

# File-by-File Mathematical and Cryptographic Guide to the HAETAE EasyCrypt Proof

HAETAE machine-checked provable-security guide

May 2026

# Abstract

This guide reorganizes the checked HAETAE EasyCrypt proof by source file. Each chapter corresponds to one `.ec` file in `provable-security/proof-files.txt`. The intended reader is a mathematician or cryptologist: the text explains the security meaning of the definitions, the mathematical role of the lemmas, and how each file contributes to the final random-oracle EUF-CMA theorem. It is not a tactic manual; for line-level EasyCrypt syntax, use the companion line-by-line guides.

# Contents

|                                                         |            |
|---------------------------------------------------------|------------|
| <b>Abstract</b>                                         | <b>ii</b>  |
| <b>Orientation</b>                                      | <b>1</b>   |
| <b>Narrative Mathematical Guide</b>                     | <b>2</b>   |
| The Theorem as a Cryptographic Statement . . . . .      | 2          |
| What EasyCrypt Is Checking . . . . .                    | 2          |
| Game-Hopping Structure . . . . .                        | 3          |
| Random-Oracle Programming . . . . .                     | 3          |
| Transcript Extraction and Forking . . . . .             | 3          |
| Lattice Assumptions . . . . .                           | 4          |
| Rejection Sampling and Distributional Support . . . . . | 4          |
| Algebraic Model and Hash Boundary . . . . .             | 4          |
| How to Read the File-by-File Chapters . . . . .         | 5          |
| Structure Diagram and Design Rationale . . . . .        | 5          |
| <b>1 HAETAE_Security.ec</b>                             | <b>8</b>   |
| <b>2 Sig_ROM.ec</b>                                     | <b>54</b>  |
| <b>3 HAETAE_HopGames.ec</b>                             | <b>67</b>  |
| <b>4 HAETAE_Reductions.ec</b>                           | <b>296</b> |
| <b>5 HAETAE_ROM.ec</b>                                  | <b>324</b> |
| <b>6 HAETAE_ROM_Programming.ec</b>                      | <b>335</b> |
| <b>7 HAETAE_Scheme.ec</b>                               | <b>408</b> |
| <b>8 HAETAE_Transcript.ec</b>                           | <b>413</b> |
| <b>9 HAETAE_Events.ec</b>                               | <b>447</b> |
| <b>10 HAETAE_Assumptions.ec</b>                         | <b>452</b> |
| <b>11 HAETAE_Distributions.ec</b>                       | <b>476</b> |
| <b>12 HAETAE_Rejection.ec</b>                           | <b>538</b> |

|                         |     |
|-------------------------|-----|
| 13 HAETAE_Algebra.ec    | 634 |
| 14 HAETAE_Params.ec     | 825 |
| 15 HAETAE_FIPS202.ec    | 836 |
| 16 HAETAE_Keccak1600.ec | 845 |
| How to Use This Guide   | 877 |

# Orientation

The checked theorem is a conditional, game-based proof for the EasyCrypt model of HAETAE. In cryptographic notation, the final bound has the form

$$\text{Adv}_{\text{EUF-CMA}}^{\text{HAETAE}}(A) \leq \epsilon_{\text{MLWE}} + \epsilon_{\text{Module-SIS}} + \delta_{\text{BimodalSelfTargetMSIS} \rightarrow \text{Module-SIS}} + \Delta_{\text{RO}}.$$

The terms  $\epsilon_{\text{MLWE}}$  and  $\epsilon_{\text{Module-SIS}}$  are assumption terms,  $\delta_{\text{BimodalSelfTargetMSIS} \rightarrow \text{Module-SIS}}$  is the loss for lifting the bimodal self-target extraction relation to Module-SIS, and  $\Delta_{\text{RO}}$  is the random-oracle/signing-simulation loss. In the counted-ROM proof path,

$$\Delta_{\text{RO}} = \frac{(q_h + q_s + 1)^2}{|\mathcal{C}|} + \frac{q_s(q_h + 1)}{|\mathcal{C}|} + \frac{q_s(q_h + 1)}{|\mathcal{C}|}.$$

The three summands correspond to collision, prequery/reprogramming, and min-entropy bad events. The checked constants instantiate  $q_h = 2$ ,  $q_s = 2$ , and  $|\mathcal{C}| \geq 2^{58}$ .

The proof does not claim implementation-level constant-time security, concrete lattice hardness estimates, or a standard-model replacement for the random oracle. It proves that the formal HAETAE model satisfies the stated unforgeability bound under the formal MLWE, Module-SIS, and random-oracle assumptions.

# Narrative Mathematical Guide

This chapter gives the continuous mathematical reading that a cryptologist would use before opening individual EasyCrypt files. The later numbered chapters refine this narrative file by file; this section explains the proof as one argument.

## The Theorem as a Cryptographic Statement

The checked result is a random-oracle EUF-CMA theorem for the HAETAE model. An adversary receives a public key and access to a signing oracle, asks a bounded number of random-oracle and signing queries, and wins if it outputs a fresh valid signature. Freshness means that the message/signature pair is not merely a replay of a signing-oracle answer.

The theorem is conditional in the standard reductionist sense. The proof separates the adversary's success probability into three conceptual sources: solving the modeled MLWE instance behind the public key, solving the modeled Module-SIS relation extracted from a successful fork, or hitting a bad random-oracle/programming event. In symbols, the proof has the shape

$$\Pr[\text{EUF-CMA}_{\text{HAETAE}}(A)] \leq \epsilon_{\text{MLWE}} + \epsilon_{\text{Module-SIS}} + \delta_{\text{BimodalSelfTargetMSIS} \rightarrow \text{Module-SIS}} + \Delta_{\text{RO}}.$$

The EasyCrypt theorem names these terms rather than hiding them in asymptotic notation. This is important for checking: each transition must either be an exact equivalence, an event inclusion, or an explicit arithmetic inequality.

## What EasyCrypt Is Checking

The proof script is not a prose proof transcribed linearly. It is a collection of typed probabilistic programs and machine-checked judgments about them. Mathematically, the main EasyCrypt forms should be read as follows:

|               |                   |                                         |
|---------------|-------------------|-----------------------------------------|
| <b>type</b>   | $\leftrightarrow$ | carrier set or abstract domain,         |
| <b>op</b>     | $\leftrightarrow$ | total function, predicate, or constant, |
| <b>module</b> | $\leftrightarrow$ | oracle machine or game,                 |
| <b>lemma</b>  | $\leftrightarrow$ | mathematical proposition,               |
| <b>hoare</b>  | $\leftrightarrow$ | partial-correctness judgment,           |
| <b>equiv</b>  | $\leftrightarrow$ | relational equality of two programs,    |
| <b>phoare</b> | $\leftrightarrow$ | probabilistic Hoare bound.              |

Thus a long EasyCrypt proof is often a short cryptographic idea decomposed into many local verification obligations. For example, a paper proof may say that a simulator's transcript is distributed as in the real game except with small bad-event probability. In EasyCrypt this becomes a sequence of relational program equivalences, preservation lemmas for maps and counters, support lemmas for samplers, and real-arithmetic facts about the accumulated loss.

## Game-Hopping Structure

The proof follows the usual game-hopping method. Starting from the real EUF-CMA game, it replaces concrete subprocedures by simulated ones until a successful forgery can be interpreted as either an MLWE distinguisher, a Module-SIS solution, or a random-oracle bad event. The individual hops have four mathematical flavors.

**Exact code transformations.** Some games differ only by inlining, dead-state removal, or re-ordering of independent computations. EasyCrypt proves these by `equiv` judgments. The mathematical claim is equality of output distributions.

**Bad-event transitions.** Some games are identical until an exceptional event occurs. The proof records the event as a predicate on game state, proves that the two games agree outside that predicate, and then upper-bounds the event probability. This is the formal version of the fundamental lemma of game playing.

**Programming transitions.** Random-oracle games must justify when an oracle value can be sampled lazily, when it can be programmed, and when programming is invalid because the adversary already queried the same point. The checked proof keeps explicit logs and counters, so the final loss can mention the number of hash and signing queries.

**Reduction transitions.** At the end of the hop chain, a successful adversary is converted into an algorithm for a hardness game. EasyCrypt checks that the constructed adversary uses the same information as the game allows and that its success predicate is the advertised algebraic predicate.

## Random-Oracle Programming

HAETAE is modeled in the random-oracle setting. The proof separates the random oracle into typed domains: message-hash queries, challenge queries, expansion queries, and signing-randomness queries. This typing is not merely cosmetic. It prevents the proof from accidentally using a value sampled for one cryptographic role as though it came from another.

The counted-ROM part of the proof introduces three losses. The collision term accounts for two relevant challenge points mapping in a way that prevents clean extraction. The prequery term accounts for the adversary querying a point before the simulator needs to program it. The min-entropy term accounts for insufficient unpredictability of the point selected for programming. In the checked parameter setting these terms are grouped as

$$\frac{(q_h + q_s + 1)^2}{|\mathcal{C}|} + \frac{q_s(q_h + 1)}{|\mathcal{C}|} + \frac{q_s(q_h + 1)}{|\mathcal{C}|}.$$

The important mathematical point is that the proof does not use an informal "random oracle behaves randomly" step. It accounts for the finite challenge space and the exact query counters that appear in the game.

## Transcript Extraction and Forking

The forking part of the proof packages an accepting signature execution as a transcript. A transcript records the public key, message, commitment-like values, challenge, response, and verification facts

needed by the algebraic extractor. If two accepting transcripts share the same target and differ in challenge, their response equations can be subtracted. This is the Fiat-Shamir special-soundness step:

$$Az_1 - c_1t = y \quad \text{and} \quad Az_2 - c_2t = y \quad \implies \quad A(z_1 - z_2) = (c_1 - c_2)t.$$

The exact symbols in the EasyCrypt model are HAETAE-specific, but the mathematical role is this standard extraction pattern. The proof then checks that the extracted vector is nonzero and norm-bounded under the rejection and verification predicates. Those facts are what connect the forgery to a Module-SIS-style relation.

## Lattice Assumptions

The model uses two abstract hardness interfaces. MLWE accounts for replacing or interpreting the public key distribution, while Module-SIS accounts for the short relation obtained from forking. A third interface, bimodal self-target MSIS, is used internally because the extracted relation is shaped by the Fiat-Shamir transcript and by HAETAE’s bimodal response structure.

The proof does not estimate the concrete hardness of these assumptions. It keeps them as symbolic advantage terms. This is a deliberate trust boundary: the machine checks the reduction from HAETAE forgery to those assumption games, but it does not replace cryptanalytic parameter assessment.

## Rejection Sampling and Distributional Support

HAETAE signing uses rejection sampling to ensure that the published response has the right distributional shape and norm bounds. In a prose proof this often appears as a short lemma saying that the accepted response is statistically close to an ideal sample. In EasyCrypt it becomes several obligations: samplers must be lossless, sampled values must lie in the promised support, acceptance predicates must imply verification-side norm predicates, and the probability of rejection must be bounded by an explicit loss term.

This decomposition explains why the distribution and rejection files contain many small lemmas. They are not independent cryptographic ideas; they are the local facts needed so that the game hops can replace idealized signing coins with structurally sampled signing attempts without silently changing the distribution.

## Algebraic Model and Hash Boundary

The algebra file supplies the ring, module, vector, matrix, packing, and verification operations used by the proof. Its mathematical purpose is to make the public-key relation, signing equation, verification predicate, and norm checks precise enough for the game proof to mention them. The proof relies on these operations as the formal HAETAE model.

The FIPS202 and Keccak files are different in character. They model deterministic SHAKE/Keccak-shaped computations needed by the EasyCrypt model, but the security theorem remains a random-oracle theorem. No part of the checked proof claims that concrete SHAKE instantiates the random oracle in the standard model.



## How to Read the File-by-File Chapters

The chapters below follow the proof manifest. They should be read as the indexed expansion of this narrative. When a chapter discusses a type, operation, lemma, Hoare judgment, or equivalence judgment, interpret it through the symbolic correspondence above: carrier sets and functions define the mathematical language, games define probability spaces, and lemmas prove either exact equalities, bad-event bounds, or reductions to assumptions.

The chapters below follow the manifest order. Read them as a dependency chain: top-level composition first, then the generic game interface, then hop games and losses, then the algebraic and distributional infrastructure.

## Structure Diagram and Design Rationale

Figure 1 gives the structural view of the active `.ec` files. An arrow  $F \rightarrow G$  means that  $F$  supplies definitions, games, predicates, or lemmas that are consumed by  $G$ . The diagram therefore shows proof-use flow, not every EasyCrypt standard-library import.

**Definition 0.1** (Layered proof-surface structure). Let  $\mathcal{F}$  be the set of active EasyCrypt files in `provable-security/proof-files.txt`. The HAETAE proof surface is organized as a stratification

$$\mathcal{F} = L_0 \cup L_1 \cup L_2 \cup L_3 \cup L_4 \cup L_5$$

where

$$\begin{aligned} L_0 &= \{\text{HAETAE\_Params.ec}, \text{HAETAE\_Keccak1600.ec}\}, \\ L_1 &= \{\text{HAETAE\_FIPS202.ec}, \text{HAETAE\_Algebra.ec}, \text{HAETAE\_Distributions.ec}\}, \\ L_2 &= \{\text{HAETAE\_Transcript.ec}, \text{HAETAE\_Events.ec}, \text{HAETAE\_Rejection.ec}\}, \\ L_3 &= \{\text{Sig\_ROM.ec}, \text{HAETAE\_ROM.ec}, \text{HAETAE\_Scheme.ec}, \text{HAETAE\_Assumptions.ec}\}, \\ L_4 &= \{\text{HAETAE\_ROM\_Programming.ec}, \text{HAETAE\_HopGames.ec}, \text{HAETAE\_Reductions.ec}\}, \\ L_5 &= \{\text{HAETAE\_Security.ec}\}. \end{aligned}$$

The intended invariant of the stratification is that lower layers introduce the mathematical language and reusable local facts, while higher layers consume those facts to define games, prove game transitions, account for losses, and assemble the final theorem.

**Theorem 0.1** (Design adequacy of the layered structure). *The layered structure above is appropriate for a machine-checked provable-security proof of HAETAE because it separates the five different mathematical tasks that occur in the reduction:*

$$\begin{aligned} &\text{model definition} \longrightarrow \text{local mathematical facts} \\ &\longrightarrow \text{game and event definitions} \longrightarrow \text{probability/loss bounds} \\ &\longrightarrow \text{final advantage composition.} \end{aligned}$$

*Consequently, the final theorem in `HAETAE_Security.ec` can be checked as a composition of named obligations rather than as a monolithic proof that hides algebra, sampling, random-oracle programming, and hardness assumptions in one script.*

*Mathematical explanation.* The lowest layer fixes finite parameters and the Keccak state vocabulary. This is necessary because later carrier sets, byte lengths, bounds, and hash-shaped operations must be well typed before any probability space is defined.

The next layer separates deterministic algebra and distribution laws from the game proof. Algebraic facts such as verification equations, packing properties, and norm predicates are pure statements about the modeled carrier sets. Distribution facts such as losslessness, support, and point-probability bounds are measure-theoretic local facts about samplers. They are reusable and should not be reproved inside every game equivalence.

The transcript, event, and rejection layer names the predicates that appear in the fundamental-lemma arguments. This is mathematically important because a bad-event proof must state exactly which event accounts for the difference between two games. Naming these predicates once prevents the proof from using slightly different informal notions of acceptance, freshness, or rejection in different hops.

The interface layer separates the generic random-oracle signature experiment from the HAETAE instantiation and from the cryptographic assumptions. This is the main trust-boundary reason for the structure. The files `Sig_ROM.ec`, `HAETAE_ROM.ec`, and `HAETAE_Scheme.ec` say what experiment is being run, while `HAETAE_Assumptions.ec` says which hardness games are assumed. The theorem therefore depends on explicit game advantages, not on hidden axioms.

The hybrid layer contains exactly the material that changes probabilities: random-oracle programming, stateful game hops, sampler replacements, bad-event bounds, and real-valued loss normalization. These arguments are kept out of `HAETAE_Security.ec` so that every loss term in the final bound can be traced to a local lemma in `HAETAE_ROM_Programming.ec`, `HAETAE_HopGames.ec`, or `HAETAE_Reductions.ec`.

Finally, `HAETAE_Security.ec` is a theorem-composition layer. Its role is to combine already checked statements by transitivity of inequalities, nonnegativity, and rewriting of named bound expressions. This is the right shape for the public theorem: a cryptologist can inspect the final statement and then follow each premise back to the lower layer where it is proved.  $\square$

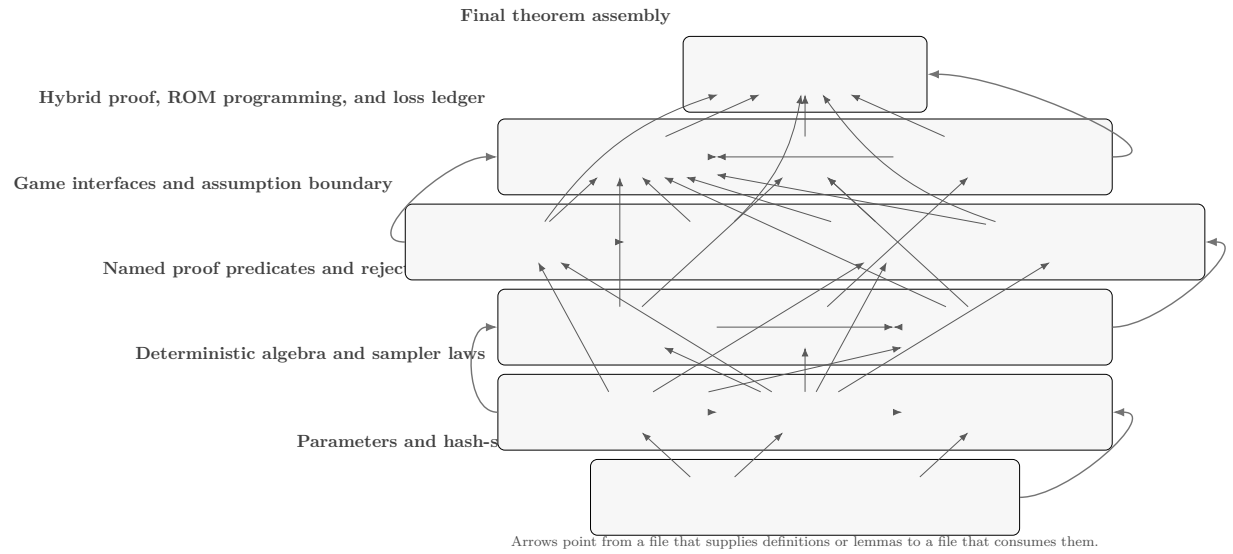


Figure 1: Layered structure of the active EasyCrypt files in the HAETAE provable-security proof surface.

# Chapter 1

## HAETAE\_Security.ec

**Mathematical role.** This is the theorem-composition file. It imports all previous definitions and lemmas and proves that the individual game hops, reductions, and arithmetic loss facts imply the final HAETAE security theorem. It is the closest file to the statement a cryptologist would put in a paper.

**Correctness.** The operation `correctness_obligation` states that an honestly generated keypair and honestly generated signature always verify:

$$\forall sd, coins, m, ctx, \quad \text{VerifyInternal}(pk, m, ctx, \sigma) = 1,$$

where

$$(pk, sk) = \text{KeyGenInternal}(sd), \quad \sigma = \text{SignInternal}(sk, m, ctx, coins).$$

The lemma `correctness_perfect` turns this deterministic obligation into the generic signature correctness game:

$$\Pr[\text{Correctness}^{\text{HAETAE}}(A) = \text{fail}] = 0.$$

**Security composition.** The file contains a sequence of security lemmas with increasingly explicit premises. The basic pattern is:

$$\Pr[\text{EUF-CMA}(A)] \leq \Pr[\text{UF-NMA}(N)] + \Delta_{\text{RO}},$$

$$\Pr[\text{UF-NMA}(N)] \leq \Pr[\text{MLWE}(B_1)] + \Pr[\text{BimodalSelfTargetMSIS}(B_2)],$$

$$\Pr[\text{BimodalSelfTargetMSIS}(B_2)] \leq \Pr[\text{Module-SIS}(B_3)] + \delta_{\text{BimodalSelfTargetMSIS} \rightarrow \text{Module-SIS}}.$$

The final real-arithmetic step substitutes the hardness bounds for MLWE and Module-SIS and rewrites the result to the named bound from `HAETAE_Reductions.ec`.

**Main theorem variants.** The many lemmas are not competing theorems. They expose different proof boundaries:

- `euf_cma_security`: a compact theorem assuming an abstract Fiat-Shamir-with-aborts hop.
- `euf_cma_security_from_rom_transcript_and_counted_bimodal`: a more mechanized theorem using counted random-oracle programming.
- `euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal`: the final checked counted-ROM path, after the explicit sampling hop has been discharged.

For a cryptologist, this file should be read as the final proof tree: every premise is either a previously checked game hop, an assumption game bound, or a real-arithmetic nonnegativity fact.

## Textbook Formal Inventory

This subsection records the exact formal material in `HAETAE_Security.ec`. It is generated from the proof-surface EasyCrypt file, so the line numbers and statements are meant to be read next to the checked source.

**Chapter role.** top-level correctness and EUF-CMA theorem composition.

**Inventory size.** 65 context declarations, 1 definitions/game objects, 40 lemmas or relational theorems, and 1 explicit Hoare/Phoare judgments were found.

## Theory Context, Imports, and Quantified Modules

**Context 1 (line 1)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import AllCore Real Distr StdOrder.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 2 (line 2)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import Sig_ROM HAETAE_Scheme HAETAE_Algebra HAETAE_Distributions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 3 (line 3)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import HAETAE_Reductions HAETAE_Assumptions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 4 (line 4)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import HAETAE_Rejection.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 5 (line 5) EasyCrypt name.**

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import HAETAE_Transcript HAETAE_ROM_Programming.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 6 (line 6) EasyCrypt name.**

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import HAETAE_HopGames.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 7 (line 8) EasyCrypt name.**

```
HAETAE_Security
```

**Checked EasyCrypt statement.**

```
theory HAETAE_Security.
```

**Formal mathematical reading.** Mathematical context. This begins a namespace for the formal development in this file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 8 (line 10) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Algebra.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 9 (line 11)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Distributions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 10 (line 12)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Reductions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 11 (line 13)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Assumptions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 12 (line 14)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Rejection.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 13 (line 15) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Transcript.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 14 (line 16) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_ROM_Programming.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 15 (line 17) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_HopGames.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 16 (line 18) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Scheme.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.



**Context 17 (line 19)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import RealOrder.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 18 (line 35)** EasyCrypt name.

```
section_line_35
```

**Checked EasyCrypt statement.**

```
section.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 19 (line 37)** EasyCrypt name.

```
H
```

**Checked EasyCrypt statement.**

```
declare module H <: SIG.Oracle.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 20 (line 38)** EasyCrypt name.

```
A
```

**Checked EasyCrypt statement.**

```
declare module A <: SIG.CORR_ADV {-H, -HAETAET}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 21 (line 73) EasyCrypt name.

```
NoSigningSecurity
```

### Checked EasyCrypt statement.

```
section NoSigningSecurity.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 22 (line 75) EasyCrypt name.

```
H
```

### Checked EasyCrypt statement.

```
declare module H <: SIG.Oracle {-SIG.EUF_CMA}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 23 (line 76) EasyCrypt name.

```
N
```

### Checked EasyCrypt statement.

```
declare module N <: SIG.NMA_Adversary {-H, -HAETAE, -SIG.EUF_CMA}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 24 (line 77) EasyCrypt name.

```
Bmlwe
```

### Checked EasyCrypt statement.

```
declare module Bmlwe <: MLWE_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 25 (line 78) EasyCrypt name.**

```
Bbimodal
```

**Checked EasyCrypt statement.**

```
declare module Bbimodal <: BimodalSelfTargetMSIS_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 26 (line 79) EasyCrypt name.**

```
Bsis
```

**Checked EasyCrypt statement.**

```
declare module Bsis <: ModuleSIS_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 27 (line 127) EasyCrypt name.**

```
section_line_127
```

**Checked EasyCrypt statement.**

```
section.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 28 (line 129) EasyCrypt name.**

```
H
```

**Checked EasyCrypt statement.**

```
declare module H <: SIG.Oracle.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 29 (line 130) EasyCrypt name.**

A

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -HAETAE}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 30 (line 131) EasyCrypt name.**

N

**Checked EasyCrypt statement.**

```
declare module N <: SIG.NMA_Adversary {-H, -HAETAE}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 31 (line 132) EasyCrypt name.**

Bmlwe

**Checked EasyCrypt statement.**

```
declare module Bmlwe <: MLWE_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 32 (line 133) EasyCrypt name.**

Bbimodal

**Checked EasyCrypt statement.**

```
declare module Bbimodal <: BimodalSelfTargetMSIS_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 33 (line 134) EasyCrypt name.

```
Bsis
```

#### Checked EasyCrypt statement.

```
declare module Bsis <: ModuleSIS_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 34 (line 211) EasyCrypt name.

```
FullReductionWithSimulatorHop
```

#### Checked EasyCrypt statement.

```
section FullReductionWithSimulatorHop.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 35 (line 213) EasyCrypt name.

```
H
```

#### Checked EasyCrypt statement.

```
declare module H <: SIG.Oracle {-SIG.EUF_CMA, -EUF_CMA_SimulatedSign,  
                                -RealSigningSimulator}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 36 (line 215) EasyCrypt name.

```
A
```

#### Checked EasyCrypt statement.

```
declare module A <: SIG.Adversary {-H, -HAETAE, -SIG.EUF_CMA,  
                                   -EUF_CMA_SimulatedSign,  
                                   -RealSigningSimulator}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 37 (line 218) EasyCrypt name.**

N

**Checked EasyCrypt statement.**

```
declare module N <: SIG.NMA_Adversary {-H, -HAETAE}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 38 (line 219) EasyCrypt name.**

Bmlwe

**Checked EasyCrypt statement.**

```
declare module Bmlwe <: MLWE_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 39 (line 220) EasyCrypt name.**

Bbimodal

**Checked EasyCrypt statement.**

```
declare module Bbimodal <: BimodalSelfTargetMSIS_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 40 (line 221) EasyCrypt name.**

Bsis

**Checked EasyCrypt statement.**

```
declare module Bsis <: ModuleSIS_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 41 (line 306) EasyCrypt name.**

```
FullReductionWithROMTranscriptHop
```

**Checked EasyCrypt statement.**

```
section FullReductionWithROMTranscriptHop.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 42 (line 308) EasyCrypt name.**

```
H
```

**Checked EasyCrypt statement.**

```
declare module H <: SIG.Oracle {-SIG.EUF_CMA,  
                                -EUF_CMA_InternalTranscriptSign,  
                                -EUF_CMA_ROMInternalTranscriptSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 43 (line 311) EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -HAETAE, -SIG.EUF_CMA,  
                                   -EUF_CMA_InternalTranscriptSign,  
                                   -EUF_CMA_ROMInternalTranscriptSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 44 (line 314) EasyCrypt name.**

```
N
```

### Checked EasyCrypt statement.

```
declare module N <: SIG.NMA_Adversary {-H, -HAETAETAE}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 45 (line 315) EasyCrypt name.

```
Bmlwe
```

### Checked EasyCrypt statement.

```
declare module Bmlwe <: MLWE_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 46 (line 316) EasyCrypt name.

```
Bbimodal
```

### Checked EasyCrypt statement.

```
declare module Bbimodal <: BimodalSelfTargetMSIS_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 47 (line 317) EasyCrypt name.

```
Bsis
```

### Checked EasyCrypt statement.

```
declare module Bsis <: ModuleSIS_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 48 (line 407) EasyCrypt name.



```
MechanizedBimodalModuleSISLift
```

**Checked EasyCrypt statement.**

```
section MechanizedBimodalModuleSISLift.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 49 (line 409) EasyCrypt name.**

```
H
```

**Checked EasyCrypt statement.**

```
declare module H <: SIG.Oracle {-SIG.EUF_CMA, -EUF_CMA_SimulatedSign,  
                                -RealSigningSimulator}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 50 (line 411) EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -HAETAE, -SIG.EUF_CMA,  
                                   -EUF_CMA_SimulatedSign,  
                                   -RealSigningSimulator}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 51 (line 414) EasyCrypt name.**

```
N
```

**Checked EasyCrypt statement.**

```
declare module N <: SIG.NMA_Adversary {-H, -HAETAE}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 52 (line 415) EasyCrypt name.

```
Bmlwe
```

### Checked EasyCrypt statement.

```
declare module Bmlwe <: MLWE_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 53 (line 416) EasyCrypt name.

```
Bbimodal
```

### Checked EasyCrypt statement.

```
declare module Bbimodal <: BimodalSelfTargetMSIS_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 54 (line 461) EasyCrypt name.

```
MechanizedROMTranscriptBimodalLift
```

### Checked EasyCrypt statement.

```
section MechanizedROMTranscriptBimodalLift.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 55 (line 463) EasyCrypt name.

```
H
```

### Checked EasyCrypt statement.

```
declare module H <: SIG.Oracle {-SIG.EUF_CMA,  
                                -EUF_CMA_InternalTranscriptSign,  
                                -EUF_CMA_ROMInternalTranscriptSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 56 (line 466)** EasyCrypt name.

A

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -HAETAE, -SIG.EUF_CMA,  
                                  -EUF_CMA_InternalTranscriptSign,  
                                  -EUF_CMA_ROMInternalTranscriptSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 57 (line 469)** EasyCrypt name.

N

**Checked EasyCrypt statement.**

```
declare module N <: SIG.NMA_Adversary {-H, -HAETAE}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 58 (line 470)** EasyCrypt name.

Bmlwe

**Checked EasyCrypt statement.**

```
declare module Bmlwe <: MLWE_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 59 (line 471)** EasyCrypt name.

Bbimodal

**Checked EasyCrypt statement.**

```
declare module Bbimodal <: BimodalSelfTargetMSIS_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 60 (line 598) EasyCrypt name.**

```
MechanizedROMTranscriptConstructedNMALift
```

**Checked EasyCrypt statement.**

```
section MechanizedROMTranscriptConstructedNMALift.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 61 (line 600) EasyCrypt name.**

```
H
```

**Checked EasyCrypt statement.**

```
declare module H <: SIG.Oracle {-SIG.EUF_CMA,
    -EUF_CMA_InternalTranscriptSign,
    -EUF_CMA_ROMInternalTranscriptSign,
    -ROMInternalTranscriptAsNMA,
    -ROMInternalTranscriptPublicSimAsNMA,
    -ROMInternalTranscriptPaperSimAsNMA,
    -ROMInternalTranscriptBudgetedPaperSimAsNMA,
    -ROMPaperSimSigningSampler,
    -RealSigningPaperSimSampler,
    -ROSigningAttemptPaperSimSampler,
    -HAETAERejectionPaperSimSampler,
    -ExactHyperballHAETAERejectionPaperSimSampler,
    -ExactHyperballPaperSimSampler,
    -ROExactHyperballPaperSimSampler}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 62 (line 614) EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -HAETAERejectionPaperSimSampler,
    -EUF_CMA_InternalTranscriptSign,
    -EUF_CMA_ROMInternalTranscriptSign,
    -ROMInternalTranscriptAsNMA,
    -ROMInternalTranscriptPublicSimAsNMA,
    -ROMInternalTranscriptPaperSimAsNMA,
    -ROMInternalTranscriptBudgetedPaperSimAsNMA,
    -ROMPaperSimSigningSampler,
    -RealSigningPaperSimSampler,
    -ROSigningAttemptPaperSimSampler,
    -HAETAERejectionPaperSimSampler,
```

```
-ExactHyperballHAETAERejectionPaperSimSampler,
-ExactHyperballPaperSimSampler,
-ROExactHyperballPaperSimSampler}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 63 (line 628) EasyCrypt name.**

```
Samp
```

**Checked EasyCrypt statement.**

```
declare module Samp <: PaperSimSigningSampler {-H, -A,
                                             -ROMInternalTranscriptPaperSimAsNMA}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 64 (line 630) EasyCrypt name.**

```
Bmlwe
```

**Checked EasyCrypt statement.**

```
declare module Bmlwe <: MLWE_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 65 (line 631) EasyCrypt name.**

```
Bbimodal
```

**Checked EasyCrypt statement.**

```
declare module Bbimodal <: BimodalSelfTargetMSIS_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

## Definitions, Carrier Sets, Operations, Games, and Procedures

**Definition 1.1** (EasyCrypt line 21). *EasyCrypt name*.

```
correctness_obligation
```

**Checked EasyCrypt statement.**

```
op correctness_obligation =  
  forall (sd : seed) (coins : random_coins)  
    (m : message) (ctx : context),  
    verify_internal haetae_mode (keygen_internal haetae_mode sd).‘1 m ctx  
      (sign_internal haetae_mode  
        (keygen_internal haetae_mode sd).‘2 m ctx coins).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

## Lemmas, Theorems, Relational Equivalences, and Their Proof Dependencies

**Theorem 1.1** (EasyCrypt line 28). *EasyCrypt name*.

```
correctness_obligation_holds
```

**Checked EasyCrypt statement.**

```
lemma correctness_obligation_holds : correctness_obligation.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
coins  
correctness_obligation  
ctx  
keygen_internal  
sd  
verify_internal_sign_internal
```

**Theorem 1.2** (EasyCrypt line 40). *EasyCrypt name*.

```
correctness_perfect
```

**Checked EasyCrypt statement.**

```
lemma correctness_perfect &m :  
  Pr[SIG.Correctness(H, HAETAE, A).main() @ &m : res] = 0%r.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byphoare*, *hoare*, *proc*, *inline*, *wp*, *call*, *rnd*, *conseq*, *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
coins
keygen_internal
result
result0
sd
valid_signature_sign_internal
verify_internal
verify_norm_ok_current
```

**Theorem 1.3** (EasyCrypt line 81). *EasyCrypt name.*

```
euf_cma_security_no_signing
```

**Checked EasyCrypt statement.**

```
lemma euf_cma_security_no_signing &m :
  Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] <=
    Pr[MLWE_Game(Bmlwe).main() @ &m : res] +
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] =>
  Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] <=
    Pr[ModuleSIS_Game(Bsis).main() @ &m : res] +
    bimodal_to_msis_reduction_loss_term =>
  Pr[MLWE_Game(Bmlwe).main() @ &m : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(Bsis).main() @ &m : res] <= module_sis_hardness_term =>
  Pr[SIG.EUF_CMA(H, HAETAE, SIG.NMA_As_CMA(N)).main() @ &m : res] <=
    haetae_euf_bound.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*, *apply*. Named identifiers syntactically cited in the proof block:

```
Bbimodal
BimodalSelfTargetMSIS_Game
Bmlwe
Bsis
HAETAE
MLWE_Game
ModuleSIS_Game
Pr
SIG
UF_NMA
bimodal_hop
bimodal_to_msis_reduction_loss_term
h_nma_msis
haetae_euf_bound_rejection_groupedE
ler_add
ler_addl
ler_trans
```

```

lerr
main
mlwe_bound
mlwe_hardness_term
module_sis_hardness_term
msis_bound
nma_as_cma_exact
nma_hop
rejection_bound_parts_nonnegative
rejection_sampling_bound_obligation_holds

```

**Theorem 1.4** (EasyCrypt line 136). *EasyCrypt name.*

```
euf_cma_security
```

**Checked EasyCrypt statement.**

```

lemma euf_cma_security  $\mathcal{E}m$  :
  Pr[SIG.EUF_CMA(H, HAETAE, A).main() @  $\mathcal{E}m$  : res] <=
    Pr[SIG.UF_NMA(H, HAETAE, N).main() @  $\mathcal{E}m$  : res] +
      (fs_with_aborts_reprogramming_term +
        fs_with_aborts_min_entropy_term) =>
    Pr[SIG.UF_NMA(H, HAETAE, N).main() @  $\mathcal{E}m$  : res] <=
      Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] +
        Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] =>
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] <=
      Pr[ModuleSIS_Game(Bsis).main() @  $\mathcal{E}m$  : res] +
        bimodal_to_msis_reduction_loss_term =>
    Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] <= mlwe_hardness_term =>
    Pr[ModuleSIS_Game(Bsis).main() @  $\mathcal{E}m$  : res] <= module_sis_hardness_term =>
    Pr[SIG.EUF_CMA(H, HAETAE, A).main() @  $\mathcal{E}m$  : res] <= haetae_euf_bound.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

Bbimodal
BimodalSelfTargetMSIS_Game
Bmlwe
Bsis
HAETAE
MLWE_Game
ModuleSIS_Game
Pr
SIG
UF_NMA
bimodal_hop
bimodal_to_msis_reduction_loss_term
fs_hop
fs_with_aborts_min_entropy_term
fs_with_aborts_reprogramming_term
h_nma_msis
haetae_euf_bound_groupedE
ler_add
ler_trans
lerr
main

```



```
mlwe_bound
msis_bound
nma_hop
```

**Theorem 1.5** (EasyCrypt line 188). *EasyCrypt name.*

```
euf_cma_security_with_hop_interfaces
```

**Checked EasyCrypt statement.**

```
lemma euf_cma_security_with_hop_interfaces  $\mathcal{E}m$  :
  fs_with_aborts_interfaces_ready haetae_mode =>
  Pr[SIG.EUF_CMA(H, HAETAE, A).main() @  $\mathcal{E}m$  : res] <=
    Pr[SIG.UF_NMA(H, HAETAE, N).main() @  $\mathcal{E}m$  : res] +
    (fs_with_aborts_reprogramming_term +
     fs_with_aborts_min_entropy_term) =>
  Pr[SIG.UF_NMA(H, HAETAE, N).main() @  $\mathcal{E}m$  : res] <=
    Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] +
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] =>
  Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] <=
    Pr[ModuleSIS_Game(Bsis).main() @  $\mathcal{E}m$  : res] +
    bimodal_to_msis_reduction_loss_term =>
  Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(Bsis).main() @  $\mathcal{E}m$  : res] <= module_sis_hardness_term =>
  Pr[SIG.EUF_CMA(H, HAETAE, A).main() @  $\mathcal{E}m$  : res] <= haetae_euf_bound.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
bimodal_hop
euf_cma_security
fs_hop
mlwe_bound
msis_bound
nma_hop
```

**Theorem 1.6** (EasyCrypt line 223). *EasyCrypt name.*

```
euf_cma_security_from_simulated_hop
```

**Checked EasyCrypt statement.**

```
lemma euf_cma_security_from_simulated_hop  $\mathcal{E}m$  :
  fs_with_aborts_interfaces_ready haetae_mode =>
  Pr[EUF_CMA_SimulatedSign(H, HAETAE, A, RealSigningSimulator(HAETAE)).main()
    @  $\mathcal{E}m$  : res] <=
    Pr[SIG.UF_NMA(H, HAETAE, N).main() @  $\mathcal{E}m$  : res] +
    (fs_with_aborts_reprogramming_term +
     fs_with_aborts_min_entropy_term) =>
  Pr[SIG.UF_NMA(H, HAETAE, N).main() @  $\mathcal{E}m$  : res] <=
    Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] +
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] =>
  Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] <=
    Pr[ModuleSIS_Game(Bsis).main() @  $\mathcal{E}m$  : res] +
```

```

bimodal_to_msis_reduction_loss_term =>
Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] <= mlwe_hardness_term =>
Pr[ModuleSIS_Game(Bsis).main() @  $\mathcal{E}m$  : res] <= module_sis_hardness_term =>
Pr[SIG.EUF_CMA(H, HAETAE, A).main() @  $\mathcal{E}m$  : res] <= haetae_euf_bound.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*, *apply*. Named identifiers syntactically cited in the proof block:

```

Bbimodal
BimodalSelfTargetMSIS_Game
Bmlwe
Bsis
HAETAE
MLWE_Game
ModuleSIS_Game
Pr
SIG
UF_NMA
bimodal_hop
bimodal_to_msis_reduction_loss_term
fs_with_aborts_min_entropy_term
fs_with_aborts_reprogramming_term
h_nma_msis
haetae_euf_bound_groupedE
iface
ler_add
ler_trans
lerr
main
mlwe_bound
msis_bound
nma_hop
real_signing_simulator_exact
sim_hop

```

**Theorem 1.7** (EasyCrypt line 278). *EasyCrypt name*.

```
euf_cma_security_from_explicit_boundaries
```

**Checked EasyCrypt statement.**

```

lemma euf_cma_security_from_explicit_boundaries  $\mathcal{E}m$  :
  Pr[EUF_CMA_SimulatedSign(H, HAETAE, A, RealSigningSimulator(HAETAE)).main()
    @  $\mathcal{E}m$  : res] <=
    Pr[SIG.UF_NMA(H, HAETAE, N).main() @  $\mathcal{E}m$  : res] +
    (fs_with_aborts_reprogramming_term +
     fs_with_aborts_min_entropy_term) =>
  Pr[SIG.UF_NMA(H, HAETAE, N).main() @  $\mathcal{E}m$  : res] <=
    Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] +
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] =>
  Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] <=
    Pr[ModuleSIS_Game(Bsis).main() @  $\mathcal{E}m$  : res] +
    bimodal_to_msis_reduction_loss_term =>
  Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(Bsis).main() @  $\mathcal{E}m$  : res] <= module_sis_hardness_term =>
  Pr[SIG.EUF_CMA(H, HAETAE, A).main() @  $\mathcal{E}m$  : res] <= haetae_euf_bound.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
bimodal_hop
euf_cma_security_from_simulated_hop
fs_with_aborts_interfaces_ready_holds
mlwe_bound
msis_bound
nma_hop
sim_hop
```

**Theorem 1.8** (EasyCrypt line 319). *EasyCrypt* name.

```
euf_cma_security_from_rom_internal_transcript_hop
```

**Checked EasyCrypt statement.**

```
lemma euf_cma_security_from_rom_internal_transcript_hop  $\mathcal{E}m$  :
  Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @  $\mathcal{E}m$  : res] <=
    Pr[SIG.UF_NMA(H, HAETAE, N).main() @  $\mathcal{E}m$  : res] +
      (fs_with_aborts_reprogramming_term +
        fs_with_aborts_min_entropy_term) =>
  Pr[SIG.UF_NMA(H, HAETAE, N).main() @  $\mathcal{E}m$  : res] <=
    Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] +
      Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] =>
  Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] <=
    Pr[ModuleSIS_Game(Bsis).main() @  $\mathcal{E}m$  : res] +
      bimodal_to_msis_reduction_loss_term =>
  Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(Bsis).main() @  $\mathcal{E}m$  : res] <= module_sis_hardness_term =>
  Pr[SIG.EUF_CMA(H, HAETAE, A).main() @  $\mathcal{E}m$  : res] <= haetae_euf_bound.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
Bbimodal
Bmlwe
Bsis
bimodal_hop
euf_cma_security
haetae_rom_internal_transcript_eraser_exact
mlwe_bound
msis_bound
nma_hop
rom_hop
```

**Theorem 1.9** (EasyCrypt line 344). *EasyCrypt name.*

```
rom_internal_transcript_hop_from_clear_site_reduction
```

**Checked EasyCrypt statement.**

```
lemma rom_internal_transcript_hop_from_clear_site_reduction &m :
  Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :
    res /\
      transcript_log_signature_programming_sites_clear haetae_mode
      EUF_CMA_ROMInternalTranscriptSign.transcripts] <=
    Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] =>
  Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res] <=
    Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] +
    (fs_with_aborts_reprogramming_term +
     fs_with_aborts_min_entropy_term).
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMInternalTranscriptSign
Pr
clear_reduction
haetae_mode
ler_add
main
mu_split
rom_internal_transcript_bad_forgery_bound
split_success
transcript_log_signature_programming_sites_clear
transcripts
```

**Theorem 1.10** (EasyCrypt line 375). *EasyCrypt name.*

```
rom_internal_transcript_hop_from_clear_site_counted_reduction
```

**Checked EasyCrypt statement.**

```
lemma rom_internal_transcript_hop_from_clear_site_counted_reduction &m :
  Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :
    res /\
      transcript_log_signature_programming_sites_clear haetae_mode
      EUF_CMA_ROMInternalTranscriptSign.transcripts] <=
    Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] =>
  Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res] <=
    Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] +
    counted_rom_programming_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **have**, **rewrite**, **apply**. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMInternalTranscriptSign
Pr
clear_reduction
haetae_mode
ler_add
main
mu_split
rom_internal_transcript_bad_forgery_counted_bound
split_success
transcript_log_signature_programming_sites_clear
transcripts
```

**Theorem 1.11** (EasyCrypt line 418). **EasyCrypt name.**

```
bimodal_to_module_sis_reduction_bound
```

**Checked EasyCrypt statement.**

```
lemma bimodal_to_module_sis_reduction_bound &m :
  Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] <=
  Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
    @ &m : res] +
  bimodal_to_msis_reduction_loss_term.
```

**Formal mathematical reading.** *Mathematical theorem.* This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **rewrite**, **have**. Named identifiers syntactically cited in the proof block:

```
Bbimodal
BimodalMSIS_As_ModuleSIS
ModuleSIS_Game
Pr
bimodal_msis_as_module_sis_exact
bimodal_to_msis_reduction_loss_termE
lerr
main
ring
```

**Theorem 1.12** (EasyCrypt line 434). **EasyCrypt name.**

```
euf_cma_security_from_mechanized_boundaries
```

**Checked EasyCrypt statement.**

```
lemma euf_cma_security_from_mechanized_boundaries &m :
  Pr[EUF_CMA_SimulatedSign(H, HAETAE, A, RealSigningSimulator(HAETAE)).main()
    @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] +
  (fs_with_aborts_reprogramming_term +
```

```

    fs_with_aborts_min_entropy_term) =>
Pr[SIG.UF_NMA(H, HAETAE, N).main() @  $\mathcal{E}m$  : res] <=
  Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] +
  Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] =>
Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] <= mlwe_hardness_term =>
Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
  @  $\mathcal{E}m$  : res] <= module_sis_hardness_term =>
Pr[SIG.EUF_CMA(H, HAETAE, A).main() @  $\mathcal{E}m$  : res] <= haetae_euf_bound.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```

Bbimodal
BimodalMSIS_As_ModuleSIS
Bmlwe
bimodal_to_module_sis_reduction_bound
euf_cma_security_from_simulated_hop
fs_with_aborts_interfaces_ready_holds
mlwe_bound
msis_bound
nma_hop
sim_hop

```

**Theorem 1.13** (EasyCrypt line 473). *EasyCrypt name.*

```

euf_cma_security_from_rom_transcript_and_mechanized_bimodal

```

**Checked EasyCrypt statement.**

```

lemma euf_cma_security_from_rom_transcript_and_mechanized_bimodal  $\mathcal{E}m$  :
  Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @  $\mathcal{E}m$  : res] <=
    Pr[SIG.UF_NMA(H, HAETAE, N).main() @  $\mathcal{E}m$  : res] +
    (fs_with_aborts_reprogramming_term +
     fs_with_aborts_min_entropy_term) =>
  Pr[SIG.UF_NMA(H, HAETAE, N).main() @  $\mathcal{E}m$  : res] <=
    Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] +
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] =>
  Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
    @  $\mathcal{E}m$  : res] <= module_sis_hardness_term =>
  Pr[SIG.EUF_CMA(H, HAETAE, A).main() @  $\mathcal{E}m$  : res] <= haetae_euf_bound.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```

Bbimodal
BimodalMSIS_As_ModuleSIS
Bmlwe
bimodal_to_module_sis_reduction_bound
euf_cma_security_from_rom_internal_transcript_hop
mlwe_bound
msis_bound
nma_hop
rom_hop

```

**Theorem 1.14** (EasyCrypt line 496). *EasyCrypt name.*

```
euf_cma_security_from_rom_transcript_and_counted_bimodal
```

**Checked EasyCrypt statement.**

```

lemma euf_cma_security_from_rom_transcript_and_counted_bimodal  $\mathcal{E}m$  :
  Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @  $\mathcal{E}m$  : res] <=
    Pr[SIG.UF_NMA(H, HAETAE, N).main() @  $\mathcal{E}m$  : res] +
      counted_rom_programming_loss_term =>
  Pr[SIG.UF_NMA(H, HAETAE, N).main() @  $\mathcal{E}m$  : res] <=
    Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] +
      Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] =>
  Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
    @  $\mathcal{E}m$  : res] <= module_sis_hardness_term =>
  Pr[SIG.EUF_CMA(H, HAETAE, A).main() @  $\mathcal{E}m$  : res] <=
    haetae_euf_counted_rom_bound.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*, *apply*. Named identifiers syntactically cited in the proof block:

```

Bbimodal
BimodalMSIS_As_ModuleSIS
BimodalSelfTargetMSIS_Game
Bmlwe
HAETAE
MLWE_Game
ModuleSIS_Game
Pr
SIG
UF_NMA
bimodal_to_module_sis_reduction_bound
bimodal_to_msis_reduction_loss_term
counted_rom_programming_loss_term
h_nma_msis
haetae_euf_counted_rom_bound_groupedE
haetae_rom_internal_transcript_erasure_exact
ler_add
ler_trans
lerr
main
mlwe_bound
msis_bound
nma_hop
rom_hop

```

**Theorem 1.15** (EasyCrypt line 547). *EasyCrypt name.*

```
euf_cma_security_from_rom_clear_site_and_mechanized_bimodal
```

**Checked EasyCrypt statement.**

```
lemma euf_cma_security_from_rom_clear_site_and_mechanized_bimodal &M :
  Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &M :
    res /\
      transcript_log_signature_programming_sites_clear haetae_mode
      EUF_CMA_ROMInternalTranscriptSign.transcripts] <=
    Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &M : res] =>
  Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &M : res] <=
    Pr[MLWE_Game(Bmlwe).main() @ &M : res] +
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &M : res] =>
  Pr[MLWE_Game(Bmlwe).main() @ &M : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
    @ &M : res] <= module_sis_hardness_term =>
  Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &M : res] <= haetae_euf_bound.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
clear_reduction
euf_cma_security_from_rom_transcript_and_mechanized_bimodal
mlwe_bound
msis_bound
nma_hop
rom_internal_transcript_hop_from_clear_site_reduction
```

**Theorem 1.16** (EasyCrypt line 571). *EasyCrypt name.*

```
euf_cma_security_from_rom_clear_site_and_counted_bimodal
```

**Checked EasyCrypt statement.**

```
lemma euf_cma_security_from_rom_clear_site_and_counted_bimodal &M :
  Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &M :
    res /\
      transcript_log_signature_programming_sites_clear haetae_mode
      EUF_CMA_ROMInternalTranscriptSign.transcripts] <=
    Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &M : res] =>
  Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &M : res] <=
    Pr[MLWE_Game(Bmlwe).main() @ &M : res] +
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &M : res] =>
  Pr[MLWE_Game(Bmlwe).main() @ &M : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
    @ &M : res] <= module_sis_hardness_term =>
  Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &M : res] <=
    haetae_euf_counted_rom_bound.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.



**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
clear_reduction
euf_cma_security_from_rom_transcript_and_counted_bimodal
mlwe_bound
msis_bound
nma_hop
rom_internal_transcript_hop_from_clear_site_counted_reduction
```

**Theorem 1.17** (EasyCrypt line 633). *EasyCrypt name.*

```
euf_cma_security_from_structural_nma_and_counted_bimodal
```

**Checked EasyCrypt statement.**

```
lemma euf_cma_security_from_structural_nma_and_counted_bimodal  $\mathcal{E}m$  :
  Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptAsNMA(A)).main()
    @  $\mathcal{E}m$  : res] <=
    Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] +
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] =>
    Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] <= mlwe_hardness_term =>
    Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
      @  $\mathcal{E}m$  : res] <= module_sis_hardness_term =>
    Pr[SIG.EUF_CMA(H, HAETAE, A).main() @  $\mathcal{E}m$  : res] <=
    haetae_euf_counted_rom_bound.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
Bbimodal
Bmlwe
ROMInternalTranscriptAsNMA
euf_cma_security_from_rom_clear_site_and_counted_bimodal
mlwe_bound
msis_bound
nma_hop
rom_internal_transcript_clear_site_structural_nma_bound
```

**Theorem 1.18** (EasyCrypt line 653). *EasyCrypt name.*

```
euf_cma_security_from_public_sim_nma_and_counted_bimodal
```

**Checked EasyCrypt statement.**

```
lemma euf_cma_security_from_public_sim_nma_and_counted_bimodal  $\mathcal{E}m$  :
  Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPublicSimAsNMA(A)).main()
    @  $\mathcal{E}m$  : res] <=
```

```

Pr[MLWE_Game(Bmlwe).main() @ &m : res] +
Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] =>
Pr[MLWE_Game(Bmlwe).main() @ &m : res] <= mlwe_hardness_term =>
Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
  @ &m : res] <= module_sis_hardness_term =>
Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m : res] <=
  haetae_euf_counted_rom_bound.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

euf_cma_security_from_structural_nma_and_counted_bimodal
mlwe_bound
msis_bound
nma_hop
rom_internal_nma_public_sim_exact

```

**Theorem 1.19** (EasyCrypt line 673). *EasyCrypt name*.

```

euf_cma_security_from_paper_sim_nma_and_counted_bimodal

```

**Checked EasyCrypt statement.**

```

lemma euf_cma_security_from_paper_sim_nma_and_counted_bimodal &m :
  Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPublicSimAsNMA(A)).main()
    @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA(A, Samp)).main() @ &m : res] +
    rejection_sampling_loss_term =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA(A, Samp)).main() @ &m : res] +
    rejection_sampling_loss_term <=
  Pr[MLWE_Game(Bmlwe).main() @ &m : res] +
  Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] =>
  Pr[MLWE_Game(Bmlwe).main() @ &m : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
    @ &m : res] <= module_sis_hardness_term =>
  Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m : res] <=
    haetae_euf_counted_rom_bound.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```

HAETAE
Pr

```

```

ROMInternalTranscriptPaperSimAsNMA
SIG
Samp
UF_NMA
euf_cma_security_from_public_sim_nma_and_counted_bimodal
ler_trans
main
mlwe_bound
msis_bound
paper_nma_hop
public_sim_to_paper_sim_nma_hop_from_sampling_hop
rejection_sampling_loss_term
sampling_hop

```

**Theorem 1.20** (EasyCrypt line 704). *EasyCrypt name.*

```
euf_cma_security_from_rom_paper_sim_nma_and_counted_bimodal
```

*Checked EasyCrypt statement.*

```

lemma euf_cma_security_from_rom_paper_sim_nma_and_counted_bimodal &m :
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROMPaperSimSigningSampler(H))).main() @ &m : res] <=
    Pr[MLWE_Game(Bmlwe).main() @ &m : res] +
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] =>
    Pr[MLWE_Game(Bmlwe).main() @ &m : res] <= mlwe_hardness_term =>
    Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
      @ &m : res] <= module_sis_hardness_term =>
    Pr[SIG.EUF_CMA(H, HAETAETAE, A).main() @ &m : res] <=
    haetae_euf_counted_rom_bound.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

euf_cma_security_from_public_sim_nma_and_counted_bimodal
mlwe_bound
msis_bound
nma_hop
rom_internal_nma_public_sim_rom_paper_sim_exact

```

**Theorem 1.21** (EasyCrypt line 725). *EasyCrypt name.*

```
euf_cma_security_from_real_signing_paper_sim_nma_and_counted_bimodal
```

*Checked EasyCrypt statement.*

```

lemma euf_cma_security_from_real_signing_paper_sim_nma_and_counted_bimodal &m :
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, RealSigningPaperSimSampler(H))).main() @ &m : res] <=
    Pr[MLWE_Game(Bmlwe).main() @ &m : res] +
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] =>
    Pr[MLWE_Game(Bmlwe).main() @ &m : res] <= mlwe_hardness_term =>

```

```
Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
  @ &m : res] <= module_sis_hardness_term =>
Pr[SIG.EUF_CMA(H, HAETAETAE, A).main() @ &m : res] <=
  haetae_euf_counted_rom_bound.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
euf_cma_security_from_structural_nma_and_counted_bimodal
mlwe_bound
msis_bound
nma_hop
rom_internal_nma_real_signing_paper_sim_exact
```

**Theorem 1.22** (EasyCrypt line 746). *EasyCrypt name.*

```
euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal
```

**Checked EasyCrypt statement.**

```
lemma euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal &m :
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, HAETAETAErejectionPaperSimSampler(H))).main() @ &m : res] <=
    Pr[MLWE_Game(Bmlwe).main() @ &m : res] +
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] =>
    Pr[MLWE_Game(Bmlwe).main() @ &m : res] <= mlwe_hardness_term =>
    Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
      @ &m : res] <= module_sis_hardness_term =>
    Pr[SIG.EUF_CMA(H, HAETAETAE, A).main() @ &m : res] <=
      haetae_euf_counted_rom_bound.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
euf_cma_security_from_real_signing_paper_sim_nma_and_counted_bimodal
mlwe_bound
msis_bound
nma_hop
rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact
```

**Theorem 1.23** (EasyCrypt line 768). *EasyCrypt name.*

```
exact_hyperball_haetae_rejection_nma_bound_from_paper_sim
```

### Checked EasyCrypt statement.

```

lemma exact_hyperball_haetae_rejection_nma_bound_from_paper_sim  $\mathcal{E}m$  :
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ExactHyperballPaperSimSampler(H))).main() @  $\mathcal{E}m$  : res] <=
    Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] +
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] =>
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ExactHyperballHAETAERejectionPaperSimSampler(H))).main()
    @  $\mathcal{E}m$  : res] <=
    Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] +
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res].

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

nma_hop
rom_internal_nma_exact_hyperball_rejection_paper_sim_no_fallback_exact

```

### Theorem 1.24 (EasyCrypt line 787). *EasyCrypt name.*

```

haetae_rejection_nma_bound_from_exact_hyperball_sampling_hop

```

### Checked EasyCrypt statement.

```

lemma haetae_rejection_nma_bound_from_exact_hyperball_sampling_hop  $\mathcal{E}m$  :
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, HAETAERejectionPaperSimSampler(H))).main() @  $\mathcal{E}m$  : res] <=
    Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ExactHyperballHAETAERejectionPaperSimSampler(H))).main()
    @  $\mathcal{E}m$  : res] + rejection_sampling_loss_term =>
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ExactHyperballPaperSimSampler(H))).main() @  $\mathcal{E}m$  : res] +
    rejection_sampling_loss_term <=
    Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] +
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] =>
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, HAETAERejectionPaperSimSampler(H))).main() @  $\mathcal{E}m$  : res] <=
    Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] +
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res].

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem

into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
ExactHyperballHAETAERejectionPaperSimSampler
HAETAER
Pr
ROMInternalTranscriptPaperSimAsNMA
SIG
UF_NMA
exact_nma_hop
ler_trans
main
rejection_sampling_loss_term
rom_internal_nma_exact_hyperball_rejection_paper_sim_no_fallback_exact
sampling_hop
```

**Theorem 1.25** (EasyCrypt line 819). *EasyCrypt name.*

```
haetae_rejection_exact_hyperball_sampling_hop_from_real_signing_exact_hyperball_hop
```

**Checked EasyCrypt statement.**

```
lemma haetae_rejection_exact_hyperball_sampling_hop_from_real_signing_exact_hyperball_hop &m :
  Pr[SIG.UF_NMA(H, HAETAER,
    ROMInternalTranscriptPaperSimAsNMA
      (A, RealSigningPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAER,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ExactHyperballPaperSimSampler(H))).main() @ &m : res] +
  rejection_sampling_loss_term =>
  Pr[SIG.UF_NMA(H, HAETAER,
    ROMInternalTranscriptPaperSimAsNMA
      (A, HAETAERejectionPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAER,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ExactHyperballHAETAERejectionPaperSimSampler(H))).main()
    @ &m : res] + rejection_sampling_loss_term.
```

**Formal mathematical reading.** *Mathematical theorem.* This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** *Use in this chapter.* It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
real_exact_hop
rom_internal_nma_exact_hyperball_rejection_paper_sim_no_fallback_exact
rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact
```

**Theorem 1.26** (EasyCrypt line 843). *EasyCrypt name.*

```
haetae_rejection_ro_exact_hyperball_sampling_hop_from_real_signing_ro_exact_hyperball_hop
```

**Checked EasyCrypt statement.**

```
lemma haetae_rejection_ro_exact_hyperball_sampling_hop_from_real_signing_ro_exact_hyperball_hop &m :
```

```

Pr[SIG.UF_NMA(H, HAETAET,
  ROMInternalTranscriptPaperSimAsNMA
    (A, RealSigningPaperSimSampler(H))).main() @ Em : res] <=
Pr[SIG.UF_NMA(H, HAETAET,
  ROMInternalTranscriptPaperSimAsNMA
    (A, ROExactHyperballPaperSimSampler(H))).main() @ Em : res] +
  rejection_sampling_loss_term =>
Pr[SIG.UF_NMA(H, HAETAET,
  ROMInternalTranscriptPaperSimAsNMA
    (A, HAETAETRejectionPaperSimSampler(H))).main() @ Em : res] <=
Pr[SIG.UF_NMA(H, HAETAET,
  ROMInternalTranscriptPaperSimAsNMA
    (A, ROExactHyperballPaperSimSampler(H))).main() @ Em : res] +
  rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

real_ro_exact_hop
rom_internal_nma_real_signing_haetaet_rejection_paper_sim_exact

```

**Theorem 1.27** (EasyCrypt line 865). *EasyCrypt name.*

```

real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop

```

**Checked EasyCrypt statement.**

```

lemma real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop Em :
  Pr[SIG.UF_NMA(H, HAETAET,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ Em : res] <=
Pr[SIG.UF_NMA(H, HAETAET,
  ROMInternalTranscriptPaperSimAsNMA
    (A, ROExactHyperballPaperSimSampler(H))).main() @ Em : res] +
  rejection_sampling_loss_term =>
Pr[SIG.UF_NMA(H, HAETAET,
  ROMInternalTranscriptPaperSimAsNMA
    (A, RealSigningPaperSimSampler(H))).main() @ Em : res] <=
Pr[SIG.UF_NMA(H, HAETAET,
  ROMInternalTranscriptPaperSimAsNMA
    (A, ROExactHyperballPaperSimSampler(H))).main() @ Em : res] +
  rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

ro_attempt_hop
rom_internal_nma_real_signing_ro_attempt_paper_sim_exact

```

**Theorem 1.28** (EasyCrypt line 886). *EasyCrypt name.*

```

real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop_loss

```

*Checked EasyCrypt statement.*

```

lemma real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop_loss
  &M (loss : real) :
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &M : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &M : res] +
  loss =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, RealSigningPaperSimSampler(H))).main() @ &M : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &M : res] +
  loss.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

ro_attempt_hop
rom_internal_nma_real_signing_ro_attempt_paper_sim_exact

```

**Theorem 1.29** (EasyCrypt line 908). *EasyCrypt name.*

```

real_signing_ro_exact_hyperball_hop_from_budgeted_lifting

```

*Checked EasyCrypt statement.*

```

lemma real_signing_ro_exact_hyperball_hop_from_budgeted_lifting &M :
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &M : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &M : res] =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &M : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &M : res] =>
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA

```



```

      (A, RealSigningPaperSimSampler(H))).main() @ &m : res] <=
Pr[SIG.UF_NMA(H, HAETAETAE,
  ROMInternalTranscriptPaperSimAsNMA
    (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
rom_signature_query_budget * rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```

attempt_unbudgeted_to_budgeted
exact_budgeted_to_unbudgeted
real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop_loss
rejection_sampling_loss_term
rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_lifting
rom_signature_query_budget
sample_loss

```

**Theorem 1.30** (EasyCrypt line 943). *EasyCrypt name.*

```

haetae_rejection_ro_exact_hyperball_sampling_hop_from_budgeted_lifting

```

**Checked EasyCrypt statement.**

```

lemma haetae_rejection_ro_exact_hyperball_sampling_hop_from_budgeted_lifting
  &m :
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] =>
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] =>
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, HAETAERejectionPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
  rom_signature_query_budget * rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
attempt_unbudgeted_to_budgeted
exact_budgeted_to_unbudgeted
real_signing_ro_exact_hyperball_hop_from_budgeted_lifting
rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact
sample_loss
```

**Theorem 1.31** (EasyCrypt line 974). *EasyCrypt* name.

```
euf_cma_security_from_exact_hyperball_paper_sim_bridge_and_counted_bimodal
```

*Checked EasyCrypt statement.*

```
lemma euf_cma_security_from_exact_hyperball_paper_sim_bridge_and_counted_bimodal  $\mathcal{E}m$  :
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, HAETAETAERejectionPaperSimSAMPLER(H))).main() @  $\mathcal{E}m$  : res] <=
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ExactHyperballHAETAETAERejectionPaperSimSAMPLER(H))).main()
    @  $\mathcal{E}m$  : res] =>
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ExactHyperballPaperSimSAMPLER(H))).main() @  $\mathcal{E}m$  : res] <=
  Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] +
  Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] =>
  Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
    @  $\mathcal{E}m$  : res] <= module_sis_hardness_term =>
  Pr[SIG.EUF_CMA(H, HAETAETAE, A).main() @  $\mathcal{E}m$  : res] <=
  haetae_euf_counted_rom_bound.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
ExactHyperballHAETAETAERejectionPaperSimSAMPLER
HAETAETAE
Pr
ROMInternalTranscriptPaperSimAsNMA
SIG
UF_NMA
euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal
exact_bridge
exact_hyperball_haetae_rejection_nma_bound_from_paper_sim
exact_nma_hop
ler_trans
main
mlwe_bound
msis_bound
```

**Theorem 1.32** (EasyCrypt line 1007). *EasyCrypt* name.

```
euf_cma_security_from_exact_hyperball_sampling_hop_and_counted_bimodal
```

### Checked EasyCrypt statement.

```
lemma euf_cma_security_from_exact_hyperball_sampling_hop_and_counted_bimodal  $\mathcal{E}m$  :
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, HAETAERejectionPaperSimSampler(H))).main() @  $\mathcal{E}m$  : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ExactHyperballHAETAERejectionPaperSimSampler(H))).main()
    @  $\mathcal{E}m$  : res] + rejection_sampling_loss_term =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ExactHyperballPaperSimSampler(H))).main() @  $\mathcal{E}m$  : res] +
  rejection_sampling_loss_term <=
  Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] +
  Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] =>
  Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
    @  $\mathcal{E}m$  : res] <= module_sis_hardness_term =>
  Pr[SIG.EUF_CMA(H, HAETAE, A).main() @  $\mathcal{E}m$  : res] <=
  haetae_euf_counted_rom_bound.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal
exact_nma_hop
haetae_rejection_nma_bound_from_exact_hyperball_sampling_hop
mlwe_bound
msis_bound
sampling_hop
```

### Theorem 1.33 (EasyCrypt line 1036). *EasyCrypt name.*

```
euf_cma_security_from_ro_exact_hyperball_sampling_hop_and_counted_bimodal
```

### Checked EasyCrypt statement.

```
lemma euf_cma_security_from_ro_exact_hyperball_sampling_hop_and_counted_bimodal  $\mathcal{E}m$  :
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, RealSigningPaperSimSampler(H))).main() @  $\mathcal{E}m$  : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @  $\mathcal{E}m$  : res] +
  rejection_sampling_loss_term =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @  $\mathcal{E}m$  : res] +
  rejection_sampling_loss_term <=
  Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] +
  Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @  $\mathcal{E}m$  : res] =>
  Pr[MLWE_Game(Bmlwe).main() @  $\mathcal{E}m$  : res] <= mlwe_hardness_term =>
```

```

Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
  @ &m : res] <= module_sis_hardness_term =>
Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m : res] <=
  haetae_euf_counted_rom_bound.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```

HAETAE
Pr
ROExactHyperballPaperSimSampler
ROMInternalTranscriptPaperSimAsNMA
SIG
UF_NMA
euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal
haetae_rejection_ro_exact_hyperball_sampling_hop_from_real_signing_ro_exact_hyperball_hop
ler_trans
main
mlwe_bound
msis_bound
real_ro_exact_hop
rejection_sampling_loss_term
ro_exact_nma_hop

```

**Theorem 1.34** (EasyCrypt line 1072). *EasyCrypt name.*

```

euf_cma_security_from_ro_signing_attempt_to_ro_exact_hyperball_hop_and_counted_bimodal

```

**Checked EasyCrypt statement.**

```

lemma euf_cma_security_from_ro_signing_attempt_to_ro_exact_hyperball_hop_and_counted_bimodal &m :
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
    rejection_sampling_loss_term =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
    rejection_sampling_loss_term <=
  Pr[MLWE_Game(Bmlwe).main() @ &m : res] +
  Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] =>
  Pr[MLWE_Game(Bmlwe).main() @ &m : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
    @ &m : res] <= module_sis_hardness_term =>
  Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m : res] <=
    haetae_euf_counted_rom_bound.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event

bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
euf_cma_security_from_ro_exact_hyperball_sampling_hop_and_counted_bimodal
mlwe_bound
msis_bound
real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop
ro_attempt_hop
ro_exact_nma_hop
```

**Theorem 1.35** (EasyCrypt line 1102). *EasyCrypt name.*

```
real_signing_ro_exact_hyperball_hop_checked_from_loss_budget
```

**Checked EasyCrypt statement.**

```
lemma real_signing_ro_exact_hyperball_hop_checked_from_loss_budget &m :
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, RealSigningPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
  rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop
rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_bound
```

**Theorem 1.36** (EasyCrypt line 1116). *EasyCrypt name.*

```
real_signing_ro_exact_hyperball_hop_from_paper_sample_lifting
```

**Checked EasyCrypt statement.**

```
lemma real_signing_ro_exact_hyperball_hop_from_paper_sample_lifting &m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
  rejection_sampling_loss_term =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, RealSigningPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
```

```

ROMInternalTranscriptPaperSimAsNMA
  (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```

lifted_loss
real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop
rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting
sample_loss

```

**Theorem 1.37** (EasyCrypt line 1140). *EasyCrypt name.*

```

euf_cma_security_from_paper_sample_lifting_and_counted_bimodal

```

**Checked EasyCrypt statement.**

```

lemma euf_cma_security_from_paper_sample_lifting_and_counted_bimodal &m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
  rejection_sampling_loss_term =>
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
  rejection_sampling_loss_term <=
  Pr[MLWE_Game(Bmlwe).main() @ &m : res] +
  Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] =>
  Pr[MLWE_Game(Bmlwe).main() @ &m : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
    @ &m : res] <= module_sis_hardness_term =>
  Pr[SIG.EUF_CMA(H, HAETAETAE, A).main() @ &m : res] <=
  haetae_euf_counted_rom_bound.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```

euf_cma_security_from_ro_exact_hyperball_sampling_hop_and_counted_bimodal
lifted_loss
mlwe_bound

```

```

msis_bound
real_signing_ro_exact_hyperball_hop_from_paper_sample_lifting
ro_exact_nma_hop
sample_loss

```

**Theorem 1.38** (EasyCrypt line 1170). *EasyCrypt name.*

```
euf_cma_security_from_budgeted_paper_sample_lifting_and_counted_bimodal
```

*Checked EasyCrypt statement.*

```

lemma euf_cma_security_from_budgeted_paper_sample_lifting_and_counted_bimodal
  <math>\mathcal{E}m</math> :
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ <math>\mathcal{E}m</math> : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ <math>\mathcal{E}m</math> : res] =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ <math>\mathcal{E}m</math> : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ <math>\mathcal{E}m</math> : res] =>
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ <math>\mathcal{E}m</math> : res] +
  rom_signature_query_budget * rejection_sampling_loss_term <=
  Pr[MLWE_Game(Bmlwe).main() @ <math>\mathcal{E}m</math> : res] +
  Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ <math>\mathcal{E}m</math> : res] =>
  Pr[MLWE_Game(Bmlwe).main() @ <math>\mathcal{E}m</math> : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
    @ <math>\mathcal{E}m</math> : res] <= module_sis_hardness_term =>
  Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ <math>\mathcal{E}m</math> : res] <=
  haetae_euf_counted_rom_bound.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```

HAETAE
Pr
ROExactHyperballPaperSimSampler
ROMInternalTranscriptPaperSimAsNMA
SIG
UF_NMA
attempt_unbudgeted_to_budgeted
euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal
exact_budgeted_to_unbudgeted
haetae_rejection_ro_exact_hyperball_sampling_hop_from_budgeted_lifting
ler_trans
main
mlwe_bound
msis_bound
rejection_sampling_loss_term

```

```

ro_exact_nma_hop
rom_signature_query_budget
sample_loss

```

**Theorem 1.39** (EasyCrypt line 1217). *EasyCrypt name.*

```
euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal
```

**Checked EasyCrypt statement.**

```

lemma euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal &M :
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &M : res] +
    rejection_sampling_loss_term <=
  Pr[MLWE_Game(Bmlwe).main() @ &M : res] +
  Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &M : res] =>
  Pr[MLWE_Game(Bmlwe).main() @ &M : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
    @ &M : res] <= module_sis_hardness_term =>
  Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &M : res] <=
  haetae_euf_counted_rom_bound.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```

euf_cma_security_from_ro_exact_hyperball_sampling_hop_and_counted_bimodal
mlwe_bound
msis_bound
real_signing_ro_exact_hyperball_hop_checked_from_loss_budget
ro_exact_nma_hop

```

**Theorem 1.40** (EasyCrypt line 1239). *EasyCrypt name.*

```
euf_cma_security_from_real_signing_exact_hyperball_hop_and_counted_bimodal
```

**Checked EasyCrypt statement.**

```

lemma euf_cma_security_from_real_signing_exact_hyperball_hop_and_counted_bimodal &M :
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, RealSigningPaperSimSampler(H))).main() @ &M : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ExactHyperballPaperSimSampler(H))).main() @ &M : res] +
    rejection_sampling_loss_term =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ExactHyperballPaperSimSampler(H))).main() @ &M : res] +
    rejection_sampling_loss_term <=
  Pr[MLWE_Game(Bmlwe).main() @ &M : res] +
  Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &M : res] =>
  Pr[MLWE_Game(Bmlwe).main() @ &M : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()

```



```
@ Em : res] <= module_sis_hardness_term =>
Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ Em : res] <=
haetae_euf_counted_rom_bound.
```

**Formal mathematical reading.** *Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.*

**Role in the proof.** *Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **apply**. Named identifiers syntactically cited in the proof block:*

```
euf_cma_security_from_exact_hyperball_sampling_hop_and_counted_bimodal
exact_nma_hop
haetae_rejection_exact_hyperball_sampling_hop_from_real_signing_exact_hyperball_hop
mlwe_bound
msis_bound
real_exact_hop
```

## Explicit Program-Logic Judgments Used Inside Proof Scripts

### Program-Logic Judgment 1.1 (EasyCrypt line 44). Checked EasyCrypt judgment.

```
hoare.
```

**Formal mathematical reading.** This is a Hoare judgment. It states partial correctness of the displayed procedure from the displayed precondition to the displayed postcondition.

*End of generated formal inventory for HAETAE\_Security. ec.*

## Chapter 2

### Sig\_ROM.ec

**Mathematical role.** This file defines the generic signature security games over a random oracle. It is intentionally independent of HAETAE-specific algebra.

**Objects.** The abstract types are

pkey, skey, message, context, signature, ro\_query, ro\_output.

The oracle is a lazy random oracle

$$H : \text{ro\_query} \rightarrow \text{ro\_output},$$

with outputs sampled from `dro_output`.

**Games.** The EUF-CMA game records signing queries and accepts a forgery only if the message/-context pair is fresh:

$$\text{win}_{\text{EUF-CMA}} \iff \text{Verify}(pk, m, ctx, \sigma) = 1 \wedge (m, ctx) \notin Q_{\text{sign}}.$$

The UF-NMA game has no signing oracle:

$$\text{win}_{\text{UF-NMA}} \iff \text{Verify}(pk, m, ctx, \sigma) = 1.$$

The strong unforgeability game is also present, with freshness over  $((m, ctx), \sigma)$ , but it is not the central theorem target here.

**Key lemma.** `nma_as_cma_exact` proves that an NMA adversary embedded into the CMA interface without using the signing oracle has exactly the same success probability. This is a bookkeeping lemma, but it matters: it lets later theorems move between generic CMA syntax and NMA-style reduction games without paying any loss.

#### Textbook Formal Inventory

This subsection records the exact formal material in `Sig_ROM.ec`. It is generated from the proof-surface EasyCrypt file, so the line numbers and statements are meant to be read next to the checked source.

**Chapter role.** generic signature, random-oracle, EUF-CMA, SUF-CMA, and UF-NMA games.

**Inventory size.** 6 context declarations, 38 definitions/game objects, 3 lemmas or relational theorems, and 0 explicit Hoare/Phoare judgments were found.

## Theory Context, Imports, and Quantified Modules

**Context 1 (line 1)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import AllCore List PROM.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 2 (line 31)** EasyCrypt name.

```
cloned theory
```

**Checked EasyCrypt statement.**

```
clone import FullRO as RO with
  type in_t <- ro_query,
  type out_t <- ro_output,
  op dout <- dro_output.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 3 (line 143)** EasyCrypt name.

```
NMA_Embedding
```

**Checked EasyCrypt statement.**

```
section NMA_Embedding.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 4 (line 145)** EasyCrypt name.

```
H
```

**Checked EasyCrypt statement.**

```
declare module H <: Oracle {-EUF_CMA}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 5 (line 146) EasyCrypt name.**

```
S
```

**Checked EasyCrypt statement.**

```
declare module S <: Scheme {-H, -EUF_CMA}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 6 (line 147) EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
declare module A <: NMA_Adversary {-H, -S, -EUF_CMA}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

## Definitions, Carrier Sets, Operations, Games, and Procedures

**Definition 2.1 (EasyCrypt line 5). EasyCrypt name.**

```
pkey
```

**Checked EasyCrypt statement.**

```
type pkey.
```

**Formal mathematical reading.** Mathematical definition. `pkey` is an abstract carrier set. The proof may quantify over this domain, but it does not expose a representation.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 2.2 (EasyCrypt line 6). EasyCrypt name.**

```
skey
```

**Checked EasyCrypt statement.**

```
type skey.
```

**Formal mathematical reading.** Mathematical definition. `skey` is an abstract carrier set. The proof may quantify over this domain, but it does not expose a representation.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 2.3 (EasyCrypt line 7). EasyCrypt name.**

message

**Checked EasyCrypt statement.**

type message.

**Formal mathematical reading.** Mathematical definition. `message` is an abstract carrier set. The proof may quantify over this domain, but it does not expose a representation.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 2.4** (EasyCrypt line 8). **EasyCrypt name.**

context

**Checked EasyCrypt statement.**

type context.

**Formal mathematical reading.** Mathematical definition. `context` is an abstract carrier set. The proof may quantify over this domain, but it does not expose a representation.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 2.5** (EasyCrypt line 9). **EasyCrypt name.**

signature

**Checked EasyCrypt statement.**

type signature.

**Formal mathematical reading.** Mathematical definition. `signature` is an abstract carrier set. The proof may quantify over this domain, but it does not expose a representation.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 2.6** (EasyCrypt line 10). **EasyCrypt name.**

ro\_query

**Checked EasyCrypt statement.**

type ro\_query.

**Formal mathematical reading.** Mathematical definition. `ro_query` is an abstract carrier set. The proof may quantify over this domain, but it does not expose a representation.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 2.7** (EasyCrypt line 11). **EasyCrypt name.**

ro\_output

**Checked EasyCrypt statement.**

type ro\_output.

**Formal mathematical reading.** Mathematical definition. `ro_output` is an abstract carrier set. The proof may quantify over this domain, but it does not expose a representation.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 2.8** (EasyCrypt line 13). **EasyCrypt name.**

```
dro_output
```

**Checked EasyCrypt statement.**

```
op [lossless] dro_output : ro_query -> ro_output distr.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present. The EasyCrypt [lossless] annotation states that this distribution has total mass 1.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 2.9** (EasyCrypt line 15). **EasyCrypt name.**

```
query
```

**Checked EasyCrypt statement.**

```
type query = message * context.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 2.10** (EasyCrypt line 16). **EasyCrypt name.**

```
signed_query
```

**Checked EasyCrypt statement.**

```
type signed_query = query * signature.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 2.11** (EasyCrypt line 18). **EasyCrypt name.**

```
fresh_msg
```

**Checked EasyCrypt statement.**

```
op fresh_msg (qs : query list) (m : message) (ctx : context) : bool =
  ! ((m, ctx) \in qs).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 2.12** (EasyCrypt line 21). **EasyCrypt name.**

```
fresh_sig
```

**Checked EasyCrypt statement.**

```
op fresh_sig (qs : signed_query list) (m : message) (ctx : context)
  (sig : signature) : bool =
  ! (((m, ctx), sig) \in qs).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 2.13** (EasyCrypt line 36). **EasyCrypt name.**

```
Oracle
```

**Checked EasyCrypt statement.**

```
module type Oracle = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It defines the oracle boundary through which adversaries and simulations interact.

**Definition 2.14** (EasyCrypt line 40). **EasyCrypt name.**

```
POracle
```

**Checked EasyCrypt statement.**

```
module type POracle = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It defines the oracle boundary through which adversaries and simulations interact.

**Definition 2.15** (EasyCrypt line 44). **EasyCrypt name.**

```
Scheme
```

**Checked EasyCrypt statement.**

```
module type Scheme(O : POracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.16** (EasyCrypt line 45). **EasyCrypt name.**

```
kg
```

**Checked EasyCrypt statement.**

```
proc kg() : pkey * skey
  proc sign(sk : skey, m : message, ctx : context) : signature
  proc verify(pk : pkey, m : message, ctx : context,
             sig : signature) : bool
}.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.17** (EasyCrypt line 51). **EasyCrypt name.**

```
CORR_ADV
```

**Checked EasyCrypt statement.**

```
module type CORR_ADV(O : POracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.18** (EasyCrypt line 52). **EasyCrypt name.**

```
find
```

**Checked EasyCrypt statement.**

```
proc find(pk : pkey, sk : skey) : message * context
}.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.19** (EasyCrypt line 55). **EasyCrypt name.**

```
Correctness
```

**Checked EasyCrypt statement.**



```
module Correctness(H : Oracle, S : Scheme, A : CORR_ADV) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.20** (EasyCrypt line 56). **EasyCrypt name.**

```
main
```

**Checked EasyCrypt statement.**

```
proc main() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.21** (EasyCrypt line 73). **EasyCrypt name.**

```
SignOracle
```

**Checked EasyCrypt statement.**

```
module type SignOracle = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It defines the oracle boundary through which adversaries and simulations interact.

**Definition 2.22** (EasyCrypt line 74). **EasyCrypt name.**

```
sign
```

**Checked EasyCrypt statement.**

```
proc sign(m : message, ctx : context) : signature
}.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.23** (EasyCrypt line 77). **EasyCrypt name.**

```
Adversary
```

**Checked EasyCrypt statement.**

```
module type Adversary(H : POracle, O : SignOracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.24** (EasyCrypt line 78). **EasyCrypt name.**

```
forge
```

**Checked EasyCrypt statement.**

```
proc forge(pk : pkey) : message * context * signature {H.get, 0.sign}
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.25** (EasyCrypt line 81). **EasyCrypt name.**

```
NMA_Adversary
```

**Checked EasyCrypt statement.**

```
module type NMA_Adversary(H : POracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 2.26** (EasyCrypt line 82). **EasyCrypt name.**

```
forge
```

**Checked EasyCrypt statement.**

```
proc forge(pk : pkey) : message * context * signature {H.get}
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.27** (EasyCrypt line 85). **EasyCrypt name.**

```
NMA_As_CMA
```

**Checked EasyCrypt statement.**

```
module NMA_As_CMA(A : NMA_Adversary) (H : POracle, 0 : SignOracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 2.28** (EasyCrypt line 86). **EasyCrypt name.**

```
forge
```

**Checked EasyCrypt statement.**

```
proc forge(pk : pkey) : message * context * signature = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.29** (EasyCrypt line 94). **EasyCrypt name.**

```
EUF_CMA
```

**Checked EasyCrypt statement.**

```
module EUF_CMA(H : Oracle, S : Scheme, A : Adversary) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 2.30** (EasyCrypt line 98). **EasyCrypt name.**

```
0
```

**Checked EasyCrypt statement.**

```
module 0 = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.31** (EasyCrypt line 99). **EasyCrypt name.**

```
sign
```

**Checked EasyCrypt statement.**

```
proc sign(m : message, ctx : context) : signature = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.32** (EasyCrypt line 108). **EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
module A = A(H, 0)

proc main() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.33** (EasyCrypt line 126). **EasyCrypt name.**

```
UF_NMA
```

**Checked EasyCrypt statement.**

```
module UF_NMA(H : Oracle, S : Scheme, A : NMA_Adversary) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 2.34** (EasyCrypt line 127). **EasyCrypt name.**

```
main
```

**Checked EasyCrypt statement.**

```
proc main() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.35** (EasyCrypt line 174). **EasyCrypt name.**

```
SUF_CMA
```

**Checked EasyCrypt statement.**

```
module SUF_CMA(H : Oracle, S : Scheme, A : Adversary) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 2.36** (EasyCrypt line 178). **EasyCrypt name.**

```
0
```

**Checked EasyCrypt statement.**

```
module O = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.37** (EasyCrypt line 179). **EasyCrypt name.**

```
sign
```

**Checked EasyCrypt statement.**

```
proc sign(m : message, ctx : context) : signature = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 2.38** (EasyCrypt line 188). **EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
module A = A(H, O)

  proc main() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

## Lemmas, Theorems, Relational Equivalences, and Their Proof Dependencies

**Theorem 2.1** (EasyCrypt line 25). **EasyCrypt name.**

```
fresh_msg_nil
```

**Checked EasyCrypt statement.**

```
lemma fresh_msg_nil m ctx : fresh_msg [] m ctx.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 2.2** (EasyCrypt line 28). **EasyCrypt name.**

```
fresh_sig_nil
```

**Checked EasyCrypt statement.**

```
lemma fresh_sig_nil m ctx sig : fresh_sig [] m ctx sig.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 2.3** (EasyCrypt line 149). **EasyCrypt name.**

```
nma_as_cma_exact
```

**Checked EasyCrypt statement.**

```
lemma nma_as_cma_exact  $\mathcal{E}m$  :  
  Pr[EUF_CMA(H, S, NMA_As_CMA(A)).main() @  $\mathcal{E}m$  : res] =  
  Pr[UF_NMA(H, S, A).main() @  $\mathcal{E}m$  : res].
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byequiv*, *proc*, *inline*, *wp*, *call*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA  
NMA_As_CMA  
arg  
forge  
glob
```

*End of generated formal inventory for Sig\_ROM. ec.*

## Chapter 3

### HAETAE\_HopGames.ec

**Mathematical role.** This is the game-hopping file. It is large because handwritten phrases such as “identical until bad”, “replace the signing oracle by a simulator”, and “program the random oracle at the challenge point” are expanded into explicit typed games, state invariants, oracle logs, and probability inequalities.

**Cryptographic chain.** The file implements the following conceptual sequence:

- $G_0$  : real EUF-CMA game with HAETAE signing,
- $G_1$  : game with transcript logging,
- $G_2$  : internal transcript and ROM-programming game,
- $G_3$  : NMA-style game after signing is simulated,
- $G_4$  : paper-simulation game,
- $G_5$  : exact-hyperball sampling game,
- $G_6$  : game exposing MLWE and bimodal MSIS extraction.

The EasyCrypt module names are more detailed than this schematic chain because some games carry counted logs, budgeted counters, or direct-sampler variants.

**Types of transitions.** The file proves four kinds of statements.

1. Exact relational equivalences:

$$\Pr[G_i = 1] = \Pr[G_{i+1} = 1].$$

These appear as **equiv** lemmas such as oracle-erasure and simulator equivalences.

2. Bad-event inequalities:

$$\Pr[G_i = 1] \leq \Pr[G_{i+1} = 1] + \Pr[\text{Bad}].$$

The bad events are reprogramming conflicts, prequeries, and entropy failures.

3. Sampler-replacement bounds:

$$\Pr[G_i = 1] \leq \Pr[G_{i+1} = 1] + \Delta_{\text{sample}}.$$

These are the rejection and exact-hyperball transitions.

4. Reduction steps:

$$\Pr[G_i = 1] \leq \Pr[\text{MLWE}(B_1) = 1] + \Pr[\text{BimodalSelfTargetMSIS}(B_2) = 1].$$

**Why so many preservation lemmas?** The adversary is arbitrary and may call the random oracle adaptively. Each oracle or signing call must preserve invariants such as query budgets, programmed-site freshness, transcript validity, and min-entropy clearance. Those invariants are what justify the eventual bad-event bounds. Mathematicians should read these lemmas as the formal version of “the simulation maintains the same view unless a listed bad event occurs.”

**Special-soundness interface.** Near the end of the file, the proof introduces forking adversaries and transcript extractors. The game `BimodalSpecialSoundness_Game` captures the standard Fiat-Shamir idea that two accepting transcripts with the same target and different challenges yield an algebraic witness. The lifting lemmas then connect that witness to Module-SIS through `HAETAE_Assumptions.ec`.

## Textbook Formal Inventory

This subsection records the exact formal material in `HAETAE_HopGames.ec`. It is generated from the proof-surface EasyCrypt file, so the line numbers and statements are meant to be read next to the checked source.

**Chapter role.** HAETAE-specific hybrid games, simulator transitions, and hop bounds.

**Inventory size.** 88 context declarations, 148 definitions/game objects, 271 lemmas or relational theorems, and 12 explicit Hoare/Phoare judgments were found.

## Theory Context, Imports, and Quantified Modules

**Context 1 (line 1)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import AllCore Distr List Real FSet FMap StdOrder FelTactic Mu_mem.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 2 (line 2)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import Sig_ROM HAETAE_Scheme HAETAE_Params HAETAE_Algebra.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 3 (line 3)** EasyCrypt name.

```
imported theories
```



### Checked EasyCrypt statement.

```
require import HAETAE_Distributions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 4 (line 4) EasyCrypt name.

```
imported theories
```

### Checked EasyCrypt statement.

```
require import HAETAE_Assumptions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 5 (line 5) EasyCrypt name.

```
imported theories
```

### Checked EasyCrypt statement.

```
require import HAETAE_Transcript.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 6 (line 6) EasyCrypt name.

```
imported theories
```

### Checked EasyCrypt statement.

```
require import HAETAE_Reductions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 7 (line 7) EasyCrypt name.

```
imported theories
```

### Checked EasyCrypt statement.

```
require import HAETAE_Rejection HAETAE_ROM HAETAE_ROM_Programming.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 8 (line 9) EasyCrypt name.**

```
HAETAE_HopGames
```

**Checked EasyCrypt statement.**

```
theory HAETAE_HopGames.
```

**Formal mathematical reading.** Mathematical context. This begins a namespace for the formal development in this file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 9 (line 11) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Scheme.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 10 (line 12) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Params.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 11 (line 13) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Algebra.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 12 (line 14)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Distributions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 13 (line 15)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Assumptions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 14 (line 16)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Transcript.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 15 (line 17)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Reductions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 16 (line 18) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Rejection.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 17 (line 19) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_ROM.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 18 (line 20) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_ROM_Programming.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 19 (line 21) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import RealOrder.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 20 (line 1766) EasyCrypt name.**

```
ROMInternalTranscriptBudgetedPaperSimNMAQueryBudget
```

### Checked EasyCrypt statement.

```
section ROMInternalTranscriptBudgetedPaperSimNMAQueryBudget.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 21 (line 1768) EasyCrypt name.

```
H
```

### Checked EasyCrypt statement.

```
declare module H <: SIG.Oracle {-ROMInternalTranscriptBudgetedPaperSimAsNMA}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 22 (line 1769) EasyCrypt name.

```
A
```

### Checked EasyCrypt statement.

```
declare module A <: SIG.Adversary {-H,  
                                   -ROMInternalTranscriptBudgetedPaperSimAsNMA}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 23 (line 1771) EasyCrypt name.

```
Samp
```

### Checked EasyCrypt statement.

```
declare module Samp <: PaperSimSigningSampler  
  {-A, -ROMInternalTranscriptBudgetedPaperSimAsNMA}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 24 (line 2979) EasyCrypt name.

```
InternalTranscriptSignSound
```

**Checked EasyCrypt statement.**

```
section InternalTranscriptSignSound.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 25 (line 2981) EasyCrypt name.**

```
H
```

**Checked EasyCrypt statement.**

```
declare module H <: SIG.Oracle {-EUF_CMA_InternalTranscriptSign,  
                                -EUF_CMA_ROMInternalTranscriptSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 26 (line 2983) EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -EUF_CMA_InternalTranscriptSign,  
                                   -EUF_CMA_ROMInternalTranscriptSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 27 (line 3576) EasyCrypt name.**

```
CountedROMInternalTranscriptDiscipline
```

**Checked EasyCrypt statement.**

```
section CountedROMInternalTranscriptDiscipline.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 28 (line 3578) EasyCrypt name.

H

#### Checked EasyCrypt statement.

```
declare module H <: Oracle {-CountedLazyROM,  
                           -EUF_CMA_ROMInternalTranscriptCountedSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 29 (line 3580) EasyCrypt name.

A

#### Checked EasyCrypt statement.

```
declare module A <: SIG.Adversary {-H, -CountedLazyROM,  
                                  -EUF_CMA_ROMInternalTranscriptCountedSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 30 (line 3771) EasyCrypt name.

ProgrammedROMTranscriptDiscipline

#### Checked EasyCrypt statement.

```
section ProgrammedROMTranscriptDiscipline.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 31 (line 3773) EasyCrypt name.

H

#### Checked EasyCrypt statement.

```
declare module H <: Oracle {-CountedLazyROM,  
                           -EUF_CMA_ROMProgrammedTranscriptCountedSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 32 (line 3775) EasyCrypt name.**

A

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -CountedLazyROM,
                                   -EUF_CMA_ROMProgrammedTranscriptCountedSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 33 (line 3906) EasyCrypt name.**

BudgetedProgrammedROMTranscriptDiscipline

**Checked EasyCrypt statement.**

```
section BudgetedProgrammedROMTranscriptDiscipline.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 34 (line 3908) EasyCrypt name.**

H

**Checked EasyCrypt statement.**

```
declare module H <: Oracle {-CountedLazyROM,
                           -DirectSigningCoinSampler,
                           -EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 35 (line 3911) EasyCrypt name.**

A

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -CountedLazyROM,
                                   -DirectSigningCoinSampler,
                                   -EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign}.
```



**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

#### Context 36 (line 4520) EasyCrypt name.

```
BudgetedSampledROMTranscriptDiscipline
```

#### Checked EasyCrypt statement.

```
section BudgetedSampledROMTranscriptDiscipline.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

#### Context 37 (line 4522) EasyCrypt name.

```
H
```

#### Checked EasyCrypt statement.

```
declare module H <: Oracle {-CountedLazyROM,  
                           -DirectSigningCoinSampler,  
                           -EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

#### Context 38 (line 4525) EasyCrypt name.

```
A
```

#### Checked EasyCrypt statement.

```
declare module A <: SIG.Adversary {-H, -CountedLazyROM,  
                                  -DirectSigningCoinSampler,  
                                  -EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

#### Context 39 (line 5230) EasyCrypt name.

```
BudgetedProgrammedToSampledDirectBridge
```

**Checked EasyCrypt statement.**

```
section BudgetedProgrammedToSampledDirectBridge.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 40 (line 5232) EasyCrypt name.**

```
H
```

**Checked EasyCrypt statement.**

```
declare module H <: Oracle {-CountedLazyROM,  
                           -DirectSigningCoinSampler,  
                           -EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign,  
                           -EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 41 (line 5236) EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -CountedLazyROM,  
                                  -DirectSigningCoinSampler,  
                                  -EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign,  
                                  -EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 42 (line 5601) EasyCrypt name.**

```
HAETAEROMInternalTranscriptExact
```

**Checked EasyCrypt statement.**

```
section HAETAEROMInternalTranscriptExact.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 43 (line 5603) EasyCrypt name.

H

#### Checked EasyCrypt statement.

```
declare module H <: SIG.Oracle {-SIG.EUF_CMA,  
                                -EUF_CMA_ROMInternalTranscriptSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 44 (line 5605) EasyCrypt name.

A

#### Checked EasyCrypt statement.

```
declare module A <: SIG.Adversary {-H, -SIG.EUF_CMA,  
                                -EUF_CMA_ROMInternalTranscriptSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 45 (line 5706) EasyCrypt name.

ROMInternalTranscriptStructuralNMA

#### Checked EasyCrypt statement.

```
section ROMInternalTranscriptStructuralNMA.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 46 (line 5708) EasyCrypt name.

H

#### Checked EasyCrypt statement.

```
declare module H <: SIG.Oracle {-EUF_CMA_ROMInternalTranscriptSign,  
                                -ROMInternalTranscriptAsNMA}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 47 (line 5710) EasyCrypt name.**

A

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -EUF_CMA_ROMInternalTranscriptSign,  
                                  -ROMInternalTranscriptAsNMA}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 48 (line 5874) EasyCrypt name.**

ROMInternalTranscriptPublicSimNMA

**Checked EasyCrypt statement.**

```
section ROMInternalTranscriptPublicSimNMA.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 49 (line 5876) EasyCrypt name.**

H

**Checked EasyCrypt statement.**

```
declare module H <: SIG.Oracle {-ROMInternalTranscriptAsNMA,  
                               -ROMInternalTranscriptPublicSimAsNMA}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 50 (line 5878) EasyCrypt name.**

A

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -ROMInternalTranscriptAsNMA,  
                                  -ROMInternalTranscriptPublicSimAsNMA}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 51 (line 6041) EasyCrypt name.**

```
ROMInternalTranscriptROMPaperSimNMA
```

**Checked EasyCrypt statement.**

```
section ROMInternalTranscriptROMPaperSimNMA.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 52 (line 6043) EasyCrypt name.**

```
H
```

**Checked EasyCrypt statement.**

```
declare module H <: SIG.Oracle {-ROMInternalTranscriptPublicSimAsNMA,  
                                -ROMInternalTranscriptPaperSimAsNMA,  
                                -ROMPaperSimSigningSampler}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 53 (line 6046) EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -ROMInternalTranscriptPublicSimAsNMA,  
                                   -ROMInternalTranscriptPaperSimAsNMA,  
                                   -ROMPaperSimSigningSampler}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 54 (line 6212) EasyCrypt name.**

```
ROMInternalTranscriptRealSigningPaperSimNMA
```

**Checked EasyCrypt statement.**

```
section ROMInternalTranscriptRealSigningPaperSimNMA.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 55 (line 6214) EasyCrypt name.**

```
H
```

**Checked EasyCrypt statement.**

```
declare module H <: SIG.Oracle {-ROMInternalTranscriptAsNMA,  
                                -ROMInternalTranscriptPaperSimAsNMA,  
                                -RealSigningPaperSimSampler}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 56 (line 6217) EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -ROMInternalTranscriptAsNMA,  
                                   -ROMInternalTranscriptPaperSimAsNMA,  
                                   -RealSigningPaperSimSampler}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 57 (line 6418) EasyCrypt name.**

```
ROMInternalTranscriptROSigningAttemptPaperSimNMA
```

**Checked EasyCrypt statement.**

```
section ROMInternalTranscriptROSigningAttemptPaperSimNMA.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 58 (line 6420) EasyCrypt name.**

H

**Checked EasyCrypt statement.**

```
declare module H <: SIG.Oracle {-ROMInternalTranscriptPaperSimAsNMA,  
                                -RealSigningPaperSimSampler,  
                                -ROSigningAttemptPaperSimSampler}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 59 (line 6423) EasyCrypt name.**

A

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -ROMInternalTranscriptPaperSimAsNMA,  
                                   -RealSigningPaperSimSampler,  
                                   -ROSigningAttemptPaperSimSampler}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 60 (line 7002) EasyCrypt name.**

ROMInternalTranscriptROSigningAttemptExactHyperballPaperSimNMA

**Checked EasyCrypt statement.**

```
section ROMInternalTranscriptROSigningAttemptExactHyperballPaperSimNMA.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 61 (line 7004) EasyCrypt name.**

H

**Checked EasyCrypt statement.**

```
declare module H <: SIG.Oracle {-ROMInternalTranscriptPaperSimAsNMA,  
                                -ROMInternalTranscriptBudgetedPaperSimAsNMA,  
                                -ROSigningAttemptPaperSimSampler,  
                                -ROExactHyperballPaperSimSampler}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 62 (line 7008) EasyCrypt name.**

A

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -ROMInternalTranscriptPaperSimAsNMA,  
                                -ROMInternalTranscriptBudgetedPaperSimAsNMA,  
                                -ROSigningAttemptPaperSimSampler,  
                                -ROExactHyperballPaperSimSampler}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 63 (line 7318) EasyCrypt name.**

ConcreteROMInternalTranscriptROSigningAttemptExactHyperballPaperSimNMA

**Checked EasyCrypt statement.**

```
section ConcreteROMInternalTranscriptROSigningAttemptExactHyperballPaperSimNMA.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 64 (line 7320) EasyCrypt name.**

A

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-HAETA_E_RO.FRO,  
                                -ROMInternalTranscriptPaperSimAsNMA,  
                                -ROMInternalTranscriptBudgetedPaperSimAsNMA,  
                                -ROSigningAttemptPaperSimSampler,  
                                -ROExactHyperballPaperSimSampler}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.



### Context 65 (line 7819) EasyCrypt name.

```
ROMInternalTranscriptHAETAERejctionPaperSimNMA
```

### Checked EasyCrypt statement.

```
section ROMInternalTranscriptHAETAERejctionPaperSimNMA.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 66 (line 7821) EasyCrypt name.

```
H
```

### Checked EasyCrypt statement.

```
declare module H <: SIG.Oracle {-ROMInternalTranscriptPaperSimAsNMA,  
                                -RealSigningPaperSimSampler,  
                                -HAETAERejctionPaperSimSampler}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 67 (line 7824) EasyCrypt name.

```
A
```

### Checked EasyCrypt statement.

```
declare module A <: SIG.Adversary {-H, -ROMInternalTranscriptPaperSimAsNMA,  
                                   -RealSigningPaperSimSampler,  
                                   -HAETAERejctionPaperSimSampler}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 68 (line 8030) EasyCrypt name.

```
ROMInternalTranscriptExactHyperballPaperSimNMA
```

### Checked EasyCrypt statement.

```
section ROMInternalTranscriptExactHyperballPaperSimNMA.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 69 (line 8032) EasyCrypt name.**

H

**Checked EasyCrypt statement.**

```
declare module H <: SIG.Oracle {-ROMInternalTranscriptPaperSimAsNMA,
                                -ExactHyperballHAETAERejectionPaperSimSampler,
                                -ExactHyperballPaperSimSampler}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 70 (line 8035) EasyCrypt name.**

A

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -ROMInternalTranscriptPaperSimAsNMA,
                                   -ExactHyperballHAETAERejectionPaperSimSampler,
                                   -ExactHyperballPaperSimSampler}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 71 (line 8232) EasyCrypt name.**

ROMInternalTranscriptPaperSimHop

**Checked EasyCrypt statement.**

```
section ROMInternalTranscriptPaperSimHop.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 72 (line 8234) EasyCrypt name.**

H

**Checked EasyCrypt statement.**

```
declare module H <: SIG.Oracle {-ROMInternalTranscriptPublicSimAsNMA,
                                -ROMInternalTranscriptPaperSimAsNMA}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 73 (line 8236) EasyCrypt name.**

A

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -ROMInternalTranscriptPublicSimAsNMA,  
                                -ROMInternalTranscriptPaperSimAsNMA}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 74 (line 8238) EasyCrypt name.**

Samp

**Checked EasyCrypt statement.**

```
declare module Samp <: PaperSimSigningSampler {-H, -A,  
                                              -ROMInternalTranscriptPaperSimAsNMA}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 75 (line 8256) EasyCrypt name.**

TranscriptLoggingErasure

**Checked EasyCrypt statement.**

```
section TranscriptLoggingErasure.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 76 (line 8258) EasyCrypt name.**

H

**Checked EasyCrypt statement.**

```
declare module H <: SIG.Oracle {-EUF_CMA_SimulatedSign,
                                -EUF_CMA_TranscriptSimulatedSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 77 (line 8260) EasyCrypt name.**

```
S
```

**Checked EasyCrypt statement.**

```
declare module S <: SIG.Scheme {-H, -EUF_CMA_SimulatedSign,
                                -EUF_CMA_TranscriptSimulatedSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 78 (line 8262) EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -S, -EUF_CMA_SimulatedSign,
                                    -EUF_CMA_TranscriptSimulatedSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 79 (line 8264) EasyCrypt name.**

```
Sim
```

**Checked EasyCrypt statement.**

```
declare module Sim <: SigningSimulator {-H, -S, -A,
                                          -EUF_CMA_SimulatedSign,
                                          -EUF_CMA_TranscriptSimulatedSign}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 80 (line 8340) EasyCrypt name.**

```
RealSigningSimulatorExact
```

**Checked EasyCrypt statement.**

```
section RealSigningSimulatorExact.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 81 (line 8342) EasyCrypt name.**

```
H
```

**Checked EasyCrypt statement.**

```
declare module H <: SIG.Oracle {-SIG.EUF_CMA, -EUF_CMA_SimulatedSign,  
                                -RealSigningSimulator}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 82 (line 8344) EasyCrypt name.**

```
S
```

**Checked EasyCrypt statement.**

```
declare module S <: SIG.Scheme {-H, -SIG.EUF_CMA, -EUF_CMA_SimulatedSign,  
                                -RealSigningSimulator}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 83 (line 8346) EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
declare module A <: SIG.Adversary {-H, -S, -SIG.EUF_CMA,  
                                   -EUF_CMA_SimulatedSign,  
                                   -RealSigningSimulator}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 84 (line 8500) EasyCrypt name.**

```
BimodalSpecialSoundnessLift
```

**Checked EasyCrypt statement.**

```
section BimodalSpecialSoundnessLift.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 85 (line 8502) EasyCrypt name.**

```
H
```

**Checked EasyCrypt statement.**

```
declare module H <: SIG.Oracle.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 86 (line 8503) EasyCrypt name.**

```
S
```

**Checked EasyCrypt statement.**

```
declare module S <: SIG.Scheme {-H}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 87 (line 8504) EasyCrypt name.**

```
F
```

**Checked EasyCrypt statement.**

```
declare module F <: ForkingAdversary {-H, -S}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 88 (line 8505) EasyCrypt name.**

```
X
```

**Checked EasyCrypt statement.**

```
declare module X <: BimodalTranscriptExtractor {-H, -S, -F}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Definitions, Carrier Sets, Operations, Games, and Procedures**

**Definition 3.1** (EasyCrypt line 23). **EasyCrypt name.**

```
SigningSimulator
```

**Checked EasyCrypt statement.**

```
module type SigningSimulator(H : SIG.POracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.2** (EasyCrypt line 24). **EasyCrypt name.**

```
init
```

**Checked EasyCrypt statement.**

```
proc init(pk : pkey, sk : skey) : unit
  proc sign(m : message, ctx : context) : signature
}.

```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.3** (EasyCrypt line 28). **EasyCrypt name.**

```
RealSigningSimulator
```

**Checked EasyCrypt statement.**

```
module RealSigningSimulator(S : SIG.Scheme) (H : SIG.POracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.4** (EasyCrypt line 31). **EasyCrypt name.**

```
init
```

**Checked EasyCrypt statement.**

```
proc init(pk : pkey, sk0 : skey) : unit = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.5** (EasyCrypt line 35). **EasyCrypt name.**

```
sign
```

**Checked EasyCrypt statement.**

```
proc sign(m : message, ctx : context) : signature = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.6** (EasyCrypt line 43). **EasyCrypt name.**

```
EUF_CMA_SimulatedSign
```

**Checked EasyCrypt statement.**

```
module EUF_CMA_SimulatedSign(
  H : SIG.Oracle,
  S : SIG.Scheme,
  A : SIG.Adversary,
  Sim : SigningSimulator
) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 3.7** (EasyCrypt line 51). **EasyCrypt name.**

```
0
```

**Checked EasyCrypt statement.**

```
module 0 = {
```



**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.8** (EasyCrypt line 52). **EasyCrypt name.**

```
sign
```

**Checked EasyCrypt statement.**

```
proc sign(m : message, ctx : context) : signature = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.9** (EasyCrypt line 61). **EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
module A = A(H, 0)

proc main() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.10** (EasyCrypt line 81). **EasyCrypt name.**

```
transcript_log_consistent
```

**Checked EasyCrypt statement.**

```
op transcript_log_consistent
  (md : mode) (pk : pkey) (qs : SIG.query list)
  (trs : transcript list) (rs : signing_transcript_record list) : bool =
  qs = signing_record_queries rs /\
  trs = signing_record_transcripts rs /\
  signing_record_log_sound md pk rs.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 3.11** (EasyCrypt line 134). **EasyCrypt name.**

```
transcript_log_ready
```

**Checked EasyCrypt statement.**

```

op transcript_log_ready (md : mode) (trs : transcript list) : bool =
  transcript_log_valid md trs /\
  transcript_log_min_entropy_clear md trs.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 3.12** (EasyCrypt line 138). **EasyCrypt name.**

```

budgeted_programmed_query_count_discipline

```

**Checked EasyCrypt statement.**

```

op budgeted_programmed_query_count_discipline
  (hash_count signing_count : int) : bool =
  0 <= hash_count /\
  hash_count <= hash_query_budget_count /\
  0 <= signing_count /\
  signing_count <= signature_query_budget_count.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.13** (EasyCrypt line 145). **EasyCrypt name.**

```

budgeted_paper_sim_signing_count_discipline

```

**Checked EasyCrypt statement.**

```

op budgeted_paper_sim_signing_count_discipline
  (signing_count : int) : bool =
  0 <= signing_count /\
  signing_count <= signature_query_budget_count.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.14** (EasyCrypt line 150). **EasyCrypt name.**

```

budgeted_paper_sim_sampler_freshness_discipline

```

**Checked EasyCrypt statement.**

```

op budgeted_paper_sim_sampler_freshness_discipline
  (hash_count signing_count : int)
  (hash_qs sampler_qs : ro_query list) : bool =
  0 <= hash_count /\
  hash_count <= hash_query_budget_count /\
  size hash_qs <= hash_count /\
  0 <= signing_count /\

```

```
signing_count <= signature_query_budget_count /\
size sampler_qs <= signing_count.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 3.15** (EasyCrypt line 160). **EasyCrypt name.**

```
sampler_rom_covered
```

**Checked EasyCrypt statement.**

```
op sampler_rom_covered
  (rom : (ro_query, ro_output * PROM.flag) fmap)
  (hash_qs sampler_qs : ro_query list) : bool =
  forall seed_coins,
    sampler_expand_query seed_coins \in rom =>
    sampler_expand_query seed_coins \in hash_qs /\
    sampler_expand_query seed_coins \in sampler_qs.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 3.16** (EasyCrypt line 205). **EasyCrypt name.**

```
budgeted_programmed_reprogram_discipline
```

**Checked EasyCrypt statement.**

```
op budgeted_programmed_reprogram_discipline
  (pk : pkey) (sk : skey)
  (programmed : programming_site list) (bad_reprogram : bool) : bool =
  pk = public_key_of_secret haetae_mode sk /\
  programming_site_log_conflict_free_for_honest haetae_mode sk programmed /\
  ! bad_reprogram.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.17** (EasyCrypt line 212). **EasyCrypt name.**

```
budgeted_programmed_challenge_query_discipline
```

**Checked EasyCrypt statement.**

```
op budgeted_programmed_challenge_query_discipline
  (hash_count : int) (queries : ro_query list) : bool =
  0 <= hash_count /\
  hash_count <= hash_query_budget_count /\
  challenge_query_log_count queries <= hash_count + 1.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.18** (EasyCrypt line 332). **EasyCrypt name.**

```
internal_transcript_state_sound
```

**Checked EasyCrypt statement.**

```
op internal_transcript_state_sound
  (md : mode) (pk : pkey) (sk : skey) (qs : SIG.query list)
  (trs : transcript list) (rs : signing_transcript_record list) : bool =
  pk = public_key_of_secret md sk /\
  transcript_log_consistent md pk qs trs rs.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 3.19** (EasyCrypt line 442). **EasyCrypt name.**

```
EUFCMA_TranscriptSimulatedSign
```

**Checked EasyCrypt statement.**

```
module EUFCMA_TranscriptSimulatedSign(
  H : SIG.Oracle,
  S : SIG.Scheme,
  A : SIG.Adversary,
  Sim : SigningSimulator
) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 3.20** (EasyCrypt line 453). **EasyCrypt name.**

```
0
```

**Checked EasyCrypt statement.**

```
module 0 = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.21** (EasyCrypt line 454). **EasyCrypt name.**

```
sign
```

**Checked EasyCrypt statement.**

```
proc sign(m : message, ctx : context) : signature = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.22** (EasyCrypt line 467). **EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
module A = A(H, 0)

proc main() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.23** (EasyCrypt line 490). **EasyCrypt name.**

```
EUFCMA_InternalTranscriptSign
```

**Checked EasyCrypt statement.**

```
module EUFCMA_InternalTranscriptSign(
  H : SIG.Oracle,
  A : SIG.Adversary
) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 3.24** (EasyCrypt line 500). **EasyCrypt name.**

```
0
```

**Checked EasyCrypt statement.**

```
module 0 = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.25** (EasyCrypt line 501). **EasyCrypt name.**

```
sign
```

**Checked EasyCrypt statement.**

```
proc sign(m : message, ctx : context) : signature = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.26** (EasyCrypt line 517). **EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
module A = A(H, 0)

proc main() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.27** (EasyCrypt line 544). **EasyCrypt name.**

```
EUFCMA_ROMInternalTranscriptSign
```

**Checked EasyCrypt statement.**

```
module EUFCMA_ROMInternalTranscriptSign(
  H : SIG.Oracle,
  A : SIG.Adversary
) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 3.28** (EasyCrypt line 554). **EasyCrypt name.**

```
0
```

**Checked EasyCrypt statement.**

```
module 0 = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.29** (EasyCrypt line 555). **EasyCrypt name.**

```
sign
```

**Checked EasyCrypt statement.**

```
proc sign(m : message, ctx : context) : signature = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.30** (EasyCrypt line 584). **EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
module A = A(H, 0)

proc main() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.31** (EasyCrypt line 613). **EasyCrypt name.**

```
ROMInternalTranscriptAsNMA
```

**Checked EasyCrypt statement.**

```
module ROMInternalTranscriptAsNMA(A : SIG.Adversary) (H : SIG.POracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 3.32** (EasyCrypt line 620). **EasyCrypt name.**

```
0
```

**Checked EasyCrypt statement.**

```
module 0 = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.33** (EasyCrypt line 621). **EasyCrypt name.**

```
sign
```

**Checked EasyCrypt statement.**

```
proc sign(m : message, ctx : context) : signature = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.34** (EasyCrypt line 650). **EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
module A = A(H, 0)

proc forge(pk : pkey) : message * context * signature = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.35** (EasyCrypt line 665). **EasyCrypt name.**

```
public_sim_source
```

**Checked EasyCrypt statement.**

```
op public_sim_source
  (pk : pkey) (m : message) (ctx : context)
  (coins : random_coins) : byte list =
  coins ++ pk.'1 ++ ctx ++ m.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.36** (EasyCrypt line 670). **EasyCrypt name.**

```
public_sim_response_aux_vector
```

**Checked EasyCrypt statement.**

```
op public_sim_response_aux_vector :
  mode -> pkey -> message -> context -> random_coins -> polyveck =
  fun md pk m ctx coins =>
    deterministic_unit_polyveck md 29
    (public_sim_source pk m ctx coins).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.37** (EasyCrypt line 676). **EasyCrypt name.**

```
public_sim_response_vector
```

**Checked EasyCrypt statement.**



```

op public_sim_response_vector :
  mode -> pkey -> message -> context -> random_coins -> polyvecl =
  fun md pk m ctx coins =>
    deterministic_unit_polyvecl md 17
    (public_sim_source pk m ctx coins).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.38** (EasyCrypt line 682). **EasyCrypt name.**

```

public_sim_hint_vector

```

**Checked EasyCrypt statement.**

```

op public_sim_hint_vector :
  mode -> pkey -> message -> context -> random_coins -> hint_t =
  fun md pk m ctx coins =>
    deterministic_unit_polyveck md 41
    (public_sim_source pk m ctx coins).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.39** (EasyCrypt line 688). **EasyCrypt name.**

```

public_sim_commitment_lowbits

```

**Checked EasyCrypt statement.**

```

op public_sim_commitment_lowbits :
  mode -> pkey -> message -> context -> random_coins -> poly =
  fun md pk m ctx coins =>
    let raw = deterministic_polyveck md
    (public_sim_source pk m ctx coins) in
    let row0 = nth poly_zero raw 0 in
    mkseq
    (fun i =>
      if i = 0 then signing_entropy_token_of_coins coins
      else poly_coeff row0 i)
    n.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.40** (EasyCrypt line 700). **EasyCrypt name.**

```

public_sim_commitment_challenge

```

**Checked EasyCrypt statement.**

```

op public_sim_commitment_challenge :
  mode -> pkey -> message -> context -> random_coins -> challenge =
  fun md pk m ctx coins =>
    challenge_hash md

```

```
(public_sim_response_aux_vector md pk m ctx coins)
(public_sim_commitment_lowbits md pk m ctx coins)
(message_hash pk ctx m).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.41** (EasyCrypt line 708). **EasyCrypt name.**

```
public_sim_commitment_highbits
```

**Checked EasyCrypt statement.**

```
op public_sim_commitment_highbits :
  mode -> pkey -> message -> context -> random_coins -> polyveck =
  fun md pk m ctx coins =>
    let ch = public_sim_commitment_challenge md pk m ctx coins in
    reconstructed_highbits md
      (public_sim_response_aux_vector md pk m ctx coins)
      (public_key_challenge_term md pk ch).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.42** (EasyCrypt line 716). **EasyCrypt name.**

```
public_sim_signature_token
```

**Checked EasyCrypt statement.**

```
op public_sim_signature_token :
  mode -> pkey -> message -> context -> random_coins -> sig_token =
  fun md pk m ctx coins =>
    ( public_sim_response_vector md pk m ctx coins,
      public_sim_response_aux_vector md pk m ctx coins,
      public_sim_hint_vector md pk m ctx coins ).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 3.43** (EasyCrypt line 723). **EasyCrypt name.**

```
public_sim_signature
```

**Checked EasyCrypt statement.**

```
op public_sim_signature :
  mode -> pkey -> message -> context -> random_coins -> signature =
  fun md pk m ctx coins =>
    let highbits = public_sim_commitment_highbits md pk m ctx coins in
    let lowbits = public_sim_commitment_lowbits md pk m ctx coins in
    let mu = message_hash pk ctx m in
    let ch = public_sim_commitment_challenge md pk m ctx coins in
    (highbits, lowbits, mu, ch,
     public_sim_signature_token md pk m ctx coins, 0, 0).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.44** (EasyCrypt line 804). **EasyCrypt name.**

```
ROMInternalTranscriptPublicSimAsNMA
```

**Checked EasyCrypt statement.**

```
module ROMInternalTranscriptPublicSimAsNMA(A : SIG.Adversary)
  (H : SIG.POracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 3.45** (EasyCrypt line 811). **EasyCrypt name.**

```
0
```

**Checked EasyCrypt statement.**

```
module 0 = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.46** (EasyCrypt line 812). **EasyCrypt name.**

```
sign
```

**Checked EasyCrypt statement.**

```
proc sign(m : message, ctx : context) : signature = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.47** (EasyCrypt line 841). **EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
module A = A(H, 0)

proc forge(pk : pkey) : message * context * signature = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.48** (EasyCrypt line 855). **EasyCrypt name.**

```
paper_sim_signature_sample
```

**Checked EasyCrypt statement.**

```
type paper_sim_signature_sample = sig_token * poly * int.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 3.49** (EasyCrypt line 857). **EasyCrypt name.**

```
paper_sim_sample_token
```

**Checked EasyCrypt statement.**

```
op paper_sim_sample_token (s : paper_sim_signature_sample) : sig_token =  
  s.'1.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 3.50** (EasyCrypt line 859). **EasyCrypt name.**

```
paper_sim_sample_lowbits_carrier
```

**Checked EasyCrypt statement.**

```
op paper_sim_sample_lowbits_carrier  
  (s : paper_sim_signature_sample) : poly =  
  s.'2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 3.51** (EasyCrypt line 862). **EasyCrypt name.**

```
paper_sim_sample_entropy
```

**Checked EasyCrypt statement.**

```
op paper_sim_sample_entropy (s : paper_sim_signature_sample) : int =  
  s.'3.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 3.52** (EasyCrypt line 864). **EasyCrypt name.**

```
paper_sim_sample_response
```

**Checked EasyCrypt statement.**

```
op paper_sim_sample_response (s : paper_sim_signature_sample) : polyvec1 =  
  (paper_sim_sample_token s).‘1.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 3.53** (EasyCrypt line 866). **EasyCrypt name.**

```
paper_sim_sample_response_aux
```

**Checked EasyCrypt statement.**

```
op paper_sim_sample_response_aux  
  (s : paper_sim_signature_sample) : polyveck =  
  (paper_sim_sample_token s).‘2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 3.54** (EasyCrypt line 869). **EasyCrypt name.**

```
paper_sim_sample_hint
```

**Checked EasyCrypt statement.**

```
op paper_sim_sample_hint (s : paper_sim_signature_sample) : hint_t =  
  (paper_sim_sample_token s).‘3.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 3.55** (EasyCrypt line 872). **EasyCrypt name.**

```
paper_sim_commitment_lowbits
```

**Checked EasyCrypt statement.**

```
op paper_sim_commitment_lowbits  
  (md : mode) (s : paper_sim_signature_sample) : poly =  
  mkseq  
    (fun i =>  
      if i = 0 then paper_sim_sample_entropy s  
      else poly_coeff (paper_sim_sample_lowbits_carrier s) i)  
  n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.56** (EasyCrypt line 880). **EasyCrypt name.**

```
paper_sim_commitment_challenge
```

**Checked EasyCrypt statement.**

```
op paper_sim_commitment_challenge
  (md : mode) (pk : pkey) (m : message) (ctx : context)
  (s : paper_sim_signature_sample) : challenge =
  challenge_hash md
    (paper_sim_sample_response_aux s)
    (paper_sim_commitment_lowbits md s)
    (message_hash pk ctx m).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.57** (EasyCrypt line 888). **EasyCrypt name.**

```
paper_sim_commitment_highbits
```

**Checked EasyCrypt statement.**

```
op paper_sim_commitment_highbits
  (md : mode) (pk : pkey) (m : message) (ctx : context)
  (s : paper_sim_signature_sample) : polyveck =
  let ch = paper_sim_commitment_challenge md pk m ctx s in
  reconstructed_highbits md
    (paper_sim_sample_response_aux s)
    (public_key_challenge_term md pk ch).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.58** (EasyCrypt line 896). **EasyCrypt name.**

```
paper_sim_signature
```

**Checked EasyCrypt statement.**

```
op paper_sim_signature
  (md : mode) (pk : pkey) (m : message) (ctx : context)
  (s : paper_sim_signature_sample) : signature =
  let highbits = paper_sim_commitment_highbits md pk m ctx s in
  let lowbits = paper_sim_commitment_lowbits md s in
  let mu = message_hash pk ctx m in
  let ch = paper_sim_commitment_challenge md pk m ctx s in
  (highbits, lowbits, mu, ch, paper_sim_sample_token s, 0, 0).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.59** (EasyCrypt line 905). **EasyCrypt name.**

```
paper_sim_signature_sample_wf
```

**Checked EasyCrypt statement.**

```
op paper_sim_signature_sample_wf
  (md : mode) (s : paper_sim_signature_sample) : bool =
  polyvecl_wf md (paper_sim_sample_response s) /\
  polyveck_wf md (paper_sim_sample_response_aux s) /\
  polyveck_wf md (paper_sim_sample_hint s) /\
  poly_wf (paper_sim_sample_lowbits_carrier s).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 3.60** (EasyCrypt line 931). **EasyCrypt name.**

```
public_sim_lowbits_carrier
```

**Checked EasyCrypt statement.**

```
op public_sim_lowbits_carrier
  (md : mode) (pk : pkey) (m : message) (ctx : context)
  (coins : random_coins) : poly =
  nth poly_zero
  (deterministic_polyveck md (public_sim_source pk m ctx coins)) 0.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.61** (EasyCrypt line 937). **EasyCrypt name.**

```
paper_sim_sample_from_public_coins
```

**Checked EasyCrypt statement.**

```
op paper_sim_sample_from_public_coins
  (md : mode) (pk : pkey) (m : message) (ctx : context)
  (coins : random_coins) : paper_sim_signature_sample =
  ( public_sim_signature_token md pk m ctx coins,
    public_sim_lowbits_carrier md pk m ctx coins,
    signing_entropy_token_of_coins coins).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 3.62** (EasyCrypt line 1044). **EasyCrypt name.**

```
real_signing_lowbits_carrier
```

**Checked EasyCrypt statement.**

```

op real_signing_lowbits_carrier
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (coins : random_coins) : poly =
  nth poly_zero (commitment_raw md sk m ctx coins) 0.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.63** (EasyCrypt line 1049). **EasyCrypt name.**

```

paper_sim_sample_from_real_signing_coins

```

**Checked EasyCrypt statement.**

```

op paper_sim_sample_from_real_signing_coins
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (coins : random_coins) : paper_sim_signature_sample =
  ( signature_token md sk m ctx coins,
    real_signing_lowbits_carrier md sk m ctx coins,
    signing_entropy_token_of_coins coins).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 3.64** (EasyCrypt line 1154). **EasyCrypt name.**

```

paper_sim_sample_from_rejection_attempt

```

**Checked EasyCrypt statement.**

```

op paper_sim_sample_from_rejection_attempt
  (st : signing_attempt_state) : paper_sim_signature_sample =
  ( signing_attempt_state_token st,
    signing_attempt_state_lowbits_carrier st,
    signing_entropy_token_of_coins (signing_attempt_state_coins st)).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

**Definition 3.65** (EasyCrypt line 1160). **EasyCrypt name.**

```

paper_sim_abort_fallback_sample

```

**Checked EasyCrypt statement.**

```

op paper_sim_abort_fallback_sample
  (md : mode) : paper_sim_signature_sample =
  ((polyvecl_zero md, polyveck_zero md, polyveck_zero md), poly_zero, 0).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.



**Definition 3.66** (EasyCrypt line 1164). **EasyCrypt name.**

```
haetae_rejection_sample_from_attempt
```

**Checked EasyCrypt statement.**

```
op haetae_rejection_sample_from_attempt
  (md : mode) (pk : pkey) (m : message) (ctx : context)
  (st : signing_attempt_state) : paper_sim_signature_sample =
  if signing_attempt_state_rejection_accepts md pk m ctx st
  then paper_sim_sample_from_rejection_attempt st
  else paper_sim_abort_fallback_sample md.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

**Definition 3.67** (EasyCrypt line 1352). **EasyCrypt name.**

```
PaperSimSigningSampler
```

**Checked EasyCrypt statement.**

```
module type PaperSimSigningSampler = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.68** (EasyCrypt line 1353). **EasyCrypt name.**

```
init
```

**Checked EasyCrypt statement.**

```
proc init(pk : pkey) : unit
  proc sample_with_seed(seed_coins : random_coins, m : message, ctx : context)
    : paper_sim_signature_sample
  proc sample(m : message, ctx : context) : paper_sim_signature_sample
}.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.69** (EasyCrypt line 1359). **EasyCrypt name.**

```
ROMPaperSimSigningSampler
```

**Checked EasyCrypt statement.**

```
module ROMPaperSimSigningSampler(H : SIG.POracle) : PaperSimSigningSampler = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the oracle boundary through which adversaries and simulations interact.

**Definition 3.70** (EasyCrypt line 1362). **EasyCrypt name.**

```
init
```

**Checked EasyCrypt statement.**

```
proc init(pk : pkey) : unit = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.71** (EasyCrypt line 1366). **EasyCrypt name.**

```
sample_with_seed
```

**Checked EasyCrypt statement.**

```
proc sample_with_seed(seed_coins : random_coins, m : message, ctx : context)
  : paper_sim_signature_sample = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.72** (EasyCrypt line 1379). **EasyCrypt name.**

```
sample
```

**Checked EasyCrypt statement.**

```
proc sample(m : message, ctx : context) : paper_sim_signature_sample = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.73** (EasyCrypt line 1389). **EasyCrypt name.**

```
RealSigningPaperSimSampler
```

**Checked EasyCrypt statement.**

```
module RealSigningPaperSimSampler(H : SIG.POracle) : PaperSimSigningSampler = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.74** (EasyCrypt line 1393). **EasyCrypt name.**

```
init
```

**Checked EasyCrypt statement.**

```
proc init(pk : pkey) : unit = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.75** (EasyCrypt line 1398). **EasyCrypt name.**

```
sample_with_seed
```

**Checked EasyCrypt statement.**

```
proc sample_with_seed(seed_coins : random_coins, m : message, ctx : context)
  : paper_sim_signature_sample = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.76** (EasyCrypt line 1411). **EasyCrypt name.**

```
sample
```

**Checked EasyCrypt statement.**

```
proc sample(m : message, ctx : context) : paper_sim_signature_sample = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.77** (EasyCrypt line 1421). **EasyCrypt name.**

```
R0SigningAttemptPaperSimSampler
```

**Checked EasyCrypt statement.**

```
module R0SigningAttemptPaperSimSampler(H : SIG.POracle)
  : PaperSimSigningSampler = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.78** (EasyCrypt line 1426). **EasyCrypt name.**

```
init
```

**Checked EasyCrypt statement.**

```
proc init(pk : pkey) : unit = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.79** (EasyCrypt line 1431). **EasyCrypt name.**

```
sample_with_seed
```

**Checked EasyCrypt statement.**

```
proc sample_with_seed(seed_coins : random_coins, m : message, ctx : context)
  : paper_sim_signature_sample = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.80** (EasyCrypt line 1446). **EasyCrypt name.**

```
sample
```

**Checked EasyCrypt statement.**

```
proc sample(m : message, ctx : context) : paper_sim_signature_sample = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.81** (EasyCrypt line 1456). **EasyCrypt name.**

```
HAETARejectionPaperSimSampler
```

**Checked EasyCrypt statement.**

```
module HAETARejectionPaperSimSampler(H : SIG.POracle)
  : PaperSimSigningSampler = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.82** (EasyCrypt line 1461). **EasyCrypt name.**

```
init
```

**Checked EasyCrypt statement.**

```
proc init(pk : pkey) : unit = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.83** (EasyCrypt line 1466). **EasyCrypt name.**

```
sample_with_seed
```

**Checked EasyCrypt statement.**

```
proc sample_with_seed(seed_coins : random_coins, m : message, ctx : context)
  : paper_sim_signature_sample = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.84** (EasyCrypt line 1482). **EasyCrypt name.**

```
sample
```

**Checked EasyCrypt statement.**

```
proc sample(m : message, ctx : context) : paper_sim_signature_sample = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.85** (EasyCrypt line 1492). **EasyCrypt name.**

```
ExactHyperballHAETAERejectionPaperSimSampler
```

**Checked EasyCrypt statement.**

```
module ExactHyperballHAETAERejectionPaperSimSampler(H : SIG.POracle)
  : PaperSimSigningSampler = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.86** (EasyCrypt line 1497). **EasyCrypt name.**

```
init
```

**Checked EasyCrypt statement.**

```
proc init(pk : pkey) : unit = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.87** (EasyCrypt line 1502). **EasyCrypt name.**

```
sample_with_seed
```

**Checked EasyCrypt statement.**

```
proc sample_with_seed(seed_coins : random_coins, m : message, ctx : context)
  : paper_sim_signature_sample = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.88** (EasyCrypt line 1513). **EasyCrypt name.**

```
sample
```

**Checked EasyCrypt statement.**

```
proc sample(m : message, ctx : context) : paper_sim_signature_sample = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.89** (EasyCrypt line 1523). **EasyCrypt name.**

```
ExactHyperballPaperSimSampler
```

**Checked EasyCrypt statement.**

```
module ExactHyperballPaperSimSampler(H : SIG.POracle)
  : PaperSimSigningSampler = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.90** (EasyCrypt line 1528). **EasyCrypt name.**

```
init
```

**Checked EasyCrypt statement.**

```
proc init(pk : pkey) : unit = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.91** (EasyCrypt line 1533). **EasyCrypt name.**

```
sample_with_seed
```

**Checked EasyCrypt statement.**

```
proc sample_with_seed(seed_coins : random_coins, m : message, ctx : context)
  : paper_sim_signature_sample = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.92** (EasyCrypt line 1544). **EasyCrypt name.**

```
sample
```

**Checked EasyCrypt statement.**

```
proc sample(m : message, ctx : context) : paper_sim_signature_sample = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.93** (EasyCrypt line 1554). **EasyCrypt name.**

```
R0ExactHyperballPaperSimSampler
```

**Checked EasyCrypt statement.**

```
module R0ExactHyperballPaperSimSampler(H : SIG.POracle)
  : PaperSimSigningSampler = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.94** (EasyCrypt line 1559). **EasyCrypt name.**

```
init
```

**Checked EasyCrypt statement.**

```
proc init(pk : pkey) : unit = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.95** (EasyCrypt line 1564). **EasyCrypt name.**

```
sample_with_seed
```

**Checked EasyCrypt statement.**

```
proc sample_with_seed(seed_coins : random_coins, m : message, ctx : context)
  : paper_sim_signature_sample = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.96** (EasyCrypt line 1577). **EasyCrypt name.**

```
sample
```

**Checked EasyCrypt statement.**

```
proc sample(m : message, ctx : context) : paper_sim_signature_sample = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.97** (EasyCrypt line 1587). **EasyCrypt name.**

```
StructuralAttemptPaperSample
```

**Checked EasyCrypt statement.**

```
module StructuralAttemptPaperSample = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.98** (EasyCrypt line 1588). **EasyCrypt name.**

```
sample
```



### Checked EasyCrypt statement.

```
proc sample(sk : skey, m : message, ctx : context)
  : paper_sim_signature_sample = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.99** (EasyCrypt line 1599). **EasyCrypt name.**

```
ExactHyperballPaperSample
```

### Checked EasyCrypt statement.

```
module ExactHyperballPaperSample = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.100** (EasyCrypt line 1600). **EasyCrypt name.**

```
sample
```

### Checked EasyCrypt statement.

```
proc sample(sk : skey, m : message, ctx : context)
  : paper_sim_signature_sample = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 3.101** (EasyCrypt line 1611). **EasyCrypt name.**

```
ROMInternalTranscriptPaperSimAsNMA
```

### Checked EasyCrypt statement.

```
module ROMInternalTranscriptPaperSimAsNMA(
  A : SIG.Adversary,
  Samp : PaperSimSigningSampler
) (H : SIG.POracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 3.102** (EasyCrypt line 1620). **EasyCrypt name.**

```
0
```

**Checked EasyCrypt statement.**

```
module O = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.103** (EasyCrypt line 1621). **EasyCrypt name.**

```
sign
```

**Checked EasyCrypt statement.**

```
proc sign(m : message, ctx : context) : signature = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.104** (EasyCrypt line 1646). **EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
module A = A(H, O)

  proc forge(pk : pkey) : message * context * signature = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.105** (EasyCrypt line 1661). **EasyCrypt name.**

```
ROMInternalTranscriptBudgetedPaperSimAsNMA
```

**Checked EasyCrypt statement.**

```
module ROMInternalTranscriptBudgetedPaperSimAsNMA(
  A : SIG.Adversary,
  Samp : PaperSimSigningSampler
) (H : SIG.POracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 3.106** (EasyCrypt line 1675). **EasyCrypt name.**

```
AH
```

**Checked EasyCrypt statement.**

```
module AH = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.107** (EasyCrypt line 1676). **EasyCrypt name.**

```
get
```

**Checked EasyCrypt statement.**

```
proc get(q : ro_query) : ro_output = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.108** (EasyCrypt line 1695). **EasyCrypt name.**

```
0
```

**Checked EasyCrypt statement.**

```
module 0 = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.109** (EasyCrypt line 1696). **EasyCrypt name.**

```
sign
```

**Checked EasyCrypt statement.**

```
proc sign(m : message, ctx : context) : signature = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.110** (EasyCrypt line 1746). **EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
module A = A(AH, 0)

proc forge(pk : pkey) : message * context * signature = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.111** (EasyCrypt line 2624). **EasyCrypt name.**

```
EUFCMA_ROMInternalTranscriptCountedSign
```

**Checked EasyCrypt statement.**

```
module EUFCMA_ROMInternalTranscriptCountedSign(  
  H : Oracle,  
  A : SIG.Adversary  
) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 3.112** (EasyCrypt line 2628). **EasyCrypt name.**

```
C
```

**Checked EasyCrypt statement.**

```
module C = CountedLazyROM(H)  
  
  var pk_current : pkey  
  var sk_current : skey  
  var queries : SIG.query list  
  var transcripts : transcript list  
  var records : signing_transcript_record list  
  
  module O = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.113** (EasyCrypt line 2637). **EasyCrypt name.**

```
sign
```

**Checked EasyCrypt statement.**

```
proc sign(m : message, ctx : context) : signature = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.114** (EasyCrypt line 2667). **EasyCrypt name.**

```
A
```

### Checked EasyCrypt statement.

```

module A = A(C, 0)

proc main() : bool = {

```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

### Definition 3.115 (EasyCrypt line 2696). EasyCrypt name.

```

EUF_CMA_ROMProgrammedTranscriptCountedSign

```

### Checked EasyCrypt statement.

```

module EUF_CMA_ROMProgrammedTranscriptCountedSign(
  H : Oracle,
  A : SIG.Adversary
) = {

```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

### Definition 3.116 (EasyCrypt line 2700). EasyCrypt name.

```

C

```

### Checked EasyCrypt statement.

```

module C = CountedLazyROM(H)

var pk_current : pkey
var sk_current : skey
var queries : SIG.query list
var transcripts : transcript list
var records : signing_transcript_record list

module 0 = {

```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

### Definition 3.117 (EasyCrypt line 2709). EasyCrypt name.

```

sign

```

### Checked EasyCrypt statement.

```

proc sign(m : message, ctx : context) : signature = {

```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.118** (EasyCrypt line 2741). **EasyCrypt name.**

A

**Checked EasyCrypt statement.**

```
module A = A(C, 0)

proc main() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.119** (EasyCrypt line 2770). **EasyCrypt name.**

EUF\_CMA\_ROMProgrammedTranscriptBudgetedCountedSign

**Checked EasyCrypt statement.**

```
module EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(
  H : Oracle,
  A : SIG.Adversary
) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 3.120** (EasyCrypt line 2774). **EasyCrypt name.**

C

**Checked EasyCrypt statement.**

```
module C = CountedLazyROM(H)

var pk_current : pkey
var sk_current : skey
var queries : SIG.query list
var transcripts : transcript list
var records : signing_transcript_record list
var adversary_hash_count : int
var signing_count : int

module AH = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.121** (EasyCrypt line 2785). **EasyCrypt name.**

get

**Checked EasyCrypt statement.**

```
proc get(q : ro_query) : ro_output = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.122** (EasyCrypt line 2803). **EasyCrypt name.**

```
0
```

**Checked EasyCrypt statement.**

```
module 0 = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.123** (EasyCrypt line 2804). **EasyCrypt name.**

```
sign
```

**Checked EasyCrypt statement.**

```
proc sign(m : message, ctx : context) : signature = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.124** (EasyCrypt line 2843). **EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
module A = A(AH, 0)

proc main() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.125** (EasyCrypt line 2874). **EasyCrypt name.**

```
EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
```

**Checked EasyCrypt statement.**

```

module EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign(
  H : Oracle,
  A : SIG.Adversary,
  Samp : SigningCoinSampler
) = {

```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 3.126** (EasyCrypt line 2879). **EasyCrypt name.**

```

C

```

**Checked EasyCrypt statement.**

```

module C = CountedLazyROM(H)

var pk_current : pkey
var sk_current : skey
var queries : SIG.query list
var transcripts : transcript list
var records : signing_transcript_record list
var adversary_hash_count : int
var signing_count : int

module AH = {

```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.127** (EasyCrypt line 2890). **EasyCrypt name.**

```

get

```

**Checked EasyCrypt statement.**

```

proc get(q : ro_query) : ro_output = {

```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.128** (EasyCrypt line 2908). **EasyCrypt name.**

```

0

```

**Checked EasyCrypt statement.**

```

module 0 = {

```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.



**Definition 3.129** (EasyCrypt line 2909). **EasyCrypt name.**

```
sign
```

**Checked EasyCrypt statement.**

```
proc sign(m : message, ctx : context) : signature = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.130** (EasyCrypt line 2948). **EasyCrypt name.**

```
A
```

**Checked EasyCrypt statement.**

```
module A = A(AH, 0)

proc main() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.131** (EasyCrypt line 6615). **EasyCrypt name.**

```
structural_to_exact_hyperball_paper_sample_loss_obligation
```

**Checked EasyCrypt statement.**

```
op structural_to_exact_hyperball_paper_sample_loss_obligation
  (md : mode) : bool =
  forall (sk : skey) (m : message) (ctx : context)
    (p : paper_sim_signature_sample -> bool),
  mu (dsigning_attempt_state md sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st)) <=
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st)) +
  rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 3.132** (EasyCrypt line 8424). **EasyCrypt name.**

```
ForkingAdversary
```

**Checked EasyCrypt statement.**

```
module type ForkingAdversary(H : SIG.POracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.133** (EasyCrypt line 8425). **EasyCrypt name.**

```
fork
```

**Checked EasyCrypt statement.**

```
proc fork(pk : pkey) : transcript * transcript
}.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.134** (EasyCrypt line 8428). **EasyCrypt name.**

```
TranscriptExtractor
```

**Checked EasyCrypt statement.**

```
module type TranscriptExtractor = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.135** (EasyCrypt line 8429). **EasyCrypt name.**

```
extract
```

**Checked EasyCrypt statement.**

```
proc extract(tr1 : transcript, tr2 : transcript) :
  module_sis_solution option
}.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.136** (EasyCrypt line 8433). **EasyCrypt name.**

```
BimodalTranscriptExtractor
```

**Checked EasyCrypt statement.**

```
module type BimodalTranscriptExtractor = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.137** (EasyCrypt line 8434). **EasyCrypt name.**

```
extract
```

**Checked EasyCrypt statement.**

```
proc extract(tr1 : transcript, tr2 : transcript) :  
  bimodal_selftarget_msis_solution option  
{.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.138** (EasyCrypt line 8438). **EasyCrypt name.**

```
BimodalSpecialSoundness_Game
```

**Checked EasyCrypt statement.**

```
module BimodalSpecialSoundness_Game(  
  H : SIG.Oracle,  
  S : SIG.Scheme,  
  F : ForkingAdversary,  
  X : BimodalTranscriptExtractor  
) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 3.139** (EasyCrypt line 8444). **EasyCrypt name.**

```
main
```

**Checked EasyCrypt statement.**

```
proc main(md : mode) : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.140** (EasyCrypt line 8463). **EasyCrypt name.**

```
SpecialSoundness_Game
```

**Checked EasyCrypt statement.**

```
module SpecialSoundness_Game(  
  H : SIG.Oracle,  
  S : SIG.Scheme,  
  F : ForkingAdversary,  
  X : TranscriptExtractor  
) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 3.141** (EasyCrypt line 8469). **EasyCrypt name.**

```
main
```

**Checked EasyCrypt statement.**

```
proc main(md : mode) : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.142** (EasyCrypt line 8488). **EasyCrypt name.**

```
BimodalTranscriptExtractor_As_ModuleSIS
```

**Checked EasyCrypt statement.**

```
module BimodalTranscriptExtractor_As_ModuleSIS(  
  X : BimodalTranscriptExtractor  
) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.143** (EasyCrypt line 8491). **EasyCrypt name.**

```
extract
```

**Checked EasyCrypt statement.**

```
proc extract(tr1 : transcript, tr2 : transcript) :  
  module_sis_solution option = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.144** (EasyCrypt line 8535). **EasyCrypt name.**

```
BimodalToModuleSIS
```

**Checked EasyCrypt statement.**

```
module type BimodalToModuleSIS = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.145** (EasyCrypt line 8536). **EasyCrypt name.**

```
lift
```

**Checked EasyCrypt statement.**

```
proc lift(x : bimodal_selftarget_msis_instance,  
         sol : bimodal_selftarget_msis_solution) :  
  module_sis_solution  
{.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 3.146** (EasyCrypt line 8541). **EasyCrypt name.**

```
special_soundness_interfaces_ready
```

**Checked EasyCrypt statement.**

```
op special_soundness_interfaces_ready (md : mode) : bool =  
  fork_public_reconstruction_obligation md /\  
  forking_extraction_obligation md /\  
  bimodal_to_module_sis_lift_obligation md.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.147** (EasyCrypt line 8587). **EasyCrypt name.**

```
signing_simulator_sound
```

**Checked EasyCrypt statement.**

```
op signing_simulator_sound (md : mode) : bool =  
  forall (pk : pkey) (m : message) (ctx : context) (sig : signature),  
    signing_simulation_relation md pk m ctx sig =>  
    verify_internal md pk m ctx sig.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 3.148** (EasyCrypt line 8615). *EasyCrypt name.*

```
fs_with_aborts_interfaces_ready
```

**Checked EasyCrypt statement.**

```
op fs_with_aborts_interfaces_ready (md : mode) : bool =
  signing_simulator_sound md /\
  special_soundness_interfaces_ready md /\
  fs_with_aborts_bound_obligation.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

## Lemmas, Theorems, Relational Equivalences, and Their Proof Dependencies

**Theorem 3.1** (EasyCrypt line 88). *EasyCrypt name.*

```
transcript_log_consistent_nil
```

**Checked EasyCrypt statement.**

```
lemma transcript_log_consistent_nil md pk :
  transcript_log_consistent md pk [] [] [].
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 3.2** (EasyCrypt line 95). *EasyCrypt name.*

```
transcript_log_consistent_cons
```

**Checked EasyCrypt statement.**

```
lemma transcript_log_consistent_cons md pk qs trs rs m ctx sig tr :
  transcript_log_consistent md pk qs trs rs =>
  signing_record_relation md pk (m, ctx, sig, tr) =>
  transcript_log_consistent md pk
    ((m, ctx) :: qs)
    (tr :: trs)
    ((m, ctx, sig, tr) :: rs).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `split`, `apply`. Named identifiers syntactically cited in the proof block:*

```
qE
rel
signing_record_context
signing_record_log_sound_cons
signing_record_message
signing_record_queries_cons
signing_record_query
signing_record_transcript
signing_record_transcripts_cons
sound
trE
transcript_log_consistent
```

**Theorem 3.3** (EasyCrypt line 114). *EasyCrypt name.*

```
transcript_log_consistent_no_min_entropy
```

**Checked EasyCrypt statement.**

```
lemma transcript_log_consistent_no_min_entropy md pk qs trs rs :
  transcript_log_consistent md pk qs trs rs =>
  transcript_log_min_entropy_clear md trs.
```

**Formal mathematical reading.** *Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `apply`. Named identifiers syntactically cited in the proof block:*

```
md
pk
rs
signing_record_log_sound_transcripts_no_min_entropy
sound
transcript_log_consistent
trsE
```

**Theorem 3.4** (EasyCrypt line 124). *EasyCrypt name.*

```
transcript_log_consistent_valid
```

**Checked EasyCrypt statement.**

```
lemma transcript_log_consistent_valid md pk qs trs rs :
  transcript_log_consistent md pk qs trs rs =>
  transcript_log_valid md trs.
```

**Formal mathematical reading.** *Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.*

**Role in the proof.** *Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.*

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
md
pk
rs
signing_record_log_sound_transcripts_valid
sound
transcript_log_consistent
trsE
```

**Theorem 3.5** (EasyCrypt line 168). *EasyCrypt* name.

```
sampler_rom_covered_fresh_after_clean_seed
```

**Checked EasyCrypt statement.**

```
lemma sampler_rom_covered_fresh_after_clean_seed
  seed_coins rom hash_qs sampler_qs bad :
  sampler_rom_covered rom hash_qs sampler_qs =>
  ! (bad \ /
    sampler_expand_query seed_coins \in hash_qs \ /
    sampler_expand_query seed_coins \in sampler_qs) =>
  sampler_expand_query seed_coins \notin rom.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
sampler_rom_covered
```

**Theorem 3.6** (EasyCrypt line 179). *EasyCrypt* name.

```
sampler_rom_covered_self_log_preserves
```

**Checked EasyCrypt statement.**

```
lemma sampler_rom_covered_self_log_preserves
  seed_coins rom hash_qs sampler_qs :
  sampler_rom_covered rom hash_qs sampler_qs =>
  sampler_rom_covered rom hash_qs
  (sampler_expand_query seed_coins :: sampler_qs).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:



```
sampler_rom_covered
```

**Theorem 3.7** (EasyCrypt line 188). *EasyCrypt name.*

```
sampler_rom_covered_old_log_clean_boundary
```

**Checked EasyCrypt statement.**

```
lemma sampler_rom_covered_old_log_clean_boundary
  seed_coins rom hash_qs sampler_qs bad :
  sampler_rom_covered rom hash_qs sampler_qs =>
  ! (bad \/  
    sampler_expand_query seed_coins \in hash_qs \/  
    sampler_expand_query seed_coins \in sampler_qs) =>  
  sampler_expand_query seed_coins \notin rom /\  
  sampler_rom_covered rom hash_qs  
  (sampler_expand_query seed_coins :: sampler_qs).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
bad  
clean  
covered  
hash_qs  
rom  
sampler_qs  
sampler_rom_covered_fresh_after_clean_seed  
sampler_rom_covered_self_log_preserves  
seed_coins
```

**Theorem 3.8** (EasyCrypt line 218). *EasyCrypt name.*

```
transcript_log_consistent_ready
```

**Checked EasyCrypt statement.**

```
lemma transcript_log_consistent_ready md pk qs trs rs :  
  transcript_log_consistent md pk qs trs rs =>  
  transcript_log_ready md trs.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
consistent  
md
```

```
pk
qs
rs
transcript_log_consistent_no_min_entropy
transcript_log_consistent_valid
transcript_log_ready
trs
```

**Theorem 3.9** (EasyCrypt line 229). *EasyCrypt name.*

```
transcript_log_valid_member
```

**Checked EasyCrypt statement.**

```
lemma transcript_log_valid_member md trs tr :
  transcript_log_valid md trs =>
  tr \in trs =>
  transcript_valid md tr.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
mem_tr
transcript_log_valid
valid_all
```

**Theorem 3.10** (EasyCrypt line 239). *EasyCrypt name.*

```
transcript_log_min_entropy_clear_member
```

**Checked EasyCrypt statement.**

```
lemma transcript_log_min_entropy_clear_member md trs tr :
  transcript_log_min_entropy_clear md trs =>
  tr \in trs =>
  ! min_entropy_failure md tr.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
clear_all
mem_tr
transcript_log_min_entropy_clear
```

**Theorem 3.11** (EasyCrypt line 249). *EasyCrypt name.*

```
transcript_log_ready_member_valid
```

**Checked EasyCrypt statement.**

```
lemma transcript_log_ready_member_valid md trs tr :  
  transcript_log_ready md trs =>  
  tr \in trs =>  
  transcript_valid md tr.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
md  
mem_tr  
tr  
transcript_log_ready  
transcript_log_valid_member  
trs  
valid
```

**Theorem 3.12** (EasyCrypt line 259). *EasyCrypt name.*

```
transcript_log_ready_member_no_min_entropy
```

**Checked EasyCrypt statement.**

```
lemma transcript_log_ready_member_no_min_entropy md trs tr :  
  transcript_log_ready md trs =>  
  tr \in trs =>  
  ! min_entropy_failure md tr.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
clear_log  
md  
mem_tr  
tr  
transcript_log_min_entropy_clear_member  
transcript_log_ready  
trs
```

**Theorem 3.13** (EasyCrypt line 269). *EasyCrypt name.*

```
transcript_log_ready_no_failure_for_matching_site
```

**Checked EasyCrypt statement.**

```
lemma transcript_log_ready_no_failure_for_matching_site md trs tr site :  
  transcript_log_ready md trs =>  
  tr \in trs =>  
  programming_site_matches_transcript md tr site =>  
  ! fs_with_aborts_failure md tr site.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
fs_with_aborts_failure  
match_site  
md  
mem_tr  
ready  
reprogramming_failure  
tr  
transcript_log_ready_member_no_min_entropy  
trs
```

**Theorem 3.14** (EasyCrypt line 280). **EasyCrypt name.**

```
transcript_log_ready_no_failure_for_programming_site
```

**Checked EasyCrypt statement.**

```
lemma transcript_log_ready_no_failure_for_programming_site md trs tr y :  
  transcript_log_ready md trs =>  
  tr \in trs =>  
  transcript_challenge_matches tr y =>  
  ! fs_with_aborts_failure md tr  
    (programming_site_of_transcript md tr y).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
match_y  
md  
mem_tr  
programming_site_of_transcript  
programming_site_self_matches  
ready  
tr  
transcript_log_ready_no_failure_for_matching_site
```

```
trs
```

**Theorem 3.15** (EasyCrypt line 293). *EasyCrypt name.*

```
transcript_log_ready_no_failure_for_signature_programming_site
```

*Checked EasyCrypt statement.*

```
lemma transcript_log_ready_no_failure_for_signature_programming_site
  md trs tr y :
  transcript_log_ready md trs =>
  tr \in trs =>
  transcript_signature_challenge_matches tr y =>
  ! fs_with_aborts_failure md tr
    (signature_programming_site_of_transcript md tr y).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *apply*. Named identifiers syntactically cited in the proof block:

```
match_y
md
mem_tr
ready
signature_programming_site_of_transcript
tr
transcript_log_ready_member_valid
transcript_log_ready_no_failure_for_matching_site
transcript_valid_signature_programming_site_matches
trs
valid
```

**Theorem 3.16** (EasyCrypt line 308). *EasyCrypt name.*

```
transcript_log_ready_actual_signature_site_clear_member
```

*Checked EasyCrypt statement.*

```
lemma transcript_log_ready_actual_signature_site_clear_member md trs tr :
  transcript_log_ready md trs =>
  tr \in trs =>
  transcript_signature_programming_site_clear md tr.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
actual_signature_programming_site
```

```
md
mem_tr
ready
tr
transcript_log_ready_no_failure_for_signature_programming_site
transcript_signature_challenge_output
transcript_signature_challenge_output_matches
transcript_signature_programming_site_clear
trs
```

**Theorem 3.17** (EasyCrypt line 322). *EasyCrypt name.*

```
transcript_log_ready_signature_sites_clear
```

**Checked EasyCrypt statement.**

```
lemma transcript_log_ready_signature_sites_clear md trs :
  transcript_log_ready md trs =>
  transcript_log_signature_programming_sites_clear md trs.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
md
mem_tr
ready
tr
transcript_log_ready_actual_signature_site_clear_member
transcript_log_signature_programming_sites_clear
trs
```

**Theorem 3.18** (EasyCrypt line 338). *EasyCrypt name.*

```
internal_transcript_state_sound_nil
```

**Checked EasyCrypt statement.**

```
lemma internal_transcript_state_sound_nil md sk :
  internal_transcript_state_sound md
  (public_key_of_secret md sk) sk [] [] [].
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
internal_transcript_state_sound
```

```
transcript_log_consistent_nil
trivial
```

**Theorem 3.19** (EasyCrypt line 348). *EasyCrypt name.*

```
internal_transcript_state_sound_keygen
```

**Checked EasyCrypt statement.**

```
lemma internal_transcript_state_sound_keygen md sd :
  internal_transcript_state_sound md
    (keygen_internal md sd).‘1
    (keygen_internal md sd).‘2
  [] [] [].
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
internal_transcript_state_sound
keygen_internal
transcript_log_consistent_nil
```

**Theorem 3.20** (EasyCrypt line 360). *EasyCrypt name.*

```
internal_transcript_state_sound_no_min_entropy
```

**Checked EasyCrypt statement.**

```
lemma internal_transcript_state_sound_no_min_entropy
  md pk sk qs trs rs :
  internal_transcript_state_sound md pk sk qs trs rs =>
  transcript_log_min_entropy_clear md trs.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
internal_transcript_state_sound
log_sound
md
pk
qs
rs
transcript_log_consistent_no_min_entropy
trs
```

**Theorem 3.21** (EasyCrypt line 370). *EasyCrypt name.*

```
internal_transcript_state_sound_valid
```

**Checked EasyCrypt statement.**

```
lemma internal_transcript_state_sound_valid md pk sk qs trs rs :  
  internal_transcript_state_sound md pk sk qs trs rs =>  
  transcript_log_valid md trs.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
internal_transcript_state_sound  
log_sound  
md  
pk  
qs  
rs  
transcript_log_consistent_valid  
trs
```

**Theorem 3.22** (EasyCrypt line 379). *EasyCrypt name.*

```
internal_transcript_state_sound_log_ready
```

**Checked EasyCrypt statement.**

```
lemma internal_transcript_state_sound_log_ready md pk sk qs trs rs :  
  internal_transcript_state_sound md pk sk qs trs rs =>  
  transcript_log_ready md trs.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
internal_transcript_state_sound  
log_sound  
md  
pk  
qs  
rs  
transcript_log_consistent_ready  
trs
```

**Theorem 3.23** (EasyCrypt line 388). *EasyCrypt name.*

```
transcript_log_consistent_sign_internal_cons
```



### Checked EasyCrypt statement.

```
lemma transcript_log_consistent_sign_internal_cons
  md sk qs trs rs m ctx coins :
  transcript_log_consistent md (public_key_of_secret md sk) qs trs rs =>
  transcript_log_consistent md
    (public_key_of_secret md sk)
    ((m, ctx) :: qs)
    (transcript_of_signature md (public_key_of_secret md sk) m ctx
      (sign_internal md sk m ctx coins) :: trs)
    ((m, ctx, sign_internal md sk m ctx coins,
      transcript_of_signature md (public_key_of_secret md sk) m ctx
      (sign_internal md sk m ctx coins)) :: rs).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
log_sound
sign_internal_record_relation
transcript_log_consistent_cons
```

### Theorem 3.24 (EasyCrypt line 406). *EasyCrypt* name.

```
internal_transcript_state_sound_sign_internal_cons
```

### Checked EasyCrypt statement.

```
lemma internal_transcript_state_sound_sign_internal_cons
  md pk sk qs trs rs m ctx coins :
  internal_transcript_state_sound md pk sk qs trs rs =>
  internal_transcript_state_sound md pk sk
    ((m, ctx) :: qs)
    (transcript_of_signature md pk m ctx
      (sign_internal md sk m ctx coins) :: trs)
    ((m, ctx, sign_internal md sk m ctx coins,
      transcript_of_signature md pk m ctx
      (sign_internal md sk m ctx coins)) :: rs).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
coins
ctx
internal_transcript_state_sound
log_sound
md
pkE
qs
```

```
rs
sk
transcript_log_consistent_sign_internal_cons
trs
```

**Theorem 3.25** (EasyCrypt line 428). *EasyCrypt name.*

```
sign_internal_transcript_no_min_entropy
```

**Checked EasyCrypt statement.**

```
lemma sign_internal_transcript_no_min_entropy md pk sk m ctx coins :
  pk = public_key_of_secret md sk =>
  ! min_entropy_failure md
    (transcript_of_signature md pk m ctx
     (sign_internal md sk m ctx coins)).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
coins
ctx
md
pkE
public_key_of_secret
sign_internal
sign_internal_transcript_relation
signing_transcript_relation_no_min_entropy_failure
sk
```

**Theorem 3.26** (EasyCrypt line 733). *EasyCrypt name.*

```
public_sim_commitment_lowbitsE
```

**Checked EasyCrypt statement.**

```
lemma public_sim_commitment_lowbitsE md sk m ctx coins :
  public_sim_commitment_lowbits md (public_key_of_secret md sk) m ctx coins =
  commitment_lowbits md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
commitment_lowbits
commitment_raw
commitment_source
```

```

public_key_of_secret
public_sim_commitment_lowbits
public_sim_source

```

**Theorem 3.27** (EasyCrypt line 742). *EasyCrypt name.*

```

public_sim_response_aux_vectorE

```

**Checked EasyCrypt statement.**

```

lemma public_sim_response_aux_vectorE md sk m ctx coins :
  public_sim_response_aux_vector md (public_key_of_secret md sk) m ctx coins =
  response_aux_vector md sk m ctx coins.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

public_key_of_secret
public_sim_response_aux_vector
public_sim_source
response_aux_vector
response_source

```

**Theorem 3.28** (EasyCrypt line 750). *EasyCrypt name.*

```

public_sim_commitment_challengeE

```

**Checked EasyCrypt statement.**

```

lemma public_sim_commitment_challengeE md sk m ctx coins :
  public_sim_commitment_challenge md (public_key_of_secret md sk) m ctx coins =
  commitment_challenge md sk m ctx coins.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

commitment_challenge
public_sim_commitment_challenge
public_sim_commitment_lowbitsE
public_sim_response_aux_vectorE

```

**Theorem 3.29** (EasyCrypt line 758). *EasyCrypt name.*

```

public_sim_commitment_highbitsE

```

### Checked EasyCrypt statement.

```
lemma public_sim_commitment_highbitsE md sk m ctx coins :  
  public_sim_commitment_highbits md (public_key_of_secret md sk) m ctx coins =  
  commitment_highbits md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
commitment_highbits  
public_sim_commitment_challengeE  
public_sim_commitment_highbits  
public_sim_response_aux_vectorE
```

### Theorem 3.30 (EasyCrypt line 767). *EasyCrypt* name.

```
public_sim_response_vectorE
```

### Checked EasyCrypt statement.

```
lemma public_sim_response_vectorE md sk m ctx coins :  
  public_sim_response_vector md (public_key_of_secret md sk) m ctx coins =  
  response_vector md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
public_key_of_secret  
public_sim_response_vector  
public_sim_source  
response_source  
response_vector
```

### Theorem 3.31 (EasyCrypt line 775). *EasyCrypt* name.

```
public_sim_hint_vectorE
```

### Checked EasyCrypt statement.

```
lemma public_sim_hint_vectorE md sk m ctx coins :  
  public_sim_hint_vector md (public_key_of_secret md sk) m ctx coins =  
  hint_vector md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
hint_vector
public_key_of_secret
public_sim_hint_vector
public_sim_source
response_source
```

**Theorem 3.32** (EasyCrypt line 783). *EasyCrypt name.*

```
public_sim_signature_tokenE
```

**Checked EasyCrypt statement.**

```
lemma public_sim_signature_tokenE md sk m ctx coins :
  public_sim_signature_token md (public_key_of_secret md sk) m ctx coins =
  signature_token md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
public_sim_hint_vectorE
public_sim_response_aux_vectorE
public_sim_response_vectorE
public_sim_signature_token
signature_token
```

**Theorem 3.33** (EasyCrypt line 793). *EasyCrypt name.*

```
public_sim_signatureE
```

**Checked EasyCrypt statement.**

```
lemma public_sim_signatureE md sk m ctx coins :
  public_sim_signature md (public_key_of_secret md sk) m ctx coins =
  sign_internal md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

public_sim_commitment_challengeE
public_sim_commitment_highbitsE
public_sim_commitment_lowbitsE
public_sim_signature
public_sim_signature_tokenE
sign_internal

```

**Theorem 3.34** (EasyCrypt line 912). *EasyCrypt name.*

```

paper_sim_commitment_lowbits_wf

```

**Checked EasyCrypt statement.**

```

lemma paper_sim_commitment_lowbits_wf md s :
  poly_wf (paper_sim_commitment_lowbits md s).

```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

paper_sim_commitment_lowbits
poly_wf
size_mkseq

```

**Theorem 3.35** (EasyCrypt line 918). *EasyCrypt name.*

```

paper_sim_signature_valid

```

**Checked EasyCrypt statement.**

```

lemma paper_sim_signature_valid md pk m ctx s :
  paper_sim_signature_sample_wf md s =>
  valid_signature md (paper_sim_signature md pk m ctx s).

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

aux_wf
challenge_hash_wf
hint_wf
low_wf
message_hash_size
paper_sim_commitment_challenge
paper_sim_commitment_highbits
paper_sim_commitment_lowbits_wf
paper_sim_signature
paper_sim_signature_sample_wf

```

```
polyveck_add_wf
reconstructed_highbits
resp_wf
valid_signature
```

**Theorem 3.36** (EasyCrypt line 944). *EasyCrypt name.*

```
public_sim_lowbits_carrier_wf
```

**Checked EasyCrypt statement.**

```
lemma public_sim_lowbits_carrier_wf md pk m ctx coins :
  poly_wf (public_sim_lowbits_carrier md pk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
coins
ctx
deterministic_polyveck
deterministic_polyveck_wf
md
mode_k_gt0
pk
polyveck_wf_nth
public_sim_lowbits_carrier
public_sim_source
```

**Theorem 3.37** (EasyCrypt line 955). *EasyCrypt name.*

```
paper_sim_sample_from_public_coins_wf
```

**Checked EasyCrypt statement.**

```
lemma paper_sim_sample_from_public_coins_wf md pk m ctx coins :
  paper_sim_signature_sample_wf md
    (paper_sim_sample_from_public_coins md pk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
deterministic_unit_polyveck_wf
deterministic_unit_polyveckl_wf
paper_sim_sample_from_public_coins
paper_sim_sample_hint
paper_sim_sample_lowbits_carrier
```

```

paper_sim_sample_response
paper_sim_sample_response_aux
paper_sim_sample_token
paper_sim_signature_sample_wf
public_sim_hint_vector
public_sim_lowbits_carrier_wf
public_sim_response_aux_vector
public_sim_response_vector
public_sim_signature_token

```

**Theorem 3.38** (EasyCrypt line 970). *EasyCrypt name.*

```
paper_sim_sample_response_public_coinsE
```

*Checked EasyCrypt statement.*

```

lemma paper_sim_sample_response_public_coinsE md pk m ctx coins :
  paper_sim_sample_response
    (paper_sim_sample_from_public_coins md pk m ctx coins) =
  public_sim_response_vector md pk m ctx coins.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 3.39** (EasyCrypt line 978). *EasyCrypt name.*

```
paper_sim_sample_response_aux_public_coinsE
```

*Checked EasyCrypt statement.*

```

lemma paper_sim_sample_response_aux_public_coinsE md pk m ctx coins :
  paper_sim_sample_response_aux
    (paper_sim_sample_from_public_coins md pk m ctx coins) =
  public_sim_response_aux_vector md pk m ctx coins.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 3.40** (EasyCrypt line 986). *EasyCrypt name.*

```
paper_sim_sample_hint_public_coinsE
```

*Checked EasyCrypt statement.*

```

lemma paper_sim_sample_hint_public_coinsE md pk m ctx coins :
  paper_sim_sample_hint
    (paper_sim_sample_from_public_coins md pk m ctx coins) =
  public_sim_hint_vector md pk m ctx coins.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.



**Theorem 3.41** (EasyCrypt line 994). *EasyCrypt name.*

```
paper_sim_sample_token_public_coinsE
```

**Checked EasyCrypt statement.**

```
lemma paper_sim_sample_token_public_coinsE md pk m ctx coins :
  paper_sim_sample_token
    (paper_sim_sample_from_public_coins md pk m ctx coins) =
  public_sim_signature_token md pk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 3.42** (EasyCrypt line 1001). *EasyCrypt name.*

```
paper_sim_commitment_lowbits_public_coinsE
```

**Checked EasyCrypt statement.**

```
lemma paper_sim_commitment_lowbits_public_coinsE md pk m ctx coins :
  paper_sim_commitment_lowbits md
    (paper_sim_sample_from_public_coins md pk m ctx coins) =
  public_sim_commitment_lowbits md pk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
paper_sim_commitment_lowbits
paper_sim_sample_entropy
paper_sim_sample_from_public_coins
paper_sim_sample_lowbits_carrier
public_sim_commitment_lowbits
public_sim_lowbits_carrier
```

**Theorem 3.43** (EasyCrypt line 1012). *EasyCrypt name.*

```
paper_sim_commitment_challenge_public_coinsE
```

**Checked EasyCrypt statement.**

```
lemma paper_sim_commitment_challenge_public_coinsE md pk m ctx coins :
  paper_sim_commitment_challenge md pk m ctx
    (paper_sim_sample_from_public_coins md pk m ctx coins) =
  public_sim_commitment_challenge md pk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
paper_sim_commitment_challenge
paper_sim_commitment_lowbits_public_coinsE
paper_sim_sample_response_aux_public_coinsE
public_sim_commitment_challenge
```

**Theorem 3.44** (EasyCrypt line 1022). *EasyCrypt name.*

```
paper_sim_commitment_highbits_public_coinsE
```

**Checked EasyCrypt statement.**

```
lemma paper_sim_commitment_highbits_public_coinsE md pk m ctx coins :
  paper_sim_commitment_highbits md pk m ctx
  (paper_sim_sample_from_public_coins md pk m ctx coins) =
  public_sim_commitment_highbits md pk m ctx coins.
```

**Formal mathematical reading.** *Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.*

**Role in the proof.** *Use in this chapter. It proves an exact game replacement or simulator equivalence.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
paper_sim_commitment_challenge_public_coinsE
paper_sim_commitment_highbits
paper_sim_sample_response_aux_public_coinsE
public_sim_commitment_highbits
```

**Theorem 3.45** (EasyCrypt line 1032). *EasyCrypt name.*

```
paper_sim_signature_public_coinsE
```

**Checked EasyCrypt statement.**

```
lemma paper_sim_signature_public_coinsE md pk m ctx coins :
  paper_sim_signature md pk m ctx
  (paper_sim_sample_from_public_coins md pk m ctx coins) =
  public_sim_signature md pk m ctx coins.
```

**Formal mathematical reading.** *Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.*

**Role in the proof.** *Use in this chapter. It proves an exact game replacement or simulator equivalence.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
paper_sim_commitment_challenge_public_coinsE
paper_sim_commitment_highbits_public_coinsE
```

```
paper_sim_commitment_lowbits_public_coinsE
paper_sim_sample_token_public_coinsE
paper_sim_signature
public_sim_signature
```

**Theorem 3.46** (EasyCrypt line 1056). *EasyCrypt name.*

```
real_signing_lowbits_carrier_wf
```

**Checked EasyCrypt statement.**

```
lemma real_signing_lowbits_carrier_wf md sk m ctx coins :
  poly_wf (real_signing_lowbits_carrier md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
coins
commitment_raw
commitment_raw_wf
ctx
md
mode_k_gt0
polyveck_wf_nth
real_signing_lowbits_carrier
sk
```

**Theorem 3.47** (EasyCrypt line 1065). *EasyCrypt name.*

```
paper_sim_sample_from_real_signing_coins_wf
```

**Checked EasyCrypt statement.**

```
lemma paper_sim_sample_from_real_signing_coins_wf md sk m ctx coins :
  paper_sim_signature_sample_wf md
    (paper_sim_sample_from_real_signing_coins md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
hint_vector_wf
paper_sim_sample_from_real_signing_coins
paper_sim_sample_hint
paper_sim_sample_lowbits_carrier
paper_sim_sample_response
paper_sim_sample_response_aux
```

```

paper_sim_sample_token
paper_sim_signature_sample_wf
real_signing_lowbits_carrier_wf
response_aux_vector_wf
response_vector_wf

```

**Theorem 3.48** (EasyCrypt line 1078). *EasyCrypt name.*

```

paper_sim_sample_response_real_signing_coinsE

```

*Checked EasyCrypt statement.*

```

lemma paper_sim_sample_response_real_signing_coinsE md sk m ctx coins :
  paper_sim_sample_response
    (paper_sim_sample_from_real_signing_coins md sk m ctx coins) =
    response_vector md sk m ctx coins.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 3.49** (EasyCrypt line 1086). *EasyCrypt name.*

```

paper_sim_sample_response_aux_real_signing_coinsE

```

*Checked EasyCrypt statement.*

```

lemma paper_sim_sample_response_aux_real_signing_coinsE md sk m ctx coins :
  paper_sim_sample_response_aux
    (paper_sim_sample_from_real_signing_coins md sk m ctx coins) =
    response_aux_vector md sk m ctx coins.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 3.50** (EasyCrypt line 1094). *EasyCrypt name.*

```

paper_sim_sample_hint_real_signing_coinsE

```

*Checked EasyCrypt statement.*

```

lemma paper_sim_sample_hint_real_signing_coinsE md sk m ctx coins :
  paper_sim_sample_hint
    (paper_sim_sample_from_real_signing_coins md sk m ctx coins) =
    hint_vector md sk m ctx coins.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 3.51** (EasyCrypt line 1102). *EasyCrypt name.*

```
paper_sim_sample_token_real_signing_coinsE
```

**Checked EasyCrypt statement.**

```
lemma paper_sim_sample_token_real_signing_coinsE md sk m ctx coins :  
  paper_sim_sample_token  
    (paper_sim_sample_from_real_signing_coins md sk m ctx coins) =  
    signature_token md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 3.52** (EasyCrypt line 1109). *EasyCrypt name.*

```
paper_sim_commitment_lowbits_real_signing_coinsE
```

**Checked EasyCrypt statement.**

```
lemma paper_sim_commitment_lowbits_real_signing_coinsE md sk m ctx coins :  
  paper_sim_commitment_lowbits md  
    (paper_sim_sample_from_real_signing_coins md sk m ctx coins) =  
    commitment_lowbits md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
commitment_lowbits  
paper_sim_commitment_lowbits  
paper_sim_sample_entropy  
paper_sim_sample_from_real_signing_coins  
paper_sim_sample_lowbits_carrier  
real_signing_lowbits_carrier
```

**Theorem 3.53** (EasyCrypt line 1120). *EasyCrypt name.*

```
paper_sim_commitment_challenge_real_signing_coinsE
```

**Checked EasyCrypt statement.**

```
lemma paper_sim_commitment_challenge_real_signing_coinsE  
  md sk m ctx coins :  
  paper_sim_commitment_challenge md (public_key_of_secret md sk) m ctx  
    (paper_sim_sample_from_real_signing_coins md sk m ctx coins) =  
    commitment_challenge md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
commitment_challenge
paper_sim_commitment_challenge
paper_sim_commitment_lowbits_real_signing_coinsE
paper_sim_sample_response_aux_real_signing_coinsE
```

**Theorem 3.54** (EasyCrypt line 1131). *EasyCrypt name.*

```
paper_sim_commitment_highbits_real_signing_coinsE
```

**Checked EasyCrypt statement.**

```
lemma paper_sim_commitment_highbits_real_signing_coinsE
  md sk m ctx coins :
  paper_sim_commitment_highbits md (public_key_of_secret md sk) m ctx
    (paper_sim_sample_from_real_signing_coins md sk m ctx coins) =
  commitment_highbits md sk m ctx coins.
```

**Formal mathematical reading.** *Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.*

**Role in the proof.** *Use in this chapter. It proves an exact game replacement or simulator equivalence.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
commitment_highbits
paper_sim_commitment_challenge_real_signing_coinsE
paper_sim_commitment_highbits
paper_sim_sample_response_aux_real_signing_coinsE
```

**Theorem 3.55** (EasyCrypt line 1142). *EasyCrypt name.*

```
paper_sim_signature_real_signing_coinsE
```

**Checked EasyCrypt statement.**

```
lemma paper_sim_signature_real_signing_coinsE md sk m ctx coins :
  paper_sim_signature md (public_key_of_secret md sk) m ctx
    (paper_sim_sample_from_real_signing_coins md sk m ctx coins) =
  sign_internal md sk m ctx coins.
```

**Formal mathematical reading.** *Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.*

**Role in the proof.** *Use in this chapter. It proves an exact game replacement or simulator equivalence.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
paper_sim_commitment_challenge_real_signing_coinsE
```

```

paper_sim_commitment_highbits_real_signing_coinsE
paper_sim_commitment_lowbits_real_signing_coinsE
paper_sim_sample_token_real_signing_coinsE
paper_sim_signature
sign_internal

```

**Theorem 3.56** (EasyCrypt line 1171). *EasyCrypt name.*

```

paper_sim_abort_fallback_sample_wf

```

**Checked EasyCrypt statement.**

```

lemma paper_sim_abort_fallback_sample_wf md :
  paper_sim_signature_sample_wf md (paper_sim_abort_fallback_sample md).

```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

paper_sim_abort_fallback_sample
paper_sim_sample_hint
paper_sim_sample_lowbits_carrier
paper_sim_sample_response
paper_sim_sample_response_aux
paper_sim_sample_token
paper_sim_signature_sample_wf
poly_zero_wf
polyveck_zero_wf
polyvecl_zero_wf

```

**Theorem 3.57** (EasyCrypt line 1182). *EasyCrypt name.*

```

paper_sim_sample_from_rejection_attempt_of_coinsE

```

**Checked EasyCrypt statement.**

```

lemma paper_sim_sample_from_rejection_attempt_of_coinsE md sk m ctx coins :
  paper_sim_sample_from_rejection_attempt
    (signing_attempt_state_of_coins md sk m ctx coins) =
  paper_sim_sample_from_real_signing_coins md sk m ctx coins.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

paper_sim_sample_from_real_signing_coins
paper_sim_sample_from_rejection_attempt
real_signing_lowbits_carrier

```

```

signing_attempt_state_coins
signing_attempt_state_of_coins
signing_attempt_state_of_coins_lowbits_carrierE
signing_attempt_state_of_coins_tokenE

```

**Theorem 3.58** (EasyCrypt line 1195). *EasyCrypt name.*

```

paper_sim_commitment_lowbits_rejection_attemptE

```

**Checked EasyCrypt statement.**

```

lemma paper_sim_commitment_lowbits_rejection_attemptE md st :
  paper_sim_commitment_lowbits md
    (paper_sim_sample_from_rejection_attempt st) =
  signing_attempt_state_programmed_lowbits st.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

paper_sim_commitment_lowbits
paper_sim_sample_entropy
paper_sim_sample_from_rejection_attempt
paper_sim_sample_lowbits_carrier
signing_attempt_state_programmed_lowbits

```

**Theorem 3.59** (EasyCrypt line 1207). *EasyCrypt name.*

```

paper_sim_commitment_challenge_rejection_attemptE

```

**Checked EasyCrypt statement.**

```

lemma paper_sim_commitment_challenge_rejection_attemptE md pk m ctx st :
  signing_attempt_state_lowbits_consistent st =>
  paper_sim_commitment_challenge md pk m ctx
    (paper_sim_sample_from_rejection_attempt st) =
  signing_attempt_state_programmed_challenge md pk m ctx st.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

lowbits_ok
paper_sim_commitment_challenge
paper_sim_commitment_lowbits_rejection_attemptE
paper_sim_sample_from_rejection_attempt
paper_sim_sample_response_aux
paper_sim_sample_token

```



```
signing_attempt_state_lowbits_consistent
signing_attempt_state_programmed_challenge
```

**Theorem 3.60** (EasyCrypt line 1225). *EasyCrypt name.*

```
paper_sim_commitment_highbits_rejection_attemptE
```

*Checked EasyCrypt statement.*

```
lemma paper_sim_commitment_highbits_rejection_attemptE md pk m ctx st :
  signing_attempt_state_lowbits_consistent st =>
  signing_attempt_state_challenge_consistent md pk m ctx st =>
  signing_attempt_state_public_equation_consistent md pk st =>
  paper_sim_commitment_highbits md pk m ctx
  (paper_sim_sample_from_rejection_attempt st) =
  signing_attempt_state_highbits st.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
challengeE
challenge_ok
ctx
lowbits_ok
md
paper_sim_commitment_challenge_rejection_attemptE
paper_sim_commitment_highbits
paper_sim_sample_from_rejection_attempt
paper_sim_sample_response_aux
paper_sim_sample_token
pk
publicE
public_ok
signing_attempt_state_challenge_consistent
signing_attempt_state_public_equation_consistent
signing_attempt_state_reconstructed_highbits
st
```

**Theorem 3.61** (EasyCrypt line 1249). *EasyCrypt name.*

```
paper_sim_signature_rejection_attempt_fieldsE
```

*Checked EasyCrypt statement.*

```
lemma paper_sim_signature_rejection_attempt_fieldsE md pk m ctx st :
  signing_attempt_state_field_accepts md pk m ctx st =>
  paper_sim_signature md pk m ctx
  (paper_sim_sample_from_rejection_attempt st) =
  signing_attempt_state_signature_of_fields pk m ctx st.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `have`. Named identifiers syntactically cited in the proof block:

```
challengeE
challenge_ok
ctx
fields_ok
lowbitsE
lowbits_ok
md
paper_sim_commitment_challenge_rejection_attemptE
paper_sim_commitment_highbits_rejection_attemptE
paper_sim_commitment_lowbits_rejection_attemptE
paper_sim_sample_from_rejection_attempt
paper_sim_sample_token
paper_sim_signature
pk
public_ok
signing_attempt_state_challenge_consistent
signing_attempt_state_field_accepts
signing_attempt_state_lowbits_consistent
signing_attempt_state_signature_of_fields
st
```

**Theorem 3.62** (EasyCrypt line 1277). *EasyCrypt name.*

```
paper_sim_signature_rejection_attempt_matches_state
```

**Checked EasyCrypt statement.**

```
lemma paper_sim_signature_rejection_attempt_matches_state md pk m ctx st :
  signing_attempt_state_field_accepts md pk m ctx st =>
  paper_sim_signature md pk m ctx
    (paper_sim_sample_from_rejection_attempt st) =
  signing_attempt_state_signature st.
```

**Formal mathematical reading.** *Mathematical theorem.* This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** *Use in this chapter.* It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:

```
ctx
field_ok
match_ok
md
paper_sim_signature_rejection_attempt_fieldsE
pk
signing_attempt_state_field_accepts
signing_attempt_state_fields_match_signature
st
```

**Theorem 3.63** (EasyCrypt line 1293). *EasyCrypt name.*

```
paper_sim_signature_rejection_attempt_of_coinsE
```

### Checked EasyCrypt statement.

```
lemma paper_sim_signature_rejection_attempt_of_coinsE md sk m ctx coins :  
  paper_sim_signature md (public_key_of_secret md sk) m ctx  
    (paper_sim_sample_from_rejection_attempt  
      (signing_attempt_state_of_coins md sk m ctx coins)) =  
    sign_internal md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
coins  
ctx  
md  
paper_sim_signature_rejection_attempt_matches_state  
public_key_of_secret  
signing_attempt_state_of_coins  
signing_attempt_state_of_coins_field_accepts  
signing_attempt_state_of_coins_signatureE  
sk
```

### Theorem 3.64 (EasyCrypt line 1306). *EasyCrypt* name.

```
haetae_rejection_sample_from_attempt_signatureE
```

### Checked EasyCrypt statement.

```
lemma haetae_rejection_sample_from_attempt_signatureE md pk m ctx st :  
  signing_attempt_state_rejection_accepts md pk m ctx st =>  
  paper_sim_signature md pk m ctx  
    (haetae_rejection_sample_from_attempt md pk m ctx st) =  
  signing_attempt_state_signature st.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
ctx  
field_ok  
haetae_rejection_sample_from_attempt  
md  
paper_sim_signature_rejection_attempt_matches_state  
pk  
reject_ok  
signing_attempt_state_rejection_accepts  
signing_attempt_state_verification_accepts  
st
```

**Theorem 3.65** (EasyCrypt line 1323). *EasyCrypt name.*

```
haetae_rejection_sample_from_attempt_no_fallback
```

**Checked EasyCrypt statement.**

```
lemma haetae_rejection_sample_from_attempt_no_fallback md pk m ctx st :
  signing_attempt_state_rejection_accepts md pk m ctx st =>
  haetae_rejection_sample_from_attempt md pk m ctx st =
  paper_sim_sample_from_rejection_attempt st.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_rejection_sample_from_attempt
reject_ok
```

**Theorem 3.66** (EasyCrypt line 1332). *EasyCrypt name.*

```
haetae_rejection_sample_from_attempt_of_coinsE
```

**Checked EasyCrypt statement.**

```
lemma haetae_rejection_sample_from_attempt_of_coinsE md sk m ctx coins :
  haetae_rejection_sample_from_attempt md (public_key_of_secret md sk) m ctx
  (signing_attempt_state_of_coins md sk m ctx coins) =
  paper_sim_sample_from_real_signing_coins md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_rejection_sample_from_attempt
paper_sim_sample_from_rejection_attempt_of_coinsE
signing_attempt_state_of_coins_rejection_accepts_current
```

**Theorem 3.67** (EasyCrypt line 1343). *EasyCrypt name.*

```
haetae_rejection_sample_from_attempt_of_coins_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_rejection_sample_from_attempt_of_coins_wf md sk m ctx coins :
  paper_sim_signature_sample_wf md
  (haetae_rejection_sample_from_attempt md (public_key_of_secret md sk) m ctx
  (signing_attempt_state_of_coins md sk m ctx coins)).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_rejection_sample_from_attempt_of_coinsE
paper_sim_sample_from_real_signing_coins_wf
```

**Theorem 3.68** (*EasyCrypt* line 1774). *EasyCrypt* name.

```
budgeted_paper_sim_sign_oracle_preserves_signing_count
```

**Checked *EasyCrypt* statement.**

```
lemma budgeted_paper_sim_sign_oracle_preserves_signing_count :
  hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).O.sign :
    budgeted_paper_sim_signing_count_discipline
    ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count ==>
    budgeted_paper_sim_signing_count_discipline
    ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
budgeted_paper_sim_signing_count_discipline
signature_query_budget_count
```

**Theorem 3.69** (*EasyCrypt* line 1795). *EasyCrypt* name.

```
adversary_budgeted_paper_sim_signing_count_preserved
```

**Checked *EasyCrypt* statement.**

```
lemma adversary_budgeted_paper_sim_signing_count_preserved :
  hoare[
    ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).A.forge :
    budgeted_paper_sim_signing_count_discipline
    ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count ==>
    budgeted_paper_sim_signing_count_discipline
    ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `wp`, `call`, `apply`. Named identifiers syntactically cited in the proof block:

```
ROMInternalTranscriptBudgetedPaperSimAsNMA
budgeted_paper_sim_sign_oracle_preserves_signing_count
budgeted_paper_sim_signing_count_discipline
signing_count
```

**Theorem 3.70** (EasyCrypt line 1816). *EasyCrypt name.*

```
budgeted_paper_sim_main_signing_count_preserved
```

**Checked EasyCrypt statement.**

```
lemma budgeted_paper_sim_main_signing_count_preserved :
  hoare [ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).forge :
    true ==>
    budgeted_paper_sim_signing_count_discipline
    ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count].
```

**Formal mathematical reading.** *Mathematical theorem.* This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** *Use in this chapter.* It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `wp`, `call`, `conseq`, `rewrite`. Named identifiers syntactically cited in the proof block:

```
ROMInternalTranscriptBudgetedPaperSimAsNMA
adversary_budgeted_paper_sim_signing_count_preserved
budgeted_paper_sim_signing_count_discipline
signature_query_budget_count
signing_count
```

**Theorem 3.71** (EasyCrypt line 1836). *EasyCrypt name.*

```
sampler_expand_query_point_bound
```

**Checked EasyCrypt statement.**

```
lemma sampler_expand_query_point_bound q :
  mu drandom_coins (fun coins => sampler_expand_query coins = q) <=
  signing_randomness_point_bound.
```

**Formal mathematical reading.** *Mathematical theorem.* This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `case`, `smt`, `apply`. Named identifiers syntactically cited in the proof block:

```

coins
drandom_coins
eq_q
ler_trans
mu
mu0
mu_le
sampler_expand_query
signing_coin_distribution_token_point_bound
signing_entropy_token_cardinality
signing_entropy_token_of_coins
signing_randomness_point_bound
target

```

**Theorem 3.72** (EasyCrypt line 1859). *EasyCrypt name.*

```
sampler_expand_query_log_fset_bound
```

**Checked EasyCrypt statement.**

```

lemma sampler_expand_query_log_fset_bound (qs : ro_query list) :
  mu drandom_coins
    (fun coins => sampler_expand_query coins \in qs) <=
    (card (oflist qs))%r * signing_randomness_point_bound.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*, *apply*. Named identifiers syntactically cited in the proof block:

```

coins
drandom_coins
mem_oflist
mu_eq
mu_mem_le_gen
oflist
qs
ro_query
sampler_expand_query
sampler_expand_query_point_bound
signing_randomness_point_bound

```

**Theorem 3.73** (EasyCrypt line 1877). *EasyCrypt name.*

```
sampler_expand_query_log_budget_bound
```

**Checked EasyCrypt statement.**

```

lemma sampler_expand_query_log_budget_bound (qs : ro_query list) hc :
  0 <= hc =>
  hc <= hash_query_budget_count =>
  size qs <= hc =>
  mu drandom_coins
    (fun coins => sampler_expand_query coins \in qs) <=
    rom_hash_query_budget / challenge_support_cardinality_lower_bound.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `apply`, `rewrite`, `smt`. Named identifiers syntactically cited in the proof block:

```
card
challenge_support_cardinality_lower_bound
fcard_oflist
hash_query_budget_count
hash_query_count
hc_budget
hc_ge0
ler_trans
oflist
qs
rom_hash_query_budget
sampler_expand_query_log_fset_bound
signing_entropy_token_cardinality
signing_randomness_point_bound
size_bound
```

**Theorem 3.74** (EasyCrypt line 1896). *EasyCrypt name.*

```
sampler_expand_query_joint_log_budget_bound
```

**Checked EasyCrypt statement.**

```
lemma sampler_expand_query_joint_log_budget_bound
  (hash_qs sampler_qs : ro_query list) hc sc :
  0 <= hc =>
  hc <= hash_query_budget_count =>
  0 <= sc =>
  sc <= signature_query_budget_count =>
  size hash_qs <= hc =>
  size sampler_qs <= sc =>
  mu random_coins
    (fun coins =>
      sampler_expand_query coins \in hash_qs \ /
      sampler_expand_query coins \in sampler_qs) <=
  (rom_hash_query_budget + rom_signature_query_budget) /
  challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `apply`, `smt`. Named identifiers syntactically cited in the proof block:

```
card
challenge_support_cardinality_lower_bound
coins
fcard_oflist
hash_qs
```



```

hash_query_budget_count
hash_query_count
hash_size
hc_budget
hc_ge0
ler_trans
mem_cat
mu_eq
oflist
rom_hash_query_budget
rom_signature_query_budget
sampler_expand_query
sampler_expand_query_log_fset_bound
sampler_qs
sampler_size
sc_budget
sc_ge0
signature_query_budget_count
signature_query_count
signing_entropy_token_cardinality
signing_randomness_point_bound
size_cat

```

**Theorem 3.75** (EasyCrypt line 1929). *EasyCrypt name.*

```
concrete_lazy_rom_sampler_expand_query_fresh_le
```

**Checked EasyCrypt statement.**

```

lemma concrete_lazy_rom_sampler_expand_query_fresh_le
  seed_coins (p : random_coins -> bool) &m :
  sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m{m} =>
  Pr[HAETAE_RO.FRO.get(sampler_expand_query seed_coins) @ &m :
    p (ro_signing_coins res)] <=
  mu drandom_coins p.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *byphoare*, *proc*, *wp*, *rnd*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

FRO
HAETAE_RO
arg
fresh
hr
lerr
notin
ro_signing_coins
sampler_expand_query
sampler_expand_query_dro_output_ro_signing_coins
seed_coins

```

**Theorem 3.76** (EasyCrypt line 1949). *EasyCrypt name.*

```
concrete_lazy_rom_sampler_expand_query_fresh_eq
```

### Checked EasyCrypt statement.

```

lemma concrete_lazy_rom_sampler_expand_query_fresh_eq
  seed_coins (p : random_coins -> bool) :
  phoare[HAETAE_RO.FRO.get :
    arg = sampler_expand_query seed_coins /\
    sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m
    ==> p (ro_signing_coins res)] =
  (mu drandom_coins p).

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

hr
sampler_expand_query_dro_output_ro_signing_coins

```

### Theorem 3.77 (EasyCrypt line 1964). *EasyCrypt* name.

```

concrete_lazy_rom_get_preserves_sampler_rom_covered

```

### Checked EasyCrypt statement.

```

lemma concrete_lazy_rom_get_preserves_sampler_rom_covered
  q hash_qs sampler_qs :
  hoare[HAETAE_RO.FRO.get :
    arg = q /\
    sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs /\
    (forall seed_coins,
      q = sampler_expand_query seed_coins =>
      q \in hash_qs /\ q \in sampler_qs) ==>
    sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

mem_set
sampler_rom_covered

```

### Theorem 3.78 (EasyCrypt line 1982). *EasyCrypt* name.

```

concrete_lazy_rom_get_preserves_sampler_rom_covered_arg

```

### Checked EasyCrypt statement.

```

lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_arg
  hash_qs sampler_qs :
  hoare[HAETAE_RO.FRO.get :
    sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs /\
    (forall seed_coins,
      arg = sampler_expand_query seed_coins =>
      arg \in hash_qs /\ arg \in sampler_qs) ==>
    sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

mem_set
sampler_rom_covered

```

**Theorem 3.79** (EasyCrypt line 1999). *EasyCrypt name.*

```

concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs

```

**Checked EasyCrypt statement.**

```

lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs :
  hoare[HAETAE_RO.FRO.get :
    sampler_rom_covered
    HAETAE_RO.FRO.m
    ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries /\
    (forall seed_coins,
      arg = sampler_expand_query seed_coins =>
      arg \in
        ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries /\
      arg \in
        ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries) ==>
    sampler_rom_covered
    HAETAE_RO.FRO.m
    ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

mem_set
sampler_rom_covered

```

**Theorem 3.80** (EasyCrypt line 2024). *EasyCrypt name.*

```
concrete_lazy_rom_get_preserves_sampler_rom_covered_hash_cons
```

**Checked EasyCrypt statement.**

```
lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_hash_cons
  q hash_qs sampler_qs :
  hoare[HAETAE_RO.FRO.get :
    arg = q /\
    sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs ==>
    sampler_rom_covered HAETAE_RO.FRO.m (q :: hash_qs) sampler_qs].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
mem_set
sampler_rom_covered
```

**Theorem 3.81** (EasyCrypt line 2039). *EasyCrypt name.*

```
concrete_lazy_rom_get_preserves_sampler_rom_covered_sampler_cons
```

**Checked EasyCrypt statement.**

```
lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_sampler_cons
  q hash_qs sampler_qs :
  hoare[HAETAE_RO.FRO.get :
    arg = q /\
    sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs ==>
    sampler_rom_covered HAETAE_RO.FRO.m hash_qs (q :: sampler_qs)].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
mem_set
sampler_rom_covered
```

**Theorem 3.82** (EasyCrypt line 2054). *EasyCrypt name.*

```
sampler_bad_prequery_one_step_bound_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma sampler_bad_prequery_one_step_bound_nonnegative :
  0%r <= rom_hash_query_budget / challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
challenge_support_cardinality_lower_bound_nonnegative
divr_ge0
rom_hash_query_budget_nonnegative
```

**Theorem 3.83** (EasyCrypt line 2062). *EasyCrypt name*.

```
budgeted_paper_sim_sampler_bad_prequery_forge_fel_bound
```

**Checked EasyCrypt statement.**

```
lemma budgeted_paper_sim_sampler_bad_prequery_forge_fel_bound pk &m :
  Pr[ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).forge(pk) @ &m :
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=
    rom_signature_query_budget *
    ((rom_hash_query_budget + rom_signature_query_budget) /
     challenge_support_cardinality_lower_bound).
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*, *inline*, *wp*, *proc*, *conseq*, *hoare*, *call*, *rnd*, *apply*. Named identifiers syntactically cited in the proof block:

```
AH
BRA
Bigreal
RField
ROMInternalTranscriptBudgetedPaperSimAsNMA
Samp
StdBigop
adversary_hash_count
adversary_hash_queries
budgeted_paper_sim_sampler_freshness_discipline
challenge_support_cardinality_lower_bound
fel
forge
get
hash_query_budget_count
hr
intmulr
old_bad
old_count
rom_hash_query_budget
```

```

rom_signature_query_budget
sampler_bad_prequery
sampler_expand_queries
sampler_expand_query
sampler_expand_query_joint_log_budget_bound
seed_coins
sign
signature_query_budget_count
signature_query_count
signing_coin_distribution_lossless
signing_count
sumri_const
trivial

```

**Theorem 3.84** (EasyCrypt line 2202). *EasyCrypt name.*

```

budgeted_paper_sim_sampler_bad_prequery_forge_fel_phoare

```

*Checked EasyCrypt statement.*

```

lemma budgeted_paper_sim_sampler_bad_prequery_forge_fel_phoare :
  phoare[ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).forge :
    true ==> ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=
    (rom_signature_query_budget *
      ((rom_hash_query_budget + rom_signature_query_budget) /
        challenge_support_cardinality_lower_bound)).

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Named identifiers syntactically cited in the proof block:

```

arg
budgeted_paper_sim_sampler_bad_prequery_forge_fel_bound
bypr
exact

```

**Theorem 3.85** (EasyCrypt line 2214). *EasyCrypt name.*

```

budgeted_paper_sim_sampler_bad_prequery_nma_fel_bound

```

*Checked EasyCrypt statement.*

```

lemma budgeted_paper_sim_sampler_bad_prequery_nma_fel_bound &m :
  Pr[SIG.UF_NMA(H, HAETA,
    ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp)).main() @ &m :
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=
    rom_signature_query_budget *
      ((rom_hash_query_budget + rom_signature_query_budget) /
        challenge_support_cardinality_lower_bound).

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byphoare*, *proc*, *inline*, *wp*, *call*, *rnd*. Named identifiers syntactically cited in the proof block:

```
HAETAE
ROMInternalTranscriptBudgetedPaperSimAsNMA
budgeted_paper_sim_sampler_bad_prequery_forge_fel_phoare
kg
sampler_bad_prequery
verify
```

**Theorem 3.86** (EasyCrypt line 2239). *EasyCrypt name.*

```
concrete_lazy_rom_sampler_expand_query_fresh_phoare
```

**Checked EasyCrypt statement.**

```
lemma concrete_lazy_rom_sampler_expand_query_fresh_phoare
  seed_coins (p : random_coins -> bool) :
  phoare[HAETAE_RO.FRO.get :
    arg = sampler_expand_query seed_coins /\
    sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m
  ==> p (ro_signing_coins res)] <=
  (mu drandom_coins p).
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
arg_eq
bypr
concrete_lazy_rom_sampler_expand_query_fresh_le
fresh
seed_coins
```

**Theorem 3.87** (EasyCrypt line 2252). *EasyCrypt name.*

```
concrete_lazy_rom_sampler_expand_query_fresh_arg_phoare
```

**Checked EasyCrypt statement.**

```
lemma concrete_lazy_rom_sampler_expand_query_fresh_arg_phoare
  (p : random_coins -> bool) :
  phoare[HAETAE_RO.FRO.get :
    (exists seed_coins,
      arg = sampler_expand_query seed_coins /\
      sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m)
  ==> p (ro_signing_coins res)] <=
  (mu drandom_coins p).
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated

postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
arg_eq
bypr
concrete_lazy_rom_sampler_expand_query_fresh_le
fresh
seed_coins
```

**Theorem 3.88** (EasyCrypt line 2266). *EasyCrypt* name.

```
concrete_lazy_rom_sampler_expand_query_fresh_arg_clean_phoare
```

**Checked EasyCrypt statement.**

```
lemma concrete_lazy_rom_sampler_expand_query_fresh_arg_clean_phoare
  (p : random_coins -> bool) :
  phoare[HAETAE_RO.FRO.get :
    (exists seed_coins,
      arg = sampler_expand_query seed_coins /\
      sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m) /\
    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery
  ==> p (ro_signing_coins res) /\
    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=
  (mu drandom_coins p).
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
arg_eq
fresh
hr
lerr
sampler_expand_query_dro_output_ro_signing_coins
seed_coins
```

**Theorem 3.89** (EasyCrypt line 2285). *EasyCrypt* name.

```
concrete_ro_signing_attempt_sample_with_seed_fresh_structural_le
```

**Checked EasyCrypt statement.**

```
lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_le
  seed_coins sk m ctx (p : paper_sim_signature_sample -> bool) :
  phoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :
    arg = (seed_coins, m, ctx) /\
    ROSigningAttemptPaperSimSampler.sk_current = sk /\
```



```
sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m
==> p res] <=
(mu (dsigning_attempt_state haetae_mode
    sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st))).
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *proc*, *wp*, *call*. Named identifiers syntactically cited in the proof block:

```
coins
concrete_lazy_rom_sampler_expand_query_fresh_phoare
ctx
dmapE
dsigning_attempt_state
haetae_mode
paper_sim_sample_from_rejection_attempt
seed_coins
signing_attempt_state_of_coins
sk
```

**Theorem 3.90** (EasyCrypt line 2307). *EasyCrypt name.*

```
concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_le
```

**Checked EasyCrypt statement.**

```
lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_le
  sk m ctx (p : paper_sim_signature_sample -> bool) :
  phoare [ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :
    arg.'2 = m /\
    arg.'3 = ctx /\
    ROSigningAttemptPaperSimSampler.sk_current = sk /\
    sampler_expand_query arg.'1 \notin HAETAE_RO.FRO.m
    ==> p res] <=
  (mu (dsigning_attempt_state haetae_mode
    sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st))).
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *proc*, *wp*, *call*, *smt*. Named identifiers syntactically cited in the proof block:

```
coins
concrete_lazy_rom_sampler_expand_query_fresh_arg_phoare
ctx
dmapE
```

```

dsigning_attempt_state
haetae_mode
paper_sim_sample_from_rejection_attempt
signing_attempt_state_of_coins
sk

```

**Theorem 3.91** (EasyCrypt line 2330). *EasyCrypt name.*

```
concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_clean_le
```

*Checked EasyCrypt statement.*

```

lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_clean_le
  sk m ctx (p : paper_sim_signature_sample -> bool) :
  phoare [ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :
    arg.'2 = m /\
    arg.'3 = ctx /\
    ROSigningAttemptPaperSimSampler.sk_current = sk /\
    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery /\
    sampler_expand_query arg.'1 \notin HAETAE_RO.FRO.m
    ==> p res /\
    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=
  (mu (dsigning_attempt_state haetae_mode
    sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st))).

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *proc*, *wp*, *call*, *smt*. Named identifiers syntactically cited in the proof block:

```

coins
concrete_lazy_rom_sampler_expand_query_fresh_arg_clean_phoare
ctx
dmapE
dsigning_attempt_state
haetae_mode
paper_sim_sample_from_rejection_attempt
signing_attempt_state_of_coins
sk

```

**Theorem 3.92** (EasyCrypt line 2355). *EasyCrypt name.*

```
concrete_lazy_rom_sampler_expand_query_guarded_clean_phoare
```

*Checked EasyCrypt statement.*

```

lemma concrete_lazy_rom_sampler_expand_query_guarded_clean_phoare
  (p : random_coins -> bool) :
  phoare [HAETAE_RO.FRO.get :
    (! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery =>
      exists seed_coins,
        arg = sampler_expand_query seed_coins /\
        sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m)
    ==> p (ro_signing_coins res) /\
    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=

```

```
(mu drandom_coins p).
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *case*, *rewrite*, *smt*, *elim*. Named identifiers syntactically cited in the proof block:

```
FRO
HAETAE_RO
ROMInternalTranscriptBudgetedPaperSimAsNMA
arg_eq
bad
clean
fresh
guarded
hr
lerr
mu0
mu_bounded
mu_eq
ro_output
sampler_bad_prequery
sampler_expand_query_dro_output_ro_signing_coins
seed_coins
```

**Theorem 3.93** (EasyCrypt line 2382). *EasyCrypt name.*

```
concrete_ro_signing_attempt_sample_with_seed_guarded_clean_structural_arg_le
```

**Checked EasyCrypt statement.**

```
lemma concrete_ro_signing_attempt_sample_with_seed_guarded_clean_structural_arg_le
  sk m ctx (p : paper_sim_signature_sample -> bool) :
  phoare [ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :
    arg.'2 = m /\
    arg.'3 = ctx /\
    ROSigningAttemptPaperSimSampler.sk_current = sk /\
    (! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery =>
      sampler_expand_query arg.'1 \notin HAETAE_RO.FRO.m)
    ==> p res /\
    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=
  (mu (dsigning_attempt_state haetae_mode
    sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st))).
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *proc*, *wp*, *call*, *smt*. Named identifiers syntactically cited in the proof block:

```

coins
concrete_lazy_rom_sampler_expand_query_guarded_clean_phoare
ctx
dmapE
dsigning_attempt_state
haetae_mode
paper_sim_sample_from_rejection_attempt
signing_attempt_state_of_coins
sk

```

**Theorem 3.94** (EasyCrypt line 2407). *EasyCrypt name.*

```
concrete_ro_signing_attempt_sample_with_seed_fresh_structural_eq
```

**Checked EasyCrypt statement.**

```

lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_eq
  seed_coins sk m ctx (p : paper_sim_signature_sample -> bool) :
  phoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :
    arg = (seed_coins, m, ctx) /\
    ROSigningAttemptPaperSimSampler.sk_current = sk /\
    sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m
    ==> p res] =
  (mu (dsigning_attempt_state haetae_mode
    sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st))).

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *proc*, *wp*, *call*. Named identifiers syntactically cited in the proof block:

```

coins
concrete_lazy_rom_sampler_expand_query_fresh_eq
ctx
dmapE
dsigning_attempt_state
haetae_mode
paper_sim_sample_from_rejection_attempt
seed_coins
signing_attempt_state_of_coins
sk

```

**Theorem 3.95** (EasyCrypt line 2429). *EasyCrypt name.*

```
concrete_ro_exact_hyperball_sample_with_seed_exact
```

**Checked EasyCrypt statement.**

```

lemma concrete_ro_exact_hyperball_sample_with_seed_exact
  seed_coins sk m ctx (p : paper_sim_signature_sample -> bool) :
  phoare[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :
    arg = (seed_coins, m, ctx) /\
    ROExactHyperballPaperSimSampler.sk_current = sk
    ==> p res] =
  (mu (dexact_hyperball_signing_attempt_state haetae_mode

```

```

      sk m ctx
    (fun st => p (paper_sim_sample_from_rejection_attempt st))).

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *call*, *smt*. Named identifiers syntactically cited in the proof block:

```

ROExactHyperballPaperSimSampler
ro_output_distribution_lossless
sk
sk_current

```

**Theorem 3.96** (EasyCrypt line 2448). *EasyCrypt* name.

```

concrete_ro_exact_hyperball_sample_with_seed_exact_no_seed

```

**Checked EasyCrypt statement.**

```

lemma concrete_ro_exact_hyperball_sample_with_seed_exact_no_seed
  sk m ctx (p : paper_sim_signature_sample -> bool) :
  phoare [ROExactHyperballPaperSimSampler(HAETAЕ.RO.FRO).sample_with_seed :
    arg. '2 = m /\
    arg. '3 = ctx /\
    ROExactHyperballPaperSimSampler.sk_current = sk
    ==> p res] =
  (mu (dexact_hyperball_signing_attempt_state haetae_mode
    sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st))).

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *call*, *smt*. Named identifiers syntactically cited in the proof block:

```

ROExactHyperballPaperSimSampler
ro_output_distribution_lossless
sk
sk_current

```

**Theorem 3.97** (EasyCrypt line 2468). *EasyCrypt* name.

```

concrete_ro_exact_hyperball_sample_with_seed_exact_pr

```

**Checked EasyCrypt statement.**

```

lemma concrete_ro_exact_hyperball_sample_with_seed_exact_pr

```

```

seed_coins m ctx (p : paper_sim_signature_sample -> bool) &m :
Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
(seed_coins, m, ctx) @ &m : p res] =
mu (deact_hyperball_signing_attempt_state haetae_mode
ROExactHyperballPaperSimSampler.sk_current{m} m ctx)
(fun st => p (paper_sim_sample_from_rejection_attempt st)).

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *bypHoare*. Named identifiers syntactically cited in the proof block:

```

ROExactHyperballPaperSimSampler
concrete_ro_exact_hyperball_sample_with_seed_exact
ctx
seed_coins
sk_current

```

**Theorem 3.98** (EasyCrypt line 2480). *EasyCrypt name.*

```
concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered
```

**Checked EasyCrypt statement.**

```

lemma concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered
  hash_qs sampler_qs :
  hoare [ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :
    sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs /\
    sampler_expand_query arg. '1 \in sampler_qs ==>
    sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *smt*. Named identifiers syntactically cited in the proof block:

```

concrete_lazy_rom_get_preserves_sampler_rom_covered_arg
hash_qs
sampler_qs

```

**Theorem 3.99** (EasyCrypt line 2494). *EasyCrypt name.*

```
concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered
```

**Checked EasyCrypt statement.**

```
lemma concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered
```

```

hash_qs sampler_qs :
hoare[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :
  sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs /\
  sampler_expand_query arg. '1 \in sampler_qs ==>
  sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *call*, *smt*. Named identifiers syntactically cited in the proof block:

```

concrete_lazy_rom_get_preserves_sampler_rom_covered_arg
hash_qs
sampler_qs

```

**Theorem 3.100** (EasyCrypt line 2510). *EasyCrypt* name.

```

concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs

```

**Checked EasyCrypt statement.**

```

lemma concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs :
  hoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :
    sampler_rom_covered
      HAETAE_RO.FRO.m
      ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries
      ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries /\
    sampler_expand_query arg. '1 \in
      ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries ==>
    sampler_rom_covered
      HAETAE_RO.FRO.m
      ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries
      ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *smt*. Named identifiers syntactically cited in the proof block:

```

concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs

```

**Theorem 3.101** (EasyCrypt line 2529). *EasyCrypt* name.

```

concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs

```

**Checked EasyCrypt statement.**

```

lemma concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs :
  hoare [ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :
    sampler_rom_covered
    HAETAE_RO.FRO.m
    ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries /\
    sampler_expand_query arg. '1 \in
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries ==>
    sampler_rom_covered
    HAETAE_RO.FRO.m
    ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *call*, *smt*. Named identifiers syntactically cited in the proof block:

```
concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs
```

**Theorem 3.102** (EasyCrypt line 2550). *EasyCrypt* name.

```
concrete_budgeted_ro_signing_attempt_o_sign_preserves_sampler_rom_covered
```

**Checked EasyCrypt statement.**

```

lemma concrete_budgeted_ro_signing_attempt_o_sign_preserves_sampler_rom_covered :
  hoare [ROMInternalTranscriptBudgetedPaperSimAsNMA
    (A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO),
    HAETAE_RO.FRO).O.sign :
    sampler_rom_covered
    HAETAE_RO.FRO.m
    ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries ==>
    sampler_rom_covered
    HAETAE_RO.FRO.m
    ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *rnd*, *smt*. Named identifiers syntactically cited in the proof block:

```

concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs
concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs

```

**Theorem 3.103** (EasyCrypt line 2577). *EasyCrypt* name.



```
concrete_budgeted_ro_exact_hyperball_o_sign_preserves_sampler_rom_covered
```

**Checked EasyCrypt statement.**

```
lemma concrete_budgeted_ro_exact_hyperball_o_sign_preserves_sampler_rom_covered :
  hoare [ROMInternalTranscriptBudgetedPaperSimAsNMA
    (A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO),
      HAETAE_RO.FRO).O.sign :
    sampler_rom_covered
    HAETAE_RO.FRO.m
    ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries ==>
    sampler_rom_covered
    HAETAE_RO.FRO.m
    ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *rnd*, *smt*. Named identifiers syntactically cited in the proof block:

```
concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs
concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs
```

**Theorem 3.104 (EasyCrypt line 2604). EasyCrypt name.**

```
concrete_budgeted_ro_signing_attempt_o_sign_clean_sampler_fresh
```

**Checked EasyCrypt statement.**

```
lemma concrete_budgeted_ro_signing_attempt_o_sign_clean_sampler_fresh :
  hoare [ROMInternalTranscriptBudgetedPaperSimAsNMA
    (A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO),
      HAETAE_RO.FRO).O.sign :
    sampler_rom_covered
    HAETAE_RO.FRO.m
    ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries /\
    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery ==>
    sampler_rom_covered
    HAETAE_RO.FRO.m
    ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *conseq*. Named identifiers syntactically cited in the proof

block:

```
concrete_budgeted_ro_signing_attempt_o_sign_preserves_sampler_rom_covered
```

**Theorem 3.105** (EasyCrypt line 2986). *EasyCrypt name.*

```
internal_transcript_sign_oracle_preserves_state
```

**Checked EasyCrypt statement.**

```
lemma internal_transcript_sign_oracle_preserves_state :  
  hoare[EUFCMA_InternalTranscriptSign(H, A).O.sign :  
    internal_transcript_state_sound haetae_mode  
    EUFCMA_InternalTranscriptSign.pk_current  
    EUFCMA_InternalTranscriptSign.sk_current  
    EUFCMA_InternalTranscriptSign.queries  
    EUFCMA_InternalTranscriptSign.transcripts  
    EUFCMA_InternalTranscriptSign.records ==>  
    internal_transcript_state_sound haetae_mode  
    EUFCMA_InternalTranscriptSign.pk_current  
    EUFCMA_InternalTranscriptSign.sk_current  
    EUFCMA_InternalTranscriptSign.queries  
    EUFCMA_InternalTranscriptSign.transcripts  
    EUFCMA_InternalTranscriptSign.records].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *apply*. Named identifiers syntactically cited in the proof block:

```
EUFCMA_InternalTranscriptSign  
coins  
ctx  
haetae_mode  
hr  
internal_transcript_state_sound_sign_internal_cons  
pk_current  
queries  
records  
sk_current  
state_sound  
transcripts
```

**Theorem 3.106** (EasyCrypt line 3016). *EasyCrypt name.*

```
adversary_internal_transcript_state_preserved
```

**Checked EasyCrypt statement.**

```
lemma adversary_internal_transcript_state_preserved :  
  hoare[A(H, EUFCMA_InternalTranscriptSign(H, A).O).forge :  
    internal_transcript_state_sound haetae_mode  
    EUFCMA_InternalTranscriptSign.pk_current  
    EUFCMA_InternalTranscriptSign.sk_current  
    EUFCMA_InternalTranscriptSign.queries  
    EUFCMA_InternalTranscriptSign.transcripts
```

```

    EUF_CMA_InternalTranscriptSign.records ==>
internal_transcript_state_sound haetae_mode
    EUF_CMA_InternalTranscriptSign.pk_current
    EUF_CMA_InternalTranscriptSign.sk_current
    EUF_CMA_InternalTranscriptSign.queries
    EUF_CMA_InternalTranscriptSign.transcripts
    EUF_CMA_InternalTranscriptSign.records].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *call*, *apply*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_InternalTranscriptSign
haetae_mode
internal_transcript_sign_oracle_preserves_state
internal_transcript_state_sound
pk_current
queries
records
sk_current
transcripts

```

**Theorem 3.107** (EasyCrypt line 3044). *EasyCrypt name.*

```

internal_transcript_sign_main_state_sound

```

**Checked EasyCrypt statement.**

```

lemma internal_transcript_sign_main_state_sound :
  hoare[EUF_CMA_InternalTranscriptSign(H, A).main :
    true ==>
    internal_transcript_state_sound haetae_mode
      EUF_CMA_InternalTranscriptSign.pk_current
      EUF_CMA_InternalTranscriptSign.sk_current
      EUF_CMA_InternalTranscriptSign.queries
      EUF_CMA_InternalTranscriptSign.transcripts
      EUF_CMA_InternalTranscriptSign.records].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *apply*, *rnd*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_InternalTranscriptSign
adversary_internal_transcript_state_preserved
haetae_mode
internal_transcript_state_sound
pk_current

```

```
queries
records
sk_current
transcripts
```

**Theorem 3.108** (EasyCrypt line 3076). *EasyCrypt name.*

```
internal_transcript_sign_main_transcripts_valid
```

**Checked EasyCrypt statement.**

```
lemma internal_transcript_sign_main_transcripts_valid :
  hoare[EUF_CMA_InternalTranscriptSign(H, A).main :
    true ==>
    transcript_log_valid haetae_mode
    EUF_CMA_InternalTranscriptSign.transcripts].
```

**Formal mathematical reading.** *Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.*

**Role in the proof.** *Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `wp`, `call`, `conseq`, `apply`, `rnd`. Named identifiers syntactically cited in the proof block:*

```
EUF_CMA_InternalTranscriptSign
adversary_internal_transcript_state_preserved
haetae_mode
hr
internal_transcript_state_sound
internal_transcript_state_sound_valid
pk_current
qs0
queries
records
rs0
sk_current
state_sound
transcript_log_valid
transcripts
trs0
```

**Theorem 3.109** (EasyCrypt line 3118). *EasyCrypt name.*

```
internal_transcript_sign_main_log_ready
```

**Checked EasyCrypt statement.**

```
lemma internal_transcript_sign_main_log_ready :
  hoare[EUF_CMA_InternalTranscriptSign(H, A).main :
    true ==>
    transcript_log_ready haetae_mode
    EUF_CMA_InternalTranscriptSign.transcripts].
```

**Formal mathematical reading.** *Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.*

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *conseq*, *apply*, *rnd*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_InternalTranscriptSign
adversary_internal_transcript_state_preserved
haetae_mode
hr
internal_transcript_state_sound
internal_transcript_state_sound_log_ready
pk_current
qs0
queries
records
rs0
sk_current
state_sound
transcript_log_ready
transcripts
trs0

```

**Theorem 3.110** (EasyCrypt line 3160). *EasyCrypt name.*

```

internal_transcript_sign_main_no_min_entropy

```

**Checked EasyCrypt statement.**

```

lemma internal_transcript_sign_main_no_min_entropy :
  hoare[EUF_CMA_InternalTranscriptSign(H, A).main :
    true ==>
    transcript_log_min_entropy_clear haetae_mode
    EUF_CMA_InternalTranscriptSign.transcripts].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *conseq*, *apply*, *rnd*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_InternalTranscriptSign
adversary_internal_transcript_state_preserved
haetae_mode
hr
internal_transcript_state_sound
internal_transcript_state_sound_no_min_entropy
pk_current
qs0
queries
records
rs0
sk_current
state_sound
transcript_log_min_entropy_clear
transcripts

```

trs0

**Theorem 3.111** (EasyCrypt line 3202). *EasyCrypt name.*

*rom\_internal\_transcript\_sign\_oracle\_preserves\_state*

**Checked EasyCrypt statement.**

```
lemma rom_internal_transcript_sign_oracle_preserves_state :
  hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).O.sign :
    internal_transcript_state_sound haetae_mode
    EUF_CMA_ROMInternalTranscriptSign.pk_current
    EUF_CMA_ROMInternalTranscriptSign.sk_current
    EUF_CMA_ROMInternalTranscriptSign.queries
    EUF_CMA_ROMInternalTranscriptSign.transcripts
    EUF_CMA_ROMInternalTranscriptSign.records ==>
    internal_transcript_state_sound haetae_mode
    EUF_CMA_ROMInternalTranscriptSign.pk_current
    EUF_CMA_ROMInternalTranscriptSign.sk_current
    EUF_CMA_ROMInternalTranscriptSign.queries
    EUF_CMA_ROMInternalTranscriptSign.transcripts
    EUF_CMA_ROMInternalTranscriptSign.records].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *rnd*, *apply*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMInternalTranscriptSign
ctx
haetae_mode
hr
internal_transcript_state_sound_sign_internal_cons
pk_current
queries
records
result
ro_signing_coins
seed_coins
sk_current
state_sound
transcripts
```

**Theorem 3.112** (EasyCrypt line 3238). *EasyCrypt name.*

*adversary\_rom\_internal\_transcript\_state\_preserved*

**Checked EasyCrypt statement.**

```
lemma adversary_rom_internal_transcript_state_preserved :
  hoare[A(H, EUF_CMA_ROMInternalTranscriptSign(H, A).O).forge :
    internal_transcript_state_sound haetae_mode
    EUF_CMA_ROMInternalTranscriptSign.pk_current
    EUF_CMA_ROMInternalTranscriptSign.sk_current
    EUF_CMA_ROMInternalTranscriptSign.queries
    EUF_CMA_ROMInternalTranscriptSign.transcripts
```

```

    EUF_CMA_ROMInternalTranscriptSign.records ==>
internal_transcript_state_sound haetae_mode
    EUF_CMA_ROMInternalTranscriptSign.pk_current
    EUF_CMA_ROMInternalTranscriptSign.sk_current
    EUF_CMA_ROMInternalTranscriptSign.queries
    EUF_CMA_ROMInternalTranscriptSign.transcripts
    EUF_CMA_ROMInternalTranscriptSign.records].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *call*, *apply*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMInternalTranscriptSign
haetae_mode
internal_transcript_state_sound
pk_current
queries
records
rom_internal_transcript_sign_oracle_preserves_state
sk_current
transcripts

```

**Theorem 3.113** (EasyCrypt line 3266). *EasyCrypt* name.

```

rom_internal_transcript_sign_main_state_sound

```

**Checked EasyCrypt statement.**

```

lemma rom_internal_transcript_sign_main_state_sound :
  hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :
    true ==>
    internal_transcript_state_sound haetae_mode
      EUF_CMA_ROMInternalTranscriptSign.pk_current
      EUF_CMA_ROMInternalTranscriptSign.sk_current
      EUF_CMA_ROMInternalTranscriptSign.queries
      EUF_CMA_ROMInternalTranscriptSign.transcripts
      EUF_CMA_ROMInternalTranscriptSign.records].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *apply*, *rnd*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMInternalTranscriptSign
adversary_rom_internal_transcript_state_preserved
haetae_mode
internal_transcript_state_sound
pk_current

```

```
queries
records
sk_current
transcripts
```

**Theorem 3.114** (EasyCrypt line 3300). *EasyCrypt name.*

```
rom_internal_transcript_sign_main_transcripts_valid
```

**Checked EasyCrypt statement.**

```
lemma rom_internal_transcript_sign_main_transcripts_valid :
  hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :
    true ==>
    transcript_log_valid haetae_mode
    EUF_CMA_ROMInternalTranscriptSign.transcripts].
```

**Formal mathematical reading.** *Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.*

**Role in the proof.** *Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `wp`, `call`, `conseq`, `apply`, `rnd`. Named identifiers syntactically cited in the proof block:*

```
EUF_CMA_ROMInternalTranscriptSign
adversary_rom_internal_transcript_state_preserved
haetae_mode
hr
internal_transcript_state_sound
internal_transcript_state_sound_valid
pk_current
qs0
queries
records
rs0
sk_current
state_sound
transcript_log_valid
transcripts
trs0
```

**Theorem 3.115** (EasyCrypt line 3344). *EasyCrypt name.*

```
rom_internal_transcript_sign_main_log_ready
```

**Checked EasyCrypt statement.**

```
lemma rom_internal_transcript_sign_main_log_ready :
  hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :
    true ==>
    transcript_log_ready haetae_mode
    EUF_CMA_ROMInternalTranscriptSign.transcripts].
```

**Formal mathematical reading.** *Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.*



**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *conseq*, *apply*, *rnd*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMInternalTranscriptSign
adversary_rom_internal_transcript_state_preserved
haetae_mode
hr
internal_transcript_state_sound
internal_transcript_state_sound_log_ready
pk_current
qs0
queries
records
rs0
sk_current
state_sound
transcript_log_ready
transcripts
trs0
```

**Theorem 3.116** (EasyCrypt line 3388). *EasyCrypt name.*

```
rom_internal_transcript_sign_main_no_min_entropy
```

**Checked EasyCrypt statement.**

```
lemma rom_internal_transcript_sign_main_no_min_entropy :
  hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :
    true ==>
    transcript_log_min_entropy_clear haetae_mode
    EUF_CMA_ROMInternalTranscriptSign.transcripts].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *conseq*, *apply*, *rnd*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMInternalTranscriptSign
adversary_rom_internal_transcript_state_preserved
haetae_mode
hr
internal_transcript_state_sound
internal_transcript_state_sound_no_min_entropy
pk_current
qs0
queries
records
rs0
sk_current
state_sound
transcript_log_min_entropy_clear
transcripts
```

trs0

**Theorem 3.117** (EasyCrypt line 3432). *EasyCrypt name.*

rom\_internal\_transcript\_sign\_main\_no\_failure\_for\_signature\_site

**Checked EasyCrypt statement.**

```
lemma rom_internal_transcript_sign_main_no_failure_for_signature_site :
  hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :
    true ==>
    forall tr y,
      tr \in EUF_CMA_ROMInternalTranscriptSign.transcripts =>
      transcript_signature_challenge_matches tr y =>
      ! fs_with_aborts_failure haetae_mode tr
        (signature_programming_site_of_transcript haetae_mode tr y)].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *conseq*, *have*, *apply*, *rnd*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMInternalTranscriptSign
adversary_rom_internal_transcript_state_preserved
fs_with_aborts_failure
haetae_mode
hr
internal_transcript_state_sound
internal_transcript_state_sound_log_ready
match_y
mem_tr
pk_current
qs0
queries
ready
records
rs0
signature_programming_site_of_transcript
sk_current
state_sound
tr
transcript_log_ready_no_failure_for_signature_programming_site
transcript_signature_challenge_matches
transcripts
trs0
```

**Theorem 3.118** (EasyCrypt line 3485). *EasyCrypt name.*

rom\_internal\_transcript\_sign\_main\_signature\_sites\_clear

**Checked EasyCrypt statement.**

```
lemma rom_internal_transcript_sign_main_signature_sites_clear :
  hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :
    true ==>
    transcript_log_signature_programming_sites_clear haetae_mode
```

```
EUF_CMA_ROMInternalTranscriptSign.transcripts].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *conseq*, *have*, *apply*, *rnd*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMInternalTranscriptSign
adversary_rom_internal_transcript_state_preserved
haetae_mode
hr
internal_transcript_state_sound
internal_transcript_state_sound_log_ready
pk_current
qs0
queries
ready
records
rs0
sk_current
state_sound
transcript_log_ready_signature_sites_clear
transcript_log_signature_programming_sites_clear
transcripts
trs0
```

**Theorem 3.119** (EasyCrypt line 3531). *EasyCrypt name.*

```
rom_internal_transcript_bad_forgery_zero
```

**Checked EasyCrypt statement.**

```
lemma rom_internal_transcript_bad_forgery_zero &m :
  Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :
    res /\ ! transcript_log_signature_programming_sites_clear haetae_mode
      EUF_CMA_ROMInternalTranscriptSign.transcripts] = 0%r.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byphoare*, *hoare*, *conseq*, *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMInternalTranscriptSign
clear_sites
forge
haetae_mode
hr
rom_internal_transcript_sign_main_signature_sites_clear
transcript_log_signature_programming_sites_clear
```

```
transcripts
trs0
```

**Theorem 3.120** (EasyCrypt line 3548). *EasyCrypt name.*

```
rom_internal_transcript_bad_forgery_bound
```

**Checked EasyCrypt statement.**

```
lemma rom_internal_transcript_bad_forgery_bound &m :
  Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :
    res /\ ! transcript_log_signature_programming_sites_clear haetae_mode
      EUF_CMA_ROMInternalTranscriptSign.transcripts] <=
    fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*, *apply*. Named identifiers syntactically cited in the proof block:

```
fs_with_aborts_min_entropy_term
fs_with_aborts_min_entropy_term_nonnegative
fs_with_aborts_reprogramming_term
fs_with_aborts_reprogramming_term_nonnegative
ge_entropy
ge_reprogram
ge_sum
ler_add
rom_internal_transcript_bad_forgery_zero
```

**Theorem 3.121** (EasyCrypt line 3564). *EasyCrypt name.*

```
rom_internal_transcript_bad_forgery_counted_bound
```

**Checked EasyCrypt statement.**

```
lemma rom_internal_transcript_bad_forgery_counted_bound &m :
  Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :
    res /\ ! transcript_log_signature_programming_sites_clear haetae_mode
      EUF_CMA_ROMInternalTranscriptSign.transcripts] <=
    counted_rom_programming_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
counted_rom_programming_loss_term_nonnegative
rom_internal_transcript_bad_forgery_zero
```

**Theorem 3.122** (EasyCrypt line 3583). *EasyCrypt name.*

```
counted_rom_internal_transcript_sign_oracle_preserves_budget_state
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_internal_transcript_sign_oracle_preserves_budget_state :
  hoare[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).O.sign :
    internal_transcript_state_sound haetae_mode
      EUF_CMA_ROMInternalTranscriptCountedSign.pk_current
      EUF_CMA_ROMInternalTranscriptCountedSign.sk_current
      EUF_CMA_ROMInternalTranscriptCountedSign.queries
      EUF_CMA_ROMInternalTranscriptCountedSign.transcripts
      EUF_CMA_ROMInternalTranscriptCountedSign.records /\
    ! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_prequery /\
    ! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_reprogram /\
    ! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_min_entropy /\
    EUF_CMA_ROMInternalTranscriptCountedSign.C.programmed_sites = [] ==>
    internal_transcript_state_sound haetae_mode
      EUF_CMA_ROMInternalTranscriptCountedSign.pk_current
      EUF_CMA_ROMInternalTranscriptCountedSign.sk_current
      EUF_CMA_ROMInternalTranscriptCountedSign.queries
      EUF_CMA_ROMInternalTranscriptCountedSign.transcripts
      EUF_CMA_ROMInternalTranscriptCountedSign.records /\
    ! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_prequery /\
    ! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_reprogram /\
    ! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_min_entropy /\
    EUF_CMA_ROMInternalTranscriptCountedSign.C.programmed_sites = []].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*, *rnd*, *have*, *case*, *apply*, *split*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMInternalTranscriptCountedSign
ctx
entropy_ok
get
haetae_mode
hr
internal_transcript_state_sound_sign_internal_cons
min_entropy_failure
no_min
observe_transcript
pkE
pk_current
preq_ok
programmed_ok
public_key_of_secret
queries
records
reprog_ok
result
ro_signing_coins
seed_coins
```

```

sign_internal
sign_internal_transcript_no_min_entropy
sk_current
state_sound
transcript_of_signature
transcripts

```

**Theorem 3.123** (EasyCrypt line 3653). *EasyCrypt name.*

```
adversary_counted_rom_internal_transcript_budget_state_preserved
```

**Checked EasyCrypt statement.**

```

lemma adversary_counted_rom_internal_transcript_budget_state_preserved :
  hoare[A (EUF_CMA_ROMInternalTranscriptCountedSign(H, A).C,
    EUF_CMA_ROMInternalTranscriptCountedSign(H, A).O).forge :
    internal_transcript_state_sound haetae_mode
      EUF_CMA_ROMInternalTranscriptCountedSign.pk_current
      EUF_CMA_ROMInternalTranscriptCountedSign.sk_current
      EUF_CMA_ROMInternalTranscriptCountedSign.queries
      EUF_CMA_ROMInternalTranscriptCountedSign.transcripts
      EUF_CMA_ROMInternalTranscriptCountedSign.records /\
    ! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_prequery /\
    ! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_reprogram /\
    ! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_min_entropy /\
    EUF_CMA_ROMInternalTranscriptCountedSign.C.programmed_sites = [] ==>
    internal_transcript_state_sound haetae_mode
      EUF_CMA_ROMInternalTranscriptCountedSign.pk_current
      EUF_CMA_ROMInternalTranscriptCountedSign.sk_current
      EUF_CMA_ROMInternalTranscriptCountedSign.queries
      EUF_CMA_ROMInternalTranscriptCountedSign.transcripts
      EUF_CMA_ROMInternalTranscriptCountedSign.records /\
    ! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_prequery /\
    ! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_reprogram /\
    ! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_min_entropy /\
    EUF_CMA_ROMInternalTranscriptCountedSign.C.programmed_sites = []].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *call*, *apply*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMInternalTranscriptCountedSign
bad_min_entropy
bad_prequery
bad_reprogram
counted_lazy_rom_get_preserves_clear
counted_rom_internal_transcript_sign_oracle_preserves_budget_state
haetae_mode
internal_transcript_state_sound
pk_current
programmed_sites
queries
records
sk_current
transcripts

```

**Theorem 3.124** (EasyCrypt line 3694). *EasyCrypt name.*

```
counted_rom_internal_transcript_main_bad_clear
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_internal_transcript_main_bad_clear :
  hoare[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).main :
    true ==> ! res].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *apply*, *rnd*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMInternalTranscriptCountedSign
adversary_counted_rom_internal_transcript_budget_state_preserved
bad_min_entropy
bad_prequery
bad_reprogram
counted_lazy_rom_bad_false_when_clear
counted_lazy_rom_get_preserves_clear
counted_lazy_rom_init_clears
haetae_mode
internal_transcript_state_sound
internal_transcript_state_sound_keygen
pk_current
programmed_sites
queries
records
sd
sk_current
transcripts
```

**Theorem 3.125** (EasyCrypt line 3733). *EasyCrypt name.*

```
counted_rom_internal_transcript_main_bad_false
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_internal_transcript_main_bad_false :
  hoare[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).main :
    true ==> res = false].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *conseq*. Named identifiers syntactically cited in the proof block:

```
counted_rom_internal_transcript_main_bad_clear
```

**Theorem 3.126** (EasyCrypt line 3740). *EasyCrypt name.*

```
counted_rom_internal_transcript_counted_bad_zero
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_internal_transcript_counted_bad_zero &m :  
  Pr[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).main() @ &m :  
    res] = 0%r.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byphoare*, *hoare*, *conseq*, *apply*. Named identifiers syntactically cited in the proof block:

```
counted_rom_internal_transcript_main_bad_false
```

**Theorem 3.127** (EasyCrypt line 3751). *EasyCrypt name.*

```
counted_rom_internal_transcript_budget_sound
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_internal_transcript_budget_sound &m :  
  counted_lazy_rom_bad_flags_budget_sound  
  (Pr[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).main() @ &m :  
    res]).
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \bowtie B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
counted_lazy_rom_bad_flags_budget_sound  
counted_rom_internal_transcript_counted_bad_zero  
counted_rom_programming_loss_term_nonnegative
```

**Theorem 3.128** (EasyCrypt line 3761). *EasyCrypt name.*

```
counted_rom_internal_transcript_counted_bad_subunit
```

**Checked EasyCrypt statement.**



```
lemma counted_rom_internal_transcript_counted_bad_subunit &m :
  Pr[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).main() @ &m :
    res] < 1%r.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\text{Pr}[G : E] \bowtie B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
counted_rom_internal_transcript_counted_bad_zero
trivial
```

**Theorem 3.129** (EasyCrypt line 3778). *EasyCrypt name*.

```
programmed_counted_sign_oracle_preserves_min_entropy_clear
```

**Checked EasyCrypt statement.**

```
lemma programmed_counted_sign_oracle_preserves_min_entropy_clear :
  hoare[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).O.sign :
    EUF_CMA_ROMProgrammedTranscriptCountedSign.pk_current =
      public_key_of_secret haetae_mode
      EUF_CMA_ROMProgrammedTranscriptCountedSign.sk_current /\
    ! EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy ==>
    EUF_CMA_ROMProgrammedTranscriptCountedSign.pk_current =
      public_key_of_secret haetae_mode
      EUF_CMA_ROMProgrammedTranscriptCountedSign.sk_current /\
    ! EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*, *rnd*, *have*, *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMProgrammedTranscriptCountedSign
ctx
entropy_ok
get
haetae_mode
hr
min_entropy_failure
no_min
observe_transcript
pkE
pk_current
program
result
ro_signing_coins
```

```
seed_coins
sign_internal
sign_internal_transcript_no_min_entropy
sk_current
transcript_of_signature
```

**Theorem 3.130** (EasyCrypt line 3818). *EasyCrypt name.*

```
adversary_programmed_counted_min_entropy_preserved
```

**Checked EasyCrypt statement.**

```
lemma adversary_programmed_counted_min_entropy_preserved :
  hoare[A (EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).C,
    EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).O).forge :
    EUF_CMA_ROMProgrammedTranscriptCountedSign.pk_current =
      public_key_of_secret haetae_mode
      EUF_CMA_ROMProgrammedTranscriptCountedSign.sk_current /\
    ! EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy ==>
    EUF_CMA_ROMProgrammedTranscriptCountedSign.pk_current =
      public_key_of_secret haetae_mode
      EUF_CMA_ROMProgrammedTranscriptCountedSign.sk_current /\
    ! EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *call*, *apply*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMProgrammedTranscriptCountedSign
bad_min_entropy
counted_lazy_rom_get_preserves_min_entropy_clear
haetae_mode
pk_current
programmed_counted_sign_oracle_preserves_min_entropy_clear
public_key_of_secret
sk_current
```

**Theorem 3.131** (EasyCrypt line 3841). *EasyCrypt name.*

```
programmed_counted_main_min_entropy_clear
```

**Checked EasyCrypt statement.**

```
lemma programmed_counted_main_min_entropy_clear :
  hoare[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).main :
    true ==>
    ! EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *conseq*, *rnd*, *apply*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMProgrammedTranscriptCountedSign
adversary_programmed_counted_min_entropy_preserved
bad_min_entropy
counted_lazy_rom_bad_preserves_min_entropy_clear
counted_lazy_rom_get_preserves_min_entropy_clear
counted_lazy_rom_init_clears
haetae_mode
pk_current
public_key_of_keygen
public_key_of_secret
sd
sk_current

```

**Theorem 3.132** (EasyCrypt line 3869). *EasyCrypt name.*

```

programmed_counted_min_entropy_bad_zero

```

**Checked EasyCrypt statement.**

```

lemma programmed_counted_min_entropy_bad_zero &m :
  Pr[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).main() @ &m :
    EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy] = 0%r.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byphoare*, *hoare*, *conseq*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMProgrammedTranscriptCountedSign
bad_min_entropy
programmed_counted_main_min_entropy_clear

```

**Theorem 3.133** (EasyCrypt line 3879). *EasyCrypt name.*

```

programmed_counted_min_entropy_bad_bound

```

**Checked EasyCrypt statement.**

```

lemma programmed_counted_min_entropy_bad_bound &m :
  Pr[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).main() @ &m :
    EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy] <=
    counted_rom_min_entropy_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
counted_rom_min_entropy_term_nonnegative
programmed_counted_min_entropy_bad_zero
```

**Theorem 3.134** (EasyCrypt line 3888). *EasyCrypt* name.

```
programmed_counted_prequery_reprogramming_bad_bound_from_budget_expr
```

**Checked EasyCrypt statement.**

```
lemma programmed_counted_prequery_reprogramming_bad_bound_from_budget_expr &m :
  Pr[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).main() @ &m :
    EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_prequery \ /
    EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_reprogram] <=
    ((rom_total_query_budget * rom_total_query_budget) +
     (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) /
    challenge_support_cardinality_lower_bound =>
  Pr[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).main() @ &m :
    EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_prequery \ /
    EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_reprogram] <=
    counted_rom_prequery_reprogramming_term.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
counted_rom_prequery_reprogramming_term_cardinalityE
```

**Theorem 3.135** (EasyCrypt line 3915). *EasyCrypt* name.

```
budgeted_counted_lazy_rom_get_true
```

**Checked EasyCrypt statement.**

```
lemma budgeted_counted_lazy_rom_get_true :
  hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.get :
    true ==> true].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
budgeted_programmed_challenge_query_discipline
```

**Theorem 3.136** (EasyCrypt line 3925). *EasyCrypt name.*

```
budgeted_counted_lazy_rom_init_true
```

**Checked EasyCrypt statement.**

```
lemma budgeted_counted_lazy_rom_init_true :
  hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.init :
    true ==> true].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `wp`, `call`.

**Theorem 3.137** (EasyCrypt line 3935). *EasyCrypt name.*

```
budgeted_counted_lazy_rom_bad_true
```

**Checked EasyCrypt statement.**

```
lemma budgeted_counted_lazy_rom_bad_true :
  hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.bad :
    true ==> true].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`.

**Theorem 3.138** (EasyCrypt line 3942). *EasyCrypt name.*

```
budgeted_programmed_hash_get_preserves_challenge_query_discipline
```

**Checked EasyCrypt statement.**

```
lemma budgeted_programmed_hash_get_preserves_challenge_query_discipline :
  hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH.get :
    budgeted_programmed_challenge_query_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries ==>
    budgeted_programmed_challenge_query_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign
budgeted_programmed_challenge_query_discipline
challenge_query_log_count_cons_le
get
hash_query_budget_count
```

**Theorem 3.139** (EasyCrypt line 3963). *EasyCrypt* name.

```
budgeted_programmed_sign_oracle_preserves_challenge_query_discipline
```

**Checked EasyCrypt statement.**

```
lemma budgeted_programmed_sign_oracle_preserves_challenge_query_discipline :
  hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O.sign :
    budgeted_programmed_challenge_query_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries ==>
    budgeted_programmed_challenge_query_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*, *rnd*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
DirectSigningCoinSampler
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign
budgeted_programmed_challenge_query_discipline
challenge_query_log_count_message_cons
get
observe_transcript
program
sample
```

**Theorem 3.140** (EasyCrypt line 3989). *EasyCrypt* name.

```
adversary_budgeted_programmed_challenge_query_discipline_preserved
```

**Checked EasyCrypt statement.**

```
lemma adversary_budgeted_programmed_challenge_query_discipline_preserved :
  hoare[
    A(EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH,
      EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O).forge :
    budgeted_programmed_challenge_query_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count
```

```

EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries ==>
budgeted_programmed_challenge_query_discipline
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *call*, *apply*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign
adversary_hash_count
budgeted_programmed_challenge_query_discipline
budgeted_programmed_hash_get_preserves_challenge_query_discipline
budgeted_programmed_sign_oracle_preserves_challenge_query_discipline
hash_queries

```

**Theorem 3.141** (EasyCrypt line 4011). *EasyCrypt name*.

```

budgeted_programmed_main_challenge_query_discipline

```

**Checked EasyCrypt statement.**

```

lemma budgeted_programmed_main_challenge_query_discipline :
  hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main :
    true ==>
    budgeted_programmed_challenge_query_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *conseq*, *rnd*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign
adversary_budgeted_programmed_challenge_query_discipline_preserved
adversary_hash_count
budgeted_counted_lazy_rom_bad_true
budgeted_programmed_challenge_query_discipline
challenge_query_log_count
counted_lazy_rom_get_nonchallenge_preserves_challenge_query_count
counted_lazy_rom_init_clears_hash_queries
hash_queries
hash_query_budget_count
matrix_expand_query
ro_query_is_challenge
sd

```

**Theorem 3.142** (EasyCrypt line 4042). *EasyCrypt name.*

```
budgeted_programmed_main_challenge_query_count_bound
```

*Checked EasyCrypt statement.*

```
lemma budgeted_programmed_main_challenge_query_count_bound :
  hoare[EUFCMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main :
    true ==>
    challenge_query_log_count
    EUFCMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries <=
    hash_query_budget_count + 1].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *conseq*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
budgeted_programmed_challenge_query_discipline
budgeted_programmed_main_challenge_query_discipline
```

**Theorem 3.143** (EasyCrypt line 4054). *EasyCrypt name.*

```
budgeted_programmed_adaptive_prequery_lift
```

*Checked EasyCrypt statement.*

```
lemma budgeted_programmed_adaptive_prequery_lift
  md sk m ctx qs hc :
  budgeted_programmed_challenge_query_discipline hc qs =>
  mu random_coins
  (fun coins =>
    programming_site_prequeried qs
    (honest_signing_programming_site md sk m ctx coins)) <=
  (rom_hash_query_budget + 1%r) /
  challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
budgeted_programmed_challenge_query_discipline
ctx
hc
hc_budget
hc_ge0
md
programmed_site_prequery_one_step_challenge_counter_bound
qs
```



```
query_count
sk
```

**Theorem 3.144** (EasyCrypt line 4070). *EasyCrypt name.*

```
budgeted_programmed_adaptive_prequery_lift_checked_31
```

**Checked EasyCrypt statement.**

```
lemma budgeted_programmed_adaptive_prequery_lift_checked_31
  md sk m ctx qs hc :
  budgeted_programmed_challenge_query_discipline hc qs =>
  mu drandom_coins
    (fun coins =>
      programming_site_prequeried qs
        (honest_signing_programming_site md sk m ctx coins)) <=
  31%r / 288230376151711744%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
budgeted_programmed_adaptive_prequery_lift
challenge_support_cardinality_lower_bound
ctx
discipline
hash_query_count
hc
ler_trans
md
qs
rom_hash_query_budget
sk
```

**Theorem 3.145** (EasyCrypt line 4089). *EasyCrypt name.*

```
budgeted_programmed_hash_get_preserves_discipline
```

**Checked EasyCrypt statement.**

```
lemma budgeted_programmed_hash_get_preserves_discipline :
  hoare [EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH.get :
    budgeted_programmed_query_count_discipline
      EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count
      EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\
    budgeted_programmed_reprogram_discipline
      EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current
      EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current
      EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites
      EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram ==>
    budgeted_programmed_query_count_discipline
      EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count
      EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\
    budgeted_programmed_reprogram_discipline
      EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current
      EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current
      EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites
```

```
EUFCMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *call*, *wp*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
budgeted_counted_lazy_rom_get_true
budgeted_programmed_query_count_discipline
hash_query_budget_count
signature_query_budget_count
```

**Theorem 3.146** (EasyCrypt line 4119). *EasyCrypt name.*

```
budgeted_programmed_reprogram_discipline_after_actual_signature
```

**Checked EasyCrypt statement.**

```
lemma budgeted_programmed_reprogram_discipline_after_actual_signature
  pk sk m ctx coins sites bad_reprogram :
  budgeted_programmed_reprogram_discipline pk sk sites bad_reprogram =>
  pk = public_key_of_secret haetae_mode sk =>
  budgeted_programmed_reprogram_discipline
    pk sk
    (actual_signature_programming_site haetae_mode
      (transcript_of_signature haetae_mode pk m ctx
        (sign_internal haetae_mode sk m ctx coins)) :: sites)
  (bad_reprogram ∨
    programming_site_reprograms sites
      (actual_signature_programming_site haetae_mode
        (transcript_of_signature haetae_mode pk m ctx
          (sign_internal haetae_mode sk m ctx coins))))).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*, *apply*, *split*. Named identifiers syntactically cited in the proof block:

```
actual_signature_programming_site
actual_signature_programming_site_sign_internal_conflict_step
budgeted_programmed_reprogram_discipline
coins
ctx
exact
haetae_mode
hclear
hconf
hconf_next
hkey
hno_reprogram
```

```

hsite
pk
programming_site_log_conflict_free_for_honest
programming_site_reprograms
sign_internal
sites
sk
transcript_of_signature

```

**Theorem 3.147** (EasyCrypt line 4157). *EasyCrypt* name.

```

budgeted_programmed_sign_oracle_preserves_discipline

```

*Checked EasyCrypt statement.*

```

lemma budgeted_programmed_sign_oracle_preserves_discipline :
  hoare [EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O.sign :
    budgeted_programmed_query_count_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\
    budgeted_programmed_reprogram_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram ==>
    budgeted_programmed_query_count_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\
    budgeted_programmed_reprogram_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*, *rnd*, *case*, *split*, *rewrite*, *smt*, *apply*. Named identifiers syntactically cited in the proof block:

```

CountedLazyROM
DirectSigningCoinSampler
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign
bad_reprogram
budgeted_programmed_query_count_discipline
budgeted_programmed_reprogram_discipline
budgeted_programmed_reprogram_discipline_after_actual_signature
ctx
exact
get
hash_query_budget_count
hcount
hdis
hr
hsign_case
hsign_lt
observe_transcript
pk_current
program

```

```

programmed_sites
sample
sampled_coins
signature_query_budget_count
signing_count
sk_current

```

**Theorem 3.148** (EasyCrypt line 4223). *EasyCrypt name.*

```
adversary_budgeted_programmed_discipline_preserved
```

*Checked EasyCrypt statement.*

```

lemma adversary_budgeted_programmed_discipline_preserved :
  hoare[
    A (EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH,
      EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O).forge :
    budgeted_programmed_query_count_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\
    budgeted_programmed_reprogram_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram ==>
    budgeted_programmed_query_count_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\
    budgeted_programmed_reprogram_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *call*, *apply*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign
adversary_hash_count
bad_reprogram
budgeted_programmed_hash_get_preserves_discipline
budgeted_programmed_query_count_discipline
budgeted_programmed_reprogram_discipline
budgeted_programmed_sign_oracle_preserves_discipline
pk_current
programmed_sites
signing_count
sk_current

```

**Theorem 3.149** (EasyCrypt line 4260). *EasyCrypt name.*

```
budgeted_programmed_main_reprogram_clear
```

*Checked EasyCrypt statement.*

```

lemma budgeted_programmed_main_reprogram_clear :
  hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main :
    true ==>
    ! EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *conseq*, *rnd*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign
adversary_budgeted_programmed_discipline_preserved
adversary_hash_count
bad_reprogram
budgeted_counted_lazy_rom_get_true
budgeted_programmed_query_count_discipline
budgeted_programmed_reprogram_discipline
counted_lazy_rom_init_clears
hash_query_budget_count
pk_current
programmed_sites
sd
signature_query_budget_count
signing_count
sk_current

```

**Theorem 3.150** (EasyCrypt line 4310). *EasyCrypt name.*

```

budgeted_programmed_bad_reprogram_zero

```

**Checked EasyCrypt statement.**

```

lemma budgeted_programmed_bad_reprogram_zero &m :
  Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] =
  0%r.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byphoare*, *hoare*, *conseq*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign
bad_reprogram
budgeted_programmed_main_reprogram_clear

```

**Theorem 3.151** (EasyCrypt line 4321). *EasyCrypt name.*

```
budgeted_programmed_prequery_reprogram_bad_bound_from_prequery
```

**Checked EasyCrypt statement.**

```
lemma budgeted_programmed_prequery_reprogram_bad_bound_from_prequery &m :
  Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery] <=
    ((rom_total_query_budget * rom_total_query_budget) +
     (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) /
    challenge_support_cardinality_lower_bound =>
  Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery /\
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] <=
    ((rom_total_query_budget * rom_total_query_budget) +
     (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) /
    challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign
Pr
bad_prequery
bad_reprogram
budgeted_programmed_bad_reprogram_zero
ler_trans
main
mu_or
prequery_bound
```

**Theorem 3.152** (EasyCrypt line 4345). *EasyCrypt name.*

```
budgeted_programmed_hash_get_preserves_query_budget
```

**Checked EasyCrypt statement.**

```
lemma budgeted_programmed_hash_get_preserves_query_budget :
  hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH.get :
    0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=
    hash_query_budget_count /\
    0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=
    signature_query_budget_count ==>
    0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=
    hash_query_budget_count /\
    0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=
    signature_query_budget_count].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `call`, `wp`, `rewrite`, `smt`. Named identifiers syntactically cited in the proof block:

```
budgeted_counted_lazy_rom_get_true
hash_query_budget_count
signature_query_budget_count
```

**Theorem 3.153** (EasyCrypt line 4368). *EasyCrypt name.*

```
budgeted_programmed_sign_oracle_preserves_query_budget
```

**Checked EasyCrypt statement.**

```
lemma budgeted_programmed_sign_oracle_preserves_query_budget :
  hoare [EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O.sign :
    0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=
    hash_query_budget_count /\
    0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=
    signature_query_budget_count ==>
    0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=
    hash_query_budget_count /\
    0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=
    signature_query_budget_count].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `inline`, `wp`, `call`, `rnd`, `rewrite`. Named identifiers syntactically cited in the proof block:

```
DirectSigningCoinSampler
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign
get
observe_transcript
program
sample
signature_query_budget_count
```

**Theorem 3.154** (EasyCrypt line 4399). *EasyCrypt name.*

```
adversary_budgeted_programmed_query_budget_preserved
```

**Checked EasyCrypt statement.**

```
lemma adversary_budgeted_programmed_query_budget_preserved :
  hoare [
    A (EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH,
```

```

    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O).forge :
0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=
    hash_query_budget_count /\
0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=
    signature_query_budget_count ==>
0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=
    hash_query_budget_count /\
0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=
    signature_query_budget_count].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *call*, *apply*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign
adversary_hash_count
budgeted_programmed_hash_get_preserves_query_budget
budgeted_programmed_sign_oracle_preserves_query_budget
hash_query_budget_count
signature_query_budget_count
signing_count

```

**Theorem 3.155** (EasyCrypt line 4430). *EasyCrypt name.*

```

budgeted_programmed_main_query_budget_preserved

```

**Checked EasyCrypt statement.**

```

lemma budgeted_programmed_main_query_budget_preserved :
  hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main :
    true ==>
0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=
    hash_query_budget_count /\
0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=
    signature_query_budget_count].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *conseq*, *rnd*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign

```



```

adversary_budgeted_programmed_query_budget_preserved
adversary_hash_count
budgeted_counted_lazy_rom_bad_true
budgeted_counted_lazy_rom_get_true
budgeted_counted_lazy_rom_init_true
hash_query_budget_count
signature_query_budget_count
signing_count

```

**Theorem 3.156** (EasyCrypt line 4467). *EasyCrypt name.*

```

budgeted_programmed_query_budget_certifies_branch_bound

```

**Checked EasyCrypt statement.**

```

lemma budgeted_programmed_query_budget_certifies_branch_bound &M :
  Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &M :
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery \ /
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] <=
    ((rom_total_query_budget * rom_total_query_budget) +
     (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) /
    challenge_support_cardinality_lower_bound =>
  Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &M :
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery \ /
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] <=
    counted_rom_prequery_reprogramming_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

counted_rom_prequery_reprogramming_term_cardinalityE

```

**Theorem 3.157** (EasyCrypt line 4483). *EasyCrypt name.*

```

budgeted_programmed_query_budget_certifies_branch_bound_from_prequery

```

**Checked EasyCrypt statement.**

```

lemma budgeted_programmed_query_budget_certifies_branch_bound_from_prequery
  &M :
  Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &M :
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery] <=
    ((rom_total_query_budget * rom_total_query_budget) +
     (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) /
    challenge_support_cardinality_lower_bound =>
  Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &M :
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery \ /
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] <=
    counted_rom_prequery_reprogramming_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
budgeted_programmed_prequery_reprogram_bad_bound_from_prequery
budgeted_programmed_query_budget_certifies_branch_bound
prequery_bound
```

**Theorem 3.158** (EasyCrypt line 4500). *EasyCrypt* name.

```
budgeted_programmed_query_budget_certifies_checked_31_bound
```

**Checked EasyCrypt statement.**

```
lemma budgeted_programmed_query_budget_certifies_checked_31_bound
  &m :
  Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery] <=
    ((rom_total_query_budget * rom_total_query_budget) +
     (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) /
    challenge_support_cardinality_lower_bound =>
  Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery \ /
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] <=
    31%r / 288230376151711744%r.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
budgeted_programmed_query_budget_certifies_branch_bound_from_prequery
counted_rom_prequery_reprogramming_term
counted_rom_prequery_reprogramming_term_concreteE
ler_trans
prequery_bound
```

**Theorem 3.159** (EasyCrypt line 4529). *EasyCrypt* name.

```
budgeted_sampled_direct_counted_lazy_rom_get_true
```

**Checked EasyCrypt statement.**

```
lemma budgeted_sampled_direct_counted_lazy_rom_get_true :
  hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).C.get :
    true ==> true].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `wp`, `call`.

**Theorem 3.160** (EasyCrypt line 4540). *EasyCrypt name.*

```
budgeted_sampled_direct_counted_lazy_rom_init_true
```

**Checked EasyCrypt statement.**

```
lemma budgeted_sampled_direct_counted_lazy_rom_init_true :
  hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).C.init :
    true ==> true].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `wp`, `call`.

**Theorem 3.161** (EasyCrypt line 4551). *EasyCrypt name.*

```
budgeted_sampled_direct_counted_lazy_rom_bad_true
```

**Checked EasyCrypt statement.**

```
lemma budgeted_sampled_direct_counted_lazy_rom_bad_true :
  hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).C.bad :
    true ==> true].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`.

**Theorem 3.162** (EasyCrypt line 4559). *EasyCrypt name.*

```
budgeted_sampled_direct_hash_get_preserves_challenge_query_discipline
```

**Checked EasyCrypt statement.**

```
lemma budgeted_sampled_direct_hash_get_preserves_challenge_query_discipline :
  hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).AH.get :
    budgeted_programmed_challenge_query_discipline
      EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count
      EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries ==>
```

```

budgeted_programmed_challenge_query_discipline
  EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count
  EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

DirectSigningCoinSampler
EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
budgeted_programmed_challenge_query_discipline
challenge_query_log_count_cons_le
get
hash_query_budget_count

```

**Theorem 3.163** (EasyCrypt line 4582). *EasyCrypt* name.

```

budgeted_sampled_direct_sign_oracle_preserves_challenge_query_discipline

```

**Checked EasyCrypt statement.**

```

lemma budgeted_sampled_direct_sign_oracle_preserves_challenge_query_discipline :
  hoare [EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).O.sign :
    budgeted_programmed_challenge_query_discipline
      EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count
      EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries ==>
    budgeted_programmed_challenge_query_discipline
      EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count
      EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*, *rnd*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

DirectSigningCoinSampler
EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
budgeted_programmed_challenge_query_discipline
challenge_query_log_count_message_cons
get
observe_transcript
program
sample

```

**Theorem 3.164** (EasyCrypt line 4612). *EasyCrypt* name.

```
adversary_budgeted_sampled_direct_challenge_query_discipline_preserved
```

### Checked EasyCrypt statement.

```
lemma adversary_budgeted_sampled_direct_challenge_query_discipline_preserved :  
  hoare[  
    A(EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign  
      (H, A, DirectSigningCoinSampler).AH,  
      EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign  
      (H, A, DirectSigningCoinSampler).D).forge :  
    budgeted_programmed_challenge_query_discipline  
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count  
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries ==>  
    budgeted_programmed_challenge_query_discipline  
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count  
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *call*, *apply*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign  
adversary_hash_count  
budgeted_programmed_challenge_query_discipline  
budgeted_sampled_direct_hash_get_preserves_challenge_query_discipline  
budgeted_sampled_direct_sign_oracle_preserves_challenge_query_discipline  
hash_queries
```

### Theorem 3.165 (EasyCrypt line 4636). *EasyCrypt* name.

```
budgeted_sampled_direct_main_challenge_query_discipline
```

### Checked EasyCrypt statement.

```
lemma budgeted_sampled_direct_main_challenge_query_discipline :  
  hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign  
    (H, A, DirectSigningCoinSampler).main :  
    true ==>  
    budgeted_programmed_challenge_query_discipline  
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count  
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *conseq*, *rnd*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
adversary_budgeted_sampled_direct_challenge_query_discipline_preserved
adversary_hash_count
budgeted_programmed_challenge_query_discipline
budgeted_sampled_direct_counted_lazy_rom_bad_true
challenge_query_log_count
counted_lazy_rom_get_nonchallenge_preserves_challenge_query_count
counted_lazy_rom_init_clears_hash_queries
hash_queries
hash_query_budget_count
matrix_expand_query
ro_query_is_challenge
sd

```

**Theorem 3.166** (EasyCrypt line 4668). *EasyCrypt name.*

```

budgeted_sampled_direct_hash_get_preserves_discipline

```

*Checked EasyCrypt statement.*

```

lemma budgeted_sampled_direct_hash_get_preserves_discipline :
  hoare [EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).AH.get :
    budgeted_programmed_query_count_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\
    budgeted_programmed_reprogram_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram ==>
    budgeted_programmed_query_count_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\
    budgeted_programmed_reprogram_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram].

```

**Formal mathematical reading.** *Mathematical theorem.* This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *call*, *wp*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

budgeted_programmed_query_count_discipline
budgeted_sampled_direct_counted_lazy_rom_get_true
hash_query_budget_count
signature_query_budget_count

```

**Theorem 3.167** (EasyCrypt line 4699). *EasyCrypt name.*

```

budgeted_sampled_direct_sign_oracle_preserves_discipline

```

*Checked EasyCrypt statement.*

```

lemma budgeted_sampled_direct_sign_oracle_preserves_discipline :
  hoare [EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).O.sign :
    budgeted_programmed_query_count_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\
    budgeted_programmed_reprogram_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram ==>
    budgeted_programmed_query_count_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\
    budgeted_programmed_reprogram_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram].

```

**Formal mathematical reading.** *Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.*

**Role in the proof.** *Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **proc**, **inline**, **wp**, **call**, **rnd**, **case**, **split**, **rewrite**, **smt**, **apply**. Named identifiers syntactically cited in the proof block:*

```

CountedLazyROM
DirectSigningCoinSampler
EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
bad_reprogram
budgeted_programmed_query_count_discipline
budgeted_programmed_reprogram_discipline
budgeted_programmed_reprogram_discipline_after_actual_signature
ctx
exact
get
hash_query_budget_count
hcount
hdis
hr
hsign_case
hsign_lt
observe_transcript
pk_current
program
programmed_sites
sample
sampled_coins
signature_query_budget_count
signing_count
sk_current

```

**Theorem 3.168** (EasyCrypt line 4769). *EasyCrypt name.*

```
adversary_budgeted_sampled_direct_discipline_preserved
```

**Checked EasyCrypt statement.**

```
lemma adversary_budgeted_sampled_direct_discipline_preserved :
```

```

hoare[
  A(EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).AH,
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).O).forge :
  budgeted_programmed_query_count_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\
  budgeted_programmed_reprogram_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram ==>
  budgeted_programmed_query_count_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\
  budgeted_programmed_reprogram_discipline
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *call*, *apply*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
adversary_hash_count
bad_reprogram
budgeted_programmed_query_count_discipline
budgeted_programmed_reprogram_discipline
budgeted_sampled_direct_hash_get_preserves_discipline
budgeted_sampled_direct_sign_oracle_preserves_discipline
pk_current
programmed_sites
signing_count
sk_current

```

**Theorem 3.169** (EasyCrypt line 4808). *EasyCrypt* name.

```

budgeted_sampled_direct_main_reprogram_clear

```

**Checked EasyCrypt statement.**

```

lemma budgeted_sampled_direct_main_reprogram_clear :
  hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).main :
    true ==>
    ! EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.



**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `wp`, `call`, `conseq`, `rnd`, `rewrite`, `smt`. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
adversary_budgeted_sampled_direct_discipline_preserved
adversary_hash_count
bad_reprogram
budgeted_programmed_query_count_discipline
budgeted_programmed_reprogram_discipline
budgeted_sampled_direct_counted_lazy_rom_get_true
counted_lazy_rom_init_clears
hash_query_budget_count
pk_current
programmed_sites
sd
signature_query_budget_count
signing_count
sk_current

```

**Theorem 3.170** (EasyCrypt line 4859). *EasyCrypt name.*

```

budgeted_sampled_direct_bad_reprogram_zero

```

**Checked EasyCrypt statement.**

```

lemma budgeted_sampled_direct_bad_reprogram_zero &m :
  Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).main() @ &m :
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram] =
  0%r.

```

**Formal mathematical reading.** *Mathematical theorem.* This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `byphoare`, `hoare`, `conseq`. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
bad_reprogram
budgeted_sampled_direct_main_reprogram_clear

```

**Theorem 3.171** (EasyCrypt line 4871). *EasyCrypt name.*

```

budgeted_sampled_direct_one_step_bound_nonnegative

```

**Checked EasyCrypt statement.**

```

lemma budgeted_sampled_direct_one_step_bound_nonnegative :
  0%r <=
    (rom_hash_query_budget + 1%r) /
    challenge_support_cardinality_lower_bound.

```

**Formal mathematical reading.** *Mathematical theorem.* This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
addr_ge0
challenge_support_cardinality_lower_bound_nonnegative
divr_ge0
rom_hash_query_budget_nonnegative
```

**Theorem 3.172** (EasyCrypt line 4881). *EasyCrypt* name.

```
budgeted_sampled_direct_sampler_message_hash_prequery_bound
```

**Checked EasyCrypt statement.**

```
lemma budgeted_sampled_direct_sampler_message_hash_prequery_bound
  pk sk qs m ctx :
  phoare[DirectSigningCoinSampler.sample :
    budgeted_programmed_challenge_query_discipline
      EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count
      qs /\
    pk = public_key_of_secret haetae_mode sk /\
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current = pk /\
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current = sk /\
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries = qs ==>
    programming_site_prequeried
      (message_hash_query
        (public_key_of_secret haetae_mode sk) ctx m :: qs)
      (actual_signature_programming_site haetae_mode
        (transcript_of_signature haetae_mode
          pk
          m ctx
          (sign_internal haetae_mode
            sk m ctx res))))] <=
  ((rom_hash_query_budget + 1%r) /
    challenge_support_cardinality_lower_bound).
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*, *smt*, *byphoare*, *conseq*. Named identifiers syntactically cited in the proof block:

```
actual_signature_programming_site
budgeted_programmed_challenge_query_discipline
bypr
challenge_query_log_count
ctx
direct_signing_coin_sampler_message_hash_budgeted_prequery_bound
discipline
haetae_mode
hash_query_budget_count
hc_budget
m0
```

```

message_hash_query
pk
programming_site_prequeried
public_key_of_secret
qs
query_budget
query_count
sign_internal
sk
transcript_of_signature

```

**Theorem 3.173** (EasyCrypt line 4920). *EasyCrypt name.*

```

budgeted_sampled_direct_bad_prequery_bound

```

**Checked EasyCrypt statement.**

```

lemma budgeted_sampled_direct_bad_prequery_bound &m :
  islossless H.get =>
  islossless H.set =>
  Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).main() @ &m :
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery] <=
    ((rom_hash_query_budget + rom_signature_query_budget) *
     (rom_hash_query_budget + 1%r)) /
    challenge_support_cardinality_lower_bound.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *conseq*, *rewrite*, *smt*, *wp*, *inline*, *call*, *rnd*, *proc*, *hoare*, *apply*. Named identifiers syntactically cited in the proof block:

```

AH
BRA
Bigreal
DirectSigningCoinSampler
EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
H_get_ll
H_get_true
H_set_ll
H_set_true
StdBigop
actual_signature_programming_site
adversary_hash_count
bad_prequery
budgeted_programmed_challenge_query_discipline
budgeted_sampled_direct_one_step_bound_nonnegative
challenge_query_log_count
challenge_query_log_count_cons_le
challenge_support_cardinality_lower_bound
coins
ctx
fel
get
h1
h2
h3
h4

```

```

h5
h6
h7
h8
haetae_mode
hash_queries
hash_query_budget_count
hash_query_count
hr
init
keygen_internal
ler_trans
matrix_expand_query
message_hash_query
observe_transcript
old_bad
old_count
phoare
pk_current
program
programmed_site_prequery_one_step_message_hash_counter_bound
programming_site_prequeried
public_key_of_secret
ro_query_is_challenge
rom_hash_query_budget
rom_signature_query_budget
sample
sampled_coins
sd
set
sign
sign_internal
signature_query_budget_count
signature_query_count
signing_coin_distribution_lossless
signing_count
sk_current
sumri_const
transcript_of_signature
trivial

```

**Theorem 3.174** (EasyCrypt line 5182). *EasyCrypt name.*

```

budgeted_sampled_direct_prequery_reprogram_bad_bound

```

*Checked EasyCrypt statement.*

```

lemma budgeted_sampled_direct_prequery_reprogram_bad_bound &m :
  islossless H.get =>
  islossless H.set =>
  Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).main() @ &m :
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery \ /
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram] <=
  counted_rom_prequery_reprogramming_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *apply*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
DirectSigningCoinSampler
EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
H_get_ll
H_set_ll
Pr
bad_prequery
bad_reprogram
budgeted_sampled_direct_bad_prequery_bound
budgeted_sampled_direct_bad_reprogram_zero
challenge_support_cardinality_lower_bound
counted_rom_prequery_reprogramming_term_cardinalityE
hash_query_count
ler_trans
main
mu_or
rom_hash_query_budget
rom_signature_query_budget
rom_total_query_budget
signature_query_count
```

**Theorem 3.175** (EasyCrypt line 5213). *EasyCrypt name.*

```
budgeted_sampled_direct_prequery_reprogram_bad_checked_31
```

**Checked EasyCrypt statement.**

```
lemma budgeted_sampled_direct_prequery_reprogram_bad_checked_31 &m :
  islossless H.get =>
  islossless H.set =>
  Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).main() @ &m :
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery \ /
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram] <=
  31%r / 288230376151711744%r.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
H_get_ll
H_set_ll
budgeted_sampled_direct_prequery_reprogram_bad_bound
counted_rom_prequery_reprogramming_term
counted_rom_prequery_reprogramming_term_concreteE
ler_trans
```

**Theorem 3.176** (EasyCrypt line 5241). *EasyCrypt name.*

```
budgeted_programmed_counted_sampled_direct_hash_get_equiv
```

**Checked EasyCrypt statement.**

```

equiv budgeted_programmed_counted_sampled_direct_hash_get_equiv :
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH.get ~
  EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).AH.get :
  ={glob H, arg} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.signature_sites{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.signature_sites{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_min_entropy{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_min_entropy{2}
  ==>
  ={glob H, res} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.signature_sites{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.signature_sites{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_min_entropy{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_min_entropy{2}.

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *inline*, *call*. Named identifiers syntactically cited in the proof block:

```

DirectSigningCoinSampler
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign
EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
arg
get
glob

```

**Theorem 3.177** (EasyCrypt line 5295). *EasyCrypt* name.

```

budgeted_programmed_counted_sampled_direct_sign_equiv

```

**Checked EasyCrypt statement.**

```

equiv budgeted_programmed_counted_sampled_direct_sign_equiv :
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O.sign ~
  EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).O.sign :
    ={glob H, arg} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.queries{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.queries{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.transcripts{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.transcripts{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.records{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.records{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.signature_sites{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.signature_sites{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_min_entropy{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_min_entropy{2}
  ==>
  ={glob H, res} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.queries{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.queries{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.transcripts{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.transcripts{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.records{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.records{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.signature_sites{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.signature_sites{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_min_entropy{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_min_entropy{2}.

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*, *rnd*. Named identifiers syntactically cited in the proof block:

```
DirectSigningCoinSampler
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign
EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
arg
get
glob
observe_transcript
program
sample
```

**Theorem 3.178** (EasyCrypt line 5380). *EasyCrypt* name.

```
adversary_budgeted_programmed_counted_sampled_direct_equiv
```

**Checked EasyCrypt statement.**

```
equiv adversary_budgeted_programmed_counted_sampled_direct_equiv :
  A(EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH,
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O).forge ~
  A(EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).AH,
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).O).forge :
  ={glob H, glob A, arg} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.queries{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.queries{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.transcripts{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.transcripts{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.records{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.records{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.signature_sites{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.signature_sites{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_min_entropy{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_min_entropy{2}
  ==>
  ={glob H, res} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.queries{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.queries{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.transcripts{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.transcripts{2} /\
```



```

EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.records{1} =
  EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.records{2} /\
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count{1} =
  EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{2} /\
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count{1} =
  EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count{2} /\
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries{1} =
  EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{2} /\
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites{1} =
  EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites{2} /\
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.signature_sites{1} =
  EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.signature_sites{2} /\
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =
  EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =
  EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2} /\
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_min_entropy{1} =
  EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_min_entropy{2}.

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *call*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign
EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
adversary_hash_count
bad_min_entropy
bad_prequery
bad_reprogram
budgeted_programmed_counted_sampled_direct_hash_get_equiv
budgeted_programmed_counted_sampled_direct_sign_equiv
glob
hash_queries
pk_current
programmed_sites
queries
records
signature_sites
signing_count
sk_current
transcripts

```

**Theorem 3.179** (EasyCrypt line 5480). *EasyCrypt name.*

```

budgeted_programmed_counted_sampled_direct_main_equiv

```

**Checked EasyCrypt statement.**

```

equiv budgeted_programmed_counted_sampled_direct_main_equiv :
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main ~
  EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
  (H, A, DirectSigningCoinSampler).main :
  ={glob H, glob A} ==>
  ={res} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\
  EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =

```

```
EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2}.
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*, *apply*, *rnd*. Named identifiers syntactically cited in the proof block:

```
DirectSigningCoinSampler
EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign
EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
adversary_budgeted_programmed_counted_sampled_direct_equiv
adversary_hash_count
arg
bad
bad_min_entropy
bad_prequery
bad_reprogram
get
glob
hash_queries
init
pk_current
programmed_sites
queries
records
signature_sites
signing_count
sk_current
transcripts
```

**Theorem 3.180** (EasyCrypt line 5572). *EasyCrypt name*.

```
budgeted_programmed_counted_sampled_direct_prequery_reprogram_bad_exact
```

**Checked EasyCrypt statement.**

```
lemma budgeted_programmed_counted_sampled_direct_prequery_reprogram_bad_exact
  &m :
  Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery \ /
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] =
  Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign
    (H, A, DirectSigningCoinSampler).main() @ &m :
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery \ /
    EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram].
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byequiv*. Named identifiers syntactically cited in the proof

block:

```
budgeted_programmed_counted_sampled_direct_main_equiv
```

**Theorem 3.181** (EasyCrypt line 5585). *EasyCrypt name.*

```
budgeted_programmed_prequery_reprogram_bad_checked_31_via_direct_sampler
```

**Checked EasyCrypt statement.**

```
lemma budgeted_programmed_prequery_reprogram_bad_checked_31_via_direct_sampler
  &m :
  islossless H.get =>
  islossless H.set =>
  Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery /\
    EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] <=
  31%r / 288230376151711744%r.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
H_get_ll
H_set_ll
budgeted_programmed_counted_sampled_direct_prequery_reprogram_bad_exact
budgeted_sampled_direct_prequery_reprogram_bad_checked_31
```

**Theorem 3.182** (EasyCrypt line 5608). *EasyCrypt name.*

```
haetae_rom_internal_oracle_erasure
```

**Checked EasyCrypt statement.**

```
equiv haetae_rom_internal_oracle_erasure :
  SIG.EUF_CMA(H, HAETA, A).O.sign ~
  EUF_CMA_ROMInternalTranscriptSign(H, A).O.sign :
  ={glob H, arg} /\
  SIG.EUF_CMA.sk{1} = EUF_CMA_ROMInternalTranscriptSign.sk_current{2} /\
  SIG.EUF_CMA.queries{1} =
    EUF_CMA_ROMInternalTranscriptSign.queries{2}
  ==>
  ={glob H, res} /\
  SIG.EUF_CMA.sk{1} = EUF_CMA_ROMInternalTranscriptSign.sk_current{2} /\
  SIG.EUF_CMA.queries{1} =
    EUF_CMA_ROMInternalTranscriptSign.queries{2}.
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `inline`, `wp`, `call`, `rnd`. Named identifiers syntactically cited in the proof block:

```
HAETAETAE
arg
glob
sign
```

**Theorem 3.183** (EasyCrypt line 5637). *EasyCrypt name.*

```
adversary_haetae_rom_internal_erasure
```

**Checked EasyCrypt statement.**

```
equiv adversary_haetae_rom_internal_erasure :
  A(H, SIG.EUF_CMA(H, HAETAETAE, A).O).forge ~
  A(H, EUF_CMA_ROMInternalTranscriptSign(H, A).O).forge :
    ={glob H, glob A, arg} /\
    SIG.EUF_CMA.sk{1} = EUF_CMA_ROMInternalTranscriptSign.sk_current{2} /\
    SIG.EUF_CMA.queries{1} =
      EUF_CMA_ROMInternalTranscriptSign.queries{2}
    ==>
    ={glob H, res} /\
    SIG.EUF_CMA.sk{1} = EUF_CMA_ROMInternalTranscriptSign.sk_current{2} /\
    SIG.EUF_CMA.queries{1} =
      EUF_CMA_ROMInternalTranscriptSign.queries{2}.
```

**Formal mathematical reading.** *Mathematical theorem.* This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** *Use in this chapter.* It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `inline`, `wp`, `call`, `rnd`. Named identifiers syntactically cited in the proof block:

```
EUF_CMA
EUF_CMA_ROMInternalTranscriptSign
HAETAETAE
SIG
arg
glob
queries
sign
sk
sk_current
```

**Theorem 3.184** (EasyCrypt line 5672). *EasyCrypt name.*

```
haetae_rom_internal_transcript_erasure_exact
```

**Checked EasyCrypt statement.**

```
lemma haetae_rom_internal_transcript_erasure_exact Em :
  Pr[SIG.EUF_CMA(H, HAETAETAE, A).main() @ Em : res] =
  Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ Em : res].
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byequiv*, *proc*, *inline*, *wp*, *call*, *apply*, *rnd*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA
EUF_CMA_ROMInternalTranscriptSign
HAETAETAE
SIG
adversary_haetae_rom_internal_erasure
arg
glob
kg
queries
sk
sk_current
verify

```

**Theorem 3.185** (EasyCrypt line 5713). *EasyCrypt name.*

```
rom_internal_oracle_structural_nma
```

**Checked EasyCrypt statement.**

```

equiv rom_internal_oracle_structural_nma :
  EUF_CMA_ROMInternalTranscriptSign(H, A).O.sign ~
  ROMInternalTranscriptAsNMA(A, H).O.sign :
    ={glob H, arg} /\
    EUF_CMA_ROMInternalTranscriptSign.pk_current{1} =
      ROMInternalTranscriptAsNMA.pk_current{2} /\
    EUF_CMA_ROMInternalTranscriptSign.sk_current{1} =
      ROMInternalTranscriptAsNMA.sk_current{2} /\
    EUF_CMA_ROMInternalTranscriptSign.queries{1} =
      ROMInternalTranscriptAsNMA.queries{2} /\
    EUF_CMA_ROMInternalTranscriptSign.transcripts{1} =
      ROMInternalTranscriptAsNMA.transcripts{2} /\
    EUF_CMA_ROMInternalTranscriptSign.records{1} =
      ROMInternalTranscriptAsNMA.records{2}
    ==>
    ={glob H, res} /\
    EUF_CMA_ROMInternalTranscriptSign.pk_current{1} =
      ROMInternalTranscriptAsNMA.pk_current{2} /\
    EUF_CMA_ROMInternalTranscriptSign.sk_current{1} =
      ROMInternalTranscriptAsNMA.sk_current{2} /\
    EUF_CMA_ROMInternalTranscriptSign.queries{1} =
      ROMInternalTranscriptAsNMA.queries{2} /\
    EUF_CMA_ROMInternalTranscriptSign.transcripts{1} =
      ROMInternalTranscriptAsNMA.transcripts{2} /\
    EUF_CMA_ROMInternalTranscriptSign.records{1} =
      ROMInternalTranscriptAsNMA.records{2}.

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *rnd*. Named identifiers syntactically cited in the proof block:

```
arg
glob
```

**Theorem 3.186** (EasyCrypt line 5755). *EasyCrypt name*.

```
adversary_rom_internal_structural_nma
```

**Checked EasyCrypt statement.**

```
equiv adversary_rom_internal_structural_nma :
  A(H, EUF_CMA_ROMInternalTranscriptSign(H, A).0).forge ~
  A(H, ROMInternalTranscriptAsNMA(A, H).0).forge :
  ={glob H, glob A, arg} /\
  EUF_CMA_ROMInternalTranscriptSign.pk_current{1} =
    ROMInternalTranscriptAsNMA.pk_current{2} /\
  EUF_CMA_ROMInternalTranscriptSign.sk_current{1} =
    ROMInternalTranscriptAsNMA.sk_current{2} /\
  EUF_CMA_ROMInternalTranscriptSign.queries{1} =
    ROMInternalTranscriptAsNMA.queries{2} /\
  EUF_CMA_ROMInternalTranscriptSign.transcripts{1} =
    ROMInternalTranscriptAsNMA.transcripts{2} /\
  EUF_CMA_ROMInternalTranscriptSign.records{1} =
    ROMInternalTranscriptAsNMA.records{2}
  ==>
  ={glob H, res} /\
  EUF_CMA_ROMInternalTranscriptSign.pk_current{1} =
    ROMInternalTranscriptAsNMA.pk_current{2} /\
  EUF_CMA_ROMInternalTranscriptSign.sk_current{1} =
    ROMInternalTranscriptAsNMA.sk_current{2} /\
  EUF_CMA_ROMInternalTranscriptSign.queries{1} =
    ROMInternalTranscriptAsNMA.queries{2} /\
  EUF_CMA_ROMInternalTranscriptSign.transcripts{1} =
    ROMInternalTranscriptAsNMA.transcripts{2} /\
  EUF_CMA_ROMInternalTranscriptSign.records{1} =
    ROMInternalTranscriptAsNMA.records{2}.
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *rnd*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMInternalTranscriptSign
ROMInternalTranscriptAsNMA
arg
glob
pk_current
queries
records
sk_current
transcripts
```

**Theorem 3.187** (EasyCrypt line 5811). *EasyCrypt name.*

```
rom_internal_transcript_structural_nma_bound
```

**Checked EasyCrypt statement.**

```
lemma rom_internal_transcript_structural_nma_bound  $\mathcal{E}m$  :
  Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @  $\mathcal{E}m$  : res] <=
  Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptAsNMA(A)).main()
    @  $\mathcal{E}m$  : res].
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `byequiv`, `proc`, `inline`, `wp`, `call`, `apply`, `rnd`, `rewrite`. Named identifiers syntactically cited in the proof block:

```
EUF_CMA_ROMInternalTranscriptSign
HAETAE
ROMInternalTranscriptAsNMA
adversary_rom_internal_structural_nma
arg
forge
glob
keygen_internal
kg
pk_current
public_key_of_secret
queries
records
secret_key_of_seed
sk_current
transcripts
verify
```

**Theorem 3.188** (EasyCrypt line 5858). *EasyCrypt name.*

```
rom_internal_transcript_clear_site_structural_nma_bound
```

**Checked EasyCrypt statement.**

```
lemma rom_internal_transcript_clear_site_structural_nma_bound  $\mathcal{E}m$  :
  Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @  $\mathcal{E}m$  :
    res /\
    transcript_log_signature_programming_sites_clear haetae_mode
    EUF_CMA_ROMInternalTranscriptSign.transcripts] <=
  Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptAsNMA(A)).main()
    @  $\mathcal{E}m$  : res].
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_ROMInternalTranscriptSign
Pr
ler_trans
main
mu_sub
rom_internal_transcript_structural_nma_bound

```

**Theorem 3.189** (EasyCrypt line 5881). *EasyCrypt name*.

```
rom_internal_public_sim_nma_oracle
```

*Checked EasyCrypt statement.*

```

equiv rom_internal_public_sim_nma_oracle :
  ROMInternalTranscriptAsNMA(A, H).O.sign ~
  ROMInternalTranscriptPublicSimAsNMA(A, H).O.sign :
    ={glob H, arg} /\
    ROMInternalTranscriptAsNMA.pk_current{1} =
      ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\
    public_key_of_secret haetae_mode
      ROMInternalTranscriptAsNMA.sk_current{1} =
      ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\
    ROMInternalTranscriptAsNMA.queries{1} =
      ROMInternalTranscriptPublicSimAsNMA.queries{2} /\
    ROMInternalTranscriptAsNMA.transcripts{1} =
      ROMInternalTranscriptPublicSimAsNMA.transcripts{2} /\
    ROMInternalTranscriptAsNMA.records{1} =
      ROMInternalTranscriptPublicSimAsNMA.records{2}
    ==>
    ={glob H, res} /\
    ROMInternalTranscriptAsNMA.pk_current{1} =
      ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\
    public_key_of_secret haetae_mode
      ROMInternalTranscriptAsNMA.sk_current{1} =
      ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\
    ROMInternalTranscriptAsNMA.queries{1} =
      ROMInternalTranscriptPublicSimAsNMA.queries{2} /\
    ROMInternalTranscriptAsNMA.transcripts{1} =
      ROMInternalTranscriptPublicSimAsNMA.transcripts{2} /\
    ROMInternalTranscriptAsNMA.records{1} =
      ROMInternalTranscriptPublicSimAsNMA.records{2}.

```

**Formal mathematical reading.** *Mathematical theorem.* This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *rnd*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

arg
glob
public_sim_commitment_highbitsE
public_sim_commitment_lowbitsE
public_sim_signatureE

```

**Theorem 3.190** (EasyCrypt line 5927). *EasyCrypt name*.



```
adversary_rom_internal_public_sim_nma
```

### Checked EasyCrypt statement.

```
equiv adversary_rom_internal_public_sim_nma :
  A(H, ROMInternalTranscriptAsNMA(A, H).0).forge ~
  A(H, ROMInternalTranscriptPublicSimAsNMA(A, H).0).forge :
    ={glob H, glob A, arg} /\
      ROMInternalTranscriptAsNMA.pk_current{1} =
        ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\
      public_key_of_secret haetae_mode
        ROMInternalTranscriptAsNMA.sk_current{1} =
          ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\
      ROMInternalTranscriptAsNMA.queries{1} =
        ROMInternalTranscriptPublicSimAsNMA.queries{2} /\
      ROMInternalTranscriptAsNMA.transcripts{1} =
        ROMInternalTranscriptPublicSimAsNMA.transcripts{2} /\
      ROMInternalTranscriptAsNMA.records{1} =
        ROMInternalTranscriptPublicSimAsNMA.records{2}
    ==>
    ={glob H, res} /\
      ROMInternalTranscriptAsNMA.pk_current{1} =
        ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\
      public_key_of_secret haetae_mode
        ROMInternalTranscriptAsNMA.sk_current{1} =
          ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\
      ROMInternalTranscriptAsNMA.queries{1} =
        ROMInternalTranscriptPublicSimAsNMA.queries{2} /\
      ROMInternalTranscriptAsNMA.transcripts{1} =
        ROMInternalTranscriptPublicSimAsNMA.transcripts{2} /\
      ROMInternalTranscriptAsNMA.records{1} =
        ROMInternalTranscriptPublicSimAsNMA.records{2}.
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *rnd*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
ROMInternalTranscriptAsNMA
ROMInternalTranscriptPublicSimAsNMA
arg
glob
haetae_mode
pk_current
public_key_of_secret
public_sim_commitment_highbitsE
public_sim_commitment_lowbitsE
public_sim_signatureE
queries
records
sk_current
transcripts
```

**Theorem 3.191** (EasyCrypt line 5988). *EasyCrypt name.*

```
rom_internal_nma_public_sim_exact
```

### Checked EasyCrypt statement.

```
lemma rom_internal_nma_public_sim_exact  $\mathcal{E}m$  :
  Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptAsNMA(A)).main()
    @  $\mathcal{E}m$  : res] =
  Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPublicSimAsNMA(A)).main()
    @  $\mathcal{E}m$  : res].
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byequiv*, *proc*, *inline*, *wp*, *call*, *apply*, *rnd*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
HAETAE
ROMInternalTranscriptAsNMA
ROMInternalTranscriptPublicSimAsNMA
adversary_rom_internal_public_sim_nma
arg
forge
glob
haetae_mode
keygen_internal
kg
pk_current
public_key_of_secret
queries
records
secret_key_of_seed
sk_current
transcripts
verify
```

### Theorem 3.192 (EasyCrypt line 6050). *EasyCrypt* name.

```
rom_internal_public_sim_rom_paper_sim_nma_oracle
```

### Checked EasyCrypt statement.

```
equiv rom_internal_public_sim_rom_paper_sim_nma_oracle :
  ROMInternalTranscriptPublicSimAsNMA(A, H).O.sign ~
  ROMInternalTranscriptPaperSimAsNMA
  (A, ROMPaperSimSigningSampler(H), H).O.sign :
  ={glob H, arg} /\
  ROMInternalTranscriptPublicSimAsNMA.pk_current{1} =
  ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
  ROMPaperSimSigningSampler.pk_current{2} =
  ROMInternalTranscriptPublicSimAsNMA.pk_current{1} /\
  ROMInternalTranscriptPublicSimAsNMA.queries{1} =
  ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
  ROMInternalTranscriptPublicSimAsNMA.transcripts{1} =
  ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
  ROMInternalTranscriptPublicSimAsNMA.records{1} =
  ROMInternalTranscriptPaperSimAsNMA.records{2}
  ==>
  ={glob H, res} /\
  ROMInternalTranscriptPublicSimAsNMA.pk_current{1} =
  ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
  ROMPaperSimSigningSampler.pk_current{2} =
```

```

    ROMInternalTranscriptPublicSimAsNMA.pk_current{1} /\
    ROMInternalTranscriptPublicSimAsNMA.queries{1} =
    ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
    ROMInternalTranscriptPublicSimAsNMA.transcripts{1} =
    ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
    ROMInternalTranscriptPublicSimAsNMA.records{1} =
    ROMInternalTranscriptPaperSimAsNMA.records{2}.

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*, *rnd*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

ROMPaperSimSigningSampler
arg
glob
paper_sim_commitment_highbits_public_coinsE
paper_sim_commitment_lowbits_public_coinsE
paper_sim_signature_public_coinsE
sample
sample_with_seed

```

**Theorem 3.193** (EasyCrypt line 6097). *EasyCrypt name.*

```

adversary_rom_internal_public_sim_rom_paper_sim_nma

```

**Checked EasyCrypt statement.**

```

equiv adversary_rom_internal_public_sim_rom_paper_sim_nma :
A(H, ROMInternalTranscriptPublicSimAsNMA(A, H).O).forge ~
A(H, ROMInternalTranscriptPaperSimAsNMA
  (A, ROMPaperSimSigningSampler(H), H).O).forge :
={glob H, glob A, arg} /\
ROMInternalTranscriptPublicSimAsNMA.pk_current{1} =
  ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
ROMPaperSimSigningSampler.pk_current{2} =
  ROMInternalTranscriptPublicSimAsNMA.pk_current{1} /\
ROMInternalTranscriptPublicSimAsNMA.queries{1} =
  ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
ROMInternalTranscriptPublicSimAsNMA.transcripts{1} =
  ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
ROMInternalTranscriptPublicSimAsNMA.records{1} =
  ROMInternalTranscriptPaperSimAsNMA.records{2}
==>
={glob H, res} /\
ROMInternalTranscriptPublicSimAsNMA.pk_current{1} =
  ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
ROMPaperSimSigningSampler.pk_current{2} =
  ROMInternalTranscriptPublicSimAsNMA.pk_current{1} /\
ROMInternalTranscriptPublicSimAsNMA.queries{1} =
  ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
ROMInternalTranscriptPublicSimAsNMA.transcripts{1} =
  ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
ROMInternalTranscriptPublicSimAsNMA.records{1} =
  ROMInternalTranscriptPaperSimAsNMA.records{2}.

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem:

two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*, *rnd*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
ROMInternalTranscriptPaperSimAsNMA
ROMInternalTranscriptPublicSimAsNMA
ROMPaperSimSigningSampler
arg
glob
paper_sim_commitment_highbits_public_coinsE
paper_sim_commitment_lowbits_public_coinsE
paper_sim_signature_public_coinsE
pk_current
queries
records
sample
sample_with_seed
transcripts
```

**Theorem 3.194** (EasyCrypt line 6158). *EasyCrypt name*.

```
rom_internal_nma_public_sim_rom_paper_sim_exact
```

**Checked EasyCrypt statement.**

```
lemma rom_internal_nma_public_sim_rom_paper_sim_exact &m :
  Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPublicSimAsNMA(A)).main()
    @ &m : res] =
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROMPaperSimSigningSampler(H))).main() @ &m : res].
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byequiv*, *proc*, *inline*, *wp*, *call*, *apply*, *rnd*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
HAETAE
ROMInternalTranscriptPaperSimAsNMA
ROMInternalTranscriptPublicSimAsNMA
ROMPaperSimSigningSampler
adversary_rom_internal_public_sim_rom_paper_sim_nma
arg
forge
glob
init
keygen_internal
kg
pk_current
public_key_of_secret
```

```

queries
records
secret_key_of_seed
transcripts
verify

```

**Theorem 3.195** (EasyCrypt line 6221). *EasyCrypt name.*

```
rom_internal_real_signing_paper_sim_nma_oracle
```

**Checked EasyCrypt statement.**

```

equiv rom_internal_real_signing_paper_sim_nma_oracle :
  ROMInternalTranscriptAsNMA(A, H).O.sign ~
  ROMInternalTranscriptPaperSimAsNMA
    (A, RealSigningPaperSimSampler(H), H).O.sign :
    ={glob H, arg} /\
  ROMInternalTranscriptAsNMA.pk_current{1} =
    ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
  public_key_of_secret haetae_mode
    ROMInternalTranscriptAsNMA.sk_current{1} =
    ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
  RealSigningPaperSimSampler.pk_current{2} =
    ROMInternalTranscriptAsNMA.pk_current{1} /\
  RealSigningPaperSimSampler.sk_current{2} =
    ROMInternalTranscriptAsNMA.sk_current{1} /\
  ROMInternalTranscriptAsNMA.queries{1} =
    ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
  ROMInternalTranscriptAsNMA.transcripts{1} =
    ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
  ROMInternalTranscriptAsNMA.records{1} =
    ROMInternalTranscriptPaperSimAsNMA.records{2}
  ==>
  ={glob H, res} /\
  ROMInternalTranscriptAsNMA.pk_current{1} =
    ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
  public_key_of_secret haetae_mode
    ROMInternalTranscriptAsNMA.sk_current{1} =
    ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
  RealSigningPaperSimSampler.pk_current{2} =
    ROMInternalTranscriptAsNMA.pk_current{1} /\
  RealSigningPaperSimSampler.sk_current{2} =
    ROMInternalTranscriptAsNMA.sk_current{1} /\
  ROMInternalTranscriptAsNMA.queries{1} =
    ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
  ROMInternalTranscriptAsNMA.transcripts{1} =
    ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
  ROMInternalTranscriptAsNMA.records{1} =
    ROMInternalTranscriptPaperSimAsNMA.records{2}.

```

**Formal mathematical reading.** *Mathematical theorem.* This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*, *rnd*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

RealSigningPaperSimSampler
arg
glob

```

```

paper_sim_commitment_highbits_real_signing_coinsE
paper_sim_commitment_lowbits_real_signing_coinsE
paper_sim_signature_real_signing_coinsE
sample
sample_with_seed

```

**Theorem 3.196** (EasyCrypt line 6278). *EasyCrypt name.*

```
adversary_rom_internal_real_signing_paper_sim_nma
```

**Checked EasyCrypt statement.**

```

equiv adversary_rom_internal_real_signing_paper_sim_nma :
  A(H, ROMInternalTranscriptAsNMA(A, H).O).forge ~
  A(H, ROMInternalTranscriptPaperSimAsNMA
    (A, RealSigningPaperSimSampler(H), H).O).forge :
    ={glob H, glob A, arg} /\
    ROMInternalTranscriptAsNMA.pk_current{1} =
      ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
    public_key_of_secret haetae_mode
      ROMInternalTranscriptAsNMA.sk_current{1} =
        ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
    RealSigningPaperSimSampler.pk_current{2} =
      ROMInternalTranscriptAsNMA.pk_current{1} /\
    RealSigningPaperSimSampler.sk_current{2} =
      ROMInternalTranscriptAsNMA.sk_current{1} /\
    ROMInternalTranscriptAsNMA.queries{1} =
      ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
    ROMInternalTranscriptAsNMA.transcripts{1} =
      ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
    ROMInternalTranscriptAsNMA.records{1} =
      ROMInternalTranscriptPaperSimAsNMA.records{2}
    ==>
    ={glob H, res} /\
    ROMInternalTranscriptAsNMA.pk_current{1} =
      ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
    public_key_of_secret haetae_mode
      ROMInternalTranscriptAsNMA.sk_current{1} =
        ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
    RealSigningPaperSimSampler.pk_current{2} =
      ROMInternalTranscriptAsNMA.pk_current{1} /\
    RealSigningPaperSimSampler.sk_current{2} =
      ROMInternalTranscriptAsNMA.sk_current{1} /\
    ROMInternalTranscriptAsNMA.queries{1} =
      ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
    ROMInternalTranscriptAsNMA.transcripts{1} =
      ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
    ROMInternalTranscriptAsNMA.records{1} =
      ROMInternalTranscriptPaperSimAsNMA.records{2}.

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*, *rnd*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

ROMInternalTranscriptAsNMA
ROMInternalTranscriptPaperSimAsNMA
RealSigningPaperSimSampler

```

```

arg
glob
haetae_mode
paper_sim_commitment_highbits_real_signing_coinsE
paper_sim_commitment_lowbits_real_signing_coinsE
paper_sim_signature_real_signing_coinsE
pk_current
public_key_of_secret
queries
records
sample
sample_with_seed
sk_current
transcripts

```

**Theorem 3.197** (EasyCrypt line 6354). *EasyCrypt name.*

```
rom_internal_nma_real_signing_paper_sim_exact
```

**Checked EasyCrypt statement.**

```

lemma rom_internal_nma_real_signing_paper_sim_exact &m :
  Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptAsNMA(A)).main()
    @ &m : res] =
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, RealSigningPaperSimSampler(H))).main() @ &m : res].

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byequiv*, *proc*, *inline*, *wp*, *call*, *apply*, *rnd*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

HAETAE
ROMInternalTranscriptAsNMA
ROMInternalTranscriptPaperSimAsNMA
RealSigningPaperSimSampler
adversary_rom_internal_real_signing_paper_sim_nma
arg
forge
glob
haetae_mode
init
keygen_internal
kg
pk_current
public_key_of_secret
queries
records
secret_key_of_seed
sk_current
transcripts
verify

```

**Theorem 3.198** (EasyCrypt line 6427). *EasyCrypt name.*

```
real_signing_ro_attempt_paper_sim_nma_oracle
```

### Checked EasyCrypt statement.

```

equiv real_signing_ro_attempt_paper_sim_nma_oracle :
  ROMInternalTranscriptPaperSimAsNMA
  (A, RealSigningPaperSimSampler(H), H).O.sign ~
  ROMInternalTranscriptPaperSimAsNMA
  (A, ROSigningAttemptPaperSimSampler(H), H).O.sign :
  ={glob H, arg} /\
  ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =
    ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
  RealSigningPaperSimSampler.pk_current{1} =
    ROSigningAttemptPaperSimSampler.pk_current{2} /\
  RealSigningPaperSimSampler.sk_current{1} =
    ROSigningAttemptPaperSimSampler.sk_current{2} /\
  RealSigningPaperSimSampler.pk_current{1} =
    public_key_of_secret haetae_mode
    RealSigningPaperSimSampler.sk_current{1} /\
  ROMInternalTranscriptPaperSimAsNMA.queries{1} =
    ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
  ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =
    ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
  ROMInternalTranscriptPaperSimAsNMA.records{1} =
    ROMInternalTranscriptPaperSimAsNMA.records{2}
  ==>
  ={glob H, res} /\
  ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =
    ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
  RealSigningPaperSimSampler.pk_current{1} =
    ROSigningAttemptPaperSimSampler.pk_current{2} /\
  RealSigningPaperSimSampler.sk_current{1} =
    ROSigningAttemptPaperSimSampler.sk_current{2} /\
  RealSigningPaperSimSampler.pk_current{1} =
    public_key_of_secret haetae_mode
    RealSigningPaperSimSampler.sk_current{1} /\
  ROMInternalTranscriptPaperSimAsNMA.queries{1} =
    ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
  ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =
    ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
  ROMInternalTranscriptPaperSimAsNMA.records{1} =
    ROMInternalTranscriptPaperSimAsNMA.records{2}.

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*, *rnd*, *smt*. Named identifiers syntactically cited in the proof block:

```

ROSigningAttemptPaperSimSampler
RealSigningPaperSimSampler
arg
glob
paper_sim_sample_from_rejection_attempt_of_coinsE
sample
sample_with_seed

```

**Theorem 3.199** (EasyCrypt line 6485). *EasyCrypt name.*

```

adversary_real_signing_ro_attempt_paper_sim_nma

```



### Checked EasyCrypt statement.

```

equiv adversary_real_signing_ro_attempt_paper_sim_nma :
  A(H, ROMInternalTranscriptPaperSimAsNMA
    (A, RealSigningPaperSimSampler(H), H).0).forge ~
  A(H, ROMInternalTranscriptPaperSimAsNMA
    (A, ROSigningAttemptPaperSimSampler(H), H).0).forge :
  ={glob H, glob A, arg} /\
  ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =
    ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
  RealSigningPaperSimSampler.pk_current{1} =
    ROSigningAttemptPaperSimSampler.pk_current{2} /\
  RealSigningPaperSimSampler.sk_current{1} =
    ROSigningAttemptPaperSimSampler.sk_current{2} /\
  RealSigningPaperSimSampler.pk_current{1} =
    public_key_of_secret haetae_mode
    RealSigningPaperSimSampler.sk_current{1} /\
  ROMInternalTranscriptPaperSimAsNMA.queries{1} =
    ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
  ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =
    ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
  ROMInternalTranscriptPaperSimAsNMA.records{1} =
    ROMInternalTranscriptPaperSimAsNMA.records{2}
  ==>
  ={glob H, res} /\
  ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =
    ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
  RealSigningPaperSimSampler.pk_current{1} =
    ROSigningAttemptPaperSimSampler.pk_current{2} /\
  RealSigningPaperSimSampler.sk_current{1} =
    ROSigningAttemptPaperSimSampler.sk_current{2} /\
  RealSigningPaperSimSampler.pk_current{1} =
    public_key_of_secret haetae_mode
    RealSigningPaperSimSampler.sk_current{1} /\
  ROMInternalTranscriptPaperSimAsNMA.queries{1} =
    ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
  ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =
    ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
  ROMInternalTranscriptPaperSimAsNMA.records{1} =
    ROMInternalTranscriptPaperSimAsNMA.records{2}.

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *call*. Named identifiers syntactically cited in the proof block:

```

ROMInternalTranscriptPaperSimAsNMA
ROSigningAttemptPaperSimSampler
RealSigningPaperSimSampler
glob
haetae_mode
pk_current
public_key_of_secret
queries
real_signing_ro_attempt_paper_sim_nma_oracle
records
sk_current
transcripts

```

**Theorem 3.200** (EasyCrypt line 6548). *EasyCrypt name.*

```
rom_internal_nma_real_signing_ro_attempt_paper_sim_exact
```

*Checked EasyCrypt statement.*

```
lemma rom_internal_nma_real_signing_ro_attempt_paper_sim_exact &m :
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, RealSigningPaperSimSampler(H))).main() @ &m : res] =
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res].
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byequiv*, *proc*, *inline*, *wp*, *call*, *apply*, *rnd*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
HAETAE
ROMInternalTranscriptPaperSimAsNMA
ROSigningAttemptPaperSimSampler
RealSigningPaperSimSampler
adversary_real_signing_ro_attempt_paper_sim_nma
arg
forge
glob
haetae_mode
init
keygen_internal
kg
pk_current
public_key_of_secret
queries
records
secret_key_of_seed
sk_current
transcripts
verify
```

**Theorem 3.201** (EasyCrypt line 6625). *EasyCrypt name.*

```
concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_fresh_loss
```

*Checked EasyCrypt statement.*

```
lemma concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_fresh_loss
  seed_coins m ctx (p : paper_sim_signature_sample -> bool) &m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m{m} =>
  ROSigningAttemptPaperSimSampler.sk_current{m} =
  ROExactHyperballPaperSimSampler.sk_current{m} =>
  Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
    (seed_coins, m, ctx) @ &m : p res] <=
  Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
    (seed_coins, m, ctx) @ &m : p res] +
  rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *byphoare*, *smt*. Named identifiers syntactically cited in the proof block:

```

FRO
HAETAЕ_RO
Pr
ROSigningAttemptPaperSimSampler
concrete_ro_exact_hyperball_sample_with_seed_exact_pr
concrete_ro_signing_attempt_sample_with_seed_fresh_structural_le
ctx
dsigning_attempt_state
exact_eq
fresh
haetae_mode
mu
paper_sim_sample_from_rejection_attempt
sample_loss
sample_step
sample_with_seed
seed_coins
signing_le
sk_current
sk_eq
st

```

**Theorem 3.202** (EasyCrypt line 6655). *EasyCrypt name*.

```
concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_clean_loss
```

**Checked EasyCrypt statement.**

```

lemma concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_clean_loss
  seed_coins m ctx (p : paper_sim_signature_sample -> bool)
  hash_qs sampler_qs bad &m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  sampler_rom_covered HAETAЕ_RO.FRO.m{m} hash_qs sampler_qs =>
  ! (bad \ /
    sampler_expand_query seed_coins \in hash_qs \ /
    sampler_expand_query seed_coins \in sampler_qs) =>
  ROSigningAttemptPaperSimSampler.sk_current{m} =
  ROExactHyperballPaperSimSampler.sk_current{m} =>
  Pr[ROSigningAttemptPaperSimSampler(HAETAЕ_RO.FRO).sample_with_seed
    (seed_coins, m, ctx) @ &m : p res] <=
  Pr[ROExactHyperballPaperSimSampler(HAETAЕ_RO.FRO).sample_with_seed
    (seed_coins, m, ctx) @ &m : p res] +
  rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem

into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
FRO
HAETAE_RO
bad
clean
concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_fresh_loss
covered
ctx
hash_qs
sample_loss
sampler_qs
sampler_rom_covered_fresh_after_clean_seed
seed_coins
sk_eq
```

**Theorem 3.203** (EasyCrypt line 6678). *EasyCrypt name*.

```
concrete_budgeted_o_sign_sample_with_seed_clean_loss_surface
```

*Checked EasyCrypt statement.*

```
lemma concrete_budgeted_o_sign_sample_with_seed_clean_loss_surface
  seed_coins m ctx (p : paper_sim_signature_sample -> bool) &m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  sampler_rom_covered
  HAETAE_RO.FRO.m{m}
  ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}
  ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m} =>
  ! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} \ /
    sampler_expand_query seed_coins \in
    ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} \ /
    sampler_expand_query seed_coins \in
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}) =>
  ROSigningAttemptPaperSimSampler.sk_current{m} =
  ROExactHyperballPaperSimSampler.sk_current{m} =>
  Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
    (seed_coins, m, ctx) @ &m : p res] <=
  Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
    (seed_coins, m, ctx) @ &m : p res] +
  rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
ROMInternalTranscriptBudgetedPaperSimAsNMA
adversary_hash_queries
clean
concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_clean_loss
covered
ctx
sample_loss
sampler_bad_prequery
```

```
sampler_expand_queries
seed_coins
sk_eq
```

**Theorem 3.204** (EasyCrypt line 6708). *EasyCrypt name.*

```
concrete_budgeted_o_sign_old_log_sample_with_seed_clean_loss_surface
```

*Checked EasyCrypt statement.*

```
lemma concrete_budgeted_o_sign_old_log_sample_with_seed_clean_loss_surface
  seed_coins m ctx (p : paper_sim_signature_sample -> bool)
  old_hash_qs old_sampler_qs old_bad &m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  old_hash_qs =
  ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} =>
  old_sampler_qs =
  ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m} =>
  old_bad =
  ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} =>
  sampler_rom_covered HAETAE_RO.FRO.m{m} old_hash_qs old_sampler_qs =>
  ! (old_bad \ /
    sampler_expand_query seed_coins \in old_hash_qs \ /
    sampler_expand_query seed_coins \in old_sampler_qs) =>
  ROSigningAttemptPaperSimSampler.sk_current{m} =
  ROExactHyperballPaperSimSampler.sk_current{m} =>
  Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
    (seed_coins, m, ctx) @ &m : p res] <=
  Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
    (seed_coins, m, ctx) @ &m : p res] +
  rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **apply**. Named identifiers syntactically cited in the proof block:

```
clean
concrete_budgeted_o_sign_sample_with_seed_clean_loss_surface
covered
ctx
old_bad
old_bad_eq
old_hash_eq
old_hash_qs
old_sampler_eq
old_sampler_qs
sample_loss
seed_coins
sk_eq
subst
```

**Theorem 3.205** (EasyCrypt line 6740). *EasyCrypt name.*

```
concrete_budgeted_o_sign_old_log_signature_from_sample_clean_loss_surface
```

*Checked EasyCrypt statement.*

```

lemma concrete_budgeted_o_sign_old_log_signature_from_sample_clean_loss_surface
  seed_coins m ctx (p : signature -> bool)
  old_hash_qs old_sampler_qs old_bad &m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  old_hash_qs =
    ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} =>
  old_sampler_qs =
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m} =>
  old_bad =
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} =>
  sampler_rom_covered HAETAE_RO.FRO.m{m} old_hash_qs old_sampler_qs =>
  ! (old_bad \ /
    sampler_expand_query seed_coins \in old_hash_qs \ /
    sampler_expand_query seed_coins \in old_sampler_qs) =>
  ROSigningAttemptPaperSimSampler.sk_current{m} =
    ROExactHyperballPaperSimSampler.sk_current{m} =>
  Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
    (seed_coins, m, ctx) @ &m :
    p (paper_sim_signature haetae_mode
      ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current{m}
      m ctx res)] <=
  Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
    (seed_coins, m, ctx) @ &m :
    p (paper_sim_signature haetae_mode
      ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current{m}
      m ctx res)] +
  rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```

ROMInternalTranscriptBudgetedPaperSimAsNMA
clean
concrete_budgeted_o_sign_old_log_sample_with_seed_clean_loss_surface
covered
ctx
haetae_mode
old_bad
old_bad_eq
old_hash_eq
old_hash_qs
old_sampler_eq
old_sampler_qs
paper_sim_signature
pk_current
sample_loss
seed_coins
sk_eq
smp

```

**Theorem 3.206** (EasyCrypt line 6779). *EasyCrypt name.*

```

concrete_budgeted_o_sign_old_log_signature_clean_event_loss_surface

```

**Checked EasyCrypt statement.**

```

lemma concrete_budgeted_o_sign_old_log_signature_clean_event_loss_surface
  seed_coins m ctx (p : signature -> bool)
  old_hash_qs old_sampler_qs old_bad &m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  old_hash_qs =
    ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} =>
  old_sampler_qs =
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m} =>
  old_bad =
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} =>
  sampler_rom_covered HAETAE_RO.FRO.m{m} old_hash_qs old_sampler_qs =>
  ! (old_bad \ /
    sampler_expand_query seed_coins \in old_hash_qs \ /
    sampler_expand_query seed_coins \in old_sampler_qs) =>
  ROSigningAttemptPaperSimSampler.sk_current{m} =
    ROExactHyperballPaperSimSampler.sk_current{m} =>
  Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
    (seed_coins, m, ctx) @ &m :
    p (paper_sim_signature haetae_mode
      ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current{m}
      m ctx res) /\
    ! (old_bad \ /
      sampler_expand_query seed_coins \in old_hash_qs \ /
      sampler_expand_query seed_coins \in old_sampler_qs)] <=
  Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
    (seed_coins, m, ctx) @ &m :
    p (paper_sim_signature haetae_mode
      ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current{m}
      m ctx res)] +
  rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

clean
concrete_budgeted_o_sign_old_log_signature_from_sample_clean_loss_surface
covered
ctx
old_bad
old_bad_eq
old_hash_eq
old_hash_qs
old_sampler_eq
old_sampler_qs
sample_loss
seed_coins
sk_eq

```

**Theorem 3.207** (EasyCrypt line 6818). *EasyCrypt name.*

```
concrete_frozen_old_log_sample_clean_event_loss_surface
```

**Checked EasyCrypt statement.**

```
lemma concrete_frozen_old_log_sample_clean_event_loss_surface
```

```

seed_coins m ctx (p : paper_sim_signature_sample -> bool)
old_hash_qs old_sampler_qs old_bad &m :
structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
sampler_rom_covered HAETAE_RO.FRO.m{m} old_hash_qs old_sampler_qs =>
! (old_bad \ /
  sampler_expand_query seed_coins \in old_hash_qs \ /
  sampler_expand_query seed_coins \in old_sampler_qs) =>
ROSigningAttemptPaperSimSampler.sk_current{m} =
ROExactHyperballPaperSimSampler.sk_current{m} =>
Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
  (seed_coins, m, ctx) @ &m :
  p res \ /
  ! (old_bad \ /
    sampler_expand_query seed_coins \in old_hash_qs \ /
    sampler_expand_query seed_coins \in old_sampler_qs)] <=
Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
  (seed_coins, m, ctx) @ &m : p res] +
rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

clean
concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_clean_loss
covered
ctx
old_bad
old_hash_qs
old_sampler_qs
sample_loss
seed_coins
sk_eq

```

**Theorem 3.208** (EasyCrypt line 6845). *EasyCrypt* name.

```

concrete_frozen_old_log_signature_clean_event_loss_surface

```

**Checked EasyCrypt statement.**

```

lemma concrete_frozen_old_log_signature_clean_event_loss_surface
  seed_coins m ctx pk (p : signature -> bool)
  old_hash_qs old_sampler_qs old_bad &m :
structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
sampler_rom_covered HAETAE_RO.FRO.m{m} old_hash_qs old_sampler_qs =>
! (old_bad \ /
  sampler_expand_query seed_coins \in old_hash_qs \ /
  sampler_expand_query seed_coins \in old_sampler_qs) =>
ROSigningAttemptPaperSimSampler.sk_current{m} =
ROExactHyperballPaperSimSampler.sk_current{m} =>
Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
  (seed_coins, m, ctx) @ &m :
  p (paper_sim_signature haetae_mode pk m ctx res) \ /
  ! (old_bad \ /
    sampler_expand_query seed_coins \in old_hash_qs \ /
    sampler_expand_query seed_coins \in old_sampler_qs)] <=
Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed

```



```

(seed_coins, m, ctx) @ &m :
  p (paper_sim_signature haetae_mode pk m ctx res)] +
  rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **rewrite**, **apply**. Named identifiers syntactically cited in the proof block:

```

clean
concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_clean_loss
covered
ctx
haetae_mode
old_bad
old_hash_qs
old_sampler_qs
paper_sim_signature
pk
sample_loss
seed_coins
sk_eq
smp

```

**Theorem 3.209** (EasyCrypt line 6875). *EasyCrypt name.*

```

concrete_frozen_old_log_self_logged_signature_clean_event_loss_surface

```

**Checked EasyCrypt statement.**

```

lemma concrete_frozen_old_log_self_logged_signature_clean_event_loss_surface
  seed_coins m ctx pk (p : signature -> bool)
  old_hash_qs old_sampler_qs old_bad &m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  sampler_rom_covered HAETAERO.FRO.m{m} old_hash_qs old_sampler_qs =>
  ! (old_bad /\
    sampler_expand_query seed_coins \in old_hash_qs /\
    sampler_expand_query seed_coins \in old_sampler_qs) =>
  ROSigningAttemptPaperSimSampler.sk_current{m} =
  ROExactHyperballPaperSimSampler.sk_current{m} =>
  sampler_expand_query seed_coins \notin HAETAERO.FRO.m{m} /\
  sampler_rom_covered HAETAERO.FRO.m{m} old_hash_qs
  (sampler_expand_query seed_coins :: old_sampler_qs) /\
  Pr[ROSigningAttemptPaperSimSampler(HAETAERO.FRO).sample_with_seed
    (seed_coins, m, ctx) @ &m :
    p (paper_sim_signature haetae_mode pk m ctx res) /\
    ! (old_bad /\
      sampler_expand_query seed_coins \in old_hash_qs /\
      sampler_expand_query seed_coins \in old_sampler_qs)] <=
  Pr[ROExactHyperballPaperSimSampler(HAETAERO.FRO).sample_with_seed
    (seed_coins, m, ctx) @ &m :
    p (paper_sim_signature haetae_mode pk m ctx res)] +
  rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *split*, *smt*, *apply*. Named identifiers syntactically cited in the proof block:

```
FRO
HAETAE_RO
boundary
clean
concrete_frozen_old_log_signature_clean_event_loss_surface
covered
ctx
old_bad
old_hash_qs
old_sampler_qs
pk
sample_loss
sampler_rom_covered_old_log_clean_boundary
seed_coins
sk_eq
```

**Theorem 3.210** (EasyCrypt line 6913). *EasyCrypt name.*

```
concrete_budgeted_self_logged_signature_clean_event_loss_surface
```

**Checked EasyCrypt statement.**

```
lemma concrete_budgeted_self_logged_signature_clean_event_loss_surface
  seed_coins m ctx pk (p : signature -> bool) &m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  sampler_rom_covered
  HAETAE_RO.FRO.m{m}
  ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}
  ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m} =>
  ! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} \ /
    sampler_expand_query seed_coins \in
    ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} \ /
    sampler_expand_query seed_coins \in
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}) =>
  ROSigningAttemptPaperSimSampler.sk_current{m} =
  ROExactHyperballPaperSimSampler.sk_current{m} =>
  sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m{m} /\
  sampler_rom_covered
  HAETAE_RO.FRO.m{m}
  ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}
  (sampler_expand_query seed_coins ::
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}) /\
  Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
    (seed_coins, m, ctx) @ &m :
    p (paper_sim_signature haetae_mode pk m ctx res) /\
    ! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} \ /
      sampler_expand_query seed_coins \in
      ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} \ /
      sampler_expand_query seed_coins \in
      ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m})] <=
  Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
    (seed_coins, m, ctx) @ &m :
    p (paper_sim_signature haetae_mode pk m ctx res)] +
  rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage

inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
ROMInternalTranscriptBudgetedPaperSimAsNMA
adversary_hash_queries
clean
concrete_frozen_old_log_self_logged_signature_clean_event_loss_surface
covered
ctx
pk
sample_loss
sampler_bad_prequery
sampler_expand_queries
seed_coins
sk_eq
```

**Theorem 3.211** (EasyCrypt line 6956). *EasyCrypt name.*

```
concrete_lazy_rom_get_lossless
```

**Checked EasyCrypt statement.**

```
lemma concrete_lazy_rom_get_lossless :
  islossless HAETAE_RO.FRO.get.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *smt*. Named identifiers syntactically cited in the proof block:

```
ro_output_distribution_lossless
```

**Theorem 3.212** (EasyCrypt line 6965). *EasyCrypt name.*

```
structural_attempt_exact_hyperball_paper_sample_one_step_loss
```

**Checked EasyCrypt statement.**

```
lemma structural_attempt_exact_hyperball_paper_sample_one_step_loss
  sk m ctx (p : paper_sim_signature_sample -> bool) :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  phoare[StructuralAttemptPaperSample.sample :
    arg = (sk, m, ctx) ==> p res] <=
  (mu (dexact_hyperball_signing_attempt_state haetae_mode sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st)) +
    rejection_sampling_loss_term).
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `byphoare`, `proc`, `wp`, `rnd`, `apply`, `rewrite`. Named identifiers syntactically cited in the proof block:

```
arg
argE
bypr
ctx
m0
sample_loss
sk
```

**Theorem 3.213** (EasyCrypt line 6986). *EasyCrypt name.*

```
exact_hyperball_paper_sample_one_step_upper
```

**Checked EasyCrypt statement.**

```
lemma exact_hyperball_paper_sample_one_step_upper
  sk m ctx (p : paper_sim_signature_sample -> bool) :
  phoare[ExactHyperballPaperSample.sample :
    arg = (sk, m, ctx) ==> p res] <=
  (mu (dexact_hyperball_signing_attempt_state haetae_mode sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st))).
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `byphoare`, `proc`, `wp`, `rnd`, `rewrite`. Named identifiers syntactically cited in the proof block:

```
arg
argE
bypr
ctx
m0
sk
```

**Theorem 3.214** (EasyCrypt line 7016). *EasyCrypt name.*

```
rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting
```

**Checked EasyCrypt statement.**

```
lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting &m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
```

```

    rejection_sampling_loss_term =>
Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=
Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
    rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Named identifiers syntactically cited in the proof block:

```
lifted
```

**Theorem 3.215** (EasyCrypt line 7036). *EasyCrypt name.*

```
rom_internal_budgeted_ro_signing_attempt_signing_call_loss_bound
```

**Checked EasyCrypt statement.**

```

lemma rom_internal_budgeted_ro_signing_attempt_signing_call_loss_bound &m :
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :
    0 < ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count] <=
    rom_signature_query_budget * rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *have*, *smt*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

HAETAE
Pr
ROMInternalTranscriptBudgetedPaperSimAsNMA
ROSigningAttemptPaperSimSampler
SIG
UF_NMA
budgeted_loss_ge1
call_event_le1
main
mu_bounded
rejection_sampling_loss_term
rejection_sampling_loss_term_ge1
rom_signature_query_budget
signature_query_budget_count
signature_query_count
signing_count

```

**Theorem 3.216** (EasyCrypt line 7057). *EasyCrypt name.*

```
rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting
```

*Checked EasyCrypt statement.*

```
lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting &m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
  rom_signature_query_budget * rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *apply*, *smt*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
HAETAETAE
Pr
ROExactHyperballPaperSimSampler
ROMInternalTranscriptBudgetedPaperSimAsNMA
ROSigningAttemptPaperSimSampler
SIG
StructuralAttemptPaperSample
UF_NMA
budgeted_loss_ge1
checked_one_step
context
ctx0
left_le1
m0
main
message
mu_bounded
p0
paper_sim_signature_sample
phoare
rejection_sampling_loss_term
rejection_sampling_loss_term_ge1
right_ge0
rom_signature_query_budget
sample
sample_loss
signature_query_budget_count
signature_query_count
sk0
skey
structural_attempt_exact_hyperball_paper_sample_one_step_loss
```

**Theorem 3.217** (EasyCrypt line 7097). *EasyCrypt name.*

```
rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting
```

*Checked EasyCrypt statement.*

```

lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting &m :
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :
    res /\
    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
    rom_signature_query_budget * rejection_sampling_loss_term =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
    rom_signature_query_budget * rejection_sampling_loss_term +
    rom_signature_query_budget *
      ((rom_hash_query_budget + rom_signature_query_budget) /
        challenge_support_cardinality_lower_bound).

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\text{Pr}[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

HAETAE
Pr
ROExactHyperballPaperSimSampler
ROMInternalTranscriptBudgetedPaperSimAsNMA
ROSigningAttemptPaperSimSampler
SIG
UF_NMA
budgeted_paper_sim_sampler_bad_prequery_nma_fel_bound
clean_lifting
ler_add
ler_trans
lerr
main
mu_or
mu_sub
rejection_sampling_loss_term
rom_signature_query_budget
sampler_bad_prequery

```

**Theorem 3.218** (EasyCrypt line 7154). *EasyCrypt* name.

```
rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting
```

**Checked EasyCrypt statement.**

```

lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting &m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :
    res /\

```

```

! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=
Pr[SIG.UF_NMA(H, HAETAETAE,
  ROMInternalTranscriptBudgetedPaperSimAsNMA
    (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
rom_signature_query_budget * rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *smt*. Named identifiers syntactically cited in the proof block:

```

HAETAETAE
Pr
ROMInternalTranscriptBudgetedPaperSimAsNMA
ROSigningAttemptPaperSimSampler
SIG
UF_NMA
budgeted_lifting
clean_restrict
main
mu_sub
rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting
sample_loss
sampler_bad_prequery

```

**Theorem 3.219** (EasyCrypt line 7183). *EasyCrypt name*.

```

rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_checked_clean_lifting

```

**Checked EasyCrypt statement.**

```

lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_checked_clean_lifting &m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
  rom_signature_query_budget * rejection_sampling_loss_term +
  rom_signature_query_budget *
    ((rom_hash_query_budget + rom_signature_query_budget) /
      challenge_support_cardinality_lower_bound).

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:



```
rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting
rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting
sample_loss
```

**Theorem 3.220** (EasyCrypt line 7205). *EasyCrypt name.*

```
rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_lifting
```

**Checked EasyCrypt statement.**

```
lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_lifting &m :
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] =>
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
  rom_signature_query_budget * rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
HAETAE
Pr
ROExactHyperballPaperSimSampler
ROMInternalTranscriptBudgetedPaperSimAsNMA
ROSigningAttemptPaperSimSampler
SIG
UF_NMA
attempt_unbudgeted_to_budgeted
exact_budgeted_to_unbudgeted
hbudgeted
main
rejection_sampling_loss_term
rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting
rom_signature_query_budget
sample_loss
```

**Theorem 3.221** (EasyCrypt line 7243). *EasyCrypt name.*

```
rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_checked_clean_lifting
```

**Checked EasyCrypt statement.**

```

lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_checked_clean_lifting  $\mathcal{E}m$  :
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @  $\mathcal{E}m$  : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @  $\mathcal{E}m$  : res] =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @  $\mathcal{E}m$  : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @  $\mathcal{E}m$  : res] =>
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(H))).main() @  $\mathcal{E}m$  : res] <=
  Pr[SIG.UF_NMA(H, HAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(H))).main() @  $\mathcal{E}m$  : res] +
  rom_signature_query_budget * rejection_sampling_loss_term +
  rom_signature_query_budget *
  ((rom_hash_query_budget + rom_signature_query_budget) /
    challenge_support_cardinality_lower_bound).

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```

HAETAE
Pr
ROExactHyperballPaperSimSampler
ROMInternalTranscriptBudgetedPaperSimAsNMA
ROSigningAttemptPaperSimSampler
SIG
UF_NMA
attempt_unbudgeted_to_budgeted
challenge_support_cardinality_lower_bound
exact_budgeted_to_unbudgeted
hbudgeted
main
rejection_sampling_loss_term
rom_hash_query_budget
rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_checked_clean_lifting
rom_signature_query_budget
sample_loss

```

**Theorem 3.222** (EasyCrypt line 7291). *EasyCrypt name.*

```
rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_bound
```

**Checked EasyCrypt statement.**

```

lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_bound  $\mathcal{E}m$  :
  Pr[SIG.UF_NMA(H, HAETAE,

```

```

ROMInternalTranscriptPaperSimAsNMA
  (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=
Pr[SIG.UF_NMA(H, HAETAE,
  ROMInternalTranscriptPaperSimAsNMA
    (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *smt*, *apply*. Named identifiers syntactically cited in the proof block:

```

HAETAE
Pr
ROExactHyperballPaperSimSampler
ROMInternalTranscriptPaperSimAsNMA
ROSigningAttemptPaperSimSampler
SIG
UF_NMA
left_le1
loss_ge1
main
mu_bounded
rejection_sampling_loss_term
rejection_sampling_loss_term_ge1
right_ge0

```

**Theorem 3.223** (EasyCrypt line 7326). *EasyCrypt name.*

```
concrete_lazy_rom_get_preserves_signature_event
```

**Checked EasyCrypt statement.**

```

lemma concrete_lazy_rom_get_preserves_signature_event
  pk m ctx smp (p : signature -> bool) :
  phoare[HAETAE_RO.FRO.get :
    p (paper_sim_signature haetae_mode pk m ctx smp) ==>
    p (paper_sim_signature haetae_mode pk m ctx smp)] = 1%r.

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *smt*. Named identifiers syntactically cited in the proof block:

```
ro_output_distribution_lossless
```

**Theorem 3.224** (EasyCrypt line 7338). *EasyCrypt name.*

```
concrete_lazy_rom_get_preserves_signature_event_with_pk
```

### Checked EasyCrypt statement.

```
lemma concrete_lazy_rom_get_preserves_signature_event_with_pk
  pk m ctx smp (p : signature -> bool) :
  phoare[HAETAE_RO.FRO.get :
    ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\
    p (paper_sim_signature haetae_mode pk m ctx smp) ==>
    ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\
    p (paper_sim_signature haetae_mode pk m ctx smp)] = 1%r.
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *smt*. Named identifiers syntactically cited in the proof block:

```
ro_output_distribution_lossless
```

### Theorem 3.225 (EasyCrypt line 7352). EasyCrypt name.

```
concrete_lazy_rom_get_preserves_current_signature_event
```

### Checked EasyCrypt statement.

```
lemma concrete_lazy_rom_get_preserves_current_signature_event
  m ctx smp (p : signature -> bool) :
  phoare[HAETAE_RO.FRO.get :
    p (paper_sim_signature haetae_mode
      ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current m ctx smp) ==>
    p (paper_sim_signature haetae_mode
      ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current m ctx smp)] = 1%r.
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rnd*, *smt*. Named identifiers syntactically cited in the proof block:

```
ro_output_distribution_lossless
```

### Theorem 3.226 (EasyCrypt line 7366). EasyCrypt name.

```
concrete_lazy_rom_get_true
```

### Checked EasyCrypt statement.

```
lemma concrete_lazy_rom_get_true :
  phoare[HAETAE_RO.FRO.get : true ==> true] = 1%r.
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `conseq`. Named identifiers syntactically cited in the proof block:

```
concrete_lazy_rom_get_lossless
```

**Theorem 3.227** (EasyCrypt line 7372). *EasyCrypt name.*

```
concrete_lazy_rom_get_preserves_budgeted_clean_signature_event
```

**Checked EasyCrypt statement.**

```
lemma concrete_lazy_rom_get_preserves_budgeted_clean_signature_event
  pk m ctx smp (p : signature -> bool) :
  phoare[HAETAE_RO.FRO.get :
    ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\
    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery /\
    p (paper_sim_signature haetae_mode pk m ctx smp) ==>
    ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\
    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery /\
    p (paper_sim_signature haetae_mode pk m ctx smp)] = 1%r.
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `wp`, `rnd`, `smt`. Named identifiers syntactically cited in the proof block:

```
ro_output_distribution_lossless
```

**Theorem 3.228** (EasyCrypt line 7388). *EasyCrypt name.*

```
concrete_lazy_rom_get_preserves_budgeted_clean_signature_event_hoare
```

**Checked EasyCrypt statement.**

```
lemma concrete_lazy_rom_get_preserves_budgeted_clean_signature_event_hoare
  pk m ctx smp (p : signature -> bool) :
  hoare[HAETAE_RO.FRO.get :
    ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\
    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery /\
    p (paper_sim_signature haetae_mode pk m ctx smp) ==>
    ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\
    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery /\
    p (paper_sim_signature haetae_mode pk m ctx smp)].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `wp`, `rnd`, `smt`. Named identifiers syntactically cited in the proof block:

```
ro_output_distribution_lossless
```

**Theorem 3.229** (EasyCrypt line 7404). *EasyCrypt name.*

```
concrete_ro_signing_attempt_sample_with_seed_lossless
```

**Checked EasyCrypt statement.**

```
lemma concrete_ro_signing_attempt_sample_with_seed_lossless :
  islossless ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `wp`, `call`. Named identifiers syntactically cited in the proof block:

```
concrete_lazy_rom_get_lossless
```

**Theorem 3.230** (EasyCrypt line 7413). *EasyCrypt name.*

```
concrete_budgeted_ro_signing_attempt_o_sign_active_clean_signature_structural_le
```

**Checked EasyCrypt statement.**

```
lemma concrete_budgeted_ro_signing_attempt_o_sign_active_clean_signature_structural_le
  pk sk msg ctxt (p : signature -> bool) :
  phoare [ROMInternalTranscriptBudgetedPaperSimAsNMA
    (A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO),
      HAETAE_RO.FRO).O.sign :
    arg = (msg, ctxt) /\
    ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count <
      signature_query_budget_count /\
    ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\
    ROSigningAttemptPaperSimSampler.sk_current = sk /\
    sampler_rom_covered
    HAETAE_RO.FRO.m
    ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries /\
    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery ==>
    p res /\
    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=
  (mu (dsigning_attempt_state haetae_mode sk msg ctxt)
    (fun st =>
      p (paper_sim_signature haetae_mode pk msg ctxt
```

```
(paper_sim_sample_from_rejection_attempt st))))).
```

**Formal mathematical reading.** *Mathematical theorem.* This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *rnd*, *smt*, *inline*, *hoare*. Named identifiers syntactically cited in the proof block:

```
FRO
HAETAE_RO
ROMInternalTranscriptBudgetedPaperSimAsNMA
ROSigningAttemptPaperSimSampler
adversary_hash_queries
concrete_ro_signing_attempt_sample_with_seed_guarded_clean_structural_arg_le
concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs
ctx
ctxt
dsigning_attempt_state
get
haetae_mode
msg
mu
mu_bounded
paper_sim_sample_from_rejection_attempt
paper_sim_signature
pk
pk_current
ro_output_distribution_lossless
sampler_bad_prequery
sampler_expand_queries
sampler_rom_covered
sampler_rom_covered_fresh_after_clean_seed
sampler_rom_covered_self_log_preserves
signing_coin_distribution_lossless
sk
sk_current
smp
st
```

**Theorem 3.231** (EasyCrypt line 7487). *EasyCrypt name.*

```
concrete_budgeted_ro_exact_hyperball_o_sign_active_signature_exact
```

**Checked EasyCrypt statement.**

```
lemma concrete_budgeted_ro_exact_hyperball_o_sign_active_signature_exact
  pk sk msg ctxt (p : signature -> bool) :
  phoare [ROMInternalTranscriptBudgetedPaperSimAsNMA
    (A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO),
    HAETAE_RO.FRO).O.sign :
    arg = (msg, ctxt) /\
    ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count <
    signature_query_budget_count /\
    ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\
    ROExactHyperballPaperSimSampler.sk_current = sk ==>
    p res] =
  (mu (dexact_hyperball_signing_attempt_state haetae_mode sk msg ctxt)
    (fun st =>
      p (paper_sim_signature haetae_mode pk msg ctxt
```

```
(paper_sim_sample_from_rejection_attempt st))).
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*, *smt*, *inline*, *rnd*. Named identifiers syntactically cited in the proof block:

```
FRO
HAETAE_RO
ROExactHyperballPaperSimSampler
ROMInternalTranscriptBudgetedPaperSimAsNMA
concrete_lazy_rom_get_true
concrete_ro_exact_hyperball_sample_with_seed_exact_no_seed
ctx
ctxt
dexact_hyperball_signing_attempt_state
get
haetae_mode
msg
mu
mu_bounded
paper_sim_sample_from_rejection_attempt
paper_sim_signature
pk
pk_current
ro_output_distribution_lossless
signing_coin_distribution_lossless
sk
sk_current
smp
st
```

**Theorem 3.232** (EasyCrypt line 7541). *EasyCrypt name.*

```
concrete_budgeted_ro_exact_hyperball_sample_with_seed_to_o_sign_active_projection
```

**Checked EasyCrypt statement.**

```
lemma concrete_budgeted_ro_exact_hyperball_sample_with_seed_to_o_sign_active_projection
  seed_coins pk sk msg ctxt (p : signature -> bool) &M :
  ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count{m} <
  signature_query_budget_count =>
  ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current{m} = pk =>
  ROExactHyperballPaperSimSampler.sk_current{m} = sk =>
  Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
    (seed_coins, msg, ctxt) @ &M :
    p (paper_sim_signature haetae_mode pk msg ctxt res)] <=
  Pr[ROMInternalTranscriptBudgetedPaperSimAsNMA
    (A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO),
    HAETAE_RO.FRO).O.sign(msg, ctxt) @ &M :
    p res].
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event



bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*, *byphoare*, *smt*. Named identifiers syntactically cited in the proof block:

```
FRO
HAETAE_RO
Pr
ROExactHyperballPaperSimSampler
ROMInternalTranscriptBudgetedPaperSimAsNMA
active
concrete_budgeted_ro_exact_hyperball_o_sign_active_signature_exact
concrete_ro_exact_hyperball_sample_with_seed_exact_pr
ctxt
dexact_hyperball_signing_attempt_state
haetae_mode
msg
mu
osign_eq
paper_sim_sample_from_rejection_attempt
paper_sim_signature
pk
pk_eq
sample_eq
sample_with_seed
seed_coins
sign
sk
sk_eq
smp
st
```

**Theorem 3.233** (EasyCrypt line 7585). *EasyCrypt* name.

```
concrete_budgeted_ro_signing_attempt_o_sign_to_self_logged_sample_active_projection
```

**Checked EasyCrypt statement.**

```
lemma concrete_budgeted_ro_signing_attempt_o_sign_to_self_logged_sample_active_projection
  seed_coins pk sk msg ctxt (p : signature -> bool) &m :
  sampler_rom_covered
  HAETAE_RO.FRO.m{m}
  ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}
  ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m} =>
  ! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} \/\
    sampler_expand_query seed_coins \in
      ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} \/\
      sampler_expand_query seed_coins \in
        ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}) =>
  ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count{m} <
    signature_query_budget_count =>
  ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current{m} = pk =>
  ROSigningAttemptPaperSimSampler.sk_current{m} = sk =>
  Pr[ROMInternalTranscriptBudgetedPaperSimAsNMA
    (A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO),
      HAETAE_RO.FRO).O.sign(msg, ctxt) @ &m :
    p res /\
    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=
  Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
    (seed_coins, msg, ctxt) @ &m :
    p (paper_sim_signature haetae_mode pk msg ctxt res) /\
    ! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} \/\
      sampler_expand_query seed_coins \in
        ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} \/\
```

```
sampler_expand_query seed_coins \in
ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m})).
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *byphoare*, *smt*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
FRO
HAETAE_RO
Pr
ROMInternalTranscriptBudgetedPaperSimAsNMA
ROSigningAttemptPaperSimSampler
active
adversary_hash_queries
clean
clean_eq
concrete_budgeted_ro_signing_attempt_o_sign_active_clean_signature_structural_le
concrete_ro_signing_attempt_sample_with_seed_fresh_structural_eq
covered
ctxt
dsigning_attempt_state
haetae_mode
msg
mu
mu_eq
osign_le
paper_sim_sample_from_rejection_attempt
paper_sim_signature
pk
pk_eq
sample_eq
sample_with_seed
sampler_bad_prequery
sampler_expand_queries
sampler_expand_query
sampler_rom_covered_fresh_after_clean_seed
seed_coins
sign
sk
sk_eq
smp
st
```

**Theorem 3.234** (EasyCrypt line 7673). *EasyCrypt name.*

```
concrete_budgeted_o_sign_clean_one_call_loss_from_self_logged_surface
```

**Checked EasyCrypt statement.**

```
lemma concrete_budgeted_o_sign_clean_one_call_loss_from_self_logged_surface
  seed_coins m ctx pk (p : signature -> bool) & m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  sampler_rom_covered
  HAETAE_RO.FRO.m{m}
  ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}
  ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m} =>
  ! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} \ /
```

```

sampler_expand_query seed_coins \in
  ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} \/\
sampler_expand_query seed_coins \in
  ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}) =>
ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count{m} <
signature_query_budget_count =>
ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current{m} = pk =>
ROSigningAttemptPaperSimSampler.sk_current{m} =
ROExactHyperballPaperSimSampler.sk_current{m} =>
Pr[ROMInternalTranscriptBudgetedPaperSimAsNMA
  (A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO),
  HAETAE_RO.FRO).O.sign(m, ct $\alpha$ ) @  $\mathcal{E}_m$  :
  p res /\
  ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=
Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed
  (seed_coins, m, ct $\alpha$ ) @  $\mathcal{E}_m$  :
  p (paper_sim_signature haetae_mode pk m ct $\alpha$  res) /\
  ! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} \/\
  sampler_expand_query seed_coins \in
    ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} \/\
  sampler_expand_query seed_coins \in
    ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m})]] =>
Pr[ROMInternalTranscriptBudgetedPaperSimAsNMA
  (A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO),
  HAETAE_RO.FRO).O.sign(m, ct $\alpha$ ) @  $\mathcal{E}_m$  :
  p res /\
  ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=
Pr[ROMInternalTranscriptBudgetedPaperSimAsNMA
  (A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO),
  HAETAE_RO.FRO).O.sign(m, ct $\alpha$ ) @  $\mathcal{E}_m$  :
  p res] +
rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```

FRO
HAETAE_RO
Pr
ROExactHyperballPaperSimSampler
ROMInternalTranscriptBudgetedPaperSimAsNMA
active
attempt_project
clean
concrete_budgeted_ro_exact_hyperball_sample_with_seed_to_o_sign_active_projection
concrete_budgeted_self_logged_signature_clean_event_loss_surface
covered
ct $\alpha$ 
exact_project
haetae_mode
paper_sim_signature
pk
pk_eq
sample_loss
sample_with_seed
seed_coins
sign

```

```
sk_current
sk_eq
surface
```

**Theorem 3.235** (EasyCrypt line 7733). *EasyCrypt name.*

```
concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting
```

**Checked EasyCrypt statement.**

```
lemma concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting &m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
  Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO))).main() @ &m :
    res /\
    ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=
  Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,
    ROMInternalTranscriptBudgetedPaperSimAsNMA
      (A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO))).main() @ &m : res] +
  rom_signature_query_budget * rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *smt*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
FRO
HAETAE
HAETAE_RO
Pr
ROExactHyperballPaperSimSampler
ROMInternalTranscriptBudgetedPaperSimAsNMA
ROSigningAttemptPaperSimSampler
SIG
UF_NMA
budgeted_loss_ge1
left_le1
main
mu_bounded
rejection_sampling_loss_term
rejection_sampling_loss_term_ge1
right_ge0
rom_signature_query_budget
sample_loss
sampler_bad_prequery
signature_query_budget_count
signature_query_count
```

**Theorem 3.236** (EasyCrypt line 7767). *EasyCrypt name.*

```
concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting
```

**Checked EasyCrypt statement.**

```
lemma concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting &m :
```

```

Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,
  ROMInternalTranscriptBudgetedPaperSimAsNMA
    (A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO))).main() @ &m :
  res /\
  ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=
Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,
  ROMInternalTranscriptBudgetedPaperSimAsNMA
    (A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO))).main() @ &m : res] +
  rom_signature_query_budget * rejection_sampling_loss_term =>
Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,
  ROMInternalTranscriptBudgetedPaperSimAsNMA
    (A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO))).main() @ &m : res] <=
Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,
  ROMInternalTranscriptBudgetedPaperSimAsNMA
    (A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO))).main() @ &m : res] +
  rom_signature_query_budget * rejection_sampling_loss_term +
  rom_signature_query_budget *
    ((rom_hash_query_budget + rom_signature_query_budget) /
      challenge_support_cardinality_lower_bound).

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```

FRO
HAETAE_RO
clean_lifting
rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting

```

**Theorem 3.237** (EasyCrypt line 7795). *EasyCrypt name.*

```

concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting

```

**Checked EasyCrypt statement.**

```

lemma concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting
  &m :
  structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>
Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,
  ROMInternalTranscriptBudgetedPaperSimAsNMA
    (A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO))).main() @ &m : res] <=
Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,
  ROMInternalTranscriptBudgetedPaperSimAsNMA
    (A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO))).main() @ &m : res] +
  rom_signature_query_budget * rejection_sampling_loss_term +
  rom_signature_query_budget *
    ((rom_hash_query_budget + rom_signature_query_budget) /
      challenge_support_cardinality_lower_bound).

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting
concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting
sample_loss
```

**Theorem 3.238** (EasyCrypt line 7828). *EasyCrypt name*.

```
real_signing_rejection_paper_sim_nma_oracle
```

**Checked EasyCrypt statement.**

```
equiv real_signing_rejection_paper_sim_nma_oracle :
  ROMInternalTranscriptPaperSimAsNMA
    (A, RealSigningPaperSimSampler(H), H).O.sign ~
  ROMInternalTranscriptPaperSimAsNMA
    (A, HAETAERejectionPaperSimSampler(H), H).O.sign :
    ={glob H, arg} /\
  ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =
    ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
  RealSigningPaperSimSampler.pk_current{1} =
    HAETAERejectionPaperSimSampler.pk_current{2} /\
  RealSigningPaperSimSampler.sk_current{1} =
    HAETAERejectionPaperSimSampler.sk_current{2} /\
  RealSigningPaperSimSampler.pk_current{1} =
    public_key_of_secret haetae_mode
    RealSigningPaperSimSampler.sk_current{1} /\
  ROMInternalTranscriptPaperSimAsNMA.queries{1} =
    ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
  ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =
    ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
  ROMInternalTranscriptPaperSimAsNMA.records{1} =
    ROMInternalTranscriptPaperSimAsNMA.records{2}
  ==>
  ={glob H, res} /\
  ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =
    ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
  RealSigningPaperSimSampler.pk_current{1} =
    HAETAERejectionPaperSimSampler.pk_current{2} /\
  RealSigningPaperSimSampler.sk_current{1} =
    HAETAERejectionPaperSimSampler.sk_current{2} /\
  RealSigningPaperSimSampler.pk_current{1} =
    public_key_of_secret haetae_mode
    RealSigningPaperSimSampler.sk_current{1} /\
  ROMInternalTranscriptPaperSimAsNMA.queries{1} =
    ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
  ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =
    ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
  ROMInternalTranscriptPaperSimAsNMA.records{1} =
    ROMInternalTranscriptPaperSimAsNMA.records{2}.
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*, *rnd*, *smt*. Named identifiers

syntactically cited in the proof block:

```
HAETAERejectionPaperSimSampler
RealSigningPaperSimSampler
arg
glob
haetae_rejection_sample_from_attempt_of_coinsE
sample
sample_with_seed
```

**Theorem 3.239** (EasyCrypt line 7886). *EasyCrypt name.*

```
adversary_real_signing_rejection_paper_sim_nma
```

*Checked EasyCrypt statement.*

```
equiv adversary_real_signing_rejection_paper_sim_nma :
  A(H, ROMInternalTranscriptPaperSimAsNMA
    (A, RealSigningPaperSimSampler(H), H).O).forge ~
  A(H, ROMInternalTranscriptPaperSimAsNMA
    (A, HAETAERejectionPaperSimSampler(H), H).O).forge :
    ={glob H, glob A, arg} /\
    ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =
      ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
    RealSigningPaperSimSampler.pk_current{1} =
      HAETAERejectionPaperSimSampler.pk_current{2} /\
    RealSigningPaperSimSampler.sk_current{1} =
      HAETAERejectionPaperSimSampler.sk_current{2} /\
    RealSigningPaperSimSampler.pk_current{1} =
      public_key_of_secret haetae_mode
      RealSigningPaperSimSampler.sk_current{1} /\
    ROMInternalTranscriptPaperSimAsNMA.queries{1} =
      ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
    ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =
      ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
    ROMInternalTranscriptPaperSimAsNMA.records{1} =
      ROMInternalTranscriptPaperSimAsNMA.records{2}
    ==>
    ={glob H, res} /\
    ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =
      ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
    RealSigningPaperSimSampler.pk_current{1} =
      HAETAERejectionPaperSimSampler.pk_current{2} /\
    RealSigningPaperSimSampler.sk_current{1} =
      HAETAERejectionPaperSimSampler.sk_current{2} /\
    RealSigningPaperSimSampler.pk_current{1} =
      public_key_of_secret haetae_mode
      RealSigningPaperSimSampler.sk_current{1} /\
    ROMInternalTranscriptPaperSimAsNMA.queries{1} =
      ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
    ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =
      ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
    ROMInternalTranscriptPaperSimAsNMA.records{1} =
      ROMInternalTranscriptPaperSimAsNMA.records{2}.
```

**Formal mathematical reading.** *Mathematical theorem.* This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** *Use in this chapter.* It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*, *rnd*, *smt*. Named identifiers

syntactically cited in the proof block:

```
HAETAERejectionPaperSimSampler
ROMInternalTranscriptPaperSimAsNMA
RealSigningPaperSimSampler
arg
glob
haetae_mode
haetae_rejection_sample_from_attempt_of_coinsE
pk_current
public_key_of_secret
queries
records
sample
sample_with_seed
sk_current
transcripts
```

**Theorem 3.240** (EasyCrypt line 7963). *EasyCrypt name.*

```
rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact
```

**Checked EasyCrypt statement.**

```
lemma rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact &m :
  Pr[SIG.UF_NMA(H, HAETAERejectionPaperSimSampler,
    ROMInternalTranscriptPaperSimAsNMA
    (A, RealSigningPaperSimSampler(H))).main() @ &m : res] =
  Pr[SIG.UF_NMA(H, HAETAERejectionPaperSimSampler,
    ROMInternalTranscriptPaperSimAsNMA
    (A, HAETAERejectionPaperSimSampler(H))).main() @ &m : res].
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byequiv*, *proc*, *inline*, *wp*, *call*, *apply*, *rnd*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
HAETAERejectionPaperSimSampler
ROMInternalTranscriptPaperSimAsNMA
RealSigningPaperSimSampler
adversary_real_signing_rejection_paper_sim_nma
arg
forge
glob
haetae_mode
init
keygen_internal
kg
pk_current
public_key_of_secret
queries
records
secret_key_of_seed
sk_current
transcripts
verify
```



**Theorem 3.241** (EasyCrypt line 8039). *EasyCrypt name.*

```
exact_hyperball_haetae_rejection_paper_sim_nma_oracle
```

*Checked EasyCrypt statement.*

```
equiv exact_hyperball_haetae_rejection_paper_sim_nma_oracle :
  ROMInternalTranscriptPaperSimAsNMA
    (A, ExactHyperballHAETAERejectionPaperSimSampler(H), H).O.sign ~
  ROMInternalTranscriptPaperSimAsNMA
    (A, ExactHyperballPaperSimSampler(H), H).O.sign :
  ={glob H, arg} /\
  ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =
    ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
  ExactHyperballHAETAERejectionPaperSimSampler.pk_current{1} =
    ExactHyperballPaperSimSampler.pk_current{2} /\
  ExactHyperballHAETAERejectionPaperSimSampler.sk_current{1} =
    ExactHyperballPaperSimSampler.sk_current{2} /\
  ExactHyperballHAETAERejectionPaperSimSampler.pk_current{1} =
    public_key_of_secret haetae_mode
      ExactHyperballHAETAERejectionPaperSimSampler.sk_current{1} /\
  ROMInternalTranscriptPaperSimAsNMA.queries{1} =
    ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
  ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =
    ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
  ROMInternalTranscriptPaperSimAsNMA.records{1} =
    ROMInternalTranscriptPaperSimAsNMA.records{2}
  ==>
  ={glob H, res} /\
  ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =
    ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
  ExactHyperballHAETAERejectionPaperSimSampler.pk_current{1} =
    ExactHyperballPaperSimSampler.pk_current{2} /\
  ExactHyperballHAETAERejectionPaperSimSampler.sk_current{1} =
    ExactHyperballPaperSimSampler.sk_current{2} /\
  ExactHyperballHAETAERejectionPaperSimSampler.pk_current{1} =
    public_key_of_secret haetae_mode
      ExactHyperballHAETAERejectionPaperSimSampler.sk_current{1} /\
  ROMInternalTranscriptPaperSimAsNMA.queries{1} =
    ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
  ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =
    ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
  ROMInternalTranscriptPaperSimAsNMA.records{1} =
    ROMInternalTranscriptPaperSimAsNMA.records{2}.
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*. Named identifiers syntactically cited in the proof block:

```
ExactHyperballHAETAERejectionPaperSimSampler
ExactHyperballPaperSimSampler
arg
glob
pk_current
sk_current
```

**Theorem 3.242** (EasyCrypt line 8101). *EasyCrypt name.*

```
adversary_exact_hyperball_haetae_rejection_paper_sim_nma
```

### Checked EasyCrypt statement.

```
equiv adversary_exact_hyperball_haetae_rejection_paper_sim_nma :
  A(H, ROMInternalTranscriptPaperSimAsNMA
    (A, ExactHyperballHAETAERejectionPaperSimSampler(H), H).O).forge ~
  A(H, ROMInternalTranscriptPaperSimAsNMA
    (A, ExactHyperballPaperSimSampler(H), H).O).forge :
  ={glob H, glob A, arg} /\
  ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =
  ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
  ExactHyperballHAETAERejectionPaperSimSampler.pk_current{1} =
  ExactHyperballPaperSimSampler.pk_current{2} /\
  ExactHyperballHAETAERejectionPaperSimSampler.sk_current{1} =
  ExactHyperballPaperSimSampler.sk_current{2} /\
  ExactHyperballHAETAERejectionPaperSimSampler.pk_current{1} =
  public_key_of_secret haetae_mode
  ExactHyperballHAETAERejectionPaperSimSampler.sk_current{1} /\
  ROMInternalTranscriptPaperSimAsNMA.queries{1} =
  ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
  ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =
  ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
  ROMInternalTranscriptPaperSimAsNMA.records{1} =
  ROMInternalTranscriptPaperSimAsNMA.records{2}
  ==>
  ={glob H, res} /\
  ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =
  ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\
  ExactHyperballHAETAERejectionPaperSimSampler.pk_current{1} =
  ExactHyperballPaperSimSampler.pk_current{2} /\
  ExactHyperballHAETAERejectionPaperSimSampler.sk_current{1} =
  ExactHyperballPaperSimSampler.sk_current{2} /\
  ExactHyperballHAETAERejectionPaperSimSampler.pk_current{1} =
  public_key_of_secret haetae_mode
  ExactHyperballHAETAERejectionPaperSimSampler.sk_current{1} /\
  ROMInternalTranscriptPaperSimAsNMA.queries{1} =
  ROMInternalTranscriptPaperSimAsNMA.queries{2} /\
  ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =
  ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\
  ROMInternalTranscriptPaperSimAsNMA.records{1} =
  ROMInternalTranscriptPaperSimAsNMA.records{2}.
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *call*. Named identifiers syntactically cited in the proof block:

```
ExactHyperballHAETAERejectionPaperSimSampler
ExactHyperballPaperSimSampler
ROMInternalTranscriptPaperSimAsNMA
exact_hyperball_haetae_rejection_paper_sim_nma_oracle
glob
haetae_mode
pk_current
public_key_of_secret
queries
records
```

```
sk_current
transcripts
```

**Theorem 3.243** (EasyCrypt line 8164). *EasyCrypt name.*

```
rom_internal_nma_exact_hyperball_rejection_paper_sim_no_fallback_exact
```

*Checked EasyCrypt statement.*

```
lemma rom_internal_nma_exact_hyperball_rejection_paper_sim_no_fallback_exact &m :
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ExactHyperballHAETAETAERejectionPaperSimSampler(H))).main()
    @ &m : res] =
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA
      (A, ExactHyperballPaperSimSampler(H))).main() @ &m : res].
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byequiv*, *proc*, *inline*, *wp*, *call*, *apply*, *rnd*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
ExactHyperballHAETAETAERejectionPaperSimSampler
ExactHyperballPaperSimSampler
HAETAETAE
ROMInternalTranscriptPaperSimAsNMA
adversary_exact_hyperball_haetae_rejection_paper_sim_nma
arg
forge
glob
haetae_mode
init
keygen_internal
kg
pk_current
public_key_of_secret
queries
records
secret_key_of_seed
sk_current
transcripts
verify
```

**Theorem 3.244** (EasyCrypt line 8241). *EasyCrypt name.*

```
public_sim_to_paper_sim_nma_hop_from_sampling_hop
```

*Checked EasyCrypt statement.*

```
lemma public_sim_to_paper_sim_nma_hop_from_sampling_hop &m :
  Pr[SIG.UF_NMA(H, HAETAETAE, ROMInternalTranscriptPublicSimAsNMA(A)).main()
    @ &m : res] <=
  Pr[SIG.UF_NMA(H, HAETAETAE,
    ROMInternalTranscriptPaperSimAsNMA(A, Samp)).main() @ &m : res] +
  rejection_sampling_loss_term =>
```

```

Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPublicSimAsNMA(A)).main()
  @  $\mathcal{E}m$  : res] <=
Pr[SIG.UF_NMA(H, HAETAE,
  ROMInternalTranscriptPaperSimAsNMA(A, Samp)).main() @  $\mathcal{E}m$  : res] +
  rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 3.245** (EasyCrypt line 8268). *EasyCrypt* name.

```
transcript_logging_oracle_erasure
```

**Checked EasyCrypt statement.**

```

equiv transcript_logging_oracle_erasure :
  EUF_CMA_SimulatedSign(H, S, A, Sim).0.sign ~
  EUF_CMA_TranscriptSimulatedSign(H, S, A, Sim).0.sign :
  ={glob H, glob S, glob Sim, arg} /\
  EUF_CMA_SimulatedSign.queries{1} =
  EUF_CMA_TranscriptSimulatedSign.queries{2}
  ==>
  ={glob H, glob S, glob Sim, res} /\
  EUF_CMA_SimulatedSign.queries{1} =
  EUF_CMA_TranscriptSimulatedSign.queries{2}.

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*. Named identifiers syntactically cited in the proof block:

```

arg
glob

```

**Theorem 3.246** (EasyCrypt line 8286). *EasyCrypt* name.

```
adversary_transcript_logging_erasure
```

**Checked EasyCrypt statement.**

```

equiv adversary_transcript_logging_erasure :
  A(H, EUF_CMA_SimulatedSign(H, S, A, Sim).0).forge ~
  A(H, EUF_CMA_TranscriptSimulatedSign(H, S, A, Sim).0).forge :
  ={glob H, glob S, glob A, glob Sim, arg} /\
  EUF_CMA_SimulatedSign.queries{1} =
  EUF_CMA_TranscriptSimulatedSign.queries{2}
  ==>
  ={glob H, glob S, glob Sim, res} /\
  EUF_CMA_SimulatedSign.queries{1} =
  EUF_CMA_TranscriptSimulatedSign.queries{2}.

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_SimulatedSign
EUF_CMA_TranscriptSimulatedSign
arg
glob
queries

```

**Theorem 3.247** (EasyCrypt line 8309). *EasyCrypt name.*

```

transcript_logging_erasure_exact

```

**Checked EasyCrypt statement.**

```

lemma transcript_logging_erasure_exact Em :
  Pr[EUF_CMA_SimulatedSign(H, S, A, Sim).main() @ Em : res] =
  Pr[EUF_CMA_TranscriptSimulatedSign(H, S, A, Sim).main() @ Em : res].

```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byequiv*, *proc*, *wp*, *call*, *apply*. Named identifiers syntactically cited in the proof block:

```

EUF_CMA_SimulatedSign
EUF_CMA_TranscriptSimulatedSign
adversary_transcript_logging_erasure
arg
glob
queries

```

**Theorem 3.248** (EasyCrypt line 8350). *EasyCrypt name.*

```

real_signing_oracle_equiv

```

**Checked EasyCrypt statement.**

```

equiv real_signing_oracle_equiv :
  SIG.EUF_CMA(H, S, A).O.sign ~
  EUF_CMA_SimulatedSign(H, S, A, RealSigningSimulator(S)).O.sign :
  ={glob H, glob S, arg} /\
  SIG.EUF_CMA.sk{1} = RealSigningSimulator.sk{2} /\
  SIG.EUF_CMA.queries{1} = EUF_CMA_SimulatedSign.queries{2}
==>
  ={glob H, glob S, res} /\
  SIG.EUF_CMA.sk{1} = RealSigningSimulator.sk{2} /\

```

```
SIG.EUF_CMA.queries{1} = EUF_CMA_SimulatedSign.queries{2}.
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*. Named identifiers syntactically cited in the proof block:

```
RealSigningSimulator
arg
glob
sign
```

**Theorem 3.249** (EasyCrypt line 8369). *EasyCrypt name.*

```
adversary_real_signing_equiv
```

**Checked EasyCrypt statement.**

```
equiv adversary_real_signing_equiv :
  A(H, SIG.EUF_CMA(H, S, A).0).forge ~
  A(H, EUF_CMA_SimulatedSign(H, S, A, RealSigningSimulator(S)).0).forge :
  ={glob H, glob S, glob A, arg} /\
  SIG.EUF_CMA.sk{1} = RealSigningSimulator.sk{2} /\
  SIG.EUF_CMA.queries{1} = EUF_CMA_SimulatedSign.queries{2}
==>
  ={glob H, glob S, res} /\
  SIG.EUF_CMA.sk{1} = RealSigningSimulator.sk{2} /\
  SIG.EUF_CMA.queries{1} = EUF_CMA_SimulatedSign.queries{2}.
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *inline*, *wp*, *call*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA
EUF_CMA_SimulatedSign
RealSigningSimulator
SIG
arg
glob
queries
sign
sk
```

**Theorem 3.250** (EasyCrypt line 8393). *EasyCrypt name.*

```
real_signing_simulator_exact
```

### Checked EasyCrypt statement.

```
lemma real_signing_simulator_exact  $\mathcal{E}m$  :  
  Pr[SIG.EUF_CMA(H, S, A).main() @  $\mathcal{E}m$  : res] =  
  Pr[EUF_CMA_SimulatedSign(H, S, A, RealSigningSimulator(S)).main() @  $\mathcal{E}m$  : res].
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byequiv*, *proc*, *inline*, *wp*, *call*, *apply*. Named identifiers syntactically cited in the proof block:

```
EUF_CMA  
EUF_CMA_SimulatedSign  
RealSigningSimulator  
SIG  
adversary_real_signing_equiv  
arg  
glob  
init  
queries  
sk
```

### Theorem 3.251 (EasyCrypt line 8507). EasyCrypt name.

```
bimodal_special_soundness_lift_exact
```

### Checked EasyCrypt statement.

```
lemma bimodal_special_soundness_lift_exact  $\mathcal{E}m$  md :  
  Pr[BimodalSpecialSoundness_Game(H, S, F, X).main(md) @  $\mathcal{E}m$  : res] =  
  Pr[SpecialSoundness_Game(  
    H, S, F, BimodalTranscriptExtractor_As_ModuleSIS(X)).main(md)  
    @  $\mathcal{E}m$  : res].
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byequiv*, *proc*, *inline*, *wp*, *call*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
BimodalTranscriptExtractor_As_ModuleSIS  
arg  
bimodal_selftarget_msis_valid  
bimodal_to_module_sis_instance  
extract  
glob  
module_sis_instance_of_fork
```

### Theorem 3.252 (EasyCrypt line 8546). EasyCrypt name.

```
special_soundness_interfaces_ready_from_forking
```

**Checked EasyCrypt statement.**

```
lemma special_soundness_interfaces_ready_from_forking md :  
  forking_extraction_obligation md =>  
  special_soundness_interfaces_ready md.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*. Named identifiers syntactically cited in the proof block:

```
bimodal_to_module_sis_lift_obligation_holds  
exact  
fork_public_reconstruction_obligation_holds  
hfork  
md  
special_soundness_interfaces_ready
```

**Theorem 3.253** (EasyCrypt line 8559). *EasyCrypt name.*

```
special_soundness_interfaces_public_reconstruction
```

**Checked EasyCrypt statement.**

```
lemma special_soundness_interfaces_public_reconstruction md :  
  special_soundness_interfaces_ready md =>  
  fork_public_reconstruction_obligation md.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
exact  
hpublic  
special_soundness_interfaces_ready
```

**Theorem 3.254** (EasyCrypt line 8568). *EasyCrypt name.*

```
special_soundness_interfaces_ready_sound
```

**Checked EasyCrypt statement.**

```
lemma special_soundness_interfaces_ready_sound md :  
  special_soundness_interfaces_ready md =>  
  special_soundness_obligation md.
```



**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
exact
hfork
hlift
md
special_soundness_from_bimodal_lift
special_soundness_interfaces_ready
```

**Theorem 3.255** (EasyCrypt line 8577). *EasyCrypt name.*

```
special_soundness_interfaces_ready_sound_with_public_reconstruction
```

**Checked EasyCrypt statement.**

```
lemma special_soundness_interfaces_ready_sound_with_public_reconstruction md :
  special_soundness_interfaces_ready md =>
  special_soundness_with_public_reconstruction_obligation md.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
exact
hfork
hlift
hpublic
md
special_soundness_interfaces_ready
special_soundness_with_public_reconstruction_from_bimodal_lift
```

**Theorem 3.256** (EasyCrypt line 8592). *EasyCrypt name.*

```
signing_simulator_sound_current
```

**Checked EasyCrypt statement.**

```
lemma signing_simulator_sound_current md :
  signing_simulator_sound md.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
ctx
exact
md
pk
sig
signing_simulation_verifies
signing_simulator_sound
```

**Theorem 3.257** (EasyCrypt line 8600). *EasyCrypt name*.

```
rejection_sampling_bound_from_fs_bound
```

**Checked EasyCrypt statement.**

```
lemma rejection_sampling_bound_from_fs_bound :
  fs_with_aborts_bound_obligation =>
  rejection_sampling_bound_obligation.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
addr_ge0
exact
fs_with_aborts_bound_obligation
ge_entropy
ge_reprogram
rejection_sampling_bound_obligation
rejection_sampling_loss_term
```

**Theorem 3.258** (EasyCrypt line 8620). *EasyCrypt name*.

```
fs_with_aborts_interfaces_ready_from_parts
```

**Checked EasyCrypt statement.**

```
lemma fs_with_aborts_interfaces_ready_from_parts md :
  special_soundness_interfaces_ready md =>
  fs_with_aborts_bound_obligation =>
  fs_with_aborts_interfaces_ready md.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*, *split*. Named identifiers syntactically cited in the proof block:

```
exact
fs_with_aborts_interfaces_ready
hfs_bound
hspecial
md
signing_simulator_sound_current
```

**Theorem 3.259** (EasyCrypt line 8634). *EasyCrypt name*.

```
fs_with_aborts_interfaces_ready_from_forking
```

**Checked EasyCrypt statement.**

```
lemma fs_with_aborts_interfaces_ready_from_forking md :
  forking_extraction_obligation md =>
  fs_with_aborts_interfaces_ready md.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Named identifiers syntactically cited in the proof block:

```
exact
fork
fs_with_aborts_bound_obligation_holds
fs_with_aborts_interfaces_ready_from_parts
md
special_soundness_interfaces_ready_from_forking
```

**Theorem 3.260** (EasyCrypt line 8644). *EasyCrypt name*.

```
fs_with_aborts_interfaces_ready_holds
```

**Checked EasyCrypt statement.**

```
lemma fs_with_aborts_interfaces_ready_holds md :
  fs_with_aborts_interfaces_ready md.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Named identifiers syntactically cited in the proof block:

```
exact
forking_extraction_obligation_holds
fs_with_aborts_interfaces_ready_from_forking
md
```

**Theorem 3.261** (EasyCrypt line 8651). *EasyCrypt name*.

```
structural_to_exact_hyperball_paper_sample_loss_from_obligation
```

**Checked EasyCrypt statement.**

```
lemma structural_to_exact_hyperball_paper_sample_loss_from_obligation
  (md : mode) :
  structural_to_exact_hyperball_paper_sample_loss_obligation md =>
  forall (sk : skey) (m : message) (ctx : context)
    (p : paper_sim_signature_sample -> bool),
  mu (dsigning_attempt_state md sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st)) <=
  mu (deexact_hyperball_signing_attempt_state md sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st)) +
  rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
ctx
exact
hop
sk
structural_to_exact_hyperball_paper_sample_loss_obligation
```

**Theorem 3.262** (EasyCrypt line 8667). **EasyCrypt name.**

```
structural_to_exact_hyperball_paper_sample_loss_from_attempt_loss
```

**Checked EasyCrypt statement.**

```
lemma structural_to_exact_hyperball_paper_sample_loss_from_attempt_loss
  (md : mode) :
  structural_to_exact_hyperball_attempt_loss_obligation =>
  forall (sk : skey) (m : message) (ctx : context)
    (p : paper_sim_signature_sample -> bool),
  mu (dsigning_attempt_state md sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st)) <=
  mu (deexact_hyperball_signing_attempt_state md sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st)) +
  rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
ctx
exact
hop
```

```
md
paper_sim_sample_from_rejection_attempt
sk
st
structural_to_exact_hyperball_attempt_loss_obligation
```

**Theorem 3.263** (EasyCrypt line 8684). *EasyCrypt name.*

```
structural_to_exact_hyperball_paper_sample_loss_obligation_from_attempt_loss
```

**Checked EasyCrypt statement.**

```
lemma structural_to_exact_hyperball_paper_sample_loss_obligation_from_attempt_loss
  (md : mode) :
  structural_to_exact_hyperball_attempt_loss_obligation =>
  structural_to_exact_hyperball_paper_sample_loss_obligation md.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
ctx
exact
hop
md
sk
structural_to_exact_hyperball_paper_sample_loss_from_attempt_loss
structural_to_exact_hyperball_paper_sample_loss_obligation
```

**Theorem 3.264** (EasyCrypt line 8696). *EasyCrypt name.*

```
structural_to_exact_hyperball_paper_sample_loss_from_coin_sample_pair_loss
```

**Checked EasyCrypt statement.**

```
lemma structural_to_exact_hyperball_paper_sample_loss_from_coin_sample_pair_loss
  (md : mode) :
  structural_to_exact_hyperball_coin_sample_pair_loss_obligation =>
  forall (sk : skey) (m : message) (ctx : context)
    (p : paper_sim_signature_sample -> bool),
  mu (dsigning_attempt_state md sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st)) <=
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st)) +
  rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
hop
structural_to_exact_hyperball_attempt_loss_from_coin_sample_pair_loss
structural_to_exact_hyperball_paper_sample_loss_from_attempt_loss
```

**Theorem 3.265** (EasyCrypt line 8712). *EasyCrypt name.*

```
structural_to_exact_hyperball_paper_sample_loss_obligation_from_coin_sample_pair_loss
```

**Checked EasyCrypt statement.**

```
lemma structural_to_exact_hyperball_paper_sample_loss_obligation_from_coin_sample_pair_loss
  (md : mode) :
  structural_to_exact_hyperball_coin_sample_pair_loss_obligation =>
  structural_to_exact_hyperball_paper_sample_loss_obligation md.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
ctx
exact
hop
md
sk
structural_to_exact_hyperball_paper_sample_loss_from_coin_sample_pair_loss
structural_to_exact_hyperball_paper_sample_loss_obligation
```

**Theorem 3.266** (EasyCrypt line 8724). *EasyCrypt name.*

```
dexact_hyperball_reference_paper_sim_attempt_boundary_ready
```

**Checked EasyCrypt statement.**

```
lemma dexact_hyperball_reference_paper_sim_attempt_boundary_ready
  (md : mode) (sk : skey) (m : message) (ctx : context) :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
  (signing_attempt_state_reference_rejection_aborts md) = 0%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
dexact_hyperball_signing_attempt_state_reference_rejection_abort_zero
```

**Theorem 3.267** (EasyCrypt line 8732). *EasyCrypt name.*

```
dexact_hyperball_full_rejection_paper_sim_attempt_boundary_ready
```

**Checked EasyCrypt statement.**

```
lemma dexact_hyperball_full_rejection_paper_sim_attempt_boundary_ready
  (md : mode) (sk : skey) (m : message) (ctx : context) :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (signing_attempt_state_rejection_aborts md
      (public_key_of_secret md sk) m ctx) = 0%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **apply**. Named identifiers syntactically cited in the proof block:

```
dexact_hyperball_signing_attempt_state_rejection_abort_zero
```

**Theorem 3.268 (EasyCrypt line 8741). EasyCrypt name.**

```
dexact_hyperball_haetae_rejection_sample_no_fallback_mu
```

**Checked EasyCrypt statement.**

```
lemma dexact_hyperball_haetae_rejection_sample_no_fallback_mu
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (p : paper_sim_signature_sample -> bool) :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (fun st =>
      p (haetae_rejection_sample_from_attempt md
          (public_key_of_secret md sk) m ctx st)) =
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (fun st => p (paper_sim_sample_from_rejection_attempt st)).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **apply**, **rewrite**. Named identifiers syntactically cited in the proof block:

```
Distr
ctx
dexact_hyperball_signing_attempt_state_rejection_accepts
haetae_rejection_sample_from_attempt
md
mu_eq_support
sk
st
st_supp
```

**Theorem 3.269 (EasyCrypt line 8758). EasyCrypt name.**

```
dexact_hyperball_haetae_rejection_sample_signature_no_fallback_mu
```

**Checked EasyCrypt statement.**

```

lemma dexact_hyperball_haetae_rejection_sample_signature_no_fallback_mu
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (p : signature -> bool) :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (fun st =>
      p (paper_sim_signature md (public_key_of_secret md sk) m ctx
        (haetae_rejection_sample_from_attempt md
          (public_key_of_secret md sk) m ctx st))) =
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (fun st =>
      p (paper_sim_signature md (public_key_of_secret md sk) m ctx
        (paper_sim_sample_from_rejection_attempt st))).

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Named identifiers syntactically cited in the proof block:

```

ctx
dexact_hyperball_haetae_rejection_sample_no_fallback_mu
exact
md
paper_sim_signature
public_key_of_secret
sk

```

**Theorem 3.270** (EasyCrypt line 8777). *EasyCrypt name.*

```

dexact_hyperball_paper_sim_rejection_attempt_signature_mu

```

**Checked EasyCrypt statement.**

```

lemma dexact_hyperball_paper_sim_rejection_attempt_signature_mu
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (p : signature -> bool) :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (fun st =>
      p (paper_sim_signature md (public_key_of_secret md sk) m ctx
        (paper_sim_sample_from_rejection_attempt st))) =
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (fun st => p (signing_attempt_state_signature st)).

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *have*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

Distr
ctx
dexact_hyperball_signing_attempt_state_rejection_accepts
field_ok
md
mu_eq_support

```



```

paper_sim_signature_rejection_attempt_matches_state
public_key_of_secret
signing_attempt_state_field_accepts
signing_attempt_state_rejection_accepts
signing_attempt_state_verification_accepts
sk
st
st_supp

```

**Theorem 3.271** (EasyCrypt line 8800). *EasyCrypt name.*

```
dexact_hyperball_haetae_rejection_sample_signature_mu
```

**Checked EasyCrypt statement.**

```

lemma dexact_hyperball_haetae_rejection_sample_signature_mu
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (p : signature -> bool) :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (fun st =>
      p (paper_sim_signature md (public_key_of_secret md sk) m ctx
        (haetae_rejection_sample_from_attempt md
          (public_key_of_secret md sk) m ctx st))) =
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (fun st => p (signing_attempt_state_signature st)).

```

**Formal mathematical reading.** *Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.*

**Role in the proof.** *Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```

ctx
dexact_hyperball_haetae_rejection_sample_signature_no_fallback_mu
dexact_hyperball_paper_sim_rejection_attempt_signature_mu
exact
md
sk

```

## Explicit Program-Logic Judgments Used Inside Proof Scripts

**Program-Logic Judgment 3.1** (EasyCrypt line 2155). **Checked EasyCrypt judgment.**

```
hoare.
```

**Formal mathematical reading.** This is a Hoare judgment. It states partial correctness of the displayed procedure from the displayed precondition to the displayed postcondition.

**Program-Logic Judgment 3.2** (EasyCrypt line 2166). **Checked EasyCrypt judgment.**

```
hoare.
```

**Formal mathematical reading.** This is a Hoare judgment. It states partial correctness of the displayed procedure from the displayed precondition to the displayed postcondition.

**Program-Logic Judgment 3.3** (EasyCrypt line 3539). **Checked EasyCrypt judgment.**

```
hoare.
```

**Formal mathematical reading.** This is a Hoare judgment. It states partial correctness of the displayed procedure from the displayed precondition to the displayed postcondition.

**Program-Logic Judgment 3.4** (EasyCrypt line 3745). **Checked EasyCrypt judgment.**

```
hoare.
```

**Formal mathematical reading.** This is a Hoare judgment. It states partial correctness of the displayed procedure from the displayed precondition to the displayed postcondition.

**Program-Logic Judgment 3.5** (EasyCrypt line 3875). **Checked EasyCrypt judgment.**

```
hoare.
```

**Formal mathematical reading.** This is a Hoare judgment. It states partial correctness of the displayed procedure from the displayed precondition to the displayed postcondition.

**Program-Logic Judgment 3.6** (EasyCrypt line 4317). **Checked EasyCrypt judgment.**

```
hoare.
```

**Formal mathematical reading.** This is a Hoare judgment. It states partial correctness of the displayed procedure from the displayed precondition to the displayed postcondition.

**Program-Logic Judgment 3.7** (EasyCrypt line 4867). **Checked EasyCrypt judgment.**

```
hoare.
```

**Formal mathematical reading.** This is a Hoare judgment. It states partial correctness of the displayed procedure from the displayed precondition to the displayed postcondition.

**Program-Logic Judgment 3.8** (EasyCrypt line 4993). **Checked EasyCrypt judgment.**

```
hoare.
```

**Formal mathematical reading.** This is a Hoare judgment. It states partial correctness of the displayed procedure from the displayed precondition to the displayed postcondition.

**Program-Logic Judgment 3.9** (EasyCrypt line 4999). **Checked EasyCrypt judgment.**

```
hoare.
```

**Formal mathematical reading.** This is a Hoare judgment. It states partial correctness of the displayed procedure from the displayed precondition to the displayed postcondition.

**Program-Logic Judgment 3.10** (EasyCrypt line 5130). **Checked EasyCrypt judgment.**

```
hoare.
```

**Formal mathematical reading.** This is a Hoare judgment. It states partial correctness of the displayed procedure from the displayed precondition to the displayed postcondition.

**Program-Logic Judgment 3.11** (EasyCrypt line 5141). **Checked EasyCrypt judgment.**

```
hoare.
```

**Formal mathematical reading.** This is a Hoare judgment. It states partial correctness of the displayed procedure from the displayed precondition to the displayed postcondition.

**Program-Logic Judgment 3.12** (EasyCrypt line 7071). **Checked EasyCrypt judgment.**

```
phoare[StructuralAttemptPaperSample.sample :
  arg = (sk0, m0, ctx0) ==> p0 res] <=
(mu (dexact_hyperball_signing_attempt_state haetae_mode sk0 m0 ctx0)
  (fun st => p0 (paper_sim_sample_from_rejection_attempt st)) +
  rejection_sampling_loss_term).
```

**Formal mathematical reading.** This is a probabilistic Hoare judgment. It states that the command satisfies the postcondition with the displayed probability bound.

*End of generated formal inventory for HAETAE\_HopGames. ec.*

## Chapter 4

### HAETAE\_Reductions.ec

**Mathematical role.** This is the loss ledger. It names every numerical term in the final theorem and proves the real-arithmetic facts needed to compose inequalities.

**Hardness and lifting terms.** The file declares abstract real numbers

$$\epsilon_{\text{MLWE}}, \quad \epsilon_{\text{Module-SIS}},$$

as `mlwe_hardness_term` and `module_sis_hardness_term`. The bimodal lifting loss is

$$\delta_{\text{BimodalSelfTargetMSIS} \rightarrow \text{Module-SIS}} = \text{bimodal\_to\_msis\_reduction\_loss\_term} = 0.$$

Thus the bimodal hardness term is defined as

$$\epsilon_{\text{BimodalSelfTargetMSIS}} = \epsilon_{\text{Module-SIS}} + \delta_{\text{BimodalSelfTargetMSIS} \rightarrow \text{Module-SIS}}.$$

**Random-oracle losses.** The counted-ROM loss is

$$\Delta_{\text{counted}} = \frac{(q_h + q_s + 1)^2}{|\mathcal{C}|} + \frac{q_s(q_h + 1)}{|\mathcal{C}|} + \frac{q_s(q_h + 1)}{|\mathcal{C}|}.$$

EasyCrypt names these summands `counted_rom_collision_term`, `counted_rom_prequery_term`, and `counted_rom_min_entropy_term`. The lemmas prove positivity, subunit bounds, and equivalent expanded forms.

**Why this file matters.** In a paper proof, the final inequality is often simplified by informal algebra. Here every simplification is a lemma, for example `haetae_euf_counted_rom_bound_groupedE`. This prevents accidental loss of a term when composing the game-hop chain.

#### Textbook Formal Inventory

This subsection records the exact formal material in `HAETAE_Reductions.ec`. It is generated from the proof-surface EasyCrypt file, so the line numbers and statements are meant to be read next to the checked source.

**Chapter role.** real-valued security-loss ledger and final arithmetic normal forms.

**Inventory size.** 5 context declarations, 26 definitions/game objects, 50 lemmas or relational theorems, and 0 explicit Hoare/Phoare judgments were found.

## Theory Context, Imports, and Quantified Modules

### Context 1 (line 1) EasyCrypt name.

```
imported theories
```

#### Checked EasyCrypt statement.

```
require import AllCore Real RealExp StdOrder.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 2 (line 2) EasyCrypt name.

```
imported theories
```

#### Checked EasyCrypt statement.

```
require import HAETAE_Events.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 3 (line 4) EasyCrypt name.

```
HAETAE_Reductions
```

#### Checked EasyCrypt statement.

```
theory HAETAE_Reductions.
```

**Formal mathematical reading.** Mathematical context. This begins a namespace for the formal development in this file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 4 (line 6) EasyCrypt name.

```
opened namespace
```

#### Checked EasyCrypt statement.

```
import HAETAE_Events.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 5 (line 7) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import RealOrder.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

## Definitions, Carrier Sets, Operations, Games, and Procedures

**Definition 4.1 (EasyCrypt line 9). EasyCrypt name.**

```
mlwe_hardness_term
```

**Checked EasyCrypt statement.**

```
op mlwe_hardness_term : real.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 4.2 (EasyCrypt line 10). EasyCrypt name.**

```
module_sis_hardness_term
```

**Checked EasyCrypt statement.**

```
op module_sis_hardness_term : real.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 4.3 (EasyCrypt line 11). EasyCrypt name.**

```
bimodal_to_msis_reduction_loss_term
```

**Checked EasyCrypt statement.**

```
op bimodal_to_msis_reduction_loss_term : real = 0%r.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 4.4 (EasyCrypt line 13). EasyCrypt name.**

```
bimodal_selftarget_msis_hardness_term
```

### Checked EasyCrypt statement.

```
op bimodal_selftarget_msis_hardness_term =  
  module_sis_hardness_term + bimodal_to_msis_reduction_loss_term.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

### Definition 4.5 (EasyCrypt line 16). EasyCrypt name.

```
signature_query_count
```

### Checked EasyCrypt statement.

```
op signature_query_count : real = 2%r.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 4.6 (EasyCrypt line 17). EasyCrypt name.

```
hash_query_count
```

### Checked EasyCrypt statement.

```
op hash_query_count : real = 2%r.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 4.7 (EasyCrypt line 18). EasyCrypt name.

```
signature_query_budget_count
```

### Checked EasyCrypt statement.

```
op signature_query_budget_count : int = 2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 4.8 (EasyCrypt line 19). EasyCrypt name.

```
hash_query_budget_count
```

### Checked EasyCrypt statement.

```
op hash_query_budget_count : int = 2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 4.9** (EasyCrypt line 20). **EasyCrypt name.**

```
abort_probability
```

**Checked EasyCrypt statement.**

```
op abort_probability : real = 0%r.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 4.10** (EasyCrypt line 21). **EasyCrypt name.**

```
min_entropy_loss_factor
```

**Checked EasyCrypt statement.**

```
op min_entropy_loss_factor : real = 2%r.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 4.11** (EasyCrypt line 22). **EasyCrypt name.**

```
challenge_support_bits_lower_bound
```

**Checked EasyCrypt statement.**

```
op challenge_support_bits_lower_bound : int = 58.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 4.12** (EasyCrypt line 23). **EasyCrypt name.**

```
challenge_support_cardinality_lower_bound
```

**Checked EasyCrypt statement.**

```
op challenge_support_cardinality_lower_bound : real =  
288230376151711744%r.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 4.13** (EasyCrypt line 26). **EasyCrypt name.**

```
rom_hash_query_budget
```



### Checked EasyCrypt statement.

```
op rom_hash_query_budget : real = hash_query_count.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 4.14 (EasyCrypt line 27). EasyCrypt name.

```
rom_signature_query_budget
```

### Checked EasyCrypt statement.

```
op rom_signature_query_budget : real = signature_query_count.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 4.15 (EasyCrypt line 28). EasyCrypt name.

```
rom_total_query_budget
```

### Checked EasyCrypt statement.

```
op rom_total_query_budget =  
  rom_hash_query_budget + rom_signature_query_budget + 1%r.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 4.16 (EasyCrypt line 31). EasyCrypt name.

```
counted_rom_collision_term
```

### Checked EasyCrypt statement.

```
op counted_rom_collision_term =  
  (rom_total_query_budget * rom_total_query_budget) /  
  challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

### Definition 4.17 (EasyCrypt line 35). EasyCrypt name.

```
counted_rom_prequery_term
```

### Checked EasyCrypt statement.

```
op counted_rom_prequery_term =  
  (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) /  
  challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 4.18** (EasyCrypt line 39). **EasyCrypt name.**

```
counted_rom_min_entropy_term
```

**Checked EasyCrypt statement.**

```
op counted_rom_min_entropy_term =
  (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) /
  challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 4.19** (EasyCrypt line 43). **EasyCrypt name.**

```
counted_rom_prequery_reprogramming_term
```

**Checked EasyCrypt statement.**

```
op counted_rom_prequery_reprogramming_term =
  counted_rom_collision_term + counted_rom_prequery_term.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 4.20** (EasyCrypt line 46). **EasyCrypt name.**

```
counted_rom_programming_loss_term
```

**Checked EasyCrypt statement.**

```
op counted_rom_programming_loss_term =
  counted_rom_collision_term +
  counted_rom_prequery_term +
  counted_rom_min_entropy_term.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 4.21** (EasyCrypt line 51). **EasyCrypt name.**

```
signing_query_scale
```

**Checked EasyCrypt statement.**

```
op signing_query_scale =
  signature_query_count / (1%r - abort_probability).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 4.22** (EasyCrypt line 54). **EasyCrypt name.**

```
fs_with_aborts_reprogramming_term
```

**Checked EasyCrypt statement.**

```
op fs_with_aborts_reprogramming_term =
  sqrt signing_query_scale *
    (hash_query_count + 1%r + sqrt signing_query_scale).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 4.23** (EasyCrypt line 58). **EasyCrypt name.**

```
fs_with_aborts_min_entropy_term
```

**Checked EasyCrypt statement.**

```
op fs_with_aborts_min_entropy_term =
  min_entropy_loss_factor *
    (signing_query_scale * (hash_query_count + 1%r)).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 4.24** (EasyCrypt line 62). **EasyCrypt name.**

```
rejection_sampling_loss_term
```

**Checked EasyCrypt statement.**

```
op rejection_sampling_loss_term =
  fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 4.25** (EasyCrypt line 65). **EasyCrypt name.**

```
haetae_euf_bound
```

**Checked EasyCrypt statement.**

```
op haetae_euf_bound =
  mlwe_hardness_term +
    bimodal_selftarget_msis_hardness_term +
    rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 4.26** (EasyCrypt line 70). *EasyCrypt name.*

```
haetae_euf_counted_rom_bound
```

**Checked EasyCrypt statement.**

```
op haetae_euf_counted_rom_bound =
  mlwe_hardness_term +
  bimodal_selftarget_msis_hardness_term +
  counted_rom_programming_loss_term.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

## Lemmas, Theorems, Relational Equivalences, and Their Proof Dependencies

**Theorem 4.1** (EasyCrypt line 75). *EasyCrypt name.*

```
haetae_euf_boundE
```

**Checked EasyCrypt statement.**

```
lemma haetae_euf_boundE :
  haetae_euf_bound =
    mlwe_hardness_term +
    module_sis_hardness_term +
    bimodal_to_msis_reduction_loss_term +
    fs_with_aborts_reprogramming_term +
    fs_with_aborts_min_entropy_term.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
bimodal_selftarget_msis_hardness_term
haetae_euf_bound
rejection_sampling_loss_term
ring
```

**Theorem 4.2** (EasyCrypt line 87). *EasyCrypt name.*

```
haetae_euf_bound_groupedE
```

**Checked EasyCrypt statement.**

```
lemma haetae_euf_bound_groupedE :
```

```

haetae_euf_bound =
  (mlwe_hardness_term +
    (module_sis_hardness_term + bimodal_to_msis_reduction_loss_term)) +
  (fs_with_aborts_reprogramming_term +
    fs_with_aborts_min_entropy_term).

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

bimodal_selftarget_msis_hardness_term
haetae_euf_bound
rejection_sampling_loss_term
ring

```

**Theorem 4.3** (EasyCrypt line 98). *EasyCrypt name.*

```

haetae_euf_bound_rejection_groupedE

```

**Checked EasyCrypt statement.**

```

lemma haetae_euf_bound_rejection_groupedE :
  haetae_euf_bound =
    (mlwe_hardness_term +
      (module_sis_hardness_term + bimodal_to_msis_reduction_loss_term)) +
    rejection_sampling_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

bimodal_selftarget_msis_hardness_term
haetae_euf_bound
ring

```

**Theorem 4.4** (EasyCrypt line 107). *EasyCrypt name.*

```

haetae_euf_counted_rom_bound_groupedE

```

**Checked EasyCrypt statement.**

```

lemma haetae_euf_counted_rom_bound_groupedE :
  haetae_euf_counted_rom_bound =
    (mlwe_hardness_term +
      (module_sis_hardness_term + bimodal_to_msis_reduction_loss_term)) +
    counted_rom_programming_loss_term.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
bimodal_selftarget_msis_hardness_term
haetae_euf_counted_rom_bound
ring
```

**Theorem 4.5** (EasyCrypt line 117). *EasyCrypt name*.

```
signing_query_scaleE
```

**Checked EasyCrypt statement.**

```
lemma signing_query_scaleE :
  signing_query_scale = signature_query_count.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
abort_probability
ring
signature_query_count
signing_query_scale
```

**Theorem 4.6** (EasyCrypt line 123). *EasyCrypt name*.

```
signing_query_scale_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma signing_query_scale_nonnegative :
  0%r <= signing_query_scale.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signature_query_count
```

```
signing_query_scaleE
```

**Theorem 4.7** (EasyCrypt line 129). *EasyCrypt name.*

```
fs_with_aborts_reprogramming_term_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma fs_with_aborts_reprogramming_term_nonnegative :  
  0%r <= fs_with_aborts_reprogramming_term.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
addr_ge0  
fs_with_aborts_reprogramming_term  
ge0_sqrt  
hash_query_count  
mulr_ge0
```

**Theorem 4.8** (EasyCrypt line 140). *EasyCrypt name.*

```
fs_with_aborts_min_entropy_term_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma fs_with_aborts_min_entropy_term_nonnegative :  
  0%r <= fs_with_aborts_min_entropy_term.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
addr_ge0  
fs_with_aborts_min_entropy_term  
hash_query_count  
min_entropy_loss_factor  
mulr_ge0  
signing_query_scale_nonnegative
```

**Theorem 4.9** (EasyCrypt line 151). *EasyCrypt name.*

```
rom_hash_query_budget_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma rom_hash_query_budget_nonnegative :  
  0%r <= rom_hash_query_budget.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 4.10** (EasyCrypt line 155). *EasyCrypt name.*

```
rom_signature_query_budget_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma rom_signature_query_budget_nonnegative :  
  0%r <= rom_signature_query_budget.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 4.11** (EasyCrypt line 159). *EasyCrypt name.*

```
hash_query_budget_countE
```

**Checked EasyCrypt statement.**

```
lemma hash_query_budget_countE :  
  hash_query_count = (hash_query_budget_count)%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 4.12** (EasyCrypt line 163). *EasyCrypt name.*

```
signature_query_budget_countE
```

**Checked EasyCrypt statement.**

```
lemma signature_query_budget_countE :  
  signature_query_count = (signature_query_budget_count)%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 4.13** (EasyCrypt line 167). *EasyCrypt name.*

```
hash_query_budget_count_nonnegative
```



**Checked EasyCrypt statement.**

```
lemma hash_query_budget_count_nonnegative :  
  0 <= hash_query_budget_count.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 4.14** (EasyCrypt line 171). **EasyCrypt name.**

```
signature_query_budget_count_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma signature_query_budget_count_nonnegative :  
  0 <= signature_query_budget_count.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 4.15** (EasyCrypt line 175). **EasyCrypt name.**

```
min_entropy_loss_factor_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma min_entropy_loss_factor_nonnegative :  
  0%r <= min_entropy_loss_factor.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 4.16** (EasyCrypt line 179). **EasyCrypt name.**

```
challenge_support_cardinality_lower_bound_positive
```

**Checked EasyCrypt statement.**

```
lemma challenge_support_cardinality_lower_bound_positive :  
  0%r < challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 4.17** (EasyCrypt line 183). **EasyCrypt name.**

```
challenge_support_cardinality_lower_bound_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma challenge_support_cardinality_lower_bound_nonnegative :  
  0%r <= challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **apply**. Named identifiers syntactically cited in the proof block:

```
challenge_support_cardinality_lower_bound_positive  
ltrW
```

**Theorem 4.18** (EasyCrypt line 189). **EasyCrypt name.**

```
rom_total_query_budget_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma rom_total_query_budget_nonnegative :  
  0%r <= rom_total_query_budget.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **rewrite**, **apply**. Named identifiers syntactically cited in the proof block:

```
addr_ge0  
rom_hash_query_budget_nonnegative  
rom_signature_query_budget_nonnegative  
rom_total_query_budget
```

**Theorem 4.19** (EasyCrypt line 200). **EasyCrypt name.**

```
counted_rom_collision_term_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_collision_term_nonnegative :  
  0%r <= counted_rom_collision_term.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
challenge_support_cardinality_lower_bound_nonnegative
counted_rom_collision_term
divr_ge0
mulr_ge0
rom_total_query_budget_nonnegative
```

**Theorem 4.20** (EasyCrypt line 209). *EasyCrypt name.*

```
counted_rom_prequery_term_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_prequery_term_nonnegative :
  0%r <= counted_rom_prequery_term.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
addr_ge0
challenge_support_cardinality_lower_bound_nonnegative
counted_rom_prequery_term
divr_ge0
mulr_ge0
rom_hash_query_budget_nonnegative
rom_signature_query_budget_nonnegative
```

**Theorem 4.21** (EasyCrypt line 220). *EasyCrypt name.*

```
counted_rom_min_entropy_term_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_min_entropy_term_nonnegative :
  0%r <= counted_rom_min_entropy_term.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
addr_ge0
challenge_support_cardinality_lower_bound_nonnegative
```

```
counted_rom_min_entropy_term
divr_ge0
mulr_ge0
rom_hash_query_budget_nonnegative
rom_signature_query_budget_nonnegative
```

**Theorem 4.22** (EasyCrypt line 231). *EasyCrypt name.*

```
counted_rom_prequery_reprogramming_term_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_prequery_reprogramming_term_nonnegative :
  0%r <= counted_rom_prequery_reprogramming_term.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
addr_ge0
counted_rom_collision_term_nonnegative
counted_rom_prequery_reprogramming_term
counted_rom_prequery_term_nonnegative
```

**Theorem 4.23** (EasyCrypt line 240). *EasyCrypt name.*

```
counted_rom_programming_loss_term_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_programming_loss_term_nonnegative :
  0%r <= counted_rom_programming_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
addr_ge0
counted_rom_collision_term_nonnegative
counted_rom_min_entropy_term_nonnegative
counted_rom_prequery_term_nonnegative
counted_rom_programming_loss_term
```

**Theorem 4.24** (EasyCrypt line 251). *EasyCrypt name.*

```
counted_rom_collision_termE
```

### Checked EasyCrypt statement.

```
lemma counted_rom_collision_termE :  
  counted_rom_collision_term =  
    25%r / challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
counted_rom_collision_term  
hash_query_count  
ring  
rom_hash_query_budget  
rom_signature_query_budget  
rom_total_query_budget  
signature_query_count
```

### Theorem 4.25 (EasyCrypt line 263). *EasyCrypt* name.

```
counted_rom_prequery_termE
```

### Checked EasyCrypt statement.

```
lemma counted_rom_prequery_termE :  
  counted_rom_prequery_term =  
    6%r / challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
counted_rom_prequery_term  
hash_query_count  
ring  
rom_hash_query_budget  
rom_signature_query_budget  
signature_query_count
```

### Theorem 4.26 (EasyCrypt line 274). *EasyCrypt* name.

```
counted_rom_min_entropy_termE
```

### Checked EasyCrypt statement.

```
lemma counted_rom_min_entropy_termE :  
  counted_rom_min_entropy_term =  
    6%r / challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:

```
counted_rom_min_entropy_term
hash_query_count
ring
rom_hash_query_budget
rom_signature_query_budget
signature_query_count
```

**Theorem 4.27** (EasyCrypt line 285). *EasyCrypt name.*

```
counted_rom_prequery_reprogramming_termE
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_prequery_reprogramming_termE :
  counted_rom_prequery_reprogramming_term =
    31%r / challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:

```
counted_rom_collision_termE
counted_rom_prequery_reprogramming_term
counted_rom_prequery_termE
ring
```

**Theorem 4.28** (EasyCrypt line 294). *EasyCrypt name.*

```
counted_rom_programming_loss_termE
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_programming_loss_termE :
  counted_rom_programming_loss_term =
    37%r / challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
counted_rom_collision_termE
counted_rom_min_entropy_termE
counted_rom_prequery_termE
counted_rom_programming_loss_term
ring
```

**Theorem 4.29** (EasyCrypt line 304). *EasyCrypt name*.

```
counted_rom_programming_loss_term_subunit
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_programming_loss_term_subunit :
  counted_rom_programming_loss_term < 1%r.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
challenge_support_cardinality_lower_bound
challenge_support_cardinality_lower_bound_positive
counted_rom_programming_loss_termE
ltr_pdivr_mulr
```

**Theorem 4.30** (EasyCrypt line 313). *EasyCrypt name*.

```
counted_rom_programming_loss_term_positive
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_programming_loss_term_positive :
  0%r < counted_rom_programming_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
challenge_support_cardinality_lower_bound_positive
counted_rom_programming_loss_termE
divr_gt0
```

**Theorem 4.31** (EasyCrypt line 322). *EasyCrypt name*.

```
counted_rom_programming_loss_term_le1
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_programming_loss_term_le1 :  
  1%r >= counted_rom_programming_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
counted_rom_programming_loss_term_subunit  
ltrW
```

**Theorem 4.32 (EasyCrypt line 328). EasyCrypt name.**

```
counted_rom_prequery_reprogramming_budget_numeratorE
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_prequery_reprogramming_budget_numeratorE :  
  (rom_total_query_budget * rom_total_query_budget) +  
  (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) =  
  31%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
hash_query_count  
ring  
rom_hash_query_budget  
rom_signature_query_budget  
rom_total_query_budget  
signature_query_count
```

**Theorem 4.33 (EasyCrypt line 340). EasyCrypt name.**

```
counted_rom_budget_numeratorE
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_budget_numeratorE :  
  (rom_total_query_budget * rom_total_query_budget) +  
  (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) +  
  (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) =  
  37%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.



**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
hash_query_count
ring
rom_hash_query_budget
rom_signature_query_budget
rom_total_query_budget
signature_query_count
```

**Theorem 4.34** (EasyCrypt line 353). *EasyCrypt* name.

```
counted_rom_programming_loss_term_cardinalityE
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_programming_loss_term_cardinalityE :
  counted_rom_programming_loss_term =
    ((rom_total_query_budget * rom_total_query_budget) +
     (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) +
     (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) /
    challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
counted_rom_budget_numeratorE
counted_rom_programming_loss_termE
```

**Theorem 4.35** (EasyCrypt line 364). *EasyCrypt* name.

```
counted_rom_prequery_reprogramming_term_cardinalityE
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_prequery_reprogramming_term_cardinalityE :
  counted_rom_prequery_reprogramming_term =
    ((rom_total_query_budget * rom_total_query_budget) +
     (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) /
    challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
counted_rom_prequery_reprogramming_budget_numeratorE
counted_rom_prequery_reprogramming_termE
```

**Theorem 4.36** (EasyCrypt line 374). *EasyCrypt name*.

```
counted_rom_programming_loss_term_concreteE
```

*Checked EasyCrypt statement.*

```
lemma counted_rom_programming_loss_term_concreteE :
  counted_rom_programming_loss_term =
    37%r / 288230376151711744%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
challenge_support_cardinality_lower_bound
counted_rom_programming_loss_termE
```

**Theorem 4.37** (EasyCrypt line 382). *EasyCrypt name*.

```
counted_rom_prequery_reprogramming_term_concreteE
```

*Checked EasyCrypt statement.*

```
lemma counted_rom_prequery_reprogramming_term_concreteE :
  counted_rom_prequery_reprogramming_term =
    31%r / 288230376151711744%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
challenge_support_cardinality_lower_bound
counted_rom_prequery_reprogramming_termE
```

**Theorem 4.38** (EasyCrypt line 390). *EasyCrypt name*.

```
counted_rom_programming_loss_term_checked_subunit
```

*Checked EasyCrypt statement.*

```
lemma counted_rom_programming_loss_term_checked_subunit :
  0%r < counted_rom_programming_loss_term /\
  counted_rom_programming_loss_term < 1%r /\
  counted_rom_programming_loss_term =
  37%r / 288230376151711744%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
counted_rom_programming_loss_term_concreteE
counted_rom_programming_loss_term_positive
counted_rom_programming_loss_term_subunit
```

**Theorem 4.39** (EasyCrypt line 403). *EasyCrypt name.*

```
counted_rom_support_lower_bound_summary
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_support_lower_bound_summary :
  challenge_support_bits_lower_bound = 58 /\
  challenge_support_cardinality_lower_bound =
  288230376151711744%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *split*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
challenge_support_bits_lower_bound
challenge_support_cardinality_lower_bound
```

**Theorem 4.40** (EasyCrypt line 413). *EasyCrypt name.*

```
counted_rom_support_dominates_budget
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_support_dominates_budget :
  37%r < challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 4.41** (EasyCrypt line 417). *EasyCrypt name.*

```
counted_rom_prequery_reprogramming_support_dominates_budget
```

*Checked EasyCrypt statement.*

```
lemma counted_rom_prequery_reprogramming_support_dominates_budget :  
  31%r < challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 4.42** (EasyCrypt line 421). *EasyCrypt name.*

```
counted_rom_prequery_reprogramming_term_subunit
```

*Checked EasyCrypt statement.*

```
lemma counted_rom_prequery_reprogramming_term_subunit :  
  counted_rom_prequery_reprogramming_term < 1%r.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
challenge_support_cardinality_lower_bound_positive  
counted_rom_prequery_reprogramming_support_dominates_budget  
counted_rom_prequery_reprogramming_termE  
ltr_pdivr_mulr
```

**Theorem 4.43** (EasyCrypt line 430). *EasyCrypt name.*

```
counted_rom_prequery_reprogramming_from_query_budgets_and_support
```

*Checked EasyCrypt statement.*

```
lemma counted_rom_prequery_reprogramming_from_query_budgets_and_support :  
  counted_rom_prequery_reprogramming_term =  
    ((rom_total_query_budget * rom_total_query_budget) +  
     (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) /  
    challenge_support_cardinality_lower_bound /\  
  ((rom_total_query_budget * rom_total_query_budget) +  
   (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) <  
  challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *split*, *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
counted_rom_prequery_reprogramming_budget_numeratorE
counted_rom_prequery_reprogramming_support_dominates_budget
counted_rom_prequery_reprogramming_term_cardinalityE
```

**Theorem 4.44** (EasyCrypt line 445). *EasyCrypt name.*

```
counted_rom_programming_loss_splitE
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_programming_loss_splitE :
  counted_rom_programming_loss_term =
    counted_rom_prequery_reprogramming_term +
    counted_rom_min_entropy_term.
```

**Formal mathematical reading.** *Mathematical theorem.* This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
counted_rom_prequery_reprogramming_term
counted_rom_programming_loss_term
ring
```

**Theorem 4.45** (EasyCrypt line 454). *EasyCrypt name.*

```
counted_rom_loss_from_query_budgets_and_support
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_loss_from_query_budgets_and_support :
  counted_rom_programming_loss_term =
    ((rom_total_query_budget * rom_total_query_budget) +
     (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) +
     (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) /
    challenge_support_cardinality_lower_bound /\
    ((rom_total_query_budget * rom_total_query_budget) +
     (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) +
     (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) <
    challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** *Mathematical theorem.* This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *split*, *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
counted_rom_budget_numeratorE
counted_rom_programming_loss_term_cardinalityE
counted_rom_support_dominates_budget
```

**Theorem 4.46** (EasyCrypt line 471). *EasyCrypt name.*

```
rejection_sampling_loss_term_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma rejection_sampling_loss_term_nonnegative :
  0%r <= rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
addr_ge0
fs_with_aborts_min_entropy_term_nonnegative
fs_with_aborts_reprogramming_term_nonnegative
rejection_sampling_loss_term
```

**Theorem 4.47** (EasyCrypt line 479). *EasyCrypt name.*

```
fs_with_aborts_min_entropy_termE
```

**Checked EasyCrypt statement.**

```
lemma fs_with_aborts_min_entropy_termE :
  fs_with_aborts_min_entropy_term = 12%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
fs_with_aborts_min_entropy_term
hash_query_count
min_entropy_loss_factor
ring
signature_query_count
signing_query_scaleE
```

**Theorem 4.48** (EasyCrypt line 486). *EasyCrypt name.*

```
fs_with_aborts_min_entropy_term_ge1
```

**Checked EasyCrypt statement.**

```
lemma fs_with_aborts_min_entropy_term_ge1 :  
  1%r <= fs_with_aborts_min_entropy_term.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
fs_with_aborts_min_entropy_termE
```

**Theorem 4.49 (EasyCrypt line 492). EasyCrypt name.**

```
rejection_sampling_loss_term_ge1
```

**Checked EasyCrypt statement.**

```
lemma rejection_sampling_loss_term_ge1 :  
  1%r <= rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
fs_with_aborts_min_entropy_term_ge1  
fs_with_aborts_reprogramming_term_nonnegative  
rejection_sampling_loss_term
```

**Theorem 4.50 (EasyCrypt line 500). EasyCrypt name.**

```
bimodal_to_msis_reduction_loss_termE
```

**Checked EasyCrypt statement.**

```
lemma bimodal_to_msis_reduction_loss_termE :  
  bimodal_to_msis_reduction_loss_term = 0%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

End of generated formal inventory for *HAETAE\_Reductions.ec*.

## Chapter 5

### HAETAE\_ROM.ec

**Mathematical role.** This file specializes the generic random oracle to HAETAE. The domain is a sum type with four query classes:

$$\begin{array}{ll} \text{MessageHashQuery}(pk, ctx, m), & \text{ChallengeHashQuery}(md, w_1, w_0, \mu), \\ \text{MatrixExpandQuery}(md, \rho), & \text{SamplerExpandQuery}(coins). \end{array}$$

The codomain is a product

$$\text{ro\_output} = \text{crh} \times \text{challenge} \times \text{matrix} \times \text{random\_coins}.$$

**Typed distributions.** The distribution `dro_output` chooses the relevant component according to the query class. A message-hash query samples a `crh`, a challenge query samples a `challenge`, a matrix-expansion query samples a `matrix`, and a sampler-expansion query samples signing coins. The unused components are set to fixed zero values.

**Cryptographic reading.** This typed oracle prevents category mistakes. The same lazy random oracle machinery is used throughout the proof, but the type of the query determines which cryptographic object the answer represents. The lemma `sampler_expand_query_dro_output_ro_signing_coins` states that reading the coin component from a sampler-expansion output has exactly the intended signing-coin distribution.

#### Textbook Formal Inventory

This subsection records the exact formal material in `HAETAE_ROM.ec`. It is generated from the proof-surface EasyCrypt file, so the line numbers and statements are meant to be read next to the checked source.

**Chapter role.** typed random-oracle query/output domains.

**Inventory size.** 7 context declarations, 27 definitions/game objects, 3 lemmas or relational theorems, and 0 explicit Hoare/Phoare judgments were found.

#### Theory Context, Imports, and Quantified Modules

**Context 1 (line 1)** EasyCrypt name.

```
imported theories
```

Checked EasyCrypt statement.



```
require import AllCore Distr List PROM.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 2 (line 2)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import HAETAE_Params HAETAE_Algebra HAETAE_Distributions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 3 (line 4)** EasyCrypt name.

```
HAETAE_ROM
```

**Checked EasyCrypt statement.**

```
theory HAETAE_ROM.
```

**Formal mathematical reading.** Mathematical context. This begins a namespace for the formal development in this file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 4 (line 6)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Algebra.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 5 (line 7)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Distributions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 6 (line 8) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Params.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 7 (line 80) EasyCrypt name.**

```
cloned theory
```

**Checked EasyCrypt statement.**

```
clone import FullRO as HAETAE_RO with
  type in_t <- ro_query,
  type out_t <- ro_output,
  op dout <- dro_output.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

## Definitions, Carrier Sets, Operations, Games, and Procedures

**Definition 5.1** (EasyCrypt line 10). **EasyCrypt name.**

```
ro_query
```

**Checked EasyCrypt statement.**

```
type ro_query =
  [ MessageHashQuery of (pkey * context * message)
  | ChallengeHashQuery of (mode * polyveck * poly * crh)
  | MatrixExpandQuery of (mode * seed)
  | SamplerExpandQuery of random_coins ].
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 5.2** (EasyCrypt line 15). **EasyCrypt name.**

```
ro_output
```

### Checked EasyCrypt statement.

```
type ro_output = crh * challenge * matrix * random_coins.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 5.3 (EasyCrypt line 17). EasyCrypt name.

```
message_hash_query
```

### Checked EasyCrypt statement.

```
op message_hash_query (pk : pkey) (ctx : context) (m : message) : ro_query =  
  MessageHashQuery (pk, ctx, m).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 5.4 (EasyCrypt line 19). EasyCrypt name.

```
challenge_hash_query
```

### Checked EasyCrypt statement.

```
op challenge_hash_query  
  (md : mode) (w1 : polyveck) (w0 : poly) (mu : crh) : ro_query =  
  ChallengeHashQuery (md, w1, w0, mu).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 5.5 (EasyCrypt line 22). EasyCrypt name.

```
matrix_expand_query
```

### Checked EasyCrypt statement.

```
op matrix_expand_query (md : mode) (sd : seed) : ro_query =  
  MatrixExpandQuery (md, sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 5.6 (EasyCrypt line 24). EasyCrypt name.

```
sampler_expand_query
```

### Checked EasyCrypt statement.

```
op sampler_expand_query (coins : random_coins) : ro_query =
  SamplerExpandQuery coins.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 5.7** (EasyCrypt line 27). **EasyCrypt name.**

```
ro_crh_zero
```

**Checked EasyCrypt statement.**

```
op ro_crh_zero : crh = nseq crhbytes 0.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 5.8** (EasyCrypt line 28). **EasyCrypt name.**

```
ro_challenge_zero
```

**Checked EasyCrypt statement.**

```
op ro_challenge_zero : challenge = poly_zero.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 5.9** (EasyCrypt line 29). **EasyCrypt name.**

```
ro_matrix_zero
```

**Checked EasyCrypt statement.**

```
op ro_matrix_zero : matrix = [].
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 5.10** (EasyCrypt line 30). **EasyCrypt name.**

```
ro_random_coins_zero
```

**Checked EasyCrypt statement.**

```
op ro_random_coins_zero : random_coins = nseq seedbytes 0.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 5.11** (EasyCrypt line 31). **EasyCrypt name.**

```
ro_output_zero
```

**Checked EasyCrypt statement.**

```
op ro_output_zero : ro_output =  
  (ro_crh_zero, ro_challenge_zero, ro_matrix_zero, ro_random_coins_zero).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 5.12** (EasyCrypt line 34). **EasyCrypt name.**

```
ro_output_to_crh
```

**Checked EasyCrypt statement.**

```
op ro_output_to_crh (y : ro_output) : crh = y.'1.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 5.13** (EasyCrypt line 35). **EasyCrypt name.**

```
ro_output_to_challenge
```

**Checked EasyCrypt statement.**

```
op ro_output_to_challenge (y : ro_output) : challenge = y.'2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 5.14** (EasyCrypt line 36). **EasyCrypt name.**

```
ro_output_to_matrix
```

**Checked EasyCrypt statement.**

```
op ro_output_to_matrix (y : ro_output) : matrix = y.'3.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 5.15** (EasyCrypt line 37). **EasyCrypt name.**

```
ro_output_to_random_coins
```

**Checked EasyCrypt statement.**

```
op ro_output_to_random_coins (y : ro_output) : random_coins = y.'4.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 5.16** (EasyCrypt line 39). **EasyCrypt name.**

```
ro_output_of_crh
```

**Checked EasyCrypt statement.**

```
op ro_output_of_crh (mu : crh) : ro_output =  
  (mu, ro_challenge_zero, ro_matrix_zero, ro_random_coins_zero).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 5.17** (EasyCrypt line 41). **EasyCrypt name.**

```
ro_output_of_challenge
```

**Checked EasyCrypt statement.**

```
op ro_output_of_challenge (ch : challenge) : ro_output =  
  (ro_crh_zero, ch, ro_matrix_zero, ro_random_coins_zero).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 5.18** (EasyCrypt line 43). **EasyCrypt name.**

```
ro_output_of_matrix
```

**Checked EasyCrypt statement.**

```
op ro_output_of_matrix (a : matrix) : ro_output =  
  (ro_crh_zero, ro_challenge_zero, a, ro_random_coins_zero).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 5.19** (EasyCrypt line 45). **EasyCrypt name.**

```
ro_output_of_random_coins
```

**Checked EasyCrypt statement.**

```
op ro_output_of_random_coins (coins : random_coins) : ro_output =  
  (ro_crh_zero, ro_challenge_zero, ro_matrix_zero, coins).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 5.20** (EasyCrypt line 48). **EasyCrypt name.**

```
dmatrix_for_query
```

**Checked EasyCrypt statement.**

```
op dmatrix_for_query (x : mode * seed) : matrix distr = dmatrix x.'1.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 5.21** (EasyCrypt line 50). **EasyCrypt name.**

```
dro_output
```

**Checked EasyCrypt statement.**

```
op dro_output (q : ro_query) : ro_output distr =
  with q = MessageHashQuery _ => dmap dcrh ro_output_of_crh
  with q = ChallengeHashQuery x =>
    dmap (dchallenge x.'1) ro_output_of_challenge
  with q = MatrixExpandQuery x =>
    dmap (dmatrix_for_query x) ro_output_of_matrix
  with q = SamplerExpandQuery _ =>
    dmap drandom_coins ro_output_of_random_coins.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 5.22** (EasyCrypt line 85). **EasyCrypt name.**

```
Oracle
```

**Checked EasyCrypt statement.**

```
module type Oracle = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It defines the oracle boundary through which adversaries and simulations interact.

**Definition 5.23** (EasyCrypt line 89). **EasyCrypt name.**

```
POracle
```

**Checked EasyCrypt statement.**

```
module type POracle = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It defines the oracle boundary through which adversaries and simulations interact.

**Definition 5.24** (EasyCrypt line 93). **EasyCrypt name.**

```
ro_message_hash
```

**Checked EasyCrypt statement.**

```
op ro_message_hash (y : ro_output) : crh = ro_output_to_crh y.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 5.25** (EasyCrypt line 94). **EasyCrypt name.**

```
ro_challenge_hash
```

**Checked EasyCrypt statement.**

```
op ro_challenge_hash (y : ro_output) : challenge = ro_output_to_challenge y.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 5.26** (EasyCrypt line 95). **EasyCrypt name.**

```
ro_matrix
```

**Checked EasyCrypt statement.**

```
op ro_matrix (y : ro_output) : matrix = ro_output_to_matrix y.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 5.27** (EasyCrypt line 96). **EasyCrypt name.**

```
ro_signing_coins
```

**Checked EasyCrypt statement.**

```
op ro_signing_coins (y : ro_output) : random_coins =  
  ro_output_to_random_coins y.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.



## Lemmas, Theorems, Relational Equivalences, and Their Proof Dependencies

**Theorem 5.1** (EasyCrypt line 59). *EasyCrypt name.*

```
matrix_query_distribution_lossless
```

*Checked EasyCrypt statement.*

```
lemma matrix_query_distribution_lossless x :  
  is_lossless (dmatrix_for_query x).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
dmatrix_for_query  
matrix_distribution_lossless  
md  
sd
```

**Theorem 5.2** (EasyCrypt line 67). *EasyCrypt name.*

```
ro_output_distribution_lossless
```

*Checked EasyCrypt statement.*

```
lemma ro_output_distribution_lossless q :  
  is_lossless (dro_output q).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
challenge_distribution_lossless  
crh_distribution_lossless  
dmap_ll  
dro_output  
matrix_query_distribution_lossless  
signing_coin_distribution_lossless
```

**Theorem 5.3** (EasyCrypt line 99). *EasyCrypt name.*

```
sampler_expand_query_dro_output_ro_signing_coins
```

*Checked EasyCrypt statement.*

```
lemma sampler_expand_query_dro_output_ro_signing_coins  
  seed_coins (p : random_coins -> bool) :
```

```
mu (dro_output (sampler_expand_query seed_coins))
  (fun y => p (ro_signing_coins y)) =
mu drandom_coins p.
```

**Formal mathematical reading.** *Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.*

**Role in the proof.** *Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **rewrite**, **apply**. Named identifiers syntactically cited in the proof block:*

```
coins
dmapE
dro_output
mu_eq
ro_output_of_random_coins
ro_output_to_random_coins
ro_signing_coins
sampler_expand_query
```

*End of generated formal inventory for HAETAE\_ROM. ec.*

## Chapter 6

# HAETAE\_ROM\_Programming.ec

**Mathematical role.** This file formalizes the events involved in programming the random oracle at signature challenge points. It is the probability-theoretic core behind the counted-ROM loss.

**Programming sites.** A programming site is a pair

$$(q, y) \in \text{ro\_query} \times \text{ro\_output}.$$

For a transcript  $tr$ , the file defines the challenge query and the output that would make the transcript consistent. The key questions are:

- was the query already asked by the adversary?
- does programming the site conflict with an old oracle value?
- does the challenge query have enough min-entropy?

**Bad events.** The central bad events are reprogramming failure, prequery failure, and min-entropy failure. Their counted version is summarized by `rom_counted_programming_bad`. In mathematical notation:

$$\Pr[\text{Bad}] \leq \Delta_{\text{collision}} + \Delta_{\text{prequery}} + \Delta_{\text{entropy}}.$$

**Counted lazy oracle.** The module `CountedLazyROM` extends the lazy random oracle with logs and counters. The preservation lemmas show that initialization clears the logs, nonchallenge oracle calls do not increase the challenge-query count, fresh programming preserves a clear state, and a clear state implies the bad flag is false. These are the formal ingredients behind the informal statement “the simulation is perfect unless the adversary hits a programmed challenge point.”

### Textbook Formal Inventory

This subsection records the exact formal material in `HAETAE_ROM_Programming.ec`. It is generated from the proof-surface EasyCrypt file, so the line numbers and statements are meant to be read next to the checked source.

**Chapter role.** lazy ROM programming, counted bad events, and programming bounds.

**Inventory size.** 19 context declarations, 68 definitions/game objects, 99 lemmas or relational theorems, and 0 explicit Hoare/Phoare judgments were found.

## Theory Context, Imports, and Quantified Modules

Context 1 (line 1) EasyCrypt name.

```
imported theories
```

Checked EasyCrypt statement.

```
require import AllCore Real Distr List FSet PROM StdOrder Mu_mem.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

Context 2 (line 2) EasyCrypt name.

```
imported theories
```

Checked EasyCrypt statement.

```
require import HAETAE_Params HAETAE_Algebra HAETAE_Distributions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

Context 3 (line 3) EasyCrypt name.

```
imported theories
```

Checked EasyCrypt statement.

```
require import HAETAE_ROM HAETAE_Transcript.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

Context 4 (line 4) EasyCrypt name.

```
imported theories
```

Checked EasyCrypt statement.

```
require import HAETAE_Rejection.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 5 (line 5)**    **EasyCrypt name.**

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import HAETAE_Reductions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 6 (line 7)**    **EasyCrypt name.**

```
HAETAE_ROM_Programming
```

**Checked EasyCrypt statement.**

```
theory HAETAE_ROM_Programming.
```

**Formal mathematical reading.** Mathematical context. This begins a namespace for the formal development in this file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 7 (line 9)**    **EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Params.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 8 (line 10)**    **EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Algebra.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 9 (line 11)**    **EasyCrypt name.**

```
opened namespace
```

### Checked EasyCrypt statement.

```
import HAETAE_Distributions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 10 (line 12) EasyCrypt name.

```
opened namespace
```

### Checked EasyCrypt statement.

```
import HAETAE_ROM.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 11 (line 13) EasyCrypt name.

```
opened namespace
```

### Checked EasyCrypt statement.

```
import HAETAE_Transcript.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 12 (line 14) EasyCrypt name.

```
opened namespace
```

### Checked EasyCrypt statement.

```
import HAETAE_Rejection.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 13 (line 15) EasyCrypt name.

```
opened namespace
```

### Checked EasyCrypt statement.

```
import HAETAE_Reductions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 14 (line 16) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import RealOrder.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 15 (line 315) EasyCrypt name.**

```
CountedLazyROMStateFacts
```

**Checked EasyCrypt statement.**

```
section CountedLazyROMStateFacts.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 16 (line 317) EasyCrypt name.**

```
0
```

**Checked EasyCrypt statement.**

```
declare module 0 <: Oracle {-CountedLazyROM}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 17 (line 1455) EasyCrypt name.**

```
CountedLazyROMBadFlags
```

**Checked EasyCrypt statement.**

```
section CountedLazyROMBadFlags.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 18 (line 1457)** EasyCrypt name.

```
0
```

**Checked EasyCrypt statement.**

```
declare module 0 <: Oracle.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 19 (line 1458)** EasyCrypt name.

```
A
```

**Checked EasyCrypt statement.**

```
declare module A <: CountedROMClient {-0, -CountedLazyROM, -CountedLazyROMBadGame}.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

## Definitions, Carrier Sets, Operations, Games, and Procedures

**Definition 6.1** (EasyCrypt line 18). EasyCrypt name.

```
programming_site
```

**Checked EasyCrypt statement.**

```
type programming_site = ro_query * ro_output.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.2** (EasyCrypt line 20). EasyCrypt name.

```
programming_site_query
```

**Checked EasyCrypt statement.**

```
op programming_site_query (site : programming_site) : ro_query = site.‘1.
```



**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.3** (EasyCrypt line 21). **EasyCrypt name.**

```
programming_site_output
```

**Checked EasyCrypt statement.**

```
op programming_site_output (site : programming_site) : ro_output = site.'2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.4** (EasyCrypt line 23). **EasyCrypt name.**

```
programming_site_of_transcript
```

**Checked EasyCrypt statement.**

```
op programming_site_of_transcript
  (md : mode) (tr : transcript) (y : ro_output) : programming_site =
  (transcript_challenge_query md tr, y).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 6.5** (EasyCrypt line 27). **EasyCrypt name.**

```
signature_programming_site_of_transcript
```

**Checked EasyCrypt statement.**

```
op signature_programming_site_of_transcript
  (md : mode) (tr : transcript) (y : ro_output) : programming_site =
  (transcript_signature_challenge_query md tr, y).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 6.6** (EasyCrypt line 31). **EasyCrypt name.**

```
transcript_signature_challenge_output
```

**Checked EasyCrypt statement.**

```
op transcript_signature_challenge_output (tr : transcript) : ro_output =
  ro_output_of_challenge (sig_challenge (transcript_signature tr)).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 6.7** (EasyCrypt line 34). **EasyCrypt name.**

```
actual_signature_programming_site
```

**Checked EasyCrypt statement.**

```
op actual_signature_programming_site
  (md : mode) (tr : transcript) : programming_site =
  signature_programming_site_of_transcript md tr
  (transcript_signature_challenge_output tr).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.8** (EasyCrypt line 39). **EasyCrypt name.**

```
honest_signing_programming_site
```

**Checked EasyCrypt statement.**

```
op honest_signing_programming_site
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (coins : random_coins) : programming_site =
  actual_signature_programming_site md
  (transcript_from_honest_signing md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.9** (EasyCrypt line 45). **EasyCrypt name.**

```
honest_signing_programming_site_query
```

**Checked EasyCrypt statement.**

```
op honest_signing_programming_site_query
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (coins : random_coins) : ro_query =
  programming_site_query
  (honest_signing_programming_site md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.10** (EasyCrypt line 51). **EasyCrypt name.**

```
programmed_site_query_min_entropy_bound_for
```

**Checked EasyCrypt statement.**

```
op programmed_site_query_min_entropy_bound_for
```

```

(d : random_coins distr)
(md : mode) (sk : skey) (m : message) (ctx : context)
(qs : ro_query list) (bd : real) : bool =
forall q, q \in qs =>
  mu d
    (fun coins =>
      honest_signing_programming_site_query md sk m ctx coins = q) <= bd.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 6.11** (EasyCrypt line 60). **EasyCrypt name.**

```
programmed_site_query_min_entropy_bound
```

**Checked EasyCrypt statement.**

```

op programmed_site_query_min_entropy_bound
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (qs : ro_query list) (bd : real) : bool =
  programmed_site_query_min_entropy_bound_for drandom_coins md sk m ctx qs bd.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 6.12** (EasyCrypt line 65). **EasyCrypt name.**

```
programming_site_matches_transcript
```

**Checked EasyCrypt statement.**

```

op programming_site_matches_transcript
  (md : mode) (tr : transcript) (site : programming_site) : bool =
  programming_site_query site = transcript_challenge_query md tr /\
  transcript_challenge_matches tr (programming_site_output site).

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 6.13** (EasyCrypt line 70). **EasyCrypt name.**

```
reprogramming_failure
```

**Checked EasyCrypt statement.**

```

op reprogramming_failure
  (md : mode) (tr : transcript) (site : programming_site) : bool =
  ! programming_site_matches_transcript md tr site.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

**Definition 6.14** (EasyCrypt line 74). **EasyCrypt name.**

```
challenge_entropy_support_ok
```

**Checked EasyCrypt statement.**

```
op challenge_entropy_support_ok (md : mode) (tr : transcript) : bool =
  challenge_sparse md (transcript_challenge tr).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 6.15** (EasyCrypt line 77). **EasyCrypt name.**

```
min_entropy_failure
```

**Checked EasyCrypt statement.**

```
op min_entropy_failure (md : mode) (tr : transcript) : bool =
  ! challenge_entropy_support_ok md tr.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

**Definition 6.16** (EasyCrypt line 80). **EasyCrypt name.**

```
transcript_log_min_entropy_clear
```

**Checked EasyCrypt statement.**

```
op transcript_log_min_entropy_clear (md : mode) (trs : transcript list) : bool =
  all (fun tr => ! min_entropy_failure md tr) trs.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 6.17** (EasyCrypt line 83). **EasyCrypt name.**

```
signing_record_log_min_entropy_clear
```

**Checked EasyCrypt statement.**

```

op signing_record_log_min_entropy_clear
  (md : mode) (rs : signing_transcript_record list) : bool =
  all (fun r => ! min_entropy_failure md (signing_record_transcript r)) rs.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.18** (EasyCrypt line 87). **EasyCrypt name.**

```

fs_with_aborts_failure

```

**Checked EasyCrypt statement.**

```

op fs_with_aborts_failure
  (md : mode) (tr : transcript) (site : programming_site) : bool =
  reprogramming_failure md tr site \/ min_entropy_failure md tr.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

**Definition 6.19** (EasyCrypt line 91). **EasyCrypt name.**

```

transcript_signature_programming_site_clear

```

**Checked EasyCrypt statement.**

```

op transcript_signature_programming_site_clear
  (md : mode) (tr : transcript) : bool =
  ! fs_with_aborts_failure md tr (actual_signature_programming_site md tr).

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 6.20** (EasyCrypt line 95). **EasyCrypt name.**

```

transcript_log_signature_programming_sites_clear

```

**Checked EasyCrypt statement.**

```

op transcript_log_signature_programming_sites_clear
  (md : mode) (trs : transcript list) : bool =
  all (transcript_signature_programming_site_clear md) trs.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 6.21** (EasyCrypt line 99). **EasyCrypt name.**

```
fs_with_aborts_bound_obligation
```

**Checked EasyCrypt statement.**

```
op fs_with_aborts_bound_obligation : bool =
  0%r <= fs_with_aborts_reprogramming_term /\
  0%r <= fs_with_aborts_min_entropy_term.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 6.22** (EasyCrypt line 103). **EasyCrypt name.**

```
counted_rom_event
```

**Checked EasyCrypt statement.**

```
type counted_rom_event =
  [ CountedHashQuery of ro_query
  | CountedSignatureSite of programming_site
  | CountedProgrammedSite of programming_site
  | CountedMinEntropyCheck of transcript ].
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.23** (EasyCrypt line 109). **EasyCrypt name.**

```
counted_event_is_hash_query
```

**Checked EasyCrypt statement.**

```
op counted_event_is_hash_query (ev : counted_rom_event) : bool =
  with ev = CountedHashQuery _ => true
  with ev = CountedSignatureSite _ => false
  with ev = CountedProgrammedSite _ => false
  with ev = CountedMinEntropyCheck _ => false.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.24** (EasyCrypt line 115). **EasyCrypt name.**

```
counted_event_is_signature_site
```

**Checked EasyCrypt statement.**

```
op counted_event_is_signature_site (ev : counted_rom_event) : bool =
  with ev = CountedHashQuery _ => false
```

```

with ev = CountedSignatureSite _ => true
with ev = CountedProgrammedSite _ => false
with ev = CountedMinEntropyCheck _ => false.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.25** (EasyCrypt line 121). **EasyCrypt name.**

```
counted_event_is_programmed_site
```

**Checked EasyCrypt statement.**

```

op counted_event_is_programmed_site (ev : counted_rom_event) : bool =
  with ev = CountedHashQuery _ => false
  with ev = CountedSignatureSite _ => false
  with ev = CountedProgrammedSite _ => true
  with ev = CountedMinEntropyCheck _ => false.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.26** (EasyCrypt line 127). **EasyCrypt name.**

```
counted_event_is_min_entropy_check
```

**Checked EasyCrypt statement.**

```

op counted_event_is_min_entropy_check (ev : counted_rom_event) : bool =
  with ev = CountedHashQuery _ => false
  with ev = CountedSignatureSite _ => false
  with ev = CountedProgrammedSite _ => false
  with ev = CountedMinEntropyCheck _ => true.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.27** (EasyCrypt line 133). **EasyCrypt name.**

```
ro_query_is_challenge
```

**Checked EasyCrypt statement.**

```

op ro_query_is_challenge (q : ro_query) : bool =
  with q = MessageHashQuery _ => false
  with q = ChallengeHashQuery _ => true
  with q = MatrixExpandQuery _ => false
  with q = SamplerExpandQuery _ => false.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.28** (EasyCrypt line 139). **EasyCrypt name.**

```
challenge_query_log_count
```

**Checked EasyCrypt statement.**

```
op challenge_query_log_count (qs : ro_query list) : int =
  card (oflist (filter ro_query_is_challenge qs)).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.29** (EasyCrypt line 193). **EasyCrypt name.**

```
counted_trace_hash_queries
```

**Checked EasyCrypt statement.**

```
op counted_trace_hash_queries (evs : counted_rom_event list) : int =
  size (filter counted_event_is_hash_query evs).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.30** (EasyCrypt line 196). **EasyCrypt name.**

```
counted_trace_signature_sites
```

**Checked EasyCrypt statement.**

```
op counted_trace_signature_sites (evs : counted_rom_event list) : int =
  size (filter counted_event_is_signature_site evs).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.31** (EasyCrypt line 199). **EasyCrypt name.**

```
counted_trace_programmed_sites
```

**Checked EasyCrypt statement.**

```
op counted_trace_programmed_sites (evs : counted_rom_event list) : int =
  size (filter counted_event_is_programmed_site evs).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.



**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.32** (EasyCrypt line 202). **EasyCrypt name.**

```
counted_trace_min_entropy_checks
```

**Checked EasyCrypt statement.**

```
op counted_trace_min_entropy_checks (evs : counted_rom_event list) : int =
  size (filter counted_event_is_min_entropy_check evs).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.33** (EasyCrypt line 205). **EasyCrypt name.**

```
programming_sites_same_query
```

**Checked EasyCrypt statement.**

```
op programming_sites_same_query
  (left right : programming_site) : bool =
  programming_site_query left = programming_site_query right.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.34** (EasyCrypt line 209). **EasyCrypt name.**

```
programming_sites_conflict
```

**Checked EasyCrypt statement.**

```
op programming_sites_conflict
  (left right : programming_site) : bool =
  programming_sites_same_query left right /\
  programming_site_output left <> programming_site_output right.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.35** (EasyCrypt line 214). **EasyCrypt name.**

```
programming_site_prequeried
```

**Checked EasyCrypt statement.**

```
op programming_site_prequeried
  (queries : ro_query list) (site : programming_site) : bool =
  programming_site_query site \in queries.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.36** (EasyCrypt line 218). **EasyCrypt name.**

```
programming_site_reprograms
```

**Checked EasyCrypt statement.**

```
op programming_site_reprograms
  (programmed : programming_site list) (site : programming_site) : bool =
  has (fun old => programming_sites_conflict site old) programmed.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.37** (EasyCrypt line 222). **EasyCrypt name.**

```
programming_site_fresh
```

**Checked EasyCrypt statement.**

```
op programming_site_fresh
  (queries : ro_query list) (programmed : programming_site list)
  (site : programming_site) : bool =
  ! programming_site_prequeried queries site /\
  ! programming_site_reprograms programmed site.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.38** (EasyCrypt line 228). **EasyCrypt name.**

```
programming_transcript_fresh
```

**Checked EasyCrypt statement.**

```
op programming_transcript_fresh
  (md : mode) (queries : ro_query list)
  (programmed : programming_site list) (tr : transcript) : bool =
  programming_site_fresh queries programmed
  (actual_signature_programming_site md tr) /\
  ! min_entropy_failure md tr.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 6.39** (EasyCrypt line 235). **EasyCrypt name.**

```
rom_counted_programming_bad
```

**Checked EasyCrypt statement.**

```
op rom_counted_programming_bad
  (queries : ro_query list) (programmed : programming_site list)
  (entropy_bad : bool) : bool =
  entropy_bad \/  
has (fun site => programming_site_prequeried queries site) programmed \/  
has (fun site => programming_site_reprograms programmed site) programmed.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

**Definition 6.40** (EasyCrypt line 242). **EasyCrypt name.**

```
rom_counted_state_bad
```

**Checked EasyCrypt statement.**

```
op rom_counted_state_bad
  (queries : ro_query list) (programmed : programming_site list)
  (bad_min_entropy : bool) : bool =
  rom_counted_programming_bad queries programmed bad_min_entropy.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

**Definition 6.41** (EasyCrypt line 247). **EasyCrypt name.**

```
counted_rom_programming_bound_obligation
```

**Checked EasyCrypt statement.**

```
op counted_rom_programming_bound_obligation : bool =
  0%r <= counted_rom_programming_loss_term.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 6.42** (EasyCrypt line 250). **EasyCrypt name.**

```
CountedLazyROM
```

**Checked EasyCrypt statement.**

```
module CountedLazyROM(O : Oracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the oracle boundary through which adversaries and simulations interact.

**Definition 6.43** (EasyCrypt line 258). **EasyCrypt name.**

```
init
```

**Checked EasyCrypt statement.**

```
proc init() : unit = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 6.44** (EasyCrypt line 268). **EasyCrypt name.**

```
get
```

**Checked EasyCrypt statement.**

```
proc get(q : ro_query) : ro_output = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 6.45** (EasyCrypt line 276). **EasyCrypt name.**

```
program
```

**Checked EasyCrypt statement.**

```
proc program(site : programming_site) : unit = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 6.46** (EasyCrypt line 285). **EasyCrypt name.**

```
observe_signature_site
```

**Checked EasyCrypt statement.**

```
proc observe_signature_site(site : programming_site) : unit = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 6.47** (EasyCrypt line 289). **EasyCrypt name.**

```
observe_transcript
```

**Checked EasyCrypt statement.**

```
proc observe_transcript(md : mode, tr : transcript) : unit = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 6.48** (EasyCrypt line 297). **EasyCrypt name.**

```
bad
```

**Checked EasyCrypt statement.**

```
proc bad() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 6.49** (EasyCrypt line 302). **EasyCrypt name.**

```
SigningCoinSampler
```

**Checked EasyCrypt statement.**

```
module type SigningCoinSampler = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 6.50** (EasyCrypt line 303). **EasyCrypt name.**

```
sample
```

**Checked EasyCrypt statement.**

```
proc sample() : random_coins  
}.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 6.51** (EasyCrypt line 306). **EasyCrypt name.**

```
DirectSigningCoinSampler
```

**Checked EasyCrypt statement.**

```
module DirectSigningCoinSampler : SigningCoinSampler = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 6.52** (EasyCrypt line 307). **EasyCrypt name.**

```
sample
```

**Checked EasyCrypt statement.**

```
proc sample() : random_coins = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It defines the sampling procedure whose distributional behavior is justified by the later loss lemmas.

**Definition 6.53** (EasyCrypt line 501). **EasyCrypt name.**

```
CountedROMInterface
```

**Checked EasyCrypt statement.**

```
module type CountedROMInterface = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It defines the oracle boundary through which adversaries and simulations interact.

**Definition 6.54** (EasyCrypt line 502). **EasyCrypt name.**

```
get
```

**Checked EasyCrypt statement.**

```
proc get(q : ro_query) : ro_output
  proc program(site : programming_site) : unit
  proc observe_signature_site(site : programming_site) : unit
  proc observe_transcript(md : mode, tr : transcript) : unit
}.

```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 6.55** (EasyCrypt line 508). **EasyCrypt name.**

```
CountedROMClient
```

**Checked EasyCrypt statement.**

```
module type CountedROMClient(C : CountedROMInterface) = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It defines the oracle boundary through which adversaries and simulations interact.

**Definition 6.56** (EasyCrypt line 509). **EasyCrypt name.**

```
run
```

**Checked EasyCrypt statement.**

```
proc run() : unit {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 6.57** (EasyCrypt line 517). **EasyCrypt name.**

```
CountedLazyROMBadGame
```

**Checked EasyCrypt statement.**

```
module CountedLazyROMBadGame(
  O : Oracle,
  A : CountedROMClient
) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 6.58** (EasyCrypt line 521). **EasyCrypt name.**

```
C
```

**Checked EasyCrypt statement.**

```
module C = CountedLazyROM(O)
  module A = A(C)

  proc main() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 6.59** (EasyCrypt line 534). **EasyCrypt name.**

```
ProgramChallenge
```

**Checked EasyCrypt statement.**

```
module ProgramChallenge(O : Oracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 6.60** (EasyCrypt line 535). **EasyCrypt name.**

```
program
```

**Checked EasyCrypt statement.**

```
proc program(site : programming_site) : unit = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 6.61** (EasyCrypt line 540). **EasyCrypt name.**

```
TranscriptProgrammer
```

**Checked EasyCrypt statement.**

```
module type TranscriptProgrammer(O : Oracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 6.62** (EasyCrypt line 541). **EasyCrypt name.**

```
program
```

**Checked EasyCrypt statement.**

```
proc program(md : mode, tr : transcript, y : ro_output) : unit  
}.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.



**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 6.63** (EasyCrypt line 544). **EasyCrypt name.**

```
TranscriptProgrammerFromSite
```

**Checked EasyCrypt statement.**

```
module TranscriptProgrammerFromSite(0 : Oracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the oracle boundary through which adversaries and simulations interact.

**Definition 6.64** (EasyCrypt line 545). **EasyCrypt name.**

```
program
```

**Checked EasyCrypt statement.**

```
proc program(md : mode, tr : transcript, y : ro_output) : unit = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 6.65** (EasyCrypt line 611). **EasyCrypt name.**

```
programmed_site_entropy_token_from_query
```

**Checked EasyCrypt statement.**

```
op programmed_site_entropy_token_from_query (q : ro_query) : int =
  with q = ChallengeHashQuery x => nth 0 x.'3 0
  with q = MessageHashQuery _ => 0
  with q = MatrixExpandQuery _ => 0
  with q = SamplerExpandQuery _ => 0.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 6.66** (EasyCrypt line 884). **EasyCrypt name.**

```
programming_site_conflict_free
```

**Checked EasyCrypt statement.**

```
op programming_site_conflict_free
  (site : programming_site) (sites : programming_site list) : bool =
  all (fun old => ! programming_sites_conflict site old) sites.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.67** (EasyCrypt line 888). **EasyCrypt name.**

```
programming_site_log_conflict_free_for_honest
```

**Checked EasyCrypt statement.**

```
op programming_site_log_conflict_free_for_honest
  (md : mode) (sk : skey) (sites : programming_site list) : bool =
  forall m ctx coins,
    programming_site_conflict_free
      (honest_signing_programming_site md sk m ctx coins) sites.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 6.68** (EasyCrypt line 1460). **EasyCrypt name.**

```
counted_lazy_rom_bad_flags_budget_sound
```

**Checked EasyCrypt statement.**

```
op counted_lazy_rom_bad_flags_budget_sound (p : real) : bool =
  p <= counted_rom_programming_loss_term.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

## Lemmas, Theorems, Relational Equivalences, and Their Proof Dependencies

**Theorem 6.1** (EasyCrypt line 142). **EasyCrypt name.**

```
challenge_query_log_count_cons_le
```

**Checked EasyCrypt statement.**

```
lemma challenge_query_log_count_cons_le q qs :
  challenge_query_log_count (q :: qs) <=
  challenge_query_log_count qs + 1.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *smt*. Named identifiers syntactically cited in the proof block:

```
challenge_query_log_count
fcardUI
fsetUC
oflist_cons
site
```

**Theorem 6.2** (EasyCrypt line 156). *EasyCrypt name.*

```
challenge_query_log_count_nonchallenge_cons
```

**Checked EasyCrypt statement.**

```
lemma challenge_query_log_count_nonchallenge_cons q qs :
  ! ro_query_is_challenge q =>
    challenge_query_log_count (q :: qs) =
    challenge_query_log_count qs.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*. Named identifiers syntactically cited in the proof block:

```
challenge_query_log_count
```

**Theorem 6.3** (EasyCrypt line 169). *EasyCrypt name.*

```
challenge_query_log_count_message_cons
```

**Checked EasyCrypt statement.**

```
lemma challenge_query_log_count_message_cons pk ctx m qs :
  challenge_query_log_count (message_hash_query pk ctx m :: qs) =
  challenge_query_log_count qs.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
challenge_query_log_count_nonchallenge_cons
message_hash_query
ro_query_is_challenge
```

**Theorem 6.4** (EasyCrypt line 177). *EasyCrypt name.*

```
challenge_query_log_count_matrix_cons
```

**Checked EasyCrypt statement.**

```
lemma challenge_query_log_count_matrix_cons md sd qs :
  challenge_query_log_count (matrix_expand_query md sd :: qs) =
    challenge_query_log_count qs.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
challenge_query_log_count_nonchallenge_cons
matrix_expand_query
ro_query_is_challenge
```

**Theorem 6.5** (EasyCrypt line 185). *EasyCrypt name.*

```
challenge_query_log_count_sampler_cons
```

**Checked EasyCrypt statement.**

```
lemma challenge_query_log_count_sampler_cons coins qs :
  challenge_query_log_count (sampler_expand_query coins :: qs) =
    challenge_query_log_count qs.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
challenge_query_log_count_nonchallenge_cons
ro_query_is_challenge
sampler_expand_query
```

**Theorem 6.6** (EasyCrypt line 319). *EasyCrypt name.*

```
counted_lazy_rom_init_clears
```

**Checked EasyCrypt statement.**

```
lemma counted_lazy_rom_init_clears :
  hoare[CountedLazyROM().init :
    true ==>
    ! CountedLazyROM.bad_prequery /\
    ! CountedLazyROM.bad_reprogram /\
    ! CountedLazyROM.bad_min_entropy /\
    CountedLazyROM.programmed_sites = []].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `wp`, `call`.

**Theorem 6.7** (EasyCrypt line 333). *EasyCrypt* name.

```
counted_lazy_rom_init_clears_hash_queries
```

**Checked EasyCrypt statement.**

```
lemma counted_lazy_rom_init_clears_hash_queries :
  hoare[CountedLazyROM(O).init :
    true ==> CountedLazyROM.hash_queries = []].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `wp`, `call`.

**Theorem 6.8** (EasyCrypt line 343). *EasyCrypt* name.

```
counted_lazy_rom_get_preserves_clear
```

**Checked EasyCrypt statement.**

```
lemma counted_lazy_rom_get_preserves_clear :
  hoare[CountedLazyROM(O).get :
    ! CountedLazyROM.bad_prequery /\
    ! CountedLazyROM.bad_reprogram /\
    ! CountedLazyROM.bad_min_entropy /\
    CountedLazyROM.programmed_sites = [] ==>
    ! CountedLazyROM.bad_prequery /\
    ! CountedLazyROM.bad_reprogram /\
    ! CountedLazyROM.bad_min_entropy /\
    CountedLazyROM.programmed_sites = []].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `wp`, `call`.

**Theorem 6.9** (EasyCrypt line 360). *EasyCrypt* name.

```
counted_lazy_rom_get_nonchallenge_preserves_challenge_query_count
```

**Checked EasyCrypt statement.**

```

lemma counted_lazy_rom_get_nonchallenge_preserves_challenge_query_count n :
  hoare[CountedLazyROM(0).get :
    ! ro_query_is_challenge q /\
    challenge_query_log_count CountedLazyROM.hash_queries <= n ==>
    challenge_query_log_count CountedLazyROM.hash_queries <= n].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `wp`, `call`, `smt`.

**Theorem 6.10** (EasyCrypt line 372). *EasyCrypt name.*

```

counted_lazy_rom_get_preserves_bad_clear

```

**Checked EasyCrypt statement.**

```

lemma counted_lazy_rom_get_preserves_bad_clear :
  hoare[CountedLazyROM(0).get :
    ! CountedLazyROM.bad_prequery /\
    ! CountedLazyROM.bad_reprogram /\
    ! CountedLazyROM.bad_min_entropy ==>
    ! CountedLazyROM.bad_prequery /\
    ! CountedLazyROM.bad_reprogram /\
    ! CountedLazyROM.bad_min_entropy].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `proc`, `wp`, `call`.

**Theorem 6.11** (EasyCrypt line 387). *EasyCrypt name.*

```

counted_lazy_rom_get_preserves_min_entropy_clear

```

**Checked EasyCrypt statement.**

```

lemma counted_lazy_rom_get_preserves_min_entropy_clear :
  hoare[CountedLazyROM(0).get :
    ! CountedLazyROM.bad_min_entropy ==>
    ! CountedLazyROM.bad_min_entropy].

```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *proc*, *wp*, *call*.

**Theorem 6.12** (EasyCrypt line 398). *EasyCrypt name*.

```
counted_lazy_rom_observe_transcript_preserves_clear
```

*Checked EasyCrypt statement.*

```
lemma counted_lazy_rom_observe_transcript_preserves_clear :
  hoare[CountedLazyROM(0).observe_transcript :
    ! min_entropy_failure md tr /\
    ! CountedLazyROM.bad_prequery /\
    ! CountedLazyROM.bad_reprogram /\
    ! CountedLazyROM.bad_min_entropy /\
    CountedLazyROM.programmed_sites = [] ==>
    ! CountedLazyROM.bad_prequery /\
    ! CountedLazyROM.bad_reprogram /\
    ! CountedLazyROM.bad_min_entropy /\
    CountedLazyROM.programmed_sites = []].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
entropy_ok
hr
min_ok
preq_ok
programmed_ok
reprog_ok
```

**Theorem 6.13** (EasyCrypt line 417). *EasyCrypt name*.

```
counted_lazy_rom_observe_transcript_preserves_min_entropy_clear
```

*Checked EasyCrypt statement.*

```
lemma counted_lazy_rom_observe_transcript_preserves_min_entropy_clear :
  hoare[CountedLazyROM(0).observe_transcript :
    ! min_entropy_failure md tr /\
    ! CountedLazyROM.bad_min_entropy ==>
    ! CountedLazyROM.bad_min_entropy].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
entropy_ok
hr
min_ok
```

**Theorem 6.14** (EasyCrypt line 430). *EasyCrypt name.*

```
counted_lazy_rom_observe_transcript_preserves_bad_clear
```

**Checked EasyCrypt statement.**

```
lemma counted_lazy_rom_observe_transcript_preserves_bad_clear :
  hoare[CountedLazyROM(0).observe_transcript :
    ! min_entropy_failure md tr /\
    ! CountedLazyROM.bad_prequery /\
    ! CountedLazyROM.bad_reprogram /\
    ! CountedLazyROM.bad_min_entropy ==>
    ! CountedLazyROM.bad_prequery /\
    ! CountedLazyROM.bad_reprogram /\
    ! CountedLazyROM.bad_min_entropy].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
entropy_ok
hr
min_ok
preq_ok
reprog_ok
```

**Theorem 6.15** (EasyCrypt line 447). *EasyCrypt name.*

```
counted_lazy_rom_program_preserves_bad_clear_if_fresh
```

**Checked EasyCrypt statement.**

```
lemma counted_lazy_rom_program_preserves_bad_clear_if_fresh :
  hoare[CountedLazyROM(0).program :
    ! CountedLazyROM.bad_prequery /\
    ! CountedLazyROM.bad_reprogram /\
    ! CountedLazyROM.bad_min_entropy /\
    programming_site_fresh
    CountedLazyROM.hash_queries
    CountedLazyROM.programmed_sites site ==>
    ! CountedLazyROM.bad_prequery /\
    ! CountedLazyROM.bad_reprogram /\
    ! CountedLazyROM.bad_min_entropy].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.



**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*.

**Theorem 6.16** (EasyCrypt line 465). *EasyCrypt name*.

```
counted_lazy_rom_program_preserves_min_entropy_clear
```

**Checked EasyCrypt statement.**

```
lemma counted_lazy_rom_program_preserves_min_entropy_clear :
  hoare[CountedLazyROM(0).program :
    ! CountedLazyROM.bad_min_entropy ==>
    ! CountedLazyROM.bad_min_entropy].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *wp*, *call*.

**Theorem 6.17** (EasyCrypt line 476). *EasyCrypt name*.

```
counted_lazy_rom_bad_preserves_min_entropy_clear
```

**Checked EasyCrypt statement.**

```
lemma counted_lazy_rom_bad_preserves_min_entropy_clear :
  hoare[CountedLazyROM(0).bad :
    ! CountedLazyROM.bad_min_entropy ==>
    ! CountedLazyROM.bad_min_entropy].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*.

**Theorem 6.18** (EasyCrypt line 486). *EasyCrypt name*.

```
counted_lazy_rom_bad_false_when_clear
```

**Checked EasyCrypt statement.**

```
lemma counted_lazy_rom_bad_false_when_clear :
  hoare[CountedLazyROM(0).bad :
    ! CountedLazyROM.bad_prequery /\
    ! CountedLazyROM.bad_reprogram /\
    ! CountedLazyROM.bad_min_entropy ==>
    ! res].
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *proc*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
entropy_ok
hr
preq_ok
reprog_ok
```

**Theorem 6.19** (EasyCrypt line 550). *EasyCrypt name*.

```
programming_site_self_matches
```

**Checked EasyCrypt statement.**

```
lemma programming_site_self_matches md tr y :
  transcript_challenge_matches tr y =>
    programming_site_matches_transcript md tr
      (programming_site_of_transcript md tr y).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
programming_site_matches_transcript
programming_site_of_transcript
programming_site_output
programming_site_query
```

**Theorem 6.20** (EasyCrypt line 560). *EasyCrypt name*.

```
programming_site_self_no_reprogramming_failure
```

**Checked EasyCrypt statement.**

```
lemma programming_site_self_no_reprogramming_failure md tr y :
  transcript_challenge_matches tr y =>
    ! reprogramming_failure md tr (programming_site_of_transcript md tr y).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
match_y
md
programming_site_self_matches
reprogramming_failure
tr
```

**Theorem 6.21** (EasyCrypt line 569). *EasyCrypt name*.

```
fs_with_aborts_failureE
```

**Checked EasyCrypt statement.**

```
lemma fs_with_aborts_failureE md tr site :
  fs_with_aborts_failure md tr site =
    (reprogramming_failure md tr site \/ min_entropy_failure md tr).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 6.22** (EasyCrypt line 574). *EasyCrypt name*.

```
programming_site_prequeried_cons
```

**Checked EasyCrypt statement.**

```
lemma programming_site_prequeried_cons qs site :
  programming_site_prequeried (programming_site_query site :: qs) site.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 6.23** (EasyCrypt line 578). *EasyCrypt name*.

```
programming_site_prequeried_cons_preserve
```

**Checked EasyCrypt statement.**

```
lemma programming_site_prequeried_cons_preserve q qs site :
  programming_site_prequeried qs site =>
    programming_site_prequeried (q :: qs) site.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 6.24** (EasyCrypt line 583). *EasyCrypt name*.

```
programming_site_prequeried_output_irrelevant
```

**Checked EasyCrypt statement.**

```
lemma programming_site_prequeried_output_irrelevant qs q y1 y2 :  
  programming_site_prequeried qs (q, y1) =  
  programming_site_prequeried qs (q, y2).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 6.25** (EasyCrypt line 588). **EasyCrypt name.**

```
programming_site_prequeried_witness
```

**Checked EasyCrypt statement.**

```
lemma programming_site_prequeried_witness qs q y :  
  q \in qs =>  
  programming_site_prequeried qs (q, y).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 6.26** (EasyCrypt line 593). **EasyCrypt name.**

```
honest_signing_programming_site_queryE
```

**Checked EasyCrypt statement.**

```
lemma honest_signing_programming_site_queryE md sk m ct\ coins :  
  honest_signing_programming_site_query md sk m ct\ coins =  
  challenge_hash_query md  
    (commitment_highbits md sk m ct\ coins)  
    (commitment_lowbits md sk m ct\ coins)  
    (message_hash (public_key_of_secret md sk) ct\ m).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
actual_signature_programming_site  
honest_signing_programming_site  
honest_signing_programming_site_query  
programming_site_query  
sig_commitment_highbits  
sig_commitment_lowbits  
sig_message_hash  
sign_internal  
signature_programming_site_of_transcript
```

```
transcript_from_honest_signing
transcript_of_signature
```

**Theorem 6.27** (EasyCrypt line 617). *EasyCrypt name.*

```
honest_signing_programming_site_query_entropy_token
```

*Checked EasyCrypt statement.*

```
lemma honest_signing_programming_site_query_entropy_token
  md sk m ctx coins :
  programmed_site_entropy_token_from_query
    (honest_signing_programming_site_query md sk m ctx coins) =
  signing_entropy_token_of_coins coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
challenge_hash_query
commitment_lowbits_entropy_token
honest_signing_programming_site_queryE
programmed_site_entropy_token_from_query
```

**Theorem 6.28** (EasyCrypt line 629). *EasyCrypt name.*

```
honest_signing_programming_site_query_token_injective
```

*Checked EasyCrypt statement.*

```
lemma honest_signing_programming_site_query_token_injective
  md sk m ctx x q :
  signing_entropy_token_in_range x =>
  honest_signing_programming_site_query md sk m ctx
    (signing_entropy_token_to_coins x) = q =>
  x = programmed_site_entropy_token_from_query q.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
ctx
honest_signing_programming_site_query_entropy_token
md
query_eq
signing_entropy_token_to_coins
signing_entropy_token_to_coins_tokenK
sk
```

```
xrange
```

**Theorem 6.29** (EasyCrypt line 644). *EasyCrypt name.*

```
signing_distribution_programming_site_query_spread
```

**Checked EasyCrypt statement.**

```
lemma signing_distribution_programming_site_query_spread
  (d : random_coins distr) md sk m ctx q :
  signing_randomness_token_spread d =>
  mu d
  (fun coins =>
    honest_signing_programming_site_query md sk m ctx coins = q) <=
  1%r / challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *have*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
challenge_support_cardinality_lower_bound
coins
ctx
honest_signing_programming_site_query_entropy_token
ler_trans
md
mu
mu_le
programmed_site_entropy_token_from_query
query_eq
signing_entropy_token_cardinality
signing_entropy_token_of_coins
signing_randomness_point_bound
signing_randomness_token_spread
sk
token_spread
```

**Theorem 6.30** (EasyCrypt line 673). *EasyCrypt name.*

```
honest_signing_programming_site_query_spread
```

**Checked EasyCrypt statement.**

```
lemma honest_signing_programming_site_query_spread
  md sk m ctx q :
  mu drandom_coins
  (fun coins =>
    honest_signing_programming_site_query md sk m ctx coins = q) <=
  1%r / challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **apply**. Named identifiers syntactically cited in the proof block:

```
signing_coin_distribution_token_spread
signing_distribution_programming_site_query_spread
```

**Theorem 6.31** (EasyCrypt line 684). *EasyCrypt name.*

```
honest_signing_programming_site_query_spread_bound
```

**Checked EasyCrypt statement.**

```
lemma honest_signing_programming_site_query_spread_bound
  md sk m ctx qs :
  programmed_site_query_min_entropy_bound md sk m ctx qs
  (1%r / challenge_support_cardinality_lower_bound).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **rewrite**, **apply**. Named identifiers syntactically cited in the proof block:

```
honest_signing_programming_site_query_spread
programmed_site_query_min_entropy_bound
```

**Theorem 6.32** (EasyCrypt line 694). *EasyCrypt name.*

```
honest_signing_programming_site_query_spread_bound_for
```

**Checked EasyCrypt statement.**

```
lemma honest_signing_programming_site_query_spread_bound_for
  (d : random_coins distr) md sk m ctx qs :
  signing_randomness_token_spread d =>
  programmed_site_query_min_entropy_bound_for d md sk m ctx qs
  (1%r / challenge_support_cardinality_lower_bound).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **apply**. Named identifiers syntactically cited in the proof block:

```
signing_distribution_programming_site_query_spread
spread
```

**Theorem 6.33** (EasyCrypt line 704). *EasyCrypt name.*

```
honest_signing_programming_site_query_is_challenge
```

**Checked EasyCrypt statement.**

```
lemma honest_signing_programming_site_query_is_challenge
  md sk m ctx coins :
  ro_query_is_challenge
  (honest_signing_programming_site_query md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
challenge_hash_query
honest_signing_programming_site_queryE
ro_query_is_challenge
```

**Theorem 6.34** (EasyCrypt line 713). *EasyCrypt name.*

```
programming_site_prequeried_honest_challenge_filter
```

**Checked EasyCrypt statement.**

```
lemma programming_site_prequeried_honest_challenge_filter
  qs md sk m ctx coins :
  programming_site_prequeried qs
  (honest_signing_programming_site md sk m ctx coins) =
  programming_site_prequeried (filter ro_query_is_challenge qs)
  (honest_signing_programming_site md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
honest_signing_programming_site_query
honest_signing_programming_site_query_is_challenge
mem_filter
programming_site_prequeried
```

**Theorem 6.35** (EasyCrypt line 726). *EasyCrypt name.*

```
programming_site_prequeried_honest_message_consE
```

**Checked EasyCrypt statement.**

```
lemma programming_site_prequeried_honest_message_consE
  pk ctx m qs md sk sm sctx coins :
```



```

programming_site_prequeried (message_hash_query pk ctx m :: qs)
  (honest_signing_programming_site md sk sm sctx coins) =
programming_site_prequeried qs
  (honest_signing_programming_site md sk sm sctx coins).

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

actual_signature_programming_site
challenge_hash_query
honest_signing_programming_site
message_hash_query
programming_site_prequeried
programming_site_query
signature_programming_site_of_transcript
transcript_from_honest_signing
transcript_of_signature
transcript_signature_challenge_query

```

**Theorem 6.36** (EasyCrypt line 745). *EasyCrypt name*.

```

honest_signing_programming_site_outputE

```

**Checked EasyCrypt statement.**

```

lemma honest_signing_programming_site_outputE md sk m ctx coins :
  programming_site_output
    (honest_signing_programming_site md sk m ctx coins) =
  ro_output_of_challenge
    (challenge_hash md
      (commitment_highbits md sk m ctx coins)
      (commitment_lowbits md sk m ctx coins)
      (message_hash (public_key_of_secret md sk) ctx m)).

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

actual_signature_programming_site
honest_signing_programming_site
programming_site_output
sig_challenge
sign_internal
signature_programming_site_of_transcript
transcript_from_honest_signing
transcript_of_signature
transcript_signature_challenge_output

```

**Theorem 6.37** (EasyCrypt line 764). *EasyCrypt name.*

```
honest_signing_programming_site_functional_output
```

**Checked EasyCrypt statement.**

```
lemma honest_signing_programming_site_functional_output
  md sk1 m1 ct1 coins1 sk2 m2 ct2 coins2 :
  honest_signing_programming_site_query md sk1 m1 ct1 coins1 =
  honest_signing_programming_site_query md sk2 m2 ct2 coins2 =>
  programming_site_output
    (honest_signing_programming_site md sk1 m1 ct1 coins1) =
  programming_site_output
    (honest_signing_programming_site md sk2 m2 ct2 coins2).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
honest_signing_programming_site_outputE
honest_signing_programming_site_queryE
```

**Theorem 6.38** (EasyCrypt line 778). *EasyCrypt name.*

```
honest_signing_programming_sites_no_conflict
```

**Checked EasyCrypt statement.**

```
lemma honest_signing_programming_sites_no_conflict
  md sk1 m1 ct1 coins1 sk2 m2 ct2 coins2 :
  ! programming_sites_conflict
    (honest_signing_programming_site md sk1 m1 ct1 coins1)
    (honest_signing_programming_site md sk2 m2 ct2 coins2).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
coins1
coins2
ct1
ct2
honest_signing_programming_site
honest_signing_programming_site_functional_output
m1
m2
md
programming_site_query
programming_sites_conflict
```

```
programming_sites_same_query
query_eq
sk1
sk2
```

**Theorem 6.39** (EasyCrypt line 795). *EasyCrypt name.*

```
actual_signature_programming_site_sign_internalE
```

**Checked EasyCrypt statement.**

```
lemma actual_signature_programming_site_sign_internalE
  md pk sk m ctx coins :
  pk = public_key_of_secret md sk =>
  actual_signature_programming_site md
    (transcript_of_signature md pk m ctx
     (sign_internal md sk m ctx coins)) =
  honest_signing_programming_site md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
honest_signing_programming_site
pkE
transcript_from_honest_signing
```

**Theorem 6.40** (EasyCrypt line 808). *EasyCrypt name.*

```
actual_signature_programming_site_sign_internal_any_pkE
```

**Checked EasyCrypt statement.**

```
lemma actual_signature_programming_site_sign_internal_any_pkE
  md pk sk m ctx coins :
  actual_signature_programming_site md
    (transcript_of_signature md pk m ctx
     (sign_internal md sk m ctx coins)) =
  honest_signing_programming_site md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
actual_signature_programming_site
honest_signing_programming_site
signature_programming_site_of_transcript
transcript_from_honest_signing
```

```
transcript_of_signature
transcript_signature_challenge_output
transcript_signature_challenge_query
```

**Theorem 6.41** (EasyCrypt line 823). *EasyCrypt name.*

```
actual_signature_programming_site_prequeried_after_cons
```

**Checked EasyCrypt statement.**

```
lemma actual_signature_programming_site_prequeried_after_cons
  md pk sk m ctx coins q qs :
  pk = public_key_of_secret md sk =>
  programming_site_prequeried qs
  (honest_signing_programming_site md sk m ctx coins) =>
  programming_site_prequeried (q :: qs)
  (actual_signature_programming_site md
    (transcript_of_signature md pk m ctx
      (sign_internal md sk m ctx coins))).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
actual_signature_programming_site_sign_internalE
coins
ctx
md
pk
pkE
pre
programming_site_prequeried_cons_preserve
sk
```

**Theorem 6.42** (EasyCrypt line 839). *EasyCrypt name.*

```
actual_signature_programming_site_prequeried_after_message_hash
```

**Checked EasyCrypt statement.**

```
lemma actual_signature_programming_site_prequeried_after_message_hash
  md pk sk m ctx coins qpk qctx qm qs :
  pk = public_key_of_secret md sk =>
  programming_site_prequeried qs
  (honest_signing_programming_site md sk m ctx coins) =>
  programming_site_prequeried (message_hash_query qpk qctx qm :: qs)
  (actual_signature_programming_site md
    (transcript_of_signature md pk m ctx
      (sign_internal md sk m ctx coins))).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **apply**. Named identifiers syntactically cited in the proof block:

```
actual_signature_programming_site_prequeried_after_cons
coins
ctx
md
message_hash_query
pk
pkE
pre
qctx
qm
qpk
qs
sk
```

**Theorem 6.43** (EasyCrypt line 854). *EasyCrypt name.*

```
actual_signature_programming_site_prequeried_message_hashE
```

**Checked EasyCrypt statement.**

```
lemma actual_signature_programming_site_prequeried_message_hashE
  md pk sk m ctx coins qpk qctx qm qs :
  pk = public_key_of_secret md sk =>
  programming_site_prequeried (message_hash_query qpk qctx qm :: qs)
    (actual_signature_programming_site md
      (transcript_of_signature md pk m ctx
        (sign_internal md sk m ctx coins))) =
  programming_site_prequeried qs
    (honest_signing_programming_site md sk m ctx coins).
```

**Formal mathematical reading.** *Mathematical theorem.* This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** *Use in this chapter.* It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **rewrite**. Named identifiers syntactically cited in the proof block:

```
actual_signature_programming_site_sign_internalE
coins
ctx
md
pk
pkE
programming_site_prequeried_honest_message_consE
sk
```

**Theorem 6.44** (EasyCrypt line 870). *EasyCrypt name.*

```
actual_signature_programming_site_prequeried_message_hash_any_pkE
```

**Checked EasyCrypt statement.**

```
lemma actual_signature_programming_site_prequeried_message_hash_any_pkE
```

```

md pk sk m ctæ coins qpk qctæ qm qs :
programming_site_prequeried (message_hash_query qpk qctæ qm :: qs)
  (actual_signature_programming_site md
    (transcript_of_signature md pk m ctæ
      (sign_internal md sk m ctæ coins))) =
programming_site_prequeried qs
  (honest_signing_programming_site md sk m ctæ coins).

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

actual_signature_programming_site_sign_internal_any_pkE
coins
ctæ
md
pk
programming_site_prequeried_honest_message_consE
sk

```

**Theorem 6.45** (EasyCrypt line 894). *EasyCrypt name.*

```
programming_site_log_conflict_free_for_honest_nil
```

**Checked EasyCrypt statement.**

```

lemma programming_site_log_conflict_free_for_honest_nil md sk :
  programming_site_log_conflict_free_for_honest md sk [].

```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 6.46** (EasyCrypt line 900). *EasyCrypt name.*

```
programming_site_log_conflict_free_for_honest_cons
```

**Checked EasyCrypt statement.**

```

lemma programming_site_log_conflict_free_for_honest_cons
  md sk m ctæ coins sites :
  programming_site_log_conflict_free_for_honest md sk sites =>
  programming_site_log_conflict_free_for_honest md sk
    (honest_signing_programming_site md sk m ctæ coins :: sites).

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
coins
ctx
honest_signing_programming_sites_no_conflict
log
programming_site_conflict_free
programming_site_log_conflict_free_for_honest
```

**Theorem 6.47** (EasyCrypt line 914). *EasyCrypt name*.

```
programming_site_conflict_free_no_reprograms
```

**Checked EasyCrypt statement.**

```
lemma programming_site_conflict_free_no_reprograms site sites :
  programming_site_conflict_free site sites =>
  ! programming_site_reprograms sites site.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
hasPn
no_conflict
old
old_mem
programming_site_conflict_free
programming_site_reprograms
```

**Theorem 6.48** (EasyCrypt line 924). *EasyCrypt name*.

```
actual_signature_programming_site_sign_internal_conflict_step
```

**Checked EasyCrypt statement.**

```
lemma actual_signature_programming_site_sign_internal_conflict_step
  md pk sk m ctx coins sites :
  pk = public_key_of_secret md sk =>
  programming_site_log_conflict_free_for_honest md sk sites =>
  ! programming_site_reprograms sites
    (actual_signature_programming_site md
      (transcript_of_signature md pk m ctx
        (sign_internal md sk m ctx coins))) /\
  programming_site_log_conflict_free_for_honest md sk
    (actual_signature_programming_site md
      (transcript_of_signature md pk m ctx
        (sign_internal md sk m ctx coins))) :: sites).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *split*, *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
actual_signature_programming_site_sign_internalE
coins
ctx
log
md
pk
pkE
programming_site_conflict_free_no_reprograms
programming_site_log_conflict_free_for_honest_cons
siteE
sk
```

**Theorem 6.49** (EasyCrypt line 949). *EasyCrypt name.*

```
programmed_site_reprogram_one_step_conflict_free_zero
```

**Checked EasyCrypt statement.**

```
lemma programmed_site_reprogram_one_step_conflict_free_zero
md sk m ctx sites :
(forall coins,
  programming_site_conflict_free
    (honest_signing_programming_site md sk m ctx coins) sites) =>
mu drandom_coins
(fun coins =>
  programming_site_reprograms sites
    (honest_signing_programming_site md sk m ctx coins)) =
0%r.
```

**Formal mathematical reading.** *Mathematical theorem.* This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** *Use in this chapter.* It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
coins
conflict_free
ctx
honest_signing_programming_site
md
mu0
mu_eq
programming_site_conflict_free_no_reprograms
random_coins
sites
sk
```

**Theorem 6.50** (EasyCrypt line 969). *EasyCrypt name.*

```
programmed_site_prequery_one_step_fset_bound_for
```

**Checked EasyCrypt statement.**



```

lemma programmed_site_prequery_one_step_fset_bound_for
  (d : random_coins distr) md sk m ctx qs bd :
  programmed_site_query_min_entropy_bound_for d md sk m ctx qs bd =>
  mu d
  (fun coins =>
    programming_site_prequeried qs
    (honest_signing_programming_site md sk m ctx coins)) <=
  (card (oflist qs))%r * bd.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*, *apply*. Named identifiers syntactically cited in the proof block:

```

bd
coins
ctx
honest_signing_programming_site
honest_signing_programming_site_query
md
mem_oflist
mu_eq
mu_mem_le_gen
oflist
programmed_site_query_min_entropy_bound_for
programming_site_prequeried
programming_site_query
q_mem
qs
sk
spread

```

**Theorem 6.51** (EasyCrypt line 999). *EasyCrypt name*.

```

programmed_site_prequery_one_step_fset_bound

```

**Checked EasyCrypt statement.**

```

lemma programmed_site_prequery_one_step_fset_bound
  md sk m ctx qs bd :
  programmed_site_query_min_entropy_bound md sk m ctx qs bd =>
  mu drandom_coins
  (fun coins =>
    programming_site_prequeried qs
    (honest_signing_programming_site md sk m ctx coins)) <=
  (card (oflist qs))%r * bd.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
programmed_site_prequery_one_step_fset_bound_for
programmed_site_query_min_entropy_bound
```

**Theorem 6.52** (EasyCrypt line 1012). *EasyCrypt name.*

```
programmed_site_prequery_one_step_budget_bound_for
```

*Checked EasyCrypt statement.*

```
lemma programmed_site_prequery_one_step_budget_bound_for
  (d : random_coins distr) md sk m ctx qs :
  (card (oflist qs))%r <= rom_hash_query_budget + 1%r =>
  programmed_site_query_min_entropy_bound_for d md sk m ctx qs
  (1%r / challenge_support_cardinality_lower_bound) =>
  mu d
  (fun coins =>
    programming_site_prequeried qs
    (honest_signing_programming_site md sk m ctx coins)) <=
  (rom_hash_query_budget + 1%r) /
  challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *apply*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
card
card_budget
challenge_support_cardinality_lower_bound
challenge_support_cardinality_lower_bound_nonnegative
ctx
inv
inv_nonneg
invr_ge0
ler_trans
ler_wpmul2r
md
oflist
programmed_site_prequery_one_step_fset_bound_for
qs
rom_hash_query_budget
sk
spread
```

**Theorem 6.53** (EasyCrypt line 1044). *EasyCrypt name.*

```
programmed_site_prequery_one_step_budget_bound
```

*Checked EasyCrypt statement.*

```
lemma programmed_site_prequery_one_step_budget_bound
  md sk m ctx qs :
  (card (oflist qs))%r <= rom_hash_query_budget + 1%r =>
  programmed_site_query_min_entropy_bound md sk m ctx qs
  (1%r / challenge_support_cardinality_lower_bound) =>
  mu random_coins
  (fun coins =>
```

```

programming_site_prequeried qs
(honest_signing_programming_site md sk m ctx coins)) <=
(rom_hash_query_budget + 1%r) /
challenge_support_cardinality_lower_bound.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

card_budget
programmed_site_prequery_one_step_budget_bound_for
programmed_site_query_min_entropy_bound

```

**Theorem 6.54** (EasyCrypt line 1061). *EasyCrypt name.*

```

programmed_site_prequery_one_step_challenge_fset_bound_for

```

**Checked EasyCrypt statement.**

```

lemma programmed_site_prequery_one_step_challenge_fset_bound_for
  (d : random_coins distr) md sk m ctx qs bd :
  programmed_site_query_min_entropy_bound_for d md sk m ctx
    (filter ro_query_is_challenge qs) bd =>
  mu d
    (fun coins =>
      programming_site_prequeried qs
        (honest_signing_programming_site md sk m ctx coins)) <=
      (card (oflist (filter ro_query_is_challenge qs)))%r * bd.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

coins
ctx
filter
honest_signing_programming_site
md
mu_eq
programmed_site_prequery_one_step_fset_bound_for
programming_site_prequeried
programming_site_prequeried_honest_challenge_filter
qs
ro_query_is_challenge
sk
spread

```

**Theorem 6.55** (EasyCrypt line 1081). *EasyCrypt name.*

```
programmed_site_prequery_one_step_challenge_fset_bound
```

**Checked EasyCrypt statement.**

```
lemma programmed_site_prequery_one_step_challenge_fset_bound
  md sk m ctx qs bd :
  programmed_site_query_min_entropy_bound md sk m ctx
    (filter ro_query_is_challenge qs) bd =>
  mu drandom_coins
    (fun coins =>
      programming_site_prequeried qs
        (honest_signing_programming_site md sk m ctx coins)) <=
    (card (oflist (filter ro_query_is_challenge qs)))%r * bd.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
programmed_site_prequery_one_step_challenge_fset_bound_for
programmed_site_query_min_entropy_bound
```

**Theorem 6.56** (EasyCrypt line 1095). *EasyCrypt name.*

```
programmed_site_prequery_one_step_challenge_budget_bound_for
```

**Checked EasyCrypt statement.**

```
lemma programmed_site_prequery_one_step_challenge_budget_bound_for
  (d : random_coins distr) md sk m ctx qs :
  (card (oflist (filter ro_query_is_challenge qs)))%r <=
    rom_hash_query_budget + 1%r =>
  programmed_site_query_min_entropy_bound_for d md sk m ctx
    (filter ro_query_is_challenge qs)
    (1%r / challenge_support_cardinality_lower_bound) =>
  mu d
    (fun coins =>
      programming_site_prequeried qs
        (honest_signing_programming_site md sk m ctx coins)) <=
    (rom_hash_query_budget + 1%r) /
      challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *apply*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
card
card_budget
challenge_support_cardinality_lower_bound
challenge_support_cardinality_lower_bound_nonnegative
```

```

ctx
filter
inv
inv_nonneg
invr_ge0
ler_trans
ler_wpmul2r
md
oflist
programmed_site_prequery_one_step_challenge_fset_bound_for
qs
ro_query_is_challenge
rom_hash_query_budget
sk
spread

```

**Theorem 6.57** (EasyCrypt line 1129). *EasyCrypt name.*

```
programmed_site_prequery_one_step_challenge_budget_bound
```

**Checked EasyCrypt statement.**

```

lemma programmed_site_prequery_one_step_challenge_budget_bound
  md sk m ctx qs :
  (card (oflist (filter ro_query_is_challenge qs)))%r <=
    rom_hash_query_budget + 1%r =>
  programmed_site_query_min_entropy_bound md sk m ctx
    (filter ro_query_is_challenge qs)
    (1%r / challenge_support_cardinality_lower_bound) =>
  mu drandom_coins
    (fun coins =>
      programming_site_prequeried qs
      (honest_signing_programming_site md sk m ctx coins)) <=
    (rom_hash_query_budget + 1%r) /
    challenge_support_cardinality_lower_bound.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

card_budget
programmed_site_prequery_one_step_challenge_budget_bound_for
programmed_site_query_min_entropy_bound

```

**Theorem 6.58** (EasyCrypt line 1148). *EasyCrypt name.*

```
programmed_site_prequery_one_step_challenge_concrete_bound
```

**Checked EasyCrypt statement.**

```

lemma programmed_site_prequery_one_step_challenge_concrete_bound
  md sk m ctx qs :
  (card (oflist (filter ro_query_is_challenge qs)))%r <=
    rom_hash_query_budget + 1%r =>
  mu drandom_coins
    (fun coins =>
      programming_site_prequeried qs

```

```
(honest_signing_programming_site md sk m ctx coins)) <=
(rom_hash_query_budget + 1%r) /
challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
card_budget
honest_signing_programming_site_query_spread_bound
programmed_site_prequery_one_step_challenge_budget_bound
```

**Theorem 6.59** (EasyCrypt line 1165). *EasyCrypt name.*

```
programmed_site_prequery_one_step_challenge_concrete_bound_for
```

**Checked EasyCrypt statement.**

```
lemma programmed_site_prequery_one_step_challenge_concrete_bound_for
  (d : random_coins distr) md sk m ctx qs :
  signing_randomness_token_spread d =>
  (card (oflist (filter ro_query_is_challenge qs)))%r <=
  rom_hash_query_budget + 1%r =>
  mu d
  (fun coins =>
    programming_site_prequeried qs
    (honest_signing_programming_site md sk m ctx coins)) <=
  (rom_hash_query_budget + 1%r) /
  challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
card_budget
honest_signing_programming_site_query_spread_bound_for
programmed_site_prequery_one_step_challenge_budget_bound_for
spread
```

**Theorem 6.60** (EasyCrypt line 1183). *EasyCrypt name.*

```
challenge_query_log_count_budget_card_bound
```

**Checked EasyCrypt statement.**

```
lemma challenge_query_log_count_budget_card_bound hc qs :
  0 <= hc =>
  hc <= hash_query_budget_count =>
```

```
challenge_query_log_count qs <= hc + 1 =>
(card (oflist (filter ro_query_is_challenge qs)))%r <=
rom_hash_query_budget + 1%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
challenge_query_log_count
hash_query_budget_count
hash_query_count
rom_hash_query_budget
```

**Theorem 6.61** (EasyCrypt line 1195). *EasyCrypt name.*

```
programmed_site_prequery_one_step_challenge_counter_bound
```

**Checked EasyCrypt statement.**

```
lemma programmed_site_prequery_one_step_challenge_counter_bound
md sk m ctx qs hc :
0 <= hc =>
hc <= hash_query_budget_count =>
challenge_query_log_count qs <= hc + 1 =>
mu drandom_coins
(fun coins =>
programming_site_prequeried qs
(honest_signing_programming_site md sk m ctx coins)) <=
(rom_hash_query_budget + 1%r) /
challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
challenge_query_log_count_budget_card_bound
hc
hc_budget
hc_ge0
programmed_site_prequery_one_step_challenge_concrete_bound
qs
query_count
```

**Theorem 6.62** (EasyCrypt line 1212). *EasyCrypt name.*

```
programmed_site_prequery_one_step_challenge_counter_bound_for
```

**Checked EasyCrypt statement.**

```

lemma programmed_site_prequery_one_step_challenge_counter_bound_for
  (d : random_coins distr) md sk m ctx qs hc :
  signing_randomness_token_spread d =>
  0 <= hc =>
  hc <= hash_query_budget_count =>
  challenge_query_log_count qs <= hc + 1 =>
  mu d
    (fun coins =>
      programming_site_prequeried qs
      (honest_signing_programming_site md sk m ctx coins)) <=
  (rom_hash_query_budget + 1%r) /
  challenge_support_cardinality_lower_bound.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **apply**. Named identifiers syntactically cited in the proof block:

```

challenge_query_log_count_budget_card_bound
hc
hc_budget
hc_ge0
programmed_site_prequery_one_step_challenge_concrete_bound_for
qs
query_count
spread

```

**Theorem 6.63** (EasyCrypt line 1231). *EasyCrypt name.*

```

programmed_site_prequery_one_step_message_hash_counter_bound

```

**Checked EasyCrypt statement.**

```

lemma programmed_site_prequery_one_step_message_hash_counter_bound
  md pk sk m ctx qpk qctx qm qs hc :
  0 <= hc =>
  hc <= hash_query_budget_count =>
  challenge_query_log_count qs <= hc + 1 =>
  mu drandom_coins
    (fun coins =>
      programming_site_prequeried (message_hash_query qpk qctx qm :: qs)
      (actual_signature_programming_site md
        (transcript_of_signature md pk m ctx
          (sign_internal md sk m ctx coins)))) <=
  (rom_hash_query_budget + 1%r) /
  challenge_support_cardinality_lower_bound.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **rewrite**, **apply**. Named identifiers syntactically cited in the proof block:



```

actual_signature_programming_site_prequeried_message_hash_any_pkE
coins
ctx
hc
hc_budget
hc_ge0
honest_signing_programming_site
md
mu_eq
pk
programmed_site_prequery_one_step_challenge_counter_bound
programming_site_prequeried
qctx
qm
qpk
qs
query_count
sk

```

**Theorem 6.64** (EasyCrypt line 1257). *EasyCrypt name.*

```
direct_signing_coin_sampler_prequery_bound
```

*Checked EasyCrypt statement.*

```

lemma direct_signing_coin_sampler_prequery_bound
  md sk m ctx qs hc :
  0 <= hc =>
  hc <= hash_query_budget_count =>
  challenge_query_log_count qs <= hc + 1 =>
  phoare[DirectSigningCoinSampler.sample :
    true ==>
    programming_site_prequeried qs
    (honest_signing_programming_site md sk m ctx res)] <=
  ((rom_hash_query_budget + 1%r) /
    challenge_support_cardinality_lower_bound).

```

**Formal mathematical reading.** *Mathematical theorem.* This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byphoare*, *proc*, *rnd*, *apply*. Named identifiers syntactically cited in the proof block:

```

bypr
coins
ctx
hc
hc_budget
hc_ge0
honest_signing_programming_site
md
programmed_site_prequery_one_step_challenge_counter_bound
programming_site_prequeried
qs
query_count
sk

```

**Theorem 6.65** (EasyCrypt line 1283). *EasyCrypt* name.

```
direct_signing_coin_sampler_budgeted_prequery_bound
```

**Checked EasyCrypt statement.**

```
lemma direct_signing_coin_sampler_budgeted_prequery_bound
  md sk m ctx qs :
  challenge_query_log_count qs <= hash_query_budget_count + 1 =>
  phoare[DirectSigningCoinSampler.sample :
    true ==>
    programming_site_prequeried qs
    (honest_signing_programming_site md sk m ctx res)] <=
  ((rom_hash_query_budget + 1%r) /
    challenge_support_cardinality_lower_bound).
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
ctx
direct_signing_coin_sampler_prequery_bound
hash_query_budget_count
md
qs
query_count
sk
```

**Theorem 6.66** (EasyCrypt line 1301). *EasyCrypt* name.

```
direct_signing_coin_sampler_message_hash_prequery_bound
```

**Checked EasyCrypt statement.**

```
lemma direct_signing_coin_sampler_message_hash_prequery_bound
  md pk sk m ctx qpk qctx qm qs hc :
  0 <= hc =>
  hc <= hash_query_budget_count =>
  challenge_query_log_count qs <= hc + 1 =>
  phoare[DirectSigningCoinSampler.sample :
    true ==>
    programming_site_prequeried (message_hash_query qpk qctx qm :: qs)
    (actual_signature_programming_site md
      (transcript_of_signature md pk m ctx
        (sign_internal md sk m ctx res)))] <=
  ((rom_hash_query_budget + 1%r) /
    challenge_support_cardinality_lower_bound).
```

**Formal mathematical reading.** Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem

into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *byphoare*, *proc*, *rnd*, *apply*. Named identifiers syntactically cited in the proof block:

```
actual_signature_programming_site
bypr
coins
ctx
hc
hc_budget
hc_ge0
m0
md
message_hash_query
pk
programmed_site_prequery_one_step_message_hash_counter_bound
programming_site_prequeried
qctx
qm
qpk
qs
query_count
sign_internal
sk
transcript_of_signature
```

**Theorem 6.67** (EasyCrypt line 1333). *EasyCrypt name.*

```
direct_signing_coin_sampler_message_hash_budgeted_prequery_bound
```

**Checked EasyCrypt statement.**

```
lemma direct_signing_coin_sampler_message_hash_budgeted_prequery_bound
  md pk sk m ctx qpk qctx qm qs :
  challenge_query_log_count qs <= hash_query_budget_count + 1 =>
  phoare [DirectSigningCoinSampler.sample :
    true ==>
    programmed_site_prequeried (message_hash_query qpk qctx qm :: qs)
    (actual_signature_programming_site md
      (transcript_of_signature md pk m ctx
        (sign_internal md sk m ctx res))) ] <=
    ((rom_hash_query_budget + 1%r) /
      challenge_support_cardinality_lower_bound).
```

**Formal mathematical reading.** *Mathematical theorem. This is a relational program theorem: two games or procedures are coupled so that a precondition on their initial states implies the stated postcondition on final states and return values.*

**Role in the proof.** *Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:*

```
ctx
direct_signing_coin_sampler_message_hash_prequery_bound
hash_query_budget_count
md
pk
qctx
qm
qpk
```

```
qs
query_count
sk
```

**Theorem 6.68** (EasyCrypt line 1353). *EasyCrypt name.*

```
programmed_site_prequery_one_step_concrete_bound
```

**Checked EasyCrypt statement.**

```
lemma programmed_site_prequery_one_step_concrete_bound
  md sk m ctx qs :
  (card (oflist qs))%r <= rom_hash_query_budget + 1%r =>
  mu drandom_coins
    (fun coins =>
      programming_site_prequeried qs
        (honest_signing_programming_site md sk m ctx coins)) <=
  (rom_hash_query_budget + 1%r) /
  challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
card_budget
honest_signing_programming_site_query_spread_bound
programmed_site_prequery_one_step_budget_bound
```

**Theorem 6.69** (EasyCrypt line 1369). *EasyCrypt name.*

```
programmed_site_prequery_reprogram_one_step_concrete_bound
```

**Checked EasyCrypt statement.**

```
lemma programmed_site_prequery_reprogram_one_step_concrete_bound
  md sk m ctx qs sites :
  (forall coins,
    programming_site_conflict_free
      (honest_signing_programming_site md sk m ctx coins) sites) =>
  (card (oflist qs))%r <= rom_hash_query_budget + 1%r =>
  mu drandom_coins
    (fun coins =>
      programming_site_prequeried qs
        (honest_signing_programming_site md sk m ctx coins) \ /
      programming_site_reprograms sites
        (honest_signing_programming_site md sk m ctx coins)) <=
  (rom_hash_query_budget + 1%r) /
  challenge_support_cardinality_lower_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *apply*, *case*, *have*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
card_budget
coins
conflict_free
ctx
drandom_coins
honest_signing_programming_site
ler_trans
md
mu
mu_le
no_reprog
preq
programmed_site_prequery_one_step_concrete_bound
programming_site_conflict_free_no_reprograms
programming_site_prequeried
programming_site_reprograms
qs
reprog
sites
sk
```

**Theorem 6.70** (EasyCrypt line 1404). *EasyCrypt* name.

```
programmed_site_query_min_entropy_bound_constant_impossible
```

**Checked EasyCrypt statement.**

```
lemma programmed_site_query_min_entropy_bound_constant_impossible
  md sk m ctx qs bd q :
  q \in qs =>
  (forall coins,
    honest_signing_programming_site_query md sk m ctx coins = q) =>
  bd < 1%r =>
  programmed_site_query_min_entropy_bound md sk m ctx qs bd => false.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
bd_lt1
coins
ctx
drandom_coins
eq1_mu
eq_sym
hit1
hit_le_bd
honest_signing_programming_site_query
md
mu
q_mem
query_constant
signing_coin_distribution_lossless
sk
spread
```

**Theorem 6.71** (EasyCrypt line 1426). *EasyCrypt name.*

```
programming_site_reprograms_conflict_cons
```

**Checked EasyCrypt statement.**

```
lemma programming_site_reprograms_conflict_cons site old programmed :  
  programming_sites_conflict site old =>  
  programming_site_reprograms (old :: programmed) site.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
conflict  
programming_site_reprograms
```

**Theorem 6.72** (EasyCrypt line 1434). *EasyCrypt name.*

```
rom_counted_programming_bad_min_entropy
```

**Checked EasyCrypt statement.**

```
lemma rom_counted_programming_bad_min_entropy queries programmed :  
  rom_counted_programming_bad queries programmed true.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 6.73** (EasyCrypt line 1438). *EasyCrypt name.*

```
rom_counted_programming_bad_prequery_cons
```

**Checked EasyCrypt statement.**

```
lemma rom_counted_programming_bad_prequery_cons queries programmed site  
  entropy_bad :  
  programming_site_prequeried queries site =>  
  rom_counted_programming_bad queries (site :: programmed) entropy_bad.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*. Named identifiers syntactically cited in the proof block:

```
entropy_bad
preg
rom_counted_programming_bad
```

**Theorem 6.74** (EasyCrypt line 1448). *EasyCrypt name.*

```
counted_rom_programming_bound_obligation_holds
```

**Checked EasyCrypt statement.**

```
lemma counted_rom_programming_bound_obligation_holds :
  counted_rom_programming_bound_obligation.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
counted_rom_programming_bound_obligation
counted_rom_programming_loss_term_nonnegative
```

**Theorem 6.75** (EasyCrypt line 1463). *EasyCrypt name.*

```
counted_lazy_rom_bad_flags_universal_bound
```

**Checked EasyCrypt statement.**

```
lemma counted_lazy_rom_bad_flags_universal_bound  $\mathcal{E}m$  :
  1%r >= Pr[CountedLazyROMBadGame(0, A).main() @  $\mathcal{E}m$  : res].
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *bypHoare*.

**Theorem 6.76** (EasyCrypt line 1469). *EasyCrypt name.*

```
counted_lazy_rom_bad_flags_query_budget_bound
```

**Checked EasyCrypt statement.**

```
lemma counted_lazy_rom_bad_flags_query_budget_bound  $\mathcal{E}m$  :
  counted_lazy_rom_bad_flags_budget_sound
    (Pr[CountedLazyROMBadGame(0, A).main() @  $\mathcal{E}m$  : res]) =>
  Pr[CountedLazyROMBadGame(0, A).main() @  $\mathcal{E}m$  : res] <=
    counted_rom_programming_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] \leq B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
counted_lazy_rom_bad_flags_budget_sound
```

**Theorem 6.77** (EasyCrypt line 1478). *EasyCrypt name.*

```
counted_lazy_rom_bad_flags_query_budget_bound_subunit
```

**Checked EasyCrypt statement.**

```
lemma counted_lazy_rom_bad_flags_query_budget_bound_subunit &m :
  counted_lazy_rom_bad_flags_budget_sound
  (Pr[CountedLazyROMBadGame(0, A).main() @ &m : res]) =>
  Pr[CountedLazyROMBadGame(0, A).main() @ &m : res] < 1%r.
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
budget_sound
counted_lazy_rom_bad_flags_budget_sound
counted_rom_programming_loss_term
counted_rom_programming_loss_term_subunit
ler_lt_trans
```

**Theorem 6.78** (EasyCrypt line 1491). *EasyCrypt name.*

```
fs_with_aborts_no_programming_failure
```

**Checked EasyCrypt statement.**

```
lemma fs_with_aborts_no_programming_failure md tr y :
  transcript_challenge_matches tr y =>
  fs_with_aborts_failure md tr (programming_site_of_transcript md tr y) =
  min_entropy_failure md tr.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.



Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
fs_with_aborts_failure
match_y
md
programming_site_self_no_reprogramming_failure
tr
```

**Theorem 6.79** (EasyCrypt line 1501). *EasyCrypt name*.

```
transcript_signature_challenge_output_matches
```

**Checked EasyCrypt statement.**

```
lemma transcript_signature_challenge_output_matches tr :
  transcript_signature_challenge_matches tr
  (transcript_signature_challenge_output tr).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
ro_challenge_hash
ro_output_of_challenge
ro_output_to_challenge
transcript_signature_challenge_matches
transcript_signature_challenge_output
```

**Theorem 6.80** (EasyCrypt line 1511). *EasyCrypt name*.

```
honest_transcript_challenge_entropy_support
```

**Checked EasyCrypt statement.**

```
lemma honest_transcript_challenge_entropy_support md sk m ctz coins :
  challenge_entropy_support_ok md
  (transcript_from_honest_signing md sk m ctz coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
challenge_entropy_support_ok
challenge_from_seed_sparse
challenge_hash
sig_challenge
sign_internal
```

```
signature_challenge
transcript_challenge
transcript_from_honest_signing
transcript_of_signature
```

**Theorem 6.81** (EasyCrypt line 1521). *EasyCrypt name.*

```
honest_transcript_no_min_entropy_failure
```

**Checked EasyCrypt statement.**

```
lemma honest_transcript_no_min_entropy_failure md sk m ctx coins :
  ! min_entropy_failure md
    (transcript_from_honest_signing md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
honest_transcript_challenge_entropy_support
min_entropy_failure
```

**Theorem 6.82** (EasyCrypt line 1529). *EasyCrypt name.*

```
signing_transcript_relation_challenge_entropy_support
```

**Checked EasyCrypt statement.**

```
lemma signing_transcript_relation_challenge_entropy_support
  md pk m ctx sig tr :
  signing_transcript_relation md pk m ctx sig tr =>
  challenge_entropy_support_ok md tr.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *elim*, *apply*. Named identifiers syntactically cited in the proof block:

```
attempt
challenge_entropy_support_ok
challenge_from_seed_sparse
challenge_hash
coins
hsim
rel
rest
sig_challenge
sign_internal
signature_challenge
```

```

signing_attempt_relation
signing_simulation_relation
signing_transcript_relation
sk
trE
transcript_challenge
transcript_of_signature
witness

```

**Theorem 6.83** (EasyCrypt line 1549). *EasyCrypt name.*

```

signing_transcript_relation_no_min_entropy_failure

```

**Checked EasyCrypt statement.**

```

lemma signing_transcript_relation_no_min_entropy_failure
  md pk m ctx sig tr :
  signing_transcript_relation md pk m ctx sig tr =>
  ! min_entropy_failure md tr.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

ctx
md
min_entropy_failure
pk
rel
sig
signing_transcript_relation_challenge_entropy_support
tr

```

**Theorem 6.84** (EasyCrypt line 1560). *EasyCrypt name.*

```

signing_record_relation_no_min_entropy_failure

```

**Checked EasyCrypt statement.**

```

lemma signing_record_relation_no_min_entropy_failure md pk r :
  signing_record_relation md pk r =>
  ! min_entropy_failure md (signing_record_transcript r).

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

md

```

```
pk
rel
signing_record_context
signing_record_message
signing_record_relation
signing_record_signature
signing_transcript_relation_no_min_entropy_failure
```

**Theorem 6.85** (EasyCrypt line 1572). *EasyCrypt name.*

```
signing_record_log_sound_no_min_entropy
```

**Checked EasyCrypt statement.**

```
lemma signing_record_log_sound_no_min_entropy md pk rs :
  signing_record_log_sound md pk rs =>
  signing_record_log_min_entropy_clear md rs.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *elim*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
ih
md
pk
rel
rs
signing_record_log_min_entropy_clear
signing_record_log_sound
signing_record_relation_no_min_entropy_failure
sound
```

**Theorem 6.86** (EasyCrypt line 1584). *EasyCrypt name.*

```
signing_record_log_sound_transcripts_no_min_entropy
```

**Checked EasyCrypt statement.**

```
lemma signing_record_log_sound_transcripts_no_min_entropy md pk rs :
  signing_record_log_sound md pk rs =>
  transcript_log_min_entropy_clear md (signing_record_transcripts rs).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *elim*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
ih
md
```

```

pk
rel
rs
signing_record_log_sound
signing_record_relation_no_min_entropy_failure
signing_record_transcripts
sound
transcript_log_min_entropy_clear

```

**Theorem 6.87** (EasyCrypt line 1597). *EasyCrypt name.*

```
fs_with_aborts_no_failure_for_signing_relation
```

**Checked EasyCrypt statement.**

```

lemma fs_with_aborts_no_failure_for_signing_relation
  md pk m ctx sig tr site :
  signing_transcript_relation md pk m ctx sig tr =>
  programming_site_matches_transcript md tr site =>
  ! fs_with_aborts_failure md tr site.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

ctx
fs_with_aborts_failure
match_site
md
pk
rel
reprogramming_failure
sig
signing_transcript_relation_no_min_entropy_failure
tr

```

**Theorem 6.88** (EasyCrypt line 1609). *EasyCrypt name.*

```
fs_with_aborts_no_failure_for_signing_record
```

**Checked EasyCrypt statement.**

```

lemma fs_with_aborts_no_failure_for_signing_record md pk r site :
  signing_record_relation md pk r =>
  programming_site_matches_transcript
    md (signing_record_transcript r) site =>
  ! fs_with_aborts_failure md (signing_record_transcript r) site.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
fs_with_aborts_no_failure_for_signing_relation
match_site
md
pk
rel
signing_record_context
signing_record_message
signing_record_relation
signing_record_signature
signing_record_transcript
site
```

**Theorem 6.89** (EasyCrypt line 1624). *EasyCrypt name.*

```
fs_with_aborts_no_failure_for_honest_programming
```

**Checked EasyCrypt statement.**

```
lemma fs_with_aborts_no_failure_for_honest_programming md sk m ctx coins y :
  transcript_challenge_matches
    (transcript_from_honest_signing md sk m ctx coins) y =>
  ! fs_with_aborts_failure md
    (transcript_from_honest_signing md sk m ctx coins)
    (programming_site_of_transcript md
      (transcript_from_honest_signing md sk m ctx coins) y).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
coins
ctx
fs_with_aborts_failure
honest_transcript_no_min_entropy_failure
match_y
md
programming_site_self_no_reprogramming_failure
sk
transcript_from_honest_signing
```

**Theorem 6.90** (EasyCrypt line 1639). *EasyCrypt name.*

```
transcript_valid_programming_site_signature_query
```

**Checked EasyCrypt statement.**

```
lemma transcript_valid_programming_site_signature_query md tr y :
  transcript_valid md tr =>
  programming_site_query (programming_site_of_transcript md tr y) =
  programming_site_query (signature_programming_site_of_transcript md tr y).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
programming_site_of_transcript
programming_site_query
signature_programming_site_of_transcript
transcript_valid_signature_challenge_query
valid
```

**Theorem 6.91** (EasyCrypt line 1650). *EasyCrypt* name.

```
transcript_valid_signature_programming_site_matches
```

**Checked EasyCrypt statement.**

```
lemma transcript_valid_signature_programming_site_matches md tr y :
  transcript_valid md tr =>
  transcript_signature_challenge_matches tr y =>
  programming_site_matches_transcript md tr
    (signature_programming_site_of_transcript md tr y).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
match_y
md
programming_site_matches_transcript
programming_site_output
programming_site_query
signature_programming_site_of_transcript
tr
transcript_valid_signature_challenge_matches
transcript_valid_signature_challenge_query
valid
```

**Theorem 6.92** (EasyCrypt line 1666). *EasyCrypt* name.

```
transcript_valid_signature_programming_site_no_reprogramming
```

**Checked EasyCrypt statement.**

```
lemma transcript_valid_signature_programming_site_no_reprogramming
  md tr y :
  transcript_valid md tr =>
  transcript_signature_challenge_matches tr y =>
  ! reprogramming_failure md tr
    (signature_programming_site_of_transcript md tr y).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
match_y
md
reprogramming_failure
tr
transcript_valid_signature_programming_site_matches
valid
```

**Theorem 6.93** (EasyCrypt line 1679). *EasyCrypt name.*

```
transcript_valid_actual_signature_programming_site_matches
```

**Checked EasyCrypt statement.**

```
lemma transcript_valid_actual_signature_programming_site_matches md tr :
  transcript_valid md tr =>
    programming_site_matches transcript md tr
    (actual_signature_programming_site md tr).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
actual_signature_programming_site
md
tr
transcript_signature_challenge_output
transcript_signature_challenge_output_matches
transcript_valid_signature_programming_site_matches
valid
```

**Theorem 6.94** (EasyCrypt line 1691). *EasyCrypt name.*

```
transcript_valid_actual_signature_programming_site_no_reprogramming
```

**Checked EasyCrypt statement.**

```
lemma transcript_valid_actual_signature_programming_site_no_reprogramming
  md tr :
  transcript_valid md tr =>
    ! reprogramming_failure md tr (actual_signature_programming_site md tr).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.



**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
md
reprogramming_failure
tr
transcript_valid_actual_signature_programming_site_matches
valid
```

**Theorem 6.95** (EasyCrypt line 1702). *EasyCrypt* name.

```
valid_forking_pair_signature_programming_site_query
```

**Checked EasyCrypt statement.**

```
lemma valid_forking_pair_signature_programming_site_query md tr1 tr2 y1 y2 :
  valid_forking_pair md tr1 tr2 =>
    programming_site_query (signature_programming_site_of_transcript md tr1 y1) =
    programming_site_query (signature_programming_site_of_transcript md tr2 y2).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
fork_pair
programming_site_query
signature_programming_site_of_transcript
valid_forking_pair_signature_challenge_query
```

**Theorem 6.96** (EasyCrypt line 1713). *EasyCrypt* name.

```
valid_forking_pair_programming_site_query
```

**Checked EasyCrypt statement.**

```
lemma valid_forking_pair_programming_site_query md tr1 tr2 y1 y2 :
  valid_forking_pair md tr1 tr2 =>
    programming_site_query (programming_site_of_transcript md tr1 y1) =
    programming_site_query (programming_site_of_transcript md tr2 y2).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

fork_pair
programming_site_of_transcript
programming_site_query
valid_forking_pair_challenge_query

```

**Theorem 6.97** (EasyCrypt line 1723). *EasyCrypt name.*

```

valid_forking_pair_self_programming_sites_no_reprogramming

```

**Checked EasyCrypt statement.**

```

lemma valid_forking_pair_self_programming_sites_no_reprogramming
  md tr1 tr2 y1 y2 :
  valid_forking_pair md tr1 tr2 =>
  transcript_challenge_matches tr1 y1 =>
  transcript_challenge_matches tr2 y2 =>
  ! reprogramming_failure md tr1
    (programming_site_of_transcript md tr1 y1) /\
  ! reprogramming_failure md tr2
    (programming_site_of_transcript md tr2 y2).

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *split*, *apply*. Named identifiers syntactically cited in the proof block:

```

match1
match2
programming_site_self_no_reprogramming_failure

```

**Theorem 6.98** (EasyCrypt line 1739). *EasyCrypt name.*

```

fs_bound_parts_nonnegative

```

**Checked EasyCrypt statement.**

```

lemma fs_bound_parts_nonnegative :
  fs_with_aborts_bound_obligation =>
  0%r <= fs_with_aborts_reprogramming_term /\
  0%r <= fs_with_aborts_min_entropy_term.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 6.99** (EasyCrypt line 1745). *EasyCrypt name.*

```

fs_with_aborts_bound_obligation_holds

```

**Checked EasyCrypt statement.**

```

lemma fs_with_aborts_bound_obligation_holds :
  fs_with_aborts_bound_obligation.

```

**Formal mathematical reading.** *Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.*

**Role in the proof.** *Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `split`, `apply`. Named identifiers syntactically cited in the proof block:*

```
fs_with_aborts_bound_obligation
fs_with_aborts_min_entropy_term_nonnegative
fs_with_aborts_reprogramming_term_nonnegative
```

*End of generated formal inventory for HAETAE\_ROM\_Programming. ec.*

# Chapter 7

## HAETAE\_Scheme.ec

**Mathematical role.** This file instantiates the generic signature interface from `Sig_ROM.ec` with the HAETAE operations and the HAETAE random-oracle interface.

**Key generation.** The modeled key-generation procedure samples a seed, queries the matrix expansion oracle, and calls the deterministic internal key-generation function:

$$sd \leftarrow D_{\text{seed}}, \quad \rho' = \text{KeyGenRhoPrime}(sd), \quad (pk, sk) = \text{KeyGenInternal}(\rho').$$

**Signing.** Signing first samples randomness and then obtains oracle-derived signing coins:

$$coins \leftarrow D_{\text{coins}}, \quad coins' = \text{ROSignCoins}(H(\text{SamplerExpandQuery}(coins))).$$

It then hashes the public key, context, and message to  $\mu$ , computes commitment high/low bits, queries the challenge oracle, and returns `SignInternal`. The important proof design is that probabilistic sampling is separated from deterministic algebra.

**Verification.** Verification is simply `VerifyInternal`( $pk, m, ctx, \sigma$ ). The correctness proof in `HAETAE_Security.ec` relies on the algebraic lemmas from `HAETAE_Algebra.ec` showing that honest signatures satisfy this predicate.

### Textbook Formal Inventory

This subsection records the exact formal material in `HAETAE_Scheme.ec`. It is generated from the proof-surface EasyCrypt file, so the line numbers and statements are meant to be read next to the checked source.

**Chapter role.** HAETAE instantiation of the generic signature scheme interface.

**Inventory size.** 9 context declarations, 5 definitions/game objects, 0 lemmas or relational theorems, and 0 explicit Hoare/Phoare judgments were found.

### Theory Context, Imports, and Quantified Modules

**Context 1 (line 1)** EasyCrypt name.

|                   |
|-------------------|
| imported theories |
|-------------------|

**Checked EasyCrypt statement.**

```
require import AllCore.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 2 (line 2)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import Sig_ROM HAETAE_Params HAETAE_Algebra HAETAE_Distributions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 3 (line 3)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import HAETAE_ROM.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 4 (line 5)** EasyCrypt name.

```
HAETAE_Scheme
```

**Checked EasyCrypt statement.**

```
theory HAETAE_Scheme.
```

**Formal mathematical reading.** Mathematical context. This begins a namespace for the formal development in this file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 5 (line 7)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Params.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 6 (line 8) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Algebra.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 7 (line 9) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Distributions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 8 (line 10) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_ROM.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 9 (line 12) EasyCrypt name.**

```
cloned theory
```

**Checked EasyCrypt statement.**

```
clone import Sig_ROM as SIG with
  type pkey <- pkey,
  type skey <- skey,
  type message <- message,
  type context <- context,
  type signature <- signature,
```

```

type ro_query <- ro_query,
type ro_output <- ro_output,
op dro_output <- dro_output.

```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

## Definitions, Carrier Sets, Operations, Games, and Procedures

**Definition 7.1** (EasyCrypt line 22). **EasyCrypt name.**

```
haetae_mode
```

**Checked EasyCrypt statement.**

```
op haetae_mode : mode.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 7.2** (EasyCrypt line 24). **EasyCrypt name.**

```
HAETAETAE
```

**Checked EasyCrypt statement.**

```
module HAETAETAE(H : SIG.POracle) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 7.3** (EasyCrypt line 25). **EasyCrypt name.**

```
kg
```

**Checked EasyCrypt statement.**

```
proc kg() : pkey * skey = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 7.4** (EasyCrypt line 38). **EasyCrypt name.**

```
sign
```

**Checked EasyCrypt statement.**

```
proc sign(sk : skey, m : message, ctx : context) : signature = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 7.5** (EasyCrypt line 60). **EasyCrypt name.**

```
verify
```

**Checked EasyCrypt statement.**

```
proc verify(pk : pkey, m : message, ctx : context,  
    sig : signature) : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

*End of generated formal inventory for HAETAE\_ Scheme. ec.*



## Chapter 8

### HAETAE\_Transcript.ec

**Mathematical role.** This file defines the transcript abstraction used for forking and special-soundness. A transcript is the tuple

$$tr = (pk, m, ctx, w_1, w_0, \mu, c, \sigma).$$

The projections `transcript_pk`, `transcript_message`, and so on are ordinary tuple projections.

**Validity.** The predicate `transcript_valid` combines three conditions:

1. the signature fields agree with the transcript;
2. the challenge query is the one determined by  $(w_1, w_0, \mu)$ ;
3. the public verification equation is satisfied.

This is the formal replacement for the paper phrase “an accepting transcript.”

**Forking and extraction.** A valid forking pair consists of two transcripts for the same target with distinct challenges. Conceptually, subtracting two valid verification equations cancels the shared commitment:

$$Az_1 - c_1t = w, \quad Az_2 - c_2t = w,$$

so

$$A(z_1 - z_2) = (c_1 - c_2)t.$$

The file defines the corresponding extracted bimodal self-target MSIS instance and solution, proves well-formedness, and then provides the special-soundness obligations consumed by `HAETAE_HopGames.ec`.

#### Textbook Formal Inventory

This subsection records the exact formal material in `HAETAE_Transcript.ec`. It is generated from the proof-surface EasyCrypt file, so the line numbers and statements are meant to be read next to the checked source.

**Chapter role.** transcripts, forking predicates, and extraction obligations.

**Inventory size.** 7 context declarations, 52 definitions/game objects, 35 lemmas or relational theorems, and 0 explicit Hoare/Phoare judgments were found.

## Theory Context, Imports, and Quantified Modules

Context 1 (line 1) EasyCrypt name.

```
imported theories
```

Checked EasyCrypt statement.

```
require import AllCore List.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

Context 2 (line 2) EasyCrypt name.

```
imported theories
```

Checked EasyCrypt statement.

```
require import HAETAE_Params HAETAE_Algebra HAETAE_ROM HAETAE_Assumptions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

Context 3 (line 4) EasyCrypt name.

```
HAETAE_Transcript
```

Checked EasyCrypt statement.

```
theory HAETAE_Transcript.
```

**Formal mathematical reading.** Mathematical context. This begins a namespace for the formal development in this file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

Context 4 (line 6) EasyCrypt name.

```
opened namespace
```

Checked EasyCrypt statement.

```
import HAETAE_Params.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 5 (line 7)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Algebra.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 6 (line 8)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_ROM.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 7 (line 9)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Assumptions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

## Definitions, Carrier Sets, Operations, Games, and Procedures

**Definition 8.1** (EasyCrypt line 11). EasyCrypt name.

```
transcript
```

**Checked EasyCrypt statement.**

```
type transcript =  
  pkey * message * context * polyveck * poly * crh * challenge * signature.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.2** (EasyCrypt line 14). **EasyCrypt name.**

```
signature_commitment_highbits
```

**Checked EasyCrypt statement.**

```
op signature_commitment_highbits :  
  mode -> pkey -> message -> context -> signature -> polyveck =  
  fun (_ : mode) (_ : pkey) (_ : message) (_ : context) sig =>  
    sig_commitment_highbits sig.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 8.3** (EasyCrypt line 18). **EasyCrypt name.**

```
signature_commitment_lowbits
```

**Checked EasyCrypt statement.**

```
op signature_commitment_lowbits :  
  mode -> pkey -> message -> context -> signature -> poly =  
  fun (_ : mode) (_ : pkey) (_ : message) (_ : context) sig =>  
    sig_commitment_lowbits sig.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 8.4** (EasyCrypt line 22). **EasyCrypt name.**

```
signature_message_hash
```

**Checked EasyCrypt statement.**

```
op signature_message_hash :  
  mode -> pkey -> message -> context -> signature -> crh =  
  fun (_ : mode) (_ : pkey) (_ : message) (_ : context) sig =>  
    sig_message_hash sig.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 8.5** (EasyCrypt line 26). **EasyCrypt name.**

```
signature_challenge
```

**Checked EasyCrypt statement.**

```
op signature_challenge :  
  mode -> pkey -> message -> context -> signature -> challenge =  
  fun (_ : mode) (_ : pkey) (_ : message) (_ : context) sig =>  
    sig_challenge sig.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 8.6** (EasyCrypt line 31). **EasyCrypt name.**

```
transcript_pk
```

**Checked EasyCrypt statement.**

```
op transcript_pk (tr : transcript) : pkey = tr.'1.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.7** (EasyCrypt line 32). **EasyCrypt name.**

```
transcript_message
```

**Checked EasyCrypt statement.**

```
op transcript_message (tr : transcript) : message = tr.'2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.8** (EasyCrypt line 33). **EasyCrypt name.**

```
transcript_context
```

**Checked EasyCrypt statement.**

```
op transcript_context (tr : transcript) : context = tr.'3.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.9** (EasyCrypt line 34). **EasyCrypt name.**

```
transcript_commitment_highbits
```

**Checked EasyCrypt statement.**

```
op transcript_commitment_highbits (tr : transcript) : polyveck = tr.'4.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.10** (EasyCrypt line 35). **EasyCrypt name.**

```
transcript_commitment_lowbits
```

### Checked EasyCrypt statement.

```
op transcript_commitment_lowbits (tr : transcript) : poly = tr.'5.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

### Definition 8.11 (EasyCrypt line 36). EasyCrypt name.

```
transcript_message_hash
```

### Checked EasyCrypt statement.

```
op transcript_message_hash (tr : transcript) : crh = tr.'6.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

### Definition 8.12 (EasyCrypt line 37). EasyCrypt name.

```
transcript_challenge
```

### Checked EasyCrypt statement.

```
op transcript_challenge (tr : transcript) : challenge = tr.'7.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

### Definition 8.13 (EasyCrypt line 38). EasyCrypt name.

```
transcript_signature
```

### Checked EasyCrypt statement.

```
op transcript_signature (tr : transcript) : signature = tr.'8.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

### Definition 8.14 (EasyCrypt line 40). EasyCrypt name.

```
transcript_of_signature
```

### Checked EasyCrypt statement.

```
op transcript_of_signature :
  mode -> pkey -> message -> context -> signature -> transcript =
  fun md pk m ctx sig =>
    (pk, m, ctx,
     signature_commitment_highbits md pk m ctx sig,
```

```
signature_commitment_lowbits md pk m ctx sig,
signature_message_hash md pk m ctx sig,
signature_challenge md pk m ctx sig,
sig).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.15** (EasyCrypt line 50). **EasyCrypt name.**

```
transcript_challenge_query
```

**Checked EasyCrypt statement.**

```
op transcript_challenge_query (md : mode) (tr : transcript) : ro_query =
  challenge_hash_query
    md
    (transcript_commitment_highbits tr)
    (transcript_commitment_lowbits tr)
    (transcript_message_hash tr).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.16** (EasyCrypt line 57). **EasyCrypt name.**

```
transcript_signature_challenge_query
```

**Checked EasyCrypt statement.**

```
op transcript_signature_challenge_query (md : mode) (tr : transcript) :
  ro_query =
  challenge_hash_query
    md
    (sig_commitment_highbits (transcript_signature tr))
    (sig_commitment_lowbits (transcript_signature tr))
    (sig_message_hash (transcript_signature tr)).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.17** (EasyCrypt line 65). **EasyCrypt name.**

```
transcript_verifies
```

**Checked EasyCrypt statement.**

```
op transcript_verifies (md : mode) (tr : transcript) : bool =
  verify_internal md
    (transcript_pk tr)
    (transcript_message tr)
    (transcript_context tr)
    (transcript_signature tr).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.18** (EasyCrypt line 72). **EasyCrypt name.**

```
transcript_challenge_matches
```

**Checked EasyCrypt statement.**

```
op transcript_challenge_matches (tr : transcript) (y : ro_output) : bool =
  ro_challenge_hash y = transcript_challenge tr.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.19** (EasyCrypt line 75). **EasyCrypt name.**

```
transcript_signature_challenge_matches
```

**Checked EasyCrypt statement.**

```
op transcript_signature_challenge_matches
  (tr : transcript) (y : ro_output) : bool =
  ro_challenge_hash y = sig_challenge (transcript_signature tr).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.20** (EasyCrypt line 79). **EasyCrypt name.**

```
transcript_fields_match_signature
```

**Checked EasyCrypt statement.**

```
op transcript_fields_match_signature (tr : transcript) : bool =
  transcript_commitment_highbits tr =
    sig_commitment_highbits (transcript_signature tr) /\
  transcript_commitment_lowbits tr =
    sig_commitment_lowbits (transcript_signature tr) /\
  transcript_message_hash tr =
    sig_message_hash (transcript_signature tr) /\
  transcript_challenge tr =
    sig_challenge (transcript_signature tr).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.



**Definition 8.21** (EasyCrypt line 89). **EasyCrypt name.**

```
transcript_valid
```

**Checked EasyCrypt statement.**

```
op transcript_valid (md : mode) (tr : transcript) : bool =
  transcript_verifies md tr /\
  valid_signature md (transcript_signature tr) /\
  transcript_fields_match_signature tr.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 8.22** (EasyCrypt line 94). **EasyCrypt name.**

```
transcript_public_equation
```

**Checked EasyCrypt statement.**

```
op transcript_public_equation (md : mode) (tr : transcript) : bool =
  verify_public_equation md (transcript_pk tr) (transcript_signature tr).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.23** (EasyCrypt line 97). **EasyCrypt name.**

```
same_fork_target
```

**Checked EasyCrypt statement.**

```
op same_fork_target (tr1 tr2 : transcript) : bool =
  transcript_pk tr1 = transcript_pk tr2 /\
  transcript_message tr1 = transcript_message tr2 /\
  transcript_context tr1 = transcript_context tr2 /\
  transcript_commitment_highbits tr1 = transcript_commitment_highbits tr2 /\
  transcript_commitment_lowbits tr1 = transcript_commitment_lowbits tr2 /\
  transcript_message_hash tr1 = transcript_message_hash tr2.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.24** (EasyCrypt line 105). **EasyCrypt name.**

```
distinct_fork_challenges
```

**Checked EasyCrypt statement.**

```
op distinct_fork_challenges (tr1 tr2 : transcript) : bool =
  transcript_challenge tr1 <> transcript_challenge tr2.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.25** (EasyCrypt line 108). **EasyCrypt name.**

```
valid_forking_pair
```

**Checked EasyCrypt statement.**

```
op valid_forking_pair (md : mode) (tr1 tr2 : transcript) : bool =
  same_fork_target tr1 tr2 /\
  distinct_fork_challenges tr1 tr2 /\
  transcript_valid md tr1 /\
  transcript_valid md tr2.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 8.26** (EasyCrypt line 114). **EasyCrypt name.**

```
transcript_response
```

**Checked EasyCrypt statement.**

```
op transcript_response (tr : transcript) : polyvecl =
  sig_response (transcript_signature tr).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.27** (EasyCrypt line 116). **EasyCrypt name.**

```
transcript_response_aux
```

**Checked EasyCrypt statement.**

```
op transcript_response_aux (tr : transcript) : polyveck =
  sig_response_aux (transcript_signature tr).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.28** (EasyCrypt line 119). **EasyCrypt name.**

```
transcript_public_key_challenge_term
```

**Checked EasyCrypt statement.**

```

op transcript_public_key_challenge_term (md : mode) (tr : transcript) :
  polyveck =
    public_key_challenge_term md
      (transcript_pk tr)
      (sig_challenge (transcript_signature tr)).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 8.29** (EasyCrypt line 125). **EasyCrypt name.**

```

active_public_reconstruction

```

**Checked EasyCrypt statement.**

```

op active_public_reconstruction (md : mode) (tr : transcript) : bool =
  transcript_public_key_challenge_term md tr <> polyveck_zero md.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 8.30** (EasyCrypt line 128). **EasyCrypt name.**

```

inactive_public_reconstruction

```

**Checked EasyCrypt statement.**

```

op inactive_public_reconstruction (md : mode) (tr : transcript) : bool =
  transcript_public_key_challenge_term md tr = polyveck_zero md.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 8.31** (EasyCrypt line 131). **EasyCrypt name.**

```

forking_difference_solution

```

**Checked EasyCrypt statement.**

```

op forking_difference_solution
  (md : mode) (tr1 tr2 : transcript) : bimodal_selftarget_msis_solution =
  polyvecl_sub md (transcript_response tr1) (transcript_response tr2).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.32** (EasyCrypt line 135). **EasyCrypt name.**

```
fork_response_aux_difference
```

**Checked EasyCrypt statement.**

```
op fork_response_aux_difference (md : mode) (tr1 tr2 : transcript) :  
  polyveck =  
  polyveck_sub md (transcript_response_aux tr1) (transcript_response_aux tr2).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.33** (EasyCrypt line 139). **EasyCrypt name.**

```
fork_public_challenge_relation
```

**Checked EasyCrypt statement.**

```
op fork_public_challenge_relation (md : mode) (tr1 tr2 : transcript) : bool =  
  polyveck_add md  
    (transcript_response_aux tr1)  
    (transcript_public_key_challenge_term md tr1) =  
  polyveck_add md  
    (transcript_response_aux tr2)  
    (transcript_public_key_challenge_term md tr2).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.34** (EasyCrypt line 147). **EasyCrypt name.**

```
fork_response_aux_relation
```

**Checked EasyCrypt statement.**

```
op fork_response_aux_relation (tr1 tr2 : transcript) : bool =  
  transcript_response_aux tr1 = transcript_response_aux tr2.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.35** (EasyCrypt line 150). **EasyCrypt name.**

```
fork_left_active_public_challenge_relation
```

**Checked EasyCrypt statement.**

```
op fork_left_active_public_challenge_relation  
  (md : mode) (tr1 tr2 : transcript) : bool =  
  polyveck_add md  
    (transcript_response_aux tr1)  
    (transcript_public_key_challenge_term md tr1) =  
  transcript_response_aux tr2.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.36** (EasyCrypt line 157). **EasyCrypt name.**

```
fork_right_active_public_challenge_relation
```

**Checked EasyCrypt statement.**

```
op fork_right_active_public_challenge_relation
  (md : mode) (tr1 tr2 : transcript) : bool =
  transcript_response_aux tr1 =
  polyveck_add md
  (transcript_response_aux tr2)
  (transcript_public_key_challenge_term md tr2).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.37** (EasyCrypt line 164). **EasyCrypt name.**

```
fork_public_reconstruction_cases
```

**Checked EasyCrypt statement.**

```
op fork_public_reconstruction_cases
  (md : mode) (tr1 tr2 : transcript) : bool =
  fork_public_challenge_relation md tr1 tr2.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.38** (EasyCrypt line 168). **EasyCrypt name.**

```
fork_matrix_source
```

**Checked EasyCrypt statement.**

```
op fork_matrix_source (tr1 tr2 : transcript) : byte list =
  encode_pkey (transcript_pk tr1) ++
  transcript_context tr1 ++
  transcript_message tr1 ++
  encode_polyveck (transcript_commitment_highbits tr1) ++
  encode_poly (transcript_commitment_lowbits tr1) ++
  transcript_message_hash tr1 ++
  encode_poly (transcript_challenge tr1) ++
  encode_poly (transcript_challenge tr2).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.39** (EasyCrypt line 178). **EasyCrypt name.**

```
fork_module_matrix
```

**Checked EasyCrypt statement.**

```
op fork_module_matrix (md : mode) (tr1 tr2 : transcript) : matrix =
  mkseq
    (fun i =>
      deterministic_unit_polyvecl md (53 + i) (fork_matrix_source tr1 tr2))
    (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.40** (EasyCrypt line 184). **EasyCrypt name.**

```
fork_module_target
```

**Checked EasyCrypt statement.**

```
op fork_module_target (md : mode) (tr1 tr2 : transcript) : polyveck =
  matrix_vec_mul md
    (fork_module_matrix md tr1 tr2)
    (forking_difference_solution md tr1 tr2).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.41** (EasyCrypt line 189). **EasyCrypt name.**

```
bimodal_selftarget_msis_instance_of_fork
```

**Checked EasyCrypt statement.**

```
op bimodal_selftarget_msis_instance_of_fork :
  mode -> transcript -> transcript -> bimodal_selftarget_msis_instance =
  fun md tr1 tr2 =>
    (fork_module_matrix md tr1 tr2, fork_module_target md tr1 tr2).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.42** (EasyCrypt line 194). **EasyCrypt name.**

```
extract_bimodal_selftarget_msis_solution
```

**Checked EasyCrypt statement.**

```
op extract_bimodal_selftarget_msis_solution :
  mode -> transcript -> transcript ->
```

```
bimodal_selftarget_msis_solution option =
fun md tr1 tr2 =>
  if valid_forking_pair md tr1 tr2
  then Some (forking_difference_solution md tr1 tr2)
  else None.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.43** (EasyCrypt line 202). **EasyCrypt name.**

```
module_sis_instance_of_fork
```

**Checked EasyCrypt statement.**

```
op module_sis_instance_of_fork :
  mode -> transcript -> transcript -> module_sis_instance =
  fun md tr1 tr2 =>
    bimodal_to_module_sis_instance md
    (bimodal_selftarget_msis_instance_of_fork md tr1 tr2).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.44** (EasyCrypt line 208). **EasyCrypt name.**

```
extract_module_sis_solution
```

**Checked EasyCrypt statement.**

```
op extract_module_sis_solution :
  mode -> transcript -> transcript -> module_sis_solution option =
  fun md tr1 tr2 =>
    let x = bimodal_selftarget_msis_instance_of_fork md tr1 tr2 in
    let sol = extract_bimodal_selftarget_msis_solution md tr1 tr2 in
    if sol = None then None
    else Some (bimodal_to_module_sis_solution md x (oget sol)).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.45** (EasyCrypt line 216). **EasyCrypt name.**

```
bimodal_extraction_success
```

**Checked EasyCrypt statement.**

```
op bimodal_extraction_success (md : mode) (tr1 tr2 : transcript) : bool =
  extract_bimodal_selftarget_msis_solution md tr1 tr2 <> None /\
  bimodal_selftarget_msis_valid md
  (bimodal_selftarget_msis_instance_of_fork md tr1 tr2)
  (oget (extract_bimodal_selftarget_msis_solution md tr1 tr2)).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.46** (EasyCrypt line 470). **EasyCrypt name.**

```
extraction_success
```

**Checked EasyCrypt statement.**

```
op extraction_success (md : mode) (tr1 tr2 : transcript) : bool =
  extract_module_sis_solution md tr1 tr2 <> None /\
  module_sis_valid md
  (module_sis_instance_of_fork md tr1 tr2)
  (oget (extract_module_sis_solution md tr1 tr2)).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.47** (EasyCrypt line 476). **EasyCrypt name.**

```
forking_extraction_obligation
```

**Checked EasyCrypt statement.**

```
op forking_extraction_obligation (md : mode) : bool =
  forall (tr1 tr2 : transcript),
    valid_forking_pair md tr1 tr2 =>
      bimodal_extraction_success md tr1 tr2.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.48** (EasyCrypt line 481). **EasyCrypt name.**

```
fork_public_reconstruction_obligation
```

**Checked EasyCrypt statement.**

```
op fork_public_reconstruction_obligation (md : mode) : bool =
  forall (tr1 tr2 : transcript),
    valid_forking_pair md tr1 tr2 =>
      fork_public_reconstruction_cases md tr1 tr2.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 8.49** (EasyCrypt line 535). **EasyCrypt name.**



```
bimodal_to_module_sis_lift_obligation
```

### Checked EasyCrypt statement.

```
op bimodal_to_module_sis_lift_obligation (md : mode) : bool =  
  forall (x : bimodal_selftarget_msis_instance)  
    (sol : bimodal_selftarget_msis_solution),  
    bimodal_selftarget_msis_valid md x sol =>  
      module_sis_valid md  
        (bimodal_to_module_sis_instance md x)  
        (bimodal_to_module_sis_solution md x sol).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 8.50** (EasyCrypt line 551). **EasyCrypt name.**

```
special_soundness_obligation
```

### Checked EasyCrypt statement.

```
op special_soundness_obligation (md : mode) : bool =  
  forall (tr1 tr2 : transcript),  
    valid_forking_pair md tr1 tr2 =>  
      extraction_success md tr1 tr2.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 8.51** (EasyCrypt line 556). **EasyCrypt name.**

```
special_soundness_with_public_reconstruction_obligation
```

### Checked EasyCrypt statement.

```
op special_soundness_with_public_reconstruction_obligation (md : mode) : bool =  
  forall (tr1 tr2 : transcript),  
    valid_forking_pair md tr1 tr2 =>  
      fork_public_reconstruction_cases md tr1 tr2 /\  
      extraction_success md tr1 tr2.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 8.52** (EasyCrypt line 607). **EasyCrypt name.**

```
transcript_from_honest_signing
```

### Checked EasyCrypt statement.

```

op transcript_from_honest_signing
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (coins : random_coins) : transcript =
  transcript_of_signature md
    (public_key_of_secret md sk)
    m ctx
    (sign_internal md sk m ctx coins).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

## Lemmas, Theorems, Relational Equivalences, and Their Proof Dependencies

**Theorem 8.1** (EasyCrypt line 222). *EasyCrypt name.*

```

extract_bimodal_forking_some

```

**Checked EasyCrypt statement.**

```

lemma extract_bimodal_forking_some md tr1 tr2 :
  valid_forking_pair md tr1 tr2 =>
  extract_bimodal_selftarget_msis_solution md tr1 tr2 =
  Some (forking_difference_solution md tr1 tr2).

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*. Named identifiers syntactically cited in the proof block:

```

extract_bimodal_selftarget_msis_solution
md
tr1
tr2
valid_forking_pair

```

**Theorem 8.2** (EasyCrypt line 231). *EasyCrypt name.*

```

extract_bimodal_forking_nonempty

```

**Checked EasyCrypt statement.**

```

lemma extract_bimodal_forking_nonempty md tr1 tr2 :
  valid_forking_pair md tr1 tr2 =>
  extract_bimodal_selftarget_msis_solution md tr1 tr2 <> None.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*, *case*. Named identifiers syntactically cited in the proof block:

```
None
Some
extract_bimodal_forking_some
forking_difference_solution
md
tr1
tr2
valid_pair
```

**Theorem 8.3** (EasyCrypt line 240). *EasyCrypt* name.

```
transcript_fields_match_signature_challenge_query
```

*Checked EasyCrypt statement.*

```
lemma transcript_fields_match_signature_challenge_query md tr :
  transcript_fields_match_signature tr =>
  transcript_challenge_query md tr =
  transcript_signature_challenge_query md tr.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*. Named identifiers syntactically cited in the proof block:

```
highE
lowE
muE
rest
transcript_challenge_query
transcript_fields_match_signature
transcript_signature_challenge_query
```

**Theorem 8.4** (EasyCrypt line 254). *EasyCrypt* name.

```
transcript_fields_match_signature_challenge_matches
```

*Checked EasyCrypt statement.*

```
lemma transcript_fields_match_signature_challenge_matches tr y :
  transcript_fields_match_signature tr =>
  transcript_challenge_matches tr y =
  transcript_signature_challenge_matches tr y.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*. Named identifiers syntactically cited in the proof block:

```

chE
rest
transcript_challenge_matches
transcript_fields_match_signature
transcript_signature_challenge_matches

```

**Theorem 8.5** (EasyCrypt line 268). *EasyCrypt name.*

```
transcript_valid_signature_challenge_query
```

*Checked EasyCrypt statement.*

```

lemma transcript_valid_signature_challenge_query md tr :
  transcript_valid md tr =>
  transcript_challenge_query md tr =
  transcript_signature_challenge_query md tr.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

fields
transcript_fields_match_signature_challenge_query
transcript_valid

```

**Theorem 8.6** (EasyCrypt line 278). *EasyCrypt name.*

```
transcript_valid_signature_challenge_matches
```

*Checked EasyCrypt statement.*

```

lemma transcript_valid_signature_challenge_matches md tr y :
  transcript_valid md tr =>
  transcript_challenge_matches tr y =
  transcript_signature_challenge_matches tr y.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

fields
transcript_fields_match_signature_challenge_matches
transcript_valid

```

**Theorem 8.7** (EasyCrypt line 288). *EasyCrypt name.*

```
transcript_valid_signature_challengeE
```

### Checked EasyCrypt statement.

```
lemma transcript_valid_signature_challengeE md tr :  
  transcript_valid md tr =>  
    transcript_challenge tr = sig_challenge (transcript_signature tr).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **rewrite**, **case**, **apply**. Named identifiers syntactically cited in the proof block:

```
chE  
rest  
transcript_fields_match_signature  
transcript_valid
```

### Theorem 8.8 (EasyCrypt line 299). *EasyCrypt* name.

```
transcript_valid_public_equation
```

### Checked EasyCrypt statement.

```
lemma transcript_valid_public_equation md tr :  
  transcript_valid md tr =>  
    transcript_public_equation md tr.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **rewrite**, **case**, **apply**. Named identifiers syntactically cited in the proof block:

```
hpubnorm  
htail  
hver  
pub  
transcript_public_equation  
transcript_valid  
transcript_verifies  
verify_internal
```

### Theorem 8.9 (EasyCrypt line 313). *EasyCrypt* name.

```
transcript_valid_commitment_reconstruction
```

### Checked EasyCrypt statement.

```
lemma transcript_valid_commitment_reconstruction md tr :  
  transcript_valid md tr =>  
    transcript_commitment_highbits tr =  
      verify_reconstructed_highbits md
```

```
(transcript_pk tr)
(transcript_signature tr).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*, *case*. Named identifiers syntactically cited in the proof block:

```
fields
highE
md
pub
pubE
rest
tr
transcript_fields_match_signature
transcript_public_equation
transcript_valid
transcript_valid_public_equation
valid
verify_public_equation
```

**Theorem 8.10** (EasyCrypt line 333). *EasyCrypt name*.

```
valid_forking_pair_public_equations
```

**Checked EasyCrypt statement.**

```
lemma valid_forking_pair_public_equations md tr1 tr2 :
  valid_forking_pair md tr1 tr2 =>
    transcript_public_equation md tr1 /\
    transcript_public_equation md tr2.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
transcript_valid_public_equation
valid1
valid2
valid_forking_pair
```

**Theorem 8.11** (EasyCrypt line 345). *EasyCrypt name*.

```
same_fork_target_challenge_query
```

**Checked EasyCrypt statement.**

```
lemma same_fork_target_challenge_query md tr1 tr2 :
```

```

same_fork_target tr1 tr2 =>
transcript_challenge_query md tr1 = transcript_challenge_query md tr2.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

highE
lowE
muE
same_fork_target
transcript_challenge_query

```

**Theorem 8.12** (EasyCrypt line 354). *EasyCrypt name*.

```

valid_forking_pair_challenge_query

```

**Checked EasyCrypt statement.**

```

lemma valid_forking_pair_challenge_query md tr1 tr2 :
  valid_forking_pair md tr1 tr2 =>
  transcript_challenge_query md tr1 = transcript_challenge_query md tr2.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

same
same_fork_target_challenge_query
valid_forking_pair

```

**Theorem 8.13** (EasyCrypt line 363). *EasyCrypt name*.

```

valid_forking_pair_signature_challenge_query

```

**Checked EasyCrypt statement.**

```

lemma valid_forking_pair_signature_challenge_query md tr1 tr2 :
  valid_forking_pair md tr1 tr2 =>
  transcript_signature_challenge_query md tr1 =
  transcript_signature_challenge_query md tr2.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **have**, **rewrite**, **apply**. Named identifiers syntactically cited in the proof block:*

```
md
queryE
tr1
tr2
transcript_valid_signature_challenge_query
valid1
valid2
valid_forking_pair
valid_forking_pair_challenge_query
valid_pair
```

**Theorem 8.14** (EasyCrypt line 378). ***EasyCrypt name.***

```
valid_forking_pair_distinct_signature_challenges
```

**Checked EasyCrypt statement.**

```
lemma valid_forking_pair_distinct_signature_challenges md tr1 tr2 :
  valid_forking_pair md tr1 tr2 =>
  sig_challenge (transcript_signature tr1) <>
  sig_challenge (transcript_signature tr2).
```

**Formal mathematical reading.** *Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.*

**Role in the proof.** *Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **rewrite**, **have**. Named identifiers syntactically cited in the proof block:*

```
ch1E
ch2E
distinct
distinct_fork_challenges
md
tr1
tr2
transcript_valid_signature_challengeE
valid1
valid2
valid_forking_pair
```

**Theorem 8.15** (EasyCrypt line 392). ***EasyCrypt name.***

```
same_fork_target_commitment_highbits
```

**Checked EasyCrypt statement.**

```
lemma same_fork_target_commitment_highbits tr1 tr2 :
  same_fork_target tr1 tr2 =>
  transcript_commitment_highbits tr1 = transcript_commitment_highbits tr2.
```

**Formal mathematical reading.** *Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.*



**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 8.16** (EasyCrypt line 397). *EasyCrypt name.*

```
valid_forking_pair_commitment_highbits
```

**Checked EasyCrypt statement.**

```
lemma valid_forking_pair_commitment_highbits md tr1 tr2 :
  valid_forking_pair md tr1 tr2 =>
    transcript_commitment_highbits tr1 = transcript_commitment_highbits tr2.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
same
same_fork_target_commitment_highbits
valid_forking_pair
```

**Theorem 8.17** (EasyCrypt line 406). *EasyCrypt name.*

```
valid_forking_pair_reconstructed_highbits_equal
```

**Checked EasyCrypt statement.**

```
lemma valid_forking_pair_reconstructed_highbits_equal md tr1 tr2 :
  valid_forking_pair md tr1 tr2 =>
    verify_reconstructed_highbits md
      (transcript_pk tr1) (transcript_signature tr1) =
    verify_reconstructed_highbits md
      (transcript_pk tr2) (transcript_signature tr2).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
highE
md
rec1
rec2
tr1
tr2
transcript_valid_commitment_reconstruction
valid1
valid2
```

```
valid_forking_pair
valid_forking_pair_commitment_highbits
valid_pair
```

**Theorem 8.18** (EasyCrypt line 423). *EasyCrypt name.*

```
transcript_reconstructionE
```

**Checked EasyCrypt statement.**

```
lemma transcript_reconstructionE md tr :
  verify_reconstructed_highbits md
    (transcript_pk tr) (transcript_signature tr) =
  polyveck_add md
    (transcript_response_aux tr)
    (transcript_public_key_challenge_term md tr).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
reconstructed_highbits
transcript_public_key_challenge_term
transcript_response_aux
verify_reconstructed_highbits
```

**Theorem 8.19** (EasyCrypt line 436). *EasyCrypt name.*

```
transcript_active_reconstructionE
```

**Checked EasyCrypt statement.**

```
lemma transcript_active_reconstructionE md tr :
  active_public_reconstruction md tr =>
  verify_reconstructed_highbits md
    (transcript_pk tr) (transcript_signature tr) =
  polyveck_add md
    (transcript_response_aux tr)
    (transcript_public_key_challenge_term md tr).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
transcript_reconstructionE
```

**Theorem 8.20** (EasyCrypt line 448). *EasyCrypt name.*

```
valid_forking_pair_public_challenge_relation
```

**Checked EasyCrypt statement.**

```
lemma valid_forking_pair_public_challenge_relation md tr1 tr2 :  
  valid_forking_pair md tr1 tr2 =>  
  fork_public_challenge_relation md tr1 tr2.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
fork_public_challenge_relation  
md  
recE  
tr1  
tr2  
transcript_reconstructionE  
valid_forking_pair_reconstructed_highbits_equal  
valid_pair
```

**Theorem 8.21** (EasyCrypt line 461). *EasyCrypt name.*

```
valid_forking_pair_public_reconstruction_cases
```

**Checked EasyCrypt statement.**

```
lemma valid_forking_pair_public_reconstruction_cases md tr1 tr2 :  
  valid_forking_pair md tr1 tr2 =>  
  fork_public_reconstruction_cases md tr1 tr2.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
fork_public_reconstruction_cases  
valid_forking_pair_public_challenge_relation  
valid_pair
```

**Theorem 8.22** (EasyCrypt line 486). *EasyCrypt name.*

```
forking_difference_solution_wf
```

**Checked EasyCrypt statement.**

```
lemma forking_difference_solution_wf md tr1 tr2 :  
  module_sis_solution_wf md (forking_difference_solution md tr1 tr2).
```

**Formal mathematical reading.** *Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.*

**Role in the proof.** *Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
forking_difference_solution
module_sis_solution_wf
polyvecl_sub_wf
```

**Theorem 8.23** (EasyCrypt line 493). *EasyCrypt name.*

```
fork_module_matrix_rows_wf
```

**Checked EasyCrypt statement.**

```
lemma fork_module_matrix_rows_wf md tr1 tr2 :
  all (polyvecl_wf md) (fork_module_matrix md tr1 tr2).
```

**Formal mathematical reading.** *Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.*

**Role in the proof.** *Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `apply`. Named identifiers syntactically cited in the proof block:*

```
allP
deterministic_unit_polyvecl_wf
fork_module_matrix
mkseqP
row
```

**Theorem 8.24** (EasyCrypt line 501). *EasyCrypt name.*

```
fork_module_matrix_size
```

**Checked EasyCrypt statement.**

```
lemma fork_module_matrix_size md tr1 tr2 :
  size (fork_module_matrix md tr1 tr2) = mode_k md.
```

**Formal mathematical reading.** *Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.*

**Role in the proof.** *Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.*

**Proof-script interpretation.** *No proof block was collected for this item.*

**Theorem 8.25** (EasyCrypt line 505). *EasyCrypt name.*

```
bimodal_forking_solution_valid
```

### *Checked EasyCrypt statement.*

```
lemma bimodal_forking_solution_valid md tr1 tr2 :  
  bimodal_selftarget_msis_valid md  
    (bimodal_selftarget_msis_instance_of_fork md tr1 tr2)  
    (forking_difference_solution md tr1 tr2).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
bimodal_selftarget_msis_instance_of_fork  
bimodal_selftarget_msis_valid  
fork_module_target  
forking_difference_solution_wf  
module_sis_valid
```

### **Theorem 8.26** (EasyCrypt line 516). *EasyCrypt name.*

```
forking_extraction_obligation_holds
```

### *Checked EasyCrypt statement.*

```
lemma forking_extraction_obligation_holds md :  
  forking_extraction_obligation md.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
bimodal_extraction_success  
bimodal_forking_solution_valid  
extract_bimodal_forking_nonempty  
extract_bimodal_forking_some  
forking_extraction_obligation  
md  
tr1  
tr2  
valid_pair
```

### **Theorem 8.27** (EasyCrypt line 527). *EasyCrypt name.*

```
fork_public_reconstruction_obligation_holds
```

### *Checked EasyCrypt statement.*

```
lemma fork_public_reconstruction_obligation_holds md :  
  fork_public_reconstruction_obligation md.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
fork_public_reconstruction_obligation
tr1
tr2
valid_forking_pair_public_reconstruction_cases
valid_pair
```

**Theorem 8.28** (EasyCrypt line 543). *EasyCrypt name.*

```
bimodal_to_module_sis_lift_obligation_holds
```

**Checked EasyCrypt statement.**

```
lemma bimodal_to_module_sis_lift_obligation_holds md :
  bimodal_to_module_sis_lift_obligation md.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
bimodal_to_module_sis_lift_obligation
bimodal_to_module_sis_valid
sol
valid
```

**Theorem 8.29** (EasyCrypt line 562). *EasyCrypt name.*

```
module_sis_extraction_from_bimodal
```

**Checked EasyCrypt statement.**

```
lemma module_sis_extraction_from_bimodal md tr1 tr2 :
  bimodal_extraction_success md tr1 tr2 =>
  bimodal_to_module_sis_lift_obligation md =>
  extraction_success md tr1 tr2.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
bimodal_extraction_success
bimodal_selftarget_msis_instance_of_fork
bimodal_to_module_sis_lift_obligation
extract_bimodal_selftarget_msis_solution
extract_module_sis_solution
extraction_success
iff
lift
md
module_sis_instance_of_fork
nz
oget
tr1
tr2
valid
```

**Theorem 8.30** (EasyCrypt line 582). *EasyCrypt name.*

```
special_soundness_from_bimodal_lift
```

**Checked EasyCrypt statement.**

```
lemma special_soundness_from_bimodal_lift md :
  forking_extraction_obligation md =>
  bimodal_to_module_sis_lift_obligation md =>
  special_soundness_obligation md.
```

**Formal mathematical reading.** *Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.*

**Role in the proof.** *Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.*

Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
fork
forking_extraction_obligation
lift
module_sis_extraction_from_bimodal
special_soundness_obligation
tr1
tr2
valid_pair
```

**Theorem 8.31** (EasyCrypt line 592). *EasyCrypt name.*

```
special_soundness_with_public_reconstruction_from_bimodal_lift
```

**Checked EasyCrypt statement.**

```
lemma special_soundness_with_public_reconstruction_from_bimodal_lift md :
  fork_public_reconstruction_obligation md =>
  forking_extraction_obligation md =>
  bimodal_to_module_sis_lift_obligation md =>
  special_soundness_with_public_reconstruction_obligation md.
```

**Formal mathematical reading.** *Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.*

**Role in the proof.** *Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `split`, `apply`. Named identifiers syntactically cited in the proof block:*

```
fork
fork_public_reconstruction_obligation
forking_extraction_obligation
lift
module_sis_extraction_from_bimodal
public
special_soundness_with_public_reconstruction_obligation
tr1
tr2
valid_pair
```

**Theorem 8.32** (EasyCrypt line 615). *EasyCrypt name.*

```
honest_transcript_verifies
```

**Checked EasyCrypt statement.**

```
lemma honest_transcript_verifies md sk m ctx coins :
  transcript_verifies md
  (transcript_from_honest_signing md sk m ctx coins).
```

**Formal mathematical reading.** *Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `apply`. Named identifiers syntactically cited in the proof block:*

```
transcript_context
transcript_from_honest_signing
transcript_message
transcript_of_signature
transcript_pk
transcript_signature
transcript_verifies
verify_internal_sign_internal
```

**Theorem 8.33** (EasyCrypt line 625). *EasyCrypt name.*

```
transcript_of_signature_fields
```

**Checked EasyCrypt statement.**

```
lemma transcript_of_signature_fields md pk m ctx sig :
  transcript_commitment_highbits
  (transcript_of_signature md pk m ctx sig) =
  sig_commitment_highbits sig /\
  transcript_commitment_lowbits
```



```

(transcript_of_signature md pk m ctx sig) =
  sig_commitment_lowbits sig /\
transcript_message_hash
(transcript_of_signature md pk m ctx sig) =
  sig_message_hash sig /\
transcript_challenge
(transcript_of_signature md pk m ctx sig) =
  sig_challenge sig.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 8.34** (EasyCrypt line 640). *EasyCrypt name.*

```
transcript_of_signature_matches_signature
```

**Checked EasyCrypt statement.**

```

lemma transcript_of_signature_matches_signature md pk m ctx sig :
  transcript_fields_match_signature
  (transcript_of_signature md pk m ctx sig).

```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

transcript_fields_match_signature
transcript_of_signature

```

**Theorem 8.35** (EasyCrypt line 647). *EasyCrypt name.*

```
honest_transcript_challenge
```

**Checked EasyCrypt statement.**

```

lemma honest_transcript_challenge md sk m ctx coins :
  transcript_challenge
  (transcript_from_honest_signing md sk m ctx coins) =
  challenge_hash
  md
  (commitment_highbits md sk m ctx coins)
  (commitment_lowbits md sk m ctx coins)
  (message_hash (public_key_of_secret md sk) ctx m).

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
sig_challenge  
sign_internal  
signature_challenge  
transcript_from_honest_signing  
transcript_of_signature
```

*End of generated formal inventory for HAETAE\_Transcript. ec.*

## Chapter 9

### HAETAE\_Events.ec

**Mathematical role.** This file names the rejection-failure predicates used throughout the proof. It separates three possible failures:

KeyGenFailure : invalid keypair or bad secret norm,  
SignFailure : invalid signature or bad response norm,  
VerifyFailure : invalid signature or bad verification norm.

**Why it is separate.** The game-hop files need compact event names. Instead of repeatedly expanding validity and norm predicates, they use these named events. The lemmas prove that current honest key generation and signing are rejection-free:

$$\Pr[\text{AnyRejectionFailure for an honest transcript}] = 0.$$

#### Textbook Formal Inventory

This subsection records the exact formal material in `HAETAE_Events.ec`. It is generated from the proof-surface EasyCrypt file, so the line numbers and statements are meant to be read next to the checked source.

**Chapter role.** named rejection-failure event predicates.

**Inventory size.** 5 context declarations, 4 definitions/game objects, 4 lemmas or relational theorems, and 0 explicit Hoare/Phoare judgments were found.

#### Theory Context, Imports, and Quantified Modules

**Context 1 (line 1)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import AllCore.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 2 (line 2)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import HAETAE_Params HAETAE_Algebra.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 3 (line 4)** EasyCrypt name.

```
HAETAE_Events
```

**Checked EasyCrypt statement.**

```
theory HAETAE_Events.
```

**Formal mathematical reading.** Mathematical context. This begins a namespace for the formal development in this file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 4 (line 6)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Params.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 5 (line 7)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Algebra.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

## Definitions, Carrier Sets, Operations, Games, and Procedures

**Definition 9.1** (EasyCrypt line 9). EasyCrypt name.

```
keygen_rejection_failure
```

### Checked EasyCrypt statement.

```
op keygen_rejection_failure (md : mode) (pk : pkey) (sk : skey) : bool =  
  ! valid_keypair md pk sk /\ ! secret_norm_ok md sk.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

**Definition 9.2** (EasyCrypt line 12). **EasyCrypt name.**

```
signing_rejection_failure
```

### Checked EasyCrypt statement.

```
op signing_rejection_failure (md : mode) (sig : signature) : bool =  
  ! valid_signature md sig /\ ! response_norm_ok md sig.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

**Definition 9.3** (EasyCrypt line 15). **EasyCrypt name.**

```
verification_rejection_failure
```

### Checked EasyCrypt statement.

```
op verification_rejection_failure (md : mode) (sig : signature) : bool =  
  ! valid_signature md sig /\ ! verify_norm_ok md sig.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

**Definition 9.4** (EasyCrypt line 18). **EasyCrypt name.**

```
any_rejection_failure
```

### Checked EasyCrypt statement.

```
op any_rejection_failure (md : mode) (pk : pkey) (sk : skey)  
  (sig : signature) : bool =  
  keygen_rejection_failure md pk sk /\  
  signing_rejection_failure md sig /\  
  verification_rejection_failure md sig.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

## Lemmas, Theorems, Relational Equivalences, and Their Proof Dependencies

**Theorem 9.1** (EasyCrypt line 24). *EasyCrypt name.*

```
keygen_rejection_free_current
```

**Checked EasyCrypt statement.**

```
lemma keygen_rejection_free_current md sd :
  ! keygen_rejection_failure md (keygen_internal md sd). '1
    (keygen_internal md sd). '2.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*. Named identifiers syntactically cited in the proof block:

```
keygen_internal
keygen_rejection_failure
md
secret_key_of_seed
secret_norm_ok
secret_norm_sq
valid_keypair
```

**Theorem 9.2** (EasyCrypt line 32). *EasyCrypt name.*

```
signing_rejection_free_current
```

**Checked EasyCrypt statement.**

```
lemma signing_rejection_free_current md sk m ctx coins :
  ! signing_rejection_failure md (sign_internal md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
response_norm_ok_current
signing_rejection_failure
valid_signature_sign_internal
```

**Theorem 9.3** (EasyCrypt line 40). *EasyCrypt name.*

```
verification_rejection_free_current
```

### Checked EasyCrypt statement.

```
lemma verification_rejection_free_current md sk m ctx coins :  
  ! verification_rejection_failure md (sign_internal md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
valid_signature_sign_internal  
verification_rejection_failure  
verify_norm_ok_current
```

### Theorem 9.4 (EasyCrypt line 48). *EasyCrypt* name.

```
accepted_transcript_rejection_free_current
```

### Checked EasyCrypt statement.

```
lemma accepted_transcript_rejection_free_current md sd m ctx coins :  
  ! any_rejection_failure md (keygen_internal md sd). '1  
    (keygen_internal md sd). '2  
    (sign_internal md (keygen_internal md sd). '2 m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
any_rejection_failure  
keygen_rejection_free_current  
signing_rejection_free_current  
verification_rejection_free_current
```

*End of generated formal inventory for HAETAE\_Events. ec.*

# Chapter 10

## HAETAE\_Assumptions.ec

**Mathematical role.** This file defines the assumption games. It does not prove lattice hardness; it defines the formal games whose probabilities are used by the final theorem.

**MLWE.** The MLWE part defines real and random instance distributions and a distinguisher interface. The security theorem later assumes that any reduction’s success in the MLWE game is bounded by `mlwe_hardness_term`.

**Module-SIS.** A Module-SIS instance is represented as  $(A, t)$ , and a solution is a vector  $z$ . The validity predicate is

$$\text{module\_sis\_valid}(A, t, z) \iff z \text{ is well formed} \wedge Az = t.$$

This is the target relation for the extraction argument.

**Bimodal self-target MSIS.** The forking extractor first yields a bimodal self-target MSIS solution. This file models that problem with the same carrier as Module-SIS and proves exact lifting lemmas:

$$\Pr[\text{BimodalSelfTargetMSIS}(B) = 1] = \Pr[\text{Module-SIS}(\text{Lift}(B)) = 1].$$

The final theorem still keeps a named loss term for this lifting step so the bound records the reduction boundary explicitly.

### Textbook Formal Inventory

This subsection records the exact formal material in `HAETAE_Assumptions.ec`. It is generated from the proof-surface EasyCrypt file, so the line numbers and statements are meant to be read next to the checked source.

**Chapter role.** formal MLWE, Module-SIS, and bimodal self-target MSIS games.

**Inventory size.** 11 context declarations, 60 definitions/game objects, 11 lemmas or relational theorems, and 0 explicit Hoare/Phoare judgments were found.

### Theory Context, Imports, and Quantified Modules

**Context 1 (line 1)** EasyCrypt name.

```
imported theories
```



### Checked EasyCrypt statement.

```
require import AllCore Distr List.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 2 (line 2) EasyCrypt name.

```
imported theories
```

### Checked EasyCrypt statement.

```
require import HAETAE_Params HAETAE_Algebra.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 3 (line 4) EasyCrypt name.

```
HAETAE_Assumptions
```

### Checked EasyCrypt statement.

```
theory HAETAE_Assumptions.
```

**Formal mathematical reading.** Mathematical context. This begins a namespace for the formal development in this file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 4 (line 6) EasyCrypt name.

```
opened namespace
```

### Checked EasyCrypt statement.

```
import HAETAE_Params.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

### Context 5 (line 7) EasyCrypt name.

```
opened namespace
```

### Checked EasyCrypt statement.

```
import HAETAE_Algebra.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 6 (line 205) EasyCrypt name.**

```
BimodalSolverLift
```

**Checked EasyCrypt statement.**

```
section BimodalSolverLift.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 7 (line 207) EasyCrypt name.**

```
B
```

**Checked EasyCrypt statement.**

```
declare module B <: BimodalSelfTargetMSIS_Solver.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 8 (line 288) EasyCrypt name.**

```
BimodalModeSolverLiftedDistribution
```

**Checked EasyCrypt statement.**

```
section BimodalModeSolverLiftedDistribution.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 9 (line 290) EasyCrypt name.**

```
B
```

**Checked EasyCrypt statement.**

```
declare module B <: BimodalSelfTargetMSIS_ModeSolver.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 10 (line 358) EasyCrypt name.**

```
BimodalModuleSISReduction
```

**Checked EasyCrypt statement.**

```
section BimodalModuleSISReduction.
```

**Formal mathematical reading.** Mathematical context. This opens a scoped block of hypotheses, module parameters, and lemmas.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 11 (line 360) EasyCrypt name.**

```
B
```

**Checked EasyCrypt statement.**

```
declare module B <: BimodalSelfTargetMSIS_Reduction.
```

**Formal mathematical reading.** Mathematical context. This is universal quantification over an adversary, oracle, scheme, sampler, or reduction module satisfying the stated interface and memory restrictions.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

## Definitions, Carrier Sets, Operations, Games, and Procedures

**Definition 10.1 (EasyCrypt line 9). EasyCrypt name.**

```
mlwe_instance
```

**Checked EasyCrypt statement.**

```
type mlwe_instance.
```

**Formal mathematical reading.** Mathematical definition. `mlwe_instance` is an abstract carrier set. The proof may quantify over this domain, but it does not expose a representation.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 10.2 (EasyCrypt line 10). EasyCrypt name.**

```
module_sis_instance
```

**Checked EasyCrypt statement.**

```
type module_sis_instance = matrix * polyveck.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 10.3** (EasyCrypt line 11). **EasyCrypt name.**

```
module_sis_solution
```

**Checked EasyCrypt statement.**

```
type module_sis_solution = polyvecl.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 10.4** (EasyCrypt line 12). **EasyCrypt name.**

```
bimodal_selftarget_msis_instance
```

**Checked EasyCrypt statement.**

```
type bimodal_selftarget_msis_instance = module_sis_instance.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 10.5** (EasyCrypt line 13). **EasyCrypt name.**

```
bimodal_selftarget_msis_solution
```

**Checked EasyCrypt statement.**

```
type bimodal_selftarget_msis_solution = module_sis_solution.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 10.6** (EasyCrypt line 15). **EasyCrypt name.**

```
dmlwe_real
```

**Checked EasyCrypt statement.**

```
op [lossless] dmlwe_real : mode -> mlwe_instance distr.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present. The EasyCrypt [lossless] annotation states that this distribution has total mass 1.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 10.7** (EasyCrypt line 16). **EasyCrypt name.**

```
dmlwe_random
```

**Checked EasyCrypt statement.**

```
op [lossless] dmlwe_random : mode -> mlwe_instance distr.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present. The EasyCrypt [lossless] annotation states that this distribution has total mass 1.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 10.8** (EasyCrypt line 17). **EasyCrypt name.**

```
dmodule_sis_instance
```

**Checked EasyCrypt statement.**

```
op [lossless] dmodule_sis_instance : mode -> module_sis_instance distr.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present. The EasyCrypt [lossless] annotation states that this distribution has total mass 1.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 10.9** (EasyCrypt line 18). **EasyCrypt name.**

```
dbimodal_selftarget_msis_instance
```

**Checked EasyCrypt statement.**

```
op [lossless] dbimodal_selftarget_msis_instance :  
  mode -> bimodal_selftarget_msis_instance distr.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present. The EasyCrypt [lossless] annotation states that this distribution has total mass 1.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 10.10** (EasyCrypt line 21). **EasyCrypt name.**

```
module_sis_eval
```

**Checked EasyCrypt statement.**

```
op module_sis_eval  
  (md : mode) (x : module_sis_instance)  
  (sol : module_sis_solution) : polyveck =  
  polyveck_sub md (matrix_vec_mul md x.'1 sol) x.'2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 10.11** (EasyCrypt line 26). **EasyCrypt name.**

```
module_sis_zero_instance
```

**Checked EasyCrypt statement.**

```
op module_sis_zero_instance (md : mode) : module_sis_instance =
  ([], polyveck_zero md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 10.12** (EasyCrypt line 29). **EasyCrypt name.**

```
module_sis_solution_wf
```

**Checked EasyCrypt statement.**

```
op module_sis_solution_wf (md : mode) (sol : module_sis_solution) : bool =
  polyvecl_wf md sol.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 10.13** (EasyCrypt line 32). **EasyCrypt name.**

```
module_sis_valid
```

**Checked EasyCrypt statement.**

```
op module_sis_valid
  (md : mode) (x : module_sis_instance)
  (sol : module_sis_solution) : bool =
  module_sis_solution_wf md sol /\
  matrix_vec_mul md x.'1 sol = x.'2.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 10.14** (EasyCrypt line 38). **EasyCrypt name.**

```
bimodal_selftarget_msis_valid
```

**Checked EasyCrypt statement.**

```

op bimodal_selftarget_msis_valid
  (md : mode) (x : bimodal_selftarget_msis_instance)
  (sol : bimodal_selftarget_msis_solution) : bool =
  module_sis_valid md x sol.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 10.15** (EasyCrypt line 72). **EasyCrypt name.**

```

bimodal_to_module_sis_instance

```

**Checked EasyCrypt statement.**

```

op bimodal_to_module_sis_instance :
  mode -> bimodal_selftarget_msis_instance -> module_sis_instance =
  fun (_ : mode) x => x.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 10.16** (EasyCrypt line 75). **EasyCrypt name.**

```

bimodal_to_module_sis_solution

```

**Checked EasyCrypt statement.**

```

op bimodal_to_module_sis_solution :
  mode -> bimodal_selftarget_msis_instance ->
  bimodal_selftarget_msis_solution -> module_sis_solution =
  fun (_ : mode) (_ : bimodal_selftarget_msis_instance) sol => sol.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 10.17** (EasyCrypt line 91). **EasyCrypt name.**

```

MLWE_Distinguisher

```

**Checked EasyCrypt statement.**

```

module type MLWE_Distinguisher = {

```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.18** (EasyCrypt line 92). **EasyCrypt name.**

```
distinguish
```

**Checked EasyCrypt statement.**

```
proc distinguish(x : mlwe_instance) : bool
}.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.19** (EasyCrypt line 95). **EasyCrypt name.**

```
MLWE_Real_Game
```

**Checked EasyCrypt statement.**

```
module MLWE_Real_Game(B : MLWE_Distinguisher) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 10.20** (EasyCrypt line 96). **EasyCrypt name.**

```
main
```

**Checked EasyCrypt statement.**

```
proc main(md : mode) : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.21** (EasyCrypt line 106). **EasyCrypt name.**

```
MLWE_Random_Game
```

**Checked EasyCrypt statement.**

```
module MLWE_Random_Game(B : MLWE_Distinguisher) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 10.22** (EasyCrypt line 107). **EasyCrypt name.**

```
main
```

**Checked EasyCrypt statement.**



```
proc main(md : mode) : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.23** (EasyCrypt line 117). **EasyCrypt name.**

```
ModuleSIS_Solver
```

**Checked EasyCrypt statement.**

```
module type ModuleSIS_Solver = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.24** (EasyCrypt line 118). **EasyCrypt name.**

```
solve
```

**Checked EasyCrypt statement.**

```
proc solve(x : module_sis_instance) : module_sis_solution option
}.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.25** (EasyCrypt line 121). **EasyCrypt name.**

```
ModuleSIS_Relation_Game
```

**Checked EasyCrypt statement.**

```
module ModuleSIS_Relation_Game(B : ModuleSIS_Solver) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 10.26** (EasyCrypt line 122). **EasyCrypt name.**

```
main
```

**Checked EasyCrypt statement.**

```
proc main(md : mode) : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.27** (EasyCrypt line 132). **EasyCrypt name.**

```
BimodalSelfTargetMSIS_Solver
```

**Checked EasyCrypt statement.**

```
module type BimodalSelfTargetMSIS_Solver = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.28** (EasyCrypt line 133). **EasyCrypt name.**

```
solve
```

**Checked EasyCrypt statement.**

```
proc solve(x : bimodal_selftarget_msis_instance) :
  bimodal_selftarget_msis_solution option
}.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.29** (EasyCrypt line 137). **EasyCrypt name.**

```
BimodalSelfTargetMSIS_Relation_Game
```

**Checked EasyCrypt statement.**

```
module BimodalSelfTargetMSIS_Relation_Game(
  B : BimodalSelfTargetMSIS_Solver
) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 10.30** (EasyCrypt line 140). **EasyCrypt name.**

```
main
```

**Checked EasyCrypt statement.**

```
proc main(md : mode) : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.31** (EasyCrypt line 150). **EasyCrypt name.**

```
lift_bimodal_solution_option
```

**Checked EasyCrypt statement.**

```
op lift_bimodal_solution_option
  (md : mode) (x : bimodal_selftarget_msis_instance)
  (sol : bimodal_selftarget_msis_solution option) :
  module_sis_solution option =
  if sol = None then None
  else Some (bimodal_to_module_sis_solution md x (oget sol)).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 10.32** (EasyCrypt line 157). **EasyCrypt name.**

```
dmodule_sis_from_bimodal
```

**Checked EasyCrypt statement.**

```
op dmodule_sis_from_bimodal (md : mode) : module_sis_instance distr =
  dmap (dbimodal_selftarget_msis_instance md)
  (bimodal_to_module_sis_instance md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 10.33** (EasyCrypt line 174). **EasyCrypt name.**

```
BimodalToModuleSIS_Lift_Game
```

**Checked EasyCrypt statement.**

```
module BimodalToModuleSIS_Lift_Game(
  B : BimodalSelfTargetMSIS_Solver
) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 10.34** (EasyCrypt line 177). **EasyCrypt name.**

```
main
```

**Checked EasyCrypt statement.**

```
proc main(md : mode) : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.35** (EasyCrypt line 227). **EasyCrypt name.**

```
ModuleSIS_ModeSolver
```

**Checked EasyCrypt statement.**

```
module type ModuleSIS_ModeSolver = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.36** (EasyCrypt line 228). **EasyCrypt name.**

```
solve
```

**Checked EasyCrypt statement.**

```
proc solve(md : mode, x : module_sis_instance) :  
  module_sis_solution option  
}.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.37** (EasyCrypt line 232). **EasyCrypt name.**

```
BimodalSelfTargetMSIS_ModeSolver
```

**Checked EasyCrypt statement.**

```
module type BimodalSelfTargetMSIS_ModeSolver = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.38** (EasyCrypt line 233). **EasyCrypt name.**

```
solve
```

**Checked EasyCrypt statement.**

```

proc solve(md : mode, x : bimodal_selftarget_msis_instance) :
  bimodal_selftarget_msis_solution option
}.

```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.39** (EasyCrypt line 237). **EasyCrypt name.**

```

ModuleSIS_ModeRelation_Game

```

**Checked EasyCrypt statement.**

```

module ModuleSIS_ModeRelation_Game(
  B : ModuleSIS_ModeSolver
) = {

```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 10.40** (EasyCrypt line 240). **EasyCrypt name.**

```

main

```

**Checked EasyCrypt statement.**

```

proc main(md : mode) : bool = {

```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.41** (EasyCrypt line 250). **EasyCrypt name.**

```

BimodalSelfTargetMSIS_ModeRelation_Game

```

**Checked EasyCrypt statement.**

```

module BimodalSelfTargetMSIS_ModeRelation_Game(
  B : BimodalSelfTargetMSIS_ModeSolver
) = {

```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 10.42** (EasyCrypt line 253). **EasyCrypt name.**

```

main

```

### Checked EasyCrypt statement.

```
proc main(md : mode) : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.43** (EasyCrypt line 263). **EasyCrypt name.**

```
ModuleSIS_LiftedDistribution_Relation_Game
```

### Checked EasyCrypt statement.

```
module ModuleSIS_LiftedDistribution_Relation_Game(  
  B : ModuleSIS_ModeSolver  
) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 10.44** (EasyCrypt line 266). **EasyCrypt name.**

```
main
```

### Checked EasyCrypt statement.

```
proc main(md : mode) : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.45** (EasyCrypt line 276). **EasyCrypt name.**

```
BimodalModeSolver_As_ModuleSIS
```

### Checked EasyCrypt statement.

```
module BimodalModeSolver_As_ModuleSIS(  
  B : BimodalSelfTargetMSIS_ModeSolver  
) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.46** (EasyCrypt line 279). **EasyCrypt name.**

```
solve
```

### Checked EasyCrypt statement.

```
proc solve(md : mode, x : module_sis_instance) :
  module_sis_solution option = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.47** (EasyCrypt line 310). **EasyCrypt name.**

```
MLWE_Reduction
```

**Checked EasyCrypt statement.**

```
module type MLWE_Reduction = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.48** (EasyCrypt line 311). **EasyCrypt name.**

```
main
```

**Checked EasyCrypt statement.**

```
proc main() : bool
}.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.49** (EasyCrypt line 314). **EasyCrypt name.**

```
BimodalSelfTargetMSIS_Reduction
```

**Checked EasyCrypt statement.**

```
module type BimodalSelfTargetMSIS_Reduction = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.50** (EasyCrypt line 315). **EasyCrypt name.**

```
main
```

**Checked EasyCrypt statement.**

```
proc main() : bool
}.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.51** (EasyCrypt line 318). **EasyCrypt name.**

```
ModuleSIS_Reduction
```

**Checked EasyCrypt statement.**

```
module type ModuleSIS_Reduction = {
```

**Formal mathematical reading.** Mathematical definition. This is an interface for an oracle, adversary, scheme, sampler, solver, or reduction. A later theorem quantified over this interface is universally quantified over all modules implementing these procedures.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.52** (EasyCrypt line 319). **EasyCrypt name.**

```
main
```

**Checked EasyCrypt statement.**

```
proc main() : bool
}.
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.53** (EasyCrypt line 322). **EasyCrypt name.**

```
MLWE_Game
```

**Checked EasyCrypt statement.**

```
module MLWE_Game(B : MLWE_Reduction) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 10.54** (EasyCrypt line 323). **EasyCrypt name.**

```
main
```

**Checked EasyCrypt statement.**



```
proc main() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.55** (EasyCrypt line 331). **EasyCrypt name.**

```
BimodalSelfTargetMSIS_Game
```

**Checked EasyCrypt statement.**

```
module BimodalSelfTargetMSIS_Game(B : BimodalSelfTargetMSIS_Reduction) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 10.56** (EasyCrypt line 332). **EasyCrypt name.**

```
main
```

**Checked EasyCrypt statement.**

```
proc main() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.57** (EasyCrypt line 340). **EasyCrypt name.**

```
ModuleSIS_Game
```

**Checked EasyCrypt statement.**

```
module ModuleSIS_Game(B : ModuleSIS_Reduction) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It defines the game or game interface whose success event is bounded by later theorems.

**Definition 10.58** (EasyCrypt line 341). **EasyCrypt name.**

```
main
```

**Checked EasyCrypt statement.**

```
proc main() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.59** (EasyCrypt line 349). **EasyCrypt name.**

```
BimodalMSIS_As_ModuleSIS
```

**Checked EasyCrypt statement.**

```
module BimodalMSIS_As_ModuleSIS(B : BimodalSelfTargetMSIS_Reduction) = {
```

**Formal mathematical reading.** Mathematical definition. This is a probabilistic program/game or a module adapter. Its procedures induce the probability space used by the game-based proof.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

**Definition 10.60** (EasyCrypt line 350). **EasyCrypt name.**

```
main
```

**Checked EasyCrypt statement.**

```
proc main() : bool = {
```

**Formal mathematical reading.** Mathematical definition. This procedure is a command in the probabilistic program. Its return value, state updates, and oracle calls are the operational content of the game.

**Role in the proof.** Use in this chapter. It supplies an executable component of the formal probabilistic model.

## Lemmas, Theorems, Relational Equivalences, and Their Proof Dependencies

**Theorem 10.1** (EasyCrypt line 43). *EasyCrypt name.*

```
module_sis_eval_zero_instance
```

*Checked EasyCrypt statement.*

```
lemma module_sis_eval_zero_instance md sol :  
  module_sis_eval md (module_sis_zero_instance md) sol = polyveck_zero md.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
matrix_vec_mul  
module_sis_eval  
module_sis_zero_instance
```

```
polyveck_add
polyveck_neg
polyveck_sub
```

**Theorem 10.2** (EasyCrypt line 50). *EasyCrypt name.*

```
module_sis_zero_valid
```

**Checked EasyCrypt statement.**

```
lemma module_sis_zero_valid md :
  module_sis_valid md (module_sis_zero_instance md) (polyvecl_zero md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
matrix_vec_mul
module_sis_solution_wf
module_sis_valid
module_sis_zero_instance
polyvecl_zero_wf
```

**Theorem 10.3** (EasyCrypt line 59). *EasyCrypt name.*

```
module_sis_eval_wf
```

**Checked EasyCrypt statement.**

```
lemma module_sis_eval_wf md x sol :
  polyveck_wf md (module_sis_eval md x sol).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 10.4** (EasyCrypt line 63). *EasyCrypt name.*

```
module_sis_valid_target_wf
```

**Checked EasyCrypt statement.**

```
lemma module_sis_valid_target_wf md x sol :
  module_sis_valid md x sol =>
  polyveck_wf md x.'2.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
matrix_vec_mul_wf
module_sis_valid
targetE
```

**Theorem 10.5** (EasyCrypt line 80). *EasyCrypt name.*

```
bimodal_to_module_sis_valid
```

**Checked EasyCrypt statement.**

```
lemma bimodal_to_module_sis_valid md x sol :
  bimodal_selftarget_msis_valid md x sol =>
  module_sis_valid md
    (bimodal_to_module_sis_instance md x)
    (bimodal_to_module_sis_solution md x sol).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
bimodal_selftarget_msis_valid
bimodal_to_module_sis_instance
bimodal_to_module_sis_solution
```

**Theorem 10.6** (EasyCrypt line 161). *EasyCrypt name.*

```
dmodule_sis_from_bimodalE
```

**Checked EasyCrypt statement.**

```
lemma dmodule_sis_from_bimodalE md :
  dmodule_sis_from_bimodal md = dbimodal_selftarget_msis_instance md.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
bimodal_to_module_sis_instance
dmap_id
dmodule_sis_from_bimodal
```

**Theorem 10.7** (EasyCrypt line 167). *EasyCrypt name.*

```
dmodule_sis_from_bimodal_lossless
```

**Checked EasyCrypt statement.**

```
lemma dmodule_sis_from_bimodal_lossless md :
  is_lossless (dmodule_sis_from_bimodal md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dbimodal_selftarget_msis_instance_ll
dmodule_sis_from_bimodalE
```

**Theorem 10.8** (EasyCrypt line 191). *EasyCrypt name.*

```
bimodal_lift_successE
```

**Checked EasyCrypt statement.**

```
lemma bimodal_lift_successE md x sol :
  (sol <> None /\ bimodal_selftarget_msis_valid md x (oget sol)) =
  (lift_bimodal_solution_option md x sol <> None /\
   module_sis_valid md
    (bimodal_to_module_sis_instance md x)
    (oget (lift_bimodal_solution_option md x sol))).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *case*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
None
bimodal_selftarget_msis_valid
bimodal_to_module_sis_instance
bimodal_to_module_sis_solution
lift_bimodal_solution_option
sol
sol_none
```

**Theorem 10.9** (EasyCrypt line 209). *EasyCrypt name.*

```
bimodal_solver_lift_exact
```

**Checked EasyCrypt statement.**

```
lemma bimodal_solver_lift_exact E m md :
```

```
Pr[BimodalSelfTargetMSIS_Relation_Game(B).main(md) @ &m : res] =
Pr[BimodalToModuleSIS_Lift_Game(B).main(md) @ &m : res].
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `byequiv`, `proc`, `wp`, `call`, `rnd`, `rewrite`. Named identifiers syntactically cited in the proof block:

```
bimodal_selftarget_msis_valid
bimodal_to_module_sis_instance
bimodal_to_module_sis_solution
glob
md
```

**Theorem 10.10** (EasyCrypt line 292). *EasyCrypt name.*

```
bimodal_mode_solver_lifted_distribution_exact
```

**Checked EasyCrypt statement.**

```
lemma bimodal_mode_solver_lifted_distribution_exact &m md :
  Pr[BimodalSelfTargetMSIS_ModeRelation_Game(B).main(md) @ &m : res] =
  Pr[ModuleSIS_LiftedDistribution_Relation_Game(
    BimodalModeSolver_As_ModuleSIS(B)).main(md) @ &m : res].
```

**Formal mathematical reading.** Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `byequiv`, `proc`, `inline`, `wp`, `call`, `rnd`, `rewrite`. Named identifiers syntactically cited in the proof block:

```
BimodalModeSolver_As_ModuleSIS
bimodal_selftarget_msis_valid
glob
md
solve
```

**Theorem 10.11** (EasyCrypt line 362). *EasyCrypt name.*

```
bimodal_msis_as_module_sis_exact
```

**Checked EasyCrypt statement.**

```
lemma bimodal_msis_as_module_sis_exact &m :
  Pr[BimodalSelfTargetMSIS_Game(B).main() @ &m : res] =
  Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(B)).main() @ &m : res].
```

**Formal mathematical reading.** *Mathematical theorem. This theorem has the form  $\Pr[G : E] = B$  is a game-probability statement. It is one of the exact hops, bad-event bounds, or final advantage inequalities used in the reduction.*

**Role in the proof.** *Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `byequiv`, `proc`, `inline`, `wp`, `call`. Named identifiers syntactically cited in the proof block:*

```
BimodalMSIS_As_ModuleSIS
glob
main
```

*End of generated formal inventory for HAETAE\_Assumptions.ec.*

# Chapter 11

## HAETAE\_Distributions.ec

**Mathematical role.** This file defines the probability distributions used by the model and proves their support, losslessness, and point-probability facts. It is the measure theory layer of the proof.

**Basic distributions.** The file defines byte, seed, CRH, coefficient, polynomial, vector, matrix, and challenge distributions. Typical lemmas state:

$$\text{is\_lossless}(D), \quad x \in \text{supp}(D) \Rightarrow P(x), \quad \mu(D, \{x\}) \leq b.$$

**Signing randomness.** Signing randomness is represented through an entropy token of cardinality

$$288230376151711744 = 2^{58}.$$

This is the source of the challenge-support lower bound used in the counted ROM losses. The point-bound lemmas are what justify replacing adversarial hit probabilities by  $1/|\mathcal{C}|$ -style bounds.

**Exact hyperball samplers.** The proof introduces structural, bounded-unit, checked-hyperball, and exact-hyperball signing-sample distributions. Cryptographically, these are different ways of sampling the signing response space. The transition from structural signing samples to exact hyperball samples is one of the main places where rejection-sampling loss enters the proof.

### Textbook Formal Inventory

This subsection records the exact formal material in `HAETAE_Distributions.ec`. It is generated from the proof-surface EasyCrypt file, so the line numbers and statements are meant to be read next to the checked source.

**Chapter role.** sampling distributions, support predicates, point bounds, and losslessness facts.

**Inventory size.** 6 context declarations, 66 definitions/game objects, 97 lemmas or relational theorems, and 0 explicit Hoare/Phoare judgments were found.

### Theory Context, Imports, and Quantified Modules

**Context 1 (line 1)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**



```
require import AllCore Distr DInterval DList IntDiv List Real StdOrder.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 2 (line 2)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import HAETAE_Params HAETAE_Algebra.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 3 (line 4)** EasyCrypt name.

```
HAETAE_Distributions
```

**Checked EasyCrypt statement.**

```
theory HAETAE_Distributions.
```

**Formal mathematical reading.** Mathematical context. This begins a namespace for the formal development in this file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 4 (line 6)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Algebra.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 5 (line 7)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Params.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 6 (line 8) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import RealOrder.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Definitions, Carrier Sets, Operations, Games, and Procedures**

**Definition 11.1 (EasyCrypt line 10). EasyCrypt name.**

```
signing_randomness_source
```

**Checked EasyCrypt statement.**

```
type signing_randomness_source = int.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 11.2 (EasyCrypt line 12). EasyCrypt name.**

```
dbyte
```

**Checked EasyCrypt statement.**

```
op dbyte : byte distr = dinter 0 255.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.3 (EasyCrypt line 13). EasyCrypt name.**

```
dseed
```

**Checked EasyCrypt statement.**

```
op dseed : seed distr = dlist dbyte seedbytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.4** (EasyCrypt line 14). **EasyCrypt name.**

```
dcrh
```

**Checked EasyCrypt statement.**

```
op dcrh : crh distr = dlist dbyte crhbytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.5** (EasyCrypt line 15). **EasyCrypt name.**

```
signing_entropy_token_cardinality
```

**Checked EasyCrypt statement.**

```
op signing_entropy_token_cardinality : int = 288230376151711744.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 11.6** (EasyCrypt line 16). **EasyCrypt name.**

```
signing_entropy_token_in_range
```

**Checked EasyCrypt statement.**

```
op signing_entropy_token_in_range (x : int) : bool =
  0 <= x /\ x < signing_entropy_token_cardinality.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 11.7** (EasyCrypt line 18). **EasyCrypt name.**

```
signing_randomness_byte_ok
```

**Checked EasyCrypt statement.**

```
op signing_randomness_byte_ok (b : byte) : bool = 0 <= b /\ b < 256.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 11.8** (EasyCrypt line 19). **EasyCrypt name.**

```
signing_entropy_token_to_coins
```

**Checked EasyCrypt statement.**

```
op signing_entropy_token_to_coins (x : int) : random_coins =
  [ x %% 256;
    (x %/ 256) %% 256;
    (x %/ 65536) %% 256;
    (x %/ 16777216) %% 256;
    (x %/ 4294967296) %% 256;
    (x %/ 1099511627776) %% 256;
    (x %/ 281474976710656) %% 256;
    x %/ 72057594037927936 ] ++
  nseq (seedbytes - 8) 0.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 11.9** (EasyCrypt line 29). **EasyCrypt name.**

```
signing_randomness_domain
```

**Checked EasyCrypt statement.**

```
op signing_randomness_domain (coins : random_coins) : bool =
  size coins = seedbytes /\
  all signing_randomness_byte_ok coins /\
  signing_entropy_token_in_range (signing_entropy_token_of_coins coins).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 11.10** (EasyCrypt line 33). **EasyCrypt name.**

```
dsigning_entropy_token
```

**Checked EasyCrypt statement.**

```
op dsigning_entropy_token : int distr =
  dinter 0 (signing_entropy_token_cardinality - 1).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 11.11** (EasyCrypt line 35). **EasyCrypt name.**

```
dsigning_randomness_source
```

**Checked EasyCrypt statement.**

```
op dsigning_randomness_source : signing_randomness_source distr =
  dsigning_entropy_token.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.12** (EasyCrypt line 37). **EasyCrypt name.**

```
signing_randomness_source_valid
```

**Checked EasyCrypt statement.**

```
op signing_randomness_source_valid (src : signing_randomness_source) : bool =
  signing_entropy_token_in_range src.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 11.13** (EasyCrypt line 39). **EasyCrypt name.**

```
signing_randomness_from_source
```

**Checked EasyCrypt statement.**

```
op signing_randomness_from_source (src : signing_randomness_source) :
  random_coins =
  signing_entropy_token_to_coins src.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 11.14** (EasyCrypt line 42). **EasyCrypt name.**

```
dsigning_randomness
```

**Checked EasyCrypt statement.**

```
op dsigning_randomness : random_coins distr =
  dmap dsigning_randomness_source signing_randomness_from_source.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.15** (EasyCrypt line 44). **EasyCrypt name.**

```
drandom_coins
```

**Checked EasyCrypt statement.**

```
op drandom_coins : random_coins distr = dsigning_randomness.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.16** (EasyCrypt line 45). **EasyCrypt name.**

```
signing_randomness_point_bound
```

**Checked EasyCrypt statement.**

```
op signing_randomness_point_bound : real =
  1/r / (signing_entropy_token_cardinality)%r.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 11.17** (EasyCrypt line 47). **EasyCrypt name.**

```
signing_randomness_token_spread
```

**Checked EasyCrypt statement.**

```
op signing_randomness_token_spread (d : random_coins distr) : bool =
  forall token,
    mu d (fun coins => signing_entropy_token_of_coins coins = token) <=
      signing_randomness_point_bound.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 11.18** (EasyCrypt line 51). **EasyCrypt name.**

```
signing_randomness_distribution_ok
```

**Checked EasyCrypt statement.**

```
op signing_randomness_distribution_ok (d : random_coins distr) : bool =
  is_lossless d /\
    (forall coins, coins \in d => signing_randomness_domain coins) /\
      signing_randomness_token_spread d.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 11.19** (EasyCrypt line 55). **EasyCrypt name.**

```
dcoeff
```

**Checked EasyCrypt statement.**

```
op dcoeff : coeff distr = dinter 0 (q - 1).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.20** (EasyCrypt line 56). **EasyCrypt name.**

```
dpoly
```

**Checked EasyCrypt statement.**

```
op dpoly : poly distr = dlist dcoeff n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.21** (EasyCrypt line 57). **EasyCrypt name.**

```
dpolyveck
```

**Checked EasyCrypt statement.**

```
op dpolyveck (md : mode) : polyveck distr = dlist dpoly (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.22** (EasyCrypt line 58). **EasyCrypt name.**

```
dpolyvecl
```

**Checked EasyCrypt statement.**

```
op dpolyvecl (md : mode) : polyvecl distr = dlist dpoly (mode_l md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.23** (EasyCrypt line 59). **EasyCrypt name.**

```
dmatrix
```

**Checked EasyCrypt statement.**

```
op dmatrix (md : mode) : matrix distr = dlist (dpolyvecl md) (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.24** (EasyCrypt line 65). **EasyCrypt name.**

```
dchallenge
```

**Checked EasyCrypt statement.**

```
op dchallenge (md : mode) : challenge distr =  
  dmap dcrh (challenge_from_seed md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.25** (EasyCrypt line 68). **EasyCrypt name.**

```
signing_sample_pair
```

**Checked EasyCrypt statement.**

```
type signing_sample_pair = polyvecl * polyveck.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.26** (EasyCrypt line 70). **EasyCrypt name.**

```
signing_sample_pair_y1
```

**Checked EasyCrypt statement.**

```
op signing_sample_pair_y1 (s : signing_sample_pair) : polyvecl = s.'1.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.27** (EasyCrypt line 71). **EasyCrypt name.**

```
signing_sample_pair_y2
```

**Checked EasyCrypt statement.**

```
op signing_sample_pair_y2 (s : signing_sample_pair) : polyveck = s.'2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.28** (EasyCrypt line 73). **EasyCrypt name.**

```
signing_sample_pair_wf
```

**Checked EasyCrypt statement.**



```
op signing_sample_pair_wf (md : mode) (s : signing_sample_pair) : bool =
  polyvecl_wf md (signing_sample_pair_y1 s) /\
  polyveck_wf md (signing_sample_pair_y2 s).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 11.29** (EasyCrypt line 77). **EasyCrypt name.**

```
signing_sample_pair_unit
```

**Checked EasyCrypt statement.**

```
op signing_sample_pair_unit (md : mode) (s : signing_sample_pair) : bool =
  polyvecl_unit md (signing_sample_pair_y1 s) /\
  polyveck_unit md (signing_sample_pair_y2 s).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.30** (EasyCrypt line 81). **EasyCrypt name.**

```
signing_sample_bound
```

**Checked EasyCrypt statement.**

```
op signing_sample_bound (md : mode) : int = mode_ln md.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 11.31** (EasyCrypt line 83). **EasyCrypt name.**

```
coeff_bounded_by
```

**Checked EasyCrypt statement.**

```
op coeff_bounded_by (b : int) (x : coeff) : bool =
  -b <= x /\ x <= b.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 11.32** (EasyCrypt line 86). **EasyCrypt name.**

```
poly_bounded_by
```

### Checked EasyCrypt statement.

```

op poly_bounded_by (b : int) (p : poly) : bool =
  poly_wf p /\
  forall i, 0 <= i < n =>
    coeff_bounded_by b (poly_coeff p i).

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

### Definition 11.33 (EasyCrypt line 91). EasyCrypt name.

```
polyvecl_bounded_by
```

### Checked EasyCrypt statement.

```

op polyvecl_bounded_by (md : mode) (b : int) (xs : polyvecl) : bool =
  polyvecl_wf md xs /\
  forall row, 0 <= row < mode_l md =>
    poly_bounded_by b (nth poly_zero xs row).

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

### Definition 11.34 (EasyCrypt line 96). EasyCrypt name.

```
polyveck_bounded_by
```

### Checked EasyCrypt statement.

```

op polyveck_bounded_by (md : mode) (b : int) (xs : polyveck) : bool =
  polyveck_wf md xs /\
  forall row, 0 <= row < mode_k md =>
    poly_bounded_by b (nth poly_zero xs row).

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

### Definition 11.35 (EasyCrypt line 101). EasyCrypt name.

```
signing_sample_pair_bounded
```

### Checked EasyCrypt statement.

```

op signing_sample_pair_bounded (md : mode) (s : signing_sample_pair) : bool =
  polyvecl_bounded_by md (signing_sample_bound md)
  (signing_sample_pair_y1 s) /\
  polyveck_bounded_by md (signing_sample_bound md)
  (signing_sample_pair_y2 s).

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 11.36** (EasyCrypt line 107). **EasyCrypt name.**

```
signing_sample_pair_sample_ok
```

**Checked EasyCrypt statement.**

```
op signing_sample_pair_sample_ok (md : mode) (s : signing_sample_pair) : bool =
  signing_sample_pair_wf md s /\
  signing_sample_pair_bounded md s.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 11.37** (EasyCrypt line 111). **EasyCrypt name.**

```
signing_sample_pair_of_coins
```

**Checked EasyCrypt statement.**

```
op signing_sample_pair_of_coins
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (coins : random_coins) : signing_sample_pair =
  (signing_sample_y1 md sk m ctx coins,
   signing_sample_y2 md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.38** (EasyCrypt line 117). **EasyCrypt name.**

```
dstructural_signing_sample_pair
```

**Checked EasyCrypt statement.**

```
op dstructural_signing_sample_pair
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (coins : random_coins) : signing_sample_pair distr =
  dunit (signing_sample_pair_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.39** (EasyCrypt line 122). **EasyCrypt name.**

```
dsigning_unit_coeff
```

### Checked EasyCrypt statement.

```
op dsigning_unit_coeff : coeff distr = duniform [-1; 1].
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

### Definition 11.40 (EasyCrypt line 123). EasyCrypt name.

```
dsigning_unit_poly
```

### Checked EasyCrypt statement.

```
op dsigning_unit_poly : poly distr = dlist dsigning_unit_coeff n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

### Definition 11.41 (EasyCrypt line 124). EasyCrypt name.

```
dsigning_unit_polyvec1
```

### Checked EasyCrypt statement.

```
op dsigning_unit_polyvec1 (md : mode) : polyvec1 distr =  
  dlist dsigning_unit_poly (mode_l md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

### Definition 11.42 (EasyCrypt line 126). EasyCrypt name.

```
dsigning_unit_polyveck
```

### Checked EasyCrypt statement.

```
op dsigning_unit_polyveck (md : mode) : polyveck distr =  
  dlist dsigning_unit_poly (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

### Definition 11.43 (EasyCrypt line 129). EasyCrypt name.

```
signing_bounded_coeff_cardinality
```

### Checked EasyCrypt statement.

```
op signing_bounded_coeff_cardinality (md : mode) : int =  
  2 * signing_sample_bound md + 1.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 11.44** (EasyCrypt line 131). **EasyCrypt name.**

```
signing_bounded_coeff_point_bound
```

**Checked EasyCrypt statement.**

```
op signing_bounded_coeff_point_bound (md : mode) : real =
  1%r / (signing_bounded_coeff_cardinality md)%r.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 11.45** (EasyCrypt line 133). **EasyCrypt name.**

```
dsigning_bounded_coeff
```

**Checked EasyCrypt statement.**

```
op dsigning_bounded_coeff (md : mode) : coeff distr =
  dinter (-(signing_sample_bound md)) (signing_sample_bound md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 11.46** (EasyCrypt line 135). **EasyCrypt name.**

```
dsigning_bounded_poly
```

**Checked EasyCrypt statement.**

```
op dsigning_bounded_poly (md : mode) : poly distr =
  dlist (dsigning_bounded_coeff md) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 11.47** (EasyCrypt line 137). **EasyCrypt name.**

```
dsigning_bounded_polyvecl
```

**Checked EasyCrypt statement.**

```
op dsigning_bounded_polyvecl (md : mode) : polyvecl distr =
  dlist (dsigning_bounded_poly md) (mode_l md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 11.48** (EasyCrypt line 139). **EasyCrypt name.**

```
dsigning_bounded_polyveck
```

**Checked EasyCrypt statement.**

```
op dsigning_bounded_polyveck (md : mode) : polyveck distr =
  dlist (dsigning_bounded_poly md) (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 11.49** (EasyCrypt line 145). **EasyCrypt name.**

```
dbounded_unit_signing_sample_pair
```

**Checked EasyCrypt statement.**

```
op dbounded_unit_signing_sample_pair (md : mode) :
  signing_sample_pair distr =
  dlet (dsigning_unit_polyveck1 md)
    (fun y1 =>
      dmap (dsigning_unit_polyveck md)
        (fun y2 => (y1, y2))).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 11.50** (EasyCrypt line 156). **EasyCrypt name.**

```
dchecked_hyperball_signing_sample_pair
```

**Checked EasyCrypt statement.**

```
op dchecked_hyperball_signing_sample_pair (md : mode) :
  signing_sample_pair distr =
  dlet (dsigning_bounded_polyveck1 md)
    (fun y1 =>
      dmap (dsigning_bounded_polyveck md)
        (fun y2 => (y1, y2))).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.51** (EasyCrypt line 163). **EasyCrypt name.**

```
hyperball_coeff_ok
```

**Checked EasyCrypt statement.**

```
op hyperball_coeff_ok (md : mode) (c : coeff) : bool =
  coeff_bounded_by (signing_sample_bound md) c.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 11.52** (EasyCrypt line 165). **EasyCrypt name.**

```
hyperball_poly_ok
```

**Checked EasyCrypt statement.**

```
op hyperball_poly_ok (md : mode) (p : poly) : bool =
  poly_bounded_by (signing_sample_bound md) p.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 11.53** (EasyCrypt line 167). **EasyCrypt name.**

```
hyperball_polyvecl_ok
```

**Checked EasyCrypt statement.**

```
op hyperball_polyvecl_ok (md : mode) (y1 : polyvecl) : bool =
  polyvecl_bounded_by md (signing_sample_bound md) y1.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 11.54** (EasyCrypt line 169). **EasyCrypt name.**

```
hyperball_polyveck_ok
```

**Checked EasyCrypt statement.**

```
op hyperball_polyveck_ok (md : mode) (y2 : polyveck) : bool =
  polyveck_bounded_by md (signing_sample_bound md) y2.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 11.55** (EasyCrypt line 171). **EasyCrypt name.**

```
hyperball_sample_ok
```

**Checked EasyCrypt statement.**

```
op hyperball_sample_ok (md : mode) (s : signing_sample_pair) : bool =
  hyperball_polyvecl_ok md (signing_sample_pair_y1 s) /\
  hyperball_polyveck_ok md (signing_sample_pair_y2 s).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 11.56** (EasyCrypt line 175). **EasyCrypt name.**

```
dhyperball_coeff
```

**Checked EasyCrypt statement.**

```
op dhyperball_coeff (md : mode) : coeff distr =
  dsigning_bounded_coeff md.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.57** (EasyCrypt line 177). **EasyCrypt name.**

```
dhyperball_poly
```

**Checked EasyCrypt statement.**

```
op dhyperball_poly (md : mode) : poly distr =
  dsigning_bounded_poly md.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.58** (EasyCrypt line 179). **EasyCrypt name.**

```
dhyperball_polyvecl
```

**Checked EasyCrypt statement.**

```
op dhyperball_polyvecl (md : mode) : polyvecl distr =
  dsigning_bounded_polyvecl md.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.59** (EasyCrypt line 181). **EasyCrypt name.**

```
dhyperball_polyveck
```

**Checked EasyCrypt statement.**



```
op dhyperball_polyveck (md : mode) : polyveck distr =
  dsigning_bounded_polyveck md.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.60** (EasyCrypt line 184). **EasyCrypt name.**

```
hyperball_coeff_point_bound
```

**Checked EasyCrypt statement.**

```
op hyperball_coeff_point_bound (md : mode) : real =
  signing_bounded_coeff_point_bound md.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 11.61** (EasyCrypt line 186). **EasyCrypt name.**

```
hyperball_poly_point_bound
```

**Checked EasyCrypt statement.**

```
op hyperball_poly_point_bound (md : mode) : real =
  hyperball_coeff_point_bound md ^ n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 11.62** (EasyCrypt line 188). **EasyCrypt name.**

```
hyperball_polyvecl_point_bound
```

**Checked EasyCrypt statement.**

```
op hyperball_polyvecl_point_bound (md : mode) : real =
  hyperball_poly_point_bound md ^ (mode_l md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 11.63** (EasyCrypt line 190). **EasyCrypt name.**

```
hyperball_polyveck_point_bound
```

**Checked EasyCrypt statement.**

```
op hyperball_polyveck_point_bound (md : mode) : real =
  hyperball_poly_point_bound md ^ (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 11.64** (EasyCrypt line 192). **EasyCrypt name.**

```
hyperball_sample_pair_point_bound
```

**Checked EasyCrypt statement.**

```
op hyperball_sample_pair_point_bound (md : mode) : real =
  hyperball_polyvecl_point_bound md *
  hyperball_polyveck_point_bound md.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 11.65** (EasyCrypt line 201). **EasyCrypt name.**

```
dexact_hyperball_signing_sample_pair
```

**Checked EasyCrypt statement.**

```
op dexact_hyperball_signing_sample_pair (md : mode) :
  signing_sample_pair distr =
  (dhyperball_polyvecl md) '*' (dhyperball_polyveck md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 11.66** (EasyCrypt line 205). **EasyCrypt name.**

```
signing_sample_pair_distribution_ok
```

**Checked EasyCrypt statement.**

```
op signing_sample_pair_distribution_ok
  (md : mode) (d : signing_sample_pair distr) : bool =
  is_lossless d /\
  (forall s, s \in d =>
    signing_sample_pair_sample_ok md s).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

## Lemmas, Theorems, Relational Equivalences, and Their Proof Dependencies

**Theorem 11.1** (EasyCrypt line 211). **EasyCrypt name.**

```
byte_distribution_lossless
```

**Checked EasyCrypt statement.**

```
lemma byte_distribution_lossless : is_lossless dbyte.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 11.2** (EasyCrypt line 214). **EasyCrypt name.**

```
seed_distribution_lossless
```

**Checked EasyCrypt statement.**

```
lemma seed_distribution_lossless : is_lossless dseed.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
byte_distribution_lossless  
dlist_ll  
dseed
```

**Theorem 11.3** (EasyCrypt line 220). **EasyCrypt name.**

```
crh_distribution_lossless
```

**Checked EasyCrypt statement.**

```
lemma crh_distribution_lossless : is_lossless dcrh.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
byte_distribution_lossless  
dcrh  
dlist_ll
```

**Theorem 11.4** (EasyCrypt line 226). **EasyCrypt name.**

```
signing_entropy_token_cardinality_positive
```

**Checked EasyCrypt statement.**

```
lemma signing_entropy_token_cardinality_positive :  
  0 < signing_entropy_token_cardinality.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 11.5 (EasyCrypt line 230). EasyCrypt name.**

```
signing_entropy_token_distribution_lossless
```

**Checked EasyCrypt statement.**

```
lemma signing_entropy_token_distribution_lossless :  
  is_lossless dsigning_entropy_token.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dinter_ll  
dsigning_entropy_token  
signing_entropy_token_cardinality
```

**Theorem 11.6 (EasyCrypt line 238). EasyCrypt name.**

```
signing_randomness_source_distribution_lossless
```

**Checked EasyCrypt statement.**

```
lemma signing_randomness_source_distribution_lossless :  
  is_lossless dsigning_randomness_source.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dsigning_randomness_source  
signing_entropy_token_distribution_lossless
```

**Theorem 11.7 (EasyCrypt line 245). EasyCrypt name.**

```
signing_coin_distribution_lossless
```

**Checked EasyCrypt statement.**

```
lemma signing_coin_distribution_lossless : is_lossless drandom_coins.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dmap_ll
drandom_coins
dsigning_randomness
signing_randomness_source_distribution_lossless
```

**Theorem 11.8** (EasyCrypt line 251). *EasyCrypt name.*

```
signing_entropy_token_to_coins_tokenK
```

**Checked EasyCrypt statement.**

```
lemma signing_entropy_token_to_coins_tokenK x :
  signing_entropy_token_of_coins (signing_entropy_token_to_coins x) = x.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
divz_eq
divz_mul
signing_entropy_token_of_coins
signing_entropy_token_to_coins
```

**Theorem 11.9** (EasyCrypt line 260). *EasyCrypt name.*

```
signing_randomness_from_source_tokenK
```

**Checked EasyCrypt statement.**

```
lemma signing_randomness_from_source_tokenK src :
  signing_entropy_token_of_coins (signing_randomness_from_source src) = src.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_entropy_token_to_coins_tokenK
signing_randomness_from_source
```

**Theorem 11.10** (EasyCrypt line 267). *EasyCrypt name.*

```
signing_entropy_token_to_coins_size
```

**Checked EasyCrypt statement.**

```
lemma signing_entropy_token_to_coins_size x :
  size (signing_entropy_token_to_coins x) = seedbytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*. Named identifiers syntactically cited in the proof block:

```
exact
hsize
max
nseq
seedbytes
signing_entropy_token_to_coins
size
size_nseq
trivial
```

**Theorem 11.11** (EasyCrypt line 279). *EasyCrypt name.*

```
signing_entropy_token_to_coins_byte_domain
```

**Checked EasyCrypt statement.**

```
lemma signing_entropy_token_to_coins_byte_domain x :
  signing_entropy_token_in_range x =>
  all signing_randomness_byte_ok (signing_entropy_token_to_coins x).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
all_nseq
divz_ge0
```

```
ltz_divLR
ltz_pmod
modz_ge0
seedbytes
signing_entropy_token_cardinality
signing_entropy_token_in_range
signing_entropy_token_to_coins
signing_randomness_byte_ok
xrange
```

**Theorem 11.12** (EasyCrypt line 292). *EasyCrypt name.*

```
signing_entropy_token_to_coins_domain
```

**Checked EasyCrypt statement.**

```
lemma signing_entropy_token_to_coins_domain x :
  signing_entropy_token_in_range x =>
  signing_randomness_domain (signing_entropy_token_to_coins x).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
signing_entropy_token_to_coins_byte_domain
signing_entropy_token_to_coins_size
signing_entropy_token_to_coins_tokenK
signing_randomness_domain
xrange
```

**Theorem 11.13** (EasyCrypt line 305). *EasyCrypt name.*

```
signing_randomness_from_source_domain
```

**Checked EasyCrypt statement.**

```
lemma signing_randomness_from_source_domain src :
  signing_randomness_source_valid src =>
  signing_randomness_domain (signing_randomness_from_source src).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
signing_entropy_token_to_coins_domain
signing_randomness_from_source
signing_randomness_source_valid
```

**Theorem 11.14** (EasyCrypt line 314). *EasyCrypt name.*

```
signing_entropy_token_distribution_point_bound
```

**Checked EasyCrypt statement.**

```
lemma signing_entropy_token_distribution_point_bound x :
  mu dsigning_entropy_token (pred1 x) <=
  signing_randomness_point_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
RField
dinter1E
div1r
divr_ge0
dsigning_entropy_token
pred1
signing_entropy_token_cardinality
signing_randomness_point_bound
trivial
```

**Theorem 11.15** (EasyCrypt line 326). *EasyCrypt name.*

```
signing_randomness_source_distribution_point_bound
```

**Checked EasyCrypt statement.**

```
lemma signing_randomness_source_distribution_point_bound src :
  mu dsigning_randomness_source (pred1 src) <=
  signing_randomness_point_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dsigning_randomness_source
signing_entropy_token_distribution_point_bound
```

**Theorem 11.16** (EasyCrypt line 334). *EasyCrypt name.*

```
signing_coin_distribution_token_point_bound
```

**Checked EasyCrypt statement.**

```
lemma signing_coin_distribution_token_point_bound token :
```



```
mu drandom_coins
  (fun coins => signing_entropy_token_of_coins coins = token) <=
    signing_randomness_point_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dmapE
drandom_coins
dsigning_entropy_token
dsigning_randomness
ler_trans
mu
mu_le
pred1
signing_entropy_token_distribution_point_bound
signing_entropy_token_to_coins_tokenK
token
token_eq
```

**Theorem 11.17** (EasyCrypt line 348). *EasyCrypt name.*

```
signing_coin_distribution_token_spread
```

**Checked EasyCrypt statement.**

```
lemma signing_coin_distribution_token_spread :
  signing_randomness_token_spread drandom_coins.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
signing_coin_distribution_token_point_bound
signing_randomness_token_spread
```

**Theorem 11.18** (EasyCrypt line 355). *EasyCrypt name.*

```
signing_coin_distribution_domain
```

**Checked EasyCrypt statement.**

```
lemma signing_coin_distribution_domain coins :
  coins \in drandom_coins =>
    signing_randomness_domain coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `apply`, `smt`. Named identifiers syntactically cited in the proof block:

```
drandom_coins
dsigning_entropy_token
dsigning_randomness
dsigning_randomness_source
signing_entropy_token_cardinality
signing_entropy_token_in_range
signing_entropy_token_to_coins_domain
signing_randomness_from_source
src
src_supp
supp_dinter
supp_dmap
```

**Theorem 11.19** (EasyCrypt line 371). *EasyCrypt name.*

```
signing_coin_distribution_domain_full
```

**Checked EasyCrypt statement.**

```
lemma signing_coin_distribution_domain_full :
  mu drandom_coins signing_randomness_domain = 1%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `apply`. Named identifiers syntactically cited in the proof block:

```
eq1_mu
signing_coin_distribution_domain
signing_coin_distribution_lossless
```

**Theorem 11.20** (EasyCrypt line 379). *EasyCrypt name.*

```
signing_coin_distribution_ok
```

**Checked EasyCrypt statement.**

```
lemma signing_coin_distribution_ok :
  signing_randomness_distribution_ok drandom_coins.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
signing_coin_distribution_domain
signing_coin_distribution_lossless
signing_coin_distribution_token_spread
signing_randomness_distribution_ok
```

**Theorem 11.21** (EasyCrypt line 390). *EasyCrypt name.*

```
coeff_distribution_lossless
```

**Checked EasyCrypt statement.**

```
lemma coeff_distribution_lossless : is_lossless dcoeff.
```

**Formal mathematical reading.** *Mathematical theorem.* This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *No proof block was collected for this item.*

**Theorem 11.22** (EasyCrypt line 393). *EasyCrypt name.*

```
poly_distribution_lossless
```

**Checked EasyCrypt statement.**

```
lemma poly_distribution_lossless : is_lossless dpoly.
```

**Formal mathematical reading.** *Mathematical theorem.* This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
coeff_distribution_lossless
dlist_ll
dpoly
```

**Theorem 11.23** (EasyCrypt line 399). *EasyCrypt name.*

```
polyveck_distribution_lossless
```

**Checked EasyCrypt statement.**

```
lemma polyveck_distribution_lossless md : is_lossless (dpolyveck md).
```

**Formal mathematical reading.** *Mathematical theorem.* This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dlist_ll
dpolyveck
poly_distribution_lossless
```

**Theorem 11.24** (EasyCrypt line 405). *EasyCrypt name.*

```
polyvecl_distribution_lossless
```

**Checked EasyCrypt statement.**

```
lemma polyvecl_distribution_lossless md : is_lossless (dpolyvecl md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dlist_ll
dpolyvecl
poly_distribution_lossless
```

**Theorem 11.25** (EasyCrypt line 411). *EasyCrypt name.*

```
matrix_distribution_lossless
```

**Checked EasyCrypt statement.**

```
lemma matrix_distribution_lossless md : is_lossless (dmatrix md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dlist_ll
dmatrix
polyvecl_distribution_lossless
```

**Theorem 11.26** (EasyCrypt line 417). *EasyCrypt name.*

```
challenge_distribution_lossless
```

**Checked EasyCrypt statement.**

```
lemma challenge_distribution_lossless md : is_lossless (dchallenge md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
crh_distribution_lossless
dchallenge
dmap_ll
```

**Theorem 11.27** (*EasyCrypt* line 423). *EasyCrypt* name.

```
signing_sample_pair_of_coins_wf
```

**Checked *EasyCrypt* statement.**

```
lemma signing_sample_pair_of_coins_wf md sk m ctx coins :
  signing_sample_pair_wf md
  (signing_sample_pair_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
signing_sample_pair_of_coins
signing_sample_pair_wf
signing_sample_pair_y1
signing_sample_pair_y2
signing_sample_y1_wf
signing_sample_y2_wf
```

**Theorem 11.28** (*EasyCrypt* line 434). *EasyCrypt* name.

```
signing_sample_pair_of_coins_unit
```

**Checked *EasyCrypt* statement.**

```
lemma signing_sample_pair_of_coins_unit md sk m ctx coins :
  signing_sample_pair_unit md
  (signing_sample_pair_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
signing_sample_pair_of_coins
signing_sample_pair_unit
signing_sample_pair_y1
signing_sample_pair_y2
signing_sample_y1_unit
signing_sample_y2_unit
```

**Theorem 11.29** (EasyCrypt line 445). *EasyCrypt name.*

```
signing_sample_bound_gt0
```

**Checked EasyCrypt statement.**

```
lemma signing_sample_bound_gt0 md :
  0 < signing_sample_bound md.
```

**Formal mathematical reading.** *Mathematical theorem.* This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *No proof block was collected for this item.*

**Theorem 11.30** (EasyCrypt line 449). *EasyCrypt name.*

```
unit_coeff_bounded_by_signing_sample_bound
```

**Checked EasyCrypt statement.**

```
lemma unit_coeff_bounded_by_signing_sample_bound md c :
  unit_coeff_ok c =>
  coeff_bounded_by (signing_sample_bound md) c.
```

**Formal mathematical reading.** *Mathematical theorem.* This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *smt*. Named identifiers syntactically cited in the proof block:

```
coeff_bounded_by
md
signing_sample_bound
unit_coeff_ok
```

**Theorem 11.31** (EasyCrypt line 457). *EasyCrypt name.*

```
poly_unit_bounded_by_signing_sample_bound
```

**Checked EasyCrypt statement.**

```
lemma poly_unit_bounded_by_signing_sample_bound md p :
  poly_unit p =>
  poly_bounded_by (signing_sample_bound md) p.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
allP
mem_nth
p_all
p_wf
poly_bounded_by
poly_coeff
poly_unit
unit_coeff_bounded_by_signing_sample_bound
```

**Theorem 11.32** (EasyCrypt line 470). *EasyCrypt name.*

```
polyvecl_unit_bounded_by_signing_sample_bound
```

**Checked EasyCrypt statement.**

```
lemma polyvecl_unit_bounded_by_signing_sample_bound md y1 :
  polyvecl_unit md y1 =>
  polyvecl_bounded_by md (signing_sample_bound md) y1.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
allP
mem_nth
poly_unit_bounded_by_signing_sample_bound
polyvecl_bounded_by
polyvecl_unit
y1_all
y1_wf
```

**Theorem 11.33** (EasyCrypt line 482). *EasyCrypt name.*

```
polyveck_unit_bounded_by_signing_sample_bound
```

**Checked EasyCrypt statement.**

```
lemma polyveck_unit_bounded_by_signing_sample_bound md y2 :
  polyveck_unit md y2 =>
  polyveck_bounded_by md (signing_sample_bound md) y2.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
allP
mem_nth
poly_unit_bounded_by_signing_sample_bound
polyveck_bounded_by
polyveck_unit
y2_all
y2_wf
```

**Theorem 11.34** (EasyCrypt line 494). *EasyCrypt name.*

```
signing_sample_pair_unit_bounded
```

**Checked EasyCrypt statement.**

```
lemma signing_sample_pair_unit_bounded md s :
  signing_sample_pair_unit md s =>
  signing_sample_pair_bounded md s.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
polyveck_unit_bounded_by_signing_sample_bound
polyvecl_unit_bounded_by_signing_sample_bound
signing_sample_pair_bounded
signing_sample_pair_unit
y1_unit
y2_unit
```

**Theorem 11.35** (EasyCrypt line 505). *EasyCrypt name.*

```
signing_sample_pair_of_coins_sample_ok
```

**Checked EasyCrypt statement.**

```
lemma signing_sample_pair_of_coins_sample_ok md sk m ctx coins :
  signing_sample_pair_sample_ok md
  (signing_sample_pair_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.



**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
signing_sample_pair_of_coins_unit
signing_sample_pair_of_coins_wf
signing_sample_pair_sample_ok
signing_sample_pair_unit_bounded
```

**Theorem 11.36** (EasyCrypt line 516). *EasyCrypt name.*

```
structural_signing_sample_pair_lossless
```

**Checked EasyCrypt statement.**

```
lemma structural_signing_sample_pair_lossless md sk m ctx coins :
  is_lossless
    (dstructural_signing_sample_pair md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dstructural_signing_sample_pair
dunit_ll
```

**Theorem 11.37** (EasyCrypt line 523). *EasyCrypt name.*

```
structural_signing_sample_pair_support
```

**Checked EasyCrypt statement.**

```
lemma structural_signing_sample_pair_support md sk m ctx coins s :
  s \in dstructural_signing_sample_pair md sk m ctx coins =>
    signing_sample_pair_wf md s /\ signing_sample_pair_unit md s.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
dstructural_signing_sample_pair
signing_sample_pair_of_coins_unit
signing_sample_pair_of_coins_wf
supp_dunit
```

**Theorem 11.38** (EasyCrypt line 534). *EasyCrypt name.*

```
structural_signing_sample_pair_sample_ok
```

**Checked EasyCrypt statement.**

```
lemma structural_signing_sample_pair_sample_ok md sk m ctx coins s :
  s \in dstructural_signing_sample_pair md sk m ctx coins =>
  signing_sample_pair_sample_ok md s.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dstructural_signing_sample_pair
signing_sample_pair_of_coins_sample_ok
supp_dunit
```

**Theorem 11.39** (EasyCrypt line 543). *EasyCrypt name.*

```
structural_signing_sample_pair_distribution_ok
```

**Checked EasyCrypt statement.**

```
lemma structural_signing_sample_pair_distribution_ok md sk m ctx coins :
  signing_sample_pair_distribution_ok md
  (dstructural_signing_sample_pair md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
signing_sample_pair_distribution_ok
structural_signing_sample_pair_lossless
structural_signing_sample_pair_sample_ok
```

**Theorem 11.40** (EasyCrypt line 553). *EasyCrypt name.*

```
signing_unit_coeff_lossless
```

**Checked EasyCrypt statement.**

```
lemma signing_unit_coeff_lossless :
  is_lossless dsigning_unit_coeff.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dsigning_unit_coeff
duniform_ll
```

**Theorem 11.41** (EasyCrypt line 560). *EasyCrypt name*.

```
signing_unit_coeff_support
```

**Checked EasyCrypt statement.**

```
lemma signing_unit_coeff_support c :
  c \in dsigning_unit_coeff =>
    unit_coeff_ok c.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
dsigning_unit_coeff
supp_duniform
unit_coeff_ok
```

**Theorem 11.42** (EasyCrypt line 568). *EasyCrypt name*.

```
signing_bounded_coeff_cardinality_positive
```

**Checked EasyCrypt statement.**

```
lemma signing_bounded_coeff_cardinality_positive md :
  0 < signing_bounded_coeff_cardinality md.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
signing_bounded_coeff_cardinality
signing_sample_bound_gt0
```

**Theorem 11.43** (EasyCrypt line 575). *EasyCrypt name*.

```
signing_bounded_coeff_lossless
```

**Checked EasyCrypt statement.**

```
lemma signing_bounded_coeff_lossless md :  
  is_lossless (dsigning_bounded_coeff md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
dinter_ll  
dsigning_bounded_coeff  
signing_sample_bound_gt0
```

**Theorem 11.44** (EasyCrypt line 583). *EasyCrypt name.*

```
signing_bounded_coeff_support
```

**Checked EasyCrypt statement.**

```
lemma signing_bounded_coeff_support md c :  
  c \in dsigning_bounded_coeff md =>  
  coeff_bounded_by (signing_sample_bound md) c.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
coeff_bounded_by  
dsigning_bounded_coeff  
supp_dinter
```

**Theorem 11.45** (EasyCrypt line 590). *EasyCrypt name.*

```
signing_bounded_coeff_supportP
```

**Checked EasyCrypt statement.**

```
lemma signing_bounded_coeff_supportP md c :  
  c \in dsigning_bounded_coeff md <=>  
  coeff_bounded_by (signing_sample_bound md) c.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:

```
coeff_bounded_by
dsigning_bounded_coeff
supp_dinter
```

**Theorem 11.46** (EasyCrypt line 597). *EasyCrypt name.*

```
signing_bounded_coeff_point_bound
```

**Checked EasyCrypt statement.**

```
lemma signing_bounded_coeff_point_bound md c :
  mu (dsigning_bounded_coeff md) (pred1 c) <=
  signing_bounded_coeff_point_bound md.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `case`, `smt`. Named identifiers syntactically cited in the proof block:

```
dinter1E
dsigning_bounded_coeff
md
mode_ln
pred1
signing_bounded_coeff_cardinality
signing_bounded_coeff_point_bound
signing_sample_bound
trivial
```

**Theorem 11.47** (EasyCrypt line 616). *EasyCrypt name.*

```
signing_bounded_coeff_point_bound_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma signing_bounded_coeff_point_bound_nonnegative md :
  0%r <= signing_bounded_coeff_point_bound md.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `apply`, `smt`. Named identifiers syntactically cited in the proof block:

```
divr_ge0
signing_bounded_coeff_cardinality_positive
signing_bounded_coeff_point_bound
trivial
```

**Theorem 11.48** (EasyCrypt line 625). *EasyCrypt name.*

```
dlist_size_point_bound
```

**Checked EasyCrypt statement.**

```
lemma dlist_size_point_bound ['a] (d : 'a distr) bd xs :
  0%r <= bd =>
  (forall x, mu d (pred1 x) <= bd) =>
  mu (dlist d (size xs)) (pred1 xs) <= bd ^ (size xs).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *elim*, *apply*. Named identifiers syntactically cited in the proof block:

```
BRM
Bigreal
RField
StdBigop
bd_ge0
big_cons
big_ind
big_nil
dlist1E
expr0
exprS
ge0_mu
ih
ler_pmul
mulr_ge0
point_bound
predT
size
size_ge0
trivial
xs
```

**Theorem 11.49** (EasyCrypt line 647). *EasyCrypt name.*

```
dlist_point_bound
```

**Checked EasyCrypt statement.**

```
lemma dlist_point_bound ['a] (d : 'a distr) len bd xs :
  0 <= len =>
  0%r <= bd =>
  (forall x, mu d (pred1 x) <= bd) =>
  mu (dlist d len) (pred1 xs) <= bd ^ len.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *case*, *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
bd_ge0
dlist1E
dlist_size_point_bound
expr_ge0
len
lenE
len_ge0
point_bound
size
xs
```

**Theorem 11.50** (EasyCrypt line 661). *EasyCrypt name*.

```
hyperball_coeff_point_bound_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma hyperball_coeff_point_bound_nonnegative md :
  0%r <= hyperball_coeff_point_bound md.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
hyperball_coeff_point_bound
signing_bounded_coeff_point_bound_nonnegative
```

**Theorem 11.51** (EasyCrypt line 668). *EasyCrypt name*.

```
hyperball_coeff_point_boundE
```

**Checked EasyCrypt statement.**

```
lemma hyperball_coeff_point_boundE md c :
  mu (dhyperball_coeff md) (pred1 c) <=
  hyperball_coeff_point_bound md.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

dhyperball_coeff
hyperball_coeff_point_bound
signing_bounded_coeff_point_bound

```

**Theorem 11.52** (EasyCrypt line 676). *EasyCrypt name.*

```

signing_unit_poly_lossless

```

**Checked EasyCrypt statement.**

```

lemma signing_unit_poly_lossless :
  is_lossless dsigning_unit_poly.

```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

dlist_ll
dsigning_unit_poly
signing_unit_coeff_lossless

```

**Theorem 11.53** (EasyCrypt line 683). *EasyCrypt name.*

```

signing_unit_poly_support

```

**Checked EasyCrypt statement.**

```

lemma signing_unit_poly_support p :
  p \in dsigning_unit_poly =>
  poly_unit p.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*, *apply*. Named identifiers syntactically cited in the proof block:

```

allP
c_mem
dsigning_unit_poly
n_gt0
p_all
p_sz
poly_unit
poly_wf
signing_unit_coeff_support
supp_dlist

```

**Theorem 11.54** (EasyCrypt line 695). *EasyCrypt name.*



```
signing_bounded_poly_lossless
```

**Checked EasyCrypt statement.**

```
lemma signing_bounded_poly_lossless md :  
  is_lossless (dsigning_bounded_poly md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dlist_ll  
dsigning_bounded_poly  
signing_bounded_coeff_lossless
```

**Theorem 11.55** (EasyCrypt line 702). *EasyCrypt name.*

```
signing_bounded_poly_support
```

**Checked EasyCrypt statement.**

```
lemma signing_bounded_poly_support md p :  
  p \in dsigning_bounded_poly md =>  
  poly_bounded_by (signing_sample_bound md) p.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP  
dsigning_bounded_poly  
i_rng  
mem_nth  
n_gt0  
p_all  
p_sz  
poly_bounded_by  
poly_coeff  
poly_wf  
signing_bounded_coeff_support  
supp_dlist
```

**Theorem 11.56** (EasyCrypt line 717). *EasyCrypt name.*

```
signing_bounded_poly_supportP
```

**Checked EasyCrypt statement.**

```

lemma signing_bounded_poly_supportP md p :
  p \in dsigning_bounded_poly md <=>
  poly_bounded_by (signing_sample_bound md) p.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```

allP
c_mem
dsigning_bounded_poly
i_rng
mem_nth
n_gt0
nthP
p_all
p_bound
p_sz
poly_bounded_by
poly_coeff
poly_wf
signing_bounded_coeff_support
signing_bounded_coeff_supportP
supp_dlist

```

**Theorem 11.57** (EasyCrypt line 738). *EasyCrypt name.*

```
hyperball_poly_point_bound_nonnegative
```

**Checked EasyCrypt statement.**

```

lemma hyperball_poly_point_bound_nonnegative md :
  0%r <= hyperball_poly_point_bound md.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

expr_ge0
hyperball_coeff_point_bound_nonnegative
hyperball_poly_point_bound

```

**Theorem 11.58** (EasyCrypt line 745). *EasyCrypt name.*

```
hyperball_poly_lossless
```

**Checked EasyCrypt statement.**

```
lemma hyperball_poly_lossless md :
  is_lossless (dhyperball_poly md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dhyperball_poly
signing_bounded_poly_lossless
```

**Theorem 11.59** (*EasyCrypt* line 751). *EasyCrypt* name.

```
hyperball_poly_support
```

**Checked *EasyCrypt* statement.**

```
lemma hyperball_poly_support md p :
  p \in dhyperball_poly md =>
  hyperball_poly_ok md p.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dhyperball_poly
hyperball_poly_ok
signing_bounded_poly_support
```

**Theorem 11.60** (*EasyCrypt* line 759). *EasyCrypt* name.

```
hyperball_poly_supportP
```

**Checked *EasyCrypt* statement.**

```
lemma hyperball_poly_supportP md p :
  p \in dhyperball_poly md <=> hyperball_poly_ok md p.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

dhyperball_poly
hyperball_poly_ok
signing_bounded_poly_supportP

```

**Theorem 11.61** (EasyCrypt line 766). *EasyCrypt name.*

```
hyperball_poly_point_boundE
```

*Checked EasyCrypt statement.*

```

lemma hyperball_poly_point_boundE md p :
  mu (dhyperball_poly md) (pred1 p) <=
  hyperball_poly_point_bound md.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```

dhyperball_poly
dlist_point_bound
dsigning_bounded_poly
hyperball_coeff_point_boundE
hyperball_coeff_point_bound_nonnegative
hyperball_poly_point_bound
n_gt0

```

**Theorem 11.62** (EasyCrypt line 778). *EasyCrypt name.*

```
signing_unit_polyvecl_lossless
```

*Checked EasyCrypt statement.*

```

lemma signing_unit_polyvecl_lossless md :
  is_lossless (dsigning_unit_polyvecl md).

```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

dlist_ll
dsigning_unit_polyvecl
signing_unit_poly_lossless

```

**Theorem 11.63** (EasyCrypt line 785). *EasyCrypt name.*

```
signing_unit_polyvecl_lossless
```

**Checked EasyCrypt statement.**

```
lemma signing_unit_polyveck_lossless md :  
  is_lossless (dsigning_unit_polyveck md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dlist_ll  
dsigning_unit_polyveck  
signing_unit_poly_lossless
```

**Theorem 11.64 (EasyCrypt line 792). EasyCrypt name.**

```
signing_bounded_polyveck_lossless
```

**Checked EasyCrypt statement.**

```
lemma signing_bounded_polyveck_lossless md :  
  is_lossless (dsigning_bounded_polyveck md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dlist_ll  
dsigning_bounded_polyveck  
signing_bounded_poly_lossless
```

**Theorem 11.65 (EasyCrypt line 799). EasyCrypt name.**

```
signing_bounded_polyveck_lossless
```

**Checked EasyCrypt statement.**

```
lemma signing_bounded_polyveck_lossless md :  
  is_lossless (dsigning_bounded_polyveck md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dlist_ll
dsigning_bounded_polyveck
signing_bounded_poly_lossless
```

**Theorem 11.66** (EasyCrypt line 806). *EasyCrypt name.*

```
signing_unit_polyvecl_support
```

**Checked EasyCrypt statement.**

```
lemma signing_unit_polyvecl_support md y1 :
  y1 \in dsigning_unit_polyvecl md =>
  polyvecl_unit md y1.
```

**Formal mathematical reading.** *Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.* Main tactics visible in the proof script: *rewrite*, *smt*, *split*, *apply*, *have*. Named identifiers syntactically cited in the proof block:

```
allP
dsigning_unit_poly
dsigning_unit_polyvecl
mode_l_gt0
p_mem
p_supp
p_wf
poly_unit
polyvecl_unit
polyvecl_wf
signing_unit_poly_support
supp_dlist
y1_all
y1_sz
```

**Theorem 11.67** (EasyCrypt line 825). *EasyCrypt name.*

```
signing_unit_polyveck_support
```

**Checked EasyCrypt statement.**

```
lemma signing_unit_polyveck_support md y2 :
  y2 \in dsigning_unit_polyveck md =>
  polyveck_unit md y2.
```

**Formal mathematical reading.** *Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.* Main tactics visible in the proof script: *rewrite*, *smt*, *split*, *apply*, *have*. Named identifiers syntactically cited in the proof block:

```

allP
dsigning_unit_poly
dsigning_unit_polyveck
mode_k_gt0
p_mem
p_supp
p_wf
poly_unit
polyveck_unit
polyveck_wf
signing_unit_poly_support
supp_dlist
y2_all
y2_sz

```

**Theorem 11.68** (EasyCrypt line 844). *EasyCrypt name.*

```
signing_bounded_polyveck_support
```

**Checked EasyCrypt statement.**

```

lemma signing_bounded_polyveck_support md y1 :
  y1 \in dsigning_bounded_polyveck md =>
  polyveck_bounded_by md (signing_sample_bound md) y1.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*, *split*, *apply*, *have*. Named identifiers syntactically cited in the proof block:

```

allP
dsigning_bounded_poly
dsigning_bounded_polyveck
md
mem_nth
mode_l_gt0
p_mem
p_supp
p_wf
poly_bounded_by
polyveck_bounded_by
polyveck_wf
row
row_rng
signing_bounded_poly_support
supp_dlist
y1_all
y1_sz

```

**Theorem 11.69** (EasyCrypt line 865). *EasyCrypt name.*

```
signing_bounded_polyveck_supportP
```

**Checked EasyCrypt statement.**

```

lemma signing_bounded_polyveck_supportP md y1 :
  y1 \in dsigning_bounded_polyveck md <=>

```

```
polyvecl_bounded_by md (signing_sample_bound md) y1.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*, *split*, *have*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
dsigning_bounded_poly
dsigning_bounded_polyvecl
md
mem_nth
mode_l_gt0
nthP
p_mem
p_supp
p_wf
poly_bounded_by
polyvecl_bounded_by
polyvecl_wf
row
row_rng
signing_bounded_poly_support
signing_bounded_poly_supportP
supp_dlist
y1_all
y1_allwf
y1_bound
y1_sz
```

**Theorem 11.70** (EasyCrypt line 892). *EasyCrypt name.*

```
signing_bounded_polyveck_support
```

**Checked EasyCrypt statement.**

```
lemma signing_bounded_polyveck_support md y2 :
  y2 \in dsigning_bounded_polyveck md =>
  polyveck_bounded_by md (signing_sample_bound md) y2.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*, *split*, *apply*, *have*. Named identifiers syntactically cited in the proof block:

```
allP
dsigning_bounded_poly
dsigning_bounded_polyveck
md
mem_nth
mode_k_gt0
p_mem
```



```

p_supp
p_wf
poly_bounded_by
polyveck_bounded_by
polyveck_wf
row
row_rng
signing_bounded_poly_support
supp_dlist
y2_all
y2_sz

```

**Theorem 11.71** (EasyCrypt line 913). *EasyCrypt name.*

```
signing_bounded_polyveck_supportP
```

**Checked EasyCrypt statement.**

```

lemma signing_bounded_polyveck_supportP md y2 :
  y2 \in dsigning_bounded_polyveck md <=>
  polyveck_bounded_by md (signing_sample_bound md) y2.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*, *split*, *have*, *apply*. Named identifiers syntactically cited in the proof block:

```

allP
dsigning_bounded_poly
dsigning_bounded_polyveck
md
mem_nth
mode_k_gt0
nthP
p_mem
p_supp
p_wf
poly_bounded_by
polyveck_bounded_by
polyveck_wf
row
row_rng
signing_bounded_poly_support
signing_bounded_poly_supportP
supp_dlist
y2_all
y2_allwf
y2_bound
y2_sz

```

**Theorem 11.72** (EasyCrypt line 940). *EasyCrypt name.*

```
hyperball_polyvecl_point_bound_nonnegative
```

**Checked EasyCrypt statement.**

```

lemma hyperball_polyvecl_point_bound_nonnegative md :
  0%r <= hyperball_polyvecl_point_bound md.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
expr_ge0
hyperball_poly_point_bound_nonnegative
hyperball_polyvecl_point_bound
```

**Theorem 11.73** (EasyCrypt line 947). *EasyCrypt name*.

```
hyperball_polyveck_point_bound_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma hyperball_polyveck_point_bound_nonnegative md :
  0%r <= hyperball_polyveck_point_bound md.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
expr_ge0
hyperball_poly_point_bound_nonnegative
hyperball_polyveck_point_bound
```

**Theorem 11.74** (EasyCrypt line 954). *EasyCrypt name*.

```
hyperball_sample_pair_point_bound_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma hyperball_sample_pair_point_bound_nonnegative md :
  0%r <= hyperball_sample_pair_point_bound md.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
hyperball_polyveck_point_bound_nonnegative
hyperball_polyvecl_point_bound_nonnegative
```

```
hyperball_sample_pair_point_bound
mulr_ge0
```

**Theorem 11.75** (EasyCrypt line 963). *EasyCrypt name.*

```
hyperball_polyvecl_lossless
```

**Checked EasyCrypt statement.**

```
lemma hyperball_polyvecl_lossless md :
  is_lossless (dhyperball_polyvecl md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dhyperball_polyvecl
signing_bounded_polyvecl_lossless
```

**Theorem 11.76** (EasyCrypt line 969). *EasyCrypt name.*

```
hyperball_polyveck_lossless
```

**Checked EasyCrypt statement.**

```
lemma hyperball_polyveck_lossless md :
  is_lossless (dhyperball_polyveck md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dhyperball_polyveck
signing_bounded_polyveck_lossless
```

**Theorem 11.77** (EasyCrypt line 975). *EasyCrypt name.*

```
hyperball_polyvecl_support
```

**Checked EasyCrypt statement.**

```
lemma hyperball_polyvecl_support md y1 :
  y1 \in dhyperball_polyvecl md =>
  hyperball_polyvecl_ok md y1.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dhypersball_polyvecl
hypersball_polyvecl_ok
signing_bounded_polyvecl_support
```

**Theorem 11.78** (EasyCrypt line 983). *EasyCrypt name.*

```
hypersball_polyvecl_supportP
```

**Checked EasyCrypt statement.**

```
lemma hypersball_polyvecl_supportP md y1 :
  y1 \in dhypersball_polyvecl md <=> hypersball_polyvecl_ok md y1.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
dhypersball_polyvecl
hypersball_polyvecl_ok
signing_bounded_polyvecl_supportP
```

**Theorem 11.79** (EasyCrypt line 990). *EasyCrypt name.*

```
hypersball_polyveck_support
```

**Checked EasyCrypt statement.**

```
lemma hypersball_polyveck_support md y2 :
  y2 \in dhypersball_polyveck md =>
  hypersball_polyveck_ok md y2.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dhypersball_polyveck
hypersball_polyveck_ok
signing_bounded_polyveck_support
```

**Theorem 11.80** (EasyCrypt line 998). *EasyCrypt name.*

```
hyperball_polyveck_supportP
```

**Checked EasyCrypt statement.**

```
lemma hyperball_polyveck_supportP md y2 :
  y2 \in dhyperball_polyveck md <=> hyperball_polyveck_ok md y2.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
dhyperball_polyveck
hyperball_polyveck_ok
signing_bounded_polyveck_supportP
```

**Theorem 11.81** (EasyCrypt line 1005). *EasyCrypt name.*

```
hyperball_polyvecl_point_boundE
```

**Checked EasyCrypt statement.**

```
lemma hyperball_polyvecl_point_boundE md y1 :
  mu (dhyperball_polyvecl md) (pred1 y1) <=
  hyperball_polyvecl_point_bound md.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
dhyperball_polyvecl
dlist_point_bound
dsigning_bounded_polyvecl
hyperball_poly_point_boundE
hyperball_poly_point_bound_nonnegative
hyperball_polyvecl_point_bound
mode_l_gt0
```

**Theorem 11.82** (EasyCrypt line 1017). *EasyCrypt name.*

```
hyperball_polyveck_point_boundE
```

**Checked EasyCrypt statement.**

```
lemma hyperball_polyveck_point_boundE md y2 :
  mu (dhyperball_polyveck md) (pred1 y2) <=
  hyperball_polyveck_point_bound md.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
dhypersball_polyveck
dlist_point_bound
dsigning_bounded_polyveck
hypersball_poly_point_boundE
hypersball_poly_point_bound_nonnegative
hypersball_polyveck_point_bound
mode_k_gt0
```

**Theorem 11.83** (EasyCrypt line 1029). *EasyCrypt name*.

```
bounded_unit_signing_sample_pair_lossless
```

**Checked EasyCrypt statement.**

```
lemma bounded_unit_signing_sample_pair_lossless md :
  is_lossless (dbounded_unit_signing_sample_pair md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dbounded_unit_signing_sample_pair
dlet_ll
dmap_ll
signing_unit_polyveck_lossless
signing_unit_polyveck_lossless
y1
```

**Theorem 11.84** (EasyCrypt line 1039). *EasyCrypt name*.

```
bounded_unit_signing_sample_pair_support
```

**Checked EasyCrypt statement.**

```
lemma bounded_unit_signing_sample_pair_support md s :
  s \in dbounded_unit_signing_sample_pair md =>
  signing_sample_pair_wf md s /\ signing_sample_pair_unit md s.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*, *apply*, *split*. Named identifiers syntactically cited in the proof block:

```
dbounded_unit_signing_sample_pair
md
polyveck_unit
polyvecl_unit
signing_sample_pair_unit
signing_sample_pair_wf
signing_sample_pair_y1
signing_sample_pair_y2
signing_unit_polyveck_support
signing_unit_polyvecl_support
supp_dlet
supp_dmap
y1
y1_supp
y1_unit
y2
y2_supp
y2_unit
```

**Theorem 11.85** (EasyCrypt line 1060). *EasyCrypt name.*

```
bounded_unit_signing_sample_pair_sample_ok
```

**Checked EasyCrypt statement.**

```
lemma bounded_unit_signing_sample_pair_sample_ok md s :
  s \in dbounded_unit_signing_sample_pair md =>
    signing_sample_pair_sample_ok md s.
```

**Formal mathematical reading.** *Mathematical theorem.* This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
bounded_unit_signing_sample_pair_support
hs
md
s_unit
s_wf
signing_sample_pair_sample_ok
signing_sample_pair_unit_bounded
```

**Theorem 11.86** (EasyCrypt line 1072). *EasyCrypt name.*

```
bounded_unit_signing_sample_pair_distribution_ok
```

**Checked EasyCrypt statement.**

```
lemma bounded_unit_signing_sample_pair_distribution_ok md :
  signing_sample_pair_distribution_ok md
    (dbounded_unit_signing_sample_pair md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
bounded_unit_signing_sample_pair_lossless
bounded_unit_signing_sample_pair_sample_ok
signing_sample_pair_distribution_ok
```

**Theorem 11.87** (*EasyCrypt* line 1082). *EasyCrypt* name.

```
checked_hyperball_signing_sample_pair_lossless
```

**Checked *EasyCrypt* statement.**

```
lemma checked_hyperball_signing_sample_pair_lossless md :
  is_lossless (dchecked_hyperball_signing_sample_pair md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dchecked_hyperball_signing_sample_pair
dlet_ll
dmap_ll
signing_bounded_polyveck_lossless
signing_bounded_polyvecl_lossless
y1
```

**Theorem 11.88** (*EasyCrypt* line 1092). *EasyCrypt* name.

```
checked_hyperball_signing_sample_pair_support
```

**Checked *EasyCrypt* statement.**

```
lemma checked_hyperball_signing_sample_pair_support md s :
  s \in dchecked_hyperball_signing_sample_pair md =>
  signing_sample_pair_sample_ok md s.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*, *apply*, *split*. Named identifiers syntactically cited in the proof block:



```

dchecked_hyperball_signing_sample_pair
md
polyveck_bounded_by
polyvecl_bounded_by
signing_bounded_polyveck_support
signing_bounded_polyvecl_support
signing_sample_bound
signing_sample_pair_bounded
signing_sample_pair_sample_ok
signing_sample_pair_wf
signing_sample_pair_y1
signing_sample_pair_y2
supp_dlet
supp_dmap
y1
y1_bounded
y1_supp
y2
y2_bounded
y2_supp

```

**Theorem 11.89** (EasyCrypt line 1114). *EasyCrypt name.*

```
checked_hyperball_signing_sample_pair_distribution_ok
```

**Checked EasyCrypt statement.**

```

lemma checked_hyperball_signing_sample_pair_distribution_ok md :
  signing_sample_pair_distribution_ok md
    (dchecked_hyperball_signing_sample_pair md).

```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```

checked_hyperball_signing_sample_pair_lossless
checked_hyperball_signing_sample_pair_support
signing_sample_pair_distribution_ok

```

**Theorem 11.90** (EasyCrypt line 1124). *EasyCrypt name.*

```
hyperball_sample_ok_sample_ok
```

**Checked EasyCrypt statement.**

```

lemma hyperball_sample_ok_sample_ok md s :
  hyperball_sample_ok md s =>
    signing_sample_pair_sample_ok md s.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*. Named identifiers syntactically cited in the proof block:

```
hyperball_polyveck_ok
hyperball_polyvecl_ok
hyperball_sample_ok
polyveck_bounded_by
polyvecl_bounded_by
signing_sample_pair_bounded
signing_sample_pair_sample_ok
signing_sample_pair_wf
y1_ok
y2_ok
```

**Theorem 11.91** (EasyCrypt line 1139). *EasyCrypt name*.

```
exact_hyperball_signing_sample_pair_lossless
```

**Checked EasyCrypt statement.**

```
lemma exact_hyperball_signing_sample_pair_lossless md :
  is_lossless (dexact_hyperball_signing_sample_pair md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
dexact_hyperball_signing_sample_pair
dprod_ll
hyperball_polyveck_lossless
hyperball_polyvecl_lossless
```

**Theorem 11.92** (EasyCrypt line 1149). *EasyCrypt name*.

```
exact_hyperball_signing_sample_pair_support
```

**Checked EasyCrypt statement.**

```
lemma exact_hyperball_signing_sample_pair_support md s :
  s \in dexact_hyperball_signing_sample_pair md =>
  hyperball_sample_ok md s.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *case*, *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```

dexact_hyperball_signing_sample_pair
hyperball_polyveck_support
hyperball_polyvecl_support
hyperball_sample_ok
signing_sample_pair_y1
signing_sample_pair_y2
supp_dprod
y1
y1_supp
y2
y2_supp

```

**Theorem 11.93** (EasyCrypt line 1163). *EasyCrypt name.*

```

exact_hyperball_signing_sample_pair_supportP

```

**Checked EasyCrypt statement.**

```

lemma exact_hyperball_signing_sample_pair_supportP md s :
  s \in dexact_hyperball_signing_sample_pair md <=>
    hyperball_sample_ok md s.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *case*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

dexact_hyperball_signing_sample_pair
hyperball_polyveck_supportP
hyperball_polyvecl_supportP
hyperball_sample_ok
signing_sample_pair_y1
signing_sample_pair_y2
supp_dprod
y1
y2

```

**Theorem 11.94** (EasyCrypt line 1173). *EasyCrypt name.*

```

exact_hyperball_signing_sample_pair_sample_ok

```

**Checked EasyCrypt statement.**

```

lemma exact_hyperball_signing_sample_pair_sample_ok md s :
  s \in dexact_hyperball_signing_sample_pair md =>
    signing_sample_pair_sample_ok md s.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
exact_hyperball_signing_sample_pair_support
hs
hyperball_sample_ok_sample_ok
```

**Theorem 11.95** (EasyCrypt line 1182). *EasyCrypt name.*

```
exact_hyperball_signing_sample_pair_distribution_ok
```

*Checked EasyCrypt statement.*

```
lemma exact_hyperball_signing_sample_pair_distribution_ok md :
  signing_sample_pair_distribution_ok md
  (dexact_hyperball_signing_sample_pair md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
exact_hyperball_signing_sample_pair_lossless
exact_hyperball_signing_sample_pair_sample_ok
signing_sample_pair_distribution_ok
```

**Theorem 11.96** (EasyCrypt line 1192). *EasyCrypt name.*

```
exact_hyperball_signing_sample_pair_point_bound
```

*Checked EasyCrypt statement.*

```
lemma exact_hyperball_signing_sample_pair_point_bound md s :
  mu (dexact_hyperball_signing_sample_pair md) (pred1 s) <=
  hyperball_sample_pair_point_bound md.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *case*, *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dexact_hyperball_signing_sample_pair
dprod1E
ge0_mu
hyperball_polyveck_point_boundE
hyperball_polyvecl_point_boundE
hyperball_sample_pair_point_bound
ler_pmul
y1
y2
```

**Theorem 11.97** (EasyCrypt line 1206). *EasyCrypt* name.

```
exact_hyperball_signing_sample_pair_checkedE
```

**Checked EasyCrypt statement.**

```
lemma exact_hyperball_signing_sample_pair_checkedE md :
  dexact_hyperball_signing_sample_pair md =
  dchecked_hyperball_signing_sample_pair md.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dchecked_hyperball_signing_sample_pair
dexact_hyperball_signing_sample_pair
dhyperball_polyveck
dhyperball_polyvecl
dmap
dprod_dlet
eq_dlet
```

*End of generated formal inventory for HAETAE\_Distributions.ec.*

# Chapter 12

## HAETAE\_Rejection.ec

**Mathematical role.** This file formalizes rejection sampling at the level of a signing attempt. It defines the state of a signing attempt, the acceptance predicates, and the distributions over attempts used in the sampling hops.

**Signing attempt state.** A signing attempt records the sampled vectors, commitment pieces, challenge, response, auxiliary response, hint, signature, and rejection-branch data. This turns the informal signing procedure into a single mathematical object  $st$  with predicates such as:

`fields_match(st), challenge_consistent(st), public_equation_consistent(st), accepts(st).`

**Acceptance and aborts.** The file separates rejection causes: response norm failure, hint norm failure, packing failure, reference rejection failure, and bimodal reject-2 branch failure. This separation lets the proof use union bounds rather than hiding all rejection events under one opaque predicate.

**Loss obligations.** The named obligations `structural_to_exact_hyperball_sample_pair_loss_obligation`, `structural_to_exact_hyperball_attempt_loss_obligation`, and `rejection_sampling_bound_obligation` are the bridge from concrete rejection predicates to the abstract loss term used in the theorem. Many lemmas prove zero-probability aborts for the current model; others show that a remaining abort probability is bounded by the named rejection-sampling term.

### Textbook Formal Inventory

This subsection records the exact formal material in `HAETAE_Rejection.ec`. It is generated from the proof-surface EasyCrypt file, so the line numbers and statements are meant to be read next to the checked source.

**Chapter role.** rejection-sampling predicates, accepted-attempt structure, and loss obligations.

**Inventory size.** 12 context declarations, 101 definitions/game objects, 139 lemmas or relational theorems, and 0 explicit Hoare/Phoare judgments were found.

### Theory Context, Imports, and Quantified Modules

**Context 1 (line 1)** EasyCrypt name.

|                   |
|-------------------|
| imported theories |
|-------------------|

**Checked EasyCrypt statement.**

```
require import AllCore Distr IntDiv List Real StdOrder RealSeries.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 2 (line 2) EasyCrypt name.**

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import HAETAE_Params HAETAE_Algebra HAETAE_Distributions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 3 (line 3) EasyCrypt name.**

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import HAETAE_Events HAETAE_Reductions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 4 (line 4) EasyCrypt name.**

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import HAETAE_Transcript.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 5 (line 6) EasyCrypt name.**

```
HAETAE_Rejection
```

**Checked EasyCrypt statement.**

```
theory HAETAE_Rejection.
```

**Formal mathematical reading.** Mathematical context. This begins a namespace for the formal development in this file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 6 (line 8) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Params.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 7 (line 9) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Algebra.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 8 (line 10) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Distributions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 9 (line 11) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Events.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.



**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 10 (line 12)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Reductions.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 11 (line 13)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Transcript.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 12 (line 14)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import RealOrder.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

## Definitions, Carrier Sets, Operations, Games, and Procedures

**Definition 12.1** (EasyCrypt line 16). EasyCrypt name.

```
signing_transcript_record
```

**Checked EasyCrypt statement.**

```
type signing_transcript_record = message * context * signature * transcript.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 12.2** (EasyCrypt line 17). **EasyCrypt name.**

```
signing_attempt_sample
```

**Checked EasyCrypt statement.**

```
type signing_attempt_sample = polyvecl * polyveck * polyveck.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.3** (EasyCrypt line 18). **EasyCrypt name.**

```
signing_attempt_state
```

**Checked EasyCrypt statement.**

```
type signing_attempt_state =
  random_coins * signing_attempt_sample * polyveck * poly * challenge *
  polyvecl * polyveck * hint_t * signature.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.4** (EasyCrypt line 22). **EasyCrypt name.**

```
signing_attempt_state_coins
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_coins
  (st : signing_attempt_state) : random_coins =
  st.'1.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.5** (EasyCrypt line 25). **EasyCrypt name.**

```
signing_attempt_state_sample
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_sample
  (st : signing_attempt_state) : signing_attempt_sample =
  st.'2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.6** (EasyCrypt line 28). **EasyCrypt name.**

```
signing_attempt_sample_y1
```

**Checked EasyCrypt statement.**

```
op signing_attempt_sample_y1 (s : signing_attempt_sample) : polyvecl =
  s.'1.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.7** (EasyCrypt line 30). **EasyCrypt name.**

```
signing_attempt_sample_y2
```

**Checked EasyCrypt statement.**

```
op signing_attempt_sample_y2 (s : signing_attempt_sample) : polyveck =
  s.'2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.8** (EasyCrypt line 32). **EasyCrypt name.**

```
signing_attempt_sample_commitment_raw
```

**Checked EasyCrypt statement.**

```
op signing_attempt_sample_commitment_raw
  (s : signing_attempt_sample) : polyveck =
  s.'3.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.9** (EasyCrypt line 35). **EasyCrypt name.**

```
signing_attempt_state_sampled_y1
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_sampled_y1
  (st : signing_attempt_state) : polyvecl =
  signing_attempt_sample_y1 (signing_attempt_state_sample st).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.10** (EasyCrypt line 38). **EasyCrypt name.**

```
signing_attempt_state_sampled_y2
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_sampled_y2
  (st : signing_attempt_state) : polyveck =
  signing_attempt_sample_y2 (signing_attempt_state_sample st).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.11** (EasyCrypt line 41). **EasyCrypt name.**

```
signing_attempt_state_sampled_y
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_sampled_y
  (st : signing_attempt_state) : polyveck =
  signing_attempt_sample_commitment_raw (signing_attempt_state_sample st).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.12** (EasyCrypt line 44). **EasyCrypt name.**

```
signing_attempt_state_highbits
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_highbits
  (st : signing_attempt_state) : polyveck =
  st.'3.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.13** (EasyCrypt line 47). **EasyCrypt name.**

```
signing_attempt_state_lowbits
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_lowbits
  (st : signing_attempt_state) : poly =
  st.'4.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.14** (EasyCrypt line 50). **EasyCrypt name.**

```
signing_attempt_state_challenge
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_challenge
  (st : signing_attempt_state) : challenge =
  st.'5.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.15** (EasyCrypt line 53). **EasyCrypt name.**

```
signing_attempt_state_response
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_response
  (st : signing_attempt_state) : polyvec1 =
  st.'6.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.16** (EasyCrypt line 56). **EasyCrypt name.**

```
signing_attempt_state_response_aux
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_response_aux
  (st : signing_attempt_state) : polyveck =
  st.'7.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.17** (EasyCrypt line 59). **EasyCrypt name.**

```
signing_attempt_state_hint
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_hint
  (st : signing_attempt_state) : hint_t =
  st.'8.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.18** (EasyCrypt line 62). **EasyCrypt name.**

```
signing_attempt_state_signature
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_signature
  (st : signing_attempt_state) : signature =
  st.'9.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.19** (EasyCrypt line 65). **EasyCrypt name.**

```
signature_rejection_branch_byte
```

**Checked EasyCrypt statement.**

```
op signature_rejection_branch_byte (sig : signature) : int =
  sig.'6.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

**Definition 12.20** (EasyCrypt line 67). **EasyCrypt name.**

```
signing_attempt_state_reject2_branch_byte
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_reject2_branch_byte
  (st : signing_attempt_state) : int =
  signature_rejection_branch_byte (signing_attempt_state_signature st).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.21** (EasyCrypt line 70). **EasyCrypt name.**

```
signing_attempt_state_reject2_branch_bit
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_reject2_branch_bit
  (st : signing_attempt_state) : bool =
  (signing_attempt_state_reject2_branch_byte st %/ 2) %% 2 = 1.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.22** (EasyCrypt line 73). **EasyCrypt name.**

```
polyvecl_double
```

**Checked EasyCrypt statement.**

```
op polyvecl_double (md : mode) (xs : polyvecl) : polyvecl =
  mkseq
    (fun i => poly_double (nth poly_zero xs i))
    (mode_l md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.23** (EasyCrypt line 77). **EasyCrypt name.**

```
signing_attempt_state_reject2_sampled_y1
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_reject2_sampled_y1
  (st : signing_attempt_state) : polyvecl =
  signing_attempt_state_sampled_y1 st.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.24** (EasyCrypt line 80). **EasyCrypt name.**

```
signing_attempt_state_reject2_sampled_y2
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_reject2_sampled_y2
  (st : signing_attempt_state) : polyveck =
  signing_attempt_state_sampled_y2 st.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.25** (EasyCrypt line 83). **EasyCrypt name.**

```
signing_attempt_state_reject2_balance_response
```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_reject2_balance_response
  (md : mode) (st : signing_attempt_state) : polyvecl =
  polyvecl_sub md
    (polyvecl_double md (signing_attempt_state_response st))
    (signing_attempt_state_reject2_sampled_y1 st).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.26** (EasyCrypt line 88). **EasyCrypt name.**

```

signing_attempt_state_reject2_balance_aux

```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_reject2_balance_aux
  (md : mode) (st : signing_attempt_state) : polyveck =
  polyveck_sub md
    (polyveck_double md (signing_attempt_state_response_aux st))
    (signing_attempt_state_reject2_sampled_y2 st).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.27** (EasyCrypt line 93). **EasyCrypt name.**

```

signing_attempt_state_lowbits_carrier

```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_lowbits_carrier
  (st : signing_attempt_state) : poly =
  nth poly_zero (signing_attempt_state_sampled_y st) 0.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.28** (EasyCrypt line 96). **EasyCrypt name.**

```

signing_attempt_state_token

```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_token
  (st : signing_attempt_state) : sig_token =
  ( signing_attempt_state_response st,
    signing_attempt_state_response_aux st,
    signing_attempt_state_hint st).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.



**Definition 12.29** (EasyCrypt line 102). **EasyCrypt name.**

```
signing_attempt_state_programmed_lowbits
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_programmed_lowbits
  (st : signing_attempt_state) : poly =
  mkseq
    (fun i =>
      if i = 0 then signing_entropy_token_of_coins
        (signing_attempt_state_coins st)
      else poly_coeff (signing_attempt_state_lowbits_carrier st) i)
  n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.30** (EasyCrypt line 111). **EasyCrypt name.**

```
signing_attempt_state_lowbits_consistent
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_lowbits_consistent
  (st : signing_attempt_state) : bool =
  signing_attempt_state_lowbits st =
  signing_attempt_state_programmed_lowbits st.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.31** (EasyCrypt line 116). **EasyCrypt name.**

```
signing_attempt_state_programmed_challenge
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_programmed_challenge
  (md : mode) (pk : pkey) (m : message) (ctx : context)
  (st : signing_attempt_state) : challenge =
  challenge_hash md
    (signing_attempt_state_response_aux st)
    (signing_attempt_state_lowbits st)
    (message_hash pk ctx m).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.32** (EasyCrypt line 124). **EasyCrypt name.**

```
signing_attempt_state_challenge_consistent
```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_challenge_consistent
  (md : mode) (pk : pkey) (m : message) (ctx : context)
  (st : signing_attempt_state) : bool =
  signing_attempt_state_challenge st =
    signing_attempt_state_programmed_challenge md pk m ctx st.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.33** (EasyCrypt line 130). **EasyCrypt name.**

```

signing_attempt_state_reconstructed_highbits

```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_reconstructed_highbits
  (md : mode) (pk : pkey) (st : signing_attempt_state) : polyveck =
  reconstructed_highbits md
    (signing_attempt_state_response_aux st)
    (public_key_challenge_term md pk (signing_attempt_state_challenge st)).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.34** (EasyCrypt line 136). **EasyCrypt name.**

```

signing_attempt_state_public_equation_consistent

```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_public_equation_consistent
  (md : mode) (pk : pkey) (st : signing_attempt_state) : bool =
  signing_attempt_state_highbits st =
    signing_attempt_state_reconstructed_highbits md pk st.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.35** (EasyCrypt line 141). **EasyCrypt name.**

```

signing_attempt_state_signature_of_fields

```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_signature_of_fields
  (pk : pkey) (m : message) (ctx : context)
  (st : signing_attempt_state) : signature =
  ( signing_attempt_state_highbits st,
    signing_attempt_state_lowbits st,
    message_hash pk ctx m,
    signing_attempt_state_challenge st,
    signing_attempt_state_token st,

```

```
0,
0).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.36** (EasyCrypt line 152). **EasyCrypt name.**

```
signing_attempt_state_fields_match_signature
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_fields_match_signature
  (pk : pkey) (m : message) (ctx : context)
  (st : signing_attempt_state) : bool =
  signing_attempt_state_signature st =
    signing_attempt_state_signature_of_fields pk m ctx st.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.37** (EasyCrypt line 158). **EasyCrypt name.**

```
signing_attempt_state_field_accepts
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_field_accepts
  (md : mode) (pk : pkey) (m : message) (ctx : context)
  (st : signing_attempt_state) : bool =
  signing_attempt_state_lowbits_consistent st /\
  signing_attempt_state_challenge_consistent md pk m ctx st /\
  signing_attempt_state_public_equation_consistent md pk st /\
  signing_attempt_state_fields_match_signature pk m ctx st.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.38** (EasyCrypt line 166). **EasyCrypt name.**

```
signing_attempt_state_of_coins
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_of_coins
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (coins : random_coins) : signing_attempt_state =
  ( coins,
    ( signing_sample_y1 md sk m ctx coins,
      signing_sample_y2 md sk m ctx coins,
      commitment_raw md sk m ctx coins),
    commitment_highbits md sk m ctx coins,
```

```

commitment_lowbits md sk m ctx coins,
commitment_challenge md sk m ctx coins,
response_vector md sk m ctx coins,
response_aux_vector md sk m ctx coins,
hint_vector md sk m ctx coins,
sign_internal md sk m ctx coins).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.39** (EasyCrypt line 181). **EasyCrypt name.**

```
signing_attempt_sample_of_pair
```

**Checked EasyCrypt statement.**

```

op signing_attempt_sample_of_pair
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (coins : random_coins) (sample : signing_sample_pair) :
  signing_attempt_sample =
  (signing_sample_pair_y1 sample,
   signing_sample_pair_y2 sample,
   commitment_raw md sk m ctx coins).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.40** (EasyCrypt line 189). **EasyCrypt name.**

```
signing_attempt_state_of_sample_pair
```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_of_sample_pair
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (coins : random_coins) (sample : signing_sample_pair) :
  signing_attempt_state =
  ( coins,
    signing_attempt_sample_of_pair md sk m ctx coins sample,
    commitment_highbits md sk m ctx coins,
    commitment_lowbits md sk m ctx coins,
    commitment_challenge md sk m ctx coins,
    response_vector md sk m ctx coins,
    response_aux_vector md sk m ctx coins,
    hint_vector md sk m ctx coins,
    sign_internal md sk m ctx coins).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.41** (EasyCrypt line 203). **EasyCrypt name.**

```
signing_attempt_state_valid_signature_accepts
```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_valid_signature_accepts
  (md : mode) (st : signing_attempt_state) : bool =
  valid_signature md (signing_attempt_state_signature st).

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 12.42** (EasyCrypt line 207). **EasyCrypt name.**

```

signing_attempt_state_response_norm_accepts

```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_response_norm_accepts
  (md : mode) (st : signing_attempt_state) : bool =
  response_norm_ok md (signing_attempt_state_signature st).

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.43** (EasyCrypt line 211). **EasyCrypt name.**

```

signing_attempt_state_hint_norm_accepts

```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_hint_norm_accepts
  (md : mode) (st : signing_attempt_state) : bool =
  verify_norm_ok md (signing_attempt_state_signature st).

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.44** (EasyCrypt line 215). **EasyCrypt name.**

```

signing_attempt_state_accepts

```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_accepts
  (md : mode) (st : signing_attempt_state) : bool =
  signing_attempt_state_valid_signature_accepts md st /\
  signing_attempt_state_response_norm_accepts md st /\
  signing_attempt_state_hint_norm_accepts md st.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.45** (EasyCrypt line 221). **EasyCrypt name.**

```
signing_attempt_reject1_bound
```

**Checked EasyCrypt statement.**

```
op signing_attempt_reject1_bound (md : mode) : int =
  mode_b1sq md * mode_ln md * mode_ln md.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 12.46** (EasyCrypt line 224). **EasyCrypt name.**

```
signing_attempt_reject2_bound
```

**Checked EasyCrypt statement.**

```
op signing_attempt_reject2_bound (md : mode) : int =
  mode_b0sq md * mode_ln md * mode_ln md.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 12.47** (EasyCrypt line 227). **EasyCrypt name.**

```
signing_attempt_reject2_balance_norm_sq
```

**Checked EasyCrypt statement.**

```
op signing_attempt_reject2_balance_norm_sq
  (md : mode) (z : polyvec1) (zaux : polyveck)
  (y1 : polyvec1) (y2 : polyveck) : int =
  polyvec1_norm_sq md (polyvec1_sub md (polyvec1_double md z) y1) +
  polyveck_norm_sq md (polyveck_sub md (polyveck_double md zaux) y2).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.48** (EasyCrypt line 233). **EasyCrypt name.**

```
signing_attempt_reject2_concrete_aborts
```

**Checked EasyCrypt statement.**

```
op signing_attempt_reject2_concrete_aborts
  (md : mode) (branch : bool) (z : polyvec1) (zaux : polyveck)
  (y1 : polyvec1) (y2 : polyveck) : bool =
  branch /\
  signing_attempt_reject2_balance_norm_sq md z zaux y1 y2 <
  signing_attempt_reject2_bound md.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.49** (EasyCrypt line 240). **EasyCrypt name.**

```
signing_attempt_state_reject1_norm_sq
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_reject1_norm_sq
  (md : mode) (st : signing_attempt_state) : int =
  response_norm_sq md (signing_attempt_state_signature st).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.50** (EasyCrypt line 244). **EasyCrypt name.**

```
signing_attempt_state_reject1_accepts
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_reject1_accepts
  (md : mode) (st : signing_attempt_state) : bool =
  0 <= signing_attempt_state_reject1_norm_sq md st /\
  signing_attempt_state_reject1_norm_sq md st <=
  signing_attempt_reject1_bound md.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.51** (EasyCrypt line 250). **EasyCrypt name.**

```
signing_attempt_state_balance_norm_sq
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_balance_norm_sq
  (md : mode) (st : signing_attempt_state) : int =
  signing_attempt_reject2_balance_norm_sq md
  (signing_attempt_state_response st)
  (signing_attempt_state_response_aux st)
  (signing_attempt_state_reject2_sampled_y1 st)
  (signing_attempt_state_reject2_sampled_y2 st).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.52** (EasyCrypt line 258). **EasyCrypt name.**

```
signing_attempt_state_bimodal_reject2_branch
```

#### Checked EasyCrypt statement.

```
op signing_attempt_state_bimodal_reject2_branch
  (st : signing_attempt_state) : bool =
  signing_attempt_state_reject2_branch_bit st.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.53** (EasyCrypt line 262). **EasyCrypt name.**

```
signing_attempt_state_reject2_under_bound
```

#### Checked EasyCrypt statement.

```
op signing_attempt_state_reject2_under_bound
  (md : mode) (st : signing_attempt_state) : bool =
  signing_attempt_state_balance_norm_sq md st <
  signing_attempt_reject2_bound md.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 12.54** (EasyCrypt line 267). **EasyCrypt name.**

```
signing_attempt_state_reject2_aborts
```

#### Checked EasyCrypt statement.

```
op signing_attempt_state_reject2_aborts
  (md : mode) (st : signing_attempt_state) : bool =
  signing_attempt_reject2_concrete_aborts md
  (signing_attempt_state_bimodal_reject2_branch st)
  (signing_attempt_state_response st)
  (signing_attempt_state_response_aux st)
  (signing_attempt_state_reject2_sampled_y1 st)
  (signing_attempt_state_reject2_sampled_y2 st).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.55** (EasyCrypt line 276). **EasyCrypt name.**

```
signing_attempt_state_reject2_accepts
```

#### Checked EasyCrypt statement.

```
op signing_attempt_state_reject2_accepts
```



```
(md : mode) (st : signing_attempt_state) : bool =
! signing_attempt_state_reject2_aborts md st.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.56** (EasyCrypt line 280). **EasyCrypt name.**

```
signing_attempt_state_pack_accepts
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_pack_accepts
  (md : mode) (st : signing_attempt_state) : bool =
  signing_attempt_state_valid_signature_accepts md st /\
  0 < mode_crypto_bytes md.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.57** (EasyCrypt line 285). **EasyCrypt name.**

```
signing_attempt_state_reference_rejection_accepts
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_reference_rejection_accepts
  (md : mode) (st : signing_attempt_state) : bool =
  signing_attempt_state_reject1_accepts md st /\
  signing_attempt_state_reject2_accepts md st /\
  signing_attempt_state_pack_accepts md st /\
  signing_attempt_state_hint_norm_accepts md st.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

**Definition 12.58** (EasyCrypt line 292). **EasyCrypt name.**

```
signing_attempt_state_reference_rejection_aborts
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_reference_rejection_aborts
  (md : mode) (st : signing_attempt_state) : bool =
  ! signing_attempt_state_reference_rejection_accepts md st.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

**Definition 12.59** (EasyCrypt line 296). **EasyCrypt name.**

```
signing_attempt_state_reject1_aborts
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_reject1_aborts
  (md : mode) (st : signing_attempt_state) : bool =
  ! signing_attempt_state_reject1_accepts md st.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.60** (EasyCrypt line 300). **EasyCrypt name.**

```
signing_attempt_state_pack_aborts
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_pack_aborts
  (md : mode) (st : signing_attempt_state) : bool =
  ! signing_attempt_state_pack_accepts md st.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.61** (EasyCrypt line 304). **EasyCrypt name.**

```
signing_attempt_state_verification_accepts
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_verification_accepts
  (md : mode) (pk : pkey) (m : message) (ctx : context)
  (st : signing_attempt_state) : bool =
  signing_attempt_state_field_accepts md pk m ctx st /\
  signing_attempt_state_accepts md st.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.62** (EasyCrypt line 310). **EasyCrypt name.**

```
signing_attempt_state_rejection_accepts
```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_rejection_accepts
  (md : mode) (pk : pkey) (m : message) (ctx : context)
  (st : signing_attempt_state) : bool =
  signing_attempt_state_verification_accepts md pk m ctx st /\
  signing_attempt_state_reference_rejection_accepts md st.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

**Definition 12.63** (EasyCrypt line 316). **EasyCrypt name.**

```

signing_attempt_state_rejection_aborts

```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_rejection_aborts
  (md : mode) (pk : pkey) (m : message) (ctx : context)
  (st : signing_attempt_state) : bool =
  ! signing_attempt_state_rejection_accepts md pk m ctx st.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

**Definition 12.64** (EasyCrypt line 321). **EasyCrypt name.**

```

signing_attempt_state_response_norm_aborts

```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_response_norm_aborts
  (md : mode) (st : signing_attempt_state) : bool =
  ! signing_attempt_state_response_norm_accepts md st.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.65** (EasyCrypt line 325). **EasyCrypt name.**

```

signing_attempt_state_hint_norm_aborts

```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_hint_norm_aborts
  (md : mode) (st : signing_attempt_state) : bool =
  ! signing_attempt_state_hint_norm_accepts md st.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.66** (EasyCrypt line 329). **EasyCrypt name.**

```
signing_attempt_state_rejection_sampling_aborts
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_rejection_sampling_aborts
  (md : mode) (st : signing_attempt_state) : bool =
  ! signing_attempt_state_valid_signature_accepts md st.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a bad event or rejection condition whose probability must be zero or explicitly bounded.

**Definition 12.67** (EasyCrypt line 333). **EasyCrypt name.**

```
signing_attempt_state_aborts
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_aborts
  (md : mode) (st : signing_attempt_state) : bool =
  signing_attempt_state_rejection_sampling_aborts md st \/  
  signing_attempt_state_response_norm_aborts md st \/  
  signing_attempt_state_hint_norm_aborts md st.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.68** (EasyCrypt line 339). **EasyCrypt name.**

```
signing_accepts
```

**Checked EasyCrypt statement.**

```
op signing_accepts
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (coins : random_coins) : bool =
  signing_attempt_state_accepts md
  (signing_attempt_state_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.69** (EasyCrypt line 345). **EasyCrypt name.**

```
dsigning_attempt_state
```

### Checked EasyCrypt statement.

```
op dsigning_attempt_state
  (md : mode) (sk : skey) (m : message) (ctx : context) :
  signing_attempt_state distr =
  dmap drandom_coins (signing_attempt_state_of_coins md sk m ctx).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

### Definition 12.70 (EasyCrypt line 350). EasyCrypt name.

```
signing_sample_pair_sampler_ok
```

### Checked EasyCrypt statement.

```
op signing_sample_pair_sampler_ok
  (md : mode) (dsample : random_coins -> signing_sample_pair distr) :
  bool =
  forall coins,
    signing_randomness_domain coins =>
    signing_sample_pair_distribution_ok md (dsample coins).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

### Definition 12.71 (EasyCrypt line 357). EasyCrypt name.

```
dsigning_attempt_state_from_sample_pair_sampler
```

### Checked EasyCrypt statement.

```
op dsigning_attempt_state_from_sample_pair_sampler
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (dsample : random_coins -> signing_sample_pair distr) :
  signing_attempt_state distr =
  dlet drandom_coins
    (fun coins =>
      dmap (dsample coins)
        (signing_attempt_state_of_sample_pair md sk m ctx coins)).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

### Definition 12.72 (EasyCrypt line 366). EasyCrypt name.

```
signing_attempt_coin_sample_pair
```

### Checked EasyCrypt statement.

```
type signing_attempt_coin_sample_pair = random_coins * signing_sample_pair.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.73** (EasyCrypt line 368). **EasyCrypt name.**

```
signing_attempt_state_of_coin_sample_pair
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_of_coin_sample_pair
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (cs : signing_attempt_coin_sample_pair) : signing_attempt_state =
  signing_attempt_state_of_sample_pair md sk m ctx cs.'1 cs.'2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.74** (EasyCrypt line 373). **EasyCrypt name.**

```
dstructural_signing_attempt_coin_sample_pair
```

**Checked EasyCrypt statement.**

```
op dstructural_signing_attempt_coin_sample_pair
  (md : mode) (sk : skey) (m : message) (ctx : context) :
  signing_attempt_coin_sample_pair distr =
  dlet drandom_coins
  (fun coins =>
    dmap (dstructural_signing_sample_pair md sk m ctx coins)
    (fun sample => (coins, sample))).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.75** (EasyCrypt line 381). **EasyCrypt name.**

```
dexact_hyperball_signing_attempt_coin_sample_pair
```

**Checked EasyCrypt statement.**

```
op dexact_hyperball_signing_attempt_coin_sample_pair
  (md : mode) (sk : skey) (m : message) (ctx : context) :
  signing_attempt_coin_sample_pair distr =
  dlet drandom_coins
  (fun coins =>
    dmap (dexact_hyperball_signing_sample_pair md)
    (fun sample => (coins, sample))).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.76** (EasyCrypt line 389). **EasyCrypt name.**

```
dstructural_signing_attempt_state
```

**Checked EasyCrypt statement.**

```
op dstructural_signing_attempt_state
  (md : mode) (sk : skey) (m : message) (ctx : context) :
  signing_attempt_state distr =
  dmap drandom_coins
    (fun coins =>
      signing_attempt_state_of_sample_pair md sk m ctx coins
        (signing_sample_pair_of_coins md sk m ctx coins)).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.77** (EasyCrypt line 397). **EasyCrypt name.**

```
dstructural_signing_attempt_state_from_sampler
```

**Checked EasyCrypt statement.**

```
op dstructural_signing_attempt_state_from_sampler
  (md : mode) (sk : skey) (m : message) (ctx : context) :
  signing_attempt_state distr =
  dsigning_attempt_state_from_sample_pair_sampler md sk m ctx
    (dstructural_signing_sample_pair md sk m ctx).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.78** (EasyCrypt line 403). **EasyCrypt name.**

```
dbounded_unit_signing_attempt_state
```

**Checked EasyCrypt statement.**

```
op dbounded_unit_signing_attempt_state
  (md : mode) (sk : skey) (m : message) (ctx : context) :
  signing_attempt_state distr =
  dsigning_attempt_state_from_sample_pair_sampler md sk m ctx
    (fun _ => dbounded_unit_signing_sample_pair md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 12.79** (EasyCrypt line 409). **EasyCrypt name.**

```
dchecked_hyperball_signing_attempt_state
```

**Checked EasyCrypt statement.**

```
op dchecked_hyperball_signing_attempt_state
  (md : mode) (sk : skey) (m : message) (ctx : context) :
```

```

signing_attempt_state distr =
dsigning_attempt_state_from_sample_pair_sampler md sk m ctx
(fun _ => dchecked_hyperball_signing_sample_pair md).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.80** (EasyCrypt line 415). **EasyCrypt name.**

```

dexact_hyperball_signing_attempt_state

```

**Checked EasyCrypt statement.**

```

op dexact_hyperball_signing_attempt_state
  (md : mode) (sk : skey) (m : message) (ctx : context) :
  signing_attempt_state distr =
  dsigning_attempt_state_from_sample_pair_sampler md sk m ctx
  (fun _ => dexact_hyperball_signing_sample_pair md).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 12.81** (EasyCrypt line 421). **EasyCrypt name.**

```

signing_attempt_state_sample_pair_wf

```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_sample_pair_wf
  (md : mode) (st : signing_attempt_state) : bool =
  polyvecl_wf md (signing_attempt_state_sampled_y1 st) /\
  polyveck_wf md (signing_attempt_state_sampled_y2 st).

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 12.82** (EasyCrypt line 426). **EasyCrypt name.**

```

signing_attempt_state_sample_pair_unit

```

**Checked EasyCrypt statement.**

```

op signing_attempt_state_sample_pair_unit
  (md : mode) (st : signing_attempt_state) : bool =
  polyvecl_unit md (signing_attempt_state_sampled_y1 st) /\
  polyveck_unit md (signing_attempt_state_sampled_y2 st).

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.



**Definition 12.83** (EasyCrypt line 431). **EasyCrypt name.**

```
signing_attempt_state_sample_pair_bounded
```

**Checked EasyCrypt statement.**

```
op signing_attempt_state_sample_pair_bounded
  (md : mode) (st : signing_attempt_state) : bool =
  polyvecl_bounded_by md (signing_sample_bound md)
    (signing_attempt_state_sampled_y1 st) /\
  polyveck_bounded_by md (signing_sample_bound md)
    (signing_attempt_state_sampled_y2 st).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 12.84** (EasyCrypt line 438). **EasyCrypt name.**

```
signing_attempt_relation
```

**Checked EasyCrypt statement.**

```
op signing_attempt_relation
  (md : mode) (pk : pkey) (sk : skey) (m : message)
  (ctx : context) (coins : random_coins) (sig : signature) : bool =
  valid_keypair md pk sk /\
  signing_accepts md sk m ctx coins /\
  sig = sign_internal md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.85** (EasyCrypt line 445). **EasyCrypt name.**

```
signing_simulation_relation
```

**Checked EasyCrypt statement.**

```
op signing_simulation_relation
  (md : mode) (pk : pkey) (m : message) (ctx : context)
  (sig : signature) : bool =
  verify_internal md pk m ctx sig /\
  exists (sk : skey) (coins : random_coins),
    signing_attempt_relation md pk sk m ctx coins sig.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.86** (EasyCrypt line 452). **EasyCrypt name.**

```
signing_transcript_relation
```

### Checked EasyCrypt statement.

```

op signing_transcript_relation
  (md : mode) (pk : pkey) (m : message) (ctx : context)
  (sig : signature) (tr : transcript) : bool =
  signing_simulation_relation md pk m ctx sig /\
  tr = transcript_of_signature md pk m ctx sig /\
  transcript_valid md tr.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

### Definition 12.87 (EasyCrypt line 459). EasyCrypt name.

```

signing_record_message

```

### Checked EasyCrypt statement.

```

op signing_record_message (r : signing_transcript_record) : message = r.'1.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 12.88 (EasyCrypt line 460). EasyCrypt name.

```

signing_record_context

```

### Checked EasyCrypt statement.

```

op signing_record_context (r : signing_transcript_record) : context = r.'2.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 12.89 (EasyCrypt line 461). EasyCrypt name.

```

signing_record_signature

```

### Checked EasyCrypt statement.

```

op signing_record_signature (r : signing_transcript_record) : signature = r.'3.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 12.90 (EasyCrypt line 462). EasyCrypt name.

```

signing_record_transcript

```

### Checked EasyCrypt statement.

```
op signing_record_transcript (r : signing_transcript_record) : transcript =
  r.'4.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

**Definition 12.91** (EasyCrypt line 465). **EasyCrypt name.**

```
signing_record_query
```

**Checked EasyCrypt statement.**

```
op signing_record_query (r : signing_transcript_record) : message * context =
  (signing_record_message r, signing_record_context r).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.92** (EasyCrypt line 468). **EasyCrypt name.**

```
signing_record_relation
```

**Checked EasyCrypt statement.**

```
op signing_record_relation
  (md : mode) (pk : pkey) (r : signing_transcript_record) : bool =
  signing_transcript_relation md pk
    (signing_record_message r)
    (signing_record_context r)
    (signing_record_signature r)
    (signing_record_transcript r).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.93** (EasyCrypt line 476). **EasyCrypt name.**

```
signing_record_queries
```

**Checked EasyCrypt statement.**

```
op signing_record_queries (rs : signing_transcript_record list) :
  (message * context) list =
  map signing_record_query rs.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 12.94** (EasyCrypt line 480). **EasyCrypt name.**

```
signing_record_transcripts
```

### Checked EasyCrypt statement.

```
op signing_record_transcripts (rs : signing_transcript_record list) :  
  transcript list =  
  map signing_record_transcript rs.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It defines transcript/extraction data for the Fiat-Shamir forking and special-soundness argument.

### Definition 12.95 (EasyCrypt line 484). EasyCrypt name.

```
signing_record_log_sound
```

### Checked EasyCrypt statement.

```
op signing_record_log_sound  
  (md : mode) (pk : pkey) (rs : signing_transcript_record list) : bool =  
  all (signing_record_relation md pk) rs.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 12.96 (EasyCrypt line 488). EasyCrypt name.

```
transcript_log_valid
```

### Checked EasyCrypt statement.

```
op transcript_log_valid (md : mode) (trs : transcript list) : bool =  
  all (transcript_valid md) trs.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

### Definition 12.97 (EasyCrypt line 491). EasyCrypt name.

```
structural_to_exact_hyperball_sample_pair_point_loss_obligation
```

### Checked EasyCrypt statement.

```
op structural_to_exact_hyperball_sample_pair_point_loss_obligation : bool =  
  forall (md : mode) (sk : skey) (m : message) (ctx : context)  
    (coins : random_coins),  
  1%r <=  
  mu (dexact_hyperball_signing_sample_pair md)  
    (pred1 (signing_sample_pair_of_coins md sk m ctx coins)) +  
  rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 12.98** (EasyCrypt line 499). **EasyCrypt name.**

```
structural_to_exact_hyperball_sample_pair_loss_obligation
```

**Checked EasyCrypt statement.**

```
op structural_to_exact_hyperball_sample_pair_loss_obligation : bool =
  forall (md : mode) (sk : skey) (m : message) (ctx : context)
    (coins : random_coins) (p : signing_sample_pair -> bool),
  mu (dstructural_signing_sample_pair md sk m ctx coins) p <=
  mu (dexact_hyperball_signing_sample_pair md) p +
  rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 12.99** (EasyCrypt line 506). **EasyCrypt name.**

```
structural_to_exact_hyperball_coin_sample_pair_loss_obligation
```

**Checked EasyCrypt statement.**

```
op structural_to_exact_hyperball_coin_sample_pair_loss_obligation : bool =
  forall (md : mode) (sk : skey) (m : message) (ctx : context)
    (p : signing_attempt_coin_sample_pair -> bool),
  mu (dstructural_signing_attempt_coin_sample_pair md sk m ctx) p <=
  mu (dexact_hyperball_signing_attempt_coin_sample_pair md sk m ctx) p +
  rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 12.100** (EasyCrypt line 513). **EasyCrypt name.**

```
structural_to_exact_hyperball_attempt_loss_obligation
```

**Checked EasyCrypt statement.**

```
op structural_to_exact_hyperball_attempt_loss_obligation : bool =
  forall (md : mode) (sk : skey) (m : message) (ctx : context)
    (p : signing_attempt_state -> bool),
  mu (dsigning_attempt_state md sk m ctx) p <=
  mu (dexact_hyperball_signing_attempt_state md sk m ctx) p +
  rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 12.101** (EasyCrypt line 520). *EasyCrypt name.*

```
rejection_sampling_bound_obligation
```

**Checked EasyCrypt statement.**

```
op rejection_sampling_bound_obligation : bool =
  0%r <= rejection_sampling_loss_term /\
  0%r <= fs_with_aborts_reprogramming_term /\
  0%r <= fs_with_aborts_min_entropy_term.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

## Lemmas, Theorems, Relational Equivalences, and Their Proof Dependencies

**Theorem 12.1** (EasyCrypt line 525). *EasyCrypt name.*

```
signing_attempt_verifies
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_verifies md pk sk m ctx coins sig :
  signing_attempt_relation md pk sk m ctx coins sig =>
  verify_internal md pk m ctx sig.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
pkE
signing_attempt_relation
valid_keypair
verify_internal_sign_internal
```

**Theorem 12.2** (EasyCrypt line 535). *EasyCrypt name.*

```
signing_simulation_verifies
```

**Checked EasyCrypt statement.**

```
lemma signing_simulation_verifies md pk m ctx sig :
  signing_simulation_relation md pk m ctx sig =>
  verify_internal md pk m ctx sig.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_simulation_relation
```

**Theorem 12.3** (EasyCrypt line 542). *EasyCrypt name*.

```
signing_attempt_simulates
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_simulates md pk sk m ctx coins sig :
  signing_attempt_relation md pk sk m ctx coins sig =>
  signing_simulation_relation md pk m ctx sig.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
coins
ctx
md
pk
sig
signing_attempt_verifies
signing_simulation_relation
sk
```

**Theorem 12.4** (EasyCrypt line 554). *EasyCrypt name*.

```
sign_internal_attempt_relation
```

**Checked EasyCrypt statement.**

```
lemma sign_internal_attempt_relation md sk m ctx coins :
  signing_attempt_relation md
    (public_key_of_secret md sk) sk m ctx coins
    (sign_internal md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *smt*. Named identifiers syntactically cited in the proof block:

```

response_norm_ok_current
signing_accepts
signing_attempt_relation
signing_attempt_state_accepts
signing_attempt_state_hint_norm_accepts
signing_attempt_state_response_norm_accepts
signing_attempt_state_valid_signature_accepts
trivial
valid_keypair
valid_signature_sign_internal
verify_norm_ok_current

```

**Theorem 12.5** (EasyCrypt line 571). *EasyCrypt name.*

```
signing_attempt_sample_of_pair_currentE
```

*Checked EasyCrypt statement.*

```

lemma signing_attempt_sample_of_pair_currentE md sk m ctx coins :
  signing_attempt_sample_of_pair md sk m ctx coins
    (signing_sample_pair_of_coins md sk m ctx coins) =
  ( signing_sample_y1 md sk m ctx coins,
    signing_sample_y2 md sk m ctx coins,
    commitment_raw md sk m ctx coins ).

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

signing_attempt_sample_of_pair
signing_sample_pair_of_coins
signing_sample_pair_y1
signing_sample_pair_y2

```

**Theorem 12.6** (EasyCrypt line 583). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_currentE
```

*Checked EasyCrypt statement.*

```

lemma signing_attempt_state_of_sample_pair_currentE md sk m ctx coins :
  signing_attempt_state_of_sample_pair md sk m ctx coins
    (signing_sample_pair_of_coins md sk m ctx coins) =
  signing_attempt_state_of_coins md sk m ctx coins.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:



```
signing_attempt_sample_of_pair_currentE
signing_attempt_state_of_coins
signing_attempt_state_of_sample_pair
```

**Theorem 12.7** (EasyCrypt line 593). *EasyCrypt name.*

```
structural_signing_sample_pair_sampler_ok
```

**Checked EasyCrypt statement.**

```
lemma structural_signing_sample_pair_sampler_ok md sk m ctx :
  signing_sample_pair_sampler_ok md
  (dstructural_signing_sample_pair md sk m ctx).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
coins
signing_sample_pair_sampler_ok
structural_signing_sample_pair_distribution_ok
```

**Theorem 12.8** (EasyCrypt line 601). *EasyCrypt name.*

```
bounded_unit_signing_sample_pair_sampler_ok
```

**Checked EasyCrypt statement.**

```
lemma bounded_unit_signing_sample_pair_sampler_ok md :
  signing_sample_pair_sampler_ok md
  (fun _ => dbounded_unit_signing_sample_pair md).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
bounded_unit_signing_sample_pair_distribution_ok
coins
signing_sample_pair_sampler_ok
```

**Theorem 12.9** (EasyCrypt line 609). *EasyCrypt name.*

```
checked_hyperball_signing_sample_pair_sampler_ok
```

**Checked EasyCrypt statement.**

```
lemma checked_hyperball_signing_sample_pair_sampler_ok md :
  signing_sample_pair_sampler_ok md
  (fun _ => dchecked_hyperball_signing_sample_pair md).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
checked_hyperball_signing_sample_pair_distribution_ok
coins
signing_sample_pair_sampler_ok
```

**Theorem 12.10** (EasyCrypt line 617). *EasyCrypt name.*

```
exact_hyperball_signing_sample_pair_sampler_ok
```

**Checked EasyCrypt statement.**

```
lemma exact_hyperball_signing_sample_pair_sampler_ok md :
  signing_sample_pair_sampler_ok md
  (fun _ => dexact_hyperball_signing_sample_pair md).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
coins
exact_hyperball_signing_sample_pair_distribution_ok
signing_sample_pair_sampler_ok
```

**Theorem 12.11** (EasyCrypt line 625). *EasyCrypt name.*

```
dsigning_attempt_state_from_sample_pair_sampler_lossless
```

**Checked EasyCrypt statement.**

```
lemma dsigning_attempt_state_from_sample_pair_sampler_lossless
  md sk m ctx dsample :
  signing_sample_pair_sampler_ok md dsample =>
  is_lossless
  (dsigning_attempt_state_from_sample_pair_sampler
   md sk m ctx dsample).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
coins
coins_supp
dlet_ll
dmap_ll
dsigning_attempt_state_from_sample_pair_sampler
hll
hsample
signing_coin_distribution_domain
signing_coin_distribution_lossless
signing_sample_pair_distribution_ok
```

**Theorem 12.12** (EasyCrypt line 643). *EasyCrypt name.*

```
dsigning_attempt_state_from_sample_pair_sampler_sample_ok
```

**Checked EasyCrypt statement.**

```
lemma dsigning_attempt_state_from_sample_pair_sampler_sample_ok
  md sk m ctæ dsample st :
  signing_sample_pair_sampler_ok md dsample =>
  st \in
    dsigning_attempt_state_from_sample_pair_sampler
    md sk m ctæ dsample =>
  signing_attempt_state_sample_pair_wf md st /\
  signing_attempt_state_sample_pair_bounded md st.
```

**Formal mathematical reading.** *Mathematical theorem.* This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*, *split*. Named identifiers syntactically cited in the proof block:

```
coins
coins_supp
dsigning_attempt_state_from_sample_pair_sampler
hsample
hsupp
sample
sample_bounded
sample_supp
sample_wf
signing_attempt_sample_of_pair
signing_attempt_sample_y1
signing_attempt_sample_y2
signing_attempt_state_of_sample_pair
signing_attempt_state_sample
signing_attempt_state_sample_pair_bounded
signing_attempt_state_sample_pair_wf
signing_attempt_state_sampled_y1
signing_attempt_state_sampled_y2
signing_coin_distribution_domain
signing_sample_pair_bounded
signing_sample_pair_distribution_ok
signing_sample_pair_sample_ok
```

```

signing_sample_pair_wf
signing_sample_pair_y1
signing_sample_pair_y2
supp_dlet
supp_dmap

```

**Theorem 12.13** (EasyCrypt line 678). *EasyCrypt name.*

```
dstructural_signing_attempt_state_lossless
```

*Checked EasyCrypt statement.*

```

lemma dstructural_signing_attempt_state_lossless md sk m ctx :
  is_lossless (dstructural_signing_attempt_state md sk m ctx).

```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

dmap_ll
dstructural_signing_attempt_state
signing_coin_distribution_lossless

```

**Theorem 12.14** (EasyCrypt line 685). *EasyCrypt name.*

```
dstructural_signing_attempt_state_from_sampler_lossless
```

*Checked EasyCrypt statement.*

```

lemma dstructural_signing_attempt_state_from_sampler_lossless
  md sk m ctx :
  is_lossless
    (dstructural_signing_attempt_state_from_sampler md sk m ctx).

```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```

dsigning_attempt_state_from_sample_pair_sampler_lossless
dstructural_signing_attempt_state_from_sampler
structural_signing_sample_pair_sampler_ok

```

**Theorem 12.15** (EasyCrypt line 695). *EasyCrypt name.*

```
dbounded_unit_signing_attempt_state_lossless
```

*Checked EasyCrypt statement.*

```
lemma dbounded_unit_signing_attempt_state_lossless md sk m ctx :
  is_lossless (dbounded_unit_signing_attempt_state md sk m ctx).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
bounded_unit_signing_sample_pair_sampler_ok
dbounded_unit_signing_attempt_state
dsigning_attempt_state_from_sample_pair_sampler_lossless
```

**Theorem 12.16** (*EasyCrypt* line 703). *EasyCrypt* name.

```
dbounded_unit_signing_attempt_state_sample_ok
```

**Checked *EasyCrypt* statement.**

```
lemma dbounded_unit_signing_attempt_state_sample_ok md sk m ctx st :
  st \in dbounded_unit_signing_attempt_state md sk m ctx =>
  signing_attempt_state_sample_pair_wf md st /\
  signing_attempt_state_sample_pair_bounded md st.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
bounded_unit_signing_sample_pair_sampler_ok
dbounded_unit_signing_attempt_state
dsigning_attempt_state_from_sample_pair_sampler_sample_ok
```

**Theorem 12.17** (*EasyCrypt* line 713). *EasyCrypt* name.

```
dchecked_hyperball_signing_attempt_state_lossless
```

**Checked *EasyCrypt* statement.**

```
lemma dchecked_hyperball_signing_attempt_state_lossless md sk m ctx :
  is_lossless (dchecked_hyperball_signing_attempt_state md sk m ctx).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
checked_hyperball_signing_sample_pair_sampler_ok
dchecked_hyperball_signing_attempt_state
dsigning_attempt_state_from_sample_pair_sampler_lossless
```

**Theorem 12.18** (EasyCrypt line 721). *EasyCrypt name.*

```
dchecked_hyperball_signing_attempt_state_sample_ok
```

*Checked EasyCrypt statement.*

```
lemma dchecked_hyperball_signing_attempt_state_sample_ok md sk m ctæ st :
  st \in dchecked_hyperball_signing_attempt_state md sk m ctæ =>
  signing_attempt_state_sample_pair_wf md st /\
  signing_attempt_state_sample_pair_bounded md st.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
checked_hyperball_signing_sample_pair_sampler_ok
dchecked_hyperball_signing_attempt_state
dsigning_attempt_state_from_sample_pair_sampler_sample_ok
```

**Theorem 12.19** (EasyCrypt line 731). *EasyCrypt name.*

```
dexact_hyperball_signing_attempt_state_lossless
```

*Checked EasyCrypt statement.*

```
lemma dexact_hyperball_signing_attempt_state_lossless md sk m ctæ :
  is_lossless (dexact_hyperball_signing_attempt_state md sk m ctæ).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dexact_hyperball_signing_attempt_state
dsigning_attempt_state_from_sample_pair_sampler_lossless
dexact_hyperball_signing_sample_pair_sampler_ok
```

**Theorem 12.20** (EasyCrypt line 739). *EasyCrypt name.*

```
dexact_hyperball_signing_attempt_state_sample_ok
```

### Checked EasyCrypt statement.

```
lemma dexact_hyperball_signing_attempt_state_sample_ok md sk m ctx st :  
  st \in dexact_hyperball_signing_attempt_state md sk m ctx =>  
    signing_attempt_state_sample_pair_wf md st /\  
    signing_attempt_state_sample_pair_bounded md st.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dexact_hyperball_signing_attempt_state  
dsigning_attempt_state_from_sample_pair_sampler_sample_ok  
exact_hyperball_signing_sample_pair_sampler_ok
```

### Theorem 12.21 (EasyCrypt line 749). *EasyCrypt* name.

```
dexact_hyperball_signing_attempt_state_supportP
```

### Checked EasyCrypt statement.

```
lemma dexact_hyperball_signing_attempt_state_supportP md sk m ctx st :  
  st \in dexact_hyperball_signing_attempt_state md sk m ctx <=>  
    exists (coins : random_coins) (sample : signing_sample_pair),  
      coins \in drandom_coins /\  
      hyperball_sample_ok md sample /\  
      st = signing_attempt_state_of_sample_pair md sk m ctx coins sample.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*. Named identifiers syntactically cited in the proof block:

```
coins  
coins_supp  
dexact_hyperball_signing_attempt_state  
dsigning_attempt_state_from_sample_pair_sampler  
exact_hyperball_signing_sample_pair_supportP  
sample  
sample_ok  
sample_supp  
stE  
supp_dlet  
supp_dmap
```

### Theorem 12.22 (EasyCrypt line 778). *EasyCrypt* name.

```
dexact_hyperball_signing_attempt_state_reference_rejection_abort_bound1
```

### Checked EasyCrypt statement.

```
lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_bound1
  md sk m ctx :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (signing_attempt_state_reference_rejection_aborts md) <= 1%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `smt`. Named identifiers syntactically cited in the proof block:

```
mu_bounded
```

**Theorem 12.23** (EasyCrypt line 786). *EasyCrypt name.*

```
dexact_hyperball_signing_attempt_state_mu_pullback
```

**Checked EasyCrypt statement.**

```
lemma dexact_hyperball_signing_attempt_state_mu_pullback
  md sk m ctx (P : signing_attempt_state -> bool) :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx) P =
  mu (dlet drandom_coins
    (fun coins =>
      dmap (dexact_hyperball_signing_sample_pair md)
        (fun sample =>
          signing_attempt_state_of_sample_pair
            md sk m ctx coins sample)))
    P.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:

```
dexact_hyperball_signing_attempt_state
dsigning_attempt_state_from_sample_pair_sampler
```

**Theorem 12.24** (EasyCrypt line 801). *EasyCrypt name.*

```
signing_attempt_state_reference_rejection_aborts_componentE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_reference_rejection_aborts_componentE md st :
  signing_attempt_state_reference_rejection_aborts md st =
  (signing_attempt_state_reject1_aborts md st \/  

   signing_attempt_state_reject2_aborts md st \/  

   signing_attempt_state_pack_aborts md st \/  

   signing_attempt_state_hint_norm_aborts md st).
```



**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_hint_norm_aborts
signing_attempt_state_pack_aborts
signing_attempt_state_reference_rejection_aborts
signing_attempt_state_reference_rejection_accepts
signing_attempt_state_reject1_aborts
signing_attempt_state_reject2_accepts
```

**Theorem 12.25** (EasyCrypt line 817). *EasyCrypt name.*

```
dexact_hyperball_signing_attempt_state_reference_rejection_abort_split
```

**Checked EasyCrypt statement.**

```
lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_split
  md sk m ctx :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (signing_attempt_state_reference_rejection_aborts md) =
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (predU
      (signing_attempt_state_reject1_aborts md)
      (predU
        (signing_attempt_state_reject2_aborts md)
        (predU
          (signing_attempt_state_pack_aborts md)
          (signing_attempt_state_hint_norm_aborts md))))).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
mu_eq
predU
signing_attempt_state_reference_rejection_aborts_componentE
st
```

**Theorem 12.26** (EasyCrypt line 834). *EasyCrypt name.*

```
dexact_hyperball_signing_attempt_state_reference_rejection_abort_union_bound
```

**Checked EasyCrypt statement.**

```
lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_union_bound
  md sk m ctx :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (signing_attempt_state_reference_rejection_aborts md) <=
```

```

mu (dexact_hyperball_signing_attempt_state md sk m ctx)
  (signing_attempt_state_reject1_aborts md) +
mu (dexact_hyperball_signing_attempt_state md sk m ctx)
  (signing_attempt_state_reject2_aborts md) +
mu (dexact_hyperball_signing_attempt_state md sk m ctx)
  (signing_attempt_state_pack_aborts md) +
mu (dexact_hyperball_signing_attempt_state md sk m ctx)
  (signing_attempt_state_hint_norm_aborts md).

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `smt`. Named identifiers syntactically cited in the proof block:

```

dexact_hyperball_signing_attempt_state_reference_rejection_abort_split
mu_bounded
mu_or

```

**Theorem 12.27** (EasyCrypt line 852). *EasyCrypt name.*

```

signing_attempt_state_of_sample_pair_signatureE

```

**Checked EasyCrypt statement.**

```

lemma signing_attempt_state_of_sample_pair_signatureE
  md sk m ctx coins sample :
  signing_attempt_state_signature
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample) =
  sign_internal md sk m ctx coins.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:

```

signing_attempt_state_of_sample_pair
signing_attempt_state_signature

```

**Theorem 12.28** (EasyCrypt line 862). *EasyCrypt name.*

```

signing_attempt_state_of_sample_pair_pack_accepts

```

**Checked EasyCrypt statement.**

```

lemma signing_attempt_state_of_sample_pair_pack_accepts
  md sk m ctx coins sample :
  signing_attempt_state_pack_accepts md
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample).

```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
mode_crypto_bytes_gt0
signing_attempt_state_of_sample_pair_signatureE
signing_attempt_state_pack_accepts
signing_attempt_state_valid_signature_accepts
valid_signature_sign_internal
```

**Theorem 12.29** (*EasyCrypt* line 875). *EasyCrypt* name.

```
signing_attempt_state_of_sample_pair_pack_no_abort
```

**Checked *EasyCrypt* statement.**

```
lemma signing_attempt_state_of_sample_pair_pack_no_abort
  md sk m ctx coins sample :
  ! signing_attempt_state_pack_aborts md
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_sample_pair_pack_accepts
signing_attempt_state_pack_aborts
```

**Theorem 12.30** (*EasyCrypt* line 884). *EasyCrypt* name.

```
dexact_hyperball_signing_attempt_state_pack_abort_zero
```

**Checked *EasyCrypt* statement.**

```
lemma dexact_hyperball_signing_attempt_state_pack_abort_zero
  md sk m ctx :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (signing_attempt_state_pack_aborts md) =
  0%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *have*, *apply*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
coins
ctx
dexact_hyperball_signing_attempt_state
dexact_hyperball_signing_attempt_state_supportP
le0
md
mu
mu0
mu_bounded
mu_le
pred0
sample
signing_attempt_state_of_sample_pair_pack_no_abort
signing_attempt_state_pack_aborts
sk
st
stE
st_supp
```

**Theorem 12.31** (EasyCrypt line 904). *EasyCrypt name.*

```
dstructural_signing_attempt_state_from_samplerE
```

*Checked EasyCrypt statement.*

```
lemma dstructural_signing_attempt_state_from_samplerE md sk m ctx :
  dstructural_signing_attempt_state_from_sampler md sk m ctx =
  dstructural_signing_attempt_state md sk m ctx.
```

**Formal mathematical reading.** *Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.*

**Role in the proof.** *Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*, *apply*. Named identifiers syntactically cited in the proof block:*

```
coins
ctx
dlet_dunit
dmap
dmap_dunit
dsigning_attempt_state_from_sample_pair_sampler
dstructural_signing_attempt_state
dstructural_signing_attempt_state_from_sampler
dstructural_signing_sample_pair
dunit
fun_ext
md
signing_attempt_state_of_sample_pair
signing_sample_pair_of_coins
sk
```

**Theorem 12.32** (EasyCrypt line 925). *EasyCrypt name.*

```
dstructural_signing_attempt_stateE
```

*Checked EasyCrypt statement.*

```
lemma dstructural_signing_attempt_stateE md sk m ctx :
  dstructural_signing_attempt_state md sk m ctx =
  dsigning_attempt_state md sk m ctx.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
coins
dsigning_attempt_state
dstructural_signing_attempt_state
eq_dmap
signing_attempt_state_of_sample_pair_currentE
```

**Theorem 12.33** (EasyCrypt line 934). *EasyCrypt name.*

```
dstructural_signing_attempt_state_from_sampler_coin_sample_pairE
```

**Checked EasyCrypt statement.**

```
lemma dstructural_signing_attempt_state_from_sampler_coin_sample_pairE
  md sk m ctx :
  dmap (dstructural_signing_attempt_coin_sample_pair md sk m ctx)
    (signing_attempt_state_of_coin_sample_pair md sk m ctx) =
  dstructural_signing_attempt_state_from_sampler md sk m ctx.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dmap_comp
dmap_dlet
dsigning_attempt_state_from_sample_pair_sampler
dstructural_signing_attempt_coin_sample_pair
dstructural_signing_attempt_state_from_sampler
eq_dlet
eq_dmap
sample
signing_attempt_state_of_coin_sample_pair
```

**Theorem 12.34** (EasyCrypt line 950). *EasyCrypt name.*

```
dexact_hyperball_signing_attempt_state_coin_sample_pairE
```

**Checked EasyCrypt statement.**

```
lemma dexact_hyperball_signing_attempt_state_coin_sample_pairE
  md sk m ctx :
```

```
dmap (dexact_hyperball_signing_attempt_coin_sample_pair md sk m ctx)
  (signing_attempt_state_of_coin_sample_pair md sk m ctx) =
dexact_hyperball_signing_attempt_state md sk m ctx.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dexact_hyperball_signing_attempt_coin_sample_pair
dexact_hyperball_signing_attempt_state
dmap_comp
dmap_dlet
dsigning_attempt_state_from_sample_pair_sampler
eq_dlet
eq_dmap
sample
signing_attempt_state_of_coin_sample_pair
```

**Theorem 12.35** (EasyCrypt line 966). *EasyCrypt name*.

```
mu_dlet_le_add_all
```

**Checked EasyCrypt statement.**

```
lemma mu_dlet_le_add_all ['a 'b 'c]
  (d : 'a distr) (F1 : 'a -> 'b distr) (F2 : 'a -> 'c distr)
  (P1 : 'b -> bool) (P2 : 'c -> bool) (eps : real) :
  0%r <= eps =>
  (forall x, mu (F1 x) P1 <= mu (F2 x) P2 + eps) =>
  mu (dlet d F1) P1 <= mu (dlet d F2) P2 + eps.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
F1
F2
P1
P2
RealSeries
dletE
eps
eps_ge
eq_sum
exact
fun_ext
ge0_mu1
hsum
ler_add2l
ler_sum
```

```

ler_trans
ler_upmul2l
mu
mu1
mu_bounded
s1_sbl
s2_sbl
s2eps_sbl
seps_sbl
sum
sumD
summable
summableD
summableZ
summable_mu1
summable_mu1_wght

```

**Theorem 12.36** (EasyCrypt line 1004). *EasyCrypt name.*

```
structural_to_exact_hyperball_sample_pair_loss_from_point_loss
```

*Checked EasyCrypt statement.*

```

lemma structural_to_exact_hyperball_sample_pair_loss_from_point_loss :
  structural_to_exact_hyperball_sample_pair_point_loss_obligation =>
    structural_to_exact_hyperball_sample_pair_loss_obligation.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *have*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```

coins
ctx
dexact_hyperball_signing_sample_pair
dstructural_signing_sample_pair
dunitE
hop
hpoint
hs
hsub
md
mu
mu_bounded
mu_sub
pred1
rejection_sampling_loss_term_nonnegative
signing_sample_pair_of_coins
sk
structural_to_exact_hyperball_sample_pair_loss_obligation
structural_to_exact_hyperball_sample_pair_point_loss_obligation

```

**Theorem 12.37** (EasyCrypt line 1023). *EasyCrypt name.*

```
structural_to_exact_hyperball_coin_sample_pair_loss_from_sample_pair_loss
```

*Checked EasyCrypt statement.*

```
lemma structural_to_exact_hyperball_coin_sample_pair_loss_from_sample_pair_loss :
  structural_to_exact_hyperball_sample_pair_loss_obligation =>
  structural_to_exact_hyperball_coin_sample_pair_loss_obligation.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
coins
ctx
dexact_hyperball_signing_attempt_coin_sample_pair
dmapE
dstructural_signing_attempt_coin_sample_pair
exact
hop
md
mu_dlet_le_add_all
rejection_sampling_loss_term_nonnegative
sample
sk
structural_to_exact_hyperball_coin_sample_pair_loss_obligation
structural_to_exact_hyperball_sample_pair_loss_obligation
```

**Theorem 12.38** (EasyCrypt line 1039). *EasyCrypt name*.

```
structural_to_exact_hyperball_attempt_loss_from_coin_sample_pair_loss
```

**Checked EasyCrypt statement.**

```
lemma structural_to_exact_hyperball_attempt_loss_from_coin_sample_pair_loss :
  structural_to_exact_hyperball_coin_sample_pair_loss_obligation =>
  structural_to_exact_hyperball_attempt_loss_obligation.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
cs
ctx
dexact_hyperball_signing_attempt_state_coin_sample_pairE
dmapE
dstructural_signing_attempt_stateE
dstructural_signing_attempt_state_from_samplerE
dstructural_signing_attempt_state_from_sampler_coin_sample_pairE
exact
hop
md
signing_attempt_state_of_coin_sample_pair
sk
```



```
structural_to_exact_hyperball_attempt_loss_obligation
structural_to_exact_hyperball_coin_sample_pair_loss_obligation
```

**Theorem 12.39** (EasyCrypt line 1055). *EasyCrypt name.*

```
signing_attempt_state_of_coins_signatureE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_signatureE md sk m ctx coins :
  signing_attempt_state_signature
    (signing_attempt_state_of_coins md sk m ctx coins) =
  sign_internal md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 12.40** (EasyCrypt line 1062). *EasyCrypt name.*

```
signing_attempt_state_of_coins_sampled_y1E
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_sampled_y1E md sk m ctx coins :
  signing_attempt_state_sampled_y1
    (signing_attempt_state_of_coins md sk m ctx coins) =
  signing_sample_y1 md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_sample_y1
signing_attempt_state_of_coins
signing_attempt_state_sample
signing_attempt_state_sampled_y1
```

**Theorem 12.41** (EasyCrypt line 1073). *EasyCrypt name.*

```
signing_attempt_state_of_coins_sampled_y2E
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_sampled_y2E md sk m ctx coins :
  signing_attempt_state_sampled_y2
    (signing_attempt_state_of_coins md sk m ctx coins) =
  signing_sample_y2 md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_sample_y2
signing_attempt_state_of_coins
signing_attempt_state_sample
signing_attempt_state_sampled_y2
```

**Theorem 12.42** (EasyCrypt line 1084). *EasyCrypt* name.

```
signing_attempt_state_of_coins_sampled_yE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_sampled_yE md sk m ctx coins :
  signing_attempt_state_sampled_y
    (signing_attempt_state_of_coins md sk m ctx coins) =
  commitment_raw md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_sample_commitment_raw
signing_attempt_state_of_coins
signing_attempt_state_sample
signing_attempt_state_sampled_y
```

**Theorem 12.43** (EasyCrypt line 1095). *EasyCrypt* name.

```
signing_attempt_state_of_coins_highbitsE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_highbitsE md sk m ctx coins :
  signing_attempt_state_highbits
    (signing_attempt_state_of_coins md sk m ctx coins) =
  commitment_highbits md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 12.44** (EasyCrypt line 1102). *EasyCrypt* name.

```
signing_attempt_state_of_coins_lowbitsE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_lowbitsE md sk m ctx coins :  
  signing_attempt_state_lowbits  
    (signing_attempt_state_of_coins md sk m ctx coins) =  
    commitment_lowbits md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 12.45** (EasyCrypt line 1109). **EasyCrypt name.**

```
signing_attempt_state_of_coins_challengeE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_challengeE md sk m ctx coins :  
  signing_attempt_state_challenge  
    (signing_attempt_state_of_coins md sk m ctx coins) =  
    commitment_challenge md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 12.46** (EasyCrypt line 1116). **EasyCrypt name.**

```
signing_attempt_state_of_coins_responseE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_responseE md sk m ctx coins :  
  signing_attempt_state_response  
    (signing_attempt_state_of_coins md sk m ctx coins) =  
    response_vector md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 12.47** (EasyCrypt line 1123). **EasyCrypt name.**

```
signing_attempt_state_of_coins_response_auxE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_response_auxE md sk m ctx coins :  
  signing_attempt_state_response_aux  
    (signing_attempt_state_of_coins md sk m ctx coins) =  
    response_aux_vector md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 12.48** (EasyCrypt line 1130). *EasyCrypt name.*

```
signing_attempt_state_of_coins_hintE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_hintE md sk m ctx coins :
  signing_attempt_state_hint
    (signing_attempt_state_of_coins md sk m ctx coins) =
  hint_vector md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 12.49** (EasyCrypt line 1137). *EasyCrypt name.*

```
signing_attempt_state_of_coins_lowbits_carrierE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_lowbits_carrierE md sk m ctx coins :
  signing_attempt_state_lowbits_carrier
    (signing_attempt_state_of_coins md sk m ctx coins) =
  nth poly_zero (commitment_raw md sk m ctx coins) 0.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_lowbits_carrier
signing_attempt_state_of_coins_sampled_yE
```

**Theorem 12.50** (EasyCrypt line 1146). *EasyCrypt name.*

```
signing_attempt_state_of_coins_tokenE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_tokenE md sk m ctx coins :
  signing_attempt_state_token
    (signing_attempt_state_of_coins md sk m ctx coins) =
  signature_token md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signature_token
signing_attempt_state_of_coins_hintE
signing_attempt_state_of_coins_responseE
signing_attempt_state_of_coins_response_auxE
signing_attempt_state_token
```

**Theorem 12.51** (EasyCrypt line 1158). *EasyCrypt name*.

```
signing_attempt_state_of_coins_programmed_lowbitsE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_programmed_lowbitsE md sk m ctx coins :
  signing_attempt_state_programmed_lowbits
    (signing_attempt_state_of_coins md sk m ctx coins) =
    commitment_lowbits md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
commitment_lowbits
signing_attempt_state_coins
signing_attempt_state_of_coins
signing_attempt_state_of_coins_lowbits_carrierE
signing_attempt_state_programmed_lowbits
```

**Theorem 12.52** (EasyCrypt line 1169). *EasyCrypt name*.

```
signing_attempt_state_of_coins_lowbits_consistent
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_lowbits_consistent md sk m ctx coins :
  signing_attempt_state_lowbits_consistent
    (signing_attempt_state_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

signing_attempt_state_lowbits_consistent
signing_attempt_state_of_coins_lowbitsE
signing_attempt_state_of_coins_programmed_lowbitsE

```

**Theorem 12.53** (EasyCrypt line 1178). *EasyCrypt name.*

```

signing_attempt_state_of_coins_programmed_challengeE

```

**Checked EasyCrypt statement.**

```

lemma signing_attempt_state_of_coins_programmed_challengeE md sk m ctx coins :
  signing_attempt_state_programmed_challenge md (public_key_of_secret md sk)
    m ctx (signing_attempt_state_of_coins md sk m ctx coins) =
    commitment_challenge md sk m ctx coins.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

commitment_challenge
signing_attempt_state_of_coins_lowbitsE
signing_attempt_state_of_coins_response_auxE
signing_attempt_state_programmed_challenge

```

**Theorem 12.54** (EasyCrypt line 1189). *EasyCrypt name.*

```

signing_attempt_state_of_coins_challenge_consistent

```

**Checked EasyCrypt statement.**

```

lemma signing_attempt_state_of_coins_challenge_consistent md sk m ctx coins :
  signing_attempt_state_challenge_consistent md (public_key_of_secret md sk)
    m ctx (signing_attempt_state_of_coins md sk m ctx coins).

```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

signing_attempt_state_challenge_consistent
signing_attempt_state_of_coins_challengeE
signing_attempt_state_of_coins_programmed_challengeE

```

**Theorem 12.55** (EasyCrypt line 1198). *EasyCrypt name.*

```

signing_attempt_state_of_coins_reconstructed_highbitsE

```

### Checked EasyCrypt statement.

```
lemma signing_attempt_state_of_coins_reconstructed_highbitsE
  md sk m ctx coins :
  signing_attempt_state_reconstructed_highbits md (public_key_of_secret md sk)
    (signing_attempt_state_of_coins md sk m ctx coins) =
  commitment_highbits md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
commitment_highbits
signing_attempt_state_of_coins_challengeE
signing_attempt_state_of_coins_response_auxE
signing_attempt_state_reconstructed_highbits
```

### Theorem 12.56 (EasyCrypt line 1210). *EasyCrypt* name.

```
signing_attempt_state_of_coins_public_equation_consistent
```

### Checked EasyCrypt statement.

```
lemma signing_attempt_state_of_coins_public_equation_consistent
  md sk m ctx coins :
  signing_attempt_state_public_equation_consistent md
    (public_key_of_secret md sk)
    (signing_attempt_state_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_coins_highbitsE
signing_attempt_state_of_coins_reconstructed_highbitsE
signing_attempt_state_public_equation_consistent
```

### Theorem 12.57 (EasyCrypt line 1221). *EasyCrypt* name.

```
signing_attempt_state_of_coins_signature_of_fieldsE
```

### Checked EasyCrypt statement.

```
lemma signing_attempt_state_of_coins_signature_of_fieldsE md sk m ctx coins :
  signing_attempt_state_signature_of_fields
    (public_key_of_secret md sk) m ctx
    (signing_attempt_state_of_coins md sk m ctx coins) =
  sign_internal md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
sign_internal
signing_attempt_state_of_coins_challengeE
signing_attempt_state_of_coins_highbitsE
signing_attempt_state_of_coins_lowbitsE
signing_attempt_state_of_coins_tokenE
signing_attempt_state_signature_of_fields
```

**Theorem 12.58** (EasyCrypt line 1234). *EasyCrypt name*.

```
signing_attempt_state_of_coins_fields_match_signature
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_fields_match_signature md sk m ctx coins :
  signing_attempt_state_fields_match_signature
    (public_key_of_secret md sk) m ctx
    (signing_attempt_state_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_fields_match_signature
signing_attempt_state_of_coins_signatureE
signing_attempt_state_of_coins_signature_of_fieldsE
```

**Theorem 12.59** (EasyCrypt line 1244). *EasyCrypt name*.

```
signing_attempt_state_of_coins_field_accepts
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_field_accepts md sk m ctx coins :
  signing_attempt_state_field_accepts md (public_key_of_secret md sk)
    m ctx (signing_attempt_state_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.



Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_field_accepts
signing_attempt_state_of_coins_challenge_consistent
signing_attempt_state_of_coins_fields_match_signature
signing_attempt_state_of_coins_lowbits_consistent
signing_attempt_state_of_coins_public_equation_consistent
```

**Theorem 12.60** (EasyCrypt line 1258). *EasyCrypt name*.

```
signing_attempt_state_of_coins_accepts_current
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_accepts_current md sk m ctx coins :
  signing_attempt_state_accepts md
    (signing_attempt_state_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
response_norm_ok_current
signing_attempt_state_accepts
signing_attempt_state_hint_norm_accepts
signing_attempt_state_of_coins_signatureE
signing_attempt_state_response_norm_accepts
signing_attempt_state_valid_signature_accepts
valid_signature_sign_internal
verify_norm_ok_current
```

**Theorem 12.61** (EasyCrypt line 1272). *EasyCrypt name*.

```
signing_attempt_reject1_bound_ge_response_bound
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_reject1_bound_ge_response_bound md :
  mode_b1sq md <= signing_attempt_reject1_bound md.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 12.62** (EasyCrypt line 1276). *EasyCrypt name*.

```
signing_attempt_state_reject1_accepts_from_response_norm_ok
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_reject1_accepts_from_response_norm_ok md st :
  signing_attempt_state_response_norm_accepts md st =>
  signing_attempt_state_reject1_accepts md st.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *smt*. Named identifiers syntactically cited in the proof block:

```
norm_ge0
norm_le
response_norm_ok
response_norm_sq
signing_attempt_reject1_bound_ge_response_bound
signing_attempt_state_reject1_accepts
signing_attempt_state_reject1_norm_sq
signing_attempt_state_response_norm_accepts
```

**Theorem 12.63** (EasyCrypt line 1289). *EasyCrypt name.*

```
signing_attempt_state_balance_norm_sq_concreteE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_balance_norm_sq_concreteE md st :
  signing_attempt_state_balance_norm_sq md st =
  signing_attempt_reject2_balance_norm_sq md
    (signing_attempt_state_response st)
    (signing_attempt_state_response_aux st)
    (signing_attempt_state_reject2_sampled_y1 st)
    (signing_attempt_state_reject2_sampled_y2 st).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 12.64** (EasyCrypt line 1298). *EasyCrypt name.*

```
signing_attempt_state_reject2_under_bound_concreteE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_reject2_under_bound_concreteE md st :
  signing_attempt_state_reject2_under_bound md st =
  (signing_attempt_reject2_balance_norm_sq md
    (signing_attempt_state_response st)
    (signing_attempt_state_response_aux st)
    (signing_attempt_state_reject2_sampled_y1 st)
    (signing_attempt_state_reject2_sampled_y2 st) <
    signing_attempt_reject2_bound md).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_balance_norm_sq_concreteE
signing_attempt_state_reject2_under_bound
```

**Theorem 12.65** (EasyCrypt line 1311). *EasyCrypt name*.

```
signing_attempt_state_reject2_aborts_concreteE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_reject2_aborts_concreteE md st :
  signing_attempt_state_reject2_aborts md st =
    signing_attempt_reject2_concrete_aborts md
      (signing_attempt_state_bimodal_reject2_branch st)
      (signing_attempt_state_response st)
      (signing_attempt_state_response_aux st)
      (signing_attempt_state_reject2_sampled_y1 st)
      (signing_attempt_state_reject2_sampled_y2 st).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 12.66** (EasyCrypt line 1321). *EasyCrypt name*.

```
signing_attempt_state_reject2_accepts_from_branch_clear
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_reject2_accepts_from_branch_clear md st :
  ! signing_attempt_state_bimodal_reject2_branch st =>
    signing_attempt_state_reject2_accepts md st.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
branch_clear
signing_attempt_reject2_concrete_aborts
signing_attempt_state_reject2_aborts
signing_attempt_state_reject2_accepts
```

**Theorem 12.67** (EasyCrypt line 1332). *EasyCrypt name*.

```
signing_attempt_state_of_sample_pair_reject2_branch_clear
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_reject2_branch_clear
  md sk m ctx coins sample :
  ! signing_attempt_state_bimodal_reject2_branch
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
sign_internal
signature_rejection_branch_byte
signing_attempt_state_bimodal_reject2_branch
signing_attempt_state_of_sample_pair_signatureE
signing_attempt_state_reject2_branch_bit
signing_attempt_state_reject2_branch_byte
```

**Theorem 12.68** (EasyCrypt line 1345). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_reject2_accepts
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_reject2_accepts
  md sk m ctx coins sample :
  signing_attempt_state_reject2_accepts md
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_sample_pair_reject2_branch_clear
signing_attempt_state_reject2_accepts_from_branch_clear
```

**Theorem 12.69** (EasyCrypt line 1354). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_reject2_no_abort
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_reject2_no_abort
  md sk m ctx coins sample :
  ! signing_attempt_state_reject2_aborts md
```

```
(signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_reject2_concrete_aborts
signing_attempt_state_of_sample_pair_reject2_branch_clear
signing_attempt_state_reject2_aborts
```

**Theorem 12.70** (EasyCrypt line 1364). *EasyCrypt* name.

```
dexact_hyperball_signing_attempt_state_reject2_abort_zero
```

**Checked *EasyCrypt* statement.**

```
lemma dexact_hyperball_signing_attempt_state_reject2_abort_zero
  md sk m ctx :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (signing_attempt_state_reject2_aborts md) =
  0%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *apply*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
coins
ctx
dexact_hyperball_signing_attempt_state
dexact_hyperball_signing_attempt_state_supportP
le0
md
mu
mu0
mu_bounded
mu_le
pred0
sample
signing_attempt_state_of_sample_pair_reject2_no_abort
signing_attempt_state_reject2_aborts
sk
st
stE
st_supp
```

**Theorem 12.71** (EasyCrypt line 1384). *EasyCrypt* name.

```
dexact_hyperball_signing_attempt_state_reference_rejection_abort_reduced_bound
```

**Checked EasyCrypt statement.**

```
lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_reduced_bound
  md sk m ctx :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (signing_attempt_state_reference_rejection_aborts md) <=
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (signing_attempt_state_reject1_aborts md) +
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (signing_attempt_state_hint_norm_aborts md).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
ctx
dexact_hyperball_signing_attempt_state_pack_abort_zero
dexact_hyperball_signing_attempt_state_reference_rejection_abort_union_bound
dexact_hyperball_signing_attempt_state_reject2_abort_zero
md
mu_bounded
sk
ub
```

**Theorem 12.72** (EasyCrypt line 1401). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_response_norm_accepts
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_response_norm_accepts
  md sk m ctx coins sample :
  signing_attempt_state_response_norm_accepts md
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
response_norm_ok_current
signing_attempt_state_of_sample_pair_signatureE
signing_attempt_state_response_norm_accepts
```

**Theorem 12.73** (EasyCrypt line 1411). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_reject1_accepts
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_reject1_accepts
  md sk m ctx coins sample :
  signing_attempt_state_reject1_accepts md
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_sample_pair_response_norm_accepts
signing_attempt_state_reject1_accepts_from_response_norm_ok
```

**Theorem 12.74** (*EasyCrypt* line 1420). *EasyCrypt* name.

```
signing_attempt_state_of_sample_pair_reject1_no_abort
```

**Checked *EasyCrypt* statement.**

```
lemma signing_attempt_state_of_sample_pair_reject1_no_abort
  md sk m ctx coins sample :
  ! signing_attempt_state_reject1_aborts md
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_sample_pair_reject1_accepts
signing_attempt_state_reject1_aborts
```

**Theorem 12.75** (*EasyCrypt* line 1429). *EasyCrypt* name.

```
dexact_hyperball_signing_attempt_state_reject1_abort_zero
```

**Checked *EasyCrypt* statement.**

```
lemma dexact_hyperball_signing_attempt_state_reject1_abort_zero
  md sk m ctx :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (signing_attempt_state_reject1_aborts md) =
  0%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *apply*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
coins
ctx
dexact_hyperball_signing_attempt_state
dexact_hyperball_signing_attempt_state_supportP
le0
md
mu
mu0
mu_bounded
mu_le
pred0
sample
signing_attempt_state_of_sample_pair_reject1_no_abort
signing_attempt_state_reject1_aborts
sk
st
stE
st_supp
```

**Theorem 12.76** (EasyCrypt line 1449). *EasyCrypt name.*

```
dexact_hyperball_signing_attempt_state_reference_rejection_abort_hint_bound
```

**Checked EasyCrypt statement.**

```
lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_hint_bound
  md sk m ctx :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (signing_attempt_state_reference_rejection_aborts md) <=
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (signing_attempt_state_hint_norm_aborts md).
```

**Formal mathematical reading.** *Mathematical theorem.* This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** *Use in this chapter.* It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
ctx
dexact_hyperball_signing_attempt_state_reference_rejection_abort_reduced_bound
dexact_hyperball_signing_attempt_state_reject1_abort_zero
md
mu_bounded
sk
ub
```

**Theorem 12.77** (EasyCrypt line 1463). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_hint_norm_accepts
```

**Checked EasyCrypt statement.**



```
lemma signing_attempt_state_of_sample_pair_hint_norm_accepts
  md sk m ctx coins sample :
  signing_attempt_state_hint_norm_accepts md
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_hint_norm_accepts
signing_attempt_state_of_sample_pair_signatureE
verify_norm_ok_current
```

**Theorem 12.78** (*EasyCrypt* line 1473). *EasyCrypt* name.

```
signing_attempt_state_of_sample_pair_hint_norm_no_abort
```

**Checked *EasyCrypt* statement.**

```
lemma signing_attempt_state_of_sample_pair_hint_norm_no_abort
  md sk m ctx coins sample :
  ! signing_attempt_state_hint_norm_aborts md
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_hint_norm_aborts
signing_attempt_state_of_sample_pair_hint_norm_accepts
```

**Theorem 12.79** (*EasyCrypt* line 1482). *EasyCrypt* name.

```
dexact_hyperball_signing_attempt_state_hint_norm_abort_zero
```

**Checked *EasyCrypt* statement.**

```
lemma dexact_hyperball_signing_attempt_state_hint_norm_abort_zero
  md sk m ctx :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (signing_attempt_state_hint_norm_aborts md) =
  0%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *apply*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
coins
ctx
dexact_hyperball_signing_attempt_state
dexact_hyperball_signing_attempt_state_supportP
le0
md
mu
mu0
mu_bounded
mu_le
pred0
sample
signing_attempt_state_hint_norm_aborts
signing_attempt_state_of_sample_pair_hint_norm_no_abort
sk
st
stE
st_supp
```

**Theorem 12.80** (EasyCrypt line 1502). *EasyCrypt name.*

```
dexact_hyperball_signing_attempt_state_reference_rejection_abort_zero
```

**Checked EasyCrypt statement.**

```
lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_zero
  md sk m ctx :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (signing_attempt_state_reference_rejection_aborts md) =
  0%r.
```

**Formal mathematical reading.** *Mathematical theorem.* This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** *Use in this chapter.* It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
ctx
dexact_hyperball_signing_attempt_state_hint_norm_abort_zero
dexact_hyperball_signing_attempt_state_reference_rejection_abort_hint_bound
md
mu_bounded
sk
ub
```

**Theorem 12.81** (EasyCrypt line 1515). *EasyCrypt name.*

```
signing_attempt_state_of_coins_reject2_branch_clear_current
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_reject2_branch_clear_current
```

```
md sk m ctx coins :
! signing_attempt_state_bimodal_reject2_branch
  (signing_attempt_state_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
sign_internal
signature_rejection_branch_byte
signing_attempt_state_bimodal_reject2_branch
signing_attempt_state_of_coins_signatureE
signing_attempt_state_reject2_branch_bit
signing_attempt_state_reject2_branch_byte
```

**Theorem 12.82** (*EasyCrypt* line 1528). *EasyCrypt* name.

```
signing_attempt_state_of_coins_reject2_sampled_y1E
```

**Checked *EasyCrypt* statement.**

```
lemma signing_attempt_state_of_coins_reject2_sampled_y1E md sk m ctx coins :
  signing_attempt_state_reject2_sampled_y1
    (signing_attempt_state_of_coins md sk m ctx coins) =
  signing_sample_y1 md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_coins_sampled_y1E
signing_attempt_state_reject2_sampled_y1
```

**Theorem 12.83** (*EasyCrypt* line 1537). *EasyCrypt* name.

```
signing_attempt_state_of_coins_reject2_sampled_y2E
```

**Checked *EasyCrypt* statement.**

```
lemma signing_attempt_state_of_coins_reject2_sampled_y2E md sk m ctx coins :
  signing_attempt_state_reject2_sampled_y2
    (signing_attempt_state_of_coins md sk m ctx coins) =
  signing_sample_y2 md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_coins_sampled_y2E
signing_attempt_state_reject2_sampled_y2
```

**Theorem 12.84** (EasyCrypt line 1546). *EasyCrypt name*.

```
signing_attempt_state_of_coins_reject2_balance_norm_sqE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_reject2_balance_norm_sqE
  md sk m ctx coins :
  signing_attempt_state_balance_norm_sq md
    (signing_attempt_state_of_coins md sk m ctx coins) =
  signing_attempt_reject2_balance_norm_sq md
    (response_vector md sk m ctx coins)
    (response_aux_vector md sk m ctx coins)
    (signing_sample_y1 md sk m ctx coins)
    (signing_sample_y2 md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_balance_norm_sq_concreteE
signing_attempt_state_of_coins_reject2_sampled_y1E
signing_attempt_state_of_coins_reject2_sampled_y2E
signing_attempt_state_of_coins_responseE
signing_attempt_state_of_coins_response_auxE
```

**Theorem 12.85** (EasyCrypt line 1563). *EasyCrypt name*.

```
signing_attempt_state_of_coins_balance_norm_sq_current
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_balance_norm_sq_current
  md sk m ctx coins :
  signing_attempt_state_balance_norm_sq md
    (signing_attempt_state_of_coins md sk m ctx coins) =
  polyvecl_norm_sq md
    (signing_attempt_state_reject2_balance_response md
      (signing_attempt_state_of_coins md sk m ctx coins)) +
  polyveck_norm_sq md
    (signing_attempt_state_reject2_balance_aux md
      (signing_attempt_state_of_coins md sk m ctx coins)).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_reject2_balance_norm_sq
signing_attempt_state_balance_norm_sq
signing_attempt_state_reject2_balance_aux
signing_attempt_state_reject2_balance_response
```

**Theorem 12.86** (EasyCrypt line 1580). *EasyCrypt name*.

```
signing_attempt_state_of_coins_reject2_accepts_current
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_reject2_accepts_current
  md sk m ctx coins :
  signing_attempt_state_reject2_accepts md
  (signing_attempt_state_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_coins_reject2_branch_clear_current
signing_attempt_state_reject2_accepts_from_branch_clear
```

**Theorem 12.87** (EasyCrypt line 1589). *EasyCrypt name*.

```
signing_attempt_state_pack_accepts_from_valid
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_pack_accepts_from_valid md st :
  signing_attempt_state_valid_signature_accepts md st =>
  signing_attempt_state_pack_accepts md st.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
mode_crypto_bytes_gt0
signing_attempt_state_pack_accepts
valid_ok
```

**Theorem 12.88** (EasyCrypt line 1599). *EasyCrypt name.*

```
signing_attempt_state_reference_rejection_accepts_from_accepts
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_reference_rejection_accepts_from_accepts md st :
  signing_attempt_state_accepts md st =>
  signing_attempt_state_reject2_accepts md st =>
  signing_attempt_state_reference_rejection_accepts md st.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
hint_ok
reject2_ok
response_ok
signing_attempt_state_accepts
signing_attempt_state_pack_accepts_from_valid
signing_attempt_state_reference_rejection_accepts
signing_attempt_state_reject1_accepts_from_response_norm_ok
valid_ok
```

**Theorem 12.89** (EasyCrypt line 1616). *EasyCrypt name.*

```
signing_attempt_state_of_coins_reference_rejection_accepts_current
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_reference_rejection_accepts_current
  md sk m ctx coins :
  signing_attempt_state_reference_rejection_accepts md
  (signing_attempt_state_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_coins_accepts_current
signing_attempt_state_of_coins_reject2_accepts_current
signing_attempt_state_reference_rejection_accepts_from_accepts
```

**Theorem 12.90** (EasyCrypt line 1626). *EasyCrypt name.*

```
signing_attempt_state_of_coins_verification_accepts_current
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_verification_accepts_current
  md sk m ctx coins :
  signing_attempt_state_verification_accepts md (public_key_of_secret md sk)
  m ctx (signing_attempt_state_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** *Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.*

**Role in the proof.** *Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **rewrite**, **split**, **apply**. Named identifiers syntactically cited in the proof block:*

```
signing_attempt_state_of_coins_accepts_current
signing_attempt_state_of_coins_field_accepts
signing_attempt_state_verification_accepts
```

**Theorem 12.91** (EasyCrypt line 1637). *EasyCrypt name.*

```
signing_attempt_state_of_coins_rejection_accepts_current
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_rejection_accepts_current
  md sk m ctx coins :
  signing_attempt_state_rejection_accepts md (public_key_of_secret md sk)
  m ctx (signing_attempt_state_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** *Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.*

**Role in the proof.** *Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **rewrite**, **split**, **apply**. Named identifiers syntactically cited in the proof block:*

```
signing_attempt_state_of_coins_reference_rejection_accepts_current
signing_attempt_state_of_coins_verification_accepts_current
signing_attempt_state_rejection_accepts
```

**Theorem 12.92** (EasyCrypt line 1648). *EasyCrypt name.*

```
signing_attempt_state_of_coins_rejection_no_abort_current
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_coins_rejection_no_abort_current
  md sk m ctx coins :
  ! signing_attempt_state_rejection_aborts md (public_key_of_secret md sk)
  m ctx (signing_attempt_state_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_coins_rejection_accepts_current
signing_attempt_state_rejection_aborts
```

**Theorem 12.93** (*EasyCrypt* line 1657). *EasyCrypt* name.

```
signing_attempt_state_of_coins_no_abort_current
```

**Checked *EasyCrypt* statement.**

```
lemma signing_attempt_state_of_coins_no_abort_current md sk m ctx coins :
  ! signing_attempt_state_aborts md
    (signing_attempt_state_of_coins md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
response_norm_ok_current
signing_attempt_state_aborts
signing_attempt_state_hint_norm_aborts
signing_attempt_state_hint_norm_accepts
signing_attempt_state_of_coins_signatureE
signing_attempt_state_rejection_sampling_aborts
signing_attempt_state_response_norm_aborts
signing_attempt_state_response_norm_accepts
signing_attempt_state_valid_signature_accepts
valid_signature_sign_internal
verify_norm_ok_current
```

**Theorem 12.94** (*EasyCrypt* line 1674). *EasyCrypt* name.

```
signing_attempt_state_distribution_lossless
```

**Checked *EasyCrypt* statement.**

```
lemma signing_attempt_state_distribution_lossless md sk m ctx :
  is_lossless (dsigning_attempt_state md sk m ctx).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.



**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
dmap_ll
dsigning_attempt_state
signing_coin_distribution_lossless
```

**Theorem 12.95** (EasyCrypt line 1681). *EasyCrypt name*.

```
signing_attempt_state_distribution_token_spread
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_distribution_token_spread
  md sk m ctx token :
  mu (dsigning_attempt_state md sk m ctx)
    (fun st =>
      signing_entropy_token_of_coins
        (signing_attempt_state_coins st) = token) <=
      signing_randomness_point_bound.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
coins
dmapE
drandom_coins
dsigning_attempt_state
ler_trans
mu
mu_le
signing_attempt_state_coins
signing_attempt_state_of_coins
signing_coin_distribution_token_point_bound
signing_entropy_token_of_coins
token
```

**Theorem 12.96** (EasyCrypt line 1698). *EasyCrypt name*.

```
signing_attempt_state_distribution_reference_rejection_abort_zero_current
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_distribution_reference_rejection_abort_zero_current
  md sk m ctx :
  mu (dsigning_attempt_state md sk m ctx)
    (signing_attempt_state_reference_rejection_aborts md) =
    0%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
coins
dmapE
dsigning_attempt_state
mu0
mu_eq
random_coins
signing_attempt_state_of_coins_reference_rejection_accepts_current
signing_attempt_state_reference_rejection_aborts
```

**Theorem 12.97** (EasyCrypt line 1712). *EasyCrypt name*.

```
signing_attempt_state_distribution_rejection_abort_zero_current
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_distribution_rejection_abort_zero_current
  md sk m ctx :
  mu (dsigning_attempt_state md sk m ctx)
    (signing_attempt_state_rejection_aborts md
      (public_key_of_secret md sk) m ctx) =
  0%r.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
coins
dmapE
dsigning_attempt_state
mu0
mu_eq
random_coins
signing_attempt_state_of_coins_rejection_no_abort_current
```

**Theorem 12.98** (EasyCrypt line 1726). *EasyCrypt name*.

```
sign_internal_simulation_relation
```

**Checked EasyCrypt statement.**

```
lemma sign_internal_simulation_relation md sk m ctx coins :
  signing_simulation_relation md
    (public_key_of_secret md sk) m ctx
    (sign_internal md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
coins
ctx
md
public_key_of_secret
sign_internal
sign_internal_attempt_relation
signing_attempt_simulates
sk
```

**Theorem 12.99** (EasyCrypt line 1736). *EasyCrypt name.*

```
signing_attempt_transcript_valid
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_transcript_valid md pk sk m ctx coins sig :
  signing_attempt_relation md pk sk m ctx coins sig =>
  transcript_valid md (transcript_of_signature md pk m ctx sig).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
pkE
signing_attempt_relation
transcript_context
transcript_fields_match_signature
transcript_message
transcript_of_signature
transcript_pk
transcript_signature
transcript_valid
transcript_verifies
valid_keypair
valid_signature_sign_internal
verify_internal_sign_internal
```

**Theorem 12.100** (EasyCrypt line 1753). *EasyCrypt name.*

```
signing_attempt_transcript_relation
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_transcript_relation md pk sk m ctx coins sig :
  signing_attempt_relation md pk sk m ctx coins sig =>
  signing_transcript_relation md pk m ctx sig
  (transcript_of_signature md pk m ctx sig).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
attempt
coins
ctx
md
pk
sig
signing_attempt_simulates
signing_attempt_transcript_valid
signing_transcript_relation
sk
```

**Theorem 12.101** (EasyCrypt line 1766). *EasyCrypt name.*

```
sign_internal_transcript_relation
```

**Checked EasyCrypt statement.**

```
lemma sign_internal_transcript_relation md sk m ctx coins :
  signing_transcript_relation md
    (public_key_of_secret md sk) m ctx
    (sign_internal md sk m ctx coins)
    (transcript_of_signature md (public_key_of_secret md sk) m ctx
      (sign_internal md sk m ctx coins)).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
coins
ctx
md
public_key_of_secret
sign_internal
sign_internal_attempt_relation
signing_attempt_transcript_relation
sk
```

**Theorem 12.102** (EasyCrypt line 1779). *EasyCrypt name.*

```
sign_internal_record_relation
```

**Checked EasyCrypt statement.**

```
lemma sign_internal_record_relation md sk m ctx coins :
  signing_record_relation md (public_key_of_secret md sk)
```

```
(m, ctx,
  sign_internal md sk m ctx coins,
  transcript_of_signature md (public_key_of_secret md sk) m ctx
  (sign_internal md sk m ctx coins)).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
sign_internal_transcript_relation
signing_record_context
signing_record_message
signing_record_relation
signing_record_signature
signing_record_transcript
```

**Theorem 12.103** (*EasyCrypt* line 1792). *EasyCrypt* name.

```
signing_record_relation_sound
```

**Checked *EasyCrypt* statement.**

```
lemma signing_record_relation_sound md pk r :
  signing_record_relation md pk r =>
  signing_simulation_relation md pk
    (signing_record_message r)
    (signing_record_context r)
    (signing_record_signature r) /\
  transcript_valid md (signing_record_transcript r).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
hsim
rest
signing_record_relation
signing_transcript_relation
valid
```

**Theorem 12.104** (*EasyCrypt* line 1809). *EasyCrypt* name.

```
signing_record_log_sound_cons
```

**Checked *EasyCrypt* statement.**

```
lemma signing_record_log_sound_cons md pk r rs :
  signing_record_relation md pk r =>
```

```
signing_record_log_sound md pk rs =>
signing_record_log_sound md pk (r :: rs).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_record_log_sound
```

**Theorem 12.105** (EasyCrypt line 1817). *EasyCrypt* name.

```
signing_record_log_sound_transcripts_valid
```

**Checked EasyCrypt statement.**

```
lemma signing_record_log_sound_transcripts_valid md pk rs :
  signing_record_log_sound md pk rs =>
  transcript_log_valid md (signing_record_transcripts rs).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *elim*, *split*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
ih
md
pk
rel
rs
signing_record_log_sound
signing_record_relation_sound
signing_record_transcripts
sound
transcript_log_valid
```

**Theorem 12.106** (EasyCrypt line 1830). *EasyCrypt* name.

```
signing_record_queries_cons
```

**Checked EasyCrypt statement.**

```
lemma signing_record_queries_cons r rs :
  signing_record_queries (r :: rs) =
  signing_record_query r :: signing_record_queries rs.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 12.107** (EasyCrypt line 1835). *EasyCrypt name.*

```
signing_record_transcripts_cons
```

**Checked EasyCrypt statement.**

```
lemma signing_record_transcripts_cons r rs :
  signing_record_transcripts (r :: rs) =
  signing_record_transcript r :: signing_record_transcripts rs.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 12.108** (EasyCrypt line 1840). *EasyCrypt name.*

```
rejection_bound_parts_nonnegative
```

**Checked EasyCrypt statement.**

```
lemma rejection_bound_parts_nonnegative :
  rejection_sampling_bound_obligation =>
  0%r <= rejection_sampling_loss_term.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 12.109** (EasyCrypt line 1845). *EasyCrypt name.*

```
rejection_sampling_bound_obligation_holds
```

**Checked EasyCrypt statement.**

```
lemma rejection_sampling_bound_obligation_holds :
  rejection_sampling_bound_obligation.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
fs_with_aborts_min_entropy_term_nonnegative
fs_with_aborts_reprogramming_term_nonnegative
rejection_sampling_bound_obligation
rejection_sampling_loss_term_nonnegative
```

**Theorem 12.110** (EasyCrypt line 1856). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_sampled_yE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_sampled_yE md sk m ctx coins sample :
  signing_attempt_state_sampled_y
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample) =
  commitment_raw md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_sample_commitment_raw
signing_attempt_sample_of_pair
signing_attempt_state_of_sample_pair
signing_attempt_state_sample
signing_attempt_state_sampled_y
```

**Theorem 12.111** (EasyCrypt line 1868). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_highbitsE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_highbitsE md sk m ctx coins sample :
  signing_attempt_state_highbits
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample) =
  commitment_highbits md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_highbits
signing_attempt_state_of_sample_pair
```

**Theorem 12.112** (EasyCrypt line 1877). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_lowbitsE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_lowbitsE md sk m ctx coins sample :
  signing_attempt_state_lowbits
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample) =
```



```
commitment_lowbits md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_lowbits
signing_attempt_state_of_sample_pair
```

**Theorem 12.113** (EasyCrypt line 1886). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_challengeE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_challengeE md sk m ctx coins sample :
  signing_attempt_state_challenge
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample) =
    commitment_challenge md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_challenge
signing_attempt_state_of_sample_pair
```

**Theorem 12.114** (EasyCrypt line 1895). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_responseE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_responseE md sk m ctx coins sample :
  signing_attempt_state_response
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample) =
    response_vector md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_sample_pair
signing_attempt_state_response
```

**Theorem 12.115** (EasyCrypt line 1904). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_response_auxE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_response_auxE
  md sk m ctx coins sample :
  signing_attempt_state_response_aux
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample) =
  response_aux_vector md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_sample_pair
signing_attempt_state_response_aux
```

**Theorem 12.116** (EasyCrypt line 1914). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_hintE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_hintE md sk m ctx coins sample :
  signing_attempt_state_hint
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample) =
  hint_vector md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_hint
signing_attempt_state_of_sample_pair
```

**Theorem 12.117** (EasyCrypt line 1923). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_lowbits_carrierE
```

**Checked EasyCrypt statement.**

```

lemma signing_attempt_state_of_sample_pair_lowbits_carrierE
  md sk m ctx coins sample :
  signing_attempt_state_lowbits_carrier
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample) =
  nth poly_zero (commitment_raw md sk m ctx coins) 0.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

signing_attempt_state_lowbits_carrier
signing_attempt_state_of_sample_pair_sampled_yE

```

**Theorem 12.118** (EasyCrypt line 1933). *EasyCrypt* name.

```

signing_attempt_state_of_sample_pair_tokenE

```

**Checked EasyCrypt statement.**

```

lemma signing_attempt_state_of_sample_pair_tokenE md sk m ctx coins sample :
  signing_attempt_state_token
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample) =
  signature_token md sk m ctx coins.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

signature_token
signing_attempt_state_of_sample_pair_hintE
signing_attempt_state_of_sample_pair_responseE
signing_attempt_state_of_sample_pair_response_auxE
signing_attempt_state_token

```

**Theorem 12.119** (EasyCrypt line 1945). *EasyCrypt* name.

```

signing_attempt_state_of_sample_pair_programmed_lowbitsE

```

**Checked EasyCrypt statement.**

```

lemma signing_attempt_state_of_sample_pair_programmed_lowbitsE
  md sk m ctx coins sample :
  signing_attempt_state_programmed_lowbits
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample) =
  commitment_lowbits md sk m ctx coins.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
commitment_lowbits
signing_attempt_state_coins
signing_attempt_state_of_sample_pair
signing_attempt_state_of_sample_pair_lowbits_carrierE
signing_attempt_state_programmed_lowbits
```

**Theorem 12.120** (EasyCrypt line 1957). *EasyCrypt name*.

```
signing_attempt_state_of_sample_pair_lowbits_consistent
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_lowbits_consistent
  md sk m ctx coins sample :
  signing_attempt_state_lowbits_consistent
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_lowbits_consistent
signing_attempt_state_of_sample_pair_lowbitsE
signing_attempt_state_of_sample_pair_programmed_lowbitsE
```

**Theorem 12.121** (EasyCrypt line 1967). *EasyCrypt name*.

```
signing_attempt_state_of_sample_pair_programmed_challengeE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_programmed_challengeE
  md sk m ctx coins sample :
  signing_attempt_state_programmed_challenge md (public_key_of_secret md sk)
    m ctx (signing_attempt_state_of_sample_pair md sk m ctx coins sample) =
  commitment_challenge md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
commitment_challenge
signing_attempt_state_of_sample_pair_lowbitsE
signing_attempt_state_of_sample_pair_response_auxE
signing_attempt_state_programmed_challenge
```

**Theorem 12.122** (EasyCrypt line 1979). *EasyCrypt name*.

```
signing_attempt_state_of_sample_pair_challenge_consistent
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_challenge_consistent
  md sk m ctx coins sample :
  signing_attempt_state_challenge_consistent md (public_key_of_secret md sk)
  m ctx (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_challenge_consistent
signing_attempt_state_of_sample_pair_challengeE
signing_attempt_state_of_sample_pair_programmed_challengeE
```

**Theorem 12.123** (EasyCrypt line 1989). *EasyCrypt name*.

```
signing_attempt_state_of_sample_pair_reconstructed_highbitsE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_reconstructed_highbitsE
  md sk m ctx coins sample :
  signing_attempt_state_reconstructed_highbits md (public_key_of_secret md sk)
  (signing_attempt_state_of_sample_pair md sk m ctx coins sample) =
  commitment_highbits md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
commitment_highbits
signing_attempt_state_of_sample_pair_challengeE
signing_attempt_state_of_sample_pair_response_auxE
signing_attempt_state_reconstructed_highbits
```

**Theorem 12.124** (EasyCrypt line 2001). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_public_equation_consistent
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_public_equation_consistent
  md sk m ctx coins sample :
  signing_attempt_state_public_equation_consistent md
    (public_key_of_secret md sk)
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_sample_pair_highbitsE
signing_attempt_state_of_sample_pair_reconstructed_highbitsE
signing_attempt_state_public_equation_consistent
```

**Theorem 12.125** (EasyCrypt line 2012). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_signature_of_fieldsE
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_signature_of_fieldsE
  md sk m ctx coins sample :
  signing_attempt_state_signature_of_fields
    (public_key_of_secret md sk) m ctx
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample) =
  sign_internal md sk m ctx coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
sign_internal
signing_attempt_state_of_sample_pair_challengeE
signing_attempt_state_of_sample_pair_highbitsE
signing_attempt_state_of_sample_pair_lowbitsE
signing_attempt_state_of_sample_pair_tokenE
signing_attempt_state_signature_of_fields
```

**Theorem 12.126** (EasyCrypt line 2026). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_fields_match_signature
```

### Checked EasyCrypt statement.

```
lemma signing_attempt_state_of_sample_pair_fields_match_signature
  md sk m ctx coins sample :
  signing_attempt_state_fields_match_signature
    (public_key_of_secret md sk) m ctx
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_fields_match_signature
signing_attempt_state_of_sample_pair_signatureE
signing_attempt_state_of_sample_pair_signature_of_fieldsE
```

### Theorem 12.127 (EasyCrypt line 2037). EasyCrypt name.

```
signing_attempt_state_of_sample_pair_field_accepts
```

### Checked EasyCrypt statement.

```
lemma signing_attempt_state_of_sample_pair_field_accepts
  md sk m ctx coins sample :
  signing_attempt_state_field_accepts md (public_key_of_secret md sk)
    m ctx (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_field_accepts
signing_attempt_state_of_sample_pair_challenge_consistent
signing_attempt_state_of_sample_pair_fields_match_signature
signing_attempt_state_of_sample_pair_lowbits_consistent
signing_attempt_state_of_sample_pair_public_equation_consistent
```

### Theorem 12.128 (EasyCrypt line 2052). EasyCrypt name.

```
signing_attempt_state_of_sample_pair_valid_signature_accepts
```

### Checked EasyCrypt statement.

```
lemma signing_attempt_state_of_sample_pair_valid_signature_accepts
  md sk m ctx coins sample :
  signing_attempt_state_valid_signature_accepts md
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_sample_pair_signatureE
signing_attempt_state_valid_signature_accepts
valid_signature_sign_internal
```

**Theorem 12.129** (*EasyCrypt* line 2062). *EasyCrypt* name.

```
signing_attempt_state_of_sample_pair_accepts_current
```

**Checked *EasyCrypt* statement.**

```
lemma signing_attempt_state_of_sample_pair_accepts_current
  md sk m ctx coins sample :
  signing_attempt_state_accepts md
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_accepts
signing_attempt_state_of_sample_pair_hint_norm_accepts
signing_attempt_state_of_sample_pair_response_norm_accepts
signing_attempt_state_of_sample_pair_valid_signature_accepts
```

**Theorem 12.130** (*EasyCrypt* line 2075). *EasyCrypt* name.

```
signing_attempt_state_of_sample_pair_verification_accepts_current
```

**Checked *EasyCrypt* statement.**

```
lemma signing_attempt_state_of_sample_pair_verification_accepts_current
  md sk m ctx coins sample :
  signing_attempt_state_verification_accepts md (public_key_of_secret md sk)
    m ctx (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.



Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_sample_pair_accepts_current
signing_attempt_state_of_sample_pair_field_accepts
signing_attempt_state_verification_accepts
```

**Theorem 12.131** (EasyCrypt line 2086). *EasyCrypt* name.

```
signing_attempt_state_of_sample_pair_reference_rejection_accepts
```

*Checked EasyCrypt statement.*

```
lemma signing_attempt_state_of_sample_pair_reference_rejection_accepts
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (coins : random_coins) (sample : signing_sample_pair) :
  signing_attempt_state_reference_rejection_accepts md
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_sample_pair_hint_norm_accepts
signing_attempt_state_of_sample_pair_pack_accepts
signing_attempt_state_of_sample_pair_reject1_accepts
signing_attempt_state_of_sample_pair_reject2_accepts
signing_attempt_state_reference_rejection_accepts
```

**Theorem 12.132** (EasyCrypt line 2102). *EasyCrypt* name.

```
signing_attempt_state_of_sample_pair_reference_rejection_no_abort
```

*Checked EasyCrypt statement.*

```
lemma signing_attempt_state_of_sample_pair_reference_rejection_no_abort
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (coins : random_coins) (sample : signing_sample_pair) :
  ! signing_attempt_state_reference_rejection_aborts md
    (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_sample_pair_reference_rejection_accepts
signing_attempt_state_reference_rejection_aborts
```

**Theorem 12.133** (EasyCrypt line 2112). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_rejection_accepts_current
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_rejection_accepts_current
  md sk m ctx coins sample :
  signing_attempt_state_rejection_accepts md (public_key_of_secret md sk)
  m ctx (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_sample_pair_reference_rejection_accepts
signing_attempt_state_of_sample_pair_verification_accepts_current
signing_attempt_state_rejection_accepts
```

**Theorem 12.134** (EasyCrypt line 2123). *EasyCrypt name.*

```
signing_attempt_state_of_sample_pair_rejection_no_abort_current
```

**Checked EasyCrypt statement.**

```
lemma signing_attempt_state_of_sample_pair_rejection_no_abort_current
  md sk m ctx coins sample :
  ! signing_attempt_state_rejection_aborts md (public_key_of_secret md sk)
  m ctx (signing_attempt_state_of_sample_pair md sk m ctx coins sample).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
signing_attempt_state_of_sample_pair_rejection_accepts_current
signing_attempt_state_rejection_aborts
```

**Theorem 12.135** (EasyCrypt line 2132). *EasyCrypt name.*

```
dexact_hyperball_signing_attempt_state_reference_rejection_accepts
```

**Checked EasyCrypt statement.**

```
lemma dexact_hyperball_signing_attempt_state_reference_rejection_accepts
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (st : signing_attempt_state) :
  st \in dexact_hyperball_signing_attempt_state md sk m ctx =>
  signing_attempt_state_reference_rejection_accepts md st.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
coins
dexact_hyperball_signing_attempt_state_supportP
sample
signing_attempt_state_of_sample_pair_reference_rejection_accepts
stE
st_supp
```

**Theorem 12.136** (EasyCrypt line 2144). *EasyCrypt name.*

```
dexact_hyperball_signing_attempt_state_reference_rejection_no_abort
```

**Checked EasyCrypt statement.**

```
lemma dexact_hyperball_signing_attempt_state_reference_rejection_no_abort
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (st : signing_attempt_state) :
  st \in dexact_hyperball_signing_attempt_state md sk m ctx =>
  ! signing_attempt_state_reference_rejection_aborts md st.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
ctx
dexact_hyperball_signing_attempt_state_reference_rejection_accepts
md
signing_attempt_state_reference_rejection_aborts
sk
st
st_supp
```

**Theorem 12.137** (EasyCrypt line 2156). *EasyCrypt name.*

```
dexact_hyperball_signing_attempt_state_rejection_accepts
```

**Checked EasyCrypt statement.**

```
lemma dexact_hyperball_signing_attempt_state_rejection_accepts
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (st : signing_attempt_state) :
  st \in dexact_hyperball_signing_attempt_state md sk m ctx =>
  signing_attempt_state_rejection_accepts md (public_key_of_secret md sk)
  m ctx st.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
coins
dexact_hyperball_signing_attempt_state_supportP
sample
signing_attempt_state_of_sample_pair_rejection_accepts_current
stE
st_supp
```

**Theorem 12.138** (EasyCrypt line 2169). *EasyCrypt name.*

```
dexact_hyperball_signing_attempt_state_rejection_no_abort
```

**Checked EasyCrypt statement.**

```
lemma dexact_hyperball_signing_attempt_state_rejection_no_abort
  (md : mode) (sk : skey) (m : message) (ctx : context)
  (st : signing_attempt_state) :
  st \in dexact_hyperball_signing_attempt_state md sk m ctx =>
  ! signing_attempt_state_rejection_aborts md (public_key_of_secret md sk)
    m ctx st.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves an exact game replacement or simulator equivalence.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
ctx
dexact_hyperball_signing_attempt_state_rejection_accepts
md
signing_attempt_state_rejection_aborts
sk
st
st_supp
```

**Theorem 12.139** (EasyCrypt line 2182). *EasyCrypt name.*

```
dexact_hyperball_signing_attempt_state_rejection_abort_zero
```

**Checked EasyCrypt statement.**

```
lemma dexact_hyperball_signing_attempt_state_rejection_abort_zero
  md sk m ctx :
  mu (dexact_hyperball_signing_attempt_state md sk m ctx)
    (signing_attempt_state_rejection_aborts md
      (public_key_of_secret md sk) m ctx) =
  0%r.
```

**Formal mathematical reading.** *Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.*

**Role in the proof.** *Use in this chapter. It proves an exact game replacement or simulator equivalence.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `have`, `apply`, `rewrite`, `smt`. Named identifiers syntactically cited in the proof block:*

```
coins
ctx
dexact_hyperball_signing_attempt_state
dexact_hyperball_signing_attempt_state_supportP
le0
md
mu
mu0
mu_bounded
mu_le
pred0
public_key_of_secret
sample
signing_attempt_state_of_sample_pair_rejection_no_abort_current
signing_attempt_state_rejection_aborts
sk
st
stE
st_supp
```

*End of generated formal inventory for HAETAE\_Rejection.ec.*

# Chapter 13

## HAETAE\_Algebra.ec

**Mathematical role.** This file is the deterministic algebraic model of HAETAE. It defines the carrier types, arithmetic operations, key-generation equations, signing equations, verification predicate, and packing/unpacking facts.

**Carrier types.** Polynomials are lists of integer coefficients, and vectors/matrices are lists of polynomials. Well-formedness predicates enforce the expected lengths:

$$\text{poly\_wf}(p) \iff |p| = n, \quad \text{polyveck\_wf}(v) \iff |v| = k.$$

Coefficient arithmetic is reduced modulo  $q = 64513$ .

**Linear algebra.** The operations `matrix_vec_mul`, `polyveck_add`, and `polyveck_sub` define the algebraic relation underlying the public key:

$$t = As + e.$$

For modes 2 and 3 the public vector is a high-bits projection of an adjusted raw vector; for mode 5 it is a negated doubled raw vector. The predicate `public_key_relation` records the mode-dependent relation.

**Signing and verification.** The later definitions build the commitment, message hash, challenge embedding, response, auxiliary response, hint, and final signature. Verification checks that the signature is well formed, that the challenge is consistent with the public reconstruction, and that the response and hint satisfy the norm conditions. The key correctness lemmas used by the top-level file are `valid_signature_sign_internal`, `verify_internal_sign_internal`, `response_norm_ok_current`, and `verify_norm_ok_current`.

**Packing and reference correspondence.** The long middle and late parts of this file prove that byte-level packing and unpacking operations preserve the mathematical public-key object. These lemmas are not the main security reduction, but they prevent the model from silently using an inconsistent public-key representation.

## Textbook Formal Inventory

This subsection records the exact formal material in `HAETAE_Algebra.ec`. It is generated from the proof-surface EasyCrypt file, so the line numbers and statements are meant to be read next to the checked source.

**Chapter role.** deterministic polynomial, vector, matrix, key, signature, and verification algebra.

**Inventory size.** 5 context declarations, 266 definitions/game objects, 282 lemmas or relational theorems, and 0 explicit Hoare/Phoare judgments were found.

### Theory Context, Imports, and Quantified Modules

**Context 1 (line 1)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import AllCore IntDiv List.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 2 (line 2)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import HAETAE_Params HAETAE_FIPS202.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 3 (line 4)** EasyCrypt name.

```
HAETAE_Algebra
```

**Checked EasyCrypt statement.**

```
theory HAETAE_Algebra.
```

**Formal mathematical reading.** Mathematical context. This begins a namespace for the formal development in this file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 4 (line 6)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Params.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 5 (line 7) EasyCrypt name.**

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_FIPS202.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

## Definitions, Carrier Sets, Operations, Games, and Procedures

**Definition 13.1** (EasyCrypt line 9). **EasyCrypt name.**

```
byte
```

**Checked EasyCrypt statement.**

```
type byte = int.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.2** (EasyCrypt line 10). **EasyCrypt name.**

```
seed
```

**Checked EasyCrypt statement.**

```
type seed = byte list.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.3** (EasyCrypt line 11). **EasyCrypt name.**

```
crh
```

**Checked EasyCrypt statement.**



```
type crh = byte list.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.4** (EasyCrypt line 12). **EasyCrypt name.**

```
random_coins
```

**Checked EasyCrypt statement.**

```
type random_coins = byte list.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 13.5** (EasyCrypt line 13). **EasyCrypt name.**

```
xof256_state
```

**Checked EasyCrypt statement.**

```
type xof256_state = byte list.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.6** (EasyCrypt line 14). **EasyCrypt name.**

```
message
```

**Checked EasyCrypt statement.**

```
type message = byte list.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.7** (EasyCrypt line 15). **EasyCrypt name.**

```
context
```

**Checked EasyCrypt statement.**

```
type context = byte list.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.8** (EasyCrypt line 16). **EasyCrypt name.**

```
coeff
```

**Checked EasyCrypt statement.**

```
type coeff = int.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.9** (EasyCrypt line 17). **EasyCrypt name.**

```
poly
```

**Checked EasyCrypt statement.**

```
type poly = coeff list.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.10** (EasyCrypt line 18). **EasyCrypt name.**

```
challenge
```

**Checked EasyCrypt statement.**

```
type challenge = poly.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.11** (EasyCrypt line 19). **EasyCrypt name.**

```
polyveck
```

**Checked EasyCrypt statement.**

```
type polyveck = poly list.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.12** (EasyCrypt line 20). **EasyCrypt name.**

```
polyvecl
```

**Checked EasyCrypt statement.**

```
type polyvecl = poly list.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.13** (EasyCrypt line 21). **EasyCrypt name.**

```
polyvecm
```

**Checked EasyCrypt statement.**

```
type polyvecm = poly list.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.14** (EasyCrypt line 22). **EasyCrypt name.**

```
matrix
```

**Checked EasyCrypt statement.**

```
type matrix = poly list list.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.15** (EasyCrypt line 23). **EasyCrypt name.**

```
hint_t
```

**Checked EasyCrypt statement.**

```
type hint_t = polyveck.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.16** (EasyCrypt line 24). **EasyCrypt name.**

```
sig_token
```

**Checked EasyCrypt statement.**

```
type sig_token = polyvecl * polyveck * hint_t.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.17** (EasyCrypt line 26). **EasyCrypt name.**

```
pkey
```

**Checked EasyCrypt statement.**

```
type pkey = seed * polyveck.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.18** (EasyCrypt line 27). **EasyCrypt name.**

```
encoded_pkey
```

**Checked EasyCrypt statement.**

```
type encoded_pkey = byte list.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.19** (EasyCrypt line 28). **EasyCrypt name.**

```
skey
```

**Checked EasyCrypt statement.**

```
type skey = seed * int.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.20** (EasyCrypt line 29). **EasyCrypt name.**

```
signature
```

**Checked EasyCrypt statement.**

```
type signature = polyveck * poly * crh * challenge * sig_token * int * int.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.21** (EasyCrypt line 31). **EasyCrypt name.**

```
secret_key_of_seed
```

**Checked EasyCrypt statement.**

```
op secret_key_of_seed (_ : mode) (sd : seed) : skey = (sd, 0).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.22** (EasyCrypt line 32). **EasyCrypt name.**

```
poly_zero
```

**Checked EasyCrypt statement.**

```
op poly_zero : poly = nseq n 0.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.23** (EasyCrypt line 33). **EasyCrypt name.**

```
polyveck_zero
```

**Checked EasyCrypt statement.**

```
op polyveck_zero (md : mode) : polyveck = nseq (mode_k md) poly_zero.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.24** (EasyCrypt line 34). **EasyCrypt name.**

```
polyvecl_zero
```

**Checked EasyCrypt statement.**

```
op polyvecl_zero (md : mode) : polyvecl = nseq (mode_l md) poly_zero.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.25** (EasyCrypt line 35). **EasyCrypt name.**

```
polyvecm_zero
```

**Checked EasyCrypt statement.**

```
op polyvecm_zero (md : mode) : polyvecm = nseq (mode_m md) poly_zero.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.26** (EasyCrypt line 36). **EasyCrypt name.**

```
matrix_zero
```

**Checked EasyCrypt statement.**

```
op matrix_zero (md : mode) : matrix =  
  nseq (mode_k md) (polyvecl_zero md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.27** (EasyCrypt line 55). **EasyCrypt name.**

```
poly_wf
```

**Checked EasyCrypt statement.**

```
op poly_wf (p : poly) : bool = size p = n.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.28** (EasyCrypt line 56). **EasyCrypt name.**

```
polyveck_wf
```

**Checked EasyCrypt statement.**

```
op polyveck_wf (md : mode) (xs : polyveck) : bool =
  size xs = mode_k md /\ all poly_wf xs.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.29** (EasyCrypt line 58). **EasyCrypt name.**

```
polyvecl_wf
```

**Checked EasyCrypt statement.**

```
op polyvecl_wf (md : mode) (xs : polyvecl) : bool =
  size xs = mode_l md /\ all poly_wf xs.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.30** (EasyCrypt line 72). **EasyCrypt name.**

```
crh_wf
```

**Checked EasyCrypt statement.**

```
op crh_wf (mu : crh) : bool = size mu = crhbytes.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.31** (EasyCrypt line 73). **EasyCrypt name.**

```
challenge_coeff_ok
```

**Checked EasyCrypt statement.**

```
op challenge_coeff_ok (x : coeff) : bool = x = -1 /\ x = 0 /\ x = 1.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.32** (EasyCrypt line 74). **EasyCrypt name.**

```
challenge_wf
```

**Checked EasyCrypt statement.**

```
op challenge_wf (ch : challenge) : bool =
  poly_wf ch /\ all challenge_coeff_ok ch.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.33** (EasyCrypt line 76). **EasyCrypt name.**

```
unit_coeff_ok
```

**Checked EasyCrypt statement.**

```
op unit_coeff_ok (x : coeff) : bool = x = -1 \/ x = 1.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.34** (EasyCrypt line 77). **EasyCrypt name.**

```
challenge_sparse_at
```

**Checked EasyCrypt statement.**

```
op challenge_sparse_at (md : mode) (i : int) (x : coeff) : bool =
  if i < mode_tau md then unit_coeff_ok x else x = 0.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.35** (EasyCrypt line 79). **EasyCrypt name.**

```
challenge_sparse
```

**Checked EasyCrypt statement.**

```
op challenge_sparse (md : mode) (ch : challenge) : bool =
  size ch = n /\
  forall (i : int), 0 <= i /\ i < n =>
    challenge_sparse_at md i (nth 0 ch i).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.36** (EasyCrypt line 83). **EasyCrypt name.**



poly\_unit

**Checked EasyCrypt statement.**

```
op poly_unit (p : poly) : bool = poly_wf p /\ all unit_coeff_ok p.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.37** (EasyCrypt line 84). **EasyCrypt name.**

polyveck\_unit

**Checked EasyCrypt statement.**

```
op polyveck_unit (md : mode) (xs : polyveck) : bool =  
  polyveck_wf md xs /\ all poly_unit xs.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.38** (EasyCrypt line 86). **EasyCrypt name.**

polyvecl\_unit

**Checked EasyCrypt statement.**

```
op polyvecl_unit (md : mode) (xs : polyvecl) : bool =  
  polyvecl_wf md xs /\ all poly_unit xs.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.39** (EasyCrypt line 108). **EasyCrypt name.**

coeff\_mod

**Checked EasyCrypt statement.**

```
op coeff_mod (x : coeff) : coeff = x %% q.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.40** (EasyCrypt line 109). **EasyCrypt name.**

```
coeff_add
```

**Checked EasyCrypt statement.**

```
op coeff_add (x y : coeff) : coeff = coeff_mod (x + y).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.41** (EasyCrypt line 110). **EasyCrypt name.**

```
coeff_neg
```

**Checked EasyCrypt statement.**

```
op coeff_neg (x : coeff) : coeff = coeff_mod (-x).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.42** (EasyCrypt line 111). **EasyCrypt name.**

```
coeff_sub
```

**Checked EasyCrypt statement.**

```
op coeff_sub (x y : coeff) : coeff = coeff_add x (coeff_neg y).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.43** (EasyCrypt line 112). **EasyCrypt name.**

```
coeff_mul
```

**Checked EasyCrypt statement.**

```
op coeff_mul (x y : coeff) : coeff = coeff_mod (x * y).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.44** (EasyCrypt line 113). **EasyCrypt name.**

```
coeff_double
```

**Checked EasyCrypt statement.**

```
op coeff_double (x : coeff) : coeff = coeff_add x x.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.45** (EasyCrypt line 115). **EasyCrypt name.**

```
poly_coeff
```

**Checked EasyCrypt statement.**

```
op poly_coeff (p : poly) (i : int) : coeff = nth 0 p i.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.46** (EasyCrypt line 116). **EasyCrypt name.**

```
poly_normalize
```

**Checked EasyCrypt statement.**

```
op poly_normalize (p : poly) : poly =  
  mkseq (fun i => coeff_mod (poly_coeff p i)) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.47** (EasyCrypt line 118). **EasyCrypt name.**

```
poly_add
```

**Checked EasyCrypt statement.**

```
op poly_add (p r : poly) : poly =  
  if p = poly_zero /\ r = poly_zero then poly_zero  
  else mkseq  
    (fun i => coeff_add (poly_coeff p i) (poly_coeff r i)) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.48** (EasyCrypt line 122). **EasyCrypt name.**

```
poly_neg
```

**Checked EasyCrypt statement.**

```
op poly_neg (p : poly) : poly =  
  if p = poly_zero then poly_zero  
  else mkseq (fun i => coeff_neg (poly_coeff p i)) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.49** (EasyCrypt line 125). **EasyCrypt name.**

```
poly_sub
```

**Checked EasyCrypt statement.**

```
op poly_sub (p r : poly) : poly = poly_add p (poly_neg r).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.50** (EasyCrypt line 126). **EasyCrypt name.**

```
poly_double
```

**Checked EasyCrypt statement.**

```
op poly_double (p : poly) : poly =
  if p = poly_zero then poly_zero
  else mkseq (fun i => coeff_double (poly_coeff p i)) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.51** (EasyCrypt line 130). **EasyCrypt name.**

```
poly_mul_term
```

**Checked EasyCrypt statement.**

```
op poly_mul_term (p r : poly) (i j : int) : coeff =
  if j <= i then poly_coeff p j * poly_coeff r (i - j)
  else - (poly_coeff p j * poly_coeff r (n + i - j)).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 13.52** (EasyCrypt line 133). **EasyCrypt name.**

```
poly_mul
```

**Checked EasyCrypt statement.**

```
op poly_mul (p r : poly) : poly =
  if p = poly_zero \/\ r = poly_zero then poly_zero
  else mkseq
    (fun i =>
      coeff_mod
        (sumz (map (fun j => poly_mul_term p r i j) (range 0 n)))) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.53** (EasyCrypt line 140). **EasyCrypt name.**

```
poly_dot
```

**Checked EasyCrypt statement.**

```
op poly_dot (row : poly list) (v : poly list) : poly =
  if row = [] \/\ v = [] then poly_zero
  else foldl
    (fun acc (xy : poly * poly) => poly_add acc (poly_mul xy.'1 xy.'2))
    poly_zero
    (zip row v).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.54** (EasyCrypt line 147). **EasyCrypt name.**

```
matrix_vec_mul
```

**Checked EasyCrypt statement.**

```
op matrix_vec_mul (md : mode) (a : matrix) (v : polyvec1) : polyveck =
  if a = [] \/\ a = matrix_zero md then polyveck_zero md
  else mkseq (fun i => poly_dot (nth [] a i) v) (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.55** (EasyCrypt line 151). **EasyCrypt name.**

```
polyveck_add
```

**Checked EasyCrypt statement.**

```
op polyveck_add (md : mode) (xs ys : polyveck) : polyveck =
  if xs = polyveck_zero md /\ ys = polyveck_zero md then polyveck_zero md
  else mkseq
    (fun i => poly_add (nth poly_zero xs i) (nth poly_zero ys i))
    (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.56** (EasyCrypt line 156). **EasyCrypt name.**

```
polyveck_neg
```

**Checked EasyCrypt statement.**

```

op polyveck_neg (md : mode) (xs : polyveck) : polyveck =
  if xs = polyveck_zero md then polyveck_zero md
  else mkseq (fun i => poly_neg (nth poly_zero xs i)) (mode_k md).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.57** (EasyCrypt line 159). **EasyCrypt name.**

```
polyveck_sub
```

**Checked EasyCrypt statement.**

```

op polyveck_sub (md : mode) (xs ys : polyveck) : polyveck =
  polyveck_add md xs (polyveck_neg md ys).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.58** (EasyCrypt line 161). **EasyCrypt name.**

```
polyveck_double
```

**Checked EasyCrypt statement.**

```

op polyveck_double (md : mode) (xs : polyveck) : polyveck =
  if xs = polyveck_zero md then polyveck_zero md
  else mkseq (fun i => poly_double (nth poly_zero xs i)) (mode_k md).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.59** (EasyCrypt line 165). **EasyCrypt name.**

```
polyvecl_sub
```

**Checked EasyCrypt statement.**

```

op polyvecl_sub (md : mode) (xs ys : polyvecl) : polyvecl =
  mkseq
    (fun i => poly_sub (nth poly_zero xs i) (nth poly_zero ys i))
    (mode_l md).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.60** (EasyCrypt line 291). **EasyCrypt name.**

```
coeff_q_bound
```

**Checked EasyCrypt statement.**

```
op coeff_q_bound (x : coeff) : bool = 0 <= x < q.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 13.61** (EasyCrypt line 292). **EasyCrypt name.**

```
poly_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
op poly_coeffs_q_bound (p : poly) : bool =
  forall i, 0 <= i < n => coeff_q_bound (poly_coeff p i).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 13.62** (EasyCrypt line 294). **EasyCrypt name.**

```
polyveck_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
op polyveck_coeffs_q_bound (md : mode) (xs : polyveck) : bool =
  forall row, 0 <= row < mode_k md =>
    poly_coeffs_q_bound (nth poly_zero xs row).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 13.63** (EasyCrypt line 427). **EasyCrypt name.**

```
z1_alpha
```

**Checked EasyCrypt statement.**

```
op z1_alpha : int = 256.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.64** (EasyCrypt line 429). **EasyCrypt name.**

```
coeff_decompose_z1_low
```

**Checked EasyCrypt statement.**

```

op coeff_decompose_z1_low (x : coeff) : coeff =
  let lb = x %% z1_alpha in
  if z1_alpha %/ 2 <= lb then lb - z1_alpha else lb.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.65** (EasyCrypt line 432). **EasyCrypt name.**

```
coeff_decompose_z1_high
```

**Checked EasyCrypt statement.**

```

op coeff_decompose_z1_high (x : coeff) : coeff =
  (x + (z1_alpha %/ 2)) %/ z1_alpha.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.66** (EasyCrypt line 434). **EasyCrypt name.**

```
coeff_lsb
```

**Checked EasyCrypt statement.**

```
op coeff_lsb (x : coeff) : coeff = x %% 2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.67** (EasyCrypt line 435). **EasyCrypt name.**

```
coeff_decompose_vk_low
```

**Checked EasyCrypt statement.**

```

op coeff_decompose_vk_low (x : coeff) : coeff =
  if x %% 2 = 0 then 0
  else if (x %/ 2) %% 2 = 0 then 1
  else -1.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.68** (EasyCrypt line 439). **EasyCrypt name.**

```
coeff_decompose_vk_high
```

**Checked EasyCrypt statement.**



```
op coeff_decompose_vk_high (x : coeff) : coeff =
  (x - coeff_decompose_vk_low x) %/ 2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.69** (EasyCrypt line 441). **EasyCrypt name.**

```
coeff_decompose_hint_high
```

**Checked EasyCrypt statement.**

```
op coeff_decompose_hint_high (md : mode) (x : coeff) : coeff =
  let alpha = mode_alpha_hint md in
  let hb = (x + (alpha %/ 2)) %/ alpha in
  let top = (dq - 2) %/ alpha in
  if top <= hb then hb - top else hb.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.70** (EasyCrypt line 446). **EasyCrypt name.**

```
coeff_compose_z1
```

**Checked EasyCrypt statement.**

```
op coeff_compose_z1 (hi lo : coeff) : coeff = hi * z1_alpha + lo.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.71** (EasyCrypt line 448). **EasyCrypt name.**

```
poly_highbits
```

**Checked EasyCrypt statement.**

```
op poly_highbits (p : poly) : poly =
  mkseq (fun i => coeff_decompose_z1_high (poly_coeff p i)) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.72** (EasyCrypt line 450). **EasyCrypt name.**

```
poly_lowbits
```

**Checked EasyCrypt statement.**

```
op poly_lowbits (p : poly) : poly =
  mkseq (fun i => coeff_decompose_z1_low (poly_coeff p i)) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.73** (EasyCrypt line 452). **EasyCrypt name.**

```
poly_lsb
```

**Checked EasyCrypt statement.**

```
op poly_lsb (p : poly) : poly =
  mkseq (fun i => coeff_lsb (poly_coeff p i)) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.74** (EasyCrypt line 454). **EasyCrypt name.**

```
poly_vk_lowbits
```

**Checked EasyCrypt statement.**

```
op poly_vk_lowbits (p : poly) : poly =
  mkseq (fun i => coeff_decompose_vk_low (poly_coeff p i)) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.75** (EasyCrypt line 456). **EasyCrypt name.**

```
poly_vk_highbits
```

**Checked EasyCrypt statement.**

```
op poly_vk_highbits (p : poly) : poly =
  mkseq (fun i => coeff_decompose_vk_high (poly_coeff p i)) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.76** (EasyCrypt line 458). **EasyCrypt name.**

```
poly_compose_z1
```

**Checked EasyCrypt statement.**

```
op poly_compose_z1 (hi lo : poly) : poly =
  mkseq
```

```
(fun i =>
  coeff_compose_z1 (poly_coeff hi i) (poly_coeff lo i)) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.77** (EasyCrypt line 462). **EasyCrypt name.**

```
poly_hint_highbits
```

**Checked EasyCrypt statement.**

```
op poly_hint_highbits (md : mode) (p : poly) : poly =
  mkseq (fun i => coeff_decompose_hint_high md (poly_coeff p i)) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.78** (EasyCrypt line 465). **EasyCrypt name.**

```
polyvecl_highbits
```

**Checked EasyCrypt statement.**

```
op polyvecl_highbits (md : mode) (xs : polyvecl) : polyvecl =
  mkseq (fun i => poly_highbits (nth poly_zero xs i)) (mode_l md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.79** (EasyCrypt line 467). **EasyCrypt name.**

```
polyvecl_lowbits
```

**Checked EasyCrypt statement.**

```
op polyvecl_lowbits (md : mode) (xs : polyvecl) : polyvecl =
  mkseq (fun i => poly_lowbits (nth poly_zero xs i)) (mode_l md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.80** (EasyCrypt line 469). **EasyCrypt name.**

```
polyveck_highbits_hint
```

**Checked EasyCrypt statement.**

```
op polyveck_highbits_hint (md : mode) (xs : polyveck) : polyveck =
  mkseq (fun i => poly_hint_highbits md (nth poly_zero xs i)) (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.81** (EasyCrypt line 471). **EasyCrypt name.**

```
polyveck_vk_lowbits
```

**Checked EasyCrypt statement.**

```
op polyveck_vk_lowbits (md : mode) (xs : polyveck) : polyveck =
  mkseq (fun i => poly_vk_lowbits (nth poly_zero xs i)) (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.82** (EasyCrypt line 473). **EasyCrypt name.**

```
polyveck_vk_highbits
```

**Checked EasyCrypt statement.**

```
op polyveck_vk_highbits (md : mode) (xs : polyveck) : polyveck =
  mkseq (fun i => poly_vk_highbits (nth poly_zero xs i)) (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.83** (EasyCrypt line 475). **EasyCrypt name.**

```
polyveck_mul_alpha
```

**Checked EasyCrypt statement.**

```
op polyveck_mul_alpha (md : mode) (xs : polyveck) : polyveck =
  mkseq
    (fun i =>
      mkseq
        (fun j => mode_alpha_hint md * poly_coeff (nth poly_zero xs i) j)
        n)
    (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.84** (EasyCrypt line 564). **EasyCrypt name.**

```
public_key_seed_poly
```

**Checked EasyCrypt statement.**

```
op public_key_seed_poly (tag : int) (src : byte list) : poly =
  mkseq (fun i => coeff_mod (tag + i + nth 0 src i)) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.85** (EasyCrypt line 566). **EasyCrypt name.**

```
public_key_seed_polyvec1
```

**Checked EasyCrypt statement.**

```
op public_key_seed_polyvec1
  (md : mode) (tag : int) (src : byte list) : polyvec1 =
  mkseq
    (fun i => public_key_seed_poly (tag + i) (i :: src))
    (mode_l md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.86** (EasyCrypt line 571). **EasyCrypt name.**

```
public_key_seed_polyveck
```

**Checked EasyCrypt statement.**

```
op public_key_seed_polyveck
  (md : mode) (tag : int) (src : byte list) : polyveck =
  mkseq
    (fun i => public_key_seed_poly (tag + i) (i :: src))
    (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.87** (EasyCrypt line 576). **EasyCrypt name.**

```
haetae_keygen_seedbuf_bytes
```

**Checked EasyCrypt statement.**

```
op haetae_keygen_seedbuf_bytes : int = 2 * seedbytes + crhbytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.88** (EasyCrypt line 577). **EasyCrypt name.**

```
haetae_reference_keygen_seedbuf_bytes
```

**Checked EasyCrypt statement.**

```
op haetae_reference_keygen_seedbuf_bytes : int =
  2 * seedbytes + crhbytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.89** (EasyCrypt line 579). **EasyCrypt name.**

```
haetae_reference_keygen_xof_absorb_bytes
```

**Checked EasyCrypt statement.**

```
op haetae_reference_keygen_xof_absorb_bytes : int = seedbytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.90** (EasyCrypt line 580). **EasyCrypt name.**

```
haetae_reference_keygen_xof_squeeze_bytes
```

**Checked EasyCrypt statement.**

```
op haetae_reference_keygen_xof_squeeze_bytes : int =  
  haetae_reference_keygen_seedbuf_bytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.91** (EasyCrypt line 582). **EasyCrypt name.**

```
haetae_keygen_rhoprime_offset
```

**Checked EasyCrypt statement.**

```
op haetae_keygen_rhoprime_offset : int = 0.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.92** (EasyCrypt line 583). **EasyCrypt name.**

```
haetae_keygen_sigma_offset
```

**Checked EasyCrypt statement.**

```
op haetae_keygen_sigma_offset : int = seedbytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.93** (EasyCrypt line 584). **EasyCrypt name.**

```
haetae_keygen_key_offset
```

**Checked EasyCrypt statement.**

```
op haetae_keygen_key_offset : int = seedbytes + crhbytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.94** (EasyCrypt line 585). **EasyCrypt name.**

```
haetae_shake256_rate_bytes
```

**Checked EasyCrypt statement.**

```
op haetae_shake256_rate_bytes : int = shake256_rate_bytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.95** (EasyCrypt line 586). **EasyCrypt name.**

```
haetae_shake256_domain_separator
```

**Checked EasyCrypt statement.**

```
op haetae_shake256_domain_separator : byte = shake256_domain_separator.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.96** (EasyCrypt line 587). **EasyCrypt name.**

```
haetae_shake256_bytes
```

**Checked EasyCrypt statement.**

```
op haetae_shake256_bytes (input : byte list) (outlen : int) : byte list =  
  shake256 input (size input) outlen.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.97** (EasyCrypt line 589). **EasyCrypt name.**

```
haetae_seed_slice
```

**Checked EasyCrypt statement.**

```
op haetae_seed_slice (src : byte list) (offset len : int) : byte list =
  mkseq (fun i => nth 0 src (offset + i)) len.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.98** (EasyCrypt line 591). **EasyCrypt name.**

```
haetae_xof256_absorb_input
```

**Checked EasyCrypt statement.**

```
op haetae_xof256_absorb_input
  (input : byte list) (inlen : int) : byte list =
  haetae_seed_slice input 0 inlen.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.99** (EasyCrypt line 594). **EasyCrypt name.**

```
haetae_xof256_absorb_once
```

**Checked EasyCrypt statement.**

```
op haetae_xof256_absorb_once
  (input : byte list) (inlen : int) : xof256_state =
  shake256_absorb_once (haetae_xof256_absorb_input input inlen) inlen.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.100** (EasyCrypt line 597). **EasyCrypt name.**

```
haetae_xof256_squeeze
```

**Checked EasyCrypt statement.**

```
op haetae_xof256_squeeze
  (st : xof256_state) (outlen : int) : byte list =
  shake256_squeeze st outlen.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.101** (EasyCrypt line 600). **EasyCrypt name.**

```
haetae_xof256_absorb_once_squeeze
```

**Checked EasyCrypt statement.**



```

op haetae_xof256_absorb_once_squeeze
  (input : byte list) (inlen outlen : int) : byte list =
  haetae_xof256_squeeze
    (haetae_xof256_absorb_once input inlen)
    outlen.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.102** (EasyCrypt line 605). **EasyCrypt name.**

```

haetae_reference_keygen_seedbuf_after_memcpy

```

**Checked EasyCrypt statement.**

```

op haetae_reference_keygen_seedbuf_after_memcpy (sd : seed) : byte list =
  mkseq
    (fun i => if i < seedbytes then nth 0 sd i else 0)
  haetae_reference_keygen_seedbuf_bytes.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.103** (EasyCrypt line 609). **EasyCrypt name.**

```

haetae_reference_keygen_xof_input

```

**Checked EasyCrypt statement.**

```

op haetae_reference_keygen_xof_input (sd : seed) : byte list =
  haetae_seed_slice
    (haetae_reference_keygen_seedbuf_after_memcpy sd)
    0 haetae_reference_keygen_xof_absorb_bytes.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.104** (EasyCrypt line 613). **EasyCrypt name.**

```

haetae_keygen_xof_seedbuf

```

**Checked EasyCrypt statement.**

```

op haetae_keygen_xof_seedbuf (sd : seed) : byte list =
  haetae_xof256_absorb_once_squeeze
    (haetae_reference_keygen_seedbuf_after_memcpy sd)
  haetae_reference_keygen_xof_absorb_bytes
  haetae_reference_keygen_xof_squeeze_bytes.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.105** (EasyCrypt line 618). **EasyCrypt name.**

```
haetae_reference_keygen_xof256_seedbuf
```

**Checked EasyCrypt statement.**

```
op haetae_reference_keygen_xof256_seedbuf (sd : seed) : byte list =  
  haetae_keygen_xof_seedbuf sd.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.106** (EasyCrypt line 620). **EasyCrypt name.**

```
haetae_keygen_seedbuf_rhoprime
```

**Checked EasyCrypt statement.**

```
op haetae_keygen_seedbuf_rhoprime (seedbuf : byte list) : seed =  
  haetae_seed_slice seedbuf haetae_keygen_rhoprime_offset seedbytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.107** (EasyCrypt line 622). **EasyCrypt name.**

```
haetae_keygen_seedbuf_sigma
```

**Checked EasyCrypt statement.**

```
op haetae_keygen_seedbuf_sigma (seedbuf : byte list) : crh =  
  haetae_seed_slice seedbuf haetae_keygen_sigma_offset crhbytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.108** (EasyCrypt line 624). **EasyCrypt name.**

```
haetae_keygen_seedbuf_key
```

**Checked EasyCrypt statement.**

```
op haetae_keygen_seedbuf_key (seedbuf : byte list) : seed =  
  haetae_seed_slice seedbuf haetae_keygen_key_offset seedbytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.109** (EasyCrypt line 626). **EasyCrypt name.**

```
haetae_keygen_seedbuf_layout_wf
```

### Checked EasyCrypt statement.

```
op haetae_keygen_seedbuf_layout_wf (seedbuf : byte list) : bool =
  size seedbuf = haetae_reference_keygen_seedbuf_bytes /\
  haetae_keygen_rho_prime_offset = 0 /\
  haetae_keygen_sigma_offset = seedbytes /\
  haetae_keygen_key_offset = seedbytes + crhbytes /\
  size (haetae_keygen_seedbuf_rho_prime seedbuf) = seedbytes /\
  size (haetae_keygen_seedbuf_sigma seedbuf) = crhbytes /\
  size (haetae_keygen_seedbuf_key seedbuf) = seedbytes.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

### Definition 13.110 (EasyCrypt line 634). EasyCrypt name.

```
haetae_reference_keygen_xof_call_wf
```

### Checked EasyCrypt statement.

```
op haetae_reference_keygen_xof_call_wf
  (sd : seed) (seedbuf : byte list) : bool =
  size sd = haetae_reference_keygen_xof_absorb_bytes /\
  size (haetae_reference_keygen_seedbuf_after_memcpy sd) =
    haetae_reference_keygen_seedbuf_bytes /\
  haetae_xof256_absorb_input
    (haetae_reference_keygen_seedbuf_after_memcpy sd)
    haetae_reference_keygen_xof_absorb_bytes =
    haetae_reference_keygen_xof_input sd /\
  shake256_absorb_once_short_domain
    (haetae_reference_keygen_xof_input sd)
    haetae_reference_keygen_xof_absorb_bytes /\
  haetae_reference_keygen_xof_squeeze_bytes < haetae_shake256_rate_bytes /\
  haetae_xof256_absorb_once
    (haetae_reference_keygen_seedbuf_after_memcpy sd)
    haetae_reference_keygen_xof_absorb_bytes =
    shake256_absorb_once
      (haetae_reference_keygen_xof_input sd)
      haetae_reference_keygen_xof_absorb_bytes /\
  seedbuf =
    shake256_squeeze
      (haetae_xof256_absorb_once
        (haetae_reference_keygen_seedbuf_after_memcpy sd)
        haetae_reference_keygen_xof_absorb_bytes)
      haetae_reference_keygen_xof_squeeze_bytes /\
  size seedbuf = haetae_reference_keygen_xof_squeeze_bytes /\
  haetae_reference_keygen_xof_absorb_bytes = seedbytes /\
  haetae_reference_keygen_xof_squeeze_bytes =
    haetae_reference_keygen_seedbuf_bytes.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

### Definition 13.111 (EasyCrypt line 663). EasyCrypt name.

```
haetae_keygen_rho_prime
```

### Checked EasyCrypt statement.

```
op haetae_keygen_rhoprime (sd : seed) : seed =  
  haetae_keygen_seedbuf_rhoprime  
    (haetae_reference_keygen_xof256_seedbuf sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 13.112 (EasyCrypt line 666). EasyCrypt name.

```
haetae_keygen_sigma
```

### Checked EasyCrypt statement.

```
op haetae_keygen_sigma (sd : seed) : crh =  
  haetae_keygen_seedbuf_sigma  
    (haetae_reference_keygen_xof256_seedbuf sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 13.113 (EasyCrypt line 669). EasyCrypt name.

```
haetae_keygen_key
```

### Checked EasyCrypt statement.

```
op haetae_keygen_key (sd : seed) : seed =  
  haetae_keygen_seedbuf_key  
    (haetae_reference_keygen_xof256_seedbuf sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 13.114 (EasyCrypt line 672). EasyCrypt name.

```
haetae_keygen_counter_next
```

### Checked EasyCrypt statement.

```
op haetae_keygen_counter_next (md : mode) (counter : int) : int =  
  counter + mode_m md + mode_k md.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 13.115 (EasyCrypt line 674). EasyCrypt name.

```
haetae_keygen_first_accept_counter
```

### Checked EasyCrypt statement.

```
op haetae_keygen_first_accept_counter (_ : mode) (_ : seed) : int = 0.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 13.116 (EasyCrypt line 675). EasyCrypt name.

```
haetae_keygen_sampler_seed
```

### Checked EasyCrypt statement.

```
op haetae_keygen_sampler_seed (sigma : crh) (counter : int) : seed =
  mkseq
    (fun i => nth 0 sigma (i %% crhbytes) + counter + i)
  seedbytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

### Definition 13.117 (EasyCrypt line 680). EasyCrypt name.

```
public_matrix_seed
```

### Checked EasyCrypt statement.

```
op public_matrix_seed (md : mode) (sd : seed) : matrix =
  mkseq
    (fun i => public_key_seed_polyvec1 md (101 + i) (i :: sd))
  (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 13.118 (EasyCrypt line 684). EasyCrypt name.

```
public_secret_vector_seed
```

### Checked EasyCrypt statement.

```
op public_secret_vector_seed (md : mode) (sd : seed) : polyvec1 =
  public_key_seed_polyvec1 md 211 sd.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 13.119 (EasyCrypt line 686). EasyCrypt name.

```
public_error_vector_seed
```

### Checked EasyCrypt statement.

```
op public_error_vector_seed (md : mode) (sd : seed) : polyveck =  
  public_key_seed_polyveck md 307 sd.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.120** (EasyCrypt line 688). **EasyCrypt name.**

```
public_rounding_vector_seed
```

### Checked EasyCrypt statement.

```
op public_rounding_vector_seed (md : mode) (sd : seed) : polyveck =  
  public_key_seed_polyveck md 401 sd.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.121** (EasyCrypt line 691). **EasyCrypt name.**

```
haetae_keygen_a0_matrix
```

### Checked EasyCrypt statement.

```
op haetae_keygen_a0_matrix (md : mode) (sd : seed) : matrix =  
  public_matrix_seed md sd.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.122** (EasyCrypt line 693). **EasyCrypt name.**

```
haetae_keygen_secret_vector
```

### Checked EasyCrypt statement.

```
op haetae_keygen_secret_vector (md : mode) (sd : seed) : polyvecl =  
  public_secret_vector_seed md sd.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.123** (EasyCrypt line 695). **EasyCrypt name.**

```
haetae_keygen_error_vector
```

### Checked EasyCrypt statement.

```
op haetae_keygen_error_vector (md : mode) (sd : seed) : polyveck =  
  public_error_vector_seed md sd.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.124** (EasyCrypt line 697). **EasyCrypt name.**

```
haetae_keygen_raw_public_vector
```

**Checked EasyCrypt statement.**

```
op haetae_keygen_raw_public_vector (md : mode) (sd : seed) : polyveck =
  polyveck_add md
    (matrix_vec_mul md
      (haetae_keygen_a0_matrix md sd)
      (haetae_keygen_secret_vector md sd))
    (haetae_keygen_error_vector md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.125** (EasyCrypt line 703). **EasyCrypt name.**

```
haetae_keygen_mode23_predecompose_vector
```

**Checked EasyCrypt statement.**

```
op haetae_keygen_mode23_predecompose_vector
  (md : mode) (sd : seed) : polyveck =
  polyveck_add md
    (public_rounding_vector_seed md sd)
    (haetae_keygen_raw_public_vector md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.126** (EasyCrypt line 708). **EasyCrypt name.**

```
haetae_keygen_mode23_public_vector
```

**Checked EasyCrypt statement.**

```
op haetae_keygen_mode23_public_vector
  (md : mode) (sd : seed) : polyveck =
  polyveck_vk_highbits md (haetae_keygen_mode23_predecompose_vector md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.127** (EasyCrypt line 711). **EasyCrypt name.**

```
haetae_keygen_mode23_rounding_lowbits
```

**Checked EasyCrypt statement.**

```

op haetae_keygen_mode23_rounding_lowbits
  (md : mode) (sd : seed) : polyveck =
  polyveck_vk_lowbits md (haetae_keygen_mode23_predecompose_vector md sd).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.128** (EasyCrypt line 714). **EasyCrypt name.**

```

haetae_keygen_mode23_adjusted_error_vector

```

**Checked EasyCrypt statement.**

```

op haetae_keygen_mode23_adjusted_error_vector
  (md : mode) (sd : seed) : polyveck =
  polyveck_sub md
    (haetae_keygen_error_vector md sd)
    (haetae_keygen_mode23_rounding_lowbits md sd).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.129** (EasyCrypt line 719). **EasyCrypt name.**

```

haetae_keygen_mode5_public_vector

```

**Checked EasyCrypt statement.**

```

op haetae_keygen_mode5_public_vector
  (md : mode) (sd : seed) : polyveck =
  polyveck_neg md (polyveck_double md (haetae_keygen_raw_public_vector md sd)).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.130** (EasyCrypt line 722). **EasyCrypt name.**

```

haetae_public_key_vector_from_relation

```

**Checked EasyCrypt statement.**

```

op haetae_public_key_vector_from_relation
  (md : mode) (sd : seed) (s : polyvecl) (e : polyveck) : polyveck =
  with md = Mode2 =>
    let raw =
      polyveck_add md
        (matrix_vec_mul md (public_matrix_seed md sd) s)
    e in
  polyveck_vk_highbits md
    (polyveck_add md (public_rounding_vector_seed md sd) raw)
  with md = Mode3 =>
    let raw =
      polyveck_add md
        (matrix_vec_mul md (public_matrix_seed md sd) s)
    e in

```



```

polyveck_vk_highbits md
  (polyveck_add md (public_rounding_vector_seed md sd) raw)
with md = Mode5 =>
  let raw =
    polyveck_add md
      (matrix_vec_mul md (public_matrix_seed md sd) s)
    e in
  polyveck_neg md (polyveck_double md raw).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.131** (EasyCrypt line 744). **EasyCrypt name.**

```
haetae_keygen_public_vector
```

**Checked EasyCrypt statement.**

```

op haetae_keygen_public_vector (md : mode) (sd : seed) : polyveck =
  haetae_public_key_vector_from_relation md sd
    (haetae_keygen_secret_vector md sd)
    (haetae_keygen_error_vector md sd).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.132** (EasyCrypt line 748). **EasyCrypt name.**

```
public_key_vector_seed
```

**Checked EasyCrypt statement.**

```

op public_key_vector_seed (md : mode) (sd : seed) : polyveck =
  haetae_keygen_public_vector md sd.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.133** (EasyCrypt line 750). **EasyCrypt name.**

```
public_key_relation
```

**Checked EasyCrypt statement.**

```

op public_key_relation
  (md : mode) (pk : pkey) (s : polyvecl) (e : polyveck) : bool =
  pk.'2 = haetae_public_key_vector_from_relation md pk.'1 s e.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.134** (EasyCrypt line 753). **EasyCrypt name.**

```
public_key_equation_holds
```

**Checked EasyCrypt statement.**

```
op public_key_equation_holds (md : mode) (pk : pkey) : bool =
  public_key_relation md pk
    (public_secret_vector_seed md pk.'1)
    (public_error_vector_seed md pk.'1).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.135** (EasyCrypt line 757). **EasyCrypt name.**

```
haetae_reference_polymatkm_expand_matA
```

**Checked EasyCrypt statement.**

```
op haetae_reference_polymatkm_expand_matA
  (md : mode) (rho_prime : seed) : matrix =
  public_matrix_seed md rho_prime.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.136** (EasyCrypt line 760). **EasyCrypt name.**

```
haetae_reference_polyveck_expand_vecA
```

**Checked EasyCrypt statement.**

```
op haetae_reference_polyveck_expand_vecA
  (md : mode) (rho_prime : seed) : polyveck =
  public_rounding_vector_seed md rho_prime.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.137** (EasyCrypt line 763). **EasyCrypt name.**

```
haetae_reference_polyvecmk_expand_S_s1
```

**Checked EasyCrypt statement.**

```
op haetae_reference_polyvecmk_expand_S_s1
  (md : mode) (sigma : crh) (counter : int) : polyveck =
  public_secret_vector_seed md (haetae_keygen_sampler_seed sigma counter).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.138** (EasyCrypt line 766). **EasyCrypt name.**

```
haetae_reference_polyvecmk_expand_S_s2
```

**Checked EasyCrypt statement.**

```
op haetae_reference_polyvecmk_expand_S_s2
  (md : mode) (sigma : crh) (counter : int) : polyveck =
  public_error_vector_seed md (haetae_keygen_sampler_seed sigma counter).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.139** (EasyCrypt line 769). **EasyCrypt name.**

```
haetae_reference_keygen_secret_vector
```

**Checked EasyCrypt statement.**

```
op haetae_reference_keygen_secret_vector
  (md : mode) (sd : seed) : polyvecl =
  haetae_reference_polyvecmk_expand_S_s1 md
  (haetae_keygen_sigma sd)
  (haetae_keygen_first_accept_counter md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.140** (EasyCrypt line 774). **EasyCrypt name.**

```
haetae_reference_keygen_error_vector
```

**Checked EasyCrypt statement.**

```
op haetae_reference_keygen_error_vector
  (md : mode) (sd : seed) : polyveck =
  haetae_reference_polyvecmk_expand_S_s2 md
  (haetae_keygen_sigma sd)
  (haetae_keygen_first_accept_counter md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.141** (EasyCrypt line 779). **EasyCrypt name.**

```
haetae_reference_keygen_a0_matrix
```

**Checked EasyCrypt statement.**

```
op haetae_reference_keygen_a0_matrix
  (md : mode) (sd : seed) : matrix =
  haetae_reference_polymatkm_expand_matA md (haetae_keygen_rho_prime sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.142** (EasyCrypt line 782). **EasyCrypt name.**

```
haetae_reference_keygen_raw_public_vector
```

**Checked EasyCrypt statement.**

```
op haetae_reference_keygen_raw_public_vector
  (md : mode) (sd : seed) : polyveck =
  polyveck_add md
    (matrix_vec_mul md
      (haetae_reference_keygen_a0_matrix md sd)
      (haetae_reference_keygen_secret_vector md sd))
    (haetae_reference_keygen_error_vector md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.143** (EasyCrypt line 789). **EasyCrypt name.**

```
haetae_reference_keygen_mode23_predecompose_vector
```

**Checked EasyCrypt statement.**

```
op haetae_reference_keygen_mode23_predecompose_vector
  (md : mode) (sd : seed) : polyveck =
  polyveck_add md
    (haetae_reference_polyveck_expand_vecA md (haetae_keygen_rhoprime sd))
    (haetae_reference_keygen_raw_public_vector md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.144** (EasyCrypt line 794). **EasyCrypt name.**

```
haetae_reference_keygen_mode23_public_vector
```

**Checked EasyCrypt statement.**

```
op haetae_reference_keygen_mode23_public_vector
  (md : mode) (sd : seed) : polyveck =
  polyveck_vk_highbits md
    (haetae_reference_keygen_mode23_predecompose_vector md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.145** (EasyCrypt line 798). **EasyCrypt name.**

```
haetae_reference_keygen_mode23_rounding_lowbits
```

### Checked EasyCrypt statement.

```
op haetae_reference_keygen_mode23_rounding_lowbits
  (md : mode) (sd : seed) : polyveck =
  polyveck_vk_lowbits md
  (haetae_reference_keygen_mode23_predecompose_vector md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 13.146 (EasyCrypt line 802). EasyCrypt name.

```
haetae_reference_keygen_mode23_adjusted_error_vector
```

### Checked EasyCrypt statement.

```
op haetae_reference_keygen_mode23_adjusted_error_vector
  (md : mode) (sd : seed) : polyveck =
  polyveck_sub md
  (haetae_reference_keygen_error_vector md sd)
  (haetae_reference_keygen_mode23_rounding_lowbits md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 13.147 (EasyCrypt line 807). EasyCrypt name.

```
haetae_reference_keygen_mode5_public_vector
```

### Checked EasyCrypt statement.

```
op haetae_reference_keygen_mode5_public_vector
  (md : mode) (sd : seed) : polyveck =
  polyveck_neg md
  (polyveck_double md (haetae_reference_keygen_raw_public_vector md sd)).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

### Definition 13.148 (EasyCrypt line 811). EasyCrypt name.

```
haetae_reference_keygen_public_vector
```

### Checked EasyCrypt statement.

```
op haetae_reference_keygen_public_vector
  (md : mode) (sd : seed) : polyveck =
  haetae_public_key_vector_from_relation md
  (haetae_keygen_rho_prime sd)
  (haetae_reference_keygen_secret_vector md sd)
  (haetae_reference_keygen_error_vector md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.149** (EasyCrypt line 817). **EasyCrypt name.**

```
packed_haetae_reference_public_key
```

**Checked EasyCrypt statement.**

```
op packed_haetae_reference_public_key
  (md : mode) (sd : seed) : pkey =
  (haetae_keygen_rhoprime sd, haetae_reference_keygen_public_vector md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.150** (EasyCrypt line 820). **EasyCrypt name.**

```
haetae_reference_keygen_seed_flow_wf
```

**Checked EasyCrypt statement.**

```
op haetae_reference_keygen_seed_flow_wf (md : mode) (sd : seed) : bool =
  haetae_reference_keygen_a0_matrix md sd =
    haetae_reference_polymatkm_expand_matA md (haetae_keygen_rhoprime sd) /\
  haetae_reference_keygen_secret_vector md sd =
    haetae_reference_polyvecmk_expand_S_s1 md
      (haetae_keygen_sigma sd)
    (haetae_keygen_first_accept_counter md sd) /\
  haetae_reference_keygen_error_vector md sd =
    haetae_reference_polyvecmk_expand_S_s2 md
      (haetae_keygen_sigma sd)
    (haetae_keygen_first_accept_counter md sd) /\
  haetae_reference_keygen_public_vector md sd =
    haetae_public_key_vector_from_relation md
      (haetae_keygen_rhoprime sd)
      (haetae_reference_keygen_secret_vector md sd)
      (haetae_reference_keygen_error_vector md sd) /\
  (packed_haetae_reference_public_key md sd).‘1 =
    haetae_keygen_rhoprime sd /\
  (packed_haetae_reference_public_key md sd).‘2 =
    haetae_reference_keygen_public_vector md sd.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.151** (EasyCrypt line 841). **EasyCrypt name.**

```
haetae_mode23_qj_vector
```

**Checked EasyCrypt statement.**

```
op haetae_mode23_qj_vector (md : mode) (sd : seed) : polyveck =
  public_rounding_vector_seed md sd.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.152** (EasyCrypt line 843). **EasyCrypt name.**

```
haetae_mode23_verification_column
```

**Checked EasyCrypt statement.**

```
op haetae_mode23_verification_column
  (md : mode) (b qj : polyveck) : polyveck =
  polyveck_double md (polyveck_sub md qj (polyveck_double md b)).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.153** (EasyCrypt line 846). **EasyCrypt name.**

```
haetae_mode5_verification_column
```

**Checked EasyCrypt statement.**

```
op haetae_mode5_verification_column
  (md : mode) (b : polyveck) : polyveck =
  mkseq (fun i => poly_normalize (nth poly_zero b i)) (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.154** (EasyCrypt line 849). **EasyCrypt name.**

```
public_key_mode23_column
```

**Checked EasyCrypt statement.**

```
op public_key_mode23_column (md : mode) (pk : pkey) : polyveck =
  haetae_mode23_verification_column md pk.'2
  (haetae_mode23_qj_vector md pk.'1).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.155** (EasyCrypt line 852). **EasyCrypt name.**

```
public_key_mode5_column
```

**Checked EasyCrypt statement.**

```
op public_key_mode5_column (md : mode) (pk : pkey) : polyveck =
  haetae_mode5_verification_column md pk.'2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.156** (EasyCrypt line 854). **EasyCrypt name.**

```
public_key_verification_column
```

**Checked EasyCrypt statement.**

```
op public_key_verification_column (md : mode) (pk : pkey) : polyveck =
  with md = Mode2 => public_key_mode23_column md pk
  with md = Mode3 => public_key_mode23_column md pk
  with md = Mode5 => public_key_mode5_column md pk.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.157** (EasyCrypt line 858). **EasyCrypt name.**

```
haetae_verification_column_from_unpacked
```

**Checked EasyCrypt statement.**

```
op haetae_verification_column_from_unpacked
  (md : mode) (sd : seed) (b : polyveck) : polyveck =
  with md = Mode2 =>
    haetae_mode23_verification_column md b
    (haetae_mode23_qj_vector md sd)
  with md = Mode3 =>
    haetae_mode23_verification_column md b
    (haetae_mode23_qj_vector md sd)
  with md = Mode5 =>
    haetae_mode5_verification_column md b.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.158** (EasyCrypt line 868). **EasyCrypt name.**

```
haetae_verification_matrix_from_unpacked
```

**Checked EasyCrypt statement.**

```
op haetae_verification_matrix_from_unpacked
  (md : mode) (sd : seed) (b : polyveck) : matrix =
mkseq
  (fun i =>
    mkseq
      (fun j =>
        if j = 0
        then nth poly_zero
          (haetae_verification_column_from_unpacked md sd b) i
        else poly_double
          (nth poly_zero
            (nth [] (haetae_keygen_a0_matrix md sd) i)
            (j - 1)))
      (mode_l md))
  (mode_k md).
```



**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.159** (EasyCrypt line 883). **EasyCrypt name.**

```
packed_haetae_public_key
```

**Checked EasyCrypt statement.**

```
op packed_haetae_public_key (md : mode) (sd : seed) : pkey =
  (sd, haetae_keygen_public_vector md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.160** (EasyCrypt line 885). **EasyCrypt name.**

```
packed_haetae_verification_matrix
```

**Checked EasyCrypt statement.**

```
op packed_haetae_verification_matrix (md : mode) (pk : pkey) : matrix =
  haetae_verification_matrix_from_unpacked md pk.'1 pk.'2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.161** (EasyCrypt line 887). **EasyCrypt name.**

```
packed_haetae_keygen_verification_matrix
```

**Checked EasyCrypt statement.**

```
op packed_haetae_keygen_verification_matrix (md : mode) (sd : seed) :
  matrix =
    packed_haetae_verification_matrix md (packed_haetae_public_key md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.162** (EasyCrypt line 890). **EasyCrypt name.**

```
public_verification_matrix
```

**Checked EasyCrypt statement.**

```
op public_verification_matrix (md : mode) (pk : pkey) : matrix =
  packed_haetae_verification_matrix md pk.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.163** (EasyCrypt line 892). **EasyCrypt name.**

```
challenge_embedding
```

**Checked EasyCrypt statement.**

```
op challenge_embedding (md : mode) (ch : challenge) : polyvec1 =
  mkseq (fun i => if i = 0 then ch else poly_zero) (mode_l md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.164** (EasyCrypt line 894). **EasyCrypt name.**

```
public_key_challenge_term
```

**Checked EasyCrypt statement.**

```
op public_key_challenge_term
  (md : mode) (pk : pkey) (ch : challenge) : polyveck =
  matrix_vec_mul md (public_verification_matrix md pk)
  (challenge_embedding md ch).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 13.165** (EasyCrypt line 898). **EasyCrypt name.**

```
reconstructed_highbits
```

**Checked EasyCrypt statement.**

```
op reconstructed_highbits
  (md : mode) (aux_pk_ch : polyveck) : polyveck =
  polyveck_add md aux_pk_ch.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.166** (EasyCrypt line 902). **EasyCrypt name.**

```
public_key_of_secret
```

**Checked EasyCrypt statement.**

```
op public_key_of_secret (md : mode) (sk : skey) : pkey =
  (sk.'1, public_key_vector_seed md sk.'1).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.167** (EasyCrypt line 904). **EasyCrypt name.**

```
haetae_public_key_body_bytes
```

**Checked EasyCrypt statement.**

```
op haetae_public_key_body_bytes (md : mode) : int =
  mode_k md * mode_polyq_packedbytes md.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.168** (EasyCrypt line 906). **EasyCrypt name.**

```
haetae_public_key_bytes
```

**Checked EasyCrypt statement.**

```
op haetae_public_key_bytes (md : mode) : int =
  mode_publickeybytes md.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.169** (EasyCrypt line 908). **EasyCrypt name.**

```
haetae_public_key_encoding_wf
```

**Checked EasyCrypt statement.**

```
op haetae_public_key_encoding_wf (md : mode) (enc : encoded_pkey) : bool =
  size enc = haetae_public_key_bytes md.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.170** (EasyCrypt line 910). **EasyCrypt name.**

```
haetae_public_key_unpacked_wf
```

**Checked EasyCrypt statement.**

```
op haetae_public_key_unpacked_wf (md : mode) (pk : pkey) : bool =
  size pk.'1 = seedbytes /\ polyveck_wf md pk.'2.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.171** (EasyCrypt line 912). **EasyCrypt name.**

```
haetae_pack_seed
```

**Checked EasyCrypt statement.**

```
op haetae_pack_seed (sd : seed) : byte list =
  mkseq (fun i => nth 0 sd i) seedbytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.172** (EasyCrypt line 914). **EasyCrypt name.**

```
haetae_unpack_seed
```

**Checked EasyCrypt statement.**

```
op haetae_unpack_seed (enc : encoded_pkey) : seed =
  mkseq (fun i => nth 0 enc i) seedbytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.173** (EasyCrypt line 916). **EasyCrypt name.**

```
haetae_pack_poly_q
```

**Checked EasyCrypt statement.**

```
op haetae_pack_poly_q (md : mode) (p : poly) : byte list =
  mkseq
    (fun i => if i < n then poly_coeff p i else 0)
    (mode_polyq_packedbytes md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.174** (EasyCrypt line 920). **EasyCrypt name.**

```
haetae_unpack_poly_q_at
```

**Checked EasyCrypt statement.**

```
op haetae_unpack_poly_q_at
  (md : mode) (enc : encoded_pkey) (offset : int) : poly =
  mkseq (fun i => nth 0 enc (offset + i)) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.175** (EasyCrypt line 924). **EasyCrypt name.**

```
haetae_byte_modulus
```

**Checked EasyCrypt statement.**

```
op haetae_byte_modulus : int = 256.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.176** (EasyCrypt line 925). **EasyCrypt name.**

```
haetae_polyq_mode23_bound
```

**Checked EasyCrypt statement.**

```
op haetae_polyq_mode23_bound : int = 32768.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 13.177** (EasyCrypt line 926). **EasyCrypt name.**

```
haetae_polyq_mode5_bound
```

**Checked EasyCrypt statement.**

```
op haetae_polyq_mode5_bound : int = 65536.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 13.178** (EasyCrypt line 928). **EasyCrypt name.**

```
haetae_byte_norm
```

**Checked EasyCrypt statement.**

```
op haetae_byte_norm (x : int) : byte = x %% haetae_byte_modulus.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.179** (EasyCrypt line 930). **EasyCrypt name.**

```
haetae_polyq_coeff_bound
```

### Checked EasyCrypt statement.

```

op haetae_polyq_coeff_bound (md : mode) : int =
  with md = Mode2 => haetae_polyq_mode23_bound
  with md = Mode3 => haetae_polyq_mode23_bound
  with md = Mode5 => haetae_polyq_mode5_bound.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

### Definition 13.180 (EasyCrypt line 935). EasyCrypt name.

```
haetae_polyq_coeffs_wf
```

### Checked EasyCrypt statement.

```

op haetae_polyq_coeffs_wf (md : mode) (p : poly) : bool =
  poly_wf p /\
  forall i, 0 <= i < n =>
    0 <= poly_coeff p i < haetae_polyq_coeff_bound md.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

### Definition 13.181 (EasyCrypt line 940). EasyCrypt name.

```
haetae_polyveck_polyq_coeffs_wf
```

### Checked EasyCrypt statement.

```

op haetae_polyveck_polyq_coeffs_wf
  (md : mode) (b : polyveck) : bool =
  polyveck_wf md b /\
  forall row, 0 <= row < mode_k md =>
    haetae_polyq_coeffs_wf md (nth poly_zero b row).

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

### Definition 13.182 (EasyCrypt line 1059). EasyCrypt name.

```
haetae_pack_poly_q_mode23_byte
```

### Checked EasyCrypt statement.

```

op haetae_pack_poly_q_mode23_byte
  (p : poly) (blk ofs : int) : byte =
  let c0 = poly_coeff p (8 * blk + 0) in
  let c1 = poly_coeff p (8 * blk + 1) in
  let c2 = poly_coeff p (8 * blk + 2) in
  let c3 = poly_coeff p (8 * blk + 3) in
  let c4 = poly_coeff p (8 * blk + 4) in
  let c5 = poly_coeff p (8 * blk + 5) in

```

```

let c6 = poly_coeff p (8 * blk + 6) in
let c7 = poly_coeff p (8 * blk + 7) in
if ofs = 0 then haetae_byte_norm c0
else if ofs = 1 then
  haetae_byte_norm ((c0 %/ 256) %% 128 + (c1 %% 2) * 128)
else if ofs = 2 then haetae_byte_norm (c1 %/ 2)
else if ofs = 3 then
  haetae_byte_norm ((c1 %/ 512) %% 64 + (c2 %% 4) * 64)
else if ofs = 4 then haetae_byte_norm (c2 %/ 4)
else if ofs = 5 then
  haetae_byte_norm ((c2 %/ 1024) %% 32 + (c3 %% 8) * 32)
else if ofs = 6 then haetae_byte_norm (c3 %/ 8)
else if ofs = 7 then
  haetae_byte_norm ((c3 %/ 2048) %% 16 + (c4 %% 16) * 16)
else if ofs = 8 then haetae_byte_norm (c4 %/ 16)
else if ofs = 9 then
  haetae_byte_norm ((c4 %/ 4096) %% 8 + (c5 %% 32) * 8)
else if ofs = 10 then haetae_byte_norm (c5 %/ 32)
else if ofs = 11 then
  haetae_byte_norm ((c5 %/ 8192) %% 4 + (c6 %% 64) * 4)
else if ofs = 12 then haetae_byte_norm (c6 %/ 64)
else if ofs = 13 then
  haetae_byte_norm ((c6 %/ 16384) %% 2 + (c7 %% 128) * 2)
else haetae_byte_norm (c7 %/ 128).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.183** (EasyCrypt line 1092). **EasyCrypt name.**

```
haetae_pack_poly_q_mode5_byte
```

**Checked EasyCrypt statement.**

```

op haetae_pack_poly_q_mode5_byte (p : poly) (j : int) : byte =
  let c = poly_coeff p (j %/ 2) in
  if j %% 2 = 0 then haetae_byte_norm c
  else haetae_byte_norm (c %/ 256).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.184** (EasyCrypt line 1097). **EasyCrypt name.**

```
haetae_pack_poly_q_ref
```

**Checked EasyCrypt statement.**

```

op haetae_pack_poly_q_ref (md : mode) (p : poly) : byte list =
  with md = Mode2 =>
    mkseq
      (fun j => haetae_pack_poly_q_mode23_byte p (j %/ 15) (j %% 15))
      (mode_polyq_packedbytes Mode2)
  with md = Mode3 =>
    mkseq
      (fun j => haetae_pack_poly_q_mode23_byte p (j %/ 15) (j %% 15))
      (mode_polyq_packedbytes Mode3)
  with md = Mode5 =>
    mkseq
      (fun j => haetae_pack_poly_q_mode5_byte p j)

```

```
(mode_polyq_packedbytes Mode5).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.185** (EasyCrypt line 1111). **EasyCrypt name.**

```
haetae_unpack_poly_q_mode23_coeff_at
```

**Checked EasyCrypt statement.**

```
op haetae_unpack_poly_q_mode23_coeff_at
  (enc : encoded_pkey) (base blk ofs : int) : coeff =
  let a0 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 0)) in
  let a1 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 1)) in
  let a2 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 2)) in
  let a3 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 3)) in
  let a4 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 4)) in
  let a5 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 5)) in
  let a6 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 6)) in
  let a7 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 7)) in
  let a8 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 8)) in
  let a9 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 9)) in
  let a10 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 10)) in
  let a11 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 11)) in
  let a12 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 12)) in
  let a13 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 13)) in
  let a14 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 14)) in
  if ofs = 0 then a0 + 256 * (a1 %% 128)
  else if ofs = 1 then
    ((a1 %% 128) %% 2) + 2 * a2 + 512 * (a3 %% 64)
  else if ofs = 2 then
    ((a3 %% 64) %% 4) + 4 * a4 + 1024 * (a5 %% 32)
  else if ofs = 3 then
    ((a5 %% 32) %% 8) + 8 * a6 + 2048 * (a7 %% 16)
  else if ofs = 4 then
    ((a7 %% 16) %% 16) + 16 * a8 + 4096 * (a9 %% 8)
  else if ofs = 5 then
    ((a9 %% 8) %% 32) + 32 * a10 + 8192 * (a11 %% 4)
  else if ofs = 6 then
    ((a11 %% 4) %% 64) + 64 * a12 + 16384 * (a13 %% 2)
  else ((a13 %% 2) %% 128) + 128 * a14.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.186** (EasyCrypt line 1143). **EasyCrypt name.**

```
haetae_unpack_poly_q_mode5_coeff_at
```

**Checked EasyCrypt statement.**

```
op haetae_unpack_poly_q_mode5_coeff_at
  (enc : encoded_pkey) (base i : int) : coeff =
  haetae_byte_norm (nth 0 enc (base + 2 * i)) +
  256 * haetae_byte_norm (nth 0 enc (base + 2 * i + 1)).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.



**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.187** (EasyCrypt line 1148). **EasyCrypt name.**

```
haetae_unpack_poly_q_ref_at
```

**Checked EasyCrypt statement.**

```
op haetae_unpack_poly_q_ref_at
  (md : mode) (enc : encoded_pkey) (offset : int) : poly =
  with md = Mode2 =>
    mkseq
      (fun i =>
        haetae_unpack_poly_q_mode23_coeff_at
          enc offset (i % 8) (i % 8))
    n
  with md = Mode3 =>
    mkseq
      (fun i =>
        haetae_unpack_poly_q_mode23_coeff_at
          enc offset (i % 8) (i % 8))
    n
  with md = Mode5 =>
    mkseq
      (fun i => haetae_unpack_poly_q_mode5_coeff_at enc offset i)
    n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.188** (EasyCrypt line 1167). **EasyCrypt name.**

```
haetae_pack_polyveck_body_ref
```

**Checked EasyCrypt statement.**

```
op haetae_pack_polyveck_body_ref (md : mode) (b : polyveck) : byte list =
  mkseq
    (fun i =>
      nth 0
        (haetae_pack_poly_q_ref md
          (nth poly_zero b (i % mode_polyq_packedbytes md)))
          (i % mode_polyq_packedbytes md))
    (haetae_public_key_body_bytes md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.189** (EasyCrypt line 1176). **EasyCrypt name.**

```
haetae_unpack_polyveck_body_ref
```

**Checked EasyCrypt statement.**

```
op haetae_unpack_polyveck_body_ref
  (md : mode) (enc : encoded_pkey) (offset : int) : polyveck =
  mkseq
    (fun row =>
```

```

    haetae_unpack_poly_q_ref_at md enc
      (offset + row * mode_polyq_packedbytes md))
    (mode_k md).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.190** (EasyCrypt line 1184). **EasyCrypt name.**

```

haetae_pack_vk_ref

```

**Checked EasyCrypt statement.**

```

op haetae_pack_vk_ref (md : mode) (b : polyveck) (sd : seed) : encoded_pkey =
  haetae_pack_seed sd ++ haetae_pack_polyveck_body_ref md b.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.191** (EasyCrypt line 1187). **EasyCrypt name.**

```

haetae_unpack_vk_public_key_ref

```

**Checked EasyCrypt statement.**

```

op haetae_unpack_vk_public_key_ref
  (md : mode) (enc : encoded_pkey) : pkey =
  (haetae_unpack_seed enc,
   haetae_unpack_polyveck_body_ref md enc seedbytes).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.192** (EasyCrypt line 1191). **EasyCrypt name.**

```

haetae_pack_polyveck_body

```

**Checked EasyCrypt statement.**

```

op haetae_pack_polyveck_body (md : mode) (b : polyveck) : byte list =
  mkseq
    (fun i =>
      if i %% mode_polyq_packedbytes md < n then
        poly_coeff
          (nth poly_zero b (i // mode_polyq_packedbytes md))
          (i %% mode_polyq_packedbytes md)
      else 0)
    (haetae_public_key_body_bytes md).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.193** (EasyCrypt line 1200). **EasyCrypt name.**

```
haetae_unpack_polyveck_body
```

**Checked EasyCrypt statement.**

```
op haetae_unpack_polyveck_body
  (md : mode) (enc : encoded_pkey) (offset : int) : polyveck =
  mkseq
    (fun row =>
      haetae_unpack_poly_q_at md enc
        (offset + row * mode_polyq_packedbytes md))
    (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.194** (EasyCrypt line 1207). **EasyCrypt name.**

```
haetae_pack_vk
```

**Checked EasyCrypt statement.**

```
op haetae_pack_vk (md : mode) (b : polyveck) (sd : seed) : encoded_pkey =
  haetae_pack_seed sd ++ haetae_unpack_polyveck_body md b.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.195** (EasyCrypt line 1209). **EasyCrypt name.**

```
haetae_unpack_vk_public_key
```

**Checked EasyCrypt statement.**

```
op haetae_unpack_vk_public_key
  (md : mode) (enc : encoded_pkey) : pkey =
  (haetae_unpack_seed enc,
   haetae_unpack_polyveck_body md enc seedbytes).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.196** (EasyCrypt line 1213). **EasyCrypt name.**

```
haetae_unpack_vk_matrix
```

**Checked EasyCrypt statement.**

```
op haetae_unpack_vk_matrix (md : mode) (enc : encoded_pkey) : matrix =
  packed_haetae_verification_matrix md
    (haetae_unpack_vk_public_key md enc).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.197** (EasyCrypt line 1216). **EasyCrypt name.**

```
haetae_pack_public_key_bytes
```

**Checked EasyCrypt statement.**

```
op haetae_pack_public_key_bytes (md : mode) (pk : pkey) : encoded_pkey =  
  haetae_pack_vk md pk.'2 pk.'1.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.198** (EasyCrypt line 1218). **EasyCrypt name.**

```
haetae_unpack_public_key_bytes
```

**Checked EasyCrypt statement.**

```
op haetae_unpack_public_key_bytes  
  (md : mode) (enc : encoded_pkey) : pkey =  
  haetae_unpack_vk_public_key md enc.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.199** (EasyCrypt line 1221). **EasyCrypt name.**

```
concrete_haetae_keygen_packed_public_key
```

**Checked EasyCrypt statement.**

```
op concrete_haetae_keygen_packed_public_key  
  (md : mode) (sd : seed) : encoded_pkey =  
  haetae_pack_public_key_bytes md (packed_haetae_public_key md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.200** (EasyCrypt line 1224). **EasyCrypt name.**

```
concrete_haetae_keygen_unpacked_public_key
```

**Checked EasyCrypt statement.**

```
op concrete_haetae_keygen_unpacked_public_key  
  (md : mode) (sd : seed) : pkey =  
  haetae_unpack_public_key_bytes md  
    (concrete_haetae_keygen_packed_public_key md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.201** (EasyCrypt line 1228). **EasyCrypt name.**

```
concrete_haetae_keygen_verification_matrix
```

**Checked EasyCrypt statement.**

```
op concrete_haetae_keygen_verification_matrix (md : mode) (sd : seed) :
  matrix =
    packed_haetae_verification_matrix md
      (concrete_haetae_keygen_unpacked_public_key md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.202** (EasyCrypt line 1232). **EasyCrypt name.**

```
haetae_pack_public_key_bytes_ref
```

**Checked EasyCrypt statement.**

```
op haetae_pack_public_key_bytes_ref (md : mode) (pk : pkey) :
  encoded_pkey =
    haetae_pack_vk_ref md pk.'2 pk.'1.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.203** (EasyCrypt line 1235). **EasyCrypt name.**

```
haetae_unpack_public_key_bytes_ref
```

**Checked EasyCrypt statement.**

```
op haetae_unpack_public_key_bytes_ref
  (md : mode) (enc : encoded_pkey) : pkey =
  haetae_unpack_vk_public_key_ref md enc.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.204** (EasyCrypt line 1238). **EasyCrypt name.**

```
concrete_haetae_keygen_packed_public_key_ref
```

**Checked EasyCrypt statement.**

```
op concrete_haetae_keygen_packed_public_key_ref
  (md : mode) (sd : seed) : encoded_pkey =
  haetae_pack_public_key_bytes_ref md (packed_haetae_public_key md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.205** (EasyCrypt line 1241). **EasyCrypt name.**

```
concrete_haetae_keygen_unpacked_public_key_ref
```

**Checked EasyCrypt statement.**

```
op concrete_haetae_keygen_unpacked_public_key_ref
  (md : mode) (sd : seed) : pkey =
  haetae_unpack_public_key_bytes_ref md
  (concrete_haetae_keygen_packed_public_key_ref md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.206** (EasyCrypt line 1245). **EasyCrypt name.**

```
concrete_haetae_keygen_verification_matrix_ref
```

**Checked EasyCrypt statement.**

```
op concrete_haetae_keygen_verification_matrix_ref
  (md : mode) (sd : seed) : matrix =
  packed_haetae_verification_matrix md
  (concrete_haetae_keygen_unpacked_public_key_ref md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.207** (EasyCrypt line 1249). **EasyCrypt name.**

```
concrete_haetae_reference_keygen_packed_public_key_ref
```

**Checked EasyCrypt statement.**

```
op concrete_haetae_reference_keygen_packed_public_key_ref
  (md : mode) (sd : seed) : encoded_pkey =
  haetae_pack_public_key_bytes_ref md
  (packed_haetae_reference_public_key md sd).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.208** (EasyCrypt line 1253). **EasyCrypt name.**

```
concrete_haetae_reference_keygen_unpacked_public_key_ref
```

**Checked EasyCrypt statement.**

```

op concrete_haetae_reference_keygen_unpacked_public_key_ref
  (md : mode) (sd : seed) : pkey =
  haetae_unpack_public_key_bytes_ref md
  (concrete_haetae_reference_keygen_packed_public_key_ref md sd).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.209** (EasyCrypt line 1257). **EasyCrypt name.**

```

concrete_haetae_reference_keygen_verification_matrix_ref

```

**Checked EasyCrypt statement.**

```

op concrete_haetae_reference_keygen_verification_matrix_ref
  (md : mode) (sd : seed) : matrix =
  packed_haetae_verification_matrix md
  (concrete_haetae_reference_keygen_unpacked_public_key_ref md sd).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.210** (EasyCrypt line 1261). **EasyCrypt name.**

```

haetae_public_key_roundtrip_ok

```

**Checked EasyCrypt statement.**

```

op haetae_public_key_roundtrip_ok (md : mode) (pk : pkey) : bool =
  haetae_unpack_public_key_bytes md
  (haetae_pack_public_key_bytes md pk) = pk.

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.211** (EasyCrypt line 1264). **EasyCrypt name.**

```

haetae_keygen_public_key_roundtrip_ok

```

**Checked EasyCrypt statement.**

```

op haetae_keygen_public_key_roundtrip_ok (md : mode) (sd : seed) : bool =
  haetae_public_key_roundtrip_ok md (packed_haetae_public_key md sd).

```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.212** (EasyCrypt line 3039). **EasyCrypt name.**

```
valid_mode
```

**Checked EasyCrypt statement.**

```
op valid_mode : mode -> bool = fun _ => true.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.213** (EasyCrypt line 3040). **EasyCrypt name.**

```
valid_keypair
```

**Checked EasyCrypt statement.**

```
op valid_keypair (md : mode) (pk : pkey) (sk : skey) : bool =  
  pk = public_key_of_secret md sk.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.214** (EasyCrypt line 3042). **EasyCrypt name.**

```
valid_signature
```

**Checked EasyCrypt statement.**

```
op valid_signature (md : mode) (sig : signature) : bool =  
  polyveck_wf md sig.'1 /\  
  poly_wf sig.'2 /\  
  crh_wf sig.'3 /\  
  challenge_wf sig.'4 /\  
  polyvecl_wf md (sig.'5).'1 /\  
  polyveck_wf md (sig.'5).'2 /\  
  polyveck_wf md (sig.'5).'3.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.215** (EasyCrypt line 3051). **EasyCrypt name.**

```
sig_commitment_highbits
```

**Checked EasyCrypt statement.**

```
op sig_commitment_highbits (sig : signature) : polyveck = sig.'1.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.



**Definition 13.216** (EasyCrypt line 3052). **EasyCrypt name.**

```
sig_commitment_lowbits
```

**Checked EasyCrypt statement.**

```
op sig_commitment_lowbits (sig : signature) : poly = sig.'2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.217** (EasyCrypt line 3053). **EasyCrypt name.**

```
sig_message_hash
```

**Checked EasyCrypt statement.**

```
op sig_message_hash (sig : signature) : crh = sig.'3.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.218** (EasyCrypt line 3054). **EasyCrypt name.**

```
sig_challenge
```

**Checked EasyCrypt statement.**

```
op sig_challenge (sig : signature) : challenge = sig.'4.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.219** (EasyCrypt line 3055). **EasyCrypt name.**

```
sig_token_value
```

**Checked EasyCrypt statement.**

```
op sig_token_value (sig : signature) : sig_token = sig.'5.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.220** (EasyCrypt line 3056). **EasyCrypt name.**

```
sig_response
```

**Checked EasyCrypt statement.**

```
op sig_response (sig : signature) : polyvecl = (sig_token_value sig).'1.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.221** (EasyCrypt line 3057). **EasyCrypt name.**

```
sig_response_aux
```

**Checked EasyCrypt statement.**

```
op sig_response_aux (sig : signature) : polyveck = (sig_token_value sig).‘2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.222** (EasyCrypt line 3058). **EasyCrypt name.**

```
sig_hint
```

**Checked EasyCrypt statement.**

```
op sig_hint (sig : signature) : hint_t = (sig_token_value sig).‘3.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.223** (EasyCrypt line 3060). **EasyCrypt name.**

```
encode_poly
```

**Checked EasyCrypt statement.**

```
op encode_poly (p : poly) : byte list = p.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.224** (EasyCrypt line 3061). **EasyCrypt name.**

```
encode_polyveck
```

**Checked EasyCrypt statement.**

```
op encode_polyveck (xs : polyveck) : byte list = flatten xs.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.225** (EasyCrypt line 3062). **EasyCrypt name.**

```
encode_pkey
```

**Checked EasyCrypt statement.**

```
op encode_pkey (pk : pkey) : byte list = pk.'1 ++ encode_polyveck pk.'2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.226** (EasyCrypt line 3064). **EasyCrypt name.**

```
deterministic_poly
```

**Checked EasyCrypt statement.**

```
op deterministic_poly (tag : int) (src : byte list) : poly =  
  mkseq (fun i => coeff_mod (tag + i + nth 0 src i)) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 13.227** (EasyCrypt line 3066). **EasyCrypt name.**

```
deterministic_polyveck
```

**Checked EasyCrypt statement.**

```
op deterministic_polyveck (md : mode) (src : byte list) : polyveck =  
  mkseq  
    (fun i => deterministic_poly i (i :: src))  
    (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 13.228** (EasyCrypt line 3070). **EasyCrypt name.**

```
deterministic_unit_coeff
```

**Checked EasyCrypt statement.**

```
op deterministic_unit_coeff (tag : int) (src : byte list) (i : int) :  
  coeff =  
  if (tag + i + nth 0 src i) %% 2 = 0 then 1 else -1.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 13.229** (EasyCrypt line 3073). **EasyCrypt name.**

```
deterministic_unit_poly
```

**Checked EasyCrypt statement.**

```
op deterministic_unit_poly (tag : int) (src : byte list) : poly =  
  mkseq (fun i => deterministic_unit_coeff tag src i) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 13.230** (EasyCrypt line 3075). **EasyCrypt name.**

```
deterministic_unit_polyveck
```

**Checked EasyCrypt statement.**

```
op deterministic_unit_polyveck  
  (md : mode) (tag : int) (src : byte list) : polyveck =  
  mkseq  
    (fun i => deterministic_unit_poly (tag + i) (i :: src))  
  (mode_k md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 13.231** (EasyCrypt line 3080). **EasyCrypt name.**

```
deterministic_unit_polyvecl
```

**Checked EasyCrypt statement.**

```
op deterministic_unit_polyvecl  
  (md : mode) (tag : int) (src : byte list) : polyvecl =  
  mkseq  
    (fun i => deterministic_unit_poly (tag + i) (i :: src))  
  (mode_l md).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a numerical term that appears in a game-hop or final security inequality.

**Definition 13.232** (EasyCrypt line 3085). **EasyCrypt name.**

```
commitment_source
```

**Checked EasyCrypt statement.**

```
op commitment_source  
  (sk : skey) (m : message) (ctx : context)  
  (coins : random_coins) : byte list =  
  coins ++ sk.'1 ++ ctx ++ m.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.233** (EasyCrypt line 3089). **EasyCrypt** name.

```
commitment_raw
```

**Checked EasyCrypt statement.**

```
op commitment_raw :
  mode -> skey -> message -> context -> random_coins -> polyveck =
  fun md sk m ctx coins =>
    deterministic_polyveck md (commitment_source sk m ctx coins).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.234** (EasyCrypt line 3093). **EasyCrypt** name.

```
response_source
```

**Checked EasyCrypt statement.**

```
op response_source
  (sk : skey) (m : message) (ctx : context)
  (coins : random_coins) : byte list =
  coins ++ sk.'1 ++ ctx ++ m.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.235** (EasyCrypt line 3097). **EasyCrypt** name.

```
response_aux_vector
```

**Checked EasyCrypt statement.**

```
op response_aux_vector :
  mode -> skey -> message -> context -> random_coins -> polyveck =
  fun md sk m ctx coins =>
    deterministic_unit_polyveck md 29 (response_source sk m ctx coins).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.236** (EasyCrypt line 3101). **EasyCrypt** name.

```
signing_sample_y1
```

**Checked EasyCrypt statement.**

```
op signing_sample_y1 :
  mode -> skey -> message -> context -> random_coins -> polyveck1 =
  fun md sk m ctx coins =>
    deterministic_unit_polyveck1 md 73 (response_source sk m ctx coins).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 13.237** (EasyCrypt line 3105). **EasyCrypt name.**

```
signing_sample_y2
```

**Checked EasyCrypt statement.**

```
op signing_sample_y2 :
  mode -> skey -> message -> context -> random_coins -> polyveck =
  fun md sk m ctx coins =>
    deterministic_unit_polyveck md 79 (response_source sk m ctx coins).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 13.238** (EasyCrypt line 3109). **EasyCrypt name.**

```
signing_entropy_token_of_coins
```

**Checked EasyCrypt statement.**

```
op signing_entropy_token_of_coins (coins : random_coins) : int =
  nth 0 coins 0 +
  256 * nth 0 coins 1 +
  65536 * nth 0 coins 2 +
  16777216 * nth 0 coins 3 +
  4294967296 * nth 0 coins 4 +
  1099511627776 * nth 0 coins 5 +
  281474976710656 * nth 0 coins 6 +
  72057594037927936 * nth 0 coins 7.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.239** (EasyCrypt line 3118). **EasyCrypt name.**

```
commitment_lowbits
```

**Checked EasyCrypt statement.**

```
op commitment_lowbits :
  mode -> skey -> message -> context -> random_coins -> poly =
  fun md sk m ctx coins =>
    let raw = nth poly_zero (commitment_raw md sk m ctx coins) 0 in
    mkseq
      (fun i =>
        if i = 0 then signing_entropy_token_of_coins coins
        else poly_coeff raw i)
    n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.240** (EasyCrypt line 3127). **EasyCrypt name.**

```
message_hash
```

**Checked EasyCrypt statement.**

```
op message_hash : pkey -> context -> message -> crh =
  fun pk ctx m => mkseq (fun i => nth 0 (encode_pkey pk ++ ctx ++ m) i) crhbytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.241** (EasyCrypt line 3129). **EasyCrypt name.**

```
challenge_source
```

**Checked EasyCrypt statement.**

```
op challenge_source (highbits : polyveck) (lowbits : poly) (mu : crh) :
  byte list =
  encode_poly lowbits ++ encode_polyveck highbits ++ mu.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.242** (EasyCrypt line 3132). **EasyCrypt name.**

```
challenge_from_seed
```

**Checked EasyCrypt statement.**

```
op challenge_from_seed (md : mode) (src : byte list) : challenge =
  mkseq
    (fun i =>
      if i < mode_tau md
      then if nth 0 src i %% 2 = 0 then 1 else -1
      else 0) n.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.243** (EasyCrypt line 3138). **EasyCrypt name.**

```
challenge_hash
```

**Checked EasyCrypt statement.**

```
op challenge_hash : mode -> polyveck -> poly -> crh -> challenge =
  fun md (_ : polyveck) lowbits mu =>
    challenge_from_seed md (encode_poly lowbits ++ mu).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.244** (EasyCrypt line 3141). **EasyCrypt name.**

```
commitment_challenge
```

**Checked EasyCrypt statement.**

```
op commitment_challenge :
  mode -> skey -> message -> context -> random_coins -> challenge =
  fun md sk m ctx coins =>
    challenge_hash md
      (response_aux_vector md sk m ctx coins)
      (commitment_lowbits md sk m ctx coins)
      (message_hash (public_key_of_secret md sk) ctx m).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.245** (EasyCrypt line 3148). **EasyCrypt name.**

```
commitment_highbits
```

**Checked EasyCrypt statement.**

```
op commitment_highbits :
  mode -> skey -> message -> context -> random_coins -> polyveck =
  fun md sk m ctx coins =>
    let pk = public_key_of_secret md sk in
    let ch = commitment_challenge md sk m ctx coins in
    reconstructed_highbits md
      (response_aux_vector md sk m ctx coins)
      (public_key_challenge_term md pk ch).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.246** (EasyCrypt line 3156). **EasyCrypt name.**

```
response_vector
```

**Checked EasyCrypt statement.**

```
op response_vector :
  mode -> skey -> message -> context -> random_coins -> polyvecl =
  fun md sk m ctx coins =>
    deterministic_unit_polyvecl md 17 (response_source sk m ctx coins).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.247** (EasyCrypt line 3160). **EasyCrypt name.**

```
hint_vector
```



### Checked EasyCrypt statement.

```

op hint_vector :
  mode -> skey -> message -> context -> random_coins -> hint_t =
  fun md sk m ctx coins =>
    deterministic_unit_polyveck md 41 (response_source sk m ctx coins).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.248** (EasyCrypt line 3164). **EasyCrypt name.**

```
signature_token
```

### Checked EasyCrypt statement.

```

op signature_token :
  mode -> skey -> message -> context -> random_coins -> sig_token =
  fun md sk m ctx coins =>
    ( response_vector md sk m ctx coins,
      response_aux_vector md sk m ctx coins,
      hint_vector md sk m ctx coins ).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.249** (EasyCrypt line 3373). **EasyCrypt name.**

```
keygen_internal
```

### Checked EasyCrypt statement.

```

op keygen_internal (md : mode) (sd : seed) : pkey * skey =
  let sk = secret_key_of_seed md sd in (public_key_of_secret md sk, sk).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.250** (EasyCrypt line 3375). **EasyCrypt name.**

```
sign_internal
```

### Checked EasyCrypt statement.

```

op sign_internal :
  mode -> skey -> message -> context -> random_coins -> signature =
  fun (md : mode) (sk : skey) (m : message) (ctx : context)
    (coins : random_coins) =>
    let pk = public_key_of_secret md sk in
    let highbits = commitment_highbits md sk m ctx coins in
    let lowbits = commitment_lowbits md sk m ctx coins in
    let mu = message_hash pk ctx m in
    let ch = commitment_challenge md sk m ctx coins in
    (highbits, lowbits, mu, ch, signature_token md sk m ctx coins, 0, 0).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.251** (EasyCrypt line 3386). **EasyCrypt name.**

```
verify_challenge_consistent
```

**Checked EasyCrypt statement.**

```
op verify_challenge_consistent
  (md : mode) (pk : pkey) (m : message) (ctx : context)
  (sig : signature) : bool =
  let mu = message_hash pk ctx m in
  sig_message_hash sig = mu /\
  sig_challenge sig =
    challenge_hash md
    (sig_commitment_highbits sig)
    (sig_commitment_lowbits sig)
  mu.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.252** (EasyCrypt line 3397). **EasyCrypt name.**

```
verify_reconstructed_highbits
```

**Checked EasyCrypt statement.**

```
op verify_reconstructed_highbits
  (md : mode) (pk : pkey) (sig : signature) : polyveck =
  let pk_ch = public_key_challenge_term md pk (sig_challenge sig) in
  reconstructed_highbits md (sig_response_aux sig) pk_ch.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.253** (EasyCrypt line 3402). **EasyCrypt name.**

```
verify_public_equation
```

**Checked EasyCrypt statement.**

```
op verify_public_equation
  (md : mode) (pk : pkey) (sig : signature) : bool =
  sig_commitment_highbits sig =
    verify_reconstructed_highbits md pk sig.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.254** (EasyCrypt line 3777). **EasyCrypt name.**

```
coeff_norm_sq
```

**Checked EasyCrypt statement.**

```
op coeff_norm_sq (x : coeff) : int = x * x.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.255** (EasyCrypt line 3778). **EasyCrypt name.**

```
poly_norm_sq
```

**Checked EasyCrypt statement.**

```
op poly_norm_sq (p : poly) : int =  
  if poly_unit p then size p  
  else if p = poly_zero then 0  
  else sumz (map coeff_norm_sq p).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.256** (EasyCrypt line 3782). **EasyCrypt name.**

```
polyvecl_norm_sq
```

**Checked EasyCrypt statement.**

```
op polyvecl_norm_sq (md : mode) (xs : polyvecl) : int =  
  if polyvecl_unit md xs then mode_l md * n  
  else if xs = polyvecl_zero md then 0  
  else sumz (map poly_norm_sq xs).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.257** (EasyCrypt line 3786). **EasyCrypt name.**

```
polyveck_norm_sq
```

**Checked EasyCrypt statement.**

```
op polyveck_norm_sq (md : mode) (xs : polyveck) : int =  
  if polyveck_unit md xs then mode_k md * n  
  else if xs = polyveck_zero md then 0  
  else sumz (map poly_norm_sq xs).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.258** (EasyCrypt line 3790). **EasyCrypt name.**

```
signature_response_norm_sq
```

**Checked EasyCrypt statement.**

```
op signature_response_norm_sq (md : mode) (sig : signature) : int =  
  polyvecl_norm_sq md (sig_response sig) +  
  polyveck_norm_sq md (sig_response_aux sig).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.259** (EasyCrypt line 3793). **EasyCrypt name.**

```
signature_verify_norm_sq
```

**Checked EasyCrypt statement.**

```
op signature_verify_norm_sq (md : mode) (sig : signature) : int =  
  signature_response_norm_sq md sig +  
  polyveck_norm_sq md (sig_hint sig).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.260** (EasyCrypt line 3797). **EasyCrypt name.**

```
secret_norm_sq
```

**Checked EasyCrypt statement.**

```
op secret_norm_sq (sk : skey) : int = sk.'2.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.261** (EasyCrypt line 3798). **EasyCrypt name.**

```
response_norm_sq
```

**Checked EasyCrypt statement.**

```
op response_norm_sq (md : mode) (sig : signature) : int =  
  signature_response_norm_sq md sig.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.262** (EasyCrypt line 3800). **EasyCrypt name.**

```
verify_norm_sq
```

**Checked EasyCrypt statement.**

```
op verify_norm_sq (md : mode) (sig : signature) : int =  
  signature_verify_norm_sq md sig.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 13.263** (EasyCrypt line 3803). **EasyCrypt name.**

```
secret_norm_ok
```

**Checked EasyCrypt statement.**

```
op secret_norm_ok (md : mode) (sk : skey) : bool =  
  0 <= secret_norm_sq sk /\ secret_norm_sq sk <= mode_b0sq md.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.264** (EasyCrypt line 3805). **EasyCrypt name.**

```
response_norm_ok
```

**Checked EasyCrypt statement.**

```
op response_norm_ok (md : mode) (sig : signature) : bool =  
  0 <= response_norm_sq md sig /\ response_norm_sq md sig <= mode_b1sq md.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.265** (EasyCrypt line 3807). **EasyCrypt name.**

```
verify_norm_ok
```

**Checked EasyCrypt statement.**

```
op verify_norm_ok (md : mode) (sig : signature) : bool =  
  0 <= verify_norm_sq md sig /\ verify_norm_sq md sig <= mode_b2sq md.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 13.266** (EasyCrypt line 3810). **EasyCrypt name.**

```
verify_internal
```

**Checked EasyCrypt statement.**

```
op verify_internal :  
  mode -> pkey -> message -> context -> signature -> bool =  
  fun (md : mode) (pk : pkey) (m : message) (ctx : context)  
    (sig : signature) =>  
    valid_signature md sig /\  
    verify_challenge_consistent md pk m ctx sig /\  
    verify_public_equation md pk sig /\  
    verify_norm_ok md sig.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

## Lemmas, Theorems, Relational Equivalences, and Their Proof Dependencies

**Theorem 13.1** (EasyCrypt line 39). *EasyCrypt name.*

```
poly_zero_size
```

**Checked EasyCrypt statement.**

```
lemma poly_zero_size :  
  size poly_zero = n.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.2** (EasyCrypt line 43). *EasyCrypt name.*

```
polyveck_zero_size
```

**Checked EasyCrypt statement.**

```
lemma polyveck_zero_size md :  
  size (polyveck_zero md) = mode_k md.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.3** (EasyCrypt line 47). *EasyCrypt name.*

```
polyvecl_zero_size
```

**Checked EasyCrypt statement.**

```
lemma polyvecl_zero_size md :  
  size (polyvecl_zero md) = mode_l md.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.4** (EasyCrypt line 51). *EasyCrypt name.*

```
polyvecm_zero_size
```

**Checked EasyCrypt statement.**

```
lemma polyvecm_zero_size md :
  size (polyvecm_zero md) = mode_m md.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.5** (EasyCrypt line 61). *EasyCrypt name.*

```
polyveck_wf_nth
```

**Checked EasyCrypt statement.**

```
lemma polyveck_wf_nth md b row :
  polyveck_wf md b =>
    0 <= row < mode_k md =>
  poly_wf (nth poly_zero b row).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
allP
b_all
b_sz
mem_nth
poly_wf
polyveck_wf
row_rng
```

**Theorem 13.6** (EasyCrypt line 89). *EasyCrypt name.*

```
poly_zero_wf
```

**Checked EasyCrypt statement.**

```
lemma poly_zero_wf : poly_wf poly_zero.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.7** (*EasyCrypt* line 92). *EasyCrypt* name.

```
polyveck_zero_wf
```

**Checked *EasyCrypt* statement.**

```
lemma polyveck_zero_wf md : polyveck_wf md (polyveck_zero md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
all_nseq
poly_zero_wf
polyveck_wf
polyveck_zero
polyveck_zero_size
```

**Theorem 13.8** (*EasyCrypt* line 100). *EasyCrypt* name.

```
polyvecl_zero_wf
```

**Checked *EasyCrypt* statement.**

```
lemma polyvecl_zero_wf md : polyvecl_wf md (polyvecl_zero md).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
all_nseq
poly_zero_wf
polyvecl_wf
polyvecl_zero
polyvecl_zero_size
```

**Theorem 13.9** (*EasyCrypt* line 170). *EasyCrypt* name.

```
polyvecl_sub_size
```



**Checked EasyCrypt statement.**

```
lemma polyvecl_sub_size md xs ys :  
  size (polyvecl_sub md xs ys) = mode_l md.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.10** (EasyCrypt line 174). **EasyCrypt name.**

```
poly_add_wf
```

**Checked EasyCrypt statement.**

```
lemma poly_add_wf p r : poly_wf (poly_add p r).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **rewrite**, **case**, **apply**. Named identifiers syntactically cited in the proof block:

```
poly_add  
poly_wf  
poly_zero  
poly_zero_size  
size_mkseq
```

**Theorem 13.11** (EasyCrypt line 182). **EasyCrypt name.**

```
poly_normalize_wf
```

**Checked EasyCrypt statement.**

```
lemma poly_normalize_wf p : poly_wf (poly_normalize p).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.12** (EasyCrypt line 185). **EasyCrypt name.**

```
poly_neg_wf
```

**Checked EasyCrypt statement.**

```
lemma poly_neg_wf p : poly_wf (poly_neg p).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `case`, `apply`. Named identifiers syntactically cited in the proof block:

```
poly_neg
poly_wf
poly_zero
poly_zero_size
size_mkseq
```

**Theorem 13.13** (EasyCrypt line 193). *EasyCrypt name.*

```
poly_sub_wf
```

**Checked EasyCrypt statement.**

```
lemma poly_sub_wf p r : poly_wf (poly_sub p r).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.14** (EasyCrypt line 196). *EasyCrypt name.*

```
poly_double_wf
```

**Checked EasyCrypt statement.**

```
lemma poly_double_wf p : poly_wf (poly_double p).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `case`, `apply`. Named identifiers syntactically cited in the proof block:

```
poly_double
poly_wf
poly_zero
poly_zero_size
size_mkseq
```

**Theorem 13.15** (EasyCrypt line 204). *EasyCrypt name.*

```
poly_mul_wf
```

**Checked EasyCrypt statement.**

```
lemma poly_mul_wf p r : poly_wf (poly_mul p r).
```

**Formal mathematical reading.** *Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.*

**Role in the proof.** *Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `case`, `apply`. Named identifiers syntactically cited in the proof block:*

```
poly_mul
poly_wf
poly_zero
poly_zero_size
size_mkseq
```

**Theorem 13.16** (EasyCrypt line 212). *EasyCrypt name.*

```
poly_dot_wf
```

**Checked EasyCrypt statement.**

```
lemma poly_dot_wf row v : poly_wf (poly_dot row v).
```

**Formal mathematical reading.** *Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.*

**Role in the proof.** *Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `case`, `apply`, `have`, `elim`. Named identifiers syntactically cited in the proof block:*

```
acc
acc0
acc_wf
foldl
ih
poly
poly_add
poly_add_wf
poly_dot
poly_mul
poly_wf
poly_zero_wf
row
xy
zs
```

**Theorem 13.17** (EasyCrypt line 231). *EasyCrypt name.*

```
matrix_vec_mul_wf
```

**Checked EasyCrypt statement.**

```
lemma matrix_vec_mul_wf md a v : polyveck_wf md (matrix_vec_mul md a v).
```

**Formal mathematical reading.** *Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.*

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*, *split*, *smt*. Named identifiers syntactically cited in the proof block:

```
allP
matrix_vec_mul
matrix_zero
md
mkseqP
mode_k_gt0
poly_dot_wf
polyveck_wf
polyveck_zero_wf
size_mkseq
```

**Theorem 13.18** (EasyCrypt line 243). *EasyCrypt name.*

```
polyveck_add_wf
```

**Checked EasyCrypt statement.**

```
lemma polyveck_add_wf md xs ys : polyveck_wf md (polyveck_add md xs ys).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*, *split*, *smt*. Named identifiers syntactically cited in the proof block:

```
allP
md
mkseqP
mode_k_gt0
poly_add_wf
polyveck_add
polyveck_wf
polyveck_zero
polyveck_zero_wf
size_mkseq
xs
ys
```

**Theorem 13.19** (EasyCrypt line 255). *EasyCrypt name.*

```
polyveck_neg_wf
```

**Checked EasyCrypt statement.**

```
lemma polyveck_neg_wf md xs : polyveck_wf md (polyveck_neg md xs).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*, *split*, *smt*. Named identifiers syntactically cited in the proof block:

```
allP
md
mkseqP
mode_k_gt0
poly_neg_wf
polyveck_neg
polyveck_wf
polyveck_zero
polyveck_zero_wf
size_mkseq
xs
```

**Theorem 13.20** (EasyCrypt line 267). *EasyCrypt* name.

```
polyveck_sub_wf
```

**Checked EasyCrypt statement.**

```
lemma polyveck_sub_wf md xs ys : polyveck_wf md (polyveck_sub md xs ys).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.21** (EasyCrypt line 270). *EasyCrypt* name.

```
polyveck_double_wf
```

**Checked EasyCrypt statement.**

```
lemma polyveck_double_wf md xs : polyveck_wf md (polyveck_double md xs).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*, *split*, *smt*. Named identifiers syntactically cited in the proof block:

```
allP
md
mkseqP
mode_k_gt0
poly_double_wf
polyveck_double
polyveck_wf
polyveck_zero
```

```
polyveck_zero_wf
size_mkseq
xs
```

**Theorem 13.22** (EasyCrypt line 282). *EasyCrypt name.*

```
polyvecl_sub_wf
```

**Checked EasyCrypt statement.**

```
lemma polyvecl_sub_wf md xs ys : polyvecl_wf md (polyvecl_sub md xs ys).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
md
mkseqP
poly_sub_wf
polyvecl_sub
polyvecl_wf
size_mkseq
```

**Theorem 13.23** (EasyCrypt line 298). *EasyCrypt name.*

```
coeff_mod_q_bound
```

**Checked EasyCrypt statement.**

```
lemma coeff_mod_q_bound x : coeff_q_bound (coeff_mod x).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.24** (EasyCrypt line 301). *EasyCrypt name.*

```
poly_zero_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
lemma poly_zero_coeffs_q_bound : poly_coeffs_q_bound poly_zero.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
nth_nseq
poly_coeff
poly_coeffs_q_bound
poly_zero
q_gt0
```

**Theorem 13.25** (EasyCrypt line 307). *EasyCrypt name.*

```
poly_add_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
lemma poly_add_coeffs_q_bound p r :
  poly_coeffs_q_bound (poly_add p r).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
coeff_mod_q_bound
i_rng
nth_mkseq
poly_add
poly_coeff
poly_coeffs_q_bound
poly_zero
poly_zero_coeffs_q_bound
```

**Theorem 13.26** (EasyCrypt line 317). *EasyCrypt name.*

```
poly_neg_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
lemma poly_neg_coeffs_q_bound p :
  poly_coeffs_q_bound (poly_neg p).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
coeff_mod_q_bound
i_rng
nth_mkseq
poly_coeff
```

```
poly_coeffs_q_bound
poly_neg
poly_zero
poly_zero_coeffs_q_bound
```

**Theorem 13.27** (EasyCrypt line 327). *EasyCrypt name.*

```
poly_sub_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
lemma poly_sub_coeffs_q_bound p r :
  poly_coeffs_q_bound (poly_sub p r).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.28** (EasyCrypt line 331). *EasyCrypt name.*

```
poly_double_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
lemma poly_double_coeffs_q_bound p :
  poly_coeffs_q_bound (poly_double p).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
coeff_mod_q_bound
i_rng
nth_mkseq
poly_coeff
poly_coeffs_q_bound
poly_double
poly_zero
poly_zero_coeffs_q_bound
```

**Theorem 13.29** (EasyCrypt line 341). *EasyCrypt name.*

```
poly_mul_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
lemma poly_mul_coeffs_q_bound p r :
  poly_coeffs_q_bound (poly_mul p r).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.



**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
coeff_mod_q_bound
i_rng
nth_mkseq
poly_coeff
poly_coeffs_q_bound
poly_mul
poly_zero
poly_zero_coeffs_q_bound
```

**Theorem 13.30** (EasyCrypt line 351). *EasyCrypt name.*

```
poly_dot_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
lemma poly_dot_coeffs_q_bound row v :
  poly_coeffs_q_bound (poly_dot row v).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*, *have*, *elim*. Named identifiers syntactically cited in the proof block:

```
acc
acc0
acc_bd
foldl
ih
poly
poly_add
poly_add_coeffs_q_bound
poly_coeffs_q_bound
poly_dot
poly_mul
poly_zero_coeffs_q_bound
row
xy
zs
```

**Theorem 13.31** (EasyCrypt line 371). *EasyCrypt name.*

```
matrix_vec_mul_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
lemma matrix_vec_mul_coeffs_q_bound md a v :
  polyveck_coeffs_q_bound md (matrix_vec_mul md a v).
```

**Formal mathematical reading.** *Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.*

**Role in the proof.** *Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `case`, `have`, `smt`, `apply`. Named identifiers syntactically cited in the proof block:*

```
matrix_vec_mul
matrix_zero
md
mode_k
nseq
nth
nth_mkseq
nth_nseq
poly_dot_coeffs_q_bound
poly_zero
poly_zero_coeffs_q_bound
polyveck_coeffs_q_bound
polyveck_zero
row
row_rng
```

**Theorem 13.32** (EasyCrypt line 384). *EasyCrypt name.*

```
polyveck_zero_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
lemma polyveck_zero_coeffs_q_bound md :
  polyveck_coeffs_q_bound md (polyveck_zero md).
```

**Formal mathematical reading.** *Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.*

**Role in the proof.** *Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `have`, `smt`, `apply`. Named identifiers syntactically cited in the proof block:*

```
md
mode_k
nseq
nth
nth_nseq
poly_zero
poly_zero_coeffs_q_bound
polyveck_coeffs_q_bound
polyveck_zero
row
row_rng
```

**Theorem 13.33** (EasyCrypt line 393). *EasyCrypt name.*

```
polyveck_add_coeffs_q_bound
```

### Checked *EasyCrypt* statement.

```
lemma polyveck_add_coeffs_q_bound md xs ys :  
  polyveck_coeffs_q_bound md (polyveck_add md xs ys).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
md  
nth_mkseq  
poly_add_coeffs_q_bound  
polyveck_add  
polyveck_coeffs_q_bound  
polyveck_zero  
polyveck_zero_coeffs_q_bound  
row  
row_rng  
xs  
ys
```

### Theorem 13.34 (*EasyCrypt* line 403). *EasyCrypt* name.

```
polyveck_neg_coeffs_q_bound
```

### Checked *EasyCrypt* statement.

```
lemma polyveck_neg_coeffs_q_bound md xs :  
  polyveck_coeffs_q_bound md (polyveck_neg md xs).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
md  
nth_mkseq  
poly_neg_coeffs_q_bound  
polyveck_coeffs_q_bound  
polyveck_neg  
polyveck_zero  
polyveck_zero_coeffs_q_bound  
row  
row_rng  
xs
```

### Theorem 13.35 (*EasyCrypt* line 413). *EasyCrypt* name.

```
polyveck_sub_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
lemma polyveck_sub_coeffs_q_bound md xs ys :  
  polyveck_coeffs_q_bound md (polyveck_sub md xs ys).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.36** (EasyCrypt line 417). **EasyCrypt name.**

```
polyveck_double_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
lemma polyveck_double_coeffs_q_bound md xs :  
  polyveck_coeffs_q_bound md (polyveck_double md xs).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
md  
nth_mkseq  
poly_double_coeffs_q_bound  
polyveck_coeffs_q_bound  
polyveck_double  
polyveck_zero  
polyveck_zero_coeffs_q_bound  
row  
row_rng  
xs
```

**Theorem 13.37** (EasyCrypt line 483). **EasyCrypt name.**

```
poly_highbits_wf
```

**Checked EasyCrypt statement.**

```
lemma poly_highbits_wf p : poly_wf (poly_highbits p).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.38** (EasyCrypt line 486). **EasyCrypt name.**

```
poly_lowbits_wf
```

**Checked EasyCrypt statement.**

```
lemma poly_lowbits_wf p : poly_wf (poly_lowbits p).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.39** (EasyCrypt line 489). **EasyCrypt name.**

```
poly_lsb_wf
```

**Checked EasyCrypt statement.**

```
lemma poly_lsb_wf p : poly_wf (poly_lsb p).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.40** (EasyCrypt line 492). **EasyCrypt name.**

```
poly_vk_lowbits_wf
```

**Checked EasyCrypt statement.**

```
lemma poly_vk_lowbits_wf p : poly_wf (poly_vk_lowbits p).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.41** (EasyCrypt line 495). **EasyCrypt name.**

```
poly_vk_highbits_wf
```

**Checked EasyCrypt statement.**

```
lemma poly_vk_highbits_wf p : poly_wf (poly_vk_highbits p).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.42** (EasyCrypt line 498). **EasyCrypt name.**

```
poly_compose_z1_wf
```

**Checked EasyCrypt statement.**

```
lemma poly_compose_z1_wf hi lo : poly_wf (poly_compose_z1 hi lo).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.43** (*EasyCrypt* line 501). *EasyCrypt* name.

```
poly_hint_highbits_wf
```

**Checked *EasyCrypt* statement.**

```
lemma poly_hint_highbits_wf md p : poly_wf (poly_hint_highbits md p).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.44** (*EasyCrypt* line 504). *EasyCrypt* name.

```
polyvecl_highbits_wf
```

**Checked *EasyCrypt* statement.**

```
lemma polyvecl_highbits_wf md xs :  
  polyvecl_wf md (polyvecl_highbits md xs).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP  
md  
mkseqP  
poly_highbits_wf  
polyvecl_highbits  
polyvecl_wf  
size_mkseq
```

**Theorem 13.45** (*EasyCrypt* line 514). *EasyCrypt* name.

```
polyvecl_lowbits_wf
```

**Checked *EasyCrypt* statement.**

```
lemma polyvecl_lowbits_wf md xs :  
  polyvecl_wf md (polyvecl_lowbits md xs).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
md
mkseqP
poly_lowbits_wf
polyvecl_lowbits
polyvecl_wf
size_mkseq
```

**Theorem 13.46** (*EasyCrypt* line 524). *EasyCrypt* name.

```
polyveck_highbits_hint_wf
```

**Checked *EasyCrypt* statement.**

```
lemma polyveck_highbits_hint_wf md xs :
  polyveck_wf md (polyveck_highbits_hint md xs).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
md
mkseqP
poly_hint_highbits_wf
polyveck_highbits_hint
polyveck_wf
size_mkseq
```

**Theorem 13.47** (*EasyCrypt* line 534). *EasyCrypt* name.

```
polyveck_vk_lowbits_wf
```

**Checked *EasyCrypt* statement.**

```
lemma polyveck_vk_lowbits_wf md xs :
  polyveck_wf md (polyveck_vk_lowbits md xs).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
md
mkseqP
poly_vk_lowbits_wf
polyveck_vk_lowbits
polyveck_wf
size_mkseq
```

**Theorem 13.48** (EasyCrypt line 544). *EasyCrypt name.*

```
polyveck_vk_highbits_wf
```

**Checked EasyCrypt statement.**

```
lemma polyveck_vk_highbits_wf md xs :
  polyveck_wf md (polyveck_vk_highbits md xs).
```

**Formal mathematical reading.** *Mathematical theorem.* This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** *Use in this chapter.* It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
md
mkseqP
poly_vk_highbits_wf
polyveck_vk_highbits
polyveck_wf
size_mkseq
```

**Theorem 13.49** (EasyCrypt line 554). *EasyCrypt name.*

```
polyveck_mul_alpha_wf
```

**Checked EasyCrypt statement.**

```
lemma polyveck_mul_alpha_wf md xs :
  polyveck_wf md (polyveck_mul_alpha md xs).
```

**Formal mathematical reading.** *Mathematical theorem.* This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** *Use in this chapter.* It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *case*, *apply*. Named identifiers syntactically cited in the proof block:



```

allP
md
mkseqP
poly_wf
polyveck_mul_alpha
polyveck_wf
size_mkseq

```

**Theorem 13.50** (EasyCrypt line 946). *EasyCrypt name.*

```
coeff_decompose_hint_high_polyq_bound
```

**Checked EasyCrypt statement.**

```

lemma coeff_decompose_hint_high_polyq_bound md x :
  coeff_q_bound x =>
  0 <= coeff_decompose_hint_high md x < haetae_polyq_coeff_bound md.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *case*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

coeff_decompose_hint_high
coeff_q_bound
dq
haetae_polyq_coeff_bound
haetae_polyq_mode23_bound
haetae_polyq_mode5_bound
md
mode_alpha_hint

```

**Theorem 13.51** (EasyCrypt line 956). *EasyCrypt name.*

```
poly_hint_highbits_polyq_wf
```

**Checked EasyCrypt statement.**

```

lemma poly_hint_highbits_polyq_wf md p :
  poly_coeffs_q_bound p =>
  haetae_polyq_coeffs_wf md (poly_hint_highbits md p).

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```

coeff_decompose_hint_high_polyq_bound
haetae_polyq_coeffs_wf
i_rng

```

```

nth_mkseq
p_bd
poly_coeff
poly_hint_highbits
poly_hint_highbits_wf

```

**Theorem 13.52** (EasyCrypt line 970). *EasyCrypt name.*

```
polyveck_highbits_hint_polyq_wf
```

**Checked EasyCrypt statement.**

```

lemma polyveck_highbits_hint_polyq_wf md xs :
  polyveck_coeffs_q_bound md xs =>
  haetae_polyveck_polyq_coeffs_wf md (polyveck_highbits_hint md xs).

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```

haetae_polyveck_polyq_coeffs_wf
nth_mkseq
poly_hint_highbits_polyq_wf
polyveck_highbits_hint
polyveck_highbits_hint_wf
row
row_rng
xs_bd

```

**Theorem 13.53** (EasyCrypt line 984). *EasyCrypt name.*

```
coeff_decompose_vk_high_mode23_polyq_bound
```

**Checked EasyCrypt statement.**

```

lemma coeff_decompose_vk_high_mode23_polyq_bound md x :
  (md = Mode2 ∨ md = Mode3) =>
  coeff_q_bound x =>
  0 <= coeff_decompose_vk_high x < haetae_polyq_coeff_bound md.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *case*, *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

coeff_decompose_vk_high
coeff_decompose_vk_low
coeff_q_bound
haetae_polyq_coeff_bound

```

```
haetae_polyq_mode23_bound
haetae_polyq_mode5_bound
md
```

**Theorem 13.54** (EasyCrypt line 995). *EasyCrypt name.*

```
poly_vk_highbits_mode23_polyq_wf
```

**Checked EasyCrypt statement.**

```
lemma poly_vk_highbits_mode23_polyq_wf md p :
  (md = Mode2 \ / md = Mode3) =>
  poly_coeffs_q_bound p =>
  haetae_polyq_coeffs_wf md (poly_vk_highbits p).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
coeff_decompose_vk_high_mode23_polyq_bound
haetae_polyq_coeffs_wf
i_rng
md23
nth_mkseq
p_bd
poly_coeff
poly_vk_highbits
poly_vk_highbits_wf
```

**Theorem 13.55** (EasyCrypt line 1010). *EasyCrypt name.*

```
polyveck_vk_highbits_mode23_polyq_wf
```

**Checked EasyCrypt statement.**

```
lemma polyveck_vk_highbits_mode23_polyq_wf md xs :
  (md = Mode2 \ / md = Mode3) =>
  polyveck_coeffs_q_bound md xs =>
  haetae_polyveck_polyq_coeffs_wf md (polyveck_vk_highbits md xs).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
haetae_polyveck_polyq_coeffs_wf
md23
nth_mkseq
poly_vk_highbits_mode23_polyq_wf
```

```
polyveck_vk_highbits
polyveck_vk_highbits_wf
row
row_rng
xs_bd
```

**Theorem 13.56** (EasyCrypt line 1025). *EasyCrypt name.*

```
coeff_q_bound_mode5_polyq_bound
```

**Checked EasyCrypt statement.**

```
lemma coeff_q_bound_mode5_polyq_bound x :
  coeff_q_bound x =>
  0 <= x < haetae_polyq_coeff_bound Mode5.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
coeff_q_bound
haetae_polyq_coeff_bound
haetae_polyq_mode5_bound
```

**Theorem 13.57** (EasyCrypt line 1033). *EasyCrypt name.*

```
poly_q_bound_mode5_polyq_wf
```

**Checked EasyCrypt statement.**

```
lemma poly_q_bound_mode5_polyq_wf p :
  poly_wf p =>
  poly_coeffs_q_bound p =>
  haetae_polyq_coeffs_wf Mode5 p.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
coeff_q_bound_mode5_polyq_bound
haetae_polyq_coeffs_wf
i_rng
p_bd
p_wf
```

**Theorem 13.58** (EasyCrypt line 1045). *EasyCrypt name.*

```
polyveck_q_bound_mode5_polyq_wf
```

**Checked EasyCrypt statement.**

```
lemma polyveck_q_bound_mode5_polyq_wf xs :
  polyveck_wf Mode5 xs =>
  polyveck_coeffs_q_bound Mode5 xs =>
  haetae_polyveck_polyq_coeffs_wf Mode5 xs.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
Mode5
haetae_polyveck_polyq_coeffs_wf
poly_q_bound_mode5_polyq_wf
polyveck_wf_nth
row
row_rng
xs
xs_bd
xs_wf
```

**Theorem 13.59** (EasyCrypt line 1267). *EasyCrypt name.*

```
public_key_seed_poly_wf
```

**Checked EasyCrypt statement.**

```
lemma public_key_seed_poly_wf tag src :
  poly_wf (public_key_seed_poly tag src).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.60** (EasyCrypt line 1271). *EasyCrypt name.*

```
public_key_seed_polyvecl_wf
```

**Checked EasyCrypt statement.**

```
lemma public_key_seed_polyvecl_wf md tag src :
  polyvecl_wf md (public_key_seed_polyvecl md tag src).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*, *split*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
md
mkseqP
polyvecl_wf
public_key_seed_poly_wf
public_key_seed_polyvecl
size_mkseq
```

**Theorem 13.61** (EasyCrypt line 1281). *EasyCrypt* name.

```
public_key_seed_polyveck_wf
```

**Checked EasyCrypt statement.**

```
lemma public_key_seed_polyveck_wf md tag src :
  polyveck_wf md (public_key_seed_polyveck md tag src).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
md
mkseqP
polyveck_wf
public_key_seed_poly_wf
public_key_seed_polyveck
size_mkseq
```

**Theorem 13.62** (EasyCrypt line 1291). *EasyCrypt* name.

```
haetae_keygen_seedbuf_bytes_gt0
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_seedbuf_bytes_gt0 :
  0 < haetae_keygen_seedbuf_bytes.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.63** (EasyCrypt line 1295). *EasyCrypt* name.

```
haetae_reference_keygen_seedbuf_bytesE
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_seedbuf_bytesE :
  haetae_reference_keygen_seedbuf_bytes = haetae_keygen_seedbuf_bytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_seedbuf_bytes
haetae_reference_keygen_seedbuf_bytes
```

**Theorem 13.64** (EasyCrypt line 1302). *EasyCrypt name.*

```
haetae_reference_keygen_xof_absorb_bytesE
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_xof_absorb_bytesE :
  haetae_reference_keygen_xof_absorb_bytes = seedbytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.65** (EasyCrypt line 1306). *EasyCrypt name.*

```
haetae_reference_keygen_xof_squeeze_bytesE
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_xof_squeeze_bytesE :
  haetae_reference_keygen_xof_squeeze_bytes =
  haetae_reference_keygen_seedbuf_bytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.66** (EasyCrypt line 1311). *EasyCrypt name.*

```
haetae_keygen_rho_prime_offsetE
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_rho_prime_offsetE :
  haetae_keygen_rho_prime_offset = 0.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.67** (EasyCrypt line 1315). *EasyCrypt name.*

```
haetae_keygen_sigma_offsetE
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_sigma_offsetE :  
  haetae_keygen_sigma_offset = seedbytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.68** (EasyCrypt line 1319). *EasyCrypt name.*

```
haetae_keygen_key_offsetE
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_key_offsetE :  
  haetae_keygen_key_offset = seedbytes + crhbytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.69** (EasyCrypt line 1323). *EasyCrypt name.*

```
haetae_shake256_rate_bytesE
```

**Checked EasyCrypt statement.**

```
lemma haetae_shake256_rate_bytesE :  
  haetae_shake256_rate_bytes = 136.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_shake256_rate_bytes  
shake256_rate_bytesE
```

**Theorem 13.70** (EasyCrypt line 1329). *EasyCrypt name.*



```
haetae_shake256_domain_separatorE
```

**Checked EasyCrypt statement.**

```
lemma haetae_shake256_domain_separatorE :  
  haetae_shake256_domain_separator = 31.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_shake256_domain_separator  
shake256_domain_separatorE
```

**Theorem 13.71** (EasyCrypt line 1335). *EasyCrypt name.*

```
haetae_shake256_bytes_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_shake256_bytes_size input outlen :  
  0 <= outlen =>  
  size (haetae_shake256_bytes input outlen) = outlen.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
ge0_outlen  
haetae_shake256_bytes  
shake256_size
```

**Theorem 13.72** (EasyCrypt line 1343). *EasyCrypt name.*

```
haetae_xof256_squeeze_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_xof256_squeeze_size st outlen :  
  0 <= outlen =>  
  size (haetae_xof256_squeeze st outlen) = outlen.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
ge0_outlen
haetae_xof256_squeeze
shake256_squeeze_size
```

**Theorem 13.73** (EasyCrypt line 1352). *EasyCrypt name.*

```
haetae_xof256_absorb_once_squeeze_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_xof256_absorb_once_squeeze_size input inlen outlen :
  0 <= outlen =>
  size (haetae_xof256_absorb_once_squeeze input inlen outlen) = outlen.
```

**Formal mathematical reading.** *Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
ge0_outlen
haetae_xof256_absorb_once_squeeze
haetae_xof256_squeeze_size
```

**Theorem 13.74** (EasyCrypt line 1361). *EasyCrypt name.*

```
haetae_reference_keygen_seedbuf_after_memcpy_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_seedbuf_after_memcpy_size sd :
  size (haetae_reference_keygen_seedbuf_after_memcpy sd) =
  haetae_reference_keygen_seedbuf_bytes.
```

**Formal mathematical reading.** *Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
haetae_reference_keygen_seedbuf_after_memcpy
size_mkseq
```

**Theorem 13.75** (EasyCrypt line 1368). *EasyCrypt name.*

```
haetae_keygen_xof_seedbuf_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_xof_seedbuf_size sd :  
  size (haetae_keygen_xof_seedbuf sd) = haetae_keygen_seedbuf_bytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_seedbuf_bytes  
haetae_keygen_xof_seedbuf  
haetae_reference_keygen_seedbuf_bytes  
haetae_reference_keygen_xof_squeeze_bytes  
haetae_xof256_absorb_once_squeeze  
haetae_xof256_squeeze  
shake256_squeeze  
size_mkseq
```

**Theorem 13.76** (EasyCrypt line 1379). *EasyCrypt name.*

```
haetae_reference_keygen_xof256_seedbuf_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_xof256_seedbuf_size sd :  
  size (haetae_reference_keygen_xof256_seedbuf sd) =  
  haetae_reference_keygen_seedbuf_bytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_xof_seedbuf_size  
haetae_reference_keygen_seedbuf_bytesE  
haetae_reference_keygen_xof256_seedbuf
```

**Theorem 13.77** (EasyCrypt line 1388). *EasyCrypt name.*

```
haetae_seed_slice_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_seed_slice_size src offset len :  
  0 <= len =>  
  size (haetae_seed_slice src offset len) = len.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `smt`. Named identifiers syntactically cited in the proof block:

```
ge0_len
haetae_seed_slice
size_mkseq
```

**Theorem 13.78** (EasyCrypt line 1397). *EasyCrypt name.*

```
haetae_reference_keygen_xof_input_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_xof_input_size sd :
  size (haetae_reference_keygen_xof_input sd) =
  haetae_reference_keygen_xof_absorb_bytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `apply`. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_xof_absorb_bytes
haetae_reference_keygen_xof_input
haetae_seed_slice_size
seedbytes
```

**Theorem 13.79** (EasyCrypt line 1406). *EasyCrypt name.*

```
haetae_xof256_absorb_input_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_xof256_absorb_input_size input inlen :
  0 <= inlen =>
  size (haetae_xof256_absorb_input input inlen) = inlen.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `apply`. Named identifiers syntactically cited in the proof block:

```
ge0_inlen
haetae_seed_slice_size
haetae_xof256_absorb_input
```

**Theorem 13.80** (EasyCrypt line 1415). *EasyCrypt name.*

```
haetae_reference_keygen_xof_absorb_inputE
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_xof_absorb_inputE sd :
  haetae_xof256_absorb_input
    (haetae_reference_keygen_seedbuf_after_memcpy sd)
  haetae_reference_keygen_xof_absorb_bytes =
  haetae_reference_keygen_xof_input sd.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_xof_input
haetae_xof256_absorb_input
```

**Theorem 13.81** (EasyCrypt line 1425). *EasyCrypt name.*

```
haetae_reference_keygen_shake256_domain
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_shake256_domain sd :
  shake256_absorb_once_short_domain
    (haetae_reference_keygen_xof_input sd)
  haetae_reference_keygen_xof_absorb_bytes.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_xof_absorb_bytes
seedbytes
shake256_absorb_once_short_domain
shake256_rate_bytesE
```

**Theorem 13.82** (EasyCrypt line 1435). *EasyCrypt name.*

```
haetae_reference_keygen_xof_squeeze_bytes_lt_rate
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_xof_squeeze_bytes_lt_rate :  
  haetae_reference_keygen_xof_squeeze_bytes < haetae_shake256_rate_bytes.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
crhbytes  
haetae_reference_keygen_seedbuf_bytes  
haetae_reference_keygen_xof_squeeze_bytes  
haetae_shake256_rate_bytes  
seedbytes  
shake256_rate_bytesE
```

**Theorem 13.83** (EasyCrypt line 1444). *EasyCrypt name.*

```
haetae_reference_keygen_xof_absorb_once_stateE
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_xof_absorb_once_stateE sd :  
  haetae_xof256_absorb_once  
    (haetae_reference_keygen_seedbuf_after_memcpy sd)  
    haetae_reference_keygen_xof_absorb_bytes =  
  shake256_absorb_once  
    (haetae_reference_keygen_xof_input sd)  
    haetae_reference_keygen_xof_absorb_bytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_xof_absorb_inputE  
haetae_xof256_absorb_once
```

**Theorem 13.84** (EasyCrypt line 1456). *EasyCrypt name.*

```
haetae_keygen_xof_seedbuf_incrementalE
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_xof_seedbuf_incrementalE sd :  
  haetae_keygen_xof_seedbuf sd =  
  haetae_xof256_squeeze  
    (haetae_xof256_absorb_once  
      (haetae_reference_keygen_seedbuf_after_memcpy sd))
```

```
haetae_reference_keygen_xof_absorb_bytes)
haetae_reference_keygen_xof_squeeze_bytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.85** (EasyCrypt line 1466). *EasyCrypt name.*

```
haetae_keygen_xof_seedbuf_shake256E
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_xof_seedbuf_shake256E sd :
  haetae_keygen_xof_seedbuf sd =
  haetae_shake256_bytes
    (haetae_reference_keygen_xof_input sd)
  haetae_reference_keygen_xof_squeeze_bytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_xof_seedbuf_incrementalE
haetae_reference_keygen_xof_absorb_once_stateE
haetae_reference_keygen_xof_input_size
haetae_shake256_bytes
shake256
```

**Theorem 13.86** (EasyCrypt line 1478). *EasyCrypt name.*

```
haetae_keygen_seedbuf_rho_prime_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_seedbuf_rho_prime_size seedbuf :
  size (haetae_keygen_seedbuf_rho_prime seedbuf) = seedbytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_seedbuf_rho_prime
haetae_seed_slice_size
seedbytes
```

**Theorem 13.87** (EasyCrypt line 1486). *EasyCrypt name.*

```
haetae_keygen_seedbuf_sigma_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_seedbuf_sigma_size seedbuf :  
  size (haetae_keygen_seedbuf_sigma seedbuf) = crhbytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
crhbytes  
haetae_keygen_seedbuf_sigma  
haetae_seed_slice_size
```

**Theorem 13.88** (EasyCrypt line 1494). *EasyCrypt name.*

```
haetae_keygen_seedbuf_key_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_seedbuf_key_size seedbuf :  
  size (haetae_keygen_seedbuf_key seedbuf) = seedbytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_seedbuf_key  
haetae_seed_slice_size  
seedbytes
```

**Theorem 13.89** (EasyCrypt line 1502). *EasyCrypt name.*

```
haetae_reference_keygen_xof_seedbuf_layout_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_xof_seedbuf_layout_wf sd :  
  haetae_keygen_seedbuf_layout_wf  
    (haetae_reference_keygen_xof256_seedbuf sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.



**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_key_offsetE
haetae_keygen_rho_prime_offsetE
haetae_keygen_seedbuf_key_size
haetae_keygen_seedbuf_layout_wf
haetae_keygen_seedbuf_rho_prime_size
haetae_keygen_seedbuf_sigma_size
haetae_keygen_sigma_offsetE
haetae_reference_keygen_xof256_seedbuf_size
```

**Theorem 13.90** (EasyCrypt line 1516). *EasyCrypt* name.

```
haetae_reference_keygen_xof_call_wf_ok
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_xof_call_wf_ok sd :
  size sd = seedbytes =>
    haetae_reference_keygen_xof_call_wf sd
    (haetae_reference_keygen_xof256_seedbuf sd).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_xof_seedbuf_incrementalE
haetae_reference_keygen_seedbuf_after_memcpy_size
haetae_reference_keygen_shake256_domain
haetae_reference_keygen_xof256_seedbuf
haetae_reference_keygen_xof256_seedbuf_size
haetae_reference_keygen_xof_absorb_bytesE
haetae_reference_keygen_xof_absorb_inputE
haetae_reference_keygen_xof_absorb_once_stateE
haetae_reference_keygen_xof_call_wf
haetae_reference_keygen_xof_squeeze_bytesE
haetae_reference_keygen_xof_squeeze_bytes_lt_rate
haetae_xof256_squeeze
sd_sz
```

**Theorem 13.91** (EasyCrypt line 1537). *EasyCrypt* name.

```
haetae_keygen_rho_prime_ref_sliceE
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_rho_prime_ref_sliceE sd :
  haetae_keygen_rho_prime sd =
    haetae_keygen_seedbuf_rho_prime
    (haetae_reference_keygen_xof256_seedbuf sd).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.92** (EasyCrypt line 1543). *EasyCrypt name.*

```
haetae_keygen_sigma_ref_sliceE
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_sigma_ref_sliceE sd :
  haetae_keygen_sigma sd =
  haetae_keygen_seedbuf_sigma
    (haetae_reference_keygen_xof256_seedbuf sd).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.93** (EasyCrypt line 1549). *EasyCrypt name.*

```
haetae_keygen_key_ref_sliceE
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_key_ref_sliceE sd :
  haetae_keygen_key sd =
  haetae_keygen_seedbuf_key
    (haetae_reference_keygen_xof256_seedbuf sd).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.94** (EasyCrypt line 1555). *EasyCrypt name.*

```
haetae_keygen_rho_prime_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_rho_prime_size sd :
  size (haetae_keygen_rho_prime sd) = seedbytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_rho_prime
haetae_keygen_seedbuf_rho_prime_size
```

**Theorem 13.95** (EasyCrypt line 1561). *EasyCrypt name.*

```
haetae_keygen_sigma_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_sigma_size sd :
  size (haetae_keygen_sigma sd) = crhbytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_seedbuf_sigma_size
haetae_keygen_sigma
```

**Theorem 13.96** (EasyCrypt line 1567). *EasyCrypt name.*

```
haetae_keygen_key_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_key_size sd :
  size (haetae_keygen_key sd) = seedbytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_key
haetae_keygen_seedbuf_key_size
```

**Theorem 13.97** (EasyCrypt line 1573). *EasyCrypt name.*

```
haetae_keygen_sampler_seed_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_sampler_seed_size sigma counter :
  size (haetae_keygen_sampler_seed sigma counter) = seedbytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.98** (EasyCrypt line 1577). *EasyCrypt name.*

```
haetae_keygen_counter_next_gt
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_counter_next_gt counter md :
  counter < haetae_keygen_counter_next md counter.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.99** (EasyCrypt line 1581). *EasyCrypt name.*

```
public_matrix_seed_size
```

**Checked EasyCrypt statement.**

```
lemma public_matrix_seed_size md sd :
  size (public_matrix_seed md sd) = mode_k md.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.100** (EasyCrypt line 1585). *EasyCrypt name.*

```
public_matrix_seed_rows_wf
```

**Checked EasyCrypt statement.**

```
lemma public_matrix_seed_rows_wf md sd :
  all (polyvecl_wf md) (public_matrix_seed md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
i_rng
mkseqP
public_key_seed_polyvecl_wf
public_matrix_seed
row
```

**Theorem 13.101** (EasyCrypt line 1593). *EasyCrypt name.*

```
public_secret_vector_seed_wf
```

*Checked EasyCrypt statement.*

```
lemma public_secret_vector_seed_wf md sd :  
  polyvecl_wf md (public_secret_vector_seed md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.102** (EasyCrypt line 1597). *EasyCrypt name.*

```
public_error_vector_seed_wf
```

*Checked EasyCrypt statement.*

```
lemma public_error_vector_seed_wf md sd :  
  polyveck_wf md (public_error_vector_seed md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.103** (EasyCrypt line 1601). *EasyCrypt name.*

```
public_rounding_vector_seed_wf
```

*Checked EasyCrypt statement.*

```
lemma public_rounding_vector_seed_wf md sd :  
  polyveck_wf md (public_rounding_vector_seed md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.104** (EasyCrypt line 1605). *EasyCrypt name.*

```
haetae_reference_polymatkm_expand_matA_size
```

*Checked EasyCrypt statement.*

```
lemma haetae_reference_polymatkm_expand_matA_size md rhoprime :  
  size (haetae_reference_polymatkm_expand_matA md rhoprime) = mode_k md.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_polymatkm_expand_matA
public_matrix_seed_size
```

**Theorem 13.105** (EasyCrypt line 1611). *EasyCrypt name.*

```
haetae_reference_polymatkm_expand_matA_rows_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_polymatkm_expand_matA_rows_wf md rhoprime :
  all (polyvecl_wf md)
    (haetae_reference_polymatkm_expand_matA md rhoprime).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_polymatkm_expand_matA
public_matrix_seed_rows_wf
```

**Theorem 13.106** (EasyCrypt line 1619). *EasyCrypt name.*

```
haetae_reference_polyveck_expand_vecA_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_polyveck_expand_vecA_wf md rhoprime :
  polyveck_wf md (haetae_reference_polyveck_expand_vecA md rhoprime).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_polyveck_expand_vecA
public_rounding_vector_seed_wf
```

**Theorem 13.107** (EasyCrypt line 1626). *EasyCrypt name.*

```
haetae_reference_polyvecmk_expand_S_sl_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_polyvecmk_expand_S_s1_wf md sigma counter :
  polyvecl_wf md
    (haetae_reference_polyvecmk_expand_S_s1 md sigma counter).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_polyvecmk_expand_S_s1
public_secret_vector_seed_wf
```

**Theorem 13.108** (EasyCrypt line 1634). *EasyCrypt* name.

```
haetae_reference_polyvecmk_expand_S_s2_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_polyvecmk_expand_S_s2_wf md sigma counter :
  polyveck_wf md
    (haetae_reference_polyvecmk_expand_S_s2 md sigma counter).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_polyvecmk_expand_S_s2
public_error_vector_seed_wf
```

**Theorem 13.109** (EasyCrypt line 1642). *EasyCrypt* name.

```
haetae_reference_keygen_secret_vector_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_secret_vector_wf md sd :
  polyvecl_wf md (haetae_reference_keygen_secret_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_secret_vector
haetae_reference_polyvecmk_expand_S_s1_wf
```

**Theorem 13.110** (EasyCrypt line 1649). *EasyCrypt name.*

```
haetae_reference_keygen_error_vector_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_error_vector_wf md sd :
  polyveck_wf md (haetae_reference_keygen_error_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_error_vector
haetae_reference_polyvecmk_expand_S_s2_wf
```

**Theorem 13.111** (EasyCrypt line 1656). *EasyCrypt name.*

```
haetae_reference_keygen_raw_public_vector_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_raw_public_vector_wf md sd :
  polyveck_wf md (haetae_reference_keygen_raw_public_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_raw_public_vector
polyveck_add_wf
```

**Theorem 13.112** (EasyCrypt line 1663). *EasyCrypt name.*

```
haetae_reference_keygen_raw_public_vector_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_raw_public_vector_coeffs_q_bound md sd :
  polyveck_coeffs_q_bound md
  (haetae_reference_keygen_raw_public_vector md sd).
```



**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_raw_public_vector
polyveck_add_coeffs_q_bound
```

**Theorem 13.113** (EasyCrypt line 1671). *EasyCrypt name*.

```
haetae_reference_keygen_mode23_precompose_vector_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_mode23_precompose_vector_wf md sd :
  polyveck_wf md
  (haetae_reference_keygen_mode23_precompose_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_mode23_precompose_vector
polyveck_add_wf
```

**Theorem 13.114** (EasyCrypt line 1679). *EasyCrypt name*.

```
haetae_reference_keygen_mode23_precompose_vector_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_mode23_precompose_vector_coeffs_q_bound
  md sd :
  polyveck_coeffs_q_bound md
  (haetae_reference_keygen_mode23_precompose_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_mode23_precompose_vector
```

```
polyveck_add_coeffs_q_bound
```

**Theorem 13.115** (EasyCrypt line 1688). *EasyCrypt name.*

```
haetae_reference_keygen_mode23_public_vector_polyq_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_mode23_public_vector_polyq_wf md sd :  
  (md = Mode2 ∨ md = Mode3) =>  
    haetae_polyveck_polyq_coeffs_wf md  
    (haetae_reference_keygen_mode23_public_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_mode23_predecompose_vector_coeffs_q_bound  
haetae_reference_keygen_mode23_public_vector  
md23  
polyveck_vk_highbits_mode23_polyq_wf
```

**Theorem 13.116** (EasyCrypt line 1699). *EasyCrypt name.*

```
haetae_reference_keygen_mode23_adjusted_error_vector_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_mode23_adjusted_error_vector_wf md sd :  
  polyveck_wf md  
    (haetae_reference_keygen_mode23_adjusted_error_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_mode23_adjusted_error_vector  
polyveck_sub_wf
```

**Theorem 13.117** (EasyCrypt line 1707). *EasyCrypt name.*

```
haetae_reference_keygen_mode5_public_vector_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_mode5_public_vector_wf sd :  
  polyveck_wf Mode5
```

```
(haetae_reference_keygen_mode5_public_vector Mode5 sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_mode5_public_vector
polyveck_neg_wf
```

**Theorem 13.118** (*EasyCrypt* line 1715). *EasyCrypt* name.

```
haetae_reference_keygen_mode5_public_vector_coeffs_q_bound
```

**Checked *EasyCrypt* statement.**

```
lemma haetae_reference_keygen_mode5_public_vector_coeffs_q_bound sd :
  polyveck_coeffs_q_bound Mode5
  (haetae_reference_keygen_mode5_public_vector Mode5 sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_mode5_public_vector
polyveck_neg_coeffs_q_bound
```

**Theorem 13.119** (*EasyCrypt* line 1723). *EasyCrypt* name.

```
haetae_reference_keygen_mode5_public_vector_polyq_wf
```

**Checked *EasyCrypt* statement.**

```
lemma haetae_reference_keygen_mode5_public_vector_polyq_wf sd :
  haetae_polyveck_polyq_coeffs_wf Mode5
  (haetae_reference_keygen_mode5_public_vector Mode5 sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_mode5_public_vector_coeffs_q_bound
haetae_reference_keygen_mode5_public_vector_wf
polyveck_q_bound_mode5_polyq_wf
```

**Theorem 13.120** (EasyCrypt line 1732). *EasyCrypt name.*

```
haetae_keygen_raw_public_vector_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_raw_public_vector_wf md sd :
  polyveck_wf md (haetae_keygen_raw_public_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.121** (EasyCrypt line 1736). *EasyCrypt name.*

```
haetae_keygen_raw_public_vector_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_raw_public_vector_coeffs_q_bound md sd :
  polyveck_coeffs_q_bound md (haetae_keygen_raw_public_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_raw_public_vector
polyveck_add_coeffs_q_bound
```

**Theorem 13.122** (EasyCrypt line 1743). *EasyCrypt name.*

```
haetae_keygen_mode23_predecompose_vector_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_mode23_predecompose_vector_wf md sd :
  polyveck_wf md (haetae_keygen_mode23_predecompose_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_mode23_predecompose_vector
polyveck_add_wf
```

**Theorem 13.123** (EasyCrypt line 1750). *EasyCrypt name*.

```
haetae_keygen_mode23_predecompose_vector_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_mode23_predecompose_vector_coeffs_q_bound md sd :
  polyveck_coeffs_q_bound md
  (haetae_keygen_mode23_predecompose_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_mode23_predecompose_vector
polyveck_add_coeffs_q_bound
```

**Theorem 13.124** (EasyCrypt line 1758). *EasyCrypt name*.

```
haetae_keygen_mode23_public_vector_polyq_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_mode23_public_vector_polyq_wf md sd :
  (md = Mode2  $\vee$  md = Mode3) =>
  haetae_polyveck_polyq_coeffs_wf md
  (haetae_keygen_mode23_public_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_mode23_predecompose_vector_coeffs_q_bound
haetae_keygen_mode23_public_vector
md23
polyveck_vk_highbits_mode23_polyq_wf
```

**Theorem 13.125** (EasyCrypt line 1769). *EasyCrypt name*.

```
haetae_keygen_mode23_rounding_lowbits_wf
```

### Checked EasyCrypt statement.

```
lemma haetae_keygen_mode23_rounding_lowbits_wf md sd :  
  polyveck_wf md (haetae_keygen_mode23_rounding_lowbits md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_mode23_rounding_lowbits  
polyveck_vk_lowbits_wf
```

### Theorem 13.126 (EasyCrypt line 1776). EasyCrypt name.

```
haetae_keygen_mode23_adjusted_error_vector_wf
```

### Checked EasyCrypt statement.

```
lemma haetae_keygen_mode23_adjusted_error_vector_wf md sd :  
  polyveck_wf md (haetae_keygen_mode23_adjusted_error_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_mode23_adjusted_error_vector  
polyveck_sub_wf
```

### Theorem 13.127 (EasyCrypt line 1783). EasyCrypt name.

```
haetae_keygen_mode5_public_vector_wf
```

### Checked EasyCrypt statement.

```
lemma haetae_keygen_mode5_public_vector_wf sd :  
  polyveck_wf Mode5 (haetae_keygen_mode5_public_vector Mode5 sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_mode5_public_vector
polyveck_neg_wf
```

**Theorem 13.128** (EasyCrypt line 1790). *EasyCrypt name.*

```
haetae_keygen_mode5_public_vector_coeffs_q_bound
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_mode5_public_vector_coeffs_q_bound sd :
  polyveck_coeffs_q_bound Mode5
  (haetae_keygen_mode5_public_vector Mode5 sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_mode5_public_vector
polyveck_neg_coeffs_q_bound
```

**Theorem 13.129** (EasyCrypt line 1798). *EasyCrypt name.*

```
haetae_keygen_mode5_public_vector_polyq_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_mode5_public_vector_polyq_wf sd :
  haetae_polyveck_polyq_coeffs_wf Mode5
  (haetae_keygen_mode5_public_vector Mode5 sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_mode5_public_vector_coeffs_q_bound
haetae_keygen_mode5_public_vector_wf
polyveck_q_bound_mode5_polyq_wf
```

**Theorem 13.130** (EasyCrypt line 1807). *EasyCrypt name.*

```
haetae_public_key_vector_from_relation_polyq_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_public_key_vector_from_relation_polyq_wf md sd s e :
  haetae_polyveck_polyq_coeffs_wf md
  (haetae_public_key_vector_from_relation md sd s e).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_public_key_vector_from_relation
md
polyveck_add_coeffs_q_bound
polyveck_neg_coeffs_q_bound
polyveck_neg_wf
polyveck_q_bound_mode5_polyq_wf
polyveck_vk_highbits_mode23_polyq_wf
```

**Theorem 13.131** (EasyCrypt line 1824). *EasyCrypt name.*

```
haetae_reference_keygen_public_vector_polyq_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_public_vector_polyq_wf md sd :
  haetae_polyveck_polyq_coeffs_wf md
    (haetae_reference_keygen_public_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_public_key_vector_from_relation_polyq_wf
haetae_reference_keygen_public_vector
```

**Theorem 13.132** (EasyCrypt line 1832). *EasyCrypt name.*

```
haetae_keygen_public_vector_polyq_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_public_vector_polyq_wf md sd :
  haetae_polyveck_polyq_coeffs_wf md (haetae_keygen_public_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:



```
haetae_keygen_public_vector
haetae_public_key_vector_from_relation_polyq_wf
```

**Theorem 13.133** (EasyCrypt line 1839). *EasyCrypt name.*

```
public_key_vector_seed_wf
```

**Checked EasyCrypt statement.**

```
lemma public_key_vector_seed_wf md sd :
  polyveck_wf md (public_key_vector_seed md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_public_vector_polyq_wf
haetae_polyveck_polyq_coeffs_wf
md
public_key_vector_seed
sd
```

**Theorem 13.134** (EasyCrypt line 1846). *EasyCrypt name.*

```
public_key_mode23_column_wf
```

**Checked EasyCrypt statement.**

```
lemma public_key_mode23_column_wf md pk :
  polyveck_wf md (public_key_mode23_column md pk).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.135** (EasyCrypt line 1851). *EasyCrypt name.*

```
haetae_verification_column_from_unpacked_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_verification_column_from_unpacked_wf md sd b :
  polyveck_wf md (haetae_verification_column_from_unpacked md sd b).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `case`, `rewrite`, `apply`, `split`. Named identifiers syntactically cited in the proof block:

```
allP
haetae_mode23_verification_column
haetae_mode5_verification_column
haetae_verification_column_from_unpacked
md
mkseqP
poly_normalize_wf
polyveck_double_wf
polyveck_wf
size_mkseq
trivial
```

**Theorem 13.136** (EasyCrypt line 1865). *EasyCrypt name.*

```
public_key_verification_column_wf
```

**Checked EasyCrypt statement.**

```
lemma public_key_verification_column_wf md pk :
  polyveck_wf md (public_key_verification_column md pk).
```

**Formal mathematical reading.** *Mathematical theorem.* This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** *Use in this chapter.* It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `case`, `rewrite`, `apply`, `split`. Named identifiers syntactically cited in the proof block:

```
allP
md
mkseqP
poly_normalize_wf
polyveck_wf
public_key_mode23_column_wf
public_key_mode5_column
public_key_verification_column
size_mkseq
trivial
```

**Theorem 13.137** (EasyCrypt line 1879). *EasyCrypt name.*

```
public_verification_matrix_size
```

**Checked EasyCrypt statement.**

```
lemma public_verification_matrix_size md pk :
  size (public_verification_matrix md pk) = mode_k md.
```

**Formal mathematical reading.** *Mathematical theorem.* This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** *Use in this chapter.* It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*. Named identifiers syntactically cited in the proof block:

```
haetae_verification_matrix_from_unpacked
md
packed_haetae_verification_matrix
public_verification_matrix
size_mkseq
```

**Theorem 13.138** (EasyCrypt line 1886). *EasyCrypt name*.

```
public_verification_matrix_rows_wf
```

**Checked EasyCrypt statement.**

```
lemma public_verification_matrix_rows_wf md pk :
  all (polyvecl_wf md) (public_verification_matrix md pk).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*, *split*, *smt*, *case*, *have*. Named identifiers syntactically cited in the proof block:

```
allP
col_wf
haetae_verification_column_from_unpacked_wf
haetae_verification_matrix_from_unpacked
i_rng
j0
md
mkseqP
mode_l_gt0
packed_haetae_verification_matrix
pk
poly_double_wf
polyveck_wf_nth
polyvecl_wf
public_verification_matrix
row
size_mkseq
```

**Theorem 13.139** (EasyCrypt line 1903). *EasyCrypt name*.

```
challenge_embedding_wf
```

**Checked EasyCrypt statement.**

```
lemma challenge_embedding_wf md ch :
  challenge_wf ch => polyvecl_wf md (challenge_embedding md ch).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *case*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
allP
ch_wf
challenge_embedding
challenge_wf
i0
mδ
mkseqP
poly_zero_wf
polyvecl_wf
size_mkseq
```

**Theorem 13.140** (EasyCrypt line 1916). *EasyCrypt* name.

```
public_key_of_secret_wf
```

**Checked EasyCrypt statement.**

```
lemma public_key_of_secret_wf mδ sk :
  polyvecl_wf mδ (public_key_of_secret mδ sk). '2.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.141** (EasyCrypt line 1920). *EasyCrypt* name.

```
haetae_pack_seed_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_pack_seed_size sd :
  size (haetae_pack_seed sd) = seedbytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.142** (EasyCrypt line 1924). *EasyCrypt* name.

```
haetae_unpack_seed_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_seed_size enc :
  size (haetae_unpack_seed enc) = seedbytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.143** (EasyCrypt line 1928). *EasyCrypt name.*

```
haetae_unpack_seed_pack
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_seed_pack sd :
  size sd = seedbytes =>
  haetae_unpack_seed (haetae_pack_seed sd) = sd.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
eq_from_nth
haetae_pack_seed
haetae_unpack_seed
i_rng
nth_mkseq
sd_sz
seedbytes
size_mkseq
```

**Theorem 13.144** (EasyCrypt line 1942). *EasyCrypt name.*

```
haetae_unpack_seed_pack_cat
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_seed_pack_cat sd tail :
  size sd = seedbytes =>
  haetae_unpack_seed (haetae_pack_seed sd ++ tail) = sd.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
eq_from_nth
haetae_pack_seed
haetae_unpack_seed
i_rng
nth_cat
nth_mkseq
sd_sz
```

```
seedbytes
size_mkseq
```

**Theorem 13.145** (EasyCrypt line 1955). *EasyCrypt name.*

```
haetae_polyq_coeff_bound_gt0
```

**Checked EasyCrypt statement.**

```
lemma haetae_polyq_coeff_bound_gt0 md :
  0 < haetae_polyq_coeff_bound md.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: **case**, **rewrite**. Named identifiers syntactically cited in the proof block:

```
haetae_polyq_coeff_bound
haetae_polyq_mode23_bound
haetae_polyq_mode5_bound
md
```

**Theorem 13.146** (EasyCrypt line 1962). *EasyCrypt name.*

```
haetae_byte_norm_bounds
```

**Checked EasyCrypt statement.**

```
lemma haetae_byte_norm_bounds x :
  0 <= haetae_byte_norm x < haetae_byte_modulus.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.147** (EasyCrypt line 1966). *EasyCrypt name.*

```
haetae_byte_norm_small
```

**Checked EasyCrypt statement.**

```
lemma haetae_byte_norm_small x :
  0 <= x < haetae_byte_modulus => haetae_byte_norm x = x.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.148** (EasyCrypt line 1970). *EasyCrypt name.*

```
haetae_byte_norm_idempotent
```

**Checked EasyCrypt statement.**

```
lemma haetae_byte_norm_idempotent x :  
  haetae_byte_norm (haetae_byte_norm x) = haetae_byte_norm x.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.149** (EasyCrypt line 1974). **EasyCrypt name.**

```
haetae_unpack15_coeff0
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack15_coeff0 c0 c1 :  
  0 <= c0 < 32768 =>  
  0 <= c1 < 32768 =>  
  haetae_byte_norm c0 +  
    256 * (haetae_byte_norm ((c0 %/ 256) %% 128 + (c1 %% 2) * 128) %% 128) =  
  c0.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.150** (EasyCrypt line 1982). **EasyCrypt name.**

```
haetae_unpack15_coeff1
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack15_coeff1 c0 c1 c2 :  
  0 <= c0 < 32768 =>  
  0 <= c1 < 32768 =>  
  0 <= c2 < 32768 =>  
  ((haetae_byte_norm ((c0 %/ 256) %% 128 + (c1 %% 2) * 128) %/ 128) %% 2) +  
    2 * haetae_byte_norm (c1 %/ 2) +  
    512 * (haetae_byte_norm ((c1 %/ 512) %% 64 + (c2 %% 4) * 64) %% 64) =  
  c1.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.151** (EasyCrypt line 1992). **EasyCrypt name.**

```
haetae_unpack15_coeff2
```

**Checked EasyCrypt statement.**

```

lemma haetae_unpack15_coeff2 c1 c2 c3 :
  0 <= c1 < 32768 =>
  0 <= c2 < 32768 =>
  0 <= c3 < 32768 =>
  ((haetae_byte_norm ((c1 %/ 512) %% 64 + (c2 %% 4) * 64) %/ 64) %% 4) +
  4 * haetae_byte_norm (c2 %/ 4) +
  1024 * (haetae_byte_norm ((c2 %/ 1024) %% 32 + (c3 %% 8) * 32) %% 32) =
  c2.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.152** (EasyCrypt line 2002). *EasyCrypt* name.

```
haetae_unpack15_coeff3
```

**Checked EasyCrypt statement.**

```

lemma haetae_unpack15_coeff3 c2 c3 c4 :
  0 <= c2 < 32768 =>
  0 <= c3 < 32768 =>
  0 <= c4 < 32768 =>
  ((haetae_byte_norm ((c2 %/ 1024) %% 32 + (c3 %% 8) * 32) %/ 32) %% 8) +
  8 * haetae_byte_norm (c3 %/ 8) +
  2048 * (haetae_byte_norm ((c3 %/ 2048) %% 16 + (c4 %% 16) * 16) %% 16) =
  c3.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.153** (EasyCrypt line 2012). *EasyCrypt* name.

```
haetae_unpack15_coeff4
```

**Checked EasyCrypt statement.**

```

lemma haetae_unpack15_coeff4 c3 c4 c5 :
  0 <= c3 < 32768 =>
  0 <= c4 < 32768 =>
  0 <= c5 < 32768 =>
  ((haetae_byte_norm ((c3 %/ 2048) %% 16 + (c4 %% 16) * 16) %/ 16) %% 16) +
  16 * haetae_byte_norm (c4 %/ 16) +
  4096 * (haetae_byte_norm ((c4 %/ 4096) %% 8 + (c5 %% 32) * 8) %% 8) =
  c4.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.154** (EasyCrypt line 2022). *EasyCrypt* name.

```
haetae_unpack15_coeff5
```



**Checked EasyCrypt statement.**

```
lemma haetae_unpack15_coeff5 c4 c5 c6 :
  0 <= c4 < 32768 =>
  0 <= c5 < 32768 =>
  0 <= c6 < 32768 =>
  ((haetae_byte_norm ((c4 %/ 4096) %% 8 + (c5 %% 32) * 8) %/ 8) %% 32) +
  32 * haetae_byte_norm (c5 %/ 32) +
  8192 * (haetae_byte_norm ((c5 %/ 8192) %% 4 + (c6 %% 64) * 4) %% 4) =
  c5.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.155** (EasyCrypt line 2032). **EasyCrypt name.**

haetae\_unpack15\_coeff6

**Checked EasyCrypt statement.**

```
lemma haetae_unpack15_coeff6 c5 c6 c7 :
  0 <= c5 < 32768 =>
  0 <= c6 < 32768 =>
  0 <= c7 < 32768 =>
  ((haetae_byte_norm ((c5 %/ 8192) %% 4 + (c6 %% 64) * 4) %/ 4) %% 64) +
  64 * haetae_byte_norm (c6 %/ 64) +
  16384 * (haetae_byte_norm ((c6 %/ 16384) %% 2 + (c7 %% 128) * 2) %% 2) =
  c6.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.156** (EasyCrypt line 2042). **EasyCrypt name.**

haetae\_unpack15\_coeff7

**Checked EasyCrypt statement.**

```
lemma haetae_unpack15_coeff7 c6 c7 :
  0 <= c6 < 32768 =>
  0 <= c7 < 32768 =>
  ((haetae_byte_norm ((c6 %/ 16384) %% 2 + (c7 %% 128) * 2) %/ 2) %% 128) +
  128 * haetae_byte_norm (c7 %/ 128) =
  c7.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.157** (EasyCrypt line 2050). **EasyCrypt name.**

haetae\_pack\_poly\_q\_ref\_size

### Checked EasyCrypt statement.

```
lemma haetae_pack_poly_q_ref_size md p :
  size (haetae_pack_poly_q_ref md p) = mode_polyq_packedbytes md.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *smt*. Named identifiers syntactically cited in the proof block:

```
haetae_pack_poly_q_ref
md
size_mkseq
```

### Theorem 13.158 (EasyCrypt line 2057). *EasyCrypt* name.

```
haetae_unpack_poly_q_ref_at_wf
```

### Checked EasyCrypt statement.

```
lemma haetae_unpack_poly_q_ref_at_wf md enc offset :
  poly_wf (haetae_unpack_poly_q_ref_at md enc offset).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*, *smt*. Named identifiers syntactically cited in the proof block:

```
haetae_unpack_poly_q_ref_at
md
poly_wf
size_mkseq
```

### Theorem 13.159 (EasyCrypt line 2064). *EasyCrypt* name.

```
haetae_pack_polyveck_body_ref_size
```

### Checked EasyCrypt statement.

```
lemma haetae_pack_polyveck_body_ref_size md b :
  size (haetae_pack_polyveck_body_ref md b) =
    haetae_public_key_body_bytes md.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*. Named identifiers syntactically cited in the proof block:

```
haetae_pack_polyveck_body_ref
haetae_public_key_body_bytes
md
size_mkseq
```

**Theorem 13.160** (EasyCrypt line 2072). *EasyCrypt* name.

```
haetae_unpack_polyveck_body_ref_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_polyveck_body_ref_wf md enc offset :
  polyveck_wf md (haetae_unpack_polyveck_body_ref md enc offset).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
haetae_unpack_poly_q_ref_at_wf
haetae_unpack_polyveck_body_ref
md
mkseqP
polyveck_wf
size_mkseq
```

**Theorem 13.161** (EasyCrypt line 2082). *EasyCrypt* name.

```
haetae_pack_vk_ref_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_pack_vk_ref_size md b sd :
  size (haetae_pack_vk_ref md b sd) = haetae_public_key_bytes md.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*. Named identifiers syntactically cited in the proof block:

```
haetae_pack_polyveck_body_ref_size
haetae_pack_seed_size
haetae_pack_vk_ref
```

```
haetae_public_key_body_bytes
haetae_public_key_bytes
md
size_cat
```

**Theorem 13.162** (EasyCrypt line 2091). *EasyCrypt name.*

```
haetae_unpack_vk_public_key_ref_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_vk_public_key_ref_wf md enc :
  haetae_public_key_unpacked_wf md
  (haetae_unpack_vk_public_key_ref md enc).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_public_key_unpacked_wf
haetae_unpack_polyveck_body_ref_wf
haetae_unpack_seed_size
haetae_unpack_vk_public_key_ref
```

**Theorem 13.163** (EasyCrypt line 2100). *EasyCrypt name.*

```
nth_cat_size_plus
```

**Checked EasyCrypt statement.**

```
lemma nth_cat_size_plus ['a] (d : 'a) (xs ys : 'a list) j :
  0 <= j =>
  nth d (xs ++ ys) (size xs + j) = nth d ys j.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.164** (EasyCrypt line 2105). *EasyCrypt name.*

```
haetae_pack_polyveck_body_ref_nth
```

**Checked EasyCrypt statement.**

```
lemma haetae_pack_polyveck_body_ref_nth md b row k :
  0 <= row < mode_k md =>
  0 <= k < mode_polyq_packedbytes md =>
  nth 0 (haetae_pack_polyveck_body_ref md b)
  (row * mode_polyq_packedbytes md + k) =
  nth 0 (haetae_pack_poly_q_ref md (nth poly_zero b row)) k.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `smt`. Named identifiers syntactically cited in the proof block:

```
haetae_pack_poly_q_ref_size
haetae_pack_polyveck_body_ref
haetae_public_key_body_bytes
k_rng
mode_polyq_packedbytes_gt0
nth_mkseq
row_rng
```

**Theorem 13.165** (EasyCrypt line 2121). *EasyCrypt* name.

```
haetae_pack_polyveck_body_ref_mode5_nth
```

**Checked EasyCrypt statement.**

```
lemma haetae_pack_polyveck_body_ref_mode5_nth b row k :
  0 <= row < mode_k Mode5 =>
  0 <= k < mode_polyq_packedbytes Mode5 =>
  nth 0 (haetae_pack_polyveck_body_ref Mode5 b)
    (row * mode_polyq_packedbytes Mode5 + k) =
  nth 0 (haetae_pack_poly_q_ref Mode5 (nth poly_zero b row)) k.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `apply`. Named identifiers syntactically cited in the proof block:

```
haetae_pack_polyveck_body_ref_nth
```

**Theorem 13.166** (EasyCrypt line 2131). *EasyCrypt* name.

```
haetae_unpack_poly_q_mode23_coeff_at_cat
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_poly_q_mode23_coeff_at_cat prefix bytes blk ofs :
  0 <= blk =>
  haetae_unpack_poly_q_mode23_coeff_at
    (prefix ++ bytes) (size prefix) blk ofs =
  haetae_unpack_poly_q_mode23_coeff_at bytes 0 blk ofs.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
blk_ge0
haetae_unpack_poly_q_mode23_coeff_at
nth_cat
```

**Theorem 13.167** (EasyCrypt line 2142). *EasyCrypt name*.

```
haetae_polyq_mode23_coeff_rng
```

**Checked EasyCrypt statement.**

```
lemma haetae_polyq_mode23_coeff_rng md p j :
  (md = Mode2  $\vee$  md = Mode3) =>
  haetae_polyq_coeffs_wf md p =>
  0 <= j < n =>
  0 <= poly_coeff p j < 32768.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
haetae_polyq_coeff_bound
haetae_polyq_coeffs_wf
haetae_polyq_mode23_bound
j_rng
md23
p_rng
p_sz
p_wf
```

**Theorem 13.168** (EasyCrypt line 2155). *EasyCrypt name*.

```
haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs0
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs0 md p blk :
  (md = Mode2  $\vee$  md = Mode3) =>
  haetae_polyq_coeffs_wf md p =>
  0 <= blk < 32 =>
  haetae_unpack_poly_q_mode23_coeff_at
    (haetae_pack_poly_q_ref md p) 0 blk 0 =
  poly_coeff p (8 * blk + 0).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *have*, *smt*, *case*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
blk
blk_rng
c0_rng
c1_rng
haetae_byte_norm_idempotent
haetae_pack_poly_q_mode23_byte
haetae_pack_poly_q_ref
haetae_polyq_mode23_coeff_rng
haetae_unpack15_coeff0
haetae_unpack_poly_q_mode23_coeff_at
md
md23
nth_mkseq
p_wf
p_wf2
p_wf3
poly_coeff
size_mkseq
```

**Theorem 13.169** (EasyCrypt line 2182). *EasyCrypt name*.

```
haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs1
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs1 md p blk :
  (md = Mode2 \/\ md = Mode3) =>
  haetae_polyq_coeffs_wf md p =>
  0 <= blk < 32 =>
  haetae_unpack_poly_q_mode23_coeff_at
    (haetae_pack_poly_q_ref md p) 0 blk 1 =
  poly_coeff p (8 * blk + 1).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *have*, *smt*, *case*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
blk
blk_rng
c0_rng
c1_rng
c2_rng
haetae_byte_norm_idempotent
haetae_pack_poly_q_mode23_byte
haetae_pack_poly_q_ref
haetae_polyq_mode23_coeff_rng
haetae_unpack15_coeff1
haetae_unpack_poly_q_mode23_coeff_at
md
md23
nth_mkseq
p_wf
p_wf2
p_wf3
poly_coeff
```

```
size_mkseq
```

**Theorem 13.170** (EasyCrypt line 2211). *EasyCrypt name.*

```
haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs2
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs2 md p blk :  
  (md = Mode2  $\vee$  md = Mode3) =>  
    haetae_polyq_coeffs_wf md p =>  
      0 <= blk < 32 =>  
        haetae_unpack_poly_q_mode23_coeff_at  
          (haetae_pack_poly_q_ref md p) 0 blk 2 =  
          poly_coeff p (8 * blk + 2).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *smt*, *case*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
blk  
blk_rng  
c1_rng  
c2_rng  
c3_rng  
haetae_byte_norm_idempotent  
haetae_pack_poly_q_mode23_byte  
haetae_pack_poly_q_ref  
haetae_polyq_mode23_coeff_rng  
haetae_unpack15_coeff2  
haetae_unpack_poly_q_mode23_coeff_at  
md  
md23  
nth_mkseq  
p_wf  
p_wf2  
p_wf3  
poly_coeff  
size_mkseq
```

**Theorem 13.171** (EasyCrypt line 2240). *EasyCrypt name.*

```
haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs3
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs3 md p blk :  
  (md = Mode2  $\vee$  md = Mode3) =>  
    haetae_polyq_coeffs_wf md p =>  
      0 <= blk < 32 =>  
        haetae_unpack_poly_q_mode23_coeff_at  
          (haetae_pack_poly_q_ref md p) 0 blk 3 =  
          poly_coeff p (8 * blk + 3).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.



**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *smt*, *case*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
blk
blk_rng
c2_rng
c3_rng
c4_rng
haetae_byte_norm_idempotent
haetae_pack_poly_q_mode23_byte
haetae_pack_poly_q_ref
haetae_polyq_mode23_coeff_rng
haetae_unpack15_coeff3
haetae_unpack_poly_q_mode23_coeff_at
md
md23
nth_mkseq
p_wf
p_wf2
p_wf3
poly_coeff
size_mkseq
```

**Theorem 13.172** (EasyCrypt line 2269). *EasyCrypt* name.

```
haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs4
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs4 md p blk :
  (md = Mode2  $\vee$  md = Mode3) =>
  haetae_polyq_coeffs_wf md p =>
  0 <= blk < 32 =>
  haetae_unpack_poly_q_mode23_coeff_at
    (haetae_pack_poly_q_ref md p) 0 blk 4 =
  poly_coeff p (8 * blk + 4).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *smt*, *case*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
blk
blk_rng
c3_rng
c4_rng
c5_rng
haetae_byte_norm_idempotent
haetae_pack_poly_q_mode23_byte
haetae_pack_poly_q_ref
haetae_polyq_mode23_coeff_rng
haetae_unpack15_coeff4
haetae_unpack_poly_q_mode23_coeff_at
```

```

md
md23
nth_mkseq
p_wf
p_wf2
p_wf3
poly_coeff
size_mkseq

```

**Theorem 13.173** (EasyCrypt line 2298). *EasyCrypt name.*

```
haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs5
```

**Checked EasyCrypt statement.**

```

lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs5 md p blk :
  (md = Mode2  $\vee$  md = Mode3) =>
  haetae_polyq_coeffs_wf md p =>
  0 <= blk < 32 =>
  haetae_unpack_poly_q_mode23_coeff_at
    (haetae_pack_poly_q_ref md p) 0 blk 5 =
  poly_coeff p (8 * blk + 5).

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *smt*, *case*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

blk
blk_rng
c4_rng
c5_rng
c6_rng
haetae_byte_norm_idempotent
haetae_pack_poly_q_mode23_byte
haetae_pack_poly_q_ref
haetae_polyq_mode23_coeff_rng
haetae_unpack15_coeff5
haetae_unpack_poly_q_mode23_coeff_at
md
md23
nth_mkseq
p_wf
p_wf2
p_wf3
poly_coeff
size_mkseq

```

**Theorem 13.174** (EasyCrypt line 2327). *EasyCrypt name.*

```
haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs6
```

**Checked EasyCrypt statement.**

```

lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs6 md p blk :
  (md = Mode2  $\vee$  md = Mode3) =>
  haetae_polyq_coeffs_wf md p =>
  0 <= blk < 32 =>

```

```

haetae_unpack_poly_q_mode23_coeff_at
(haetae_pack_poly_q_ref md p) 0 blk 6 =
poly_coeff p (8 * blk + 6).

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *smt*, *case*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

blk
blk_rng
c5_rng
c6_rng
c7_rng
haetae_byte_norm_idempotent
haetae_pack_poly_q_mode23_byte
haetae_pack_poly_q_ref
haetae_polyq_mode23_coeff_rng
haetae_unpack15_coeff6
haetae_unpack_poly_q_mode23_coeff_at
md
md23
nth_mkseq
p_wf
p_wf2
p_wf3
poly_coeff
size_mkseq

```

**Theorem 13.175** (EasyCrypt line 2356). *EasyCrypt name*.

```

haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs7

```

**Checked EasyCrypt statement.**

```

lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs7 md p blk :
  (md = Mode2 \ / md = Mode3) =>
  haetae_polyq_coeffs_wf md p =>
  0 <= blk < 32 =>
  haetae_unpack_poly_q_mode23_coeff_at
    (haetae_pack_poly_q_ref md p) 0 blk 7 =
  poly_coeff p (8 * blk + 7).

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *smt*, *case*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

blk
blk_rng
c6_rng
c7_rng

```

```

haetae_byte_norm_idempotent
haetae_pack_poly_q_mode23_byte
haetae_pack_poly_q_ref
haetae_polyq_mode23_coeff_rng
haetae_unpack15_coeff7
haetae_unpack_poly_q_mode23_coeff_at
md
md23
nth_mkseq
p_wf
p_wf2
p_wf3
poly_coeff
size_mkseq

```

**Theorem 13.176** (EasyCrypt line 2383). *EasyCrypt name.*

```
haetae_unpack_poly_q_ref_pack_mode23_coeff
```

*Checked EasyCrypt statement.*

```

lemma haetae_unpack_poly_q_ref_pack_mode23_coeff md p i :
  (md = Mode2 ∨ md = Mode3) =>
  haetae_polyq_coeffs_wf md p =>
  0 <= i < n =>
  haetae_unpack_poly_q_mode23_coeff_at
    (haetae_pack_poly_q_ref md p) 0 (i % 8) (i %% 8) =
  poly_coeff p i.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*, *smt*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```

blk_rng
haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs0
haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs1
haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs2
haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs3
haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs4
haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs5
haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs6
haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs7
i_rng
md23
ofs0
ofs1
ofs2
ofs3
ofs4
ofs5
ofs6
ofs7
ofs_cases
p_wf
poly_coeff
rhs0
rhs1
rhs2

```

```

rhs3
rhs4
rhs5
rhs6
rhs7

```

**Theorem 13.177** (EasyCrypt line 2446). *EasyCrypt name.*

```
haetae_unpack_poly_q_ref_pack_mode23_at
```

*Checked EasyCrypt statement.*

```

lemma haetae_unpack_poly_q_ref_pack_mode23_at md prefix p :
  (md = Mode2 ∨ md = Mode3) =>
    haetae_polyq_coeffs_wf md p =>
      haetae_unpack_poly_q_ref_at md
        (prefix ++ haetae_pack_poly_q_ref md p) (size prefix) = p.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *case*, *rewrite*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```

eq_from_nth
haetae_polyq_coeffs_wf
haetae_unpack_poly_q_mode23_coeff_at_cat
haetae_unpack_poly_q_ref_at
haetae_unpack_poly_q_ref_pack_mode23_coeff
i_rng
md
md23
nth_mkseq
p_sz
p_wf
p_wf2
p_wf3
poly_wf
size_mkseq

```

**Theorem 13.178** (EasyCrypt line 2479). *EasyCrypt name.*

```
haetae_unpack_poly_q_mode23_coeff_at_body
```

*Checked EasyCrypt statement.*

```

lemma haetae_unpack_poly_q_mode23_coeff_at_body md prefix b row blk ofs :
  (md = Mode2 ∨ md = Mode3) =>
    0 <= row < mode_k md =>
      0 <= blk < 32 =>
        haetae_unpack_poly_q_mode23_coeff_at
          (prefix ++ haetae_pack_polyveck_body_ref md b)
            (size prefix + row * mode_polyq_packedbytes md) blk ofs =
          haetae_unpack_poly_q_mode23_coeff_at
            (haetae_pack_poly_q_ref md (nth poly_zero b row)) 0 blk ofs.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
blk_rng
haetae_pack_polyveck_body_ref_nth
haetae_unpack_poly_q_mode23_coeff_at
md23
mode_polyq_packedbytes_gt0
nth_cat
row_rng
```

**Theorem 13.179** (EasyCrypt line 2495). *EasyCrypt* name.

```
haetae_unpack_polyveck_body_ref_pack_mode23_coeff
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_polyveck_body_ref_pack_mode23_coeff md prefix b row i :
  (md = Mode2 ∨ md = Mode3) =>
  haetae_polyveck_polyq_coeffs_wf md b =>
  0 <= row < mode_k md =>
  0 <= i < n =>
  haetae_unpack_poly_q_mode23_coeff_at
    (prefix ++ haetae_pack_polyveck_body_ref md b)
    (size prefix + row * mode_polyq_packedbytes md) (i % 8) (i % 8) =
  poly_coeff (nth poly_zero b row) i.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
b_rng
b_wf
haetae_polyq_coeffs_wf
haetae_polyveck_polyq_coeffs_wf
haetae_unpack_poly_q_mode23_coeff_at_body
haetae_unpack_poly_q_ref_pack_mode23_coeff
i_rng
md
md23
nth
poly_zero
prefix
row
row_rng
row_wf
```

**Theorem 13.180** (EasyCrypt line 2518). *EasyCrypt* name.

```
haetae_unpack_polyveck_body_ref_pack_mode23
```

### Checked EasyCrypt statement.

```
lemma haetae_unpack_polyveck_body_ref_pack_mode23 md prefix b :  
  (md = Mode2 ∨ md = Mode3) =>  
    haetae_polyveck_polyq_coeffs_wf md b =>  
      haetae_unpack_polyveck_body_ref md  
        (prefix ++ haetae_pack_polyveck_body_ref md b)  
        (size prefix) = b.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*, *smt*, *case*, *have*. Named identifiers syntactically cited in the proof block:

```
b_rng  
b_sz  
b_wf  
b_wf2  
b_wf3  
eq_from_nth  
haetae_polyq_coeffs_wf  
haetae_polyveck_polyq_coeffs_wf  
haetae_unpack_poly_q_ref_at  
haetae_unpack_polyveck_body_ref  
haetae_unpack_polyveck_body_ref_pack_mode23_coeff  
i_rng  
md  
md23  
nth_mkseq  
poly_wf  
poly_zero  
polyveck_wf  
row  
row_p_wf  
row_rng  
row_rng2  
row_rng3  
row_sz  
size_mkseq
```

### Theorem 13.181 (EasyCrypt line 2567). *EasyCrypt* name.

```
haetae_pack_vk_ref_roundtrip_mode23
```

### Checked EasyCrypt statement.

```
lemma haetae_pack_vk_ref_roundtrip_mode23 md sd b :  
  (md = Mode2 ∨ md = Mode3) =>  
    size sd = seedbytes =>  
      haetae_polyveck_polyq_coeffs_wf md b =>  
        haetae_unpack_vk_public_key_ref md  
          (haetae_pack_vk_ref md b sd) = (sd, b).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*. Named identifiers syntactically cited in the proof block:

```
b_wf
haetae_pack_polyveck_body_ref
haetae_pack_seed
haetae_pack_seed_size
haetae_pack_vk_ref
haetae_unpack_polyveck_body_ref_pack_mode23
haetae_unpack_seed_pack_cat
haetae_unpack_vk_public_key_ref
md
md23
sd
sd_sz
seed_sz
seedbytes
size
```

**Theorem 13.182** (EasyCrypt line 2585). *EasyCrypt name.*

```
haetae_unpack_poly_q_ref_pack_mode5_coeff
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_poly_q_ref_pack_mode5_coeff p i :
  haetae_polyq_coeffs_wf Mode5 p =>
    0 <= i < n =>
      haetae_unpack_poly_q_mode5_coeff_at
        (haetae_pack_poly_q_ref Mode5 p) 0 i = poly_coeff p i.
```

**Formal mathematical reading.** *Mathematical theorem.* This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** *Use in this chapter.* It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*, *smt*. Named identifiers syntactically cited in the proof block:

```
ci_rng
haetae_byte_modulus
haetae_byte_norm
haetae_pack_poly_q_mode5_byte
haetae_pack_poly_q_ref
haetae_polyq_coeff_bound
haetae_polyq_coeffs_wf
haetae_polyq_mode5_bound
haetae_unpack_poly_q_mode5_coeff_at
i_rng
nth_mkseq
p_rng
p_sz
p_wf
poly_coeff
size_mkseq
```

**Theorem 13.183** (EasyCrypt line 2603). *EasyCrypt name.*



```
haetae_unpack_poly_q_ref_pack_mode5_at
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_poly_q_ref_pack_mode5_at prefix p :
  haetae_polyq_coeffs_wf Mode5 p =>
  haetae_unpack_poly_q_ref_at Mode5
  (prefix ++ haetae_pack_poly_q_ref Mode5 p) (size prefix) = p.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*, *smt*, *have*. Named identifiers syntactically cited in the proof block:

```
ci_rng
eq_from_nth
haetae_byte_modulus
haetae_byte_norm
haetae_pack_poly_q_mode5_byte
haetae_pack_poly_q_ref
haetae_polyq_coeff_bound
haetae_polyq_coeffs_wf
haetae_polyq_mode5_bound
haetae_unpack_poly_q_mode5_coeff_at
haetae_unpack_poly_q_ref_at
i_rng
nth_cat
nth_mkseq
p_rng
p_sz
p_wf
poly_coeff
poly_wf
size_mkseq
```

**Theorem 13.184** (EasyCrypt line 2628). *EasyCrypt name.*

```
haetae_unpack_polyveck_body_ref_pack_mode5_coeff
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_polyveck_body_ref_pack_mode5_coeff prefix b row i :
  haetae_polyveck_polyq_coeffs_wf Mode5 b =>
  0 <= row < mode_k Mode5 =>
  0 <= i < n =>
  haetae_unpack_poly_q_mode5_coeff_at
  (prefix ++ haetae_pack_polyveck_body_ref Mode5 b)
  (size prefix + row * mode_polyq_packedbytes Mode5) i =
  poly_coeff (nth poly_zero b row) i.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *have*, *rewrite*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
Mode5
b_rng
b_wf
catE
haetae_pack_poly_q_ref
haetae_pack_polyveck_body_ref
haetae_pack_polyveck_body_ref_mode5_nth
haetae_polyq_coeffs_wf
haetae_polyveck_polyq_coeffs_wf
haetae_unpack_poly_q_mode5_coeff_at
haetae_unpack_poly_q_ref_pack_mode5_coeff
hi
i_rng
idxE
k_rng
lo
mode_polyq_packedbytes
nth
nth_cat_size_plus
poly_zero
prefix
row
row_rng
row_wf
size
```

**Theorem 13.185** (EasyCrypt line 2684). *EasyCrypt name*.

```
haetae_unpack_polyveck_body_ref_pack_mode5
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_polyveck_body_ref_pack_mode5 prefix b :
  haetae_polyveck_polyq_coeffs_wf Mode5 b =>
  haetae_unpack_polyveck_body_ref Mode5
    (prefix ++ haetae_pack_polyveck_body_ref Mode5 b)
    (size prefix) = b.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*, *smt*, *have*. Named identifiers syntactically cited in the proof block:

```
b_rng
b_sz
b_wf
eq_from_nth
haetae_polyq_coeffs_wf
haetae_polyveck_polyq_coeffs_wf
haetae_unpack_poly_q_ref_at
haetae_unpack_polyveck_body_ref
haetae_unpack_polyveck_body_ref_pack_mode5_coeff
i_rng
nth_mkseq
poly_wf
poly_zero
```

```

polyveck_wf
row
row_p_wf
row_rng
row_sz
size_mkseq

```

**Theorem 13.186** (EasyCrypt line 2715). *EasyCrypt name.*

```
haetae_pack_vk_ref_roundtrip_mode5
```

*Checked EasyCrypt statement.*

```

lemma haetae_pack_vk_ref_roundtrip_mode5 sd b :
  size sd = seedbytes =>
    haetae_polyveck_polyq_coeffs_wf Mode5 b =>
      haetae_unpack_vk_public_key_ref Mode5
        (haetae_pack_vk_ref Mode5 b sd) = (sd, b).

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*. Named identifiers syntactically cited in the proof block:

```

Mode5
b_wf
haetae_pack_polyveck_body_ref
haetae_pack_seed
haetae_pack_seed_size
haetae_pack_vk_ref
haetae_unpack_polyveck_body_ref_pack_mode5
haetae_unpack_seed_pack_cat
haetae_unpack_vk_public_key_ref
sd
sd_sz
seed_sz
seedbytes
size

```

**Theorem 13.187** (EasyCrypt line 2732). *EasyCrypt name.*

```
haetae_public_key_ref_roundtrip
```

*Checked EasyCrypt statement.*

```

lemma haetae_public_key_ref_roundtrip md (pk : pkey) :
  size pk.'1 = seedbytes =>
    haetae_polyveck_polyq_coeffs_wf md pk.'2 =>
      haetae_unpack_public_key_bytes_ref md
        (haetae_pack_public_key_bytes_ref md pk) = pk.

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `case`, `rewrite`, `apply`. Named identifiers syntactically cited in the proof block:*

```
b_wf
haetae_pack_public_key_bytes_ref
haetae_pack_vk_ref_roundtrip_mode23
haetae_pack_vk_ref_roundtrip_mode5
haetae_unpack_public_key_bytes_ref
md
pk
sd
sd_sz
```

**Theorem 13.188** (EasyCrypt line 2755). ***EasyCrypt name.***

```
haetae_keygen_public_key_ref_roundtrip
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_public_key_ref_roundtrip md sd :
  size sd = seedbytes =>
    haetae_unpack_vk_public_key_ref md
      (haetae_pack_vk_ref md (haetae_keygen_public_vector md sd) sd) =
      packed_haetae_public_key md sd.
```

**Formal mathematical reading.** *Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `have`. Named identifiers syntactically cited in the proof block:*

```
haetae_keygen_public_vector
haetae_keygen_public_vector_polyq_wf
haetae_pack_public_key_bytes_ref
haetae_public_key_ref_roundtrip
haetae_unpack_public_key_bytes_ref
md
packed_haetae_public_key
sd
sd_sz
```

**Theorem 13.189** (EasyCrypt line 2771). ***EasyCrypt name.***

```
haetae_pack_polyveck_body_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_pack_polyveck_body_size md b :
  size (haetae_pack_polyveck_body md b) = haetae_public_key_body_bytes md.
```

**Formal mathematical reading.** *Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `case`. Named identifiers syntactically cited in the proof block:

```
haetae_pack_polyveck_body
haetae_public_key_body_bytes
md
size_mkseq
```

**Theorem 13.190** (EasyCrypt line 2778). *EasyCrypt name.*

```
haetae_pack_poly_q_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_pack_poly_q_size md p :
  size (haetae_pack_poly_q md p) = mode_polyq_packedbytes md.
```

**Formal mathematical reading.** *Mathematical theorem.* This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** *Use in this chapter.* It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** *No proof block was collected for this item.*

**Theorem 13.191** (EasyCrypt line 2782). *EasyCrypt name.*

```
haetae_unpack_poly_q_at_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_poly_q_at_wf md enc offset :
  poly_wf (haetae_unpack_poly_q_at md enc offset).
```

**Formal mathematical reading.** *Mathematical theorem.* This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** *Use in this chapter.* It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** *No proof block was collected for this item.*

**Theorem 13.192** (EasyCrypt line 2786). *EasyCrypt name.*

```
haetae_unpack_poly_q_pack_at
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_poly_q_pack_at md prefix p :
  poly_wf p =>
  haetae_unpack_poly_q_at md (prefix ++ haetae_pack_poly_q md p)
  (size prefix) = p.
```

**Formal mathematical reading.** *Mathematical theorem.* This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** *Use in this chapter.* It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
eq_from_nth
haetae_pack_poly_q
haetae_unpack_poly_q_at
i_rng
n_le_mode_polyq_packedbytes
nth_cat
nth_mkseq
p_wf
poly_coeff
poly_wf
size_mkseq
```

**Theorem 13.193** (EasyCrypt line 2803). *EasyCrypt* name.

```
haetae_unpack_polyveck_body_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_polyveck_body_wf md enc offset :
  polyveck_wf md (haetae_unpack_polyveck_body md enc offset).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
haetae_unpack_polyveck_body
md
mkseqP
poly_wf
polyveck_wf
size_mkseq
```

**Theorem 13.194** (EasyCrypt line 2813). *EasyCrypt* name.

```
haetae_body_index_div
```

**Checked EasyCrypt statement.**

```
lemma haetae_body_index_div md row col :
  0 <= row =>
  0 <= col < n =>
  (row * mode_polyq_packedbytes md + col)
  %/ mode_polyq_packedbytes md = row.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.195** (EasyCrypt line 2820). *EasyCrypt* name.

```
haetae_body_index_mod
```

**Checked EasyCrypt statement.**

```
lemma haetae_body_index_mod md row col :  
  0 <= row =>  
  0 <= col < n =>  
  (row * mode_polyq_packedbytes md + col)  
  %% mode_polyq_packedbytes md = col.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.196** (EasyCrypt line 2827). *EasyCrypt name.*

```
haetae_body_index_lt
```

**Checked EasyCrypt statement.**

```
lemma haetae_body_index_lt md row col :  
  0 <= row < mode_k md =>  
  0 <= col < n =>  
  row * mode_polyq_packedbytes md + col <  
  haetae_public_key_body_bytes md.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
haetae_public_key_body_bytes  
mode_polyq_packedbytes_gt0  
n_le_mode_polyq_packedbytes
```

**Theorem 13.197** (EasyCrypt line 2837). *EasyCrypt name.*

```
haetae_unpack_polyveck_body_pack_at
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_polyveck_body_pack_at md prefix b :  
  polyveck_wf md b =>  
  haetae_unpack_polyveck_body md  
  (prefix ++ haetae_pack_polyveck_body md b) (size prefix) = b.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*, *apply*, *smt*. Named identifiers syntactically cited in the proof block:

```
b_wf
col
col_rng
eq_from_nth
haetae_body_index_div
haetae_body_index_lt
haetae_body_index_mod
haetae_pack_polyveck_body
haetae_unpack_polyveck_body
mode_polyq_packedbytes_gt0
n_le_mode_polyq_packedbytes
nth_cat
nth_mkseq
poly_coeff
poly_zero
polyveck_wf_nth
row
row_rng
size_mkseq
```

**Theorem 13.198** (EasyCrypt line 2860). *EasyCrypt name*.

```
haetae_unpack_polyveck_body_pack
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_polyveck_body_pack md sd b :
  polyveck_wf md b =>
  haetae_unpack_polyveck_body md
    (haetae_pack_seed sd ++ haetae_pack_polyveck_body md b)
  seedbytes = b.
```

**Formal mathematical reading.** *Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.* Main tactics visible in the proof script: *have*, *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
b_wf
haetae_pack_seed_size
haetae_unpack_polyveck_body_pack_at
sd
seed_sz
```

**Theorem 13.199** (EasyCrypt line 2872). *EasyCrypt name*.

```
haetae_public_key_roundtrip
```

**Checked EasyCrypt statement.**

```
lemma haetae_public_key_roundtrip md pk :
  haetae_public_key_unpacked_wf md pk =>
  haetae_public_key_roundtrip_ok md pk.
```



**Formal mathematical reading.** *Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `case`, `rewrite`. Named identifiers syntactically cited in the proof block:*

```
b_wf
haetae_pack_polyveck_body
haetae_pack_public_key_bytes
haetae_pack_vk
haetae_public_key_roundtrip_ok
haetae_public_key_unpacked_wf
haetae_unpack_polyveck_body_pack
haetae_unpack_public_key_bytes
haetae_unpack_seed_pack_cat
haetae_unpack_vk_public_key
md
pk
sd
sd_sz
```

**Theorem 13.200** (EasyCrypt line 2886). *EasyCrypt name.*

```
haetae_pack_public_key_bytes_size
```

**Checked EasyCrypt statement.**

```
lemma haetae_pack_public_key_bytes_size md pk :
  size (haetae_pack_public_key_bytes md pk) = haetae_public_key_bytes md.
```

**Formal mathematical reading.** *Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
haetae_pack_polyveck_body_size
haetae_pack_public_key_bytes
haetae_pack_seed_size
haetae_pack_vk
haetae_public_key_bytes
mode_publickeybytes
size_cat
```

**Theorem 13.201** (EasyCrypt line 2894). *EasyCrypt name.*

```
haetae_pack_public_key_bytes_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_pack_public_key_bytes_wf md pk :
  haetae_public_key_encoding_wf md (haetae_pack_public_key_bytes md pk).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_pack_public_key_bytes_size
haetae_public_key_encoding_wf
```

**Theorem 13.202** (*EasyCrypt* line 2900). *EasyCrypt* name.

```
haetae_pack_public_key_bytes_ref_size
```

**Checked *EasyCrypt* statement.**

```
lemma haetae_pack_public_key_bytes_ref_size md pk :
  size (haetae_pack_public_key_bytes_ref md pk) =
    haetae_public_key_bytes md.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_pack_public_key_bytes_ref
haetae_pack_vk_ref_size
```

**Theorem 13.203** (*EasyCrypt* line 2907). *EasyCrypt* name.

```
haetae_pack_public_key_bytes_ref_wf
```

**Checked *EasyCrypt* statement.**

```
lemma haetae_pack_public_key_bytes_ref_wf md pk :
  haetae_public_key_encoding_wf md
    (haetae_pack_public_key_bytes_ref md pk).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_pack_public_key_bytes_ref_size
haetae_public_key_encoding_wf
```

**Theorem 13.204** (EasyCrypt line 2915). *EasyCrypt name.*

```
haetae_unpack_public_key_bytes_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_public_key_bytes_wf md enc :  
  haetae_public_key_unpacked_wf md  
  (haetae_unpack_public_key_bytes md enc).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*. Named identifiers syntactically cited in the proof block:

```
body_wf  
enc  
haetae_public_key_unpacked_wf  
haetae_unpack_polyveck_body_wf  
haetae_unpack_public_key_bytes  
haetae_unpack_seed_size  
haetae_unpack_vk_public_key  
md  
seed_sz  
seedbytes
```

**Theorem 13.205** (EasyCrypt line 2926). *EasyCrypt name.*

```
haetae_unpack_public_key_bytes_ref_wf
```

**Checked EasyCrypt statement.**

```
lemma haetae_unpack_public_key_bytes_ref_wf md enc :  
  haetae_public_key_unpacked_wf md  
  (haetae_unpack_public_key_bytes_ref md enc).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_unpack_public_key_bytes_ref  
haetae_unpack_vk_public_key_ref_wf
```

**Theorem 13.206** (EasyCrypt line 2934). *EasyCrypt name.*

```
concrete_haetae_keygen_packed_public_key_wf
```

**Checked EasyCrypt statement.**

```
lemma concrete_haetae_keygen_packed_public_key_wf md sd :
  haetae_public_key_encoding_wf md
  (concrete_haetae_keygen_packed_public_key md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
concrete_haetae_keygen_packed_public_key
haetae_pack_public_key_bytes_wf
```

**Theorem 13.207** (EasyCrypt line 2942). *EasyCrypt name.*

```
concrete_haetae_keygen_unpacked_public_key_wf
```

**Checked EasyCrypt statement.**

```
lemma concrete_haetae_keygen_unpacked_public_key_wf md sd :
  haetae_public_key_unpacked_wf md
  (concrete_haetae_keygen_unpacked_public_key md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
concrete_haetae_keygen_unpacked_public_key
haetae_unpack_public_key_bytes_wf
```

**Theorem 13.208** (EasyCrypt line 2950). *EasyCrypt name.*

```
concrete_haetae_keygen_packed_public_key_ref_wf
```

**Checked EasyCrypt statement.**

```
lemma concrete_haetae_keygen_packed_public_key_ref_wf md sd :
  haetae_public_key_encoding_wf md
  (concrete_haetae_keygen_packed_public_key_ref md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
concrete_haetae_keygen_packed_public_key_ref
haetae_pack_public_key_bytes_ref_wf
```

**Theorem 13.209** (EasyCrypt line 2958). *EasyCrypt name*.

```
concrete_haetae_keygen_unpacked_public_key_ref_wf
```

*Checked EasyCrypt statement.*

```
lemma concrete_haetae_keygen_unpacked_public_key_ref_wf md sd :
  haetae_public_key_unpacked_wf md
    (concrete_haetae_keygen_unpacked_public_key_ref md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
concrete_haetae_keygen_unpacked_public_key_ref
haetae_unpack_public_key_bytes_ref_wf
```

**Theorem 13.210** (EasyCrypt line 2966). *EasyCrypt name*.

```
packed_haetae_reference_public_key_wf
```

*Checked EasyCrypt statement.*

```
lemma packed_haetae_reference_public_key_wf md sd :
  polyveck_wf md (packed_haetae_reference_public_key md sd). '2.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_polyveck_polyq_coeffs_wf
haetae_reference_keygen_public_vector_polyq_wf
md
packed_haetae_reference_public_key
sd
```

**Theorem 13.211** (EasyCrypt line 2974). *EasyCrypt name*.

```
packed_haetae_reference_public_key_polyq_wf
```

*Checked EasyCrypt statement.*

```
lemma packed_haetae_reference_public_key_polyq_wf md sd :
  haetae_polyveck_polyq_coeffs_wf md
  (packed_haetae_reference_public_key md sd). '2.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_public_vector_polyq_wf
packed_haetae_reference_public_key
```

**Theorem 13.212** (EasyCrypt line 2982). *EasyCrypt name.*

```
packed_haetae_reference_public_key_unpacked_wf
```

**Checked EasyCrypt statement.**

```
lemma packed_haetae_reference_public_key_unpacked_wf md sd :
  haetae_public_key_unpacked_wf md
  (packed_haetae_reference_public_key md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*, *have*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_rho_prime_size
haetae_polyveck_polyq_coeffs_wf
haetae_public_key_unpacked_wf
haetae_reference_keygen_public_vector_polyq_wf
md
packed_haetae_reference_public_key
sd
```

**Theorem 13.213** (EasyCrypt line 2994). *EasyCrypt name.*

```
concrete_haetae_reference_keygen_packed_public_key_ref_wf
```

**Checked EasyCrypt statement.**

```
lemma concrete_haetae_reference_keygen_packed_public_key_ref_wf md sd :
  haetae_public_key_encoding_wf md
  (concrete_haetae_reference_keygen_packed_public_key_ref md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
concrete_haetae_reference_keygen_packed_public_key_ref
haetae_pack_public_key_bytes_ref_wf
```

**Theorem 13.214** (EasyCrypt line 3002). *EasyCrypt name.*

```
concrete_haetae_reference_keygen_unpacked_public_key_ref_wf
```

**Checked EasyCrypt statement.**

```
lemma concrete_haetae_reference_keygen_unpacked_public_key_ref_wf md sd :
  haetae_public_key_unpacked_wf md
  (concrete_haetae_reference_keygen_unpacked_public_key_ref md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
concrete_haetae_reference_keygen_unpacked_public_key_ref
haetae_unpack_public_key_bytes_ref_wf
```

**Theorem 13.215** (EasyCrypt line 3010). *EasyCrypt name.*

```
haetae_reference_keygen_public_key_ref_roundtrip
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_public_key_ref_roundtrip md sd :
  haetae_unpack_public_key_bytes_ref md
  (haetae_pack_public_key_bytes_ref md
   (packed_haetae_reference_public_key md sd)) =
  packed_haetae_reference_public_key md sd.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *apply*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_rho_prime_size
haetae_public_key_ref_roundtrip
packed_haetae_reference_public_key
packed_haetae_reference_public_key_polyq_wf
```

**Theorem 13.216** (EasyCrypt line 3022). *EasyCrypt name.*

```
public_key_of_secret_relation
```

**Checked EasyCrypt statement.**

```
lemma public_key_of_secret_relation md sk :
  public_key_relation md
    (public_key_of_secret md sk)
    (public_secret_vector_seed md sk. '1)
    (public_error_vector_seed md sk. '1).
```

**Formal mathematical reading.** *Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
haetae_keygen_error_vector
haetae_keygen_public_vector
haetae_keygen_secret_vector
public_key_of_secret
public_key_relation
public_key_vector_seed
```

**Theorem 13.217** (EasyCrypt line 3033). *EasyCrypt name.*

```
public_key_of_secret_equation_holds
```

**Checked EasyCrypt statement.**

```
lemma public_key_of_secret_equation_holds md sk :
  public_key_equation_holds md (public_key_of_secret md sk).
```

**Formal mathematical reading.** *Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
public_key_equation_holds
public_key_of_secret_relation
```

**Theorem 13.218** (EasyCrypt line 3171). *EasyCrypt name.*

```
deterministic_poly_wf
```

**Checked EasyCrypt statement.**

```
lemma deterministic_poly_wf tag src :
  poly_wf (deterministic_poly tag src).
```



**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.219** (EasyCrypt line 3175). *EasyCrypt name.*

```
deterministic_polyveck_wf
```

**Checked EasyCrypt statement.**

```
lemma deterministic_polyveck_wf md src :
  polyveck_wf md (deterministic_polyveck md src).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
deterministic_poly_wf
deterministic_polyveck
md
mkseqP
polyveck_wf
size_mkseq
```

**Theorem 13.220** (EasyCrypt line 3185). *EasyCrypt name.*

```
deterministic_unit_poly_wf
```

**Checked EasyCrypt statement.**

```
lemma deterministic_unit_poly_wf tag src :
  poly_wf (deterministic_unit_poly tag src).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.221** (EasyCrypt line 3189). *EasyCrypt name.*

```
deterministic_unit_poly_unit
```

**Checked EasyCrypt statement.**

```
lemma deterministic_unit_poly_unit tag src :
  poly_unit (deterministic_unit_poly tag src).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*, *case*. Named identifiers syntactically cited in the proof block:

```
allP
deterministic_unit_coeff
deterministic_unit_poly
deterministic_unit_poly_wf
mkseqP
nth
poly_unit
src
tag
unit_coeff_ok
```

**Theorem 13.222** (EasyCrypt line 3200). *EasyCrypt name*.

```
deterministic_unit_polyveck_wf
```

**Checked EasyCrypt statement.**

```
lemma deterministic_unit_polyveck_wf md tag src :
  polyveck_wf md (deterministic_unit_polyveck md tag src).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *case*, *apply*. Named identifiers syntactically cited in the proof block:

```
allP
deterministic_unit_poly_wf
deterministic_unit_polyveck
md
mkseqP
polyveck_wf
size_mkseq
```

**Theorem 13.223** (EasyCrypt line 3210). *EasyCrypt name*.

```
deterministic_unit_polyveck_unit
```

**Checked EasyCrypt statement.**

```
lemma deterministic_unit_polyveck_unit md tag src :
  polyveck_unit md (deterministic_unit_polyveck md tag src).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `split`, `apply`. Named identifiers syntactically cited in the proof block:*

```
allP
deterministic_unit_poly_unit
deterministic_unit_polyveck_wf
mkseqP
polyveck_unit
```

**Theorem 13.224** (EasyCrypt line 3220). *EasyCrypt name.*

```
deterministic_unit_polyveckl_wf
```

**Checked EasyCrypt statement.**

```
lemma deterministic_unit_polyveckl_wf md tag src :
  polyveckl_wf md (deterministic_unit_polyveckl md tag src).
```

**Formal mathematical reading.** *Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.*

**Role in the proof.** *Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `split`, `case`, `apply`. Named identifiers syntactically cited in the proof block:*

```
allP
deterministic_unit_poly_wf
deterministic_unit_polyveckl
md
mkseqP
polyveckl_wf
size_mkseq
```

**Theorem 13.225** (EasyCrypt line 3230). *EasyCrypt name.*

```
deterministic_unit_polyveckl_unit
```

**Checked EasyCrypt statement.**

```
lemma deterministic_unit_polyveckl_unit md tag src :
  polyveckl_unit md (deterministic_unit_polyveckl md tag src).
```

**Formal mathematical reading.** *Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `split`, `apply`. Named identifiers syntactically cited in the proof block:*

```
allP
deterministic_unit_poly_unit
```

```
deterministic_unit_polyvecl_wf
mkseqP
polyvecl_unit
```

**Theorem 13.226** (EasyCrypt line 3240). *EasyCrypt name.*

```
commitment_raw_wf
```

**Checked EasyCrypt statement.**

```
lemma commitment_raw_wf md sk m ctx coins :
  polyveck_wf md (commitment_raw md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
commitment_raw
deterministic_polyveck_wf
```

**Theorem 13.227** (EasyCrypt line 3246). *EasyCrypt name.*

```
commitment_highbits_wf
```

**Checked EasyCrypt statement.**

```
lemma commitment_highbits_wf md sk m ctx coins :
  polyveck_wf md (commitment_highbits md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
commitment_highbits
polyveck_add_wf
reconstructed_highbits
```

**Theorem 13.228** (EasyCrypt line 3253). *EasyCrypt name.*

```
commitment_lowbits_wf
```

**Checked EasyCrypt statement.**

```
lemma commitment_lowbits_wf md sk m ctx coins :
  poly_wf (commitment_lowbits md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
commitment_lowbits
poly_wf
```

**Theorem 13.229** (EasyCrypt line 3260). *EasyCrypt* name.

```
commitment_lowbits_entropy_token
```

**Checked *EasyCrypt* statement.**

```
lemma commitment_lowbits_entropy_token md sk m ctx coins :
  nth 0 (commitment_lowbits md sk m ctx coins) 0 =
  signing_entropy_token_of_coins coins.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*, *case*. Named identifiers syntactically cited in the proof block:

```
commitment_lowbits
n_gt0
nth_mkseq
```

**Theorem 13.230** (EasyCrypt line 3270). *EasyCrypt* name.

```
challenge_from_seed_wf
```

**Checked *EasyCrypt* statement.**

```
lemma challenge_from_seed_wf md src :
  challenge_wf (challenge_from_seed md src).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*, *case*. Named identifiers syntactically cited in the proof block:

```
allP
challenge_coeff_ok
```

```

challenge_from_seed
challenge_wf
md
mkseqP
mode_tau
nth
poly_wf
size_mkseq
src

```

**Theorem 13.231** (EasyCrypt line 3282). *EasyCrypt name.*

```
challenge_from_seed_sparse
```

*Checked EasyCrypt statement.*

```

lemma challenge_from_seed_sparse md src :
  challenge_sparse md (challenge_from_seed md src).

```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *case*. Named identifiers syntactically cited in the proof block:

```

challenge_from_seed
challenge_sparse
challenge_sparse_at
ge0_i
h_tau
lt_i
md
mode_tau
nth
nth_mkseq
size_mkseq
src
unit_coeff_ok

```

**Theorem 13.232** (EasyCrypt line 3297). *EasyCrypt name.*

```
challenge_hash_wf
```

*Checked EasyCrypt statement.*

```

lemma challenge_hash_wf md w1 w0 mu :
  challenge_wf (challenge_hash md w1 w0 mu).

```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.233** (EasyCrypt line 3301). *EasyCrypt name.*

```
challenge_hash_sparse
```

**Checked EasyCrypt statement.**

```
lemma challenge_hash_sparse md w1 w0 mu :  
  challenge_sparse md (challenge_hash md w1 w0 mu).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.234** (EasyCrypt line 3305). **EasyCrypt name.**

```
response_vector_wf
```

**Checked EasyCrypt statement.**

```
lemma response_vector_wf md sk m ctx coins :  
  polyvecl_wf md (response_vector md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
deterministic_unit_polyvecl_wf  
response_vector
```

**Theorem 13.235** (EasyCrypt line 3311). **EasyCrypt name.**

```
response_vector_unit
```

**Checked EasyCrypt statement.**

```
lemma response_vector_unit md sk m ctx coins :  
  polyvecl_unit md (response_vector md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
deterministic_unit_polyvecl_unit  
response_vector
```

**Theorem 13.236** (EasyCrypt line 3317). *EasyCrypt name.*

```
response_aux_vector_wf
```

**Checked EasyCrypt statement.**

```
lemma response_aux_vector_wf md sk m ct $\alpha$  coins :  
  polyveck_wf md (response_aux_vector md sk m ct $\alpha$  coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
deterministic_unit_polyveck_wf  
response_aux_vector
```

**Theorem 13.237** (EasyCrypt line 3323). *EasyCrypt name.*

```
response_aux_vector_unit
```

**Checked EasyCrypt statement.**

```
lemma response_aux_vector_unit md sk m ct $\alpha$  coins :  
  polyveck_unit md (response_aux_vector md sk m ct $\alpha$  coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
deterministic_unit_polyveck_unit  
response_aux_vector
```

**Theorem 13.238** (EasyCrypt line 3329). *EasyCrypt name.*

```
signing_sample_y1_wf
```

**Checked EasyCrypt statement.**

```
lemma signing_sample_y1_wf md sk m ct $\alpha$  coins :  
  polyvecl_wf md (signing_sample_y1 md sk m ct $\alpha$  coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.



**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
deterministic_unit_polyvecl_wf
signing_sample_y1
```

**Theorem 13.239** (EasyCrypt line 3335). *EasyCrypt name*.

```
signing_sample_y1_unit
```

**Checked EasyCrypt statement.**

```
lemma signing_sample_y1_unit md sk m ctx coins :
  polyvecl_unit md (signing_sample_y1 md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
deterministic_unit_polyvecl_unit
signing_sample_y1
```

**Theorem 13.240** (EasyCrypt line 3341). *EasyCrypt name*.

```
signing_sample_y2_wf
```

**Checked EasyCrypt statement.**

```
lemma signing_sample_y2_wf md sk m ctx coins :
  polyveck_wf md (signing_sample_y2 md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
deterministic_unit_polyveck_wf
signing_sample_y2
```

**Theorem 13.241** (EasyCrypt line 3347). *EasyCrypt name*.

```
signing_sample_y2_unit
```

**Checked EasyCrypt statement.**

```
lemma signing_sample_y2_unit md sk m ctx coins :
  polyveck_unit md (signing_sample_y2 md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
deterministic_unit_polyveck_unit
signing_sample_y2
```

**Theorem 13.242** (*EasyCrypt* line 3353). *EasyCrypt* name.

```
hint_vector_wf
```

**Checked *EasyCrypt* statement.**

```
lemma hint_vector_wf md sk m ctx coins :
  polyveck_wf md (hint_vector md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
deterministic_unit_polyveck_wf
hint_vector
```

**Theorem 13.243** (*EasyCrypt* line 3359). *EasyCrypt* name.

```
hint_vector_unit
```

**Checked *EasyCrypt* statement.**

```
lemma hint_vector_unit md sk m ctx coins :
  polyveck_unit md (hint_vector md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
deterministic_unit_polyveck_unit  
hint_vector
```

**Theorem 13.244** (EasyCrypt line 3365). *EasyCrypt name.*

```
message_hash_size
```

*Checked EasyCrypt statement.*

```
lemma message_hash_size pk ctx m :  
  size (message_hash pk ctx m) = crhbytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.245** (EasyCrypt line 3369). *EasyCrypt name.*

```
challenge_hash_size
```

*Checked EasyCrypt statement.*

```
lemma challenge_hash_size md w1 w0 mu :  
  size (challenge_hash md w1 w0 mu) = n.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.246** (EasyCrypt line 3407). *EasyCrypt name.*

```
keygen_internal_valid
```

*Checked EasyCrypt statement.*

```
lemma keygen_internal_valid md sd :  
  valid_keypair md (keygen_internal md sd).‘1 (keygen_internal md sd).‘2.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.247** (EasyCrypt line 3411). *EasyCrypt name.*

```
public_key_of_keygen
```

*Checked EasyCrypt statement.*

```
lemma public_key_of_keygen md sd :  
  public_key_of_secret md (keygen_internal md sd).‘2 =  
  (keygen_internal md sd).‘1.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.248** (EasyCrypt line 3416). *EasyCrypt* name.

```
public_key_vector_seed_keygen_equation
```

**Checked EasyCrypt statement.**

```
lemma public_key_vector_seed_keygen_equation md sd :
  public_key_vector_seed md sd =
    haetae_public_key_vector_from_relation md sd
      (haetae_keygen_secret_vector md sd)
      (haetae_keygen_error_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.249** (EasyCrypt line 3423). *EasyCrypt* name.

```
public_key_vector_seed_mode23_keygen_equation
```

**Checked EasyCrypt statement.**

```
lemma public_key_vector_seed_mode23_keygen_equation md sd :
  (md = Mode2 /\ md = Mode3) =>
    public_key_vector_seed md sd =
      polyveck_vk_highbits md
        (polyveck_add md
          (public_rounding_vector_seed md sd)
          (polyveck_add md
            (matrix_vec_mul md
              (haetae_keygen_a0_matrix md sd)
              (haetae_keygen_secret_vector md sd))
            (haetae_keygen_error_vector md sd))).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_a0_matrix
haetae_keygen_error_vector
haetae_keygen_public_vector
haetae_keygen_secret_vector
haetae_public_key_vector_from_relation
md
md23
public_key_vector_seed
```

**Theorem 13.250** (EasyCrypt line 3443). *EasyCrypt name.*

```
public_key_vector_seed_mode5_keygen_equation
```

**Checked EasyCrypt statement.**

```
lemma public_key_vector_seed_mode5_keygen_equation sd :
  public_key_vector_seed Mode5 sd =
  polyveck_neg Mode5
    (polyveck_double Mode5
      (polyveck_add Mode5
        (matrix_vec_mul Mode5
          (haetae_keygen_a0_matrix Mode5 sd)
          (haetae_keygen_secret_vector Mode5 sd))
        (haetae_keygen_error_vector Mode5 sd))).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_a0_matrix
haetae_keygen_error_vector
haetae_keygen_public_vector
haetae_keygen_secret_vector
haetae_public_key_vector_from_relation
public_key_vector_seed
```

**Theorem 13.251** (EasyCrypt line 3459). *EasyCrypt name.*

```
haetae_keygen_raw_public_vector_keygen_equation
```

**Checked EasyCrypt statement.**

```
lemma haetae_keygen_raw_public_vector_keygen_equation md sd :
  haetae_keygen_raw_public_vector md sd =
  polyveck_add md
    (matrix_vec_mul md
      (haetae_keygen_a0_matrix md sd)
      (haetae_keygen_secret_vector md sd))
    (haetae_keygen_error_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.252** (EasyCrypt line 3468). *EasyCrypt name.*

```
haetae_reference_keygen_raw_public_vector_equation
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_raw_public_vector_equation md sd :
  haetae_reference_keygen_raw_public_vector md sd =
```

```

polyveck_add md
  (matrix_vec_mul md
    (haetae_reference_polymatkm_expand_matA md
      (haetae_keygen_rho_prime sd))
    (haetae_reference_keygen_secret_vector md sd))
    (haetae_reference_keygen_error_vector md sd)).

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:

```

haetae_reference_keygen_a0_matrix
haetae_reference_keygen_raw_public_vector

```

**Theorem 13.253** (EasyCrypt line 3481). *EasyCrypt name.*

```

haetae_reference_keygen_mode23_public_vector_equation

```

**Checked EasyCrypt statement.**

```

lemma haetae_reference_keygen_mode23_public_vector_equation md sd :
  (md = Mode2 /\ md = Mode3) =>
    haetae_reference_keygen_public_vector md sd =
      polyveck_vk_highbits md
        (polyveck_add md
          (haetae_reference_polyveck_expand_vecA md
            (haetae_keygen_rho_prime sd))
          (polyveck_add md
            (matrix_vec_mul md
              (haetae_reference_polymatkm_expand_matA md
                (haetae_keygen_rho_prime sd))
              (haetae_reference_keygen_secret_vector md sd))
            (haetae_reference_keygen_error_vector md sd))).

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `case`. Named identifiers syntactically cited in the proof block:

```

haetae_public_key_vector_from_relation
haetae_reference_keygen_public_vector
haetae_reference_polymatkm_expand_matA
haetae_reference_polyveck_expand_vecA
md
md23

```

**Theorem 13.254** (EasyCrypt line 3503). *EasyCrypt name.*

```

haetae_reference_keygen_mode23_adjusted_error_equation

```

### Checked EasyCrypt statement.

```
lemma haetae_reference_keygen_mode23_adjusted_error_equation md sd :  
  (md = Mode2 \ / md = Mode3) =>  
  haetae_reference_keygen_mode23_adjusted_error_vector md sd =  
  polyveck_sub md  
    (haetae_reference_keygen_error_vector md sd)  
    (polyveck_vk_lowbits md  
      (polyveck_add md  
        (haetae_reference_polyveck_expand_vecA md  
          (haetae_keygen_rho_prime sd))  
        (haetae_reference_keygen_raw_public_vector md sd))))).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_mode23_adjusted_error_vector  
haetae_reference_keygen_mode23_predecompose_vector  
haetae_reference_keygen_mode23_rounding_lowbits
```

### Theorem 13.255 (EasyCrypt line 3520). *EasyCrypt* name.

```
haetae_reference_keygen_mode5_public_vector_equation
```

### Checked EasyCrypt statement.

```
lemma haetae_reference_keygen_mode5_public_vector_equation sd :  
  haetae_reference_keygen_public_vector Mode5 sd =  
  polyveck_neg Mode5  
    (polyveck_double Mode5  
      (polyveck_add Mode5  
        (matrix_vec_mul Mode5  
          (haetae_reference_polymatkm_expand_matA Mode5  
            (haetae_keygen_rho_prime sd))  
          (haetae_reference_keygen_secret_vector Mode5 sd))  
        (haetae_reference_keygen_error_vector Mode5 sd))))).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_public_key_vector_from_relation  
haetae_reference_keygen_public_vector  
haetae_reference_polymatkm_expand_matA
```

### Theorem 13.256 (EasyCrypt line 3536). *EasyCrypt* name.

```
packed_haetae_reference_public_key_seedE
```

**Checked EasyCrypt statement.**

```
lemma packed_haetae_reference_public_key_seedE md sd :  
  (packed_haetae_reference_public_key md sd). '1 =  
  haetae_keygen_rhoprime sd.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.257** (EasyCrypt line 3541). **EasyCrypt name.**

```
packed_haetae_reference_public_key_bodyE
```

**Checked EasyCrypt statement.**

```
lemma packed_haetae_reference_public_key_bodyE md sd :  
  (packed_haetae_reference_public_key md sd). '2 =  
  haetae_reference_keygen_public_vector md sd.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.258** (EasyCrypt line 3546). **EasyCrypt name.**

```
haetae_reference_keygen_seed_flow_wf_ok
```

**Checked EasyCrypt statement.**

```
lemma haetae_reference_keygen_seed_flow_wf_ok md sd :  
  haetae_reference_keygen_seed_flow_wf md sd.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_reference_keygen_a0_matrix  
haetae_reference_keygen_error_vector  
haetae_reference_keygen_public_vector  
haetae_reference_keygen_secret_vector  
haetae_reference_keygen_seed_flow_wf  
packed_haetae_reference_public_key
```

**Theorem 13.259** (EasyCrypt line 3557). **EasyCrypt name.**

```
packed_haetae_public_key_wf
```

**Checked EasyCrypt statement.**



```
lemma packed_haetae_public_key_wf md sd :
  polyveck_wf md (packed_haetae_public_key md sd). '2.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_public_vector_polyq_wf
haetae_polyveck_polyq_coeffs_wf
md
packed_haetae_public_key
sd
```

**Theorem 13.260** (EasyCrypt line 3564). *EasyCrypt* name.

```
packed_haetae_public_key_polyq_wf
```

**Checked *EasyCrypt* statement.**

```
lemma packed_haetae_public_key_polyq_wf md sd :
  haetae_polyveck_polyq_coeffs_wf md (packed_haetae_public_key md sd). '2.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the *EasyCrypt* statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_public_vector_polyq_wf
packed_haetae_public_key
```

**Theorem 13.261** (EasyCrypt line 3571). *EasyCrypt* name.

```
packed_haetae_public_key_unpacked_wf
```

**Checked *EasyCrypt* statement.**

```
lemma packed_haetae_public_key_unpacked_wf md sd :
  size sd = seedbytes =>
  haetae_public_key_unpacked_wf md (packed_haetae_public_key md sd).
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
haetae_public_key_unpacked_wf
packed_haetae_public_key_wf
sd_sz
```

**Theorem 13.262** (EasyCrypt line 3579). *EasyCrypt name*.

```
haetae_keygen_public_key_roundtrip
```

*Checked EasyCrypt statement.*

```
lemma haetae_keygen_public_key_roundtrip md sd :
  size sd = seedbytes =>
  haetae_keygen_public_key_roundtrip_ok md sd.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*. Named identifiers syntactically cited in the proof block:

```
haetae_keygen_public_key_roundtrip_ok
haetae_public_key_roundtrip
packed_haetae_public_key_unpacked_wf
sd_sz
```

**Theorem 13.263** (EasyCrypt line 3589). *EasyCrypt name*.

```
public_key_of_secret_packed
```

*Checked EasyCrypt statement.*

```
lemma public_key_of_secret_packed md sk :
  public_key_of_secret md sk = packed_haetae_public_key md sk.'1.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
packed_haetae_public_key
public_key_of_secret
public_key_vector_seed
```

**Theorem 13.264** (EasyCrypt line 3596). *EasyCrypt name*.

```
keygen_public_key_packed
```

**Checked EasyCrypt statement.**

```
lemma keygen_public_key_packed md sd :  
  (keygen_internal md sd). '1 = packed_haetae_public_key md sd.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
keygen_internal  
packed_haetae_public_key  
public_key_of_secret  
public_key_vector_seed  
secret_key_of_seed
```

**Theorem 13.265 (EasyCrypt line 3603). EasyCrypt name.**

```
public_verification_matrix_packedE
```

**Checked EasyCrypt statement.**

```
lemma public_verification_matrix_packedE md pk :  
  public_verification_matrix md pk = packed_haetae_verification_matrix md pk.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.266 (EasyCrypt line 3607). EasyCrypt name.**

```
packed_haetae_keygen_verification_matrixE
```

**Checked EasyCrypt statement.**

```
lemma packed_haetae_keygen_verification_matrixE md sd :  
  packed_haetae_keygen_verification_matrix md sd =  
  haetae_verification_matrix_from_unpacked md sd  
  (haetae_keygen_public_vector md sd).
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
packed_haetae_keygen_verification_matrix  
packed_haetae_public_key
```

```
packed_haetae_verification_matrix
```

**Theorem 13.267** (EasyCrypt line 3617). *EasyCrypt name.*

```
honest_public_verification_matrix_eq_packed_keygen
```

**Checked EasyCrypt statement.**

```
lemma honest_public_verification_matrix_eq_packed_keygen md sd :  
  public_verification_matrix md (keygen_internal md sd). '1 =  
  packed_haetae_keygen_verification_matrix md sd.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
keygen_internal  
packed_haetae_keygen_verification_matrix  
packed_haetae_public_key  
packed_haetae_verification_matrix  
public_key_of_secret  
public_key_vector_seed  
public_verification_matrix  
secret_key_of_seed
```

**Theorem 13.268** (EasyCrypt line 3627). *EasyCrypt name.*

```
concrete_keygen_public_verification_matrix_correct
```

**Checked EasyCrypt statement.**

```
lemma concrete_keygen_public_verification_matrix_correct md sd :  
  size sd = seedbytes =>  
  public_verification_matrix md (keygen_internal md sd). '1 =  
  concrete_haetae_keygen_verification_matrix md sd.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
concrete_haetae_keygen_packed_public_key  
concrete_haetae_keygen_unpacked_public_key  
concrete_haetae_keygen_verification_matrix  
haetae_keygen_public_key_roundtrip  
haetae_keygen_public_key_roundtrip_ok  
haetae_public_key_roundtrip_ok  
honest_public_verification_matrix_eq_packed_keygen  
md
```

```
packed_haetae_keygen_verification_matrix
rt
sd
sd_sz
```

**Theorem 13.269** (EasyCrypt line 3644). *EasyCrypt name.*

```
concrete_keygen_public_verification_matrix_ref_correct
```

**Checked EasyCrypt statement.**

```
lemma concrete_keygen_public_verification_matrix_ref_correct md sd :
  size sd = seedbytes =>
  public_verification_matrix md (keygen_internal md sd). '1 =
  concrete_haetae_keygen_verification_matrix_ref md sd.
```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*. Named identifiers syntactically cited in the proof block:

```
concrete_haetae_keygen_packed_public_key_ref
concrete_haetae_keygen_unpacked_public_key_ref
concrete_haetae_keygen_verification_matrix_ref
haetae_keygen_public_key_ref_roundtrip
haetae_pack_public_key_bytes_ref
haetae_unpack_public_key_bytes_ref
honest_public_verification_matrix_eq_packed_keygen
md
packed_haetae_keygen_verification_matrix
rt
sd
sd_sz
```

**Theorem 13.270** (EasyCrypt line 3661). *EasyCrypt name.*

```
concrete_reference_keygen_public_verification_matrix_ref_correct
```

**Checked EasyCrypt statement.**

```
lemma concrete_reference_keygen_public_verification_matrix_ref_correct md sd :
  packed_haetae_verification_matrix md
  (packed_haetae_reference_public_key md sd) =
  concrete_haetae_reference_keygen_verification_matrix_ref md sd.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *have*, *rewrite*. Named identifiers syntactically cited in the proof block:

```

concrete_haetae_reference_keygen_packed_public_key_ref
concrete_haetae_reference_keygen_unpacked_public_key_ref
concrete_haetae_reference_keygen_verification_matrix_ref
haetae_reference_keygen_public_key_ref_roundtrip
md
rt
sd

```

**Theorem 13.271** (EasyCrypt line 3673). *EasyCrypt name.*

```
concrete_keygen_public_verification_matrix_size
```

**Checked EasyCrypt statement.**

```

lemma concrete_keygen_public_verification_matrix_size md sd :
  size (concrete_haetae_keygen_verification_matrix md sd) = mode_k md.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*. Named identifiers syntactically cited in the proof block:

```

concrete_haetae_keygen_verification_matrix
haetae_verification_matrix_from_unpacked
md
packed_haetae_verification_matrix
size_mkseq

```

**Theorem 13.272** (EasyCrypt line 3681). *EasyCrypt name.*

```
concrete_keygen_public_verification_matrix_ref_size
```

**Checked EasyCrypt statement.**

```

lemma concrete_keygen_public_verification_matrix_ref_size md sd :
  size (concrete_haetae_keygen_verification_matrix_ref md sd) = mode_k md.

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*. Named identifiers syntactically cited in the proof block:

```

concrete_haetae_keygen_verification_matrix_ref
haetae_verification_matrix_from_unpacked
md
packed_haetae_verification_matrix
size_mkseq

```

**Theorem 13.273** (EasyCrypt line 3689). *EasyCrypt name.*

```
concrete_reference_keygen_public_verification_matrix_ref_size
```

**Checked EasyCrypt statement.**

```
lemma concrete_reference_keygen_public_verification_matrix_ref_size md sd :
  size (concrete_haetae_reference_keygen_verification_matrix_ref md sd) =
    mode_k md.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *case*. Named identifiers syntactically cited in the proof block:

```
concrete_haetae_reference_keygen_verification_matrix_ref
haetae_verification_matrix_from_unpacked
md
packed_haetae_verification_matrix
size_mkseq
```

**Theorem 13.274 (EasyCrypt line 3698). EasyCrypt name.**

```
concrete_keygen_public_verification_matrix_rows_wf
```

**Checked EasyCrypt statement.**

```
lemma concrete_keygen_public_verification_matrix_rows_wf md sd :
  all (polyvecl_wf md) (concrete_haetae_keygen_verification_matrix md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*, *split*, *smt*, *case*, *have*. Named identifiers syntactically cited in the proof block:

```
allP
col_wf
concrete_haetae_keygen_unpacked_public_key
concrete_haetae_keygen_verification_matrix
haetae_verification_column_from_unpacked_wf
haetae_verification_matrix_from_unpacked
i_rng
j0
md
mkseqP
mode_l_gt0
packed_haetae_verification_matrix
poly_double_wf
polyveck_wf_nth
polyvecl_wf
row
sd
size_mkseq
```

**Theorem 13.275** (EasyCrypt line 3718). *EasyCrypt name.*

```
concrete_keygen_public_verification_matrix_ref_rows_wf
```

**Checked EasyCrypt statement.**

```
lemma concrete_keygen_public_verification_matrix_ref_rows_wf md sd :
  all (polyvecl_wf md)
    (concrete_haetae_keygen_verification_matrix_ref md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*, *split*, *smt*, *case*, *have*. Named identifiers syntactically cited in the proof block:

```
allP
col_wf
concrete_haetae_keygen_unpacked_public_key_ref
concrete_haetae_keygen_verification_matrix_ref
haetae_verification_column_from_unpacked_wf
haetae_verification_matrix_from_unpacked
i_rng
j0
md
mkseqP
mode_l_gt0
packed_haetae_verification_matrix
poly_double_wf
polyveck_wf_nth
polyvecl_wf
row
sd
size_mkseq
```

**Theorem 13.276** (EasyCrypt line 3739). *EasyCrypt name.*

```
concrete_reference_keygen_public_verification_matrix_ref_rows_wf
```

**Checked EasyCrypt statement.**

```
lemma concrete_reference_keygen_public_verification_matrix_ref_rows_wf md sd :
  all (polyvecl_wf md)
    (concrete_haetae_reference_keygen_verification_matrix_ref md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *apply*, *split*, *smt*, *case*, *have*. Named identifiers syntactically cited in the proof block:

```
allP
col_wf
```



```

concrete_haetae_reference_keygen_unpacked_public_key_ref
concrete_haetae_reference_keygen_verification_matrix_ref
haetae_verification_column_from_unpacked_wf
haetae_verification_matrix_from_unpacked
i_rng
j0
md
mkseqP
mode_l_gt0
packed_haetae_verification_matrix
poly_double_wf
polyveck_wf_nth
polyvecl_wf
row
sd
size_mkseq

```

**Theorem 13.277** (EasyCrypt line 3760). *EasyCrypt name.*

```
valid_signature_sign_internal
```

**Checked EasyCrypt statement.**

```

lemma valid_signature_sign_internal md sk m ctx coins :
  valid_signature md (sign_internal md sk m ctx coins).

```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

challenge_hash_wf
commitment_challenge
commitment_highbits_wf
commitment_lowbits_wf
crh_wf
hint_vector_wf
message_hash_size
response_aux_vector_wf
response_vector_wf
sign_internal
signature_token
valid_signature

```

**Theorem 13.278** (EasyCrypt line 3769). *EasyCrypt name.*

```
verify_challenge_consistent_sign_internal
```

**Checked EasyCrypt statement.**

```

lemma verify_challenge_consistent_sign_internal md sk m ctx coins :
  verify_challenge_consistent md (public_key_of_secret md sk) m ctx
    (sign_internal md sk m ctx coins).

```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
challenge_hash
commitment_challenge
sign_internal
verify_challenge_consistent
```

**Theorem 13.279** (EasyCrypt line 3819). *EasyCrypt name*.

```
secret_norm_ok_current
```

**Checked EasyCrypt statement.**

```
lemma secret_norm_ok_current md sd :
  secret_norm_ok md (secret_key_of_seed md sd).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 13.280** (EasyCrypt line 3823). *EasyCrypt name*.

```
response_norm_ok_current
```

**Checked EasyCrypt statement.**

```
lemma response_norm_ok_current md sk m ctx coins :
  response_norm_ok md (sign_internal md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*, *apply*, *case*. Named identifiers syntactically cited in the proof block:

```
coins
ctx
deterministic_unit_polyveck
deterministic_unit_polyveck_unit
deterministic_unit_polyvecl
deterministic_unit_polyvecl_unit
haux
hresp
md
polyveck_norm_sq
polyveck_unit
polyvecl_norm_sq
polyvecl_unit
```

```

response_aux_vector
response_norm_ok
response_norm_sq
response_source
response_vector
sig_response
sig_response_aux
sig_token_value
sign_internal
signature_response_norm_sq
signature_token
sk

```

**Theorem 13.281** (EasyCrypt line 3843). *EasyCrypt name.*

```

verify_norm_ok_current

```

**Checked EasyCrypt statement.**

```

lemma verify_norm_ok_current md sk m ctx coins :
  verify_norm_ok md (sign_internal md sk m ctx coins).

```

**Formal mathematical reading.** *Mathematical theorem.* This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** *Use in this chapter.* It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** *Proof-script reading.* The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *have*, *apply*, *case*. Named identifiers syntactically cited in the proof block:

```

coins
ctx
deterministic_unit_polyveck
deterministic_unit_polyveck_unit
deterministic_unit_polyvecl
deterministic_unit_polyvecl_unit
haux
hhint
hint_vector
hresp
md
polyveck_norm_sq
polyveck_unit
polyvecl_norm_sq
polyvecl_unit
response_aux_vector
response_source
response_vector
sig_hint
sig_response
sig_response_aux
sig_token_value
sign_internal
signature_response_norm_sq
signature_token
signature_verify_norm_sq
sk
verify_norm_ok
verify_norm_sq

```

**Theorem 13.282** (EasyCrypt line 3868). *EasyCrypt name.*

```
verify_internal_sign_internal
```

**Checked EasyCrypt statement.**

```
lemma verify_internal_sign_internal md sk m ctx coins :  
  verify_internal md (public_key_of_secret md sk) m ctx  
    (sign_internal md sk m ctx coins).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *split*, *apply*. Named identifiers syntactically cited in the proof block:

```
commitment_challenge  
commitment_highbits  
sig_response_aux  
sig_token_value  
sign_internal  
signature_token  
valid_signature_sign_internal  
verify_challenge_consistent_sign_internal  
verify_internal  
verify_norm_ok_current  
verify_public_equation  
verify_reconstructed_highbits
```

*End of generated formal inventory for HAETAE\_Algebra.ec.*

# Chapter 14

## HAETAE\_Params.ec

**Mathematical role.** This file fixes the public parameter functions used by all other files. It defines the seed length, CRH length, polynomial dimension, modulus, and the three HAETAE modes.

**Parameters.** The global constants are

$$\text{seedbytes} = 32, \quad \text{crhbytes} = 64, \quad n = 256, \quad q = 64513.$$

The modes determine vector dimensions and bounds:

| mode | $k$ | $\ell$ | $\tau$ |
|------|-----|--------|--------|
| 2    | 2   | 4      | 58     |
| 3    | 3   | 6      | 80     |
| 5    | 4   | 7      | 128    |

**Proof obligations.** The lemmas are small but important arithmetic facts: positivity of lengths and bounds,  $\tau \leq n$ , and public-key byte-size positivity. These facts are used by distribution, list-size, and packing proofs throughout the development.

### Textbook Formal Inventory

This subsection records the exact formal material in HAETAE\_Params.ec. It is generated from the proof-surface EasyCrypt file, so the line numbers and statements are meant to be read next to the checked source.

**Chapter role.** mode-dependent numerical parameters and parameter inequalities.

**Inventory size.** 2 context declarations, 18 definitions/game objects, 19 lemmas or relational theorems, and 0 explicit Hoare/Phoare judgments were found.

### Theory Context, Imports, and Quantified Modules

**Context 1 (line 1)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import AllCore.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 2 (line 3) EasyCrypt name.**

```
HAETAE_Params
```

**Checked EasyCrypt statement.**

```
theory HAETAE_Params.
```

**Formal mathematical reading.** Mathematical context. This begins a namespace for the formal development in this file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

## Definitions, Carrier Sets, Operations, Games, and Procedures

**Definition 14.1** (EasyCrypt line 5). **EasyCrypt name.**

```
seedbytes
```

**Checked EasyCrypt statement.**

```
op seedbytes : int = 32.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 14.2** (EasyCrypt line 6). **EasyCrypt name.**

```
crhbytes
```

**Checked EasyCrypt statement.**

```
op crhbytes : int = 64.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 14.3** (EasyCrypt line 7). **EasyCrypt name.**

```
n
```

**Checked EasyCrypt statement.**

```
op n : int = 256.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 14.4** (EasyCrypt line 8). **EasyCrypt name.**

```
q
```

**Checked EasyCrypt statement.**

```
op q : int = 64513.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 14.5** (EasyCrypt line 9). **EasyCrypt name.**

```
dq
```

**Checked EasyCrypt statement.**

```
op dq : int = 2 * q.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It contributes a sampler or sampler-derived object to the distributional model.

**Definition 14.6** (EasyCrypt line 11). **EasyCrypt name.**

```
mode
```

**Checked EasyCrypt statement.**

```
type mode = [ Mode2 | Mode3 | Mode5 ].
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 14.7** (EasyCrypt line 13). **EasyCrypt name.**

```
mode_k
```

**Checked EasyCrypt statement.**

```
op mode_k (md : mode) : int =  
  with md = Mode2 => 2  
  with md = Mode3 => 3  
  with md = Mode5 => 4.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 14.8** (EasyCrypt line 18). **EasyCrypt name.**

```
mode_l
```

**Checked EasyCrypt statement.**

```
op mode_l (md : mode) : int =  
  with md = Mode2 => 4  
  with md = Mode3 => 6  
  with md = Mode5 => 7.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 14.9** (EasyCrypt line 23). **EasyCrypt name.**

```
mode_tau
```

**Checked EasyCrypt statement.**

```
op mode_tau (md : mode) : int =  
  with md = Mode2 => 58  
  with md = Mode3 => 80  
  with md = Mode5 => 128.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 14.10** (EasyCrypt line 28). **EasyCrypt name.**

```
mode_b0sq
```

**Checked EasyCrypt statement.**

```
op mode_b0sq (md : mode) : int =  
  with md = Mode2 => 96944109  
  with md = Mode3 => 335438492  
  with md = Mode5 => 499239142.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 14.11** (EasyCrypt line 33). **EasyCrypt name.**

```
mode_b1sq
```

**Checked EasyCrypt statement.**

```
op mode_b1sq (md : mode) : int =  
  with md = Mode2 => 96805527  
  with md = Mode3 => 335171879  
  with md = Mode5 => 498849991.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.



**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 14.12** (EasyCrypt line 38). **EasyCrypt name.**

```
mode_b2sq
```

**Checked EasyCrypt statement.**

```
op mode_b2sq (md : mode) : int =  
  with md = Mode2 => 163265017  
  with md = Mode3 => 479901314  
  with md = Mode5 => 597386433.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 14.13** (EasyCrypt line 43). **EasyCrypt name.**

```
mode_ln
```

**Checked EasyCrypt statement.**

```
op mode_ln (_ : mode) : int = 8192.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 14.14** (EasyCrypt line 45). **EasyCrypt name.**

```
mode_alpha_hint
```

**Checked EasyCrypt statement.**

```
op mode_alpha_hint (md : mode) : int =  
  with md = Mode2 => 512  
  with md = Mode3 => 512  
  with md = Mode5 => 256.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 14.15** (EasyCrypt line 50). **EasyCrypt name.**

```
mode_m
```

**Checked EasyCrypt statement.**

```
op mode_m (md : mode) : int = mode_l md - 1.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 14.16** (EasyCrypt line 52). **EasyCrypt name.**

```
mode_crypto_bytes
```

**Checked EasyCrypt statement.**

```
op mode_crypto_bytes (md : mode) : int =  
  with md = Mode2 => 1474  
  with md = Mode3 => 2349  
  with md = Mode5 => 2948.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 14.17** (EasyCrypt line 57). **EasyCrypt name.**

```
mode_polyq_packedbytes
```

**Checked EasyCrypt statement.**

```
op mode_polyq_packedbytes (md : mode) : int =  
  with md = Mode2 => 480  
  with md = Mode3 => 480  
  with md = Mode5 => 512.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 14.18** (EasyCrypt line 62). **EasyCrypt name.**

```
mode_publickeybytes
```

**Checked EasyCrypt statement.**

```
op mode_publickeybytes (md : mode) : int =  
  seedbytes + mode_k md * mode_polyq_packedbytes md.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

## Lemmas, Theorems, Relational Equivalences, and Their Proof Dependencies

**Theorem 14.1** (EasyCrypt line 65). **EasyCrypt name.**

```
seedbytes_gt0
```

**Checked EasyCrypt statement.**

```
lemma seedbytes_gt0 : 0 < seedbytes by rewrite /seedbytes.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.2** (EasyCrypt line 66). **EasyCrypt name.**

```
crhbytes_gt0
```

**Checked EasyCrypt statement.**

```
lemma crhbytes_gt0 : 0 < crhbytes by rewrite /crhbytes.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.3** (EasyCrypt line 67). **EasyCrypt name.**

```
n_gt0
```

**Checked EasyCrypt statement.**

```
lemma n_gt0 : 0 < n by rewrite /n.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.4** (EasyCrypt line 68). **EasyCrypt name.**

```
q_gt0
```

**Checked EasyCrypt statement.**

```
lemma q_gt0 : 0 < q by rewrite /q.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.5** (EasyCrypt line 69). **EasyCrypt name.**

```
dqE
```

**Checked EasyCrypt statement.**

```
lemma dqE : dq = 129026 by rewrite /dq /q.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.6** (EasyCrypt line 71). **EasyCrypt name.**

```
mode_k_gt0
```

**Checked EasyCrypt statement.**

```
lemma mode_k_gt0 mδ : 0 < mode_k mδ.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.7** (EasyCrypt line 74). **EasyCrypt name.**

```
mode_l_gt0
```

**Checked EasyCrypt statement.**

```
lemma mode_l_gt0 mδ : 0 < mode_l mδ.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.8** (EasyCrypt line 77). **EasyCrypt name.**

```
mode_tau_gt0
```

**Checked EasyCrypt statement.**

```
lemma mode_tau_gt0 mδ : 0 < mode_tau mδ.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.9** (EasyCrypt line 80). **EasyCrypt name.**

```
mode_tau_le_n
```

**Checked EasyCrypt statement.**

```
lemma mode_tau_le_n mδ : mode_tau mδ <= n.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.10** (EasyCrypt line 83). *EasyCrypt name.*

```
mode_b0sq_gt0
```

**Checked EasyCrypt statement.**

```
lemma mode_b0sq_gt0 md : 0 < mode_b0sq md.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.11** (EasyCrypt line 86). *EasyCrypt name.*

```
mode_b1sq_gt0
```

**Checked EasyCrypt statement.**

```
lemma mode_b1sq_gt0 md : 0 < mode_b1sq md.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.12** (EasyCrypt line 89). *EasyCrypt name.*

```
mode_b2sq_gt0
```

**Checked EasyCrypt statement.**

```
lemma mode_b2sq_gt0 md : 0 < mode_b2sq md.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.13** (EasyCrypt line 92). *EasyCrypt name.*

```
mode_ln_gt0
```

**Checked EasyCrypt statement.**

```
lemma mode_ln_gt0 md : 0 < mode_ln md.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.14** (EasyCrypt line 95). *EasyCrypt name.*

```
mode_alpha_hint_gt0
```

**Checked EasyCrypt statement.**

```
lemma mode_alpha_hint_gt0 md : 0 < mode_alpha_hint md.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.15** (EasyCrypt line 98). *EasyCrypt name.*

```
mode_m_gt0
```

**Checked EasyCrypt statement.**

```
lemma mode_m_gt0 md : 0 < mode_m md.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.16** (EasyCrypt line 101). *EasyCrypt name.*

```
mode_crypto_bytes_gt0
```

**Checked EasyCrypt statement.**

```
lemma mode_crypto_bytes_gt0 md : 0 < mode_crypto_bytes md.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.17** (EasyCrypt line 104). *EasyCrypt name.*

```
mode_polyq_packedbytes_gt0
```

**Checked EasyCrypt statement.**

```
lemma mode_polyq_packedbytes_gt0 md : 0 < mode_polyq_packedbytes md.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.18** (EasyCrypt line 107). *EasyCrypt name.*

```
n_le_mode_polyq_packedbytes
```

**Checked EasyCrypt statement.**

```
lemma n_le_mode_polyq_packedbytes md : n <= mode_polyq_packedbytes md.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It contributes directly to an advantage bound, bad-event bound, or real-arithmetic composition step.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 14.19** (EasyCrypt line 110). *EasyCrypt name.*

```
mode_publickeybytes_gt0
```

**Checked EasyCrypt statement.**

```
lemma mode_publickeybytes_gt0 md : 0 < mode_publickeybytes md.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

End of generated formal inventory for HAETAE\_Params.ec.

# Chapter 15

## HAETAE\_FIPS202.ec

**Mathematical role.** This file gives a compact model of the FIPS202/SHAKE256 interface used by the HAETAE algebraic model. It is not a cryptographic proof of SHAKE; it supplies deterministic functions and size facts so the EasyCrypt model can talk about hash-derived byte strings.

**SHAKE model.** The state has 200 bytes, the SHAKE256 rate is 136 bytes, and the capacity is 64 bytes. The one-block absorb operation adds the SHAKE domain separator and final padding, applies the modeled Keccak permutation, and squeezes bytes by taking the requested state prefix.

**Use in the proof.** The size lemmas guarantee that the seed-expansion operations in HAETAE\_Algebra.ec produce byte strings of the expected length. The security proof treats the random oracle abstractly; this file is support for the deterministic hash-shaped computations inside the model.

### Textbook Formal Inventory

This subsection records the exact formal material in HAETAE\_FIPS202.ec. It is generated from the proof-surface EasyCrypt file, so the line numbers and statements are meant to be read next to the checked source.

**Chapter role.** deterministic FIPS202/SHAKE support for the formal model.

**Inventory size.** 4 context declarations, 14 definitions/game objects, 9 lemmas or relational theorems, and 0 explicit Hoare/Phoare judgments were found.

### Theory Context, Imports, and Quantified Modules

**Context 1 (line 1)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import AllCore IntDiv List.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.



**Context 2 (line 2)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import HAETAE_Keccak1600.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 3 (line 4)** EasyCrypt name.

```
HAETAE_FIPS202
```

**Checked EasyCrypt statement.**

```
theory HAETAE_FIPS202.
```

**Formal mathematical reading.** Mathematical context. This begins a namespace for the formal development in this file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 4 (line 6)** EasyCrypt name.

```
opened namespace
```

**Checked EasyCrypt statement.**

```
import HAETAE_Keccak1600.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

## Definitions, Carrier Sets, Operations, Games, and Procedures

**Definition 15.1** (EasyCrypt line 8). EasyCrypt name.

```
fips_byte
```

**Checked EasyCrypt statement.**

```
type fips_byte = int.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 15.2** (EasyCrypt line 9). EasyCrypt name.

```
fips202_state
```

**Checked EasyCrypt statement.**

```
type fips202_state = fips_byte list.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 15.3** (EasyCrypt line 11). **EasyCrypt name.**

```
fips202_state_bytes
```

**Checked EasyCrypt statement.**

```
op fips202_state_bytes : int = 200.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 15.4** (EasyCrypt line 12). **EasyCrypt name.**

```
shake256_rate_bytes
```

**Checked EasyCrypt statement.**

```
op shake256_rate_bytes : int = 136.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 15.5** (EasyCrypt line 13). **EasyCrypt name.**

```
shake256_capacity_bytes
```

**Checked EasyCrypt statement.**

```
op shake256_capacity_bytes : int = 64.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 15.6** (EasyCrypt line 14). **EasyCrypt name.**

```
shake256_domain_separator
```

**Checked EasyCrypt statement.**

```
op shake256_domain_separator : fips_byte = 31.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 15.7** (EasyCrypt line 15). **EasyCrypt name.**

```
fips202_final_padding_byte
```

**Checked EasyCrypt statement.**

```
op fips202_final_padding_byte : fips_byte = 128.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 15.8** (EasyCrypt line 17). **EasyCrypt name.**

```
fips202_keccak_f1600
```

**Checked EasyCrypt statement.**

```
op fips202_keccak_f1600 (st : fips202_state) : fips202_state =  
  keccak_f1600_bytes st.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 15.9** (EasyCrypt line 20). **EasyCrypt name.**

```
shake256_absorb_once_short_domain
```

**Checked EasyCrypt statement.**

```
op shake256_absorb_once_short_domain  
  (_ : fips_byte list) (inlen : int) : bool =  
  0 <= inlen /\ inlen < shake256_rate_bytes.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 15.10** (EasyCrypt line 24). **EasyCrypt name.**

```
shake256_absorb_once_short_block_byte
```

**Checked EasyCrypt statement.**

```
op shake256_absorb_once_short_block_byte  
  (input : fips_byte list) (inlen i : int) : fips_byte =  
  if 0 <= i /\ i < inlen then nth 0 input i  
  else if i = inlen /\ i = shake256_rate_bytes - 1 then  
    shake256_domain_separator + fips202_final_padding_byte
```

```

else if i = inlen then shake256_domain_separator
else if i = shake256_rate_bytes - 1 then fips202_final_padding_byte
else 0.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 15.11** (EasyCrypt line 33). **EasyCrypt name.**

```
shake256_absorb_once_short_state
```

**Checked EasyCrypt statement.**

```

op shake256_absorb_once_short_state
  (input : fips_byte list) (inlen : int) : fips202_state =
  mkseq
    (fun i =>
      if i < shake256_rate_bytes
      then shake256_absorb_once_short_block_byte input inlen i
      else 0)
    fips202_state_bytes.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 15.12** (EasyCrypt line 42). **EasyCrypt name.**

```
shake256_absorb_once
```

**Checked EasyCrypt statement.**

```

op shake256_absorb_once
  (input : fips_byte list) (inlen : int) : fips202_state =
  fips202_keccak_f1600
    (shake256_absorb_once_short_state input inlen).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 15.13** (EasyCrypt line 47). **EasyCrypt name.**

```
shake256_squeeze
```

**Checked EasyCrypt statement.**

```

op shake256_squeeze (st : fips202_state) (outlen : int) : fips_byte list =
  mkseq (fun i => nth 0 st i) outlen.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 15.14** (EasyCrypt line 50). **EasyCrypt name.**

shake256

**Checked EasyCrypt statement.**

```
op shake256
  (input : fips_byte list) (inlen outlen : int) : fips_byte list =
  shake256_squeeze (shake256_absorb_once input inlen) outlen.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.  
**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

## Lemmas, Theorems, Relational Equivalences, and Their Proof Dependencies

**Theorem 15.1** (EasyCrypt line 54). *EasyCrypt name.*

fips202\_state\_bytesE

**Checked EasyCrypt statement.**

```
lemma fips202_state_bytesE :
  fips202_state_bytes = 200.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 15.2** (EasyCrypt line 58). *EasyCrypt name.*

shake256\_rate\_bytesE

**Checked EasyCrypt statement.**

```
lemma shake256_rate_bytesE :
  shake256_rate_bytes = 136.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 15.3** (EasyCrypt line 62). *EasyCrypt name.*

shake256\_capacity\_bytesE

**Checked EasyCrypt statement.**

```
lemma shake256_capacity_bytesE :
  shake256_capacity_bytes = 64.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 15.4** (EasyCrypt line 66). *EasyCrypt name.*

```
shake256_domain_separatorE
```

**Checked EasyCrypt statement.**

```
lemma shake256_domain_separatorE :  
  shake256_domain_separator = 31.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 15.5** (EasyCrypt line 70). *EasyCrypt name.*

```
fips202_final_padding_byteE
```

**Checked EasyCrypt statement.**

```
lemma fips202_final_padding_byteE :  
  fips202_final_padding_byte = 128.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 15.6** (EasyCrypt line 74). *EasyCrypt name.*

```
shake256_rate_capacity_state_bytesE
```

**Checked EasyCrypt statement.**

```
lemma shake256_rate_capacity_state_bytesE :  
  shake256_rate_bytes + shake256_capacity_bytes = fips202_state_bytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
fips202_state_bytes  
shake256_capacity_bytes  
shake256_rate_bytes
```

**Theorem 15.7** (EasyCrypt line 81). *EasyCrypt name.*

```
shake256_absorb_once_short_state_size
```

**Checked EasyCrypt statement.**

```
lemma shake256_absorb_once_short_state_size input inlen :  
  size (shake256_absorb_once_short_state input inlen) =  
  fips202_state_bytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:

```
shake256_absorb_once_short_state  
size_mkseq
```

**Theorem 15.8** (EasyCrypt line 88). **EasyCrypt name.**

```
shake256_squeeze_size
```

**Checked EasyCrypt statement.**

```
lemma shake256_squeeze_size st outlen :  
  0 <= outlen =>  
  size (shake256_squeeze st outlen) = outlen.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `smt`. Named identifiers syntactically cited in the proof block:

```
ge0_outlen  
shake256_squeeze  
size_mkseq
```

**Theorem 15.9** (EasyCrypt line 97). **EasyCrypt name.**

```
shake256_size
```

**Checked EasyCrypt statement.**

```
lemma shake256_size input inlen outlen :  
  0 <= outlen =>  
  size (shake256 input inlen outlen) = outlen.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
ge0_outlen
shake256
shake256_squeeze_size
```

*End of generated formal inventory for HAETAE\_FIPS202. ec.*



# Chapter 16

## HAETAE\_Keccak1600.ec

**Mathematical role.** This file models the Keccak-f1600 permutation at the lane/bit level. It defines lanes, states, byte/lane conversions, and the theta, rho-pi, chi, and iota round functions.

**Bit-level equations.** The file proves equations such as `keccak_theta_bitE`, `keccak_rho_pi_bitE`, `keccak_chi_bitE`, and `keccak_round_bitE`. Mathematically these are expansion lemmas: they let later proofs replace a named Keccak step by its bitwise definition.

**Cryptographic boundary.** The HAETAE security theorem does not rely on proving Keccak collision resistance or indistinguishability. The random oracle is still the idealized object used in the reduction. This file only keeps the modeled FIPS202 support well typed and internally consistent.

### Textbook Formal Inventory

This subsection records the exact formal material in `HAETAE_Keccak1600.ec`. It is generated from the proof-surface EasyCrypt file, so the line numbers and statements are meant to be read next to the checked source.

**Chapter role.** Keccak-f1600 state and bit-level round equations.

**Inventory size.** 2 context declarations, 37 definitions/game objects, 50 lemmas or relational theorems, and 0 explicit Hoare/Phoare judgments were found.

### Theory Context, Imports, and Quantified Modules

**Context 1 (line 1)** EasyCrypt name.

```
imported theories
```

**Checked EasyCrypt statement.**

```
require import AllCore IntDiv List.
```

**Formal mathematical reading.** Mathematical context. This line imports or specializes earlier theories, making their carrier sets, functions, modules, and theorems available to the current file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Context 2 (line 3) EasyCrypt name.**

```
HAETAE_Keccak1600
```

**Checked EasyCrypt statement.**

```
theory HAETAE_Keccak1600.
```

**Formal mathematical reading.** Mathematical context. This begins a namespace for the formal development in this file.

**Role in the proof.** Use in this chapter. It fixes the proof context, imported mathematical language, or quantified module parameters for the following statements.

**Definitions, Carrier Sets, Operations, Games, and Procedures**

**Definition 16.1 (EasyCrypt line 5). EasyCrypt name.**

```
keccak_byte
```

**Checked EasyCrypt statement.**

```
type keccak_byte = int.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.2 (EasyCrypt line 6). EasyCrypt name.**

```
keccak_lane
```

**Checked EasyCrypt statement.**

```
type keccak_lane = bool list.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.3 (EasyCrypt line 7). EasyCrypt name.**

```
keccak_state
```

**Checked EasyCrypt statement.**

```
type keccak_state = keccak_lane list.
```

**Formal mathematical reading.** Mathematical definition. This is a definitional type abbreviation. The exact displayed EasyCrypt statement gives the right-hand side; later proof steps may unfold it without adding an assumption.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.4** (EasyCrypt line 9). **EasyCrypt name.**

```
keccak_lane_bits
```

**Checked EasyCrypt statement.**

```
op keccak_lane_bits : int = 64.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.5** (EasyCrypt line 10). **EasyCrypt name.**

```
keccak_state_lanes
```

**Checked EasyCrypt statement.**

```
op keccak_state_lanes : int = 25.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.6** (EasyCrypt line 11). **EasyCrypt name.**

```
keccak_state_bytes
```

**Checked EasyCrypt statement.**

```
op keccak_state_bytes : int = 200.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.7** (EasyCrypt line 13). **EasyCrypt name.**

```
keccak_byte_norm
```

**Checked EasyCrypt statement.**

```
op keccak_byte_norm (b : keccak_byte) : keccak_byte = b %% 256.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.8** (EasyCrypt line 14). **EasyCrypt name.**

```
keccak_byte_bit
```

**Checked EasyCrypt statement.**

```
op keccak_byte_bit (b : keccak_byte) (i : int) : bool =
  ((keccak_byte_norm b) %/ (2 ^ i)) %% 2 <> 0.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.9** (EasyCrypt line 16). **EasyCrypt name.**

```
keccak_word_norm
```

**Checked EasyCrypt statement.**

```
op keccak_word_norm (w : int) : int = w %% (2 ^ keccak_lane_bits).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.10** (EasyCrypt line 17). **EasyCrypt name.**

```
keccak_word_bit
```

**Checked EasyCrypt statement.**

```
op keccak_word_bit (w : int) (i : int) : bool =
  ((keccak_word_norm w) %/ (2 ^ i)) %% 2 <> 0.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.11** (EasyCrypt line 20). **EasyCrypt name.**

```
keccak_lane_zero
```

**Checked EasyCrypt statement.**

```
op keccak_lane_zero : keccak_lane = nseq keccak_lane_bits false.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.12** (EasyCrypt line 21). **EasyCrypt name.**

```
keccak_lane_bit
```

**Checked EasyCrypt statement.**

```
op keccak_lane_bit (lane : keccak_lane) (i : int) : bool =
  nth false lane i.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.13** (EasyCrypt line 23). **EasyCrypt name.**

```
keccak_lane_wf
```

**Checked EasyCrypt statement.**

```
op keccak_lane_wf (lane : keccak_lane) : bool =
  size lane = keccak_lane_bits.
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It names a well-formedness, support, or validity predicate needed by later preservation lemmas.

**Definition 16.14** (EasyCrypt line 26). **EasyCrypt name.**

```
keccak_lane_xor
```

**Checked EasyCrypt statement.**

```
op keccak_lane_xor (a b : keccak_lane) : keccak_lane =
  mkseq
    (fun i => keccak_lane_bit a i <> keccak_lane_bit b i)
  keccak_lane_bits.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.15** (EasyCrypt line 30). **EasyCrypt name.**

```
keccak_lane_and
```

**Checked EasyCrypt statement.**

```
op keccak_lane_and (a b : keccak_lane) : keccak_lane =
  mkseq
    (fun i => keccak_lane_bit a i /\ keccak_lane_bit b i)
  keccak_lane_bits.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.16** (EasyCrypt line 34). **EasyCrypt name.**

```
keccak_lane_not
```

**Checked EasyCrypt statement.**

```

op keccak_lane_not (a : keccak_lane) : keccak_lane =
  mkseq
    (fun i => if keccak_lane_bit a i then false else true)
    keccak_lane_bits.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.17** (EasyCrypt line 38). **EasyCrypt name.**

```
keccak_lane_rotl
```

**Checked EasyCrypt statement.**

```

op keccak_lane_rotl (a : keccak_lane) (offset : int) : keccak_lane =
  mkseq
    (fun i => keccak_lane_bit a ((i - offset) %% keccak_lane_bits))
    keccak_lane_bits.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.18** (EasyCrypt line 43). **EasyCrypt name.**

```
keccak_lane_of_int
```

**Checked EasyCrypt statement.**

```

op keccak_lane_of_int (x : int) : keccak_lane =
  mkseq (fun i => keccak_word_bit x i) keccak_lane_bits.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.19** (EasyCrypt line 46). **EasyCrypt name.**

```
keccak_lane_to_byte
```

**Checked EasyCrypt statement.**

```

op keccak_lane_to_byte (lane : keccak_lane) (byte_index : int) : keccak_byte =
  sumz
    (map
      (fun j =>
        if keccak_lane_bit lane (8 * byte_index + j)
        then 2 ^ j
        else 0)
      (range 0 8)).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.20** (EasyCrypt line 55). **EasyCrypt name.**

```
keccak_bytes_to_lane
```

**Checked EasyCrypt statement.**

```
op keccak_bytes_to_lane (bs : keccak_byte list) (lane_index : int) :  
  keccak_lane =  
  mkseq  
    (fun bit =>  
      keccak_byte_bit  
        (nth 0 bs (8 * lane_index + bit % 8))  
        (bit % 8))  
    keccak_lane_bits.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.21** (EasyCrypt line 64). **EasyCrypt name.**

```
keccak_lanes_of_bytes
```

**Checked EasyCrypt statement.**

```
op keccak_lanes_of_bytes (bs : keccak_byte list) : keccak_state =  
  mkseq (fun lane => keccak_bytes_to_lane bs lane) keccak_state_lanes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.22** (EasyCrypt line 67). **EasyCrypt name.**

```
keccak_bytes_of_lanes
```

**Checked EasyCrypt statement.**

```
op keccak_bytes_of_lanes (st : keccak_state) : keccak_byte list =  
  mkseq  
    (fun i =>  
      keccak_lane_to_byte  
        (nth keccak_lane_zero st (i % 8))  
        (i % 8))  
    keccak_state_bytes.
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.23** (EasyCrypt line 75). **EasyCrypt name.**

```
keccak_lane_index
```

**Checked EasyCrypt statement.**

```
op keccak_lane_index (x y : int) : int =  
  (x % 5) + 5 * (y % 5).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.24** (EasyCrypt line 77). **EasyCrypt name.**

```
keccak_state_lane
```

**Checked EasyCrypt statement.**

```
op keccak_state_lane (st : keccak_state) (x y : int) : keccak_lane =
  nth keccak_lane_zero st (keccak_lane_index x y).
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.25** (EasyCrypt line 80). **EasyCrypt name.**

```
keccak_rho_offsets
```

**Checked EasyCrypt statement.**

```
op keccak_rho_offsets : int list =
  0 :: 1 :: 62 :: 28 :: 27 ::
  36 :: 44 :: 6 :: 55 :: 20 ::
  3 :: 10 :: 43 :: 25 :: 39 ::
  41 :: 45 :: 15 :: 21 :: 8 ::
  18 :: 2 :: 61 :: 56 :: 14 :: [].
```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.26** (EasyCrypt line 87). **EasyCrypt name.**

```
keccak_round_constants
```

**Checked EasyCrypt statement.**

```
op keccak_round_constants : int list =
  1 ::
  32898 ::
  9223372036854808714 ::
  9223372039002292224 ::
  32907 ::
  2147483649 ::
  9223372039002292353 ::
  9223372036854808585 ::
  138 ::
  136 ::
  2147516425 ::
  2147483658 ::
  2147516555 ::
  9223372036854775947 ::
  9223372036854808713 ::
  9223372036854808579 ::
  9223372036854808578 ::
  9223372036854775936 ::
```



```

32778 ::
9223372039002259466 ::
9223372039002292353 ::
9223372036854808704 ::
2147483649 ::
9223372039002292232 :: [].

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.27** (EasyCrypt line 113). **EasyCrypt name.**

```
keccak_theta_c
```

**Checked EasyCrypt statement.**

```

op keccak_theta_c (st : keccak_state) (x : int) : keccak_lane =
  keccak_lane_xor
    (keccak_lane_xor
      (keccak_lane_xor
        (keccak_lane_xor
          (keccak_state_lane st x 0)
          (keccak_state_lane st x 1))
        (keccak_state_lane st x 2))
      (keccak_state_lane st x 3))
    (keccak_state_lane st x 4).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.28** (EasyCrypt line 124). **EasyCrypt name.**

```
keccak_theta_d
```

**Checked EasyCrypt statement.**

```

op keccak_theta_d (st : keccak_state) (x : int) : keccak_lane =
  keccak_lane_xor
    (keccak_theta_c st (x + 4))
    (keccak_lane_rotl (keccak_theta_c st (x + 1)) 1).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.29** (EasyCrypt line 129). **EasyCrypt name.**

```
keccak_theta
```

**Checked EasyCrypt statement.**

```

op keccak_theta (st : keccak_state) : keccak_state =
  mkseq
    (fun i =>
      let x = i %% 5 in
      let y = i %/ 5 in

```

```

keccak_lane_xor
  (keccak_state_lane st x y)
  (keccak_theta_d st x))
keccak_state_lanes.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.30** (EasyCrypt line 139). **EasyCrypt name.**

```
keccak_rho_pi
```

**Checked EasyCrypt statement.**

```

op keccak_rho_pi (st : keccak_state) : keccak_state =
mkseq
  (fun i =>
    let x = i %% 5 in
    let y = i %/ 5 in
    let sx = (x + 3 * y) %% 5 in
    let sy = x in
    keccak_lane_rotl
      (keccak_state_lane st sx sy)
      (nth 0 keccak_rho_offsets (keccak_lane_index sx sy)))
keccak_state_lanes.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.31** (EasyCrypt line 151). **EasyCrypt name.**

```
keccak_chi
```

**Checked EasyCrypt statement.**

```

op keccak_chi (st : keccak_state) : keccak_state =
mkseq
  (fun i =>
    let x = i %% 5 in
    let y = i %/ 5 in
    keccak_lane_xor
      (keccak_state_lane st x y)
      (keccak_lane_and
        (keccak_lane_not (keccak_state_lane st (x + 1) y))
        (keccak_state_lane st (x + 2) y)))
keccak_state_lanes.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.32** (EasyCrypt line 163). **EasyCrypt name.**

```
keccak_iota
```

**Checked EasyCrypt statement.**

```

op keccak_iota (st : keccak_state) (rc : keccak_lane) : keccak_state =
mkseq
  (fun i =>
    if i = 0
    then keccak_lane_xor (nth keccak_lane_zero st 0) rc
    else nth keccak_lane_zero st i)
  keccak_state_lanes.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.33** (EasyCrypt line 171). **EasyCrypt name.**

```
keccak_round
```

**Checked EasyCrypt statement.**

```

op keccak_round (st : keccak_state) (rc : keccak_lane) : keccak_state =
  keccak_iota (keccak_chi (keccak_rho_pi (keccak_theta st))) rc.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.34** (EasyCrypt line 174). **EasyCrypt name.**

```
keccak_f1600_lanes
```

**Checked EasyCrypt statement.**

```

op keccak_f1600_lanes (st : keccak_state) : keccak_state =
  foldl
    (fun acc rc => keccak_round acc (keccak_lane_of_int rc))
    st
    keccak_round_constants.

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.35** (EasyCrypt line 180). **EasyCrypt name.**

```
keccak_f1600_bytes
```

**Checked EasyCrypt statement.**

```

op keccak_f1600_bytes (st : keccak_byte list) : keccak_byte list =
  keccak_bytes_of_lanes (keccak_f1600_lanes (keccak_lanes_of_bytes st)).

```

**Formal mathematical reading.** Mathematical definition. This is a total EasyCrypt operation. The exact displayed statement gives the domain, codomain, and defining equation when present.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.36** (EasyCrypt line 381). **EasyCrypt name.**

```
keccak_theta_c_bit_index_value
```

**Checked EasyCrypt statement.**

```
op keccak_theta_c_bit_index_value
  (st : keccak_state) (x bit : int) : bool =
  (((keccak_lane_bit
    (nth keccak_lane_zero st (keccak_lane_index x 0)) bit <>
    keccak_lane_bit
    (nth keccak_lane_zero st (keccak_lane_index x 1)) bit) <>
    keccak_lane_bit
    (nth keccak_lane_zero st (keccak_lane_index x 2)) bit) <>
    keccak_lane_bit
    (nth keccak_lane_zero st (keccak_lane_index x 3)) bit) <>
    keccak_lane_bit
    (nth keccak_lane_zero st (keccak_lane_index x 4)) bit)).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

**Definition 16.37** (EasyCrypt line 394). **EasyCrypt name.**

```
keccak_theta_d_bit_index_value
```

**Checked EasyCrypt statement.**

```
op keccak_theta_d_bit_index_value
  (st : keccak_state) (x bit : int) : bool =
  keccak_theta_c_bit_index_value st (x + 4) bit <>
  keccak_theta_c_bit_index_value
    st
    (x + 1)
    ((bit - 1) %% keccak_lane_bits).
```

**Formal mathematical reading.** Mathematical definition. This is a Boolean predicate. The exact displayed EasyCrypt statement gives its arguments; mathematically it is an event, invariant, support condition, or well-formedness predicate over those arguments.

**Role in the proof.** Use in this chapter. It is part of the exact formal vocabulary used by subsequent lemmas and games.

## Lemmas, Theorems, Relational Equivalences, and Their Proof Dependencies

**Theorem 16.1** (EasyCrypt line 183). **EasyCrypt name.**

```
keccak_lane_bitsE
```

**Checked EasyCrypt statement.**

```
lemma keccak_lane_bitsE :
  keccak_lane_bits = 64.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 16.2** (EasyCrypt line 187). *EasyCrypt name.*

```
keccak_state_lanesE
```

*Checked EasyCrypt statement.*

```
lemma keccak_state_lanesE :  
  keccak_state_lanes = 25.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 16.3** (EasyCrypt line 191). *EasyCrypt name.*

```
keccak_state_bytesE
```

*Checked EasyCrypt statement.*

```
lemma keccak_state_bytesE :  
  keccak_state_bytes = 200.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 16.4** (EasyCrypt line 195). *EasyCrypt name.*

```
keccak_lane_zero_wf
```

*Checked EasyCrypt statement.*

```
lemma keccak_lane_zero_wf :  
  keccak_lane_wf keccak_lane_zero.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 16.5** (EasyCrypt line 199). *EasyCrypt name.*

```
keccak_lane_xor_wf
```

*Checked EasyCrypt statement.*

```
lemma keccak_lane_xor_wf a b :  
  keccak_lane_wf (keccak_lane_xor a b).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

*Proof-script interpretation.* No proof block was collected for this item.

**Theorem 16.6** (EasyCrypt line 203). *EasyCrypt name.*

```
keccak_lane_and_wf
```

*Checked EasyCrypt statement.*

```
lemma keccak_lane_and_wf a b :  
  keccak_lane_wf (keccak_lane_and a b).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

*Proof-script interpretation.* No proof block was collected for this item.

**Theorem 16.7** (EasyCrypt line 207). *EasyCrypt name.*

```
keccak_lane_not_wf
```

*Checked EasyCrypt statement.*

```
lemma keccak_lane_not_wf a :  
  keccak_lane_wf (keccak_lane_not a).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

*Proof-script interpretation.* No proof block was collected for this item.

**Theorem 16.8** (EasyCrypt line 211). *EasyCrypt name.*

```
keccak_lane_rotl_wf
```

*Checked EasyCrypt statement.*

```
lemma keccak_lane_rotl_wf a offset :  
  keccak_lane_wf (keccak_lane_rotl a offset).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

*Proof-script interpretation.* No proof block was collected for this item.

**Theorem 16.9** (EasyCrypt line 215). *EasyCrypt name.*

```
keccak_lane_of_int_wf
```

*Checked EasyCrypt statement.*

```
lemma keccak_lane_of_int_wf x :  
  keccak_lane_wf (keccak_lane_of_int x).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 16.10** (EasyCrypt line 219). *EasyCrypt name.*

```
keccak_lane_mod_range
```

**Checked EasyCrypt statement.**

```
lemma keccak_lane_mod_range x :
  0 <= x %% keccak_lane_bits < keccak_lane_bits.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 16.11** (EasyCrypt line 223). *EasyCrypt name.*

```
keccak_word_modulus_pos
```

**Checked EasyCrypt statement.**

```
lemma keccak_word_modulus_pos :
  0 < 2 ^ keccak_lane_bits.
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 16.12** (EasyCrypt line 227). *EasyCrypt name.*

```
keccak_word_norm_small
```

**Checked EasyCrypt statement.**

```
lemma keccak_word_norm_small w :
  0 <= w < 2 ^ keccak_lane_bits =>
  keccak_word_norm w = w.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
keccak_word_norm
pmod_small
w_range
```

**Theorem 16.13** (EasyCrypt line 235). *EasyCrypt name.*

```
keccak_pow2_0E
```

*Checked EasyCrypt statement.*

```
lemma keccak_pow2_0E :  
  2 ^ 0 = 1.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 16.14** (EasyCrypt line 239). *EasyCrypt name.*

```
keccak_pow2_1E
```

*Checked EasyCrypt statement.*

```
lemma keccak_pow2_1E :  
  2 ^ 1 = 2.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 16.15** (EasyCrypt line 243). *EasyCrypt name.*

```
keccak_pow2_2E
```

*Checked EasyCrypt statement.*

```
lemma keccak_pow2_2E :  
  2 ^ 2 = 4.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 16.16** (EasyCrypt line 247). *EasyCrypt name.*

```
keccak_pow2_3E
```

*Checked EasyCrypt statement.*

```
lemma keccak_pow2_3E :  
  2 ^ 3 = 8.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.



**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
exprS
keccak_pow2_2E
```

**Theorem 16.17** (EasyCrypt line 254). *EasyCrypt name.*

```
keccak_pow2_4E
```

**Checked EasyCrypt statement.**

```
lemma keccak_pow2_4E :
  2 ^ 4 = 16.
```

**Formal mathematical reading.** *Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
exprS
keccak_pow2_3E
```

**Theorem 16.18** (EasyCrypt line 261). *EasyCrypt name.*

```
keccak_pow2_5E
```

**Checked EasyCrypt statement.**

```
lemma keccak_pow2_5E :
  2 ^ 5 = 32.
```

**Formal mathematical reading.** *Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:*

```
exprS
keccak_pow2_4E
```

**Theorem 16.19** (EasyCrypt line 268). *EasyCrypt name.*

```
keccak_pow2_6E
```

**Checked EasyCrypt statement.**

```
lemma keccak_pow2_6E :  
  2 ^ 6 = 64.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
exprS  
keccak_pow2_5E
```

**Theorem 16.20** (EasyCrypt line 275). *EasyCrypt name.*

```
keccak_pow2_7E
```

**Checked EasyCrypt statement.**

```
lemma keccak_pow2_7E :  
  2 ^ 7 = 128.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
exprS  
keccak_pow2_6E
```

**Theorem 16.21** (EasyCrypt line 282). *EasyCrypt name.*

```
keccak_bytes_to_lane_wf
```

**Checked EasyCrypt statement.**

```
lemma keccak_bytes_to_lane_wf bs lane_index :  
  keccak_lane_wf (keccak_bytes_to_lane bs lane_index).
```

**Formal mathematical reading.** Mathematical theorem. This lemma is a universally quantified proposition over the variables shown in the EasyCrypt statement.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 16.22** (EasyCrypt line 286). *EasyCrypt name.*

```
keccak_rho_offsets_size
```

**Checked EasyCrypt statement.**

```
lemma keccak_rho_offsets_size :  
  size keccak_rho_offsets = keccak_state_lanes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 16.23** (EasyCrypt line 290). **EasyCrypt name.**

```
keccak_round_constants_size
```

**Checked EasyCrypt statement.**

```
lemma keccak_round_constants_size :  
  size keccak_round_constants = 24.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 16.24** (EasyCrypt line 294). **EasyCrypt name.**

```
keccak_lanes_of_bytes_size
```

**Checked EasyCrypt statement.**

```
lemma keccak_lanes_of_bytes_size bs :  
  size (keccak_lanes_of_bytes bs) = keccak_state_lanes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 16.25** (EasyCrypt line 298). **EasyCrypt name.**

```
keccak_lanes_of_bytes_lane_wf
```

**Checked EasyCrypt statement.**

```
lemma keccak_lanes_of_bytes_lane_wf bs lane :  
  0 <= lane < keccak_state_lanes =>  
    keccak_lane_wf  
      (nth keccak_lane_zero (keccak_lanes_of_bytes bs) lane).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It proves a structural correctness fact needed by later extraction or verification arguments.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions.

Main tactics visible in the proof script: *rewrite*, *smt*, *apply*. Named identifiers syntactically cited in the proof block:

```
keccak_bytes_to_lane_wf
keccak_lanes_of_bytes
lane_range
nth_mkseq
```

**Theorem 16.26** (EasyCrypt line 308). *EasyCrypt* name.

```
keccak_lanes_of_bytes_bitE
```

**Checked EasyCrypt statement.**

```
lemma keccak_lanes_of_bytes_bitE bs lane bit :
  0 <= lane < keccak_state_lanes =>
  0 <= bit < keccak_lane_bits =>
  keccak_lane_bit
    (nth keccak_lane_zero (keccak_lanes_of_bytes bs) lane)
  bit =
  keccak_byte_bit
    (nth 0 bs (8 * lane + bit % 8))
  (bit % 8).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
bit_range
keccak_bytes_to_lane
keccak_lane_bit
keccak_lanes_of_bytes
lane_range
nth_mkseq
```

**Theorem 16.27** (EasyCrypt line 324). *EasyCrypt* name.

```
keccak_lane_xor_bitE
```

**Checked EasyCrypt statement.**

```
lemma keccak_lane_xor_bitE a b bit :
  0 <= bit < keccak_lane_bits =>
  keccak_lane_bit (keccak_lane_xor a b) bit =
  (keccak_lane_bit a bit <> keccak_lane_bit b bit).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

bit_range
keccak_lane_bit
keccak_lane_xor
nth_mkseq

```

**Theorem 16.28** (EasyCrypt line 334). *EasyCrypt name.*

```
keccak_lane_and_bitE
```

*Checked EasyCrypt statement.*

```

lemma keccak_lane_and_bitE a b bit :
  0 <= bit < keccak_lane_bits =>
    keccak_lane_bit (keccak_lane_and a b) bit =
      (keccak_lane_bit a bit /\ keccak_lane_bit b bit).

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

bit_range
keccak_lane_and
keccak_lane_bit
nth_mkseq

```

**Theorem 16.29** (EasyCrypt line 344). *EasyCrypt name.*

```
keccak_lane_not_bitE
```

*Checked EasyCrypt statement.*

```

lemma keccak_lane_not_bitE a bit :
  0 <= bit < keccak_lane_bits =>
    keccak_lane_bit (keccak_lane_not a) bit =
      (if keccak_lane_bit a bit then false else true).

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

bit_range
keccak_lane_bit
keccak_lane_not
nth_mkseq

```

**Theorem 16.30** (EasyCrypt line 354). *EasyCrypt name.*

```
keccak_lane_rotl_bitE
```

**Checked EasyCrypt statement.**

```
lemma keccak_lane_rotl_bitE a offset bit :  
  0 <= bit < keccak_lane_bits =>  
    keccak_lane_bit (keccak_lane_rotl a offset) bit =  
    keccak_lane_bit a ((bit - offset) %% keccak_lane_bits).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
bit_range  
keccak_lane_bit  
keccak_lane_rotl  
nth_mkseq
```

**Theorem 16.31** (EasyCrypt line 364). *EasyCrypt name.*

```
keccak_lane_of_int_bitE
```

**Checked EasyCrypt statement.**

```
lemma keccak_lane_of_int_bitE x bit :  
  0 <= bit < keccak_lane_bits =>  
    keccak_lane_bit (keccak_lane_of_int x) bit =  
    keccak_word_bit x bit.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
bit_range  
keccak_lane_bit  
keccak_lane_of_int  
nth_mkseq
```

**Theorem 16.32** (EasyCrypt line 374). *EasyCrypt name.*

```
keccak_state_lane_bitE
```

**Checked EasyCrypt statement.**

```
lemma keccak_state_lane_bitE st x y bit :  
  keccak_lane_bit (keccak_state_lane st x y) bit =  
  keccak_lane_bit
```

```
(nth keccak_lane_zero st (keccak_lane_index x y))
bit.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It preserves an invariant across an oracle call, adversary call, or game procedure.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 16.33** (EasyCrypt line 402). *EasyCrypt name.*

```
keccak_theta_c_bitE
```

**Checked EasyCrypt statement.**

```
lemma keccak_theta_c_bitE st x bit :
  0 <= bit < keccak_lane_bits =>
  keccak_lane_bit (keccak_theta_c st x) bit =
  (((keccak_lane_bit (keccak_state_lane st x 0) bit <>
    keccak_lane_bit (keccak_state_lane st x 1) bit) <>
    keccak_lane_bit (keccak_state_lane st x 2) bit) <>
    keccak_lane_bit (keccak_state_lane st x 3) bit) <>
    keccak_lane_bit (keccak_state_lane st x 4) bit).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
bit_range
keccak_lane_xor_bitE
keccak_theta_c
```

**Theorem 16.34** (EasyCrypt line 415). *EasyCrypt name.*

```
keccak_theta_c_bit_indexE
```

**Checked EasyCrypt statement.**

```
lemma keccak_theta_c_bit_indexE st x bit :
  0 <= bit < keccak_lane_bits =>
  keccak_lane_bit (keccak_theta_c st x) bit =
  keccak_theta_c_bit_index_value st x bit.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

bit_range
keccak_theta_c_bitE
keccak_theta_c_bit_index_value

```

**Theorem 16.35** (EasyCrypt line 425). *EasyCrypt name.*

```
keccak_theta_d_bitE
```

**Checked EasyCrypt statement.**

```

lemma keccak_theta_d_bitE st x bit :
  0 <= bit < keccak_lane_bits =>
    keccak_lane_bit (keccak_theta_d st x) bit =
    (keccak_lane_bit (keccak_theta_c st (x + 4)) bit <>
     keccak_lane_bit
      (keccak_theta_c st (x + 1))
      ((bit - 1) %% keccak_lane_bits)).

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

bit_range
keccak_lane_rotl_bitE
keccak_lane_xor_bitE
keccak_theta_d

```

**Theorem 16.36** (EasyCrypt line 439). *EasyCrypt name.*

```
keccak_theta_d_bit_indexE
```

**Checked EasyCrypt statement.**

```

lemma keccak_theta_d_bit_indexE st x bit :
  0 <= bit < keccak_lane_bits =>
    keccak_lane_bit (keccak_theta_d st x) bit =
    keccak_theta_d_bit_index_value st x bit.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```

bit_range
keccak_lane_bits
keccak_theta_c_bit_indexE
keccak_theta_d_bitE
keccak_theta_d_bit_index_value

```



**Theorem 16.37** (EasyCrypt line 452). *EasyCrypt name.*

```
keccak_theta_bitE
```

**Checked EasyCrypt statement.**

```
lemma keccak_theta_bitE st lane bit :
  0 <= lane < keccak_state_lanes =>
  0 <= bit < keccak_lane_bits =>
  keccak_lane_bit
    (nth keccak_lane_zero (keccak_theta st) lane)
  bit =
  (let x = lane %% 5 in
   let y = lane %/ 5 in
   keccak_lane_bit (keccak_state_lane st x y) bit <>
   keccak_lane_bit (keccak_theta_d st x) bit).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
bit_range
keccak_lane_xor_bitE
keccak_theta
lane_range
nth_mkseq
```

**Theorem 16.38** (EasyCrypt line 469). *EasyCrypt name.*

```
keccak_theta_bit_indexE
```

**Checked EasyCrypt statement.**

```
lemma keccak_theta_bit_indexE st lane bit :
  0 <= lane < keccak_state_lanes =>
  0 <= bit < keccak_lane_bits =>
  keccak_lane_bit
    (nth keccak_lane_zero (keccak_theta st) lane)
  bit =
  (let x = lane %% 5 in
   let y = lane %/ 5 in
   keccak_lane_bit (nth keccak_lane_zero st (keccak_lane_index x y)) bit <>
   keccak_lane_bit (keccak_theta_d st x) bit).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
bit_range
```

```
keccak_theta_bitE
lane_range
```

**Theorem 16.39** (EasyCrypt line 484). *EasyCrypt name.*

```
keccak_rho_pi_bitE
```

**Checked EasyCrypt statement.**

```
lemma keccak_rho_pi_bitE st lane bit :
  0 <= lane < keccak_state_lanes =>
  0 <= bit < keccak_lane_bits =>
  keccak_lane_bit
    (nth keccak_lane_zero (keccak_rho_pi st) lane)
  bit =
  (let x = lane %% 5 in
   let y = lane %/ 5 in
   let sx = (x + 3 * y) %% 5 in
   let sy = x in
   keccak_lane_bit
     (keccak_state_lane st sx sy)
    ((bit - nth 0 keccak_rho_offsets (keccak_lane_index sx sy)) %%
     keccak_lane_bits)).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
bit_range
keccak_lane_rotl_bitE
keccak_rho_pi
lane_range
nth_mkseq
```

**Theorem 16.40** (EasyCrypt line 505). *EasyCrypt name.*

```
keccak_rho_pi_bit_indexE
```

**Checked EasyCrypt statement.**

```
lemma keccak_rho_pi_bit_indexE st lane bit :
  0 <= lane < keccak_state_lanes =>
  0 <= bit < keccak_lane_bits =>
  keccak_lane_bit
    (nth keccak_lane_zero (keccak_rho_pi st) lane)
  bit =
  (let x = lane %% 5 in
   let y = lane %/ 5 in
   let sx = (x + 3 * y) %% 5 in
   let sy = x in
   keccak_lane_bit
     (nth keccak_lane_zero st (keccak_lane_index sx sy))
    ((bit - nth 0 keccak_rho_offsets (keccak_lane_index sx sy)) %%
     keccak_lane_bits)).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```
bit_range
keccak_rho_pi_bitE
lane_range
```

**Theorem 16.41** (EasyCrypt line 524). *EasyCrypt name*.

```
keccak_chi_bitE
```

**Checked EasyCrypt statement.**

```
lemma keccak_chi_bitE st lane bit :
  0 <= lane < keccak_state_lanes =>
  0 <= bit < keccak_lane_bits =>
  keccak_lane_bit
    (nth keccak_lane_zero (keccak_chi st) lane)
  bit =
  (let x = lane %% 5 in
   let y = lane %/ 5 in
   keccak_lane_bit (keccak_state_lane st x y) bit <>
   ((if keccak_lane_bit (keccak_state_lane st (x + 1) y) bit
    then false else true) /\
    keccak_lane_bit (keccak_state_lane st (x + 2) y) bit)).
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*. Named identifiers syntactically cited in the proof block:

```
bit_range
keccak_chi
keccak_lane_and_bitE
keccak_lane_not_bitE
keccak_lane_xor_bitE
lane_range
nth_mkseq
```

**Theorem 16.42** (EasyCrypt line 545). *EasyCrypt name*.

```
keccak_chi_bit_indexE
```

**Checked EasyCrypt statement.**

```
lemma keccak_chi_bit_indexE st lane bit :
  0 <= lane < keccak_state_lanes =>
  0 <= bit < keccak_lane_bits =>
  keccak_lane_bit
    (nth keccak_lane_zero (keccak_chi st) lane)
  bit =
  (let x = lane %% 5 in
```

```

let y = lane % 5 in
keccak_lane_bit (nth keccak_lane_zero st (keccak_lane_index x y)) bit <>
((if keccak_lane_bit
    (nth keccak_lane_zero st (keccak_lane_index (x + 1) y))
    bit
  then false else true) /\
 keccak_lane_bit
  (nth keccak_lane_zero st (keccak_lane_index (x + 2) y))
 bit)).

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

bit_range
keccak_chi_bitE
lane_range

```

**Theorem 16.43** (EasyCrypt line 566). *EasyCrypt name*.

```

keccak_iota_bitE

```

**Checked EasyCrypt statement.**

```

lemma keccak_iota_bitE st rc lane bit :
  0 <= lane < keccak_state_lanes =>
  0 <= bit < keccak_lane_bits =>
  keccak_lane_bit
    (nth keccak_lane_zero (keccak_iota st rc) lane)
  bit =
  if lane = 0
  then keccak_lane_bit (nth keccak_lane_zero st 0) bit <>
    keccak_lane_bit rc bit
  else keccak_lane_bit (nth keccak_lane_zero st lane) bit.

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*, *smt*, *case*. Named identifiers syntactically cited in the proof block:

```

bit_range
keccak_iota
keccak_lane_xor_bitE
lane
lane0
lane_range
nth_mkseq

```

**Theorem 16.44** (EasyCrypt line 585). *EasyCrypt name*.

keccak\_round\_bitE

**Checked EasyCrypt statement.**

```
lemma keccak_round_bitE st rc lane bit :
  0 <= lane < keccak_state_lanes =>
  0 <= bit < keccak_lane_bits =>
  keccak_lane_bit
    (nth keccak_lane_zero (keccak_round st rc) lane)
  bit =
  if lane = 0
  then
    keccak_lane_bit
      (nth keccak_lane_zero
        (keccak_chi (keccak_rho_pi (keccak_theta st)))
        0)
      bit <>
    keccak_lane_bit rc bit
  else
    keccak_lane_bit
      (nth keccak_lane_zero
        (keccak_chi (keccak_rho_pi (keccak_theta st)))
        lane)
      bit.
```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

bit\_range  
keccak\_iota\_bitE  
keccak\_round  
lane\_range

**Theorem 16.45** (EasyCrypt line 611). **EasyCrypt name.**

keccak\_bytes\_of\_lanes\_size

**Checked EasyCrypt statement.**

```
lemma keccak_bytes_of_lanes_size st :
  size (keccak_bytes_of_lanes st) = keccak_state_bytes.
```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** No proof block was collected for this item.

**Theorem 16.46** (EasyCrypt line 615). **EasyCrypt name.**

keccak\_bytes\_of\_lanes\_byteE

**Checked EasyCrypt statement.**

```

lemma keccak_bytes_of_lanes_byteE st i :
  0 <= i < keccak_state_bytes =>
    nth 0 (keccak_bytes_of_lanes st) i =
      keccak_lane_to_byte
        (nth keccak_lane_zero st (i % 8))
        (i % 8).

```

**Formal mathematical reading.** Mathematical theorem. This is an inequality, usually an arithmetic loss bound, event inclusion bound, or monotonicity fact used to compose probabilities.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:

```

i_range
keccak_bytes_of_lanes
nth_mkseq

```

**Theorem 16.47** (EasyCrypt line 626). *EasyCrypt name.*

```

keccak_lane_to_byte_bitsE

```

**Checked EasyCrypt statement.**

```

lemma keccak_lane_to_byte_bitsE lane byte_index :
  keccak_lane_to_byte lane byte_index =
    (if keccak_lane_bit lane (8 * byte_index) then 2 ^ 0 else 0) +
    ((if keccak_lane_bit lane (8 * byte_index + 1) then 2 ^ 1 else 0) +
    ((if keccak_lane_bit lane (8 * byte_index + 2) then 2 ^ 2 else 0) +
    ((if keccak_lane_bit lane (8 * byte_index + 3) then 2 ^ 3 else 0) +
    ((if keccak_lane_bit lane (8 * byte_index + 4) then 2 ^ 4 else 0) +
    ((if keccak_lane_bit lane (8 * byte_index + 5) then 2 ^ 5 else 0) +
    ((if keccak_lane_bit lane (8 * byte_index + 6) then 2 ^ 6 else 0) +
    (if keccak_lane_bit lane (8 * byte_index + 7) then 2 ^ 7 else 0)))))).

```

**Formal mathematical reading.** Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`. Named identifiers syntactically cited in the proof block:

```

iota0
iotaS
keccak_lane_to_byte
range
sumz

```

**Theorem 16.48** (EasyCrypt line 650). *EasyCrypt name.*

```

keccak_lane_to_byte_from_bitsE

```

**Checked EasyCrypt statement.**

```

lemma keccak_lane_to_byte_from_bitsE
  lane byte_index b0 b1 b2 b3 b4 b5 b6 b7 :
  keccak_lane_bit lane (8 * byte_index) = b0 =>
  keccak_lane_bit lane (8 * byte_index + 1) = b1 =>
  keccak_lane_bit lane (8 * byte_index + 2) = b2 =>
  keccak_lane_bit lane (8 * byte_index + 3) = b3 =>
  keccak_lane_bit lane (8 * byte_index + 4) = b4 =>
  keccak_lane_bit lane (8 * byte_index + 5) = b5 =>
  keccak_lane_bit lane (8 * byte_index + 6) = b6 =>
  keccak_lane_bit lane (8 * byte_index + 7) = b7 =>
  keccak_lane_to_byte lane byte_index =
  (if b0 then 1 else 0) +
  ((if b1 then 2 else 0) +
  ((if b2 then 4 else 0) +
  ((if b3 then 8 else 0) +
  ((if b4 then 16 else 0) +
  ((if b5 then 32 else 0) +
  ((if b6 then 64 else 0) +
  (if b7 then 128 else 0)))))).

```

**Formal mathematical reading.** Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.

**Role in the proof.** Use in this chapter. It is a reusable checked theorem cited by later proof scripts.

**Proof-script interpretation.** Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: *rewrite*. Named identifiers syntactically cited in the proof block:

```

h0
h1
h2
h3
h4
h5
h6
h7
keccak_lane_to_byte_bitsE
keccak_pow2_0E
keccak_pow2_1E
keccak_pow2_2E
keccak_pow2_3E
keccak_pow2_4E
keccak_pow2_5E
keccak_pow2_6E
keccak_pow2_7E

```

**Theorem 16.49** (EasyCrypt line 676). *EasyCrypt name.*

```
keccak_lane_to_byte_70E
```

**Checked EasyCrypt statement.**

```

lemma keccak_lane_to_byte_70E lane byte_index :
  keccak_lane_bit lane (8 * byte_index) = false =>
  keccak_lane_bit lane (8 * byte_index + 1) = true =>
  keccak_lane_bit lane (8 * byte_index + 2) = true =>
  keccak_lane_bit lane (8 * byte_index + 3) = false =>
  keccak_lane_bit lane (8 * byte_index + 4) = false =>
  keccak_lane_bit lane (8 * byte_index + 5) = false =>
  keccak_lane_bit lane (8 * byte_index + 6) = true =>
  keccak_lane_bit lane (8 * byte_index + 7) = false =>
  keccak_lane_to_byte lane byte_index = 70.

```

---

**Formal mathematical reading.** *Mathematical theorem. This is an implication, normally turning one invariant, validity predicate, or proof obligation into another.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *Proof-script reading. The machine proof decomposes the theorem into rewrites, program-logic obligations, relational equivalences, and arithmetic side conditions. Main tactics visible in the proof script: `rewrite`, `smt`. Named identifiers syntactically cited in the proof block:*

```
b0
b1
b2
b3
b4
b5
b6
b7
keccak_lane_to_byte_bitsE
keccak_pow2_1E
keccak_pow2_2E
keccak_pow2_6E
```

**Theorem 16.50** (EasyCrypt line 693). *EasyCrypt name.*

```
keccak_f1600_bytes_size
```

**Checked EasyCrypt statement.**

```
lemma keccak_f1600_bytes_size st :
  size (keccak_f1600_bytes st) = keccak_state_bytes.
```

**Formal mathematical reading.** *Mathematical theorem. This is an equality or definitional expansion used for rewriting and normalization.*

**Role in the proof.** *Use in this chapter. It is a reusable checked theorem cited by later proof scripts.*

**Proof-script interpretation.** *No proof block was collected for this item.*

*End of generated formal inventory for HAETAE\_Keccak1600.ec.*



# How to Use This Guide

For conceptual reading, start with `HAETAE_Security.ec`, `Sig_ROM.ec`, `HAETAE_Reductions.ec`, and `HAETAE_Assumptions.ec`. Then read `HAETAE_Transcript.ec` and `HAETAE_HopGames.ec` together: the former explains what is extracted from two accepting transcripts, while the latter shows how the games reach that extraction setting. Finally read the infrastructure files `HAETAE_Distributions.ec`, `HAETAE_Rejection.ec`, and `HAETAE_Algebra.ec` when a theorem premise refers to sampling support, rejection loss, or deterministic verification.

To rebuild the checked proof, run:

```
sh provable-security/verify-provable-security.sh
```

To rebuild this guide from `haetae-security/guide`, run:

```
python3 generate-mathematical-chapter-details.py
pdflatex -interaction=nonstopmode -halt-on-error haetae-mathematical-cryptology-guide.
tex
pdflatex -interaction=nonstopmode -halt-on-error haetae-mathematical-cryptology-guide.
tex
```